



CuentaVoz

Tu voz cuenta

Guía técnica

Cómo está construido CuentaVoz por dentro: arquitectura, agente de voz, modelo de datos, pruebas y despliegue.

VERSIÓN **5.0**

AGOSTO DE 2026

DIRIGIDO AL **EQUIPO TÉCNICO**

QUE MANTIENE LA PLATAFORMA



PREPARADO POR STOCKXPERS

Contenido

Las secciones 1 a 3 son el panorama y el arranque. De la 4 a la 11 va el sistema por dentro, pieza por pieza. La 12 recorre las catorce pantallas una por una. Las secciones 13 a 16 son la operación: pruebas, despliegue, decisiones y qué hacer cuando algo falla. La 17 es el código completo del proyecto, generado leyendo el repositorio.

1	Qué es CuentaVoz	PANORAMA
2	La arquitectura en una página	PANORAMA
3	Poner el proyecto a andar	ARRANQUE
4	Identidad y permisos	POR DENTRO
5	El agente de voz	POR DENTRO
6	El modelo de datos	POR DENTRO
7	La API	POR DENTRO
8	El frontend	POR DENTRO
9	Trabajar sin conexión	POR DENTRO
10	Reportes y archivos	POR DENTRO
11	Trazabilidad y auditoría	POR DENTRO
12	Las catorce vistas, una por una	POR DENTRO
13	Pruebas	OPERACIÓN
14	Despliegue	OPERACIÓN
15	Decisiones de diseño	OPERACIÓN
16	Cuando algo falla	OPERACIÓN
17	El código completo	APÉNDICE

1 Qué es CuentaVoz

El problema que resuelve, y por qué se resuelve hablando.

CuentaVoz reemplaza la planilla de papel del conteo de inventario en las bodegas de alimentos y bebidas. En vez de anotar en una hoja y después transcribir a un archivo, la persona **dicta lo que ve** — «arroz, veinte kilos» — y la plataforma lo entiende, lo confirma en voz alta y lo guarda con el código oficial del catálogo.

Eso quita los dos puntos donde se pierde la información: la letra en la planilla y la transcripción posterior. Y como quien cuenta tiene las manos ocupadas, hablar es la única interfaz que no lo obliga a soltar el producto.

LA REGLA QUE GOBIERNA TODO EL SISTEMA

CuentaVoz **siempre confirma antes de guardar, y nunca sobrescribe**. Cada dato que entra pasa por una confirmación explícita de la persona, y cuando se corrige una cantidad el valor anterior no se borra: queda en el registro de trazabilidad. Si alguna vez una función nueva rompe esa regla, es la función la que está mal.

Los dos perfiles

PERFIL	QUÉ HACE	QUÉ VE
Auxiliar de inventarios auxiliar	Cuenta las bodegas que le asignaron, pide insumos al almacén y legaliza el servicio del turno.	9 opciones de menú.
Administrador de bodega auditor	Todo lo anterior, más auditar, aprobar lo que se creó durante el conteo, sacar los reportes y administrar la configuración.	13 opciones de menú.

El perfil vive en `usuario.perfil` y vale `auxiliar` o `auditor`. En el código el administrador se llama siempre `auditor`; «Administrador de bodega» es solo la etiqueta que se le muestra a la gente.

2 La arquitectura en una página

Cuatro piezas y tres servicios externos. Nada más.

PIEZA	TECNOLOGÍA	DE QUÉ RESPONDE
Frontend frontend/	React 18 + Vite, PWA	Las 14 vistas, el reconocimiento de voz del navegador, la cola sin conexión y el diálogo con Cognito.
Backend backend/	FastAPI + Uvicorn	90 endpoints, las reglas del negocio, la conversación con el agente y la generación de archivos.
Base de datos	SQLAlchemy 2 sobre SQLite (local) o PostgreSQL (producción)	18 tablas. El mismo código sirve para las dos: solo cambia DB_URL .
Servicios backend/servicios/	Python puro	Intérprete local, recetas, conciliación, analítica, validación y archivos. Sin FastAPI adentro, por eso se pueden probar solos.

Los tres servicios externos

SERVICIO	PARA QUÉ	SI NO ESTÁ
AWS Cognito	Identidad: registro, ingreso, cambio de clave, verificación en dos pasos.	No se puede entrar. Es la única dependencia dura.
Google Gemini	Entender lo que la persona dice, en lenguaje libre.	Entra el intérprete local: entiende menos variantes, pero el flujo completo sigue funcionando.
Brevo (correo)	Avisar por correo de un mensaje al administrador.	El mensaje igual queda en la bandeja; se ofrece abrir el correo del propio dispositivo.

EL PATRÓN QUE SE REPITE: DEGRADARSE SIN ROMPERSE

Ninguna función se cae por completo cuando falla algo de afuera. Sin Gemini hay intérprete local; sin internet hay cola local que se sincroniza sola; sin servicio de correo el mensaje queda igual registrado. Es la diferencia entre una demo y algo que se puede usar en una bodega con señal irregular.

3 Poner el proyecto a andar

De un clon a la aplicación abierta en el navegador.

Qué necesita instalado

HERRAMIENTA	VERSIÓN	PARA QUÉ
Python	3.12 o superior	El backend.
Node.js	18 o superior	El frontend y las pruebas.
Git	cualquiera reciente	Traer el proyecto.

Backend

- 1 Cree el entorno virtual**
`cd backend` y luego `python -m venv .venv`.
- 2 Actívelo**
en Windows, `.venv\Scripts\activate`; en Linux o macOS, `source .venv/bin/activate`.
- 3 Instale las dependencias**
`pip install -r requirements.txt`.
- 4 Copie el archivo de configuración**
`.env.ejemplo` a `.env` y llene las llaves (ver el cuadro siguiente).
- 5 Arranque**
`uvicorn main:app --reload`. Queda en `http://127.0.0.1:8000`.

Frontend

- 1 Instale**
`cd frontend` y `npm install`.
- 2 Arranque**
`npm run dev`. Queda en `http://localhost:5173`.

El frontend apunta por defecto a `http://127.0.0.1:8000`. Para apuntarlo a otro backend, defina `VITE_API_URL` antes de arrancar.

Las variables de entorno

VARIABLE	OBLIGATORIA	QUÉ CONTROLA
COGNITO_REGION COGNITO_USER_POOL_ID COGNITO_APP_CLIENT_ID	Sí	Identifican el User Pool. No son secretos: viajan dentro del JavaScript que descarga cualquiera.
AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY	Sí	Credenciales de una cuenta IAM propia del backend, con permiso solo sobre ese User Pool. Nunca la cuenta que creó el pool.
DB_URL	No	Por defecto <code>sqlite:///cuentavoz.db</code> . En producción, la cadena de PostgreSQL.
GOOGLE_API_KEY	No	Activa Gemini. Sin ella entra el intérprete local.
UMBRAL_ANOMALIA	No	Qué tan grande debe ser una diferencia para disparar una alerta. Por defecto <code>0.5</code> (50 %).
ORIGEN_PERMITIDO	No	Orígenes extra para CORS, separados por coma. Los puertos 5173 y 4173 ya vienen permitidos.
SENTRY_DSN	No	Monitoreo de errores. Vacía, no se inicializa nada.

CUENTAS DE DEMOSTRACIÓN

Con la variable de siembra activa y la tabla de usuarios vacía, el backend crea `luis` , `diana` , `stephanie` y `valentina` con la clave `StockXperts1` . No la active en un despliegue con datos reales: `diana` tiene perfil de administrador.

4 Identidad y permisos

Quién es quién, y qué puede tocar cada quien.

La clave nunca pasa por el backend

El registro, el ingreso, el cambio de clave y la verificación en dos pasos los maneja **AWS Cognito directamente**: el frontend habla con el SDK de Cognito y la clave nunca llega a este servidor. El backend no puede filtrar una clave que nunca ve.

Lo único que hace **backend/seguridad.py** es comprobar que el *access token* que llega en la cabecera **Authorization** de verdad lo firmó nuestro User Pool: firma RS256 contra las llaves públicas que Cognito publica, emisor, audiencia, y que sea un access token y no un id token.

Los dos niveles de permiso

NIVEL	CÓMO SE APLICA	QUÉ PROTEGE
Por perfil	<code>Depends(requiere_perfil("auditor"))</code> en el endpoint.	Auditoría, Panel, Reportes y Ajustes.
Por bodega	La tabla <code>asignacion_bodega</code> .	Un auxiliar solo puede contar las zonas que le asignaron.

EL PERMISO POR BODEGA SE APLICA EN CUATRO SITIOS, NO EN UNO

Listar bodegas, abrir una bodega, ver su detalle y la **búsqueda por voz**. Ocultar la opción en el menú no es seguridad: si la restricción no está también en el endpoint, basta con pedir la URL a mano. Por eso el agente de voz también la respeta — es la puerta que más fácil se olvida.

Cerrar sesión de verdad

«Cerrar sesión» no se limita a borrar el token de este navegador: llama a `/api/usuarios/yo/cerrar-todas`, que revoca la sesión en Cognito. Sin eso, el access token seguía siendo válido hasta vencer solo — hasta una hora. En una tableta que se pasan entre turnos, eso es una cuenta abierta para el siguiente.

5 El agente de voz

De «arroz, veinte kilos» a una fila en la base de datos.

El recorrido de una frase

- 1 El navegador escucha**
con la API `SpeechRecognition`, en español de Colombia. El audio no sale del dispositivo: lo que viaja es el texto.
- 2 El texto llega al backend**
por `POST /api/agente/turno`, junto con la sesión de conteo y el contexto de lo que quedó pendiente de confirmar.
- 3 El agente interpreta**
Gemini recibe la frase, el contexto y las reglas, y devuelve una intención con sus datos. Sin llave de Gemini, ese trabajo lo hace el intérprete local.
- 4 Se resuelve el artículo**
el nombre dictado se empareja contra el catálogo con `rapidfuzz` y la tabla de alias. Si hay varias candidatas, se devuelven como opciones para que la persona elija.
- 5 Se valida**
si la cantidad se sale del umbral, o la unidad no coincide, o el número es imposible, se genera una alerta
antes
de guardar.
- 6 Se confirma y se guarda**
la plataforma repite lo que entendió y espera un «confirmo». Solo entonces se escribe la fila y se registra la traza.

Por qué hay dos intérpretes

`backend/servicios/interprete.py` es un intérprete escrito a mano: reconoce números en palabras («veinte» → 20), unidades («kilos» → `Kilogram`) y las intenciones más frecuentes. No reemplaza al modelo — entiende menos variantes de frase — pero garantiza que **la demo nunca se cae por falta de internet o de cuota**.

El mismo intérprete corre además **dentro del navegador** (`frontend/src/interpreteLocal.js`) para el modo sin conexión, donde no hay backend con quien hablar.

POR QUÉ EL TURNO DEL AGENTE CORRE EN UN HILO APARTE

Llamar a Gemini puede tardar varios segundos. Hacerlo directo dentro de la ruta **async** bloqueaba **todo** el bucle de eventos: cualquier otra petición, de cualquier otra persona, a cualquier endpoint — hasta **/api/salud** — se quedaba esperando. Por eso **procesar_turno** se saca con **run_in_threadpool**.

El asistente contextual

Aparte del conteo, **POST /api/agente/asistente** responde preguntas sobre la pantalla en la que está la persona y sabe navegar: «lléveme al panel» cambia de vista. Las claves de destino tienen que calzar exactamente con las que usa **ir(destino)** en **App.jsx**; si no calzan, la navegación por voz apunta a una vista que no existe.

6 El modelo de datos

Dieciocho tablas, en `backend/modelos.py`.

TABLA	GUARDA
usuario	Las personas, su perfil y su estado. La clave no está aquí: la guarda Cognito.
asignacion_bodega	Qué bodegas puede contar cada auxiliar.
bodega	Las zonas y su estado: pendiente, en conteo, en auditoría o cerrada.
articulo	El catálogo oficial, con su código.
stock_sistema	Lo que el sistema cree que hay, contra lo que se compara el conteo.
alias_articulo	Cómo llama la gente a las cosas, contra el nombre oficial. Es lo que hace que «papa criolla» encuentre PAPA CRIOLLA PRECOCIDA .
sesion_conteo	Una toma de una bodega por una persona. Es el candado que evita conteos duplicados.
conteo	Cada renglón contado. Las correcciones no borran: apuntan al anterior con corrige_a .
alerta	Las diferencias que quedaron marcadas durante el conteo.
traza	Quién hizo qué y cuándo. Nunca se borra.
receta receta_ingredient	Los platos con sus ingredientes y su rendimiento. Es la base del cálculo de pedidos.
linea_servicio	Lo pedido contra lo usado en un servicio. Es lo que compara la legalización.
aprobacion	Productos y bodegas creados en medio del conteo, esperando el visto bueno.
historial_cierre	El cierre de cada bodega, con su doble firma.
config_clave	Configuración del sistema por clave y valor: umbral de anomalía, modo sin conexión.
archivo_generado	Los reportes que se han sacado, para poder previsualizarlos y volver a descargarlos.
mensaje_soporte	La bandeja interna entre auxiliares y administrador.

Cómo se conectan

Tres cadenas explican casi todo el sistema:

CADENA	RECORRIDO
El conteo	bodega → sesion_conteo → conteo → alerta → historial_cierre . Una bodega se abre en una sesión, la sesión acumula renglones, los renglones que se salen del umbral generan alertas, y el cierre con doble firma congela el resultado.
El pedido	receta → receta_ingrediente → stock_sistema → linea_servicio . La receta dice cuánto se necesita por porción, el stock dice cuánto hay, y la diferencia es lo que hay que pedir. Al cerrar el turno, esa misma línea guarda lo que de verdad se usó.
El nombre	lo dictado → alias_articulo → articulo . Es la cadena que convierte «papa criolla» en un código oficial. Cuando no encuentra nada, nace una fila en aprobacion en vez de detener el conteo.

NADA SE BORRA

Corregir un conteo escribe una fila nueva que apunta a la anterior.
Desactivar a una persona pone **activo = 0**, no borra la fila. Rechazar una aprobación la deja marcada como rechazada. Un inventario que pierde su historia no sirve para auditar.

7 La API

Noventa endpoints en `backend/main.py`, agrupados por familia.

FAMILIA	CUÁNTOS	PARA QUÉ
<code>/api/usuarios</code>	16	El perfil propio, la administración de cuentas y la asignación de bodegas.
<code>/api/bodegas</code>	15	Listar, abrir, cerrar, detalle y exportaciones.
<code>/api/reportes</code>	6	Generar, previsualizar y descargar.
<code>/api/pedidos</code>	6	Calcular desde la receta, enviar y aprobar.
<code>/api/soporte</code>	5	Reportar un problema, escribirle al administrador y la bandeja.
<code>/api/recetas</code>	5	Mantener los platos y sus ingredientes.
<code>/api/articulos</code>	4	Buscar en el catálogo y consultar existencias.
<code>/api/legalizacion</code>	3	Comparar, ajustar y confirmar el cierre del servicio.
<code>/api/aprobaciones · /api/alertas · /api/trazabilidad</code>	9	El control del administrador.
<code>/api/agente · /api/voz</code>	4	El turno de conversación, el asistente contextual y las voces.
<code>/api/panel · /api/analisis · /api/ajustes</code>	6	Indicadores, análisis de consumo y configuración.
<code>/api/salud</code>	1	Estado del sistema. Es el que consulta la pantalla de Ayuda.

Todos exigen un access token válido salvo `/api/salud` y `/api/registro-completado`. Los que solo puede usar el administrador llevan `Depends(requiere_perfil("auditor"))`.

LOS ERRORES LLEGAN CON SUS CABECERAS CORS

Un error que sale sin las cabeceras de CORS llega al navegador como «error de red», y la pantalla dice «Sin conexión con el servidor. Revise el Wi-Fi» — mandando a buscar el problema en el router mientras el servidor respondía perfecto. Por eso los manejadores de excepción las ponen a mano: para que el error se vea tal cual es.

8 El frontend

Catorce vistas, una barra lateral y un puñado de módulos compartidos.

Cómo está organizado

ARCHIVO	DE QUÉ RESPONDE
App.jsx	El menú (MENU), qué vista está activa, la sesión y el contexto que se pasa a las vistas.
vistas/	Las 14 pantallas, una por archivo.
api.js	Todas las llamadas al backend, y esFalloRed , que distingue un fallo de red de un error del servidor.
cognito.js	Registro, ingreso, clave y verificación en dos pasos.
voz.js	Escuchar y hablar.
interpreteLocal.js	El intérprete del modo sin conexión.
colaOffline.js	La cola de conteos pendientes de sincronizar.
accesibilidad.js	Alto contraste y tamaño de letra.
tutorial.js · VideoRecorrido.jsx	El recorrido en video que se abre al ingresar.

El menú es la única fuente de verdad de la navegación

La lista **MENU** de **App.jsx** define el orden, el título, la vista y si la opción es solo del administrador (**soloAuditor: true**). Agregar una pantalla es agregar una entrada ahí y su componente en **vistas/**.

QUIEN DESPLAZA ES .CONTENIDO, NO LA VENTANA

El contenedor raíz ocupa **100dvh** y el que tiene la barra de desplazamiento es **.contenido**. Un **window.scrollTo** no mueve nada. Vale la pena saberlo antes de perder una tarde.

9 Trabajar sin conexión

Lo que pasa cuando se cae el internet en plena bodega.

El conteo no se detiene. El navegador dispara el evento `offline`, la pantalla de Conteo cambia sola a un formulario de respaldo, y lo que se registre se guarda en `localStorage` bajo la clave de la **sesión de conteo activa**. Un aviso muestra cuántos registros quedan pendientes, tanto en la pantalla como en la barra lateral.

Al volver la señal, el evento `online` dispara la sincronización: cada renglón se reenvía y se confirma, y **solo se saca de la cola lo que de verdad se confirmó**. Si uno falla, o vuelve a caerse la red a mitad, el resto queda guardado para el próximo intento — antes, cualquier falla borraba toda la cola sin distinguir qué alcanzó a guardarse.

LA COLA VA POR SESIÓN DE CONTEO, NO POR USUARIO

La clave es `cv_offline_<sesionId>`. Sin una bodega abierta no hay sesión, y por lo tanto no hay dónde encolar: el modo sin conexión solo tiene sentido dentro de un conteo en curso.

10 Reportes y archivos

Seis salidas, repartidas en las pantallas a las que pertenecen.

ARCHIVO	SE GENERA DESDE
Consolidado para My Inventory	Reportes › Consolidado de la toma
Diferencias por bodega	Reportes › Consolidado de la toma
Análisis de consumo	Reportes › Análisis de consumo
Estado del tablero	Bodegas
Detalle de una bodega	Bodegas › detalle
Registro de trazabilidad	Ajustes › Registro de trazabilidad

Todo lo generado queda en **archivo_generado**, así que se puede **previsualizar en pantalla antes de descargar** y volver a bajar después. Los archivos salen en XLSX con **openpyxl**, con los códigos oficiales del catálogo — que es lo que My Inventory espera recibir.

11 Trazabilidad y auditoría

Cómo se responde a «¿quién cambió esto?».

Cada acción con consecuencia llama a **registrar**(usuario, accion, detalle, nivel), que escribe en **traza**. Ahí quedan los conteos, las correcciones, las aprobaciones, los cierres, los reportes de soporte y los mensajes. El administrador lo consulta y lo exporta desde **Ajustes > Registro de trazabilidad**.

Las cuatro pestañas de Auditoría

PESTAÑA	QUÉ RESUELVE
Recuento ciego y cierre	Volver a contar una bodega sin ver los números del auxiliar. La comparación solo se revela al terminar, y el cierre lleva doble firma.
Aprobaciones	Los productos y bodegas que alguien creó en medio del conteo para no detenerse. Al aprobarlos entran al catálogo con su código definitivo.
Pedidos pendientes	Autorizar lo solicitado antes de que salga del almacén.
Bandeja de alertas	Las diferencias marcadas durante el conteo, agrupadas por tipo.

POR QUÉ EL RECUESTO ES CIEGO

Si quien audita ve el número del primer conteo, lo confirma. El sesgo de anclaje convierte la auditoría en un trámite. Ocultar el dato hasta el final es lo que hace que la segunda pasada valga algo.

12 Las catorce vistas, una por una

Qué hay en cada pantalla, qué problema resuelve y con qué endpoints habla. En el orden del menú.

Cada vista es un archivo en **frontend/src/vistas/**. Reciben por props el **token**, el **usuario**, el contexto de la sesión y la función **ir()** para navegar. Ninguna guarda estado global: lo que tiene que sobrevivir a un cambio de pantalla vive en **App.jsx**.

12.1 Ingreso SIN SESIÓN

ARCHIVO	Ingreso.jsx · 657 líneas
QUÉ TIENE	Siete modos en una sola pantalla: login , login-mfa , registro , registro-codigo , registro-mfa , recuperar y recuperar-codigo . El perfil se detecta mientras se escribe el usuario.
QUÉ RESUELVE	Todo el ciclo de identidad sin salir de una pantalla ni depender de soporte: crear la cuenta, confirmar el correo, activar el segundo factor y recuperar la clave.
CON QUÉ HABLA	Con Cognito directamente (vía cognito.js), no con el backend. La clave nunca llega al servidor. Al terminar llama a /api/registro-completado para crear la fila local.

12.2 Inicio MENÚ 1

ARCHIVO	Inicio.jsx · 171 líneas
QUÉ TIENE	El saludo con la fecha, cuatro indicadores del día (bodegas asignadas, referencias contadas, alertas por revisar, exactitud del mes), cuatro accesos directos y la actividad reciente del equipo.
QUÉ RESUELVE	Que la persona sepa en diez segundos qué le toca hoy, sin recorrer el menú.
CON QUÉ HABLA	/api/usuarios/yo/resumen y /api/trazabilidad para la actividad.

12.3 Pedidos MENÚ 2

ARCHIVO	Pedido.jsx · 849 líneas
QUÉ TIENE	El micrófono para dictar el plato y las porciones, la burbuja con lo que entendió el agente, la tabla de insumos calculados (necesario, hay en bodega, falta pedir) y el botón de enviar al almacén.
QUÉ RESUELVE	El caso del reto: «hoy preparamos cincuenta ajiacos». La persona no dicta ingredientes — dicta el plato, y la receta hace el resto.
CON QUÉ HABLA	<code>/api/agente/turno</code> para entender la frase, <code>/api/pedidos/calcular</code> para la tabla y <code>/api/pedidos</code> para enviarlo.

La cuenta que hace el backend: **cantidad por porción × porciones – lo que hay en esa bodega**. Si el resultado es cero o negativo, el insumo no entra al pedido: no se pide lo que ya está.

12.4 Conteo MENÚ 3

ARCHIVO	Conteo.jsx · 1.174 líneas — la vista más grande del proyecto
QUÉ TIENE	El selector de bodega (tarjetas o nombre exacto), el avance y las alertas arriba, la burbuja del agente, los cuatro botones (Confirmar , Corregir , Teclado , Pedir ayuda), la desambiguación por tarjetas, el aviso de anomalía, el alta de producto nuevo y el formulario de respaldo sin conexión.
QUÉ RESUELVE	Es el corazón del sistema: convertir lo que alguien dice frente a un estante en una fila con código oficial, sin que nada se guarde sin confirmación.
CON QUÉ HABLA	<code>/api/bodegas/abrir</code> , <code>/api/agente/turno</code> , <code>/api/sesiones/{id}/avance</code> y <code>/api/conteo/crear-producto</code> .

LOS CUATRO CAMINOS DE UN TURNO DE CONTEO

El agente puede responder cuatro cosas distintas a una frase: **confirmar** (entendió, pregunta si guarda), **desambiguar** (varias candidatas en el catálogo, muestra tarjetas), **alertar** (la cantidad se sale del umbral, pide revisar) o **proponer crear** (no existe en el catálogo). Los cuatro terminan en una pregunta, nunca en una escritura silenciosa.

12.5 Legalización MENÚ 4

ARCHIVO	Legalizacion.jsx · 297 líneas
QUÉ TIENE	La tabla de lo pedido contra lo usado, con la diferencia y su nombre (cuadro exacto, sobrante que vuelve a bodega, o merma). Cada renglón se puede ajustar dictando.
QUÉ RESUELVE	Cerrar el turno con las cuentas claras: lo que sobró vuelve al inventario y lo que faltó queda explicado, en vez de desaparecer.
CON QUÉ HABLA	/api/legalizacion/{servicio} , /api/legalizacion/ajustar y /api/legalizacion/confirmar .

12.6 Bodegas MENÚ 5

ARCHIVO	Bodegas.jsx · 848 líneas
QUÉ TIENE	Una tarjeta por zona con su estado por color (pendiente, en conteo, en auditoría, cerrada) y su avance; el detalle al tocar una; el buscador de artículos por voz («¿cuánto arroz hay y en qué bodegas?»); y el alta de una bodega nueva, que queda pendiente de aprobación.
QUÉ RESUELVE	Saber en qué va el parque completo sin llamar a nadie, y encontrar un artículo sin recorrer estantes.
CON QUÉ HABLA	/api/bodegas y sus derivados, /api/articulos/buscar . Se actualiza sola por WebSocket cuando alguien abre o cierra una bodega.

12.7 Auditoría MENÚ 6 · ADMINISTRADOR

ARCHIVO	Auditoria.jsx · 949 líneas
PESTAÑAS	Recuento ciego y cierre · Aprobaciones · Pedidos pendientes · Bandeja de alertas
QUÉ RESUELVE	El control que solo puede hacer el administrador: verificar una bodega sin ver los números del auxiliar, dar visto bueno a lo que se creó sobre la marcha, autorizar lo que sale del almacén y resolver las diferencias marcadas.
CON QUÉ HABLA	/api/aprobaciones , /api/alertas , /api/pedidos/pendientes y los endpoints de cierre de bodega.

12.8 Reportes MENÚ 7 · ADMINISTRADOR

ARCHIVO	Reportes.jsx · 453 líneas
PESTAÑAS	Consolidado de la toma · Análisis de consumo
QUÉ TIENE	Los botones de generar (consolidado y diferencias), la lista de archivos ya generados con su estado, y el panel de vista previa a la derecha.
QUÉ RESUELVE	La salida hacia My Inventory, con los códigos oficiales. La previsualización evita cargar el archivo equivocado.
CON QUÉ HABLA	/api/reportes, /api/reportes/diferencias, /api/reportes/vista-previa y /api/reportes/descargar.

12.9 Panel **MENÚ 8 · ADMINISTRADOR**

ARCHIVO	Panel.jsx · 396 líneas
PESTAÑAS	Resumen ejecutivo · Bodegas y alertas
QUÉ TIENE	Exactitud de primera pasada, referencias contadas, alertas gestionadas y la gráfica de diferencia absoluta por bodega. En la segunda pestaña, los saldos negativos y las alertas por tipo.
QUÉ RESUELVE	Decidir con datos qué zona revisar primero, en vez de revisarlas todas.
CON QUÉ HABLA	/api/panel y /api/reportes/diferencias-por-bodega.

12.10 Ajustes **MENÚ 9 · ADMINISTRADOR**

ARCHIVO	Ajustes.jsx · 1.074 líneas
PESTAÑAS	Configuración · Gestión de usuarios · Recetas · Registro de trazabilidad
QUÉ RESUELVE	Adaptar la plataforma a la operación sin tocar código: el umbral que dispara las alertas, quién existe y qué bodegas le tocan, los platos con sus ingredientes, y el registro de todo lo que pasó.
CON QUÉ HABLA	/api/ajustes, /api/usuarios, /api/recetas y /api/trazabilidad.

POR QUÉ EL AUXILIAR NO VE ESTA PANTALLA

El umbral y el modo sin conexión ya estaban bloqueados en el backend con `requiere_perfil`. Mostrarle la pantalla igual, en solo lectura, le enseñaba controles administrativos que no le sirven y que no puede tocar. Se oculta del todo, como Auditoría, Reportes y Panel.

12.11 Ayuda MENÚ 10

ARCHIVO	Ayuda.jsx · 364 líneas
QUÉ TIENE	Preguntas frecuentes filtrables, el cuadro para preguntarle al agente por voz o por escrito, la guía de frases que entiende, el estado del sistema en vivo, y tres botones: Escribirle al administrador , Reportar un problema y Ver mensajes . Arriba a la derecha, Ver el recorrido en video .
QUÉ RESUELVE	Que una duda no obligue a salir de la aplicación ni a llamar a nadie.
CON QUÉ HABLA	<code>/api/salud</code> , <code>/api/agente/asistente</code> , <code>/api/soporte/reportar</code> , <code>/api/soporte/mensaje-administrador</code> y <code>/api/soporte/administrador</code> .

12.12 Mensajes MENÚ 11

ARCHIVO	Mensajes.jsx · 248 líneas
QUÉ TIENE	Tres contadores que cambian de significado según el perfil (enviados/recibidos, esperando/por responder, respondidos/resueltos) y la lista de mensajes. Para el administrador, el botón Responder y, si hay uno solo pendiente, un cuadro de responder por voz.
QUÉ RESUELVE	Comunicación de la operación sin salir a un correo que nadie revisa en bodega.
CON QUÉ HABLA	<code>/api/soporte/mensajes</code> y <code>/api/soporte/mensajes/{id}/responder</code> .

12.13 Mi perfil MENÚ 12

ARCHIVO	MiPerfil.jsx · 429 líneas
QUÉ TIENE	Datos personales y foto, cambio de clave, cerrar sesión en todos los dispositivos, verificación en dos pasos, preferencias de voz y el bloque de accesibilidad (alto contraste y tamaño de letra).
QUÉ RESUELVE	Que cada quien administre su propia cuenta sin pedirle nada a un administrador.
CON QUÉ HABLA	/api/usuarios/yo y sus derivados; el cambio de clave y el segundo factor van directo a Cognito.

12.14 Cerrar sesión MENÚ 13

ARCHIVO	CerrarSesion.jsx
QUÉ TIENE	Un diálogo que primero consulta si hay trabajo abierto y lo dice con la bodega y cuántas referencias lleva, antes de ofrecer salir.
QUÉ RESUELVE	Que nadie se vaya con la duda de si perdió lo contado. Y que la sesión se cierre también en el servidor, no solo en ese navegador.
CON QUÉ HABLA	/api/sesiones/{id}/avance para el aviso y /api/usuarios/yo/cerrar-todas para revocar en Cognito.

SI NO SE PUEDE COMPROBAR, NO SE DICE QUE TODO ESTÁ BIEN

Cuando la consulta de trabajo pendiente falla (sin conexión, sesión vencida), el diálogo **no** asume que no hay nada abierto: avisa que no se pudo comprobar. «Puede salir sin problema» es la respuesta más tranquilizadora posible justo cuando menos se sabe, y por eso es la que no se da.

13 Pruebas

Qué hay, cómo se corre y qué cubre.

SUITE	CÓMO SE CORRE	QUÉ CUBRE
Backend 128 pruebas	<code>cd backend</code> <code>python -m pytest tests/ -q</code>	El agente y sus conversaciones, el intérprete, la conciliación, las recetas, la seguridad y las regresiones de legalización.
Frontend 5 archivos	<code>cd frontend</code> <code>npm test</code>	Lógica pura: intérprete local, cola sin conexión, accesibilidad, confirmación por voz y tutorial.

Las pruebas de backend usan **conftest.py** con una base en memoria, así que no tocan **cuentavoz.db**. Las de frontend usan **node:test**, sin infraestructura de renderizado: se prueba la lógica que no depende del DOM.

LAS PRUEBAS CON NOMBRE REGRESIONES NO SON OPCIONALES

`test_regresiones_seguridad.py` y `test_regresiones_legalizacion.py` existen porque esos fallos ya ocurrieron una vez. Si una de ellas se pone en rojo, no es una prueba frágil: es el mismo error volviendo.

14 Despliegue

Cómo llega esto a producción.

El despliegue está descrito en **render.yaml** y detallado en **DESPLIEGUE.md**.

Son tres servicios en Render:

SERVICIO	TIPO	QUÉ ES
cuentavoz-db	PostgreSQL	La base de producción.
cuentavoz-api	Web (Python)	El backend con Uvicorn.
cuentavoz	Estático	El build del frontend, con reescritura para que las rutas de la aplicación funcionen.

La misma base de código sirve en local y en producción: lo único que cambia es **DB_URL** y las llaves. Las credenciales se cargan como variables de entorno del servicio, nunca en el repositorio.

EL SERVIDOR SE DUERME

En el plan gratuito de Render el backend entra en reposo tras un rato sin tráfico, y la primera petición puede tardar hasta un minuto. Por eso las pantallas que dependen de la red avisan «el servidor estaba en reposo y está despertando» en vez de dar un error seco.

15 Decisiones de diseño

Por qué las cosas son como son. Útil antes de cambiarlas.

DECISIÓN	POR QUÉ
Confirmar siempre antes de guardar	El reconocimiento de voz se equivoca. Un inventario con datos que nadie aprobó es peor que no tener inventario.
Nunca sobrescribir	Sin el valor anterior no hay auditoría posible.
Intérprete local además de Gemini	Una bodega con mala señal, o una cuota agotada, no pueden dejar a nadie sin poder contar.
La clave la guarda Cognito	Lo que no se almacena no se puede filtrar.
Permisos también en el endpoint	Ocultar una opción del menú no es seguridad.
El auxiliar no ve pantallas que no puede usar	Mostrarle Ajustes en solo lectura le enseña controles que no puede tocar y hace ver la herramienta más complicada de lo que es para él.
Los reportes se previsualizan	Descargar el archivo equivocado y cargarlo a My Inventory cuesta más que un clic de más.
Un mensaje se responde una sola vez	Dos respuestas al mismo mensaje pisaban la anterior en silencio, y quien ya la había leído no se enteraba del cambio.

16 Cuando algo falla

Los tropiezos más frecuentes, y qué revisar primero.

SÍNTOMA	CAUSA PROBABLE	QUÉ HACER
«Sesión inválida o vencida» al entrar	Quedó un token viejo en <code>localStorage</code> .	Limpiar el almacenamiento del sitio y volver a entrar.
El navegador dice «error de red» pero el servidor responde	El origen no está en la lista de CORS.	Agregarlo a <code>ORIGEN_PERMITIDO</code> y reiniciar el backend.
El agente entiende menos de lo esperado	No hay <code>GOOGLE_API_KEY</code> : está corriendo el intérprete local.	Ponerla en el <code>.env</code> . Lo confirma Ayuda › Estado del sistema .
Un auxiliar no ve su bodega	No se la asignaron.	Ajustes › Gestión de usuarios › asignar bodegas.
La primera petición tarda un minuto	El servidor estaba en reposo.	Esperar. La aplicación ya lo avisa en pantalla.
El correo al administrador no sale	No hay <code>BREVO_API_KEY</code> .	Es esperado: el mensaje igual queda en la bandeja y en la trazabilidad.
Una prueba de regresiones se pone en rojo	Volvió un fallo que ya se había corregido.	No ajustar la prueba. Arreglar el código.

CuentaVoz · Tu voz cuenta

Guía técnica, versión 5.0 — agosto de 2026.
Elaborada por el equipo StockXperts para la
operación de bodegas de Colsubsidio.

El código manda

Esta guía explica el porqué y el cómo, no reemplaza al repositorio. Cuando una y otro no coincidan, el código tiene la razón — y esta guía tiene un error que vale la pena corregir.

17 El código completo

El proyecto entero, archivo por archivo. Este apéndice **se genera leyendo el repositorio** cada vez que se compila la guía: no está transcrito a mano, así que no puede quedar desfasado del código real.

17.1 Backend · el nucleo

backend/main.py

4988 LÍNEAS

La aplicacion FastAPI entera: los 90 endpoints, el orquestador del agente, los middlewares de CORS y cabeceras, y el arranque.

```

1  """La API de CuentaVoz. Aqui se conectan la tableta, el agente y la base."""
2  import json
3  import os
4  import re
5  import secrets
6  import socket
7  import threading
8  import unicodedata
9  import urllib.error
10 import urllib.request
11 from datetime import timedelta
12 from horario import ahora
13
14 from dotenv import load_dotenv
15 load_dotenv()
16
17 # Apagado por defecto (sin SENTRY_DSN no llama a init ni manda nada a
18 # ningun lado) - se deja cableado para que activarlo en produccion sea
19 # solo poner la variable de entorno, sin tocar codigo. send_default_pii
20 # en False a proposito: Usuario.correo es un correo real de empleado, no
21 # hay que dejarlo viajar a un tercero por defecto.
22 import sentry_sdk
23 _SENTRY_DSN = os.environ.get("SENTRY_DSN", "").strip()
24 if _SENTRY_DSN:
25     sentry_sdk.init(dsn=_SENTRY_DSN, traces_sample_rate=0.0, send_default_pii=False)
26
27 from fastapi import (FastAPI, WebSocket, WebSocketDisconnect, Depends,
28                     HTTPException, UploadFile, File, Request)
29 from fastapi.concurrency import run_in_threadpool
30 from fastapi.middleware.cors import CORSMiddleware
31 from fastapi.responses import JSONResponse, Response
32 from pydantic import BaseModel, Field
33 from slowapi import Limiter
34 from slowapi.util import get_remote_address
35 from slowapi.errors import RatelimitExceeded
36
37 from bd import Sesion, iniciar_bd
38 from modelos import (Usuario, Bodega, Artículo, StockSistema, SesionConteo,
39                     Conteo, Alerta, Traza, LineaServicio, AsignacionBodega,
40                     Aprobacion, HistorialCierre, ConfigClave,
41                     Receta, RecetaIngrediente, MensajeSoporte)
42 from seguridad import (usuario_actual, requiere_perfil, registrar, esquema,
43                       COGNITO_REGION, COGNITO_USER_POOL_ID)
44 from agente.orquestador import procesar_turno, ESTADOS, avance
45 from servicios.recetas import (calcular_pedido, comparar_legalizacion,
46                               analisis_consumo, detalle_receta, _filas_a_legalizar)
47 from servicios.validacion import umbral_actual, validar_conteo
48 from servicios import analitica
49 from servicios.interprete import _numero as _numero_de_texto
50 import reportes
51
52
53 def _cliente_cognito():
54     """El backend administra Cognito (sembrar las cuentas demo, cerrar
55     todas las sesiones de una persona) con SU PROPIA credencial de AWS -
56     distinta de la que se uso para crear el User Pool: esta solo puede
57     AdminCreateUser/AdminSetUserPassword/AdminGetUser/
58     AdminUserGlobalSignOut sobre este User Pool en particular, nada mas
59     (ver el usuario de IAM "cuentavoz-backend"). Sin AWS_ACCESS_KEY_ID
60     configurada (desarrollo local sin AWS a mano), boto3 revienta al
61     hacer la primera llamada real, no aqui - se deja igual que el patron
62     ya usado para Gemini/Brevo: se degrada en el momento de usarla, no al

```

```

63     arrancar.""
64     import boto3
65     return boto3.client("cognito-idp", region_name=COGNITO_REGION)
66
67 app = FastAPI(title="CuentaVoz", version="1.0.0",
68               description="Asistente por voz para inventarios · Colsubsidio")
69
70 limiter = Limiter(key_func=get_remote_address)
71 app.state.limiter = limiter
72
73
74 @app.exception_handler(RateLimitExceeded)
75 def _manejador_limite_excedido(request: Request, exc: RateLimitExceeded):
76     # El manejador por defecto de slowapi responde {"error": "..."} en vez
77     # de {"detail": "..."}, que es la clave que pedir() (api.js) y toda la
78     # app ya saben leer para mostrar el mensaje real - sin esto, cualquier
79     # pantalla que tropieza con un limite (ingreso, huella, busqueda de
80     # perfil...) mostraba "Error 429" en vez de avisar que hay que esperar.
81     respuesta = JSONResponse(
82         {"detail": "Demasiados intentos. Espere un momento y vuelva a intentarlo."},
83         status_code=429)
84     _poner_cabeceras_cors(respuesta, request) # mismo motivo que en el de 500
85     return respuesta
86
87
88 @app.exception_handler(Exception)
89 def _manejador_error_no_capturado(request: Request, exc: Exception):
90     # FastAPI ya responde 500 solo ante una excepcion no manejada, pero sin
91     # la forma {"detail": "..."} que pedir() (api.js) espera - sin esto, un
92     # error real (no uno ya convertido a HTTPException) se veia en pantalla
93     # como un mensaje crudo en vez del aviso en español de siempre.
94     if _SENTRY_DSN:
95         sentry_sdk.capture_exception(exc)
96     print(f"[error no capturado] {request.method} {request.url.path}: "
97           f"{type(exc).__name__}: {exc}")
98     respuesta = JSONResponse({"detail": "Ocurrió un error inesperado. Intente de nuevo."},
99                             status_code=500)
100     _poner_cabeceras_cors(respuesta, request)
101     return respuesta
102
103
104 def _poner_cabeceras_cors(respuesta, request: Request) -> None:
105     """Le pega las cabeceras CORS a mano a una respuesta de error.
106
107     Los manejadores de excepcion de Starlette corren POR FUERA del
108     CORSMiddleware, así que sus respuestas salen sin esas cabeceras. El
109     navegador entonces no bloquea el error: bloquea la RESPUESTA ENTERA, y
110     fetch() lanza un TypeError indistinguible de quedarse sin red. Costo
111     real de ese detalle: un 500 de verdad (una insercion que fallaba en
112     Postgres) llego a la pantalla como "Sin conexion con el servidor.
113     Revise el Wi-Fi", mandando a buscar el problema en el router mientras
114     el servidor respondia perfecto. Con las cabeceras puestas, el error se
115     ve tal cual es."""
116     origen = request.headers.get("origin")
117     if origen and origen in _origenes:
118         respuesta.headers["Access-Control-Allow-Origin"] = origen
119         respuesta.headers["Access-Control-Allow-Credentials"] = "true"
120         respuesta.headers["Vary"] = "Origin"
121
122
123 _origenes = {o.strip() for o in os.getenv("ORIGEN_PERMITIDO", "").split(",") if o.strip()}
124 # 5173 es "npm run dev"; 4173 es "npm run preview" (el build de produccion,
125 # el unico que arma el service worker real - hace falta para probar el
126 # modo sin conexion tal como queda desplegado, no solo en modo desarrollo).
127 _origenes |= {"http://localhost:5173", "http://127.0.0.1:5173",
128              "http://localhost:4173", "http://127.0.0.1:4173"}
129
130 app.add_middleware(
131     CORSMiddleware,
132     allow_origins=list(_origenes),
133     allow_credentials=True, allow_methods=["*"], allow_headers=["*"],
134 )
135
136
137 @app.middleware("http")
138 async def cabeceras(request: Request, llamar):
139     resp = await llamar(request)
140     resp.headers["X-Content-Type-Options"] = "nosniff"
141     resp.headers["X-Frame-Options"] = "SAMEORIGIN"
142     resp.headers["Referrer-Policy"] = "no-referrer"
143     # obliga HTTPS en cada visita futura del navegador, no solo en esta -
144     # sin esto, un enlace http:// (o un portal cautivo de Wi-Fi) puede
145     # interceptar la primera peticion antes de que el servidor la redirija.
146     resp.headers["Strict-Transport-Security"] = "max-age=31536000; includeSubDomains"
147     return resp
148

```

```

149
150 # Las 4 cuentas de demostracion (luis/diana/stephanie/valentina, clave
151 # "StockXperts1" - la misma que aparece en el README) solo se siembran si
152 # esta variable esta activa. Por defecto NO lo esta: sin esto, cualquier
153 # despliegue con la base de usuarios vacia -incluido un Render de
154 # produccion real, con datos reales, antes de que alguien cree las
155 # cuentas de verdad- quedaba con una cuenta de perfil auditor (diana, con
156 # permisos de administrador) accesible con una clave publicada. Para el
157 # hackathon esto se activa a proposito en render.yaml; para un uso real
158 # despues, basta con no ponerla (o quitarla) para que esas cuentas nunca
159 # aparezcan solas.
160 SEMBRAR_DEMO = os.getenv("SEMBRAR_DEMO", "").strip() == "1"
161 # La clave de las cuentas de demostracion vive en Cognito, no en esta base -
162 # tiene que cumplir la politica del User Pool (min. 8, mayuscula+minuscula+
163 # numero). "StockXperts" sola no trae numero; se le agrego un "1" al
164 # migrar a Cognito (ver README.md/LEEME_PRIMERO.md, ya actualizados).
165 CLAVE_DEMO = "StockXperts1"
166
167
168 def _crear_usuario_cognito(nombre: str, correo: str, clave: str) -> bool:
169     """Crea la cuenta en Cognito con la clave dada, ya confirmada
170     (MessageAction=SUPPRESS: no le manda el correo de bienvenida
171     automatico de Cognito - esta app ya avisa la clave por su cuenta,
172     por voz o en pantalla - y Permanent=True: entra de una con esa clave,
173     sin el paso extra de "cambie su clave temporal" que exige Cognito
174     para claves no permanentes). Usado tanto al sembrar las cuentas de
175     demo como al crear un usuario nuevo por voz o desde Ajustes. Si ya
176     existe, no revienta: se ignora y se avisa por consola."""
177     cliente = _cliente_cognito()
178     try:
179         cliente.admin_create_user(
180             UserPoolId=COGNITO_USER_POOL_ID, Username=nombre,
181             UserAttributes=[{"Name": "email", "Value": correo},
182                             {"Name": "email_verified", "Value": "true"},
183                             {"Name": "name", "Value": nombre}],
184             MessageAction="SUPPRESS",
185         )
186         cliente.admin_set_user_password(
187             UserPoolId=COGNITO_USER_POOL_ID, Username=nombre,
188             Password=clave, Permanent=True,
189         )
190         return True
191     except cliente.exceptions.UsernameExistsException:
192         return False
193     except Exception as e:
194         print(f"[cognito] no se pudo crear la cuenta de {nombre}: {e}")
195         return False
196
197
198 def _actualizar_correo_cognito(nombre: str, correo: str) -> bool:
199     """Cognito es quien manda el código al recuperar clave o al confirmar
200     un registro - si el correo cambia aquí (Mi perfil o Ajustes) sin
201     avisarle a Cognito, esos códigos seguirían yendo al correo viejo para
202     siempre, sin que nada en la pantalla lo delatara. email_verified=true
203     porque este cambio lo hace un administrador o la propia persona ya
204     autenticada, no alguien sin verificar tratando de robar la cuenta."""
205     try:
206         _cliente_cognito().admin_update_user_attributes(
207             UserPoolId=COGNITO_USER_POOL_ID, Username=nombre,
208             UserAttributes=[{"Name": "email", "Value": correo},
209                             {"Name": "email_verified", "Value": "true"}],
210         )
211         return True
212     except Exception as e:
213         print(f"[cognito] no se pudo actualizar el correo de {nombre}: {e}")
214         return False
215
216
217 @app.on_event("startup")
218 def arranque():
219     iniciar_bd()
220     with Sesión() as s:
221         if SEMBRAR_DEMO and s.query(Usuario).count() == 0:
222             s.add_all([
223                 Usuario(nombre="luis", perfil="auxiliar",
224                         correo="lnieto@colsubsidio.com", codigo="CS-48127"),
225                 Usuario(nombre="diana", perfil="auditor",
226                         correo="diana@colsubsidio.com", codigo="CS-48200"),
227                 Usuario(nombre="stephanie", perfil="auxiliar",
228                         correo="stephanie@colsubsidio.com", codigo="CS-48311"),
229                 Usuario(nombre="valentina", perfil="auxiliar",
230                         correo="valentina@colsubsidio.com", codigo="CS-48342"),
231             ])
232     s.commit()
233     for nombre, correo in (("luis", "lnieto@colsubsidio.com"),
234                           ("diana", "diana@colsubsidio.com"),

```

```

235         ("stephanie", "stephanie@colsubsidio.com"),
236         ("valentina", "valentina@colsubsidio.com")):
237     _crear_usuario_cognito(nombre, correo, CLAVE_DEMO)
238     print(f"[arranque] usuarios de prueba creados (clave {CLAVE_DEMO})")
239     elif s.query(Usuario).count() == 0:
240         print("[arranque] base de usuarios vacia y SEMBRAR_DEMO no esta activo: "
241               "no se crearon cuentas de prueba. Regístrese desde Ingreso, "
242               "o active SEMBRAR_DEMO=1 para la demo.")
243
244     # bodegas asignadas por persona: solo la primera vez, solo si ya hay
245     # bodegas cargadas (cargar_excel.py corre antes que la API), y solo
246     # si las cuentas de prueba de arriba existen de verdad - sin
247     # SEMBRAR_DEMO no hay "luis"/"stephanie"/"valentina" a quien
248     # asignarles nada.
249     if SEMBRAR_DEMO and s.query(AsignacionBodega).count() == 0 and s.query(Bodega).count() > 0:
250         # del Excel real de Colsubsidio, solo un subconjunto de bodegas
251         # trae existencia de sistema (SD) cargada - las demas son
252         # ubicaciones del catalogo sin stock_sistema asociado. Priorizar
253         # esas para el reparto demo evita que un auxiliar de prueba abra
254         # una bodega con "0 referencias" y no pueda mostrar diferencias
255         # contra el sistema en una demo en vivo.
256         con_stock = {bid for (bid,) in s.query(StockSistema.bodega_id).distinct()}
257         todas = s.query(Bodega).order_by(Bodega.nombre_oficial).all()
258         bodegas = sorted(todas, key=lambda b: (b.id not in con_stock, b.nombre_oficial))
259         usuarios = {u.nombre: u for u in s.query(Usuario).all()}
260         reparto = {"luis": bodegas[0:3], "stephanie": bodegas[3:6],
261                   "valentina": bodegas[6:9]}
262         total = 0
263         for nombre, asignadas in reparto.items():
264             u = usuarios.get(nombre)
265             if not u:
266                 continue
267             for b in asignadas:
268                 s.add(AsignacionBodega(usuario_id=u.id, bodega_id=b.id))
269                 total += 1
270         s.commit()
271         if total:
272             print(f"[arranque] {total} bodegas asignadas a los auxiliares de prueba")
273
274     # el conteo inicial de negativos, para que el panel muestre "N -> 0"
275     if s.query(ConfigClave).filter_by(clave="negativos_iniciales").first() is None:
276         neg = s.query(StockSistema).filter(StockSistema.cantidad_sd < 0).count()
277         s.add(ConfigClave(clave="negativos_iniciales", valor=str(neg)))
278         s.commit()
279
280     # un par de cierres de ejemplo, para que el histórico de exactitud
281     # del panel no arranque vacío (etiquetados como semilla, no reales)
282     if s.query(HistorialCierre).count() == 0 and s.query(Bodega).count() > 0:
283         primera = s.query(Bodega).order_by(Bodega.nombre_oficial).first()
284         from datetime import timedelta
285         base = ahora() - timedelta(days=120)
286         for i, exact in enumerate([94.2, 95.8, 96.5, 97.1]):
287             s.add(HistorialCierre(bodega_id=primera.id, exactitud=exact,
288                                   referencias=0, diferencias=0,
289                                   fecha=base + timedelta(days=30 * i)))
290         s.commit()
291         print("[arranque] histórico de exactitud sembrado (4 puntos de ejemplo)")
292
293
294 @app.get("/api/salud")
295 def salud():
296     with Sesion() as s:
297         return {"api": "ok",
298               "bodegas": s.query(Bodega).count(),
299               "articulos": s.query(Articulo).count(),
300               "stock": s.query(StockSistema).count(),
301               "gemini": bool(os.getenv("GOOGLE_API_KEY", "").strip()),
302               "cognito": _estado_cognito()}
303
304
305 def _estado_cognito() -> dict:
306     """Con que User Pool y App Client esta trabajando ESTE backend.
307
308     No es informacion secreta: el User Pool Id y el App Client Id viajan en
309     el JavaScript que se descarga cualquiera que abra la aplicacion (ver
310     frontend/src/cognito.js) - por eso pueden ir aqui sin problema, y no se
311     expone nada mas (ni llaves de AWS, ni claves).
312
313     Existe porque estas tres variables van como sync:false en render.yaml,
314     o sea que viven solo en el panel de Render: si una queda vacia o
315     desactualizada, TODOS los tokens se rechazan con "Sesion invalida o
316     vencida.", que suena a problema de la persona cuando en realidad es de
317     configuracion. Comparar esto contra el frontend responde en un segundo
318     algo que si no toca adivinar."""
319     from seguridad import (COGNITO_REGION as reg, COGNITO_USER_POOL_ID as pool,
320                           COGNITO_APP_CLIENT_ID as cli, _jwks_client)

```

```

321     return {"region": reg,
322            "user_pool_id": pool or "(sin definir)",
323            "app_client_id": cli or "(sin definir)",
324            "puede_verificar_tokens": _jwks_client is not None}
325
326
327 # ----- identidad -----
328 def _buscar_usuario_por_entrada(s, entrada: str):
329     """Se puede ingresar con el nombre de usuario o con el código de
330     empleado (ej. CS-48127) - lo que la persona tenga a la mano."""
331     entrada = entrada.strip()
332     return s.query(Usuario).filter(
333         (Usuario.nombre == entrada.lower()) | (Usuario.codigo == entrada.upper())
334     ).first()
335
336
337 @app.get("/api/usuarios/perfil")
338 @limiter.limit("20/minute")
339 def perfil_por_usuario(request: Request, usuario: str = ""):
340     """Para que la pantalla de ingreso muestre «Auxiliar» o «Administrador»
341     apenas se escribe el usuario, sin esperar a iniciar sesión. Solo
342     devuelve el perfil (nada más sensible: ni nombre, ni correo) y esta
343     limitado por minuto para no servir de lista para adivinar usuarios
344     válidos. El login mismo (usuario+clave) lo resuelve Cognito
345     directamente desde el frontend - este backend nunca ve la clave."""
346     if not usuario.strip():
347         return {"perfil": None}
348     with Sesión() as s:
349         u = _buscar_usuario_por_entrada(s, usuario)
350         if not u or not u.activo:
351             return {"perfil": None}
352     return {"perfil": u.perfil}
353
354
355 class RegistroCompletadoIn(BaseModel):
356     nombre_completo: str
357     codigo: str = ""
358     correo: str
359
360
361 @app.post("/api/registro-completado")
362 @limiter.limit("10/minute")
363 def registro_completado(request: Request, datos: RegistroCompletadoIn,
364                        token: str = Depends(esquema)):
365     """Cognito ya creo y confirmo la cuenta (el código que Cognito manda al
366     correo al registrarse) antes de llegar aquí - el frontend inicia
367     sesión contra Cognito justo después de confirmar y llama esto con ese
368     token, no con datos sueltos. Aquí solo se crea la fila LOCAL que sabe
369     el perfil y las bodegas asignadas, cosas que Cognito no conoce. El
370     nombre de usuario sale del token ya verificado, nunca del cuerpo de
371     la petición - y el perfil queda SIEMPRE "auxiliar": nadie puede
372     autoasignarse el perfil de auditor solo registrándose, ese ascenso
373     lo hace un administrador después desde Ajustes.
374
375     La cuenta queda ACTIVA de inmediato: quien confirmo el código que
376     Cognito le mando al correo ya puede entrar, sin esperar a que un
377     administrador la habilite. Se probó la variante con aprobación previa
378     y se descartó por decisión de producto - agregaba una espera entre
379     registrarse y poder trabajar que no compensaba, dado que el perfil
380     siempre nace como auxiliar y sus permisos están limitados a las
381     bodegas que un auditor le asigne (ver AsignacionBodega). Si alguna vez
382     hay que bloquear a alguien, editar_usuario ya permite desactivarlo."""
383     from seguridad import _reclamos_cognito
384     reclamos = _reclamos_cognito(token) if token else None
385     if reclamos is None:
386         raise HTTPException(401, "Sesión de Cognito inválida o vencida.")
387     nombre = (reclamos.get("username") or "").strip().lower()
388     if not nombre:
389         raise HTTPException(400, "No se pudo identificar el usuario de Cognito.")
390     with Sesión() as s:
391         if s.query(Usuario).filter_by(nombre=nombre).first():
392             # ya existe (ej. reintento de red tras un primer registro que
393             # si alcanzo a crear la fila) - no es un error, solo confirma.
394             return {"ok": True, "ya_existia": True}
395         nuevo = Usuario(nombre=nombre, perfil="auxiliar",
396                        correo=(datos.correo or "").strip(),
397                        codigo=(datos.codigo or "").strip().upper())
398         s.add(nuevo)
399         s.commit()
400         s.refresh(nuevo)
401         if not nuevo.codigo:
402             nuevo.codigo = f"CS-{48000 + nuevo.id}"
403         s.commit()
404     registrar(nuevo, "USUARIO", f"{nombre} se registro por su cuenta", "ok")
405     return {"ok": True, "ya_existia": False}
406

```

```

407
408 # ----- el agente -----
409 class TurnoIn(BaseModel):
410     texto: str
411     sesion_id: int = 1
412     # respaldo de resiliencia: si el backend se reinicio a mitad de una
413     # pregunta "¿cual de los dos?", el frontend ya sabe cuales opciones
414     # estaban pendientes y se las recuerda - ver Pedido.jsx/Conteo.jsx.
415     opciones_pendientes: list[dict] | None = None
416     opciones_para: str | None = None
417     # el mismo respaldo mas la bodega abierta: sin esto, un reinicio del
418     # backend deja el frontend mostrando "en conteo" mientras el servidor
419     # ya no sabe cual bodega es - el agente termina diciendo "no hay
420     # ninguna activa" con la bodega bien visible en la pantalla.
421     bodega_id_respaldo: int | None = None
422     bodega_nombre_respaldo: str | None = None
423     # lo mismo para Pedidos: el plato y las porciones casi siempre se dictan
424     # en frases separadas ("arroz con pollo" y despues "para cuatro"); sin
425     # este respaldo, un reinicio del backend a mitad de esas dos frases deja
426     # al agente sin memoria del plato aunque la pantalla lo siga mostrando.
427     preparacion_respaldo: str | None = None
428     porciones_respaldo: int | None = None
429
430
431 @app.post("/api/agente/turno")
432 async def turno(t: TurnoIn, u: Usuario = Depends(usuario_actual)):
433     # procesar_turno() es sincrónico y llama a Gemini (pensar()), que puede
434     # tardar varios segundos - llamarlo directo aquí (una ruta async def)
435     # bloqueaba TODO el event loop mientras tanto: cualquier otra petición,
436     # de cualquier otra persona, a cualquier endpoint (hasta /api/salud) se
437     # quedaba esperando. run_in_threadpool lo saca del hilo principal para
438     # que el resto de la aplicación siga respondiendo mientras Gemini piensa.
439     r = await run_in_threadpool(procesar_turno, t.texto, t.sesion_id, u,
440                                opciones_respaldo=t.opciones_pendientes,
441                                opciones_para_respaldo=t.opciones_para,
442                                bodega_id_respaldo=t.bodega_id_respaldo,
443                                bodega_nombre_respaldo=t.bodega_nombre_respaldo,
444                                preparacion_respaldo=t.preparacion_respaldo,
445                                porciones_respaldo=t.porciones_respaldo)
446     if r.get("bodega"):
447         await difundir_estado()
448     return r
449
450
451 class PreguntarAsistenteIn(BaseModel):
452     texto: str
453     vista: str
454
455
456 # claves = lo mismo que usa ir(destino) en App.jsx; deben calzar exacto,
457 # si no la navegacion por voz apunta a una vista que no existe.
458 _DESTINOS_ASISTENTE = {
459     "inicio": "Inicio", "pedido": "Pedidos", "conteo": "Conteo",
460     "legalizacion": "Legalización", "bodegas": "Bodegas",
461     "auditoria": "Auditoría", "reportes": "Reportes", "panel": "Panel",
462     "ajustes": "Ajustes", "ayuda": "Ayuda", "perfil": "Mi perfil",
463 }
464 _SOLO_AUDITOR_ASISTENTE = {"auditoria", "reportes", "panel", "ajustes"}
465
466 # pantallas con pestañas propias: para no obligar a la persona a llegar y
467 # después dar clic en la pestaña que quería, si la pide de una vez
468 # ("llévame al análisis de consumo") el agente navega directo a ella. La
469 # clave interna debe ser la misma que usa cada vista en su useState(tab).
470 _PESTANAS_ASISTENTE = {
471     "reportes": {"consolidado": "Consolidado de la toma", "analisis": "Análisis de consumo"},
472     "ajustes": {"config": "Configuración", "usuarios": "Gestión de usuarios",
473                "recetas": "Recetas", "traza": "Registro de trazabilidad"},
474     "panel": {"resumen": "Resumen ejecutivo", "alertas": "Bodegas y alertas"},
475     "auditoria": {"recuento": "Recuento ciego y cierre", "aprobaciones": "Aprobaciones",
476                 "pedidos": "Pedidos pendientes", "alertas": "Bandeja de alertas"},
477 }
478
479 _FAQ_ASISTENTE = [
480     ("¿Cómo corrijo un conteo ya confirmado?",
481      "Diga «corregir» y luego el valor correcto; el valor anterior se conserva."),
482     ("El agente no entiende un producto",
483      "Dígalo como aparece en la etiqueta; si no existe se puede crear, queda pendiente."),
484     ("¿Qué hago si sale una alerta?",
485      "Recuente; si el número es correcto, confirme: queda marcado para el administrador."),
486     ("Se cayó el internet en plena bodega",
487      "Siga contando: el intérprete local mantiene el flujo y sincroniza al volver la señal."),
488     ("¿Puedo contar dos bodegas a la vez?",
489      "No en el mismo dispositivo; el candado de sesión evita conteos duplicados."),
490 ]
491
492

```

```

493 def _datos_panel_narrados() -> dict:
494     """Los mismos datos de /api/panel/resumen y /api/panel/alertas, ya
495     convertidos a frases listas para hablar - los usan tanto el contexto
496     completo que recibe Gemini como las respuestas determinísticas de
497     _responder_panel_por_voz, para no calcular el texto dos veces."""
498     r = analitica.resumen_ejecutivo()
499     a = analitica.resumen_alertas_panel()
500     dif_bod = r["diferencia_por_bodega"]
501     texto_dif = ("sin diferencias registradas todavía" if not dif_bod else
502                 "; ".join(f"{x['bodega'].title(): {x['diferencia']}" for x in dif_bod[:6]))
503     stock = r["stock_por_unidad"]
504     texto_stock = ("sin datos de stock todavía" if not stock else
505                   "; ".join(f"{x['unidad']} {x['pct']}% ({x['cantidad']})" for x in stock))
506     hist = r["historial_exactitud"]
507     if len(hist) < 2:
508         texto_hist = "todavía no hay suficientes cierres para ver una tendencia"
509     else:
510         tendencia = "mejorando" if hist[-1]["exactitud"] >= hist[0]["exactitud"] else "bajando"
511         texto_hist = (f"{len(hist)} tomas registradas, la más reciente en "
512                      f"{hist[-1]['bodega'].title()} con {hist[-1]['exactitud']}%, "
513                      f"tendencia {tendencia}")
514     etiquetas_estado = {"cerrada": "cerradas", "en conteo": "en conteo",
515                        "en auditoria": "en auditoría", "pendiente": "pendientes"}
516     texto_estado = (" ".join(f"{n} {etiquetas_estado.get(e, e)}"
517                              for e, n in a["estado_bodegas"].items())
518                    or "sin bodegas registradas")
519     alertas_tipo = a["alertas_por_tipo"]
520     texto_alertas_tipo = ("sin alertas registradas todavía" if not alertas_tipo else
521                           "; ".join(f"{x['tipo']}: {x['cantidad']}" for x in alertas_tipo))
522     descuadres = a["descuadres_recurrentes"]
523     texto_descuadres = ("sin descuadres repetidos todavía" if not descuadres else
524                         "; ".join(f"{x['articulo'].title(): ({x['tipo']}, diferencia "
525                                f"{x['diferencia']}) en {x['tomas']} tomas, acción sugerida: "
526                                f"{x['accion']})" for x in descuadres[:5]))
527     return {"r": r, "a": a, "texto_dif": texto_dif, "texto_stock": texto_stock,
528            "texto_hist": texto_hist, "texto_estado": texto_estado,
529            "texto_alertas_tipo": texto_alertas_tipo, "texto_descuadres": texto_descuadres}
530
531 # Preguntas frecuentes del Panel gerencial, resueltas sin pasar por Gemini.
532 # Sin esto, una pregunta tan común como "¿cuántas bodegas están cerradas?"
533 # a veces terminaba navegando a la pantalla Bodegas en vez de contestar
534 # (confirmado en pruebas: al mencionar el nombre de otra pantalla, Gemini
535 # no siempre distingue preguntar de navegar) - las mismas frases que se
536 # ofrecen en el desplegable "Ver frases exactas" de Panel quedan así
537 # garantizadas.
538 _PANEL_PREGUNTAS = [
539     (re.compile(r"\bprimera\s+pasada\b", re.IGNORECASE),
540      lambda d: f"La exactitud primera pasada es del {d['r']['exactitud_primera_pasada']}%, "
541                "promedio de cierres."),
542     (re.compile(r"\breferencias?\b.*\bcontad\b.*\bcontad\b.*\bcontad\b.*\bcontad\b", re.IGNORECASE),
543      lambda d: f"Se han contado {d['r']['referencias_contadas']} referencias en toda la "
544                "operación."),
545     (re.compile(r"\balertas?\b.*\bgestionad\b", re.IGNORECASE),
546      lambda d: f"Van {d['r']['alertas_gestionadas']} alertas gestionadas de "
547                f"{d['r']['alertas_total']} en total."),
548     (re.compile(r"\bcontad\b.*\bcontad\b.*\bcontad\b.*\bcontad\b", re.IGNORECASE),
549      lambda d: f"Estado de las bodegas: {d['texto_estado']}.",
550      (re.compile(r"\bcontad\b.*\bcontad\b.*\bcontad\b.*\bcontad\b", re.IGNORECASE),
551       lambda d: f"Van {d['r']['bodegas_cerradas']} bodegas cerradas de {d['r']['bodegas_total']} "
552                 "en total, con doble firma digital."),
553      (re.compile(r"\bdiferencia\b.*\bdiferencia\b.*\bdiferencia\b", re.IGNORECASE),
554       lambda d: f"Diferencia absoluta por bodega: {d['texto_dif']}.",
555       (re.compile(r"\bstock\b.*\bstock\b.*\bstock\b", re.IGNORECASE),
556        lambda d: f"Stock por unidad de medida: {d['texto_stock']}.",
557        (re.compile(r"\btendencia\b.*\btendencia\b", re.IGNORECASE),
558         lambda d: f"Exactitud por toma de inventario: {d['texto_hist']}.",
559         (re.compile(r"\bnegativos?\b.*\bnegativos\b", re.IGNORECASE),
560          lambda d: (f"Saldo negativo en el sistema: {d['a']['negativos_actuales']} artículos. "
561                    "La corrección se hace en My Inventory, no en CuentaVoz."
562                    if d['a']['negativos_iniciales'] == d['a']['negativos_actuales'] else
563                    f"Saldo negativo: {d['a']['negativos_iniciales']} al inicio de este período, "
564                    f"{d['a']['negativos_actuales']} ahora."),
565          (re.compile(r"\btipo\b.*\btipo\b", re.IGNORECASE),
566           lambda d: f"El tiempo promedio de conteo por bodega es de "
567                     f"{d['a']['tiempo_promedio_min']} minutos."),
568          (re.compile(r"\balias\b.*\balias\b", re.IGNORECASE),
569           lambda d: f"El agente tiene {d['a']['alias_aprendidos']} alias aprendidos."),
570          (re.compile(r"\balertas\b.*\balertas\b", re.IGNORECASE),
571           lambda d: f"Alertas por tipo: {d['texto_alertas_tipo']}.",
572          (re.compile(r"\bdescuadre\b.*\bdescuadre\b", re.IGNORECASE),
573           lambda d: f"Descuadres recurrentes: {d['texto_descuadres']}.")
574          )
575          )
576          )
577          )
578 ]

```

```

579
580
581 def _responder_panel_por_voz(texto: str, u: Usuario) -> dict | None:
582     if u.perfil != "auditor":
583         return None
584     for patron, respuesta in _PANEL_PREGUNTAS:
585         if patron.search(texto):
586             return {"respuesta_hablada": respuesta(_datos_panel_narrados()),
587                 "accion": None, "destino": None, "pestana": None}
588     return None
589
590
591 def _formato_acceso_hablado(fecha_dt) -> str | None:
592     """Mismo criterio que formatoAcceso() en MiPerfil.jsx (hoy HH:MM vs
593     DD/MM HH:MM) - para que lo que diga el agente coincida con lo que la
594     pantalla ya muestra escrito."""
595     if not fecha_dt:
596         return None
597     if fecha_dt.date() == ahora().date():
598         return f'hoy a las {fecha_dt.strftime('%H:%M')}"
599     return f"el {fecha_dt.strftime('%d/%m')} a las {fecha_dt.strftime('%H:%M')}"
600
601
602 def _datos_perfil_narrados(u: Usuario) -> dict:
603     from agente.cerebro import VOCES, VOZ_DEFEECTO
604     with Sesion() as s:
605         usr = s.get(Usuario, u.id)
606         asigs = s.query(AsignacionBodega).filter_by(usuario_id=u.id).all()
607         nombres_bodegas = [b.nombre_oficial.title() for b in
608             (s.get(Bodega, a.bodega_id) for a in asigs) if b]
609         dias_pin = (ahora() - (usr.pin_actualizado or ahora())).days
610         voz = usr.idioma_voz if usr.idioma_voz in VOCES else VOZ_DEFEECTO
611         return {
612             "ultimo_acceso_hablado": _formato_acceso_hablado(usr.ultimo_acceso),
613             "pin_vence_en_dias": max(90 - dias_pin, 0),
614             "n_bodegas": len(nombres_bodegas),
615             "texto_bodegas": "; ".join(nombres_bodegas),
616             "perfil": usr.perfil,
617             "velocidad_voz": usr.velocidad_voz,
618             "confirmacion_hablada": bool(usr.confirmacion_hablada),
619             "voz_nombre": VOCES[voz]["nombre"],
620             "voz_etiqueta": VOCES[voz]["etiqueta"],
621         }
622
623
624 _PERFIL_PREGUNTAS = [
625     (re.compile(r"[uú]ltimo\s+acceso|cu[áá]ndo\s+(entr[eé]|ingres[eé])", re.IGNORECASE),
626         lambda d: f"Su último acceso fue {d['ultimo_acceso_hablado']}" if d["ultimo_acceso_hablado"]
627         else "Todavía no hay un acceso anterior registrado."),
628     (re.compile(r"\bpin\b.*\bvence|vence\b.*\bpin\b|cu[áá]nto\s+de[íi]as.*\bpin\b", re.IGNORECASE),
629         lambda d: f"Su PIN vence en {d['pin_vence_en_dias']} días."),
630     (re.compile(r"bodegas?\s+(tengo|asignad\w*)|cu[áá]nto\s+bodegas", re.IGNORECASE),
631         lambda d: (f"tiene {d['n_bodegas']} bodegas asignadas: {d['texto_bodegas']}"
632             if d["n_bodegas"] else "Todavía no tiene bodegas asignadas.")),
633     (re.compile(r"mi\s+perfil\b|qu[eé]\s+perfil|soy\s+(auxiliar|admin\w*)", re.IGNORECASE),
634         lambda d: f"Su perfil es {'administrador de bodega' if d['perfil'] == 'auditor' else 'auxiliar de inventarios'}."),
635     (re.compile(r"qu[eé]\s+voz\s+(tengo|est[áá]|uso)|voz\s+actual", re.IGNORECASE),
636         lambda d: f"Está usando la voz {d['voz_nombre']}: {d['voz_etiqueta'].lower()}."),
637 ]
638
639
640 def _responder_perfil_por_voz(texto: str, u: Usuario) -> dict | None:
641     for patron, respuesta in _PERFIL_PREGUNTAS:
642         if patron.search(texto):
643             return {"respuesta_hablada": respuesta(_datos_perfil_narrados(u)),
644                 "accion": None, "destino": None, "pestana": None}
645     return None
646
647
648 _VOCES_HABLADAS = ("kore", "aoede", "puck", "charon")
649 _VERBO_CAMBIAR_VOZ = re.compile(
650     r"\b(cambia\w*|usa\w*|pon\w*|quiero|eli[jg]\w*|selecciona\w*)\b", re.IGNORECASE)
651
652
653 def _cambiar_voz_por_voz(texto: str, u: Usuario) -> dict | None:
654     """Cambia mi voz a puck / "usa la voz aoede" - exige el nombre EXACTO
655     de una de las cuatro voces (no un genero o descripcion), para no
656     competir con "¿qué voz tengo?" ni con "prueba la voz" (probarVoz solo
657     demuestra la que ya está elegida, no cambia nada)."""
658     if not _VERBO_CAMBIAR_VOZ.search(texto):
659         return None
660     clave = next((k for k in _VOCES_HABLADAS if re.search(rf"\b{k}\b", texto, re.IGNORECASE)), None)
661     if not clave:
662         return None
663     with Sesion() as s:
664         usr = s.get(Usuario, u.id)

```

```

665         usr.idioma_voz = clave
666         s.commit()
667     from agente.cerebro import VOCES
668     registrar(u, "PERFIL", f"{u.nombre} cambió su voz a {VOCES[clave]['nombre']} por voz", "ok")
669     return {"respuesta_hablada": f"Listo, ahora hablo con la voz {VOCES[clave]['nombre']}: "
670           f"{VOCES[clave]['etiqueta'].lower()}.",
671           "accion": "actualizar", "destino": None, "pestana": None, "idioma_voz": clave}
672
673
674     _VELOCIDAD_LENTA = re.compile(r"\blenta\b|m[aá]\s+despacio|m[aá]\s+lento", re.IGNORECASE)
675     _VELOCIDAD_RAPIDA = re.compile(r"\br[aá]pida\b|m[aá]\s+r[aá]pido", re.IGNORECASE)
676     _VELOCIDAD_NORMAL = re.compile(r"velocidad\s+normal|voz\s+normal", re.IGNORECASE)
677
678
679 def _cambiar_velocidad_por_voz(texto: str, u: Usuario) -> dict | None:
680     if _VELOCIDAD_LENTA.search(texto):
681         nueva = "lenta"
682     elif _VELOCIDAD_RAPIDA.search(texto):
683         nueva = "rapida"
684     elif _VELOCIDAD_NORMAL.search(texto):
685         nueva = "normal"
686     else:
687         return None
688     with Sesion() as s:
689         usr = s.get(Usuario, u.id)
690         usr.velocidad_voz = nueva
691         s.commit()
692     registrar(u, "PERFIL", f"{u.nombre} cambió la velocidad de voz a {nueva} por voz", "ok")
693     return {"respuesta_hablada": f"Listo, la velocidad queda en {nueva}.",
694           "accion": "actualizar", "destino": None, "pestana": None, "velocidad_voz": nueva}
695
696
697     _CONFIRMACION_HABLADA_FRASE = re.compile(r"confirmaci[oó]n\s+hablada", re.IGNORECASE)
698
699
700 def _cambiar_confirmacion_hablada_por_voz(texto: str, u: Usuario) -> dict | None:
701     if not _CONFIRMACION_HABLADA_FRASE.search(texto):
702         return None
703     if _ACTIVAR.search(texto):
704         nuevo = True
705     elif _DESACTIVAR.search(texto):
706         nuevo = False
707     else:
708         return None
709     with Sesion() as s:
710         usr = s.get(Usuario, u.id)
711         usr.confirmacion_hablada = 1 if nuevo else 0
712         s.commit()
713     estado = "activada" if nuevo else "desactivada"
714     registrar(u, "PERFIL", f"{u.nombre} dejó la confirmación hablada {estado} por voz", "ok")
715     return {"respuesta_hablada": f"Listo, la confirmación hablada quedó {estado}.",
716           "accion": "actualizar", "destino": None, "pestana": None,
717           "confirmacion_hablada": nuevo}
718
719
720 def _contexto_asistente(vista: str, u: Usuario) -> str:
721     """Un resumen honesto de lo que hay AHORA en esa pantalla, para que el
722     agente conteste con datos reales y no invente nada. Reusa las mismas
723     funciones que ya alimentan cada pantalla, no vuelve a calcular nada."""
724     if vista == "inicio":
725         with Sesion() as s:
726             n_bodegas = s.query(AsignacionBodega).filter_by(usuario_id=u.id).count()
727             alertas = s.query(Alerta).filter_by(resuelta=0).count()
728             historial = s.query(HistorialCierre).all()
729             exact = (round(sum(h.exactitud for h in historial) / len(historial), 1)
730                      if historial else 100.0)
731             return (f"Pantalla: Inicio, de {u.nombre} ({u.perfil}). "
732                   f"Bodegas asignadas a esta persona: {n_bodegas}. "
733                   f"Alertas por revisar en toda la operación: {alertas}. "
734                   f"Exactitud del mes: {exact}%")
735     if vista == "ajustes":
736         a = ver_ajustes(u)
737         # El permiso real de cambiar el modo sin conexión por voz ya se
738         # resuelve antes de llegar aquí (_cambiar_modos_sin_conexion, que
739         # exige perfil auditor) - este texto solo llega a Gemini cuando
740         # esa orden exacta no aplicó (una pregunta suelta, o quien
741         # pregunta no es administrador). Sin condicionarlo al perfil,
742         # Gemini le decía a CUALQUIERA "ya lo activé" aunque el cambio de
743         # verdad solo lo hace un administrador - la misma deshonestidad
744         # que la Regla 6 ya evita en otras respuestas.
745         permiso_offline = (
746             "El modo sin conexión sí se puede cambiar por voz, pero solo por un "
747             "administrador: decir «activa/desactiva el modo sin conexión» lo aplica de "
748             "una, sin tocar el interruptor. En Gestión de usuarios, un administrador "
749             "también puede decir «activa/desactiva a <nombre>» para cambiar el estado de "
750             "otra persona, «crea un usuario llamado <nombre> perfil <auxiliar o "

```

```

751 "auditor»» para crear una cuenta nueva (con PIN temporal), «cambia el "
752 "perfil de <nombre> a auxiliar/administrador» para lo mismo que el botón "
753 "«Editar» (el correo no se cambia por voz), «asigne/quítale la bodega "
754 "<bodega> a <nombre>» para lo mismo que el botón «Asignar bodegas» pero "
755 "sumando o quitando solo esa bodega, no reemplazando toda la lista, y "
756 "«¿quién tiene la bodega <bodega>?» o «¿cuántas bodegas sin asignar hay?» "
757 "para contestar una pregunta puntual sin abrir nada, y «muéstrame/oculta "
758 "la asignación por bodega» para abrir o cerrar la tabla completa - lo "
759 "mismo que el botón «Ver/Ocultar asignación por bodega». En Recetas, un "
760 "administrador también puede decir «crea una receta llamada "
761 "<nombre>, rendimiento <N> porciones, con <cantidad> de <ingrediente>» "
762 "(crea la receta con ese primer ingrediente ya resuelto contra el "
763 "catálogo), «agrega <cantidad> de <ingrediente> a la receta <nombre>» "
764 "(para sumarle más ingredientes después), «quita el ingrediente "
765 "<ingrediente> de la receta <nombre>», «cambia el rendimiento de la "
766 "receta <nombre> a <N> porciones», «agrega la preparación a la receta "
767 "<nombre>: <pasos>» (reemplaza los pasos de cocina completos, tal cual "
768 "se dicten, con los dos puntos como separador del nombre), y «elimina la "
769 "receta <nombre>». "
770 "En Registro de trazabilidad, «exporta el registro de trazabilidad» "
771 "descarga el archivo con los filtros de fecha/persona/acción que ya "
772 "estén puestos en la pantalla en ese momento (no los que diga la "
773 "orden), «acciones por persona» resume cuántas hizo cada quien "
774 "(hoy por defecto; «esta semana»/«este mes»/«en total» cambian el "
775 "periodo), y «¿cuántas acciones tiene <nombre>?» contesta solo por esa "
776 "persona. "
777 "Todo esto SÓLO funciona si la orden usa exactamente esas formas («crea un "
778 "usuario llamado...», «cambia el perfil de... a...», «asigne/quítale la "
779 "bodega... a...», «crea una receta llamada..., rendimiento... porciones, "
780 "con... de...», «agrega... a la receta...», «quita el ingrediente... de "
781 "la receta...», «cambia el rendimiento de la receta... a...», «agrega la "
782 "preparación a la receta...: ...», «elimina la receta...», «exporta el "
783 "registro de trazabilidad», «acciones por persona», «cuántas acciones "
784 "tiene...») - si "
785 "el mensaje llega hasta usted es porque esa frase no se dijo así, el "
786 "nombre de la persona/bodega no coincidió con ninguna existente, o el "
787 "ingrediente no se encontró en el catálogo, así que no invente que ya "
788 "creó, cambió, asignó, agregó o eliminó nada: pida que lo repitan con ese "
789 "formato exacto y el nombre completo tal como aparece en la pantalla."
790 if u.perfil == "auditor" else
791 "El modo sin conexión, el estado de otras personas, crear/editar usuarios, "
792 "asignar bodegas y crear/editar/eliminar recetas solo los puede hacer un "
793 "administrador - esta persona no lo es, así que si lo pide, dígalos con "
794 "honestidad en vez de decir que ya quedó hecho.")
795 return (f"Pantalla: Ajustes. Sección Validación de datos: umbral de anomalía "
796 f"[a['umbral']]%; bloquear cantidades negativas activado (regla fija, no "
797 f"se puede desactivar); exigir confirmación en alertas activado (regla "
798 f"fija); permitir crear productos pendientes activado (regla fija). "
799 f"Sección Conexión y sincronización: modo sin conexión "
800 f"{'activado' if a['offline'] else 'desactivado'}; refresco del tablero "
801 f"en tiempo real. "
802 f"Sección Administración: usuarios activos {a['usuarios_activos']}; "
803 f"aprobarciones pendientes {a['aprobaciones_pendientes']}. Sección Acerca "
804 f"de CuentaVoz: versión {a['version']}, modelo del agente {a['modelo']}. "
805 "El umbral solo lo puede CAMBIAR un administrador, y únicamente con los "
806 "controles de la pantalla (número y botón «Guardar configuración») - eso "
807 f"todavía no cambia por voz. {permiso_offline}")
808 if vista == "panel" and u.perfil == "auditor":
809     d = _datos_panel_narrados()
810     r, a = d["r"], d["a"]
811     return (
812         "Pantalla: Panel gerencial, con dos pestañas: «Resumen ejecutivo» y «Bodegas y "
813         "alertas» - decir el nombre de cualquiera de las dos (o «llévame a...») navega "
814         "directo a ella. "
815         f"Pestaña Resumen ejecutivo - tarjetas: exactitud primera pasada "
816         f"{r['exactitud_primera_pasada']}%, referencias contadas "
817         f"{r['referencias_contadas']}, alertas gestionadas {r['alertas_gestionadas']} de "
818         f"f{r['alertas_total']}, bodegas cerradas {r['bodegas_cerradas']} de "
819         f"f{r['bodegas_total']}. Gráfica «Diferencia absoluta por bodega»: {d['texto_dif']}. "
820         f"Gráfica «Stock por unidad de medida»: {d['texto_stock']}. Gráfica «Exactitud por "
821         f"toma de inventario»: {d['texto_hist']}. "
822         f"Pestaña Bodegas y alertas - tarjetas: saldos negativos en el sistema "
823         f"f{a['negativos_actuales']} artículos (la corrección se hace en My Inventory, no "
824         f"en CuentaVoz - dentro de una sola toma este número normalmente no se mueve), "
825         f"tiempo promedio de conteo por bodega {a['tiempo_promedio_min']} minutos, "
826         f"alias aprendidos por el agente {a['alias_aprendidos']}. Tarjeta «Estado de las "
827         f"bodegas»: {d['texto_estado']}. Tarjeta «Alertas por tipo»: {d['texto_alertas_tipo']}. "
828         f"Tabla «Descuadres recurrentes»: {d['texto_descuadres']}.")
829     if vista == "reportes" and u.perfil == "auditor":
830         d = _datos_reportes_narrados(u)
831         a = d["análisis"]
832         return ("Pantalla: Reportes, con dos pestañas: «Consolidado de la toma» y «Análisis de "
833             "consumo». Aquí se genera el consolidado para My "
834             "Inventory, las diferencias por bodega, y el análisis de consumo "
835             "de los últimos 30 días. Los tres SÍ se pueden generar por voz "
836             "(«genera el consolidado», «exporta las diferencias», «exporta el ")

```

```

837         "análisis de consumo») - queda igual que si le hubiera dado clic "
838         f"al botón. Archivos generados recientemente: {d['texto_recientes']}. "
839         f"Pestaña Análisis de consumo, últimos {a['dias']} días: pedido total "
840         f"{a['pedido_total']} kg en {a['servicios_periodo']} servicios, usado "
841         f"realmente {a['usado_total']} kg ({a['aprovechamiento']}% de "
842         f"aprovechamiento), ahorro potencial {a['ahorro_potencial']} kg al mes. "
843         f"Insumos subutilizados: {d['texto_subutil']}. También se puede pedir "
844         f"«muéstrame el archivo de <consolidado o diferencias>» para ver su "
845         f"contenido, igual que darle clic a la tarjeta.")
846     if vista == "auditoria" and u.perfil == "auditor":
847         d = _datos_auditoria_narrados(u)
848         ra = d["resumen_alertas"]
849         nombres_aprob = ("; ".join(a["nombre"].title() for a in d["aprobaciones"]))
850             if d["aprobaciones"] else "ninguna")
851         nombres_pedidos = ("; ".join(f"{p['persona'].title()}: {p['plato']}" for p in d["pedidos"]))
852             if d["pedidos"] else "ninguno")
853     return (f"Pantalla: Auditoría, con cuatro pestañas: «Recuento ciego y cierre», "
854            f"«Aprobaciones», «Pedidos pendientes» y «Bandeja de alertas» - decir el "
855            f"nombre de cualquiera (o «llévame a...») navega directo a ella. "
856            f"Pestaña Aprobaciones: {len(d['aprobaciones'])} pendientes "
857            f"{(nombres_aprob)} - «aprueba/rechaza <nombre>» resuelve una. "
858            f"Pestaña Pedidos pendientes: {len(d['pedidos'])} pendientes "
859            f"{(nombres_pedidos)} - «aprueba/rechaza el pedido de <persona o plato>» "
860            f"resuelve uno. Pestaña Bandeja de alertas: {ra['abiertas']} abiertas, "
861            f"{ra['resueltas_hoy']} resueltas hoy, tiempo medio "
862            f"{ra['tiempo_medio_min']} if ra['tiempo_medio_min'] is not None else '-' "
863            f"minutos - «resuelve la alerta de <artículo o bodega>» resuelve una. "
864            f"Pestaña Recuento ciego y cierre: «audita <bodega>» o «abre <bodega>» "
865            f"selecciona esa bodega lista para recuento o cierre, igual que darle clic "
866            f"a «Abrir» en su tarjeta - desde ahí, dictar los productos SÍ es por voz "
867            f"con el micrófono normal de esa pantalla (igual que en Conteo). Pero "
868            f"«Iniciar recuento ciego», «Ver comparación», «Aceptar todas», «Cerrar con "
869            f"doble firma», firmar con PIN, y «Cerrar bodega definitivamente» "
870            f"son A PROPÓSITO solo manuales - nadie cierra una bodega con doble firma "
871            f"por accidente con la voz. Si el mensaje llegó hasta usted es porque "
872            f"«aprueba/rechaza <nombre>», «aprueba/rechaza el pedido de <...>», "
873            f"«resuelve la alerta de <...>» o «audita <bodega>» NO encontraron ese "
874            f"nombre exacto entre lo recién listado - esas acciones YA se resuelven "
875            f"antes de llegar aquí cuando el nombre coincide, así que usted NUNCA debe "
876            f"decir que ya aprobó, rechazó, resolvió o abrió algo (sería falso: no "
877            f"tiene esa capacidad desde aquí). Dígalo con honestidad y sugiera repetirlo "
878            f"con el nombre completo tal como aparece en la pantalla.")
879     if vista == "ayuda":
880         faq = "; ".join(f"{p} -> {r}" for p, r in _FAQ_ASISTENTE)
881         return f"Pantalla: Ayuda. Preguntas frecuentes conocidas: {faq}."
882     if vista == "perfil":
883         d = _datos_perfil_narrados(u)
884         return (
885             f"Pantalla: Mi perfil, de {u.nombre} ({u.perfil}). Datos personales, seguridad "
886             f"de la cuenta y preferencias de voz. "
887             f"Último acceso: {d['ultimo_acceso_hablado']} or 'sin registro'. PIN vence en "
888             f"{d['pin_vence_en_dias']} días. Bodegas asignadas: {d['n_bodegas']}"
889             f"+ (f" ({d['texto_bodegas']})" if d["texto_bodegas"] else "") + ". "
890             f"Voz actual: {d['voz_nombre']} ({d['voz_etiqueta'].lower()}), velocidad "
891             f"{d['velocidad_voz']}, confirmación hablada "
892             f"{('activada' if d['confirmacion_hablada'] else 'desactivada')}. "
893             f"«SÍ se puede cambiar por voz: la voz neuronal («cambia mi voz a puck/kore/aoede/"
894             f"charon»), la velocidad («habla más lento/rápido», «velocidad normal») y la "
895             f"confirmación hablada («activa/desactiva la confirmación hablada»). Cambiar el "
896             f"PIN, subir una foto, y editar nombre/correo/teléfono son A PROPÓSITO solo "
897             f"manuales, por seguridad - nunca diga que ya cambió el PIN por voz, eso sería "
898             f"falso.")
899     if vista == "bodegas":
900         with Sesion() as s:
901             ids_permitidos = _ids_permitidos_para_buscar(s, u)
902             q = s.query(Bodega)
903             if ids_permitidos is not None:
904                 q = q.filter(Bodega.id.in_(ids_permitidos))
905             bodegas = q.all()
906             n_pendientes = sum(1 for b in bodegas if b.estado == "pendiente")
907             n_en_conteo = sum(1 for b in bodegas if b.estado == "en_conteo")
908         return (f"Pantalla: Bodegas. Aquí se busca el stock de un artículo en todo "
909                f"el catálogo, y se ve el estado/historial de cada bodega (no se "
910                f"cuenta desde aquí, eso es en Conteo). De las bodegas de esta "
911                f"persona: {n_pendientes} pendientes, {n_en_conteo} en conteo. Si "
912                f"piden abrir o seguir contando una bodega por nombre, se resuelve "
913                f"aparte antes de este mensaje - esto solo se usa para explicar o "
914                f"navegar a otra pantalla.")
915     return f"Pantalla: {vista}."
916
917
918 _VERBOS_ABRIR_BODEGA = re.compile(
919     r"\b(abr[ae]|abrir|contar|cuenta|siga|sigue|contin[ae]|continuar)\b",
920     re.IGNORECASE)
921
922 _MODOSIN_CONEXION = re.compile(r"sin conexi[oó]n", re.IGNORECASE)

```

```

923 _ACTIVAR = re.compile(
924     r"\b(act[ií]v\w*|enc[ie][eé]nd\w*|prend\w*)\b", re.IGNORECASE)
925 _DESACTIVAR = re.compile(
926     r"\b(desact[ií]v\w*|ap[aá]g\w*|qu[ií]t\w*)\b", re.IGNORECASE)
927
928
929 def _cambiar_modosinconexion(texto: str, u: Usuario) -> dict | None:
930     """Único ajuste que de verdad se puede cambiar por voz en esta
931     pantalla (el resto - umbral, reglas fijas - sigue exigiendo los
932     controles de la pantalla): solo un administrador, y solo si la
933     orden nombra "sin conexión" junto con un verbo de activar/desactivar
934     inequívoco. None si no aplica, para caer al agente general."""
935     if u.perfil != "auditor" or not _MODOSINCONEXION.search(texto):
936         return None
937     if _ACTIVAR.search(texto):
938         nuevo = True
939     elif _DESACTIVAR.search(texto):
940         nuevo = False
941     else:
942         return None
943     with Sesión() as s:
944         existente = s.get(ConfigClave, "offline")
945         valor = "1" if nuevo else "0"
946         if existente:
947             existente.valor = valor
948         else:
949             s.add(ConfigClave(clave="offline", valor=valor))
950         s.commit()
951     estado = "activado" if nuevo else "desactivado"
952     registrar(u, "AJUSTE", f"{u.nombre} {estado} el modo sin conexión por voz", "ok")
953     return {"respuesta_hablada": f"Listo, el modo sin conexión quedó {estado}.",
954           "accion": "actualizar", "destino": None, "pestana": None}
955
956
957 def _normalizar_nombre(t: str) -> str:
958     """Sin quitar la puntuación, una pausa natural al hablar ("Cebolla,
959     cabezona blanca.") deja una coma que rompe la comparación por
960     substring contra "CEBOLLA CABEZONA BLANCA" (sin coma) - confirmado
961     con una captura real donde por eso no lograba resolver una
962     ambigüedad pendiente aunque dijo el nombre correcto. Mismo arreglo
963     que normalizar() ya tiene en servicios/conciliacion.py, por la
964     misma razón (el reconocimiento de voz también suele cerrar la
965     frase con un punto que nadie dijo)."""
966     t = str(t).lower().strip()
967     t = re.sub(r"[.,;:;!¿?¡]+", " ", t)
968     t = re.sub(r"\s+", " ", t).strip()
969     return "".join(c for c in unicodedata.normalize("NFD", t)
970                  if unicodedata.category(c) != "Mn")
971
972
973 _COLA_RELLENO = re.compile(
974     r"\s*,?\s*(?:por\s+favor|porfa\w*|gracias|si\s+puedes|si\s+es\s+posible)"
975     r"\s*[.,!¿]*\s*$", re.IGNORECASE)
976
977
978 def _quitar_relleno(texto: str) -> str:
979     """Los comandos de una sola frase («crea un usuario llamado...»,
980     «crea una receta llamada..., con... de...») quedan anclados al
981     FINAL del texto para extraer nombre/cantidad/ingrediente con
982     precisión - pero cualquier cortesía dicha al final ("...por favor",
983     "...gracias", cosa que la gente realmente dice) caía fuera de ese
984     ancla y el campo capturado se tragaba esas palabras de más (un
985     usuario terminaba llamándose "juan perfil auxiliar por favor" en
986     vez de "juan"). Se quita ANTES de intentar cualquier comando, en un
987     bucle por si hay más de una coletilla seguida ("por favor, gracias")."""
988     anterior = None
989     t = texto
990     while anterior != t:
991         anterior = t
992         t = _COLA_RELLENO.sub("", t)
993     return t
994
995
996 def _cambiar_estado_usuario(texto: str, u: Usuario) -> dict | None:
997     """Activa/desactiva a <nombre> desde Gestión de usuarios - mismo
998     patrón determinístico que el modo sin conexión (sin pasar por
999     Gemini, así no depende de su cuota ni arriesga un cambio real por
1000     una mala interpretación del modelo). Reusa las mismas reglas que ya
1001     protegen el PUT /api/usuarios/{id}: nadie se desactiva a sí mismo
1002     por voz, y solo un administrador puede tocar a otra persona.
1003     "Quita" es sinónimo de desactivar aquí, pero "quítale la BODEGA X a
1004     Luis" no debe desactivar a Luis por accidente solo porque comparte
1005     el verbo - de ahí la exclusión explícita cuando se menciona una
1006     bodega, que es harina de otro costal (_asignar_bodega_por_voz)."""
1007     if (u.perfil != "auditor" or _MODOSINCONEXION.search(texto)
1008         or re.search(r"\bbodega\b", texto, re.IGNORECASE)):

```

```

1009         return None
1010     if _ACTIVAR.search(texto):
1011         nuevo = True
1012     elif _DESACTIVAR.search(texto):
1013         nuevo = False
1014     else:
1015         return None
1016     t = _normalizar_nombre(texto)
1017     with Sesion() as s:
1018         candidatos = s.query(Usuario).filter(Usuario.id != u.id).all()
1019         encontrado = next(
1020             (c for c in candidatos
1021              if re.search(rf"\b{re.escape(_normalizar_nombre(c.nombre))}\b", t)),
1022             None)
1023         if not encontrado:
1024             return None
1025         if bool(encontrado.activo) == nuevo:
1026             estado = "activo" if nuevo else "inactivo"
1027             return {"respuesta_hablada": f"{encontrado.nombre.capitalize()} ya estaba {estado}.",
1028                     "accion": "actualizar", "destino": None, "pestaña": None}
1029         encontrado.activo = int(nuevo)
1030         nombre = encontrado.nombre
1031         s.commit()
1032     if not nuevo:
1033         try:
1034             _cliente_cognito().admin_user_global_sign_out(
1035                 UserPoolId=COGNITO_USER_POOL_ID, Username=nombre)
1036         except Exception as e:
1037             print(f"[cognito] no se pudo cerrar la sesion de {nombre}: {e}")
1038     estado = "activado" if nuevo else "desactivado"
1039     registrar(u, "USUARIO", f"{nombre} {estado} por voz", "ok")
1040     return {"respuesta_hablada": f"Listo, {nombre.capitalize()} quedó {estado}.",
1041            "accion": "actualizar", "destino": None, "pestaña": None}
1042
1043
1044 # Cuando alguien encadena dos órdenes en una sola frase ("...con 2 kg
1045 # de papa, agrega 300 g de cebolla a la receta sopa" - confirmado con
1046 # una captura real), el último grupo capturado (.+? anclado al final
1047 # con $) se tragaba TODO lo que seguía, incluida la segunda orden
1048 # completa, como si fuera parte del dato ("no encontré «papa, agrega
1049 # 300 g de cebolla a la receta sopa» en el catálogo"). Este sufijo se
1050 # detiene apenas aparece una coma seguida de otro verbo reconocido, y
1051 # descarta el resto - la primera orden se cumple bien, y la segunda
1052 # queda pendiente de decirse aparte.
1053 _CORTE_ORDEN_ENCADENADA = (
1054     r"(?:,\s*(?:cre[ae]|agr[eé]ga|a[ñn]ad|elimin|borr|cambia|as[ií]gn|qu[ií]t|"
1055     r"muestra|ense[ñn]a|oculta|act[ii]v|desact[ii]v)\w*.*)?")
1056 _CREAR_USUARIO = re.compile(
1057     r"\b(?:cre[ae]\w*\s+)?(?:un\s+|una\s+)?(?:nuev[oa]\s+)?usuario\s+llamad[oa]\s+(?P<nombre>.+)"
1058     r"(?:\s+(?:de\s+|con\s+)?perfil\s+(?P<perfil>auxiliar|auditor|administrador))?"
1059     r"\s*" + _CORTE_ORDEN_ENCADENADA + r"[!]?s*$", re.IGNORECASE)
1060 _INTENCION_CREAR_USUARIO = re.compile(
1061     r"\b(cre[ae]\w*|necesito|quiero)\b.*\busuarios?\b|busuario\s+llamad[oa]\b",
1062     re.IGNORECASE)
1063
1064
1065 def _crear_usuario_por_voz(texto: str, u: Usuario) -> dict | None:
1066     """Crea un usuario llamado <nombre> perfil <auxiliar/auditor> -
1067     determinístico, sin pasar por Gemini. A diferencia de
1068     activar/desactivar (que busca un nombre YA existente), aquí no hay
1069     nada contra qué validar el nombre, así que se exige un formato de
1070     frase concreto en vez de adivinar de cualquier forma - más fácil de
1071     aprender a decir que arriesgar un nombre mal extraído. Sin perfil
1072     dicho, queda auxiliar (el caso más común y el de menor alcance).
1073
1074     Decir solo "crea un usuario" (sin nombre) coincidía por casualidad
1075     con la navegación a la pestaña Gestión de usuarios (por la palabra
1076     "usuario") - si ya estaba en esa pestaña, no pasaba nada visible y
1077     parecía que el agente no hacía nada. Ahora, si se nota la intención
1078     de crear pero falta el nombre, se explica el formato exacto en vez
1079     de caer en la navegación por casualidad."""
1080     if u.perfil != "auditor":
1081         return None
1082     m = _CREAR_USUARIO.search(texto.strip())
1083     if not m:
1084         if (_INTENCION_CREAR_USUARIO.search(texto)
1085             and not _ES_PREGUNTA_PESTANA.search(texto)):
1086             return {"respuesta_hablada": "Para crear un usuario diga el nombre completo así: "
1087                     "«crea un usuario llamado» y el nombre, «perfil» y "
1088                     "«auxiliar o administrador.",
1089                     "accion": None, "destino": None, "pestaña": None}
1090         return None
1091     nombre = re.sub(r"[.,;:!?]+$", "", m.group("nombre")).strip().lower()
1092     if not nombre:
1093         return None
1094     perfil_dicho = (m.group("perfil") or "").lower()

```

```

1095 perfil = "auditor" if perfil_dicho in ("auditor", "administrador") else "auxiliar"
1096 with Sesion() as s:
1097     if s.query(Usuario).filter_by(nombre=nombre).first():
1098         return {"respuesta_hablada": f"Ya existe un usuario llamado {nombre.capitalize()}.",
1099                 "accion": None, "destino": None, "pestana": None}
1100     pin_generado = secrets.token_urlsafe(6) + "1Aa" # cumple la politica de Cognito
1101     # Cognito exige un correo real para crear la cuenta (no lo pide la
1102     # frase por voz) - se usa un correo institucional predecible; el
1103     # administrador lo puede corregir despues desde Ajustes si no es
1104     # el correcto. La cuenta en si SI queda usable de una: el PIN
1105     # dictado aqui es real, no un marcador de posicion.
1106     correo = f"{nombre.replace(' ', '.')}@colsubsidio.com"
1107     nuevo = Usuario(nombre=nombre, perfil=perfil, correo=correo)
1108     s.add(nuevo)
1109     s.commit()
1110     s.refresh(nuevo)
1111     nuevo.codigo = f"CS-{48000 + nuevo.id}"
1112     s.commit()
1113 if not _crear_usuario_cognito(nombre, correo, pin_generado):
1114     # la fila local ya quedo creada (igual que en POST /api/usuarios) -
1115     # se avisa por voz en vez de anunciar un PIN que en realidad no
1116     # sirve para entrar.
1117     registrar(u, "USUARIO", f"Usuario {nombre} creado ({perfil}) por voz, "
1118                  f"pero fallo en Cognito", "alerta")
1119     return {"respuesta_hablada": f"Creé a {nombre.capitalize()} en el sistema, pero no "
1120                                f"pude generar su acceso. Créelo de nuevo desde Gestión "
1121                                f"de usuarios.",
1122            "accion": "actualizar", "destino": None, "pestana": None}
1123 registrar(u, "USUARIO", f"Usuario {nombre} creado ({perfil}) por voz", "ok")
1124 return {"respuesta_hablada": f"Listo, creé a {nombre.capitalize()} como {perfil}. "
1125                             f"Su PIN temporal es {pin_generado} - dígaselo para que "
1126                             f"lo cambie en Mi perfil.",
1127        "accion": "actualizar", "destino": None, "pestana": None}
1128
1129
1130 _VERBOS_GENERAR_REPORTE = re.compile(
1131     r"\b(gener[ae]|generar|exporta|exportar|crea|crear|descarga|descargar|saca|sacar)\b",
1132     re.IGNORECASE)
1133 _REP_DIFERENCIAS = re.compile(r"diferencia", re.IGNORECASE)
1134 _REP_ANALISIS = re.compile(r"an[al]lisis|consumo", re.IGNORECASE)
1135 _REP_CONSOLIDADO = re.compile(r"consolidado", re.IGNORECASE)
1136 _REP_TABLERO = re.compile(r"tablero", re.IGNORECASE)
1137 _REP_DETALLE_BODEGA = re.compile(r"detalle\s+d?el?\s+(la\s+)?bodega|detalle\s+de\s+almac[eén]", re.IGNORECASE)
1138 _REP_TRAZA_ARCHIVO = re.compile(r"trazabilidad", re.IGNORECASE)
1139
1140
1141 def _generar_reporte_por_voz(texto: str, u: Usuario) -> dict | None:
1142     """Genera el consolidado" / "exporta las diferencias" / "exporta el
1143     análisis de consumo" desde Reportes - mismo patrón determinístico
1144     que Ajustes (sin pasar por Gemini): llama la MISMA función que ya
1145     usa el botón correspondiente, así el resultado es idéntico al de
1146     darle clic, solo que solo un administrador puede pedirlo por voz."""
1147     if u.perfil != "auditor" or not _VERBOS_GENERAR_REPORTE.search(texto):
1148         return None
1149     if _REP_DIFERENCIAS.search(texto):
1150         d = reporte_diferencias(formato="xlsx", u=u)
1151         # archivo/titulo_archivo (ademas de "actualizar") - para que el
1152         # panel de vista previa muestre este archivo de una, igual que ya
1153         # pasa al darle clic al boton "Generar diferencias" (que llama a
1154         # previsualizar() apenas termina). Sin esto, pedirlo por voz lo
1155         # generaba pero la vista previa se quedaba vacia hasta un clic
1156         # manual en la tarjeta.
1157         return {"respuesta_hablada": f"Listo, generé las diferencias: {d['filas']} filas en "
1158                                    f"f{d['bodegas_con_descuadre']} bodegas con descuadre.",
1159                "accion": "actualizar", "destino": None, "pestana": "consolidado",
1160                "archivo": d["archivo"], "titulo_archivo": "Diferencias por bodega"}
1161     if _REP_ANALISIS.search(texto):
1162         # a diferencia del consolidado/diferencias (que solo generan y
1163         # dejan una vista previa - descargar es un clic aparte, igual por
1164         # voz que a mano), el botón "Exportar análisis" SÍ descarga de
1165         # una vez: sin "descargar_reporte" aquí, pedirlo por voz creaba
1166         # el archivo en el servidor pero nunca llegaba al dispositivo -
1167         # la persona no notaba nada distinto de antes de pedirlo.
1168         d = exportar_analisis(formato="xlsx", dias=30, u=u)
1169         return {"respuesta_hablada": "Listo, exporté el análisis de consumo de los últimos 30 días.",
1170                "accion": "descargar_reporte", "destino": None, "pestana": "analisis",
1171                "archivo": d["archivo"]}
1172     if _REP_CONSOLIDADO.search(texto):
1173         d = reporte(formato="xlsx", u=u)
1174         return {"respuesta_hablada": f"Listo, generé el consolidado: {d['filas']} filas.",
1175                "accion": "actualizar", "destino": None, "pestana": "consolidado",
1176                "archivo": d["archivo"], "titulo_archivo": "Consolidado para My Inventory"}
1177     return None
1178
1179
1180 def _datos_reportes_narrados(u: Usuario) -> dict:

```

```

1181 """Los archivos recientes (misma fuente que la lista de Reportes) y el
1182 análisis de consumo de los últimos 30 días, ya en frases listas para
1183 hablar - los usan tanto el contexto completo que recibe Gemini como
1184 las respuestas determinísticas de _responder_reportes_por_voz."""
1185 recientes = reportes_recientes(u=u)
1186 analisis = api_analisis(dias=30, u=u)
1187
1188 def _fila(a):
1189     extra = f", {a['filas']} filas" if a["filas"] is not None else ""
1190     return f"{a['titulo']} ({a['formato']}, hoy {a['hora']}{extra})"
1191
1192 texto_recientes = ("todavía no se ha generado ningún archivo" if not recientes else
1193                    "; ".join(_fila(x) for x in recientes[:5]))
1194 ultimo_consolidado = next((x for x in recientes if x["titulo"] == "Consolidado para My Inventory"), None)
1195 ultimas_diferencias = next((x for x in recientes if x["titulo"] == "Diferencias por bodega"), None)
1196 subtil = analisis["subutilizados"]
1197 texto_subtil = ("sin insumos subutilizados en el período" if not subtil else
1198                "; ".join(f"{s['nombre'].title(): sobran {s['sobra']} kg "
1199                           f"({s['sobrepedido_pct']}% de sobrepedido, en {s['veces']} servicios)"
1200                           for s in subtil[:5]))
1201
1202 return {"recientes": recientes, "analisis": analisis, "texto_recientes": texto_recientes,
1203        "ultimo_consolidado": ultimo_consolidado, "ultimas_diferencias": ultimas_diferencias,
1204        "texto_subtil": texto_subtil}
1205
1206 _REPORTES_PREGUNTAS = [
1207     (re.compile(r"bcu[aá]ntw*s+filas\b.*b[uú]ltimo\s+consolidado\b|"
1208                r"bfilas\s+tiene\s+el\s+consolidado\b", re.IGNORECASE),
1209      lambda d: (f"El último consolidado tiene {d['ultimo_consolidado']['filas']} filas, "
1210                 f"generado hoy {d['ultimo_consolidado']['hora']}."
1211                 if d["ultimo_consolidado"] else "Todavía no se ha generado ningún consolidado.")),
1212     (re.compile(r"bcu[aá]ntw*s+(filas|bodegas)\b.*bdiferencias\b|"
1213                r"bbodegas\s+tienen\s+descuadre\b", re.IGNORECASE),
1214      lambda d: (f"Las últimas diferencias exportadas tienen {d['ultimas_diferencias']['filas']} "
1215                 f"filas, generadas hoy {d['ultimas_diferencias']['hora']}."
1216                 if d["ultimas_diferencias"] else "Todavía no se han exportado diferencias por bodega.")),
1217     (re.compile(r"bqu[eé]\s+archivos\b.*bgeneradw*\barchivos\s+(recientes|generados)\b",
1218                re.IGNORECASE),
1219      lambda d: f"Archivos recientes: {d['texto_recientes']}."),
1220     (re.compile(r"bcu[aá]nto\b.*bpidi[oó]\b\bpedido\s+total\b\bpedido\s+en\s+el\s+per[ii]odo\b",
1221                re.IGNORECASE),
1222      lambda d: f"En el período se pidieron {d['analisis']['pedido_total']} kilogramos, en "
1223               f"{d['analisis']['servicios_periodo']} servicios."),
1224     (re.compile(r"bcu[aá]nto\b.*bus[oó]\b\baprovechamiento\b\busado\s+realmente\b",
1225                re.IGNORECASE),
1226      lambda d: f"Se usaron realmente {d['analisis']['usado_total']} kilogramos, el "
1227               f"{d['analisis']['aprovechamiento']}% de lo pedido."),
1228     (re.compile(r"bahorro\s+potencial\b", re.IGNORECASE),
1229      lambda d: f"El ahorro potencial es de {d['analisis']['ahorro_potencial']} kilogramos al "
1230               "mes si se ajustan las recetas."),
1231     (re.compile(r"bcu[aá]ntw*s+insumos\s+(est[aá]n\s+)?subutilizadw*\b|binsumos\s+subutilizados\b",
1232                re.IGNORECASE),
1233      lambda d: f"Hay {len(d['analisis']['subutilizados'])} insumos subutilizados: "
1234               f"{d['texto_subtil']}."),
1235     (re.compile(r"b(m[aá]s|mayor)\s+sobrepedido\b\brecomendaci[oó]n\s+(del\s+)?agente\b",
1236                re.IGNORECASE),
1237      lambda d: (f"El insumo con más sobrepedido es "
1238                 f"{d['analisis']['subutilizados'][0]['nombre'].title(): sobra en "
1239                 f"{d['analisis']['subutilizados'][0]['veces']} servicios, "
1240                 f"{d['analisis']['subutilizados'][0]['sobrepedido_pct']}% más de lo que se usa."
1241                 if d["analisis"]["subutilizados"] else
1242                 "No hay insumos subutilizados en el período.")),
1243 ]
1244
1245
1246 def _responder_reportes_por_voz(texto: str, u: Usuario) -> dict | None:
1247     """Preguntas frecuentes de Reportes (ambas pestañas), resueltas sin
1248     pasar por Gemini - mismo motivo que _responder_panel_por_voz: estas
1249     frases ya se ofrecen como garantizadas en el desplegable de ejemplos."""
1250     if u.perfil != "auditor":
1251         return None
1252     for patron, respuesta in _REPORTES_PREGUNTAS:
1253         if patron.search(texto):
1254             return {"respuesta_hablada": respuesta(_datos_reportes_narrados(u)),
1255                    "accion": None, "destino": None, "pestaña": None}
1256     return None
1257
1258
1259 _VER_ARCHIVO_VERBO = re.compile(
1260     r"b(mu[eé]stra|ense[ñ]a|w*|[aá]bre|w*)\b", re.IGNORECASE)
1261 _VER_ARCHIVO_PALABRA = re.compile(r"barchivo\b", re.IGNORECASE)
1262
1263
1264 def _previsualizar_reporte_por_voz(texto: str, u: Usuario) -> dict | None:
1265     """Muéstrame el detalle de bodega" / "ábreme el archivo de diferencias" -
1266     lo mismo que darle clic a una tarjeta de "Archivos generados", sin

```

```

1267     tocar la pantalla.
1268
1269     Diferencias, tablero, detalle de bodega y trazabilidad NO son el
1270     nombre de ninguna pestaña de esta pantalla, así que decir su nombre
1271     con un verbo de "mostrar" no compite con nada más - se abren directo,
1272     sin exigir la palabra "archivo". Consolidado y análisis Sí son
1273     pestañas reales («muéstrame el consolidado» ya significa cambiar de
1274     pestaña, ver _navegar_pestana_por_voz), así que para esos dos la
1275     palabra "archivo" sigue siendo obligatoria para desambiguar - sin
1276     ella, decir solo el nombre de la pestaña dejaría de poder cambiarla.
1277     Antes la palabra "archivo" era obligatoria para los seis tipos por
1278     igual, así que pedir "muéstrame el detalle de bodega" (la frase más
1279     natural, sin decir "archivo") no abría nada y caía al agente
1280     general, que solo podía describir el archivo en palabras y mandar a
1281     dar clic a mano - no a abrirlo de verdad.
1282
1283     Cubre los cinco tipos que puede mostrar la lista (antes solo distinguía
1284     diferencias y consolidado; el resto - tablero, detalle de bodega,
1285     análisis, trazabilidad - siempre caía al último archivo generado, que
1286     con frecuencia NO era el que se pedía)."""
1287     if u.perfil != "auditor" or not _VER_ARCHIVO_VERBO.search(texto):
1288         return None
1289     recientes = reportes_recientes(u=u)
1290     if not recientes:
1291         return {"respuesta_hablada": "Todavía no se ha generado ningún archivo.",
1292                "accion": None, "destino": None, "pestana": "consolidado"}
1293     tiene_archivo = bool(_VER_ARCHIVO_PALABRA.search(texto))
1294     if _REP_DIFERENCIAS.search(texto):
1295         elegido = next((x for x in recientes if x["titulo"] == "Diferencias por bodega"), None)
1296     elif _REP_TABLERO.search(texto):
1297         elegido = next((x for x in recientes if x["titulo"] == "Estado del tablero"), None)
1298     elif _REP_DETALLE_BODEGA.search(texto):
1299         elegido = next((x for x in recientes if x["titulo"] == "Detalle de bodega"), None)
1300     elif _REP_TRAZA_ARCHIVO.search(texto):
1301         elegido = next((x for x in recientes if x["titulo"] == "Registro de trazabilidad"), None)
1302     elif _REP_ANALISIS.search(texto) and tiene_archivo:
1303         elegido = next((x for x in recientes if x["titulo"] == "Análisis de consumo"), None)
1304     elif _REP_CONSOLIDADO.search(texto) and tiene_archivo:
1305         elegido = next((x for x in recientes if x["titulo"] == "Consolidado para My Inventory"), None)
1306     elif tiene_archivo:
1307         elegido = recientes[0]
1308     else:
1309         return None
1310     if not elegido:
1311         return {"respuesta_hablada": "No encontré ese archivo entre los generados recientemente.",
1312                "accion": None, "destino": None, "pestana": "consolidado"}
1313     detalle_filas = f", {elegido['filas']} filas" if elegido["filas"] is not None else ""
1314     return {"respuesta_hablada": f"Aquí está: {elegido['titulo']}{detalle_filas}. "
1315            "¿Desea descargarlo?",
1316            "accion": "previsualizar_reporte", "destino": None, "pestana": "consolidado",
1317            "archivo": elegido["archivo"], "titulo_archivo": elegido["titulo"]}
1318
1319
1320 # Palabras que identifican cada pestaña, para navegar entre ellas sin
1321 # pasar por Gemini - la cuota de Gemini ha fallado justo en este tipo de
1322 # orden ("llévame a gestión de usuarios"), dejando la pestaña sin
1323 # cambiar aunque la respuesta hablada dijera lo contrario.
1324 _PALABRAS_PESTANA = {
1325     "ajustes": {
1326         "config": re.compile(r"configuraci[ó]n", re.IGNORECASE),
1327         "usuarios": re.compile(r"usuarios?", re.IGNORECASE),
1328         "recetas": re.compile(r"recetas?", re.IGNORECASE),
1329         "traza": re.compile(r"trazabilidad", re.IGNORECASE),
1330     },
1331     "reportes": {
1332         "consolidado": re.compile(r"consolidado", re.IGNORECASE),
1333         "análisis": re.compile(r"an[á]lisis|consumo", re.IGNORECASE),
1334     },
1335     "panel": {
1336         "resumen": re.compile(r"resumen", re.IGNORECASE),
1337         "alertas": re.compile(r"\balertas\b", re.IGNORECASE),
1338     },
1339     "auditoria": {
1340         "recuento": re.compile(r"recuento|recont[ae]o|\bcierre\b", re.IGNORECASE),
1341         "aprobaciones": re.compile(r"aprobaciones", re.IGNORECASE),
1342         "pedidos": re.compile(r"pedidos", re.IGNORECASE),
1343         "alertas": re.compile(r"\balertas\b", re.IGNORECASE),
1344     },
1345 }
1346 _VERBOS_IR_PESTANA = re.compile(
1347     r"\b(ir|ll[eé]va(me)?|vamos|ve|mu[eé]stra(me)?|[aá]bre(me)?|ens[eé][ñn]a(me)?)\b",
1348     re.IGNORECASE)
1349 _ES_PREGUNTA_PESTANA = re.compile(r"[?]\bqu[ae]nta\b|qu[ae]nta\b|qu[ae]nta\b|qu[ae]nta\b", re.IGNORECASE)
1350
1351
1352 def _navegar_pestana_por_voz(vista: str, texto: str) -> dict | None:

```

```

1353     """Llévame a gestión de usuarios" (o cualquier pestaña de Ajustes,
1354     Reportes o Panel) - determinístico, sin pasar por Gemini, para que
1355     la navegación entre pestañas no dependa de su cuota. También navega
1356     si lo dicho es prácticamente el nombre de la pestaña solo ("Gestión
1357     de usuarios.", sin decir "Llévame a" - la persona lo dice tal como
1358     lo lee en la pantalla), siempre que sea corto y no suene a pregunta
1359     ("¿cuántos usuarios hay?" no debe navegar, solo responderse)."""
1360     palabras = _PALABRAS_PESTANA.get(vista)
1361     if not palabras:
1362         return None
1363     tiene_verbo = bool(_VERBOS_IR_PESTANA.search(texto))
1364     limpio = re.sub(r"[.,;:!?;]+", "", texto.strip())
1365     es_solo_el_nombre = (not _ES_PREGUNTA_PESTANA.search(texto)
1366                         and 0 < len(limpio.split()) <= 4)
1367     if not tiene_verbo and not es_solo_el_nombre:
1368         return None
1369     for clave, patron in palabras.items():
1370         if patron.search(texto):
1371             etiqueta = _PESTANAS_ASISTENTE[vista][clave]
1372             return {"respuesta_hablada": f"Listo, vamos a {etiqueta}.",
1373                     "accion": "navegar", "destino": vista, "pestaña": clave}
1374     return None
1375
1376
1377 # Los 4 "accesos rápidos" de Inicio ("Iniciar un conteo", "Ver el tablero",
1378 # "Continuar auditoría"/"Hacer un pedido", "Generar reporte"/"Legalizar
1379 # servicio") - las mismas tarjetas que ya se navegan con un clic.
1380 _ACCESOS_INICIO = {
1381     "conteo": re.compile(r"\bconteo\b", re.IGNORECASE),
1382     "bodegas": re.compile(r"\btablero\b|\bbodegas\b", re.IGNORECASE),
1383     "auditoria": re.compile(r"auditor[i]a", re.IGNORECASE),
1384     "pedido": re.compile(r"\bpedidos?\b", re.IGNORECASE),
1385     "reportes": re.compile(r"reporte", re.IGNORECASE),
1386     "legalizacion": re.compile(r"legaliza", re.IGNORECASE),
1387 }
1388 _VERBOS_ACCESO_INICIO = re.compile(
1389     r"\b(inicia|iniciar|continu[ae]|continuar|genera|generar|legaliza|legalizar|hacer|haga)\b",
1390     re.IGNORECASE)
1391
1392
1393 def _acceso_rapido_inicio_por_voz(texto: str, u: Usuario) -> dict | None:
1394     """Decir el nombre de una tarjeta de "¿Qué desea hacer?" navega directo
1395     a ella - determinístico, sin pasar por Gemini: se confirmó con
1396     pruebas que a veces decidía contestar en vez de navegar aunque la
1397     frase mencionara el destino tal cual ("iniciar un conteo" se quedaba
1398     en Inicio). Mismo filtro que _navegar_pestaña_por_voz para no
1399     robarle la frase a las preguntas de KPI de esta pantalla («¿cuántas
1400     bodegas tengo?» debe contestarse, no navegar a Bodegas)."""
1401     tiene_verbo = bool(_VERBOS_IR_PESTANA.search(texto)) or bool(_VERBOS_ACCESO_INICIO.search(texto))
1402     limpio = re.sub(r"[.,;:!?;]+", "", texto.strip())
1403     es_solo_el_nombre = (not _ES_PREGUNTA_PESTANA.search(texto)
1404                         and 0 < len(limpio.split()) <= 4)
1405     if not tiene_verbo and not es_solo_el_nombre:
1406         return None
1407     for destino, patron in _ACCESOS_INICIO.items():
1408         if not patron.search(texto):
1409             continue
1410         if destino in _SOLO_AUDITOR_ASISTENTE and u.perfil != "auditor":
1411             continue
1412         etiqueta = _DESTINOS_ASISTENTE[destino]
1413         return {"respuesta_hablada": f"Vamos a {etiqueta.lower()}.",
1414                 "accion": "navegar", "destino": destino, "pestaña": None}
1415     return None
1416
1417
1418 _APROBAR_ITEM = re.compile(r"\bapru[eé]b[w*]|\baprobar\b", re.IGNORECASE)
1419 _RECHAZAR_ITEM = re.compile(r"\brech[aá]z[w*]", re.IGNORECASE)
1420
1421
1422 def _resolver_aprobacion_por_voz(texto: str, u: Usuario) -> dict | None:
1423     """Aprueba/rechaza <nombre> desde Auditoría (pestaña Aprobaciones) -
1424     mismo patrón determinístico que Ajustes/Reportes: busca el nombre
1425     completo entre las aprobaciones pendientes (como frase exacta, no
1426     palabra por palabra - "aprueba costilla de res" no debe aprobar
1427     cualquier cosa que contenga "res") y llama la MISMA función que ya
1428     usa el botón, solo administrador."""
1429     if u.perfil != "auditor":
1430         return None
1431     if _APROBAR_ITEM.search(texto):
1432         verbo = "aprobar"
1433     elif _RECHAZAR_ITEM.search(texto):
1434         verbo = "rechazar"
1435     else:
1436         return None
1437     t = _normalizar_nombre(texto)
1438     with Sesion() as s:

```

```

1439     pendientes = s.query(Aprobacion).filter_by(estado="pendiente").all()
1440     encontrada = next(
1441         (a for a in pendientes
1442          if re.search(rf"\b{re.escape(_normalizar_nombre(a.nombre))}\b", t)),
1443         None)
1444     if not encontrada:
1445         return None
1446     aid, nombre = encontrada.id, encontrada.nombre
1447     if verbo == "aprobar":
1448         aprobar(aprobacion_id=aid, u=u)
1449         return {"respuesta_hablada": f"Listo, aprobé {nombre.title()}. Ya entra al catálogo oficial.",
1450               "accion": "actualizar", "destino": None, "pestana": None}
1451     rechazar(aprobacion_id=aid, u=u)
1452     return {"respuesta_hablada": f"Listo, rechacé {nombre.title()}.",
1453           "accion": "actualizar", "destino": None, "pestana": None}
1454
1455
1456 _APROBAR_RECHAZAR_PEDIDO = re.compile(
1457     r"\b(apru[eé]b\w*|rech[aá]z\w*)\b.*\bpedido\b\bpedido\b.*\b(apru[eé]b\w*|rech[aá]z\w*)\b",
1458     re.IGNORECASE)
1459
1460
1461 def _resolver_pedido_pendiente_por_voz(texto: str, u: Usuario) -> dict | None:
1462     """Apueba/rechaza el pedido de <persona o plato> desde Auditoría
1463     (pestaña Pedidos pendientes) - mismo patrón que _resolver_aprobacion_por_voz,
1464     pero exige la palabra "pedido" para no competir con aprobar/rechazar
1465     un producto o bodega nueva, que es otra lista con los mismos verbos."""
1466     if u.perfil != "auditor" or not _APROBAR_RECHAZAR_PEDIDO.search(texto):
1467         return None
1468     if _APROBAR_ITEM.search(texto):
1469         aprobar_bool = True
1470     elif _RECHAZAR_ITEM.search(texto):
1471         aprobar_bool = False
1472     else:
1473         return None
1474     t = _normalizar_nombre(texto)
1475     pendientes = pedidos_pendientes(u=u)
1476     if not pendientes:
1477         return {"respuesta_hablada": "No hay pedidos pendientes de aprobación.",
1478               "accion": None, "destino": None, "pestana": "pedidos"}
1479     encontrado = next(
1480         (p for p in pendientes
1481          if (p["persona"] and p["persona"] != "-"
1482              and re.search(rf"\b{re.escape(_normalizar_nombre(p['persona']))}\b", t))
1483           or re.search(rf"\b{re.escape(_normalizar_nombre(p['plato']))}\b", t)),
1484         None)
1485     if not encontrado:
1486         return None
1487     resolver_pedido(numero_pedido=encontrado["numero_pedido"],
1488                     datos=ResolverPedidoIn(aprobar=aprobar_bool), u=u)
1489     verbo_pasado = "aprobé" if aprobar_bool else "rechacé"
1490     return {"respuesta_hablada": f"Listo, {verbo_pasado} el pedido de "
1491                               f"{encontrado['persona'].title(): {encontrado['plato']}.",
1492           "accion": "actualizar", "destino": None, "pestana": "pedidos"}
1493
1494
1495 _RESOLVER_ALERTA = re.compile(r"\bresuelve\w*|\bresolver\b", re.IGNORECASE)
1496
1497
1498 def _resolver_alerta_por_voz(texto: str, u: Usuario) -> dict | None:
1499     """Resuelve la alerta de <artículo o bodega> desde Auditoría
1500     (pestaña Bandeja de alertas) - mismo patrón determinístico, busca
1501     entre las alertas abiertas por el artículo o la bodega que mencionan."""
1502     if u.perfil != "auditor" or not _RESOLVER_ALERTA.search(texto):
1503         return None
1504     abiertas = ver_alertas(resueltas=0, u=u)
1505     if not abiertas:
1506         return {"respuesta_hablada": "No hay alertas abiertas.",
1507               "accion": None, "destino": None, "pestana": "alertas"}
1508     t = _normalizar_nombre(texto)
1509     encontrada = next(
1510         (a for a in abiertas
1511          if (a["articulo"] and re.search(rf"\b{re.escape(_normalizar_nombre(a['articulo']))}\b", t))
1512           or (a["bodega"] and re.search(rf"\b{re.escape(_normalizar_nombre(a['bodega']))}\b", t))),
1513         None)
1514     if not encontrada:
1515         return None
1516     resolver_alerta(alerta_id=encontrada["id"], u=u)
1517     detalle = encontrada["articulo"] or encontrada["bodega"] or encontrada["titulo"]
1518     return {"respuesta_hablada": f"Listo, resolví la alerta de {detalle.title()}.",
1519           "accion": "actualizar", "destino": None, "pestana": "alertas"}
1520
1521
1522 _VERBOS_AUDITAR_BODEGA = re.compile(r"\b(abr[ae]|abrir|audit\w*)\b", re.IGNORECASE)
1523
1524

```

```

1525 def _abrir_bodega_para_auditoria_por_voz(texto: str, u: Usuario) -> dict | None:
1526     """Audita <bodega> / "abre <bodega>" desde Auditoría (pestaña
1527     Recuento ciego y cierre) - selecciona esa bodega, lo mismo que darle
1528     clic a "Abrir" en su tarjeta. A propósito NO llega hasta firmar ni
1529     cerrar: eso sigue siendo manual, es la garantía de un cierre con
1530     doble firma que nadie dispara sin querer con la voz."""
1531     if u.perfil != "auditor" or not _VERBOS_AUDITAR_BODEGA.search(texto):
1532         return None
1533     from servicios.conciliacion import buscar_bodega
1534     with Sesion() as s:
1535         candidatas_ids = {b.id for b in s.query(Bodega)
1536                           .filter(Bodega.estado.in_(["en_auditoria", "cerrada"]))}.all()}
1537         if not candidatas_ids:
1538             return {"respuesta_hablada": "No hay ninguna bodega lista para recuento ciego o "
1539                                     "cierre en este momento.",
1540                     "accion": None, "destino": None, "pestaña": "recuento"}
1541         bodega = buscar_bodega(s, texto, candidatas_ids)
1542         if not bodega:
1543             return None
1544         return {"respuesta_hablada": f"Vamos a auditar {bodega.nombre_oficial.title()}.",
1545                 "accion": "navegar", "destino": "auditoria", "pestaña": "recuento",
1546                 "bodega_auditar": bodega.nombre_oficial}
1547
1548
1549 def _datos_auditoria_narrados(u: Usuario) -> dict:
1550     return {"resumen_alertas": resumen_alertas(u=u), "pedidos": pedidos_pendientes(u=u),
1551            "aprobaciones": listar_aprobaciones(estado="pendiente", u=u)}
1552
1553
1554 _AUDITORIA_PREGUNTAS = [
1555     (re.compile(r"\bcu[aá]nt\w*s+alertas\b.*\babiertas\b|\balertas\s+abiertas\b", re.IGNORECASE),
1556      lambda d: f"Hay {d['resumen_alertas']['abiertas']} alertas abiertas."),
1557     (re.compile(r"\bcu[aá]nt\w*b.*\bresolvieron\s+hoy\b|\bresueltas\s+hoy\b", re.IGNORECASE),
1558      lambda d: f"Se resolvieron {d['resumen_alertas']['resueltas_hoy']} alertas hoy."),
1559     (re.compile(r"\btipo\s+medio\b", re.IGNORECASE),
1560      lambda d: {f"El tiempo medio de resolución es de "
1561                  f"{d['resumen_alertas']['tiempo_medio_min']} minutos."
1562                  if d["resumen_alertas"]["tiempo_medio_min"] is not None else
1563                  "Todavía no se ha resuelto ninguna alerta hoy para calcular el tiempo medio."}),
1564     (re.compile(r"\bcu[aá]nt\w*s+pedidos\b.*\bpendientes\b", re.IGNORECASE),
1565      lambda d: {f"Hay {len(d['pedidos'])} pedidos pendientes de aprobación."
1566                  if d["pedidos"] else "No hay pedidos pendientes de aprobación."}),
1567     (re.compile(r"\bcu[aá]nt\w*s+aprobaciones\b", re.IGNORECASE),
1568      lambda d: {f"Hay {len(d['aprobaciones'])} aprobaciones pendientes: " +
1569                  "; ".join(a["nombre"].title() for a in d["aprobaciones"][:6]) + "."
1570                  if d["aprobaciones"] else "No hay aprobaciones pendientes."}),
1571 ]
1572
1573
1574 def _responder_auditoria_por_voz(texto: str, u: Usuario) -> dict | None:
1575     if u.perfil != "auditor":
1576         return None
1577     for patron, respuesta in _AUDITORIA_PREGUNTAS:
1578         if patron.search(texto):
1579             return {"respuesta_hablada": respuesta(_datos_auditoria_narrados(u)),
1580                     "accion": None, "destino": None, "pestaña": None}
1581     return None
1582
1583
1584 _UNIDADES_RECETA = r"kilos?|kg|litros?|lt|unidades?|und?|gramos?|gr"
1585 # Cuando alguien encadena dos órdenes en una sola frase ("...con 2 kg
1586 # de papa, agrega 300 g de cebolla a la receta sopa" - confirmado con
1587 # una captura real), el último grupo capturado (.+? anclado al final
1588 # con $) se tragaba TODO lo que seguía, incluida la segunda orden
1589 # completa, como si fuera parte del dato ("no encontré «papa, agrega
1590 # 300 g de cebolla a la receta sopa» en el catálogo"). Este sufijo se
1591 # detiene apenas aparece una coma seguida de otro verbo reconocido, y
1592 # descarta el resto - la primera orden se cumple bien, y la segunda
1593 # queda pendiente de decirse aparte (tal como ya se explica: "puede
1594 # agregarle más ingredientes diciendo «agrega...").
1595 # El verbo "crea/crear" es OPCIONAL: el reconocimiento de voz a veces
1596 # recorta la primera palabra si la persona empieza a hablar apenas
1597 # hace clic en el micrófono, antes de que el navegador esté listo -
1598 # confirmado con una captura real ("Una receta llamada sopa,
1599 # rendimiento, cuatro porciones." sin "crea" al inicio). El resto de
1600 # la frase (receta llamada... rendimiento... con... de...) ya es lo
1601 # bastante específico para no confundirse con una pregunta suelta.
1602 _CREAR_RECETA = re.compile(
1603     r"\b(?:crea[ae]\w*(s+)?(?:una\s+)?(?:nuev[oa]\s+)?receta\s+llamad[ao]\s+(?<nombre>.+))"
1604     r"\s*[.,y]*\s*(?:con\s+)?rendimiento\s+(?:de\s+)?(?<rend>.+)?\s*porciones?"
1605     r"\s*[.,y]*\s*con\s+(?<cant>.+)?\s*"
1606     rf"({_UNIDADES_RECETA})?\s*de\s+(?<ing>.+)?\s*" + _CORTE_ORDEN_ENCADENADA + r"(!)?\s*$",
1607     re.IGNORECASE)
1608 _AGREGAR_INGREDIENTE = re.compile(
1609     r"\b(?:agr[eé]\w*[a[ñn]ad\w*)\s+(?<cant>.+)?\s*"
1610     rf"({_UNIDADES_RECETA})?\s*de\s+(?<ing>.+)"

```

```

1611     r"\s+a\s+la\s+receta\s+(?P<receta>.+?)\s*" + _CORTE_ORDEN_ENCadenada + r"[.!?]\s*$",
1612     re.IGNORECASE)
1613 _ELIMINAR_RECETA = re.compile(
1614     r"\b(elimin|w*|borr|w*)\s+(?:la\s+)?receta\s+(?P<receta>.+?)\s*"
1615     + _CORTE_ORDEN_ENCadenada + r"[.!?]\s*$",
1616     re.IGNORECASE)
1617
1618
1619 # Memoria viva de una sola pregunta pendiente por persona: "crea una
1620 # receta... con papa" -> ambiguo -> la persona contesta solo "papa
1621 # criolla" en el siguiente turno, y eso completa la orden original en
1622 # vez de tener que repetirla entera. Mismo patron que ESTADOS en
1623 # agente/orquestador.py (memoria en RAM, se pierde si el servidor se
1624 # reinicia - aceptable aqui: en el peor caso, toca repetir la orden
1625 # completa, que es lo que ya se pedia antes de esto). Expira sola
1626 # despues de _VENCIMIENTO_AMBIGUEDAD segundos para no quedar pegada a
1627 # una respuesta que en realidad era para otra cosa, dicha mucho despues.
1628 _PENDIENTE_AMBIGUEDAD: dict[int, dict] = {}
1629 _VENCIMIENTO_AMBIGUEDAD = 90
1630
1631
1632 def _elegir_ingrediente_sin_ambiguedad(dicho: str) -> dict | None:
1633     """buscar_articulo() puntúa por cobertura de palabras: una consulta
1634     de una sola palabra genérica ("papa") saca 100% de cobertura contra
1635     CUALQUIER articulo que la contenga, sin importar cuánto más tenga
1636     ese nombre alrededor - confirmado con una captura real donde "papa"
1637     empataba a 100 de confianza con "EMPANADA DE PAPA Y CARNE X50 GR",
1638     "PAPA A LA FRANCESA" y "PAPA CABELLO DE ANGEL" por igual, y se
1639     quedaba con el primero de la lista sin ningún criterio real. Igual
1640     que buscar_bodega ya hace con bodegas ambiguas: mejor decir "sea
1641     más específico" que adivinar mal en silencio y crear una receta con
1642     el ingrediente equivocado."""
1643     from servicios.conciliacion import buscar_articulo, normalizar
1644     # limite alto: buscar_articulo por defecto solo devuelve los 3
1645     # mejores, y con un empate a 100 (el caso ambiguo que nos interesa
1646     # aquí) esos 3 son un subconjunto ARBITRARIO del catálogo real -
1647     # confirmado con una captura real donde salían "Empanada De Papa Y
1648     # Carne" y "Papa A La Francesa" en vez de las papas de verdad
1649     # (Criolla, Sabanera, Pastusa) que sí existen en el catálogo. Con
1650     # más candidatos a la vista, se puede priorizar los nombres más
1651     # simples (probablemente el ingrediente crudo) al armar la lista.
1652     candidatos = buscar_articulo(dicho, limite=20)
1653     if not candidatos or candidatos[0]["confianza"] < 60:
1654         return {"error": f"No encontré «{dicho}» en el catálogo. Dígallo como aparece "
1655                 "en la etiqueta."}
1656     tope = candidatos[0]["confianza"]
1657     empatados = [c for c in candidatos if c["confianza"] == tope]
1658     if len(empatados) > 1:
1659         # Un empate NO es ambiguo si uno de ellos es el nombre EXACTO
1660         # dicho ("papa criolla" == "PAPA CRIOLLA") - eso gana solo,
1661         # aunque "PAPA CRIOLLA PRECOCIDA" haya empatado en puntaje.
1662         clave = normalizar(dicho)
1663         exacto = next((c for c in empatados if normalizar(c["nombre"]) == clave), None)
1664         if exacto:
1665             return {"articulo": exacto}
1666         # De los empatados, primero los nombres más cortos: un
1667         # ingrediente crudo ("PAPA CRIOLLA") es más probable que sea lo
1668         # que la persona quiso decir que un producto elaborado con
1669         # media frase de más ("EMPANADA DE PAPA Y CARNE X50 GR").
1670         empatados.sort(key=lambda c: len(c["nombre"].split()))
1671         mostrados = empatados[:5]
1672         opciones = ", ".join(c["nombre"].title() for c in mostrados)
1673         return {"error": f"«{dicho}» es ambiguo, encontré varios parecidos: {opciones}. "
1674                 "Dígallo más específico, como aparece completo en la etiqueta.",
1675                 "opciones": mostrados}
1676     return {"articulo": candidatos[0]}
1677
1678
1679 def _buscar_receta_por_nombre(s, nombre_dicho: str):
1680     """Una coincidencia EXACTA siempre gana primero: con dos recetas
1681     "Sopa" y "Sopa de la casa", decir "Sopa de la casa" no debe caer en
1682     "Sopa" solo porque su nombre es substring - eso pasaba antes (se
1683     quedaba con la PRIMERA que calzara, sin importar el orden de
1684     especificidad) y edita/borraba la receta equivocada en silencio."""
1685     t = _normalizar_nombre(nombre_dicho)
1686     recetas = s.query(Receta).all()
1687     exacta = next((r for r in recetas if _normalizar_nombre(r.nombre) == t), None)
1688     if exacta:
1689         return exacta
1690     return next(
1691         (r for r in recetas if re.search(rf"\b{re.escape(_normalizar_nombre(r.nombre))}\b", t)
1692          or re.search(rf"\b{re.escape(t)}\b", _normalizar_nombre(r.nombre))),
1693         None)
1694
1695
1696 _INTENCION_CREAR_RECETA = re.compile(

```

```

1697     r"\b(cre[ae]\w*|necesito|quiero)\b.*\brecetas?\b|\breceta\s+llamad[ao]\b",
1698     re.IGNORECASE)
1699
1700
1701 def _completar_crear_receta(nombre: str, rend: float, cant: float, articulo: dict,
1702                             u: Usuario) -> dict:
1703     with Sesión() as s:
1704         if s.query(Receta).filter(Receta.nombre.ilike(nombre)).first():
1705             return {"respuesta_hablada": f"Ya existe una receta llamada {nombre}.",
1706                     "accion": None, "destino": None, "pestana": None}
1707         r = Receta(nombre=nombre, rendimiento=int(rend), preparacion="")
1708         s.add(r)
1709         s.flush()
1710         s.add(RecetaIngrediente(receta_id=r.id, articulo_codigo=articulo["codigo"],
1711                                 cantidad_por_porcion=float(cant)))
1712         s.commit()
1713     registrar(u, "RECETA", f"Receta creada: {nombre} (1 ingrediente) por voz", "ok")
1714     return {"respuesta_hablada": f"Listo, creé la receta {nombre} con {articulo['nombre'].title()}. "
1715                                   "Puede agregarle más ingredientes diciendo «agrega... a la "
1716                                   f"receta {nombre}», o editarla desde la pantalla.",
1717             "accion": "actualizar", "destino": None, "pestana": None}
1718
1719
1720 def _crear_receta_por_voz(texto: str, u: Usuario) -> dict | None:
1721     """Crea una receta llamada <nombre>, rendimiento <N> porciones, con
1722     <cantidad> de <ingrediente>" - determinístico, con el mismo criterio
1723     que crear usuario: un formato de frase concreto, en vez de adivinar
1724     de cualquier forma un nombre y una lista de ingredientes que no
1725     existen todavía contra qué validar. El ingrediente SÍ se busca
1726     contra el catálogo real (misma función que usa Conteo/Pedido) - si
1727     no lo encuentra con confianza suficiente, no crea nada a medias.
1728     Decir solo "crea una receta" (sin los demás datos) coincidía por
1729     casualidad con la navegación a la pestaña Recetas - mismo arreglo
1730     que crear usuario."""
1731     if u.perfil != "auditor":
1732         return None
1733     m = _CREAR_RECETA.search(texto.strip())
1734     if not m:
1735         if (_INTENCION_CREAR_RECETA.search(texto)
1736             and not _ES_PREGUNTA_PESTANA.search(texto)):
1737             return {"respuesta_hablada": "Para crear una receta diga: «crea una receta "
1738                                         "llamada» el nombre, «rendimiento» el número de "
1739                                         "porciones, «con» la cantidad «de» el primer "
1740                                         "ingrediente.",
1741                     "accion": None, "destino": None, "pestana": None}
1742         return None
1743     nombre = re.sub(r"[.,;:!?;]+$", "", m.group("nombre")).strip()
1744     if not nombre:
1745         return None
1746     # El rendimiento y la cantidad se dicen casi siempre en palabras
1747     # ("cuatro porciones", no "4 porciones" - así habla la gente de
1748     # verdad), así que se interpretan con el mismo parser de números
1749     # que ya usa Conteo/Pedido, no un \d+ que solo entiende dígitos.
1750     rend = _numero_de_texto(m.group("rend"))
1751     cant = _numero_de_texto(m.group("cant"))
1752     if rend is None or cant is None:
1753         return {"respuesta_hablada": "No entendí el rendimiento o la cantidad. Dígalos en "
1754                                     "número, por ejemplo «cuatro porciones». ",
1755                 "accion": None, "destino": None, "pestana": None}
1756     elegido = _elegir_ingrediente_sin_ambigüedad(m.group("ing"))
1757     if "error" in elegido:
1758         if "opciones" in elegido:
1759             _PENDIENTE_AMBIGÜEDAD[u.id] = {
1760                 "tipo": "crear_receta", "nombre": nombre, "rend": rend, "cant": cant,
1761                 "opciones": elegido["opciones"], "ts": ahora()}
1762             return {"respuesta_hablada": elegido["error"],
1763                     "accion": None, "destino": None, "pestana": None}
1764     return _completar_crear_receta(nombre, rend, cant, elegido["articulo"], u)
1765
1766
1767 _INTENCION_AGREGAR_INGREDIENTE = re.compile(
1768     r"\b(agr[eé]ga\w*|añad\w*)\b.*\breceta\b", re.IGNORECASE)
1769
1770
1771 def _completar_agregar_ingrediente(cantidad_nueva: float, receta_dicha: str,
1772                                     articulo: dict, u: Usuario) -> dict | None:
1773     with Sesión() as s:
1774         r = _buscar_receta_por_nombre(s, receta_dicha)
1775         if not r:
1776             return None
1777         lineas = s.query(RecetaIngrediente).filter_by(receta_id=r.id).all()
1778         ya_tiene = next((li for li in lineas if li.articulo_codigo == articulo["codigo"]), None)
1779         if ya_tiene:
1780             ya_tiene.cantidad_por_porcion = cantidad_nueva
1781         else:
1782             s.add(RecetaIngrediente(receta_id=r.id, articulo_codigo=articulo["codigo"],

```

```

1783         cantidad_por_porcion=cantidad_nueva))
1784     s.commit()
1785     nombre_receta = r.nombre
1786     registrar(u, "RECETA", f"Receta editada: {nombre_receta} (ingrediente agregado por voz)", "ok")
1787     return {"respuesta_hablada": f"Listo, agregué {articulo['nombre'].title()} a la receta {nombre_receta}.",
1788           "accion": "actualizar", "destino": None, "pestana": None}
1789
1790
1791 def _agregar_ingredientes_por_voz(texto: str, u: Usuario) -> dict | None:
1792     """Agregale <cantidad> de <ingrediente> a la receta <nombre> -
1793     determinístico. Reusa la misma búsqueda de artículo que crear
1794     receta, y conserva los ingredientes que ya tenía (PUT reemplaza
1795     toda la lista, así que se lee la receta primero)."""
1796     if u.perfil != "auditor":
1797         return None
1798     m = _AGREGAR_INGREDIENTE.search(texto.strip())
1799     if not m:
1800         # "preparación" en la frase es de _agregar_preparacion_por_voz,
1801         # no de esta - sin la exclusión, "agrega la preparación a la
1802         # receta X: ..." caía aquí primero (comparte "agrega...receta")
1803         # y nunca le daba la oportunidad a la función correcta.
1804         if (_INTENCION_AGREGAR_INGREDIENTE.search(texto)
1805             and not re.search(r"preparaci[ó]n", texto, re.IGNORECASE)
1806             and not _ES_PREGUNTA_PESTANA.search(texto)):
1807             return {"respuesta_hablada": "Para agregar un ingrediente diga: «agrega» la "
1808                   "cantidad «de» el ingrediente «a la receta» el "
1809                   "nombre de la receta.",
1810                   "accion": None, "destino": None, "pestana": None}
1811         return None
1812     cantidad_nueva = _numero_de_texto(m.group("cant"))
1813     if cantidad_nueva is None:
1814         return {"respuesta_hablada": "No entendí la cantidad. Dígala en número, por "
1815               "ejemplo «trescientos gramos». ",
1816               "accion": None, "destino": None, "pestana": None}
1817     elegido = _elegir_ingredientes_sin_ambigüedad(m.group("ing"))
1818     if "error" in elegido:
1819         if "opciones" in elegido:
1820             _PENDIENTE_AMBIGÜEDAD[u.id] = {
1821                 "tipo": "agregar_ingredientes", "cant": cantidad_nueva,
1822                 "receta": m.group("receta"), "opciones": elegido["opciones"],
1823                 "ts": ahora()}
1824             return {"respuesta_hablada": elegido["error"],
1825                   "accion": None, "destino": None, "pestana": None}
1826     return _completar_agregar_ingredientes(cantidad_nueva, m.group("receta"),
1827                                           elegido["articulo"], u)
1828
1829
1830 def _resolver_ambigüedad_pendiente(texto: str, u: Usuario) -> dict | None:
1831     """Si la última respuesta de esta persona fue "es ambiguo, encontré
1832     Papa Sabanera, Papa Criolla, Papa Pastusa..." y en este turno dice
1833     solo "papa pastusa" (sin repetir la orden completa), esto retoma la
1834     orden original con ese ingrediente ya resuelto. Se revisa AL FINAL
1835     de la cadena de comandos de Ajustes, después de que crear/agregar
1836     ya tuvieron su oportunidad de matchear la frase completa - así una
1837     orden repetida entera (en vez de solo el nombre corto) sigue
1838     tomando el camino normal, con sus datos frescos, no los guardados
1839     aquí de la vez anterior."""
1840     pendiente = _PENDIENTE_AMBIGÜEDAD.get(u.id)
1841     if not pendiente:
1842         return None
1843     if (ahora() - pendiente["ts"]).total_seconds() > _VENCIMIENTO_AMBIGÜEDAD:
1844         del _PENDIENTE_AMBIGÜEDAD[u.id]
1845         return None
1846     t = _normalizar_nombre(texto)
1847     elegido = next(
1848         (c for c in pendiente["opciones"]
1849          if re.search(rf"\b{re.escape(_normalizar_nombre(c['nombre']))}\b", t)
1850          or re.search(rf"\b{re.escape(t)}\b", _normalizar_nombre(c["nombre"]))),
1851         None)
1852     if not elegido:
1853         return None
1854     del _PENDIENTE_AMBIGÜEDAD[u.id]
1855     if pendiente["tipo"] == "crear_receta":
1856         return _completar_crear_receta(pendiente["nombre"], pendiente["rend"],
1857                                       pendiente["cant"], elegido, u)
1858     return _completar_agregar_ingredientes(pendiente["cant"], pendiente["receta"], elegido, u)
1859
1860
1861 def _eliminar_receta_por_voz(texto: str, u: Usuario) -> dict | None:
1862     """Elimina la receta <nombre> - determinístico, busca por nombre
1863     completo entre las recetas existentes, igual que aprobaciones. NUNCA
1864     borra en el mismo turno que se pide: pregunta primero y solo borra si
1865     el siguiente turno confirma (ver _resolver_confirmacion_eliminar_receta)
1866     - es una acción irreversible (se lleva también los ingredientes) y un
1867     "elimina la receta X" mal escuchado, o una intención mal adivinada por
1868     Gemini, no debe bastar para borrarla sin confirmación explícita, igual

```

```

1869     que ya exige el botón equivalente en pantalla (window.confirm)."""
1870     if u.perfil != "auditor":
1871         return None
1872     m = _ELIMINAR_RECETA.search(texto.strip())
1873     if not m:
1874         return None
1875     with Sesión() as s:
1876         r = _buscar_receta_por_nombre(s, m.group("receta"))
1877         if not r:
1878             return None
1879         rid, nombre = r.id, r.nombre
1880         _PENDIENTE_AMBIGUEDAD[u.id] = {"tipo": "eliminar_receta", "receta_id": rid,
1881                                         "nombre": nombre, "ts": ahora()}
1882     return {"respuesta_hablada": f"¿Confirma que elimina la receta {nombre}? ",
1883            "Esta acción no se puede deshacer.",
1884            "accion": None, "destino": None, "pestana": None}
1885
1886
1887 def _resolver_confirmacion_eliminar_receta(texto: str, u: Usuario) -> dict | None:
1888     """El sí/no al "¿confirma que elimina la receta X?" de arriba. Se
1889     revisa ANTES que cualquier otra orden de esta pantalla: un "sí" o un
1890     "no" sueltos no deben caer en ningún otro reconocedor mientras esta
1891     confirmación sigue pendiente."""
1892     pendiente = _PENDIENTE_AMBIGUEDAD.get(u.id)
1893     if not pendiente or pendiente.get("tipo") != "eliminar_receta":
1894         return None
1895     if (ahora() - pendiente["ts"]).total_seconds() > _VENCIMIENTO_AMBIGUEDAD:
1896         del _PENDIENTE_AMBIGUEDAD[u.id]
1897         return None
1898     t = texto.lower()
1899     palabras = t.replace(" ", "").replace(".", "").split()
1900     dice_si = (any(p in ("sí", "si", "claro", "listo", "dale", "correcto", "confirmando") for p in palabras)
1901               or "confirmando" in t or "así es" in t or "así es" in t)
1902     dice_no = "no" in palabras or "mejor no" in t or "cancela" in t
1903     if not (dice_si or dice_no):
1904         return None
1905     del _PENDIENTE_AMBIGUEDAD[u.id]
1906     if dice_no:
1907         return {"respuesta_hablada": f"Entendido, no elimino la receta {pendiente['nombre']}.",
1908                "accion": None, "destino": None, "pestana": None}
1909     with Sesión() as s:
1910         r = s.get(Receta, pendiente["receta_id"])
1911         if r is None:
1912             return {"respuesta_hablada": "Esa receta ya no existe.",
1913                    "accion": None, "destino": None, "pestana": None}
1914         nombre = r.nombre
1915         s.query(RecetaIngrediente).filter_by(receta_id=r.id).delete()
1916         s.delete(r)
1917         s.commit()
1918     registrar(u, "RECETA", f"Receta eliminada: {nombre} por voz (confirmada)", "ok")
1919     return {"respuesta_hablada": f"Listo, eliminé la receta {nombre}.",
1920            "accion": "actualizar", "destino": None, "pestana": None}
1921
1922
1923 _QUITAR_INGREDIENTE = re.compile(
1924     r"\b(?:qu|ii|ta|w*|elimin|w*|borr|w*)\s+(?:el\s+)?ingredient\w*\s+(?P<ing>.+)"
1925     r"\s+de\s+la\s+receta\s+(?P<receta>.+)\s*" + _CORTE_ORDEN_ENCADENADA + r"[.!]?s*$",
1926     re.IGNORECASE)
1927
1928
1929 def _quitar_ingredientes_por_voz(texto: str, u: Usuario) -> dict | None:
1930     """Quita/elimina el ingrediente <ingrediente> de la receta <nombre> -
1931     mismo botón «Editar» de Recetas, pero solo para sacar UN ingrediente
1932     (no reemplaza toda la lista). Busca el ingrediente entre los que YA
1933     tiene esa receta, no contra todo el catálogo - así "quita la
1934     cebolla" no confunde una cebolla que ni siquiera está en esa
1935     receta. Nunca deja una receta sin ingredientes (la misma regla que
1936     ya exige el botón)."""
1937     if u.perfil != "auditor":
1938         return None
1939     m = _QUITAR_INGREDIENTE.search(texto.strip())
1940     if not m:
1941         return None
1942     with Sesión() as s:
1943         r = _buscar_receta_por_nombre(s, m.group("receta"))
1944         if not r:
1945             return None
1946         lineas = s.query(RecetaIngrediente).filter_by(receta_id=r.id).all()
1947         t_ing = _normalizar_nombre(m.group("ing"))
1948         objetivo = None
1949         nombre_articulo = None
1950         for li in lineas:
1951             art = s.get(Articulo, li.articulo_codigo)
1952             if not art:
1953                 continue
1954             nombre_art = _normalizar_nombre(art.nombre_oficial)

```

```

1955         if (re.search(rf"\b{re.escape(nombre_art)}\b", t_ing)
1956             or re.search(rf"\b{re.escape(t_ing)}\b", nombre_art)):
1957             objetivo, nombre_articulo = li, art.nombre_oficial
1958             break
1959     if not objetivo:
1960         return {"respuesta_hablada": f"{r.nombre.title()} no tiene un ingrediente llamado "
1961             f"«{m.group('ing')}».",
1962             "accion": None, "destino": None, "pestana": None}
1963     if len(lineas) <= 1:
1964         return {"respuesta_hablada": f"No puedo quitar {nombre_articulo.title()}: "
1965             f"{r.nombre.title()} se quedaría sin ingredientes.",
1966             "accion": None, "destino": None, "pestana": None}
1967     s.delete(objetivo)
1968     nombre_receta = r.nombre
1969     s.commit()
1970     registrar(u, "RECETA", f"Receta editada: {nombre_receta} ({nombre_articulo} quitado por voz)", "ok")
1971     return {"respuesta_hablada": f"Listo, quité {nombre_articulo.title()} de la receta {nombre_receta}.",
1972           "accion": "actualizar", "destino": None, "pestana": None}
1973
1974
1975 _CAMBIAR_RENDIMIENTO = re.compile(
1976     r"\bcambia\w*\s+el\s+rendimiento\w*\s+de\s+la\s+receta\s+(?P<receta>.+)"
1977     r"\s+a\s+(?P<rend>.+)\s+porcion\w*\s*" + _CORTE_ORDEN_ENCADENADA + r"[.!?]\s*$",
1978     re.IGNORECASE)
1979
1980
1981 def _cambiar_rendimiento_por_voz(texto: str, u: Usuario) -> dict | None:
1982     """Cambia el rendimiento de la receta <nombre> a <N> porciones" -
1983     otro campo editable del botón «Editar» de Recetas (además de los
1984     ingredientes, que se agregan/quitan aparte, y la preparación, que
1985     se dicta con _agregar_preparacion_por_voz)."""
1986     if u.perfil != "auditor":
1987         return None
1988     m = _CAMBIAR_RENDIMIENTO.search(texto.strip())
1989     if not m:
1990         return None
1991     nuevo = _numero_de_texto(m.group("rend"))
1992     if nuevo is None:
1993         return {"respuesta_hablada": "No entendí el rendimiento. Dígalo en número, por "
1994             "ejemplo «seis porciones».",
1995             "accion": None, "destino": None, "pestana": None}
1996     nuevo = int(nuevo)
1997     with Sesion() as s:
1998         r = _buscar_receta_por_nombre(s, m.group("receta"))
1999         if not r:
2000             return None
2001         if r.rendimiento == nuevo:
2002             return {"respuesta_hablada": f"{r.nombre.title()} ya rinde {nuevo} porciones.",
2003                 "accion": "actualizar", "destino": None, "pestana": None}
2004         r.rendimiento = nuevo
2005         nombre = r.nombre
2006         s.commit()
2007     registrar(u, "RECETA", f"Receta editada: {nombre} (rendimiento -> {nuevo} por voz)", "ok")
2008     return {"respuesta_hablada": f"Listo, {nombre.title()} ahora rinde {nuevo} porciones.",
2009           "accion": "actualizar", "destino": None, "pestana": None}
2010
2011
2012 _AGREGAR_PREPARACION = re.compile(
2013     r"\b(?:agr[eé]ga\w*|cambia\w*|pon\w*|dicta\w*|escrib\w*)\s+(?:la\s+)?preparaci[oó]n"
2014     r"\s+(?:de\s+|a\s+)?(?:la\s+receta\s+)?(?P<receta>.+)\s*[:;]\s*(?P<pasos>.+)\s*$",
2015     re.IGNORECASE)
2016 _INTENCION_AGREGAR_PREPARACION = re.compile(
2017     r"\b(?:agr[eé]ga\w*|cambia\w*|pon\w*|dicta\w*|escrib\w*)\b.*\bpreparaci[oó]n\b",
2018     re.IGNORECASE)
2019
2020
2021 def _agregar_preparacion_por_voz(texto: str, u: Usuario) -> dict | None:
2022     """Agrega la preparación a la receta <nombre>: <pasos>" - a
2023     diferencia de nombre/ingrediente/rendimiento (frases cortas), aquí
2024     todo lo que sigue a los dos puntos ES el dato, tal cual, sin cortar
2025     en la primera coma o punto (los pasos de cocina naturalmente tienen
2026     varias oraciones: "primero pele las papas. Luego sofía...") - por
2027     eso esta es la ÚNICA función de Recetas que NO usa
2028     _CORTE_ORDEN_ENCADENADA. Reemplaza la preparación completa (como el
2029     textarea del botón «Editar»), no la suma a lo que ya había."""
2030     if u.perfil != "auditor":
2031         return None
2032     m = _AGREGAR_PREPARACION.search(texto.strip())
2033     if not m:
2034         if (_INTENCION_AGREGAR_PREPARACION.search(texto)
2035             and not _ES_PREGUNTA_PESTANA.search(texto)):
2036             return {"respuesta_hablada": "Para dictar la preparación diga: «agrega la "
2037                 "preparación a la receta» el nombre, dos puntos, y "
2038                 "los pasos - por ejemplo «agrega la preparación a "
2039                 "la receta Sopa: pele las papas, sofía la cebolla, "
2040                 "y cocine todo junto veinte minutos».",

```

```

2041         "accion": None, "destino": None, "pestana": None}
2042     return None
2043     pasos = re.sub(r"[.,;:!?;]+", "", m.group("pasos")).strip()
2044     if not pasos:
2045         return None
2046     with Sesion() as s:
2047         r = _buscar_receta_por_nombre(s, m.group("receta"))
2048         if not r:
2049             return None
2050         r.preparacion = pasos
2051         nombre = r.nombre
2052         s.commit()
2053     registrar(u, "RECETA", f"Receta editada: {nombre} (preparación dictada por voz)", "ok")
2054     return {"respuesta_hablada": f"Listo, guardé la preparación de la receta {nombre}.",
2055           "accion": "actualizar", "destino": None, "pestana": None}
2056
2057
2058 def _cambiar_perfil_por_voz(texto: str, u: Usuario) -> dict | None:
2059     """Cambia el perfil de <nombre> a auxiliar/administrador" - mismo
2060     botón «Editar» de Gestión de usuarios, solo el campo perfil (el
2061     correo no se toca por voz: un correo mal transcrito rompería el
2062     contacto sin que se note). Llama la MISMA función que usa el botón
2063     (editar_usuario), así que hereda su regla de no poder cambiarse el
2064     propio rol - excluida además desde la búsqueda del nombre."""
2065     if u.perfil != "auditor" or not re.search(r"\bperfil\b", texto, re.IGNORECASE):
2066         return None
2067     m = re.search(r"\b(auxiliar|auditor|administrador)\b", texto, re.IGNORECASE)
2068     if not m:
2069         if not _ES_PREGUNTA_PESTANA.search(texto):
2070             return {"respuesta_hablada": "Diga a qué perfil: «cambia el perfil de» el "
2071                   "nombre «a» auxiliar o administrador.",
2072                   "accion": None, "destino": None, "pestana": None}
2073         return None
2074     perfil_nuevo = "auditor" if m.group(1).lower() in ("auditor", "administrador") else "auxiliar"
2075     t = _normalizar_nombre(texto)
2076     with Sesion() as s:
2077         candidatos = s.query(Usuario).filter(Usuario.id != u.id).all()
2078         encontrado = next(
2079             (c for c in candidatos
2080              if re.search(rf"\b{re.escape(_normalizar_nombre(c.nombre))}\b", t)),
2081             None)
2082     if not encontrado:
2083         return {"respuesta_hablada": "No encontré a esa persona. Diga el nombre completo "
2084               "tal como aparece en la lista.",
2085               "accion": None, "destino": None, "pestana": None}
2086     etiqueta = "administrador" if perfil_nuevo == "auditor" else "auxiliar"
2087     if encontrado.perfil == perfil_nuevo:
2088         return {"respuesta_hablada": f"{encontrado.nombre.capitalize()} ya era {etiqueta}.",
2089               "accion": "actualizar", "destino": None, "pestana": None}
2090     eid, nombre = encontrado.id, encontrado.nombre
2091     editar_usuario(usuario_id=eid, datos=EditarUsuarioIn(perfil=perfil_nuevo), u=u)
2092     return {"respuesta_hablada": f"Listo, {nombre.capitalize()} ahora es {etiqueta}.",
2093           "accion": "actualizar", "destino": None, "pestana": None}
2094
2095
2096 _ASIGNAR_BODEGA = re.compile(
2097     r"\bas[íí]gn\w*\s+(?:le\s+)?(?:la\s+)?bodega\s+(?P<bodega>.+?)\s+a\s+(?P<nombre>.+?)"
2098     r"\s*" + _CORTE_ORDEN_ENCADENADA + r"[.!]?s*$", re.IGNORECASE)
2099 _QUITAR_BODEGA = re.compile(
2100     r"\bqu[íí]t\w*\s+(?:le\s+)?(?:la\s+)?bodega\s+(?P<bodega>.+?)\s+a\s+(?P<nombre>.+?)"
2101     r"\s*" + _CORTE_ORDEN_ENCADENADA + r"[.!]?s*$", re.IGNORECASE)
2102
2103
2104 def _asignar_bodega_por_voz(texto: str, u: Usuario) -> dict | None:
2105     """Asigne/quítale la bodega <bodega> a <nombre>" - mismo botón
2106     «Asignar bodegas», pero sumando o quitando SOLO la bodega dicha en
2107     vez de reemplazar toda la lista (que es lo que hace el botón, que
2108     parte de las bodegas ya marcadas): así una orden de voz nunca borra
2109     por accidente asignaciones hechas por otro medio. El nombre de la
2110     bodega se busca con el mismo buscador que «abra <bodega>» en
2111     Conteo, así que entiende apodos parciales igual ("kiosco taquilla"
2112     encuentra "KIOSCO TAQUILLA AYB")."""
2113     if u.perfil != "auditor":
2114         return None
2115     m = _ASIGNAR_BODEGA.search(texto.strip())
2116     quitar = False
2117     if not m:
2118         m = _QUITAR_BODEGA.search(texto.strip())
2119         quitar = True
2120     if not m:
2121         # OJO: el verbo debe terminar justo en "a/ale/ar" (con \b después)
2122         # para no confundirse con el SUSTANTIVO "asignación" - sin ese
2123         # límite, "muéstrame la asignación por bodega" (que es la orden
2124         # de abrir el panel, no de asignar nada) caía aquí por error,
2125         # porque "asign" es raíz común de las dos palabras.
2126         if (re.search(r"\bbodega\b", texto, re.IGNORECASE)

```

```

2127         and re.search(r"\bas[í]gn(?:a(?:le)?|ar)\b|\bqu[í]t(?:a(?:le)?|ar)\b",
2128             texto, re.IGNORECASE)
2129         and not _ES_PREGUNTA_PESTANA.search(texto)):
2130     return {"respuesta_hablada": "Diga: «asigne/quítele la bodega» el nombre de "
2131         "la bodega «a» el nombre de la persona.",
2132         "accion": None, "destino": None, "pestana": None}
2133     return None
2134 from servicios.conciliacion import buscar_bodega
2135 t_nombre = _normalizar_nombre(m.group("nombre"))
2136 with Sesion() as s:
2137     bodega = buscar_bodega(s, m.group("bodega"), None)
2138     if not bodega:
2139         return {"respuesta_hablada": f"No encontré una bodega llamada «{m.group('bodega')}».",
2140             "accion": None, "destino": None, "pestana": None}
2141     candidatos = s.query(Usuario).filter(Usuario.id != u.id).all()
2142     persona = next(
2143         (c for c in candidatos
2144          if re.search(rf"\b{re.escape(_normalizar_nombre(c.nombre))}\b", t_nombre)),
2145         None)
2146     if not persona:
2147         return {"respuesta_hablada": "No encontré a esa persona. Diga el nombre completo "
2148             "tal como aparece en la lista.",
2149             "accion": None, "destino": None, "pestana": None}
2150     actuales = {a.bodega_id for a in
2151         s.query(AsignacionBodega).filter_by(usuario_id=persona.id).all()}
2152     ya_tenia = bodega.id in actuales
2153     if quitar and not ya_tenia:
2154         return {"respuesta_hablada": f"{persona.nombre.capitalize()} no tenía asignada "
2155             f"{bodega.nombre_oficial.title()}.",
2156             "accion": "actualizar", "destino": None, "pestana": None}
2157     if not quitar and ya_tenia:
2158         return {"respuesta_hablada": f"{persona.nombre.capitalize()} ya tenía asignada "
2159             f"{bodega.nombre_oficial.title()}.",
2160             "accion": "actualizar", "destino": None, "pestana": None}
2161     if quitar:
2162         actuales.discard(bodega.id)
2163     else:
2164         actuales.add(bodega.id)
2165     pid, pnombre, bnombre = persona.id, persona.nombre, bodega.nombre_oficial
2166     nuevos_ids = list(actuales)
2167     asignar_bodegas(usuario_id=pid, body={"bodega_ids": nuevos_ids, u=u)
2168     verbo = "quitó" if quitar else "asignó"
2169     return {"respuesta_hablada": f"Listo, le {verbo} {bnombre.title()} a {pnombre.capitalize()}.",
2170         "accion": "actualizar", "destino": None, "pestana": None}
2171
2172
2173 _QUIEN_TIENE_BODEGA = re.compile(
2174     r"\bqui[é]n\s+(?:tiene|es\s+responsable\s+de|maneja)\s+(?:la\s+)?bodega\s+"
2175     r"(?:<P>bodega.+?)\s*[.!?]*\s*$", re.IGNORECASE)
2176 _BODEGAS_SIN_ASIGNAR = re.compile(r"\bbodegas?\b.*\bsin\s+asignar\b", re.IGNORECASE)
2177
2178
2179 def _consultar_asignacion_bodega_por_voz(texto: str, u: Usuario) -> dict | None:
2180     """Lo mismo que el botón «Ver asignación por bodega», pero como
2181     pregunta hablada en vez de abrir la tabla completa de 54 filas:
2182     "¿quién tiene la bodega <bodega>?" contesta solo esa fila, y
2183     "¿cuántas bodegas sin asignar hay?" resume las que no tiene nadie -
2184     de solo lectura, así que no exige ninguna confirmación extra."""
2185     if u.perfil != "auditor":
2186         return None
2187     if _BODEGAS_SIN_ASIGNAR.search(texto):
2188         with Sesion() as s:
2189             asignadas = {a.bodega_id for a in s.query(AsignacionBodega).all()}
2190             sin = [b.nombre_oficial for b in s.query(Bodega).all() if b.id not in asignadas]
2191         if not sin:
2192             return {"respuesta_hablada": "Todas las bodegas tienen al menos una persona asignada.",
2193                 "accion": None, "destino": None, "pestana": None}
2194         listado = ", ".join(n.title() for n in sin[:8])
2195         extra = f" y {len(sin) - 8} más" if len(sin) > 8 else ""
2196         return {"respuesta_hablada": f"{len(sin)} bodegas sin asignar: {listado}{extra}.",
2197             "accion": None, "destino": None, "pestana": None}
2198     m = _QUIEN_TIENE_BODEGA.search(texto.strip())
2199     if not m:
2200         return None
2201     from servicios.conciliacion import buscar_bodega
2202     with Sesion() as s:
2203         bodega = buscar_bodega(s, m.group("bodega"), None)
2204         if not bodega:
2205             return {"respuesta_hablada": f"No encontré una bodega llamada «{m.group('bodega')}».",
2206                 "accion": None, "destino": None, "pestana": None}
2207         asignados = [s.get(Usuario, a.usuario_id) for a in
2208             s.query(AsignacionBodega).filter_by(bodega_id=bodega.id).all()]
2209         nombres = [p.nombre.capitalize() for p in asignados if p]
2210         titulo = bodega.nombre_oficial.title()
2211     if not nombres:
2212         return {"respuesta_hablada": f"{titulo} no tiene a nadie asignado todavía.",

```

```

2213         "accion": None, "destino": None, "pestaña": None}
2214     return {"respuesta_hablada": f"{titulo} está asignada a {'', '.join(nombres)}.",
2215            "accion": None, "destino": None, "pestaña": None}
2216
2217
2218     _ABRIR_COBERTURA = re.compile(
2219         r"\b(mu[é]stra[w*]|ens[é] [ñ]a[w*]| [aá]bre[w*]|ver)\b.*\basignaci[oó]n\b.*\bbodega",
2220         re.IGNORECASE)
2221     _CERRAR_COBERTURA = re.compile(
2222         r"\b(oc[ú]lta[w*]|ci[é]rra[w*]|esc[oó]nde[w*])\b.*\basignaci[oó]n\b.*\bbodega",
2223         re.IGNORECASE)
2224
2225
2226 def _alternar_cobertura_por_voz(texto: str, u: Usuario) -> dict | None:
2227     """El botón «Ver/Ocultar asignación por bodega» abre o cierra la
2228     tabla completa de 54 filas (quién tiene cada bodega) - a diferencia
2229     de _consultar_asignacion_bodega_por_voz (que contesta una pregunta
2230     puntual sin tocar la pantalla), esto es la orden de abrir/cerrar el
2231     panel mismo, igual que el clic al botón."""
2232     if u.perfil != "auditor":
2233         return None
2234     if _ABRIR_COBERTURA.search(texto):
2235         return {"respuesta_hablada": "Listo, ahí tiene la asignación por bodega.",
2236                "accion": "mostrar_cobertura", "destino": None, "pestaña": None}
2237     if _CERRAR_COBERTURA.search(texto):
2238         return {"respuesta_hablada": "Listo, la oculto.",
2239                "accion": "ocultar_cobertura", "destino": None, "pestaña": None}
2240     return None
2241
2242
2243     _EXPORTAR_TRAZA = re.compile(
2244         r"\b(export[w*]|descarg[w*]|genera[w*])\b.*\b(traza|trazabilidad)\b", re.IGNORECASE)
2245
2246
2247 def _exportar_trazabilidad_por_voz(texto: str, u: Usuario) -> dict | None:
2248     """El botón «Exportar» de Registro de trazabilidad exporta con los
2249     filtros (persona/acción/rango) que YA están marcados en pantalla -
2250     esos viven como estado de React en el frontend, no llegan en el
2251     texto de esta orden, así que no se puede armar el archivo aquí en
2252     el backend. En cambio, se avisa con la misma acción genérica que ya
2253     usa mostrar/ocultar cobertura, y el frontend llama a su propia
2254     función exportar() con los filtros que tenga puestos - igual que si
2255     hubiera dado clic al botón."""
2256     if u.perfil != "auditor":
2257         return None
2258     if not _EXPORTAR_TRAZA.search(texto):
2259         return None
2260     return {"respuesta_hablada": "Listo, exportando el registro de trazabilidad con los "
2261            "filtros que tiene puestos.",
2262            "accion": "exportar_trazabilidad", "destino": None, "pestaña": None}
2263
2264
2265     _ETIQUETA_RANGO_TRAZA = {"hoy": "hoy", "semana": "en la última semana",
2266                              "mes": "en el último mes", "": "en total"}
2267
2268
2269 def _rango_dicho_traza(texto: str) -> str:
2270     t = texto.lower()
2271     if re.search(r"\bsemana\b", t):
2272         return "semana"
2273     if re.search(r"\bmese\b", t):
2274         return "mes"
2275     if re.search(r"\btodo\b|\bsiempre\b", t):
2276         return ""
2277     return "hoy" # mismo valor por defecto con el que abre la pestaña
2278
2279
2280     _ACCIONES_DE_PERSONA = re.compile(
2281         r"\bcu[á]ntas\s+acciones\s+tiene\s+(?P<persona>.+?)\s*[!?]*\s*$", re.IGNORECASE)
2282     _ACCIONES_POR_PERSONA = re.compile(
2283         r"\bacciones\s+por\s+persona\b|\bcu[á]ntas\s+acciones\s+tiene\s+cada\s+persona\b|"
2284         r"\bresumen\s+de\s+acciones\b", re.IGNORECASE)
2285
2286
2287 def _acciones_por_persona_por_voz(texto: str, u: Usuario) -> dict | None:
2288     """Pedido explícito del usuario: "acciones por persona" contesta un
2289     resumen con cuántas hizo cada quien, y "¿cuántas acciones tiene
2290     <nombre>?" contesta solo por esa persona - los dos de solo lectura,
2291     sin abrir ni cambiar nada en pantalla, así que no hace falta
2292     confirmación. El periodo por defecto es "hoy" (igual que la
2293     pestaña al abrirse); decir "esta semana"/"este mes"/"en total"
2294     lo cambia."""
2295     if u.perfil != "auditor":
2296         return None
2297     m = _ACCIONES_DE_PERSONA.search(texto.strip())
2298     if m:

```

```

2299     rango = _rango_dicho_traza(texto)
2300     t_nombre = _normalizar_nombre(m.group("persona"))
2301     with Sesion() as s:
2302         candidatos = s.query(Usuario).all()
2303         persona_obj = next(
2304             (c for c in candidatos
2305              if re.search(rf"\b{re.escape(_normalizar_nombre(c.nombre))}\b", t_nombre)),
2306             None)
2307         if not persona_obj:
2308             return None
2309         total = _filtro_traza(s, persona_obj.nombre, "", rango).count()
2310         nombre = persona_obj.nombre
2311         return {"respuesta_hablada": f"{nombre.capitalize()} tiene {total} acciones "
2312                 f"{_ETIQUETA_RANGO_TRAZA[rango]}.",
2313                 "accion": None, "destino": None, "pestana": None}
2314 if _ACCIONES_POR_PERSONA.search(texto):
2315     rango = _rango_dicho_traza(texto)
2316     with Sesion() as s:
2317         filas = _filtro_traza(s, "", "", rango).all()
2318     if not filas:
2319         return {"respuesta_hablada": f"No hay acciones registradas {_ETIQUETA_RANGO_TRAZA[rango]}.",
2320                 "accion": None, "destino": None, "pestana": None}
2321     conteos: dict[str, int] = {}
2322     for f in filas:
2323         conteos[f.persona] = conteos.get(f.persona, 0) + 1
2324     ordenado = sorted(conteos.items(), key=lambda par: -par[1])
2325     resumen = ", ".join(f"{p.capitalize()}: {n}" for p, n in ordenado[:8])
2326     extra = f", y {len(ordenado) - 8} personas más" if len(ordenado) > 8 else ""
2327     return {"respuesta_hablada": f"Acciones por persona {_ETIQUETA_RANGO_TRAZA[rango]}: "
2328             f"{resumen}{extra}.",
2329             "accion": None, "destino": None, "pestana": None}
2330 return None
2331
2332
2333 def _resolver_asistente(vista: str, texto: str, u: Usuario) -> dict:
2334     """Lo que responde el agente liviano ante una pregunta suelta o una
2335     orden de navegar - usado tanto por /api/agente/asistente (Inicio,
2336     Ajustes, Ayuda, Reportes, Panel) como, cuando ni un artículo ni una
2337     bodega coinciden, por el buscador de Bodegas (ver consulta_articulo):
2338     un solo buscador que primero prueba si es un ingrediente, luego si
2339     es una bodega, y si no es ninguna de las dos, cae aquí."""
2340     texto = _quitar_relleno(texto)
2341     if vista == "inicio":
2342         acceso = _acceso_rapido_inicio_por_voz(texto, u)
2343         if acceso:
2344             return acceso
2345         # Desde Bodegas, "abra/siga contando <nombre>" es una orden concreta y
2346         # frecuente - se resuelve aparte, ANTES de Gemini, con la misma
2347         # búsqueda restringida que usa el resto de la app (nunca ofrece una
2348         # bodega ajena a lo asignado). El verbo es obligatorio: sin él, decir
2349         # el nombre de una bodega podría ser una PREGUNTA sobre ella ("¿cómo
2350         # va kiosco taquilla ayb?"), no una orden de ir a contarla.
2351     if vista == "bodegas" and _VERBOS_ABRIR_BODEGA.search(texto):
2352         from servicios.conciliacion import buscar_bodega
2353         with Sesion() as s:
2354             bodega = buscar_bodega(s, texto, _ids_permitidos_para_buscar(s, u))
2355         if bodega:
2356             return {"respuesta_hablada": f"Vamos a Conteo a abrir {bodega.nombre_oficial.title()}.",
2357                     "accion": "navegar", "destino": "conteo", "pestana": None,
2358                     "bodega": bodega.nombre_oficial}
2359     if vista == "ajustes":
2360         # el sí/no a "¿confirma que elimina la receta X?" tiene prioridad
2361         # sobre cualquier otra cosa: mientras esa confirmación siga
2362         # pendiente, un "sí" o un "no" sueltos son la respuesta a ESO, no
2363         # una orden nueva - revisarlo antes evita que caiga en otro
2364         # reconocedor de la cadena de abajo.
2365         confirmacion = _resolver_confirmacion_eliminar_receta(texto, u)
2366         if confirmacion:
2367             return confirmacion
2368         # identidad (no solo presencia) de lo que había pendiente ANTES
2369         # de este turno - crear_receta/agregar_ingredientes pueden dejar
2370         # una ambigüedad NUEVA en el mismo turno (cambio ya sale
2371         # "truthy" con el mensaje de "es ambiguo...") y esa no se debe
2372         # borrar; solo se limpia la que YA estaba ahí sin que nada de
2373         # este turno la haya tocado.
2374         pendiente_antes = _PENDIENTE_AMBIGÜEDAD.get(u.id)
2375         cambio = (_cambiar_modos_sin_conexion(texto, u) or _cambiar_estado_usuario(texto, u)
2376                  or _crear_usuario_por_voz(texto, u) or _cambiar_perfil_por_voz(texto, u)
2377                  or _asignar_bodega_por_voz(texto, u) or _alternar_cobertura_por_voz(texto, u)
2378                  or _consultar_asignacion_bodega_por_voz(texto, u)
2379                  or _crear_receta_por_voz(texto, u)
2380                  or _agregar_ingredientes_por_voz(texto, u) or _quitar_ingredientes_por_voz(texto, u)
2381                  or _cambiar_rendimiento_por_voz(texto, u) or _agregar_preparacion_por_voz(texto, u)
2382                  or _eliminar_receta_por_voz(texto, u) or _exportar_trazabilidad_por_voz(texto, u)
2383                  or _acciones_por_persona_por_voz(texto, u))
2384         if cambio:

```

```

2385         # una orden distinta que sí coincidió significa que la
2386         # persona siguió para otra cosa - una pregunta ambigua sin
2387         # contestar de hace un rato no debe "resucitar" más tarde
2388         # por una frase corta que por casualidad se parezca a una
2389         # de esas opciones viejas.
2390         if _PENDIENTE_AMBIGUEDAD.get(u.id) is pendiente_antes:
2391             _PENDIENTE_AMBIGUEDAD.pop(u.id, None)
2392         return cambio
2393     resuelta_pendiente = _resolver_ambiguedad_pendiente(texto, u)
2394     if resuelta_pendiente:
2395         return resuelta_pendiente
2396     if vista == "reportes":
2397         generado = _generar_reporte_por_voz(texto, u)
2398         if generado:
2399             return generado
2400     previsualizado = _previsualizar_reporte_por_voz(texto, u)
2401     if previsualizado:
2402         return previsualizado
2403     respondida = _responder_reportes_por_voz(texto, u)
2404     if respondida:
2405         return respondida
2406     if vista == "auditoria":
2407         resuelta = _resolver_aprobacion_por_voz(texto, u)
2408         if resuelta:
2409             return resuelta
2410         resuelto_pedido = _resolver_pedido_pendiente_por_voz(texto, u)
2411         if resuelto_pedido:
2412             return resuelto_pedido
2413         resuelta_alerta = _resolver_alerta_por_voz(texto, u)
2414         if resuelta_alerta:
2415             return resuelta_alerta
2416         auditada = _abrir_bodega_para_auditoria_por_voz(texto, u)
2417         if auditada:
2418             return auditada
2419         respondida = _responder_auditoria_por_voz(texto, u)
2420         if respondida:
2421             return respondida
2422     if vista == "panel":
2423         respondida = _responder_panel_por_voz(texto, u)
2424         if respondida:
2425             return respondida
2426     if vista == "perfil":
2427         cambio = (_cambiar_voz_por_voz(texto, u) or _cambiar_velocidad_por_voz(texto, u)
2428                 or _cambiar_confirmacion_hablada_por_voz(texto, u))
2429         if cambio:
2430             return cambio
2431         respondida = _responder_perfil_por_voz(texto, u)
2432         if respondida:
2433             return respondida
2434     navegada = _navegar_pestana_por_voz(vista, texto)
2435     if navegada:
2436         return navegada
2437     destinos = dict(_DESTINOS_ASISTENTE)
2438     if u.perfil != "auditor":
2439         for k in _SOLO_AUDITOR_ASISTENTE:
2440             destinos.pop(k, None)
2441     # Sin esto, Gemini podía "sugerir" ir a la pantalla en la que la
2442     # persona YA está (confirmado: ofreció "llevarla a Gestión de
2443     # usuarios" estando ella ahí mismo) - una orden real de cambiar de
2444     # PESTANA dentro de la misma pantalla ya se resolvió antes, arriba
2445     # (_navegar_pestana_por_voz), así que si el mensaje llega hasta aquí
2446     # nunca es eso.
2447     destinos.pop(vista, None)
2448     contexto = _contexto_asistente(vista, u)
2449     from agente.cerebro import pensar_asistente
2450     turno = pensar_asistente(contexto, texto, destinos, _PESTANAS_ASISTENTE)
2451     destino = turno.get("destino")
2452     if (turno.get("intencion") or "").lower() == "navegar" and destino in destinos:
2453         pestana = turno.get("pestana")
2454         pestanas_validas = _PESTANAS_ASISTENTE.get(destino, {})
2455         if pestana not in pestanas_validas:
2456             pestana = None
2457         return {"respuesta_hablada": turno.get("respuesta_hablada", ""),
2458               "accion": "navegar", "destino": destino, "pestana": pestana}
2459     return {"respuesta_hablada": turno.get("respuesta_hablada", ""),
2460           "accion": None, "destino": None, "pestana": None}
2461
2462
2463 @app.post("/api/agente/asistente")
2464 def api_asistente(p: PreguntarAsistenteIn, u: Usuario = Depends(usuario_actual)):
2465     """Version liviana del agente para pantallas que no cuentan ni piden
2466     (Inicio, Ajustes, Ayuda, Reportes, Panel): responde preguntas sobre lo
2467     que hay en esa pantalla y navega a otra si se lo piden. Sin el estado
2468     multi-turno de /api/agente/turno, que es específico del conteo/pedido."""
2469     return _resolver_asistente(p.vista, p.texto, u)
2470

```

```

2471
2472 @app.get("/api/sesiones/{sesion_id}/avance")
2473 def ver_avance(sesion_id: int, bodega_id_respaldo: int | None = None,
2474               u: Usuario = Depends(usuario_actual)):
2475     est = ESTADOS.setdefault(sesion_id, {})
2476     # mismo respaldo que en /agente/turno: si el backend se reinicio, el
2477     # frontend todavia sabe en que bodega estaba y este numero deja de
2478     # verse roto ("avance: 4 de 0") en vez de exigir que se reabra.
2479     if bodega_id_respaldo and not est.get("bodega_id"):
2480         est["bodega_id"] = bodega_id_respaldo
2481     with Sesion() as s:
2482         hechas = s.query(Conteo).filter_by(sesion_id=sesion_id,
2483                                           estado="confirmado").count()
2484         alertas = s.query(Alerta).filter_by(resuelta=0).count()
2485         total = (s.query(StockSistema).filter_by(bodega_id=est.get("bodega_id")).count()
2486                 if est.get("bodega_id") else 0)
2487         ultimos = (s.query(Conteo).filter_by(sesion_id=sesion_id, estado="confirmado")
2488                   .order_by(Conteo.id.desc()).limit(5).all())
2489         detalle = []
2490         for c in ultimos:
2491             a = s.get(Articulo, c.articulo_codigo)
2492             detalle.append({"nombre": a.nombre_oficial if a else c.articulo_codigo,
2493                           "cantidad": c.cantidad, "unidad": c.unidad})
2494     return {"hechas": hechas, "total": total, "alertas": alertas,
2495           "bodega": est.get("bodega_nombre"), "ultimos": detalle}
2496
2497
2498 def _limpiar_nombre_dictado(t: str) -> str:
2499     """El reconocimiento de voz del navegador cierra la frase con
2500     puntuación que nadie dijo ("Juguetería."). Ya se le quita para
2501     BUSCAR una bodega (servicios.conciliacion.normalizar); esto hace lo
2502     mismo pero para GUARDAR un nombre nuevo - sin tocar tildes ni
2503     mayúsculas, que sí importan en lo que queda escrito en el catálogo."""
2504     import re
2505     return re.sub(r"[.,;:!?;]+\\s*$", "", t.strip()).strip()
2506
2507
2508 class CrearProductoIn(BaseModel):
2509     nombre: str
2510     unidad_medida: str
2511     cantidad_inicial: float
2512     sesion_id: int = 1
2513
2514
2515 @app.post("/api/conteo/crear-producto")
2516 def crear_producto_pendiente(p: CrearProductoIn, u: Usuario = Depends(usuario_actual)):
2517     """El conteo no se detiene: el producto entra pendiente y sigue contando.
2518     Si quien cuenta es el administrador, el producto se confirma de una -
2519     igual que en crear_bodega_pendiente, dejarlo pendiente significaría que
2520     solo el mismo administrador puede aprobarlo despues."""
2521     est = ESTADOS.get(p.sesion_id, {})
2522     bodega_id = est.get("bodega_id")
2523     if not bodega_id:
2524         raise HTTPException(409, "Abra una bodega antes de crear un producto.")
2525     if p.cantidad_inicial < 0:
2526         raise HTTPException(400, "La cantidad inicial no puede ser negativa.")
2527     nombre = _limpiar_nombre_dictado(p.nombre).upper()
2528     # el sufijo de hora sin el sufijo aleatorio solo tenia ~100 microsegundos
2529     # de resolucion real (se truncaba a 10 caracteres): dos personas creando
2530     # un producto casi al mismo tiempo podian generar el mismo codigo y
2531     # tumbar la peticion con un IntegrityError sin capturar.
2532     codigo = f"PEND-{ahora().strftime('%Y%m%d%H%M%S')}-{secrets.token_hex(3).upper()}"
2533     es_auditor = u.perfil == "auditor"
2534     with Sesion() as s:
2535         s.add(Articulo(codigo=codigo, nombre_oficial=nombre,
2536                      unidad_medida=p.unidad_medida))
2537         s.commit()
2538         conteo = Conteo(sesion_id=p.sesion_id, articulo_codigo=codigo,
2539                        cantidad=p.cantidad_inicial, unidad=p.unidad_medida,
2540                        estado="confirmado" if es_auditor else "pendiente_aprobacion")
2541         s.add(conteo)
2542         s.commit()
2543         s.refresh(conteo)
2544         if es_auditor:
2545             s.add(StockSistema(articulo_codigo=codigo, bodega_id=bodega_id, cantidad_sd=0))
2546         else:
2547             s.add(Aprobacion(tipo="producto", nombre=nombre,
2548                             unidad_medida=p.unidad_medida, cantidad_inicial=p.cantidad_inicial,
2549                             bodega_id=bodega_id, articulo_codigo=codigo,
2550                             conteo_id=conteo.id, creado_por_id=u.id))
2551         s.commit()
2552     if es_auditor:
2553         registrar(u, "CREACION", f"{nombre} creado")
2554         return {"ok": True, "codigo": codigo,
2555               "respuesta_hablada": "Creado y confirmado en el catálogo."}
2556     registrar(u, "CREACION", f"{nombre} creado, pendiente de aprobacion")

```

```

2557     return {"ok": True, "codigo": codigo,
2558            "respuesta_hablada": "Creado. Queda pendiente de aprobación del administrador; "
2559                               "el conteo sigue sin interrupción."}
2560
2561
2562 # ----- bodegas -----
2563 class AbrirIn(BaseModel):
2564     bodega: str
2565
2566
2567 def _contexto_bodega(s, b, refs):
2568     """Segun el estado, lo que hace falta para entender de un vistazo quien
2569     esta en que y cuanto lleva - se usa tanto en el tablero completo como
2570     en «mis bodegas asignadas» de Conteo."""
2571     item = {}
2572     if b.estado == "en_conteo":
2573         ses = (s.query(SesionConteo)
2574               .filter_by(bodega_id=b.id, tipo="conteo", estado="abierta")
2575               .order_by(SesionConteo.id.desc()).first())
2576         if ses:
2577             persona = s.get(Usuario, ses.usuario_id)
2578             hechas = s.query(Conteo).filter_by(
2579                 sesion_id=ses.id, estado="confirmado").count()
2580             item["persona"] = persona.nombre if persona else None
2581             item["avance_pct"] = round(hechas / refs * 100) if refs else 0
2582     elif b.estado == "en_auditoria":
2583         ses = (s.query(SesionConteo).filter_by(bodega_id=b.id, tipo="auditoria")
2584               .order_by(SesionConteo.id.desc()).first())
2585         if ses:
2586             persona = s.get(Usuario, ses.usuario_id)
2587             item["persona"] = persona.nombre if persona else None
2588             conteo_ses = (s.query(SesionConteo).filter_by(bodega_id=b.id, tipo="conteo")
2589                           .order_by(SesionConteo.id.desc()).first())
2590             difs = 0
2591             if conteo_ses:
2592                 for c in (s.query(Conteo).filter_by(
2593                     sesion_id=conteo_ses.id, estado="confirmado").all()):
2594                     st = s.query(StockSistema).filter_by(
2595                         articulo_codigo=c.articulo_codigo, bodega_id=b.id).first()
2596                     if abs((c.cantidad or 0) - (st.cantidad_sd if st else 0)) > 0.001:
2597                         difs += 1
2598             item["diferencias"] = difs
2599     elif b.estado == "cerrada":
2600         hist = (s.query(HistorialCierre).filter_by(bodega_id=b.id)
2601               .order_by(HistorialCierre.fecha.desc()).first())
2602         if hist:
2603             item["hora_cierre"] = hist.fecha.strftime("%H:%M")
2604     return item
2605
2606
2607 @app.get("/api/bodegas")
2608 def listar_bodegas(propias: int = 0, u: Usuario = Depends(usuario_actual)):
2609     """El tablero en vivo: cada tarjeta necesita un dato distinto segun su
2610     estado (quien cuenta y cuanto lleva, quien audita y cuantas diferencias
2611     encontro, o a que hora se cerro) - no solo el total de referencias.
2612
2613     ?propias=1 limita el tablero a las bodegas asignadas a quien pregunta -
2614     para un auditor/administrador. Si la persona no tiene ninguna bodega
2615     asignada, ve todas (administrador general, sin zona propia); si tiene
2616     algunas asignadas, ve solo esas (un supervisor de zona con varios
2617     administradores repartiendo el parque). El resto de pantallas
2618     (Pedidos elige donde descontar, Ajustes arma la lista para asignar)
2619     siguen pidiendo la lista completa sin este parametro - pantallas que
2620     solo un auditor/administrador alcanza a abrir.
2621
2622     Un auxiliar, en cambio, nunca ve bodegas fuera de su asignacion: para
2623     ese perfil la restriccion aplica siempre, con o sin ?propias=1, para
2624     que ninguna pantalla (incluida Pedidos) filtre por accidente el parque
2625     completo a quien solo debe ver su zona."""
2626     with Sesion() as s:
2627         ids_asignadas = {a.bodega_id for a in
2628                          s.query(AsignacionBodega).filter_by(usuario_id=u.id).all()}
2629         restringir = propias or u.perfil == "auxiliar"
2630         if restringir and ids_asignadas:
2631             bodegas = (s.query(Bodega).filter(Bodega.id.in_(ids_asignadas))
2632                       .order_by(Bodega.nombre_oficial).all())
2633         else:
2634             bodegas = s.query(Bodega).order_by(Bodega.nombre_oficial).all()
2635         salida = []
2636         for b in bodegas:
2637             refs = s.query(StockSistema).filter_by(bodega_id=b.id).count()
2638             item = {"id": b.id, "bodega": b.nombre_oficial,
2639                   "estado": b.estado, "referencias": refs}
2640             item.update(_contexto_bodega(s, b, refs))
2641             salida.append(item)
2642     return salida

```

```

2643
2644
2645 @app.post("/api/bodegas/buscar")
2646 def buscar_bodega_endpoint(a: AbrirIn, u: Usuario = Depends(usuario_actual)):
2647     """Solo resuelve el nombre, no abre nada - para que el selector de
2648     bodega pueda preguntar "¿confirma que abro X?" con el nombre REAL que
2649     de verdad se va a abrir, en vez de repetir a ciegas lo que se
2650     reconoció (decir «kiosco» no debe confirmarse como si "KIOSCO" fuera
2651     una bodega real, cuando lo que existe es "KIOSCO TAQUILLA AYB").
2652
2653     Si hay varias candidatas igual de razonables ("restaurante" a secas
2654     encaja en "RESTAURANTE FUENTES AYB" Y "RESTAURANTE FUENTES SUMIN"),
2655     se devuelven como opciones en vez de decir "no la encuentro" a secas
2656     - así la persona puede elegir la que quería, no repetir a ciegas."""
2657     from servicios.conciliacion import buscar_bodegas_candidatas
2658     with Sesion() as s:
2659         ids_permitidos = _ids_permitidos_para_buscar(s, u)
2660         candidatas = buscar_bodegas_candidatas(s, a.bodega, ids_permitidos)
2661         if len(candidatas) == 1:
2662             b = candidatas[0]
2663             return {"encontrada": True, "bodega": b.nombre_oficial, "bodega_id": b.id,
2664                     "opciones": []}
2665         opciones = [{"bodega": c.nombre_oficial, "bodega_id": c.id} for c in candidatas[:6]]
2666         return {"encontrada": False, "bodega": None, "bodega_id": None, "opciones": opciones}
2667
2668
2669 @app.post("/api/bodegas/abrir")
2670 async def abrir(a: AbrirIn, u: Usuario = Depends(usuario_actual)):
2671     from servicios.conciliacion import buscar_bodega
2672     with Sesion() as s:
2673         # buscar_bodega() (misma funcion que usa el agente conversacional):
2674         # quita tildes y puntuación de sobra (el reconocimiento de voz a
2675         # veces cierra la frase con un "." que nadie dijo) y prioriza la
2676         # coincidencia exacta - sin esto, "Zoológico" podía no encontrarse
2677         # por el punto final, o abrir "ZOOLOGICO SUMINISTROS" en vez de la
2678         # bodega "ZOOLOGICO" que de verdad se pidió. Restringsida a lo
2679         # asignado si es auxiliar: no debe ni encontrar, ni ofrecer,
2680         # bodegas de otra zona que igual no podría abrir.
2681         b = buscar_bodega(s, a.bodega, _ids_permitidos_para_buscar(s, u))
2682         if b is None:
2683             raise HTTPException(404, "No encuentro esa bodega.")
2684         if u.perfil == "auxiliar" and not s.query(AsignacionBodega).filter_by(
2685             usuario_id=u.id, bodega_id=b.id).first():
2686             raise HTTPException(403, "Esa bodega no esta asignada a usted.")
2687         abierta = s.query(SesionConteo).filter_by(bodega_id=b.id, tipo="conteo",
2688             estado="abierta").first()
2689         if abierta and abierta.usuario_id != u.id:
2690             raise HTTPException(409, "Esa bodega ya esta en conteo por otra persona.")
2691         ses = abierta or SesionConteo(bodega_id=b.id, usuario_id=u.id)
2692         if not abierta:
2693             s.add(ses)
2694             b.estado = "en_conteo"
2695             s.commit()
2696             s.refresh(ses)
2697         refs = s.query(StockSistema).filter_by(bodega_id=b.id).count()
2698         sid, nombre, bid = ses.id, b.nombre_oficial, b.id
2699         ESTADOS.setdefault(sid, {}).update(
2700             {"bodega_id": bid, "bodega_nombre": nombre, "sesion_bd": sid})
2701         registrar(u, "APERTURA", f"{nombre} abierta - sesion bloqueada")
2702         await difundir_estado()
2703         return {"sesion_id": sid, "bodega": nombre, "referencias": refs, "bodega_id": bid}
2704
2705
2706 class CrearBodegaIn(BaseModel):
2707     nombre: str
2708
2709
2710 @app.post("/api/bodegas/crear-pendiente")
2711 def crear_bodega_pendiente(p: CrearBodegaIn, u: Usuario = Depends(usuario_actual)):
2712     """La bodega no se crea de una para un auxiliar: queda pendiente de
2713     aprobacion del administrador (Aprobacion tipo "bodega", ya soportada
2714     en aprobar()/rechazar()) - asi el catalogo no crece con lo que
2715     cualquiera escriba sin control. Un administrador ya tiene esa
2716     autoridad, asi que para el la bodega se crea de inmediato - de lo
2717     contrario terminaria aprobando su propia solicitud, lo cual no
2718     verifica nada (visto en produccion: Diana pidio "ALIMENTOS" y quedo
2719     esperando su propia firma en su propia bandeja)."""
2720     nombre = _limpiar_nombre_dictado(p.nombre).upper()
2721     if not nombre:
2722         raise HTTPException(400, "Dígame el nombre de la bodega.")
2723     with Sesion() as s:
2724         if s.query(Bodega).filter_by(nombre_oficial=nombre).first():
2725             raise HTTPException(409, "Ya existe una bodega con ese nombre.")
2726         if u.perfil == "auditor":
2727             s.add(Bodega(nombre_oficial=nombre, estado="pendiente"))
2728             s.commit()

```

```

2729         registrar(u, "CREACION", f"Bodega {nombre} creada")
2730         return {"ok": True,
2731                 "respuesta_hablada": f"{nombre.capitalize()} creada. Ya está en el catálogo."}
2732     s.add(Aprobacion(tipo="bodega", nombre=nombre, creado_por_id=u.id))
2733     s.commit()
2734     registrar(u, "CREACION", f"Bodega {nombre} solicitada, pendiente de aprobacion")
2735     return {"ok": True,
2736            "respuesta_hablada": f"Solicitud de {nombre.lower()} enviada. Queda "
2737                                "pendiente de aprobación del administrador."}
2738
2739
2740 def _ultima_sesion(s, bodega_id: int, tipo: str):
2741     return (s.query(SesionConteo).filter_by(bodega_id=bodega_id, tipo=tipo)
2742            .order_by(SesionConteo.id.desc()).first())
2743
2744
2745 def _requiere_acceso_bodega(s, u: Usuario, bodega_id: int):
2746     """El tablero (listar_bodegas) ya oculta las bodegas ajenas a un
2747     auxiliar, pero eso no basta: sin este chequeo, pedir el detalle,
2748     las firmas o el inventario completo por su bodega_id directamente
2749     (sin pasar por el tablero) dejaba ver auditoria y stock de cualquier
2750     zona con solo cambiar el numero en la URL."""
2751     if u.perfil == "auxiliar" and not s.query(AsignacionBodega).filter_by(
2752         usuario_id=u.id, bodega_id=bodega_id).first():
2753         raise HTTPException(403, "Esa bodega no esta asignada a usted.")
2754
2755
2756 def _ids_permitidos_para_buscar(s, u: Usuario) -> set[int] | None:
2757     """Para restringir buscar_bodega()/buscar_bodegas_candidatas() a lo
2758     que un auxiliar de verdad puede abrir - sin esto, decir "restaurante"
2759     ofrecia como opciones bodegas de zonas ajenas que ni siquiera podia
2760     llegar a abrir, puro ruido (y un nombre que no tenia por qué ver).
2761     None para un auditor: puede buscar en todo el catálogo."""
2762     if u.perfil != "auxiliar":
2763         return None
2764     return {a.bodega_id for a in
2765            s.query(AsignacionBodega).filter_by(usuario_id=u.id).all()}
2766
2767
2768 @app.get("/api/bodegas/{bodega_id}/firmas")
2769 def ver_firmas(bodega_id: int, u: Usuario = Depends(usuario_actual)):
2770     with Sesion() as s:
2771         _requiere_acceso_bodega(s, u, bodega_id)
2772         b = s.get(Bodega, bodega_id)
2773         conteo = _ultima_sesion(s, bodega_id, "conteo")
2774         auditoria = _ultima_sesion(s, bodega_id, "auditoria")
2775
2776         def _lado(ses):
2777             if ses is None:
2778                 return None
2779             au = s.get(Usuario, ses.usuario_id)
2780             return {"sesion_id": ses.id, "persona": au.nombre if au else "?",
2781                    "firmada": bool(ses.firmada),
2782                    "hora": ses.fin.strftime("%H:%M") if ses.fin else None}
2783
2784         referencias = s.query(StockSistema).filter_by(bodega_id=bodega_id).count()
2785         alertas_resueltas = (s.query(Alerta).join(Conteo, Alerta.conteo_id == Conteo.id)
2786                             .join(SesionConteo, Conteo.sesion_id == SesionConteo.id)
2787                             .filter(SesionConteo.bodega_id == bodega_id,
2788                                    Alerta.resuelta == 1).count())
2789         return {"bodega": b.nombre_oficial if b else None, "referencias": referencias,
2790                "alertas_resueltas": alertas_resueltas,
2791                "conteo": _lado(conteo), "auditoria": _lado(auditoria),
2792                "lista_para_cerrar": bool(conteo and conteo.firmada
2793                                           and auditoria and auditoria.firmada)}
2794
2795
2796 @app.post("/api/sesiones/{sesion_id}/firmar")
2797 def firmar_sesion(sesion_id: int, u: Usuario = Depends(usuario_actual)):
2798     """El auxiliar firma su conteo; el administrador firma su recuento ciego."""
2799     with Sesion() as s:
2800         ses = s.get(SesionConteo, sesion_id)
2801         if ses is None:
2802             raise HTTPException(404, "Sesion no encontrada.")
2803         if ses.usuario_id != u.id and u.perfil != "auditor":
2804             raise HTTPException(403, "Solo quien contó puede firmar este conteo.")
2805         ses.firmada = 1
2806         ses.fin = ahora()
2807         b = s.get(Bodega, ses.bodega_id)
2808         if ses.tipo == "conteo" and b and b.estado == "en_conteo":
2809             b.estado = "en_auditoria"
2810         s.commit()
2811         nombre = b.nombre_oficial if b else "?"
2812         tipo = ses.tipo
2813         etiqueta = "Conteo" if tipo == "conteo" else "Auditoria"
2814         registrar(u, "FIRMA", f"{etiqueta} firmado - {nombre}", "ok")

```

```

2815     return {"ok": True}
2816
2817
2818 @app.post("/api/bodegas/{bodega_id}/auditoria/iniciar")
2819 async def iniciar_auditoria(bodega_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
2820     with Sesion() as s:
2821         b = s.get(Bodega, bodega_id)
2822         if b is None:
2823             raise HTTPException(404, "Bodega no encontrada.")
2824         existente = s.query(SesionConteo).filter_by(
2825             bodega_id=bodega_id, tipo="auditoria", estado="abierta").first()
2826         ses = existente or SesionConteo(bodega_id=bodega_id, usuario_id=u.id, tipo="auditoria")
2827         if not existente:
2828             s.add(ses)
2829             s.commit()
2830             s.refresh(ses)
2831         refs = s.query(StockSistema).filter_by(bodega_id=bodega_id).count()
2832         sid, nombre = ses.id, b.nombre_oficial
2833         ESTADOS.setdefault(sid, {}).update({"bodega_id": bodega_id, "bodega_nombre": nombre})
2834         registrar(u, "AUDITORIA", f"Recuento ciego iniciado en {nombre}")
2835         await difundir_estado()
2836         return {"sesion_id": sid, "bodega": nombre, "referencias": refs}
2837
2838
2839 @app.get("/api/bodegas/{bodega_id}/auditoria/comparar")
2840 def comparar_auditoria(bodega_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
2841     with Sesion() as s:
2842         b = s.get(Bodega, bodega_id)
2843         ses_conteo = _ultima_sesion(s, bodega_id, "conteo")
2844         ses_audit = _ultima_sesion(s, bodega_id, "auditoria")
2845
2846     def _mapa(ses):
2847         m = {}
2848         if not ses:
2849             return m
2850         for c in (s.query(Conteo).filter_by(sesion_id=ses.id, estado="confirmado")
2851                 .order_by(Conteo.id).all()):
2852             m[c.articulo_codigo] = c.cantidad
2853         return m
2854     m1, m2 = _mapa(ses_conteo), _mapa(ses_audit)
2855     codigos = set(m1) | set(m2)
2856     filas = []
2857     diagnosticos = []
2858     for cod in codigos:
2859         art = s.get(Articulo, cod)
2860         nombre = art.nombre_oficial if art else cod
2861         st = s.query(StockSistema).filter_by(articulo_codigo=cod, bodega_id=bodega_id).first()
2862         sistema = st.cantidad_sd if st else 0
2863         c1, c2 = m1.get(cod), m2.get(cod)
2864         autoridad = c2 if c2 is not None else c1
2865         dif = round((autoridad or 0) - sistema, 3)
2866         if abs(dif) < 0.01 and (c1 == c2 or c2 is None):
2867             continue
2868         filas.append({"codigo": cod, "articulo": nombre,
2869                     "conteo1": c1, "conteo2": c2, "sistema": sistema,
2870                     "diferencia": dif,
2871                     "accion": "Revisar" if (c1 is not None and c2 is not None and c1 != c2)
2872                     else "Aceptar"})
2873         # el sistema ya traia un saldo negativo antes de esta toma (una
2874         # salida sin su entrada correspondiente): no es un error del
2875         # conteo fisico, es un dato roto que My Inventory debe corregir.
2876         if sistema < 0:
2877             diagnosticos.append(
2878                 f"El {dif:+g} de {nombre.lower()} no viene de su conteo: el sistema "
2879                 f"ya traía saldo negativo ({sistema:g}, salida registrada sin entrada). "
2880                 f"Su conteo físico es el que vale; el reporte lo marca como negativo "
2881                 f"del sistema para corrección en My Inventory.")
2882     filas.sort(key=lambda f: -abs(f["diferencia"]))
2883     return {"bodega": b.nombre_oficial if b else None,
2884           "filas": filas, "coinciden": len(codigos) - len(filas) if codigos else 0,
2885           "total": len(codigos), "diagnosticos": diagnosticos}
2886
2887
2888 @app.post("/api/bodegas/{bodega_id}/auditoria/firmar")
2889 def firmar_auditoria(bodega_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
2890     with Sesion() as s:
2891         ses = _ultima_sesion(s, bodega_id, "auditoria")
2892         if ses is None:
2893             raise HTTPException(404, "No hay un recuento de auditoria iniciado.")
2894         ses.firmada = 1
2895         ses.fin = ahora()
2896         s.commit()
2897         b = s.get(Bodega, bodega_id)
2898         nombre = b.nombre_oficial if b else "?"
2899         registrar(u, "FIRMA", f"Auditoria firmada - {nombre}", "ok")
2900     return {"ok": True}

```

```

2901
2902
2903 @app.post("/api/bodegas/{bodega_id}/cerrar")
2904 async def cerrar(bodega_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
2905     with Sesion() as s:
2906         b = s.get(Bodega, bodega_id)
2907         if b is None:
2908             raise HTTPException(404, "Bodega no encontrada.")
2909         ses_conteo = _ultima_sesion(s, bodega_id, "conteo")
2910         ses_audit = _ultima_sesion(s, bodega_id, "auditoria")
2911         if not (ses_conteo and ses_conteo.firmada and ses_audit and ses_audit.firmada):
2912             raise HTTPException(409, "Faltan firmas: el conteo y la auditoría deben "
2913                                 "estar firmados antes de cerrar.")
2914         conteos = (s.query(Conteo).filter_by(sesion_id=ses_conteo.id, estado="confirmado").all())
2915         difs = 0
2916         for c in conteos:
2917             st = s.query(StockSistema).filter_by(
2918                 articulo_codigo=c.articulo_codigo, bodega_id=bodega_id).first()
2919             if abs((c.cantidad or 0) - (st.cantidad_sd if st else 0)) > 0.001:
2920                 difs += 1
2921         exact = round((len(conteos) - difs) / len(conteos) * 100, 1) if conteos else 100.0
2922         b.estado = "cerrada"
2923         for ses in s.query(SesionConteo).filter_by(bodega_id=bodega_id,
2924                                                    estado="abierta").all():
2925             ses.estado = "cerrada"
2926             if not ses.fin:
2927                 ses.fin = ahora()
2928         s.add(HistorialCierre(bodega_id=bodega_id, exactitud=exact,
2929                             referencias=len(conteos), diferencias=difs))
2930         s.commit()
2931         nombre = b.nombre_oficial
2932         registrar(u, "CIERRE", f"{nombre} cerrada con doble firma", "ok")
2933         await difundir_estado()
2934         return {"ok": True, "bodega": nombre}
2935
2936
2937 @app.post("/api/bodegas/{bodega_id}/reabrir")
2938 async def reabrir_bodega(bodega_id: int, body: dict,
2939                          u: Usuario = Depends(requiere_perfil("auditor"))):
2940     motivo = (body.get("motivo") or "").strip()
2941     if not motivo:
2942         raise HTTPException(400, "Reabrir una bodega cerrada exige una justificación escrita.")
2943     with Sesion() as s:
2944         b = s.get(Bodega, bodega_id)
2945         if b is None:
2946             raise HTTPException(404, "Bodega no encontrada.")
2947         if b.estado != "cerrada":
2948             raise HTTPException(409, "Solo se reabre una bodega que esté cerrada.")
2949         # las sesiones firmadas quedan como historial (nada se borra), pero
2950         # se marcan como reemplazadas para que abrir() arme una ronda de
2951         # conteo realmente nueva: reabrir sin esto no devolvía nada útil al
2952         # auxiliar, porque "abrir" seguía encontrando su sesion ya firmada.
2953         for ses in s.query(SesionConteo).filter_by(bodega_id=bodega_id, estado="abierta").all():
2954             ses.estado = "reemplazada"
2955         b.estado = "en_conteo"
2956         s.commit()
2957         nombre = b.nombre_oficial
2958         registrar(u, "REAPERTURA", f"{nombre} reabierta - motivo: {motivo}", "alerta")
2959         await difundir_estado()
2960         return {"ok": True, "bodega": nombre}
2961
2962
2963 def _narrar_hito(t):
2964     p = (t.persona or "alguien").title()
2965     if t.accion == "APERTURA":
2966         return f"{p} abre la bodega"
2967     if t.accion == "AUDITORIA":
2968         return f"{p} inicia el recuento ciego"
2969     if t.accion == "CORRECCION":
2970         return "Alerta: " + t.detalle.split(" (valor anterior)")[0]
2971     if t.accion == "FIRMA" and "Conteo firmado" in t.detalle:
2972         return f"{p} cierra el conteo"
2973     if t.accion == "FIRMA" and "Auditoria firmada" in t.detalle:
2974         return f"{p} firma el recuento"
2975     if t.accion == "CIERRE":
2976         return "Cierre con doble firma"
2977     if t.accion == "REAPERTURA":
2978         return f"{p}: {t.detalle}"
2979     return t.detalle
2980
2981
2982 def _color_hito(t):
2983     if t.accion == "CIERRE":
2984         return "verde"
2985     if t.accion in ("CORRECCION", "REAPERTURA"):
2986         return "oro"

```

```

2987     return "azul"
2988
2989
2990 @app.get("/api/bodegas/{bodega_id}/detalle")
2991 def detalle_bodega(bodega_id: int, u: Usuario = Depends(usuario_actual)):
2992     with Sesion() as s:
2993         _requiere_acceso_bodega(s, u, bodega_id)
2994         b = s.get(Bodega, bodega_id)
2995         if b is None:
2996             raise HTTPException(404, "Bodega no encontrada.")
2997         total = s.query(StockSistema).filter_by(bodega_id=bodega_id).count()
2998         # solo la ronda de conteo mas reciente: tras un reabrir(), la sesion
2999         # anterior queda como historial y no debe mezclarse con el recuento
3000         # nuevo en la exactitud/diferencias que se muestran aqui.
3001         ses_conteo_actual = _ultima_sesion(s, bodega_id, "conteo")
3002         conteos = (s.query(Conteo).filter_by(
3003             sesion_id=ses_conteo_actual.id, estado="confirmado")).all()
3004         if ses_conteo_actual else []
3005     difs = []
3006     for c in conteos:
3007         st = s.query(StockSistema).filter_by(
3008             articulo_codigo=c.articulo_codigo, bodega_id=bodega_id).first()
3009         esperado = st.cantidad_sd if st else 0
3010         if abs((c.cantidad or 0) - esperado) > 0.001:
3011             a = s.get(Articulo, c.articulo_codigo)
3012             difs.append({"articulo": a.nombre_oficial if a else c.articulo_codigo,
3013                 "contado": c.cantidad, "sistema": esperado,
3014                 "diferencia": round(c.cantidad - esperado, 3)})
3015
3016     sesiones = s.query(SesionConteo).filter_by(bodega_id=bodega_id).all()
3017     usuario_ids = {ses.usuario_id for ses in sesiones if ses.usuario_id}
3018     personas_nombres = []
3019     for uid in usuario_ids:
3020         us = s.get(Usuario, uid)
3021         if us and us.nombre.title() not in personas_nombres:
3022             personas_nombres.append(us.nombre.title())
3023     if len(personas_nombres) <= 1:
3024         personas = personas_nombres[0] if personas_nombres else None
3025     else:
3026         personas = ", ".join(personas_nombres[:-1]) + " y " + personas_nombres[-1]
3027
3028     # hitos: los que ya mencionan la bodega por nombre (apertura/cierre/etc)
3029     # mas las correcciones de la persona en el rango de su sesion (esas
3030     # no mencionan la bodega, solo el articulo).
3031     hitos_t = list(s.query(Traza).filter(
3032         Traza.detalle.contains(b.nombre_oficial)).all())
3033     # el nombre de una bodega puede ser substring del de otra (ej.
3034     # "ZOOLOGICO" de "ZOOLOGICO SUMINISTROS" y "ZOOLOGICO PISCILAGO";
3035     # tambien "MOVIL FONDA", "MOVIL MIRADOR", "PANADERIA" con su propia
3036     # "... SUMINISTROS") - confirmado en produccion: abrir "ZOOLOGICO
3037     # SUMINISTROS" hacia aparecer ese evento en la linea de tiempo de
3038     # "ZOOLOGICO" a secas, una bodega distinta que ni se habia tocado.
3039     # Cuando el texto contiene el nombre de mas de una bodega real, se
3040     # prefiere siempre el mas largo (el mas especifico) como dueño real
3041     # del evento.
3042     if hitos_t:
3043         _todos_nombres = [x.nombre_oficial for x in s.query(Bodega).all()]
3044         def _es_de_esta_bodega(detalle):
3045             candidatos = [n for n in _todos_nombres if n in detalle]
3046             return not candidatos or max(candidatos, key=len) == b.nombre_oficial
3047         hitos_t = [t for t in hitos_t if _es_de_esta_bodega(t.detalle)]
3048     vistos = {t.id for t in hitos_t}
3049     for ses in sesiones:
3050         if not ses.usuario_id:
3051             continue
3052         fin = ses.fin or ahora()
3053         correcciones = (s.query(Traza)
3054             .filter(Traza.accion == "CORRECCION",
3055                 Traza.usuario_id == ses.usuario_id,
3056                 Traza.creado >= ses.inicio, Traza.creado <= fin)
3057             .all())
3058         for t in correcciones:
3059             if t.id not in vistos:
3060                 hitos_t.append(t)
3061                 vistos.add(t.id)
3062     hitos_t.sort(key=lambda t: t.creado)
3063     hitos = [{"hora": t.creado.strftime("%H:%M"), "texto": _narrar_hito(t),
3064         "tipo": _color_hito(t)} for t in hitos_t]
3065
3066     alertas_resueltas = (s.query(Alerta).join(Conteo, Alerta.conteo_id == Conteo.id)
3067         .join(SesionConteo, Conteo.sesion_id == SesionConteo.id)
3068         .filter(SesionConteo.bodega_id == bodega_id,
3069             Alerta.resuelta == 1).count())
3070
3071     exact = round((len(conteos) - len(difs)) / len(conteos) * 100, 1) if conteos else 0
3072     duracion_min = None

```

```

3073     if ses_conteo_actual and ses_conteo_actual.fin:
3074         duracion_min = round((ses_conteo_actual.fin - ses_conteo_actual.inicio).total_seconds() / 60)
3075     # referencia fija de un conteo manual en papel (no hay dato real de
3076     # esto: es un supuesto declarado, no una medicion de Colsubsidio)
3077     tiempo_papel_min = round(total * 20 / 60) if total else None
3078
3079     unidades_stock = sum(f.cantidad_sd or 0 for f in
3080                          s.query(StockSistema).filter_by(bodega_id=bodega_id).all())
3081
3082     cierre_t = next((t for t in reversed(hitos_t) if t.accion == "CIERRE"), None)
3083     hora_cierre = cierre_t.creado.strftime("%H:%M") if cierre_t else None
3084
3085     ses_auditoria = next((ses for ses in sesiones if ses.tipo == "auditoria"), None)
3086     revisor = None
3087     if ses_auditoria and ses_auditoria.usuario_id:
3088         us = s.get(Usuario, ses_auditoria.usuario_id)
3089         revisor = us.nombre.title() if us else None
3090
3091     historial = (s.query(HistorialCierre).filter_by(bodega_id=bodega_id)
3092                .order_by(HistorialCierre.fecha.desc()).all())
3093     anterior = historial[1] if len(historial) > 1 else None
3094     return {"bodega": b.nombre_oficial, "estado": b.estado,
3095            "referencias": total, "contadas": len(conteos),
3096            "exactitud": f"{exact} %", "diferencias": difs, "hitos": hitos,
3097            "duracion_min": duracion_min, "tiempo_papel_min": tiempo_papel_min,
3098            "unidades_stock": round(unidades_stock, 1),
3099            "personas": personas, "alertas_resueltas": alertas_resueltas,
3100            "hora_cierre": hora_cierre, "revisor": revisor,
3101            "ultima_toma_anterior": ({'fecha': anterior.fecha.strftime("%Y-%m-%d"),
3102                                     "exactitud": anterior.exactitud}
                                     if anterior else None)}
3103
3104
3105
3106 @app.get("/api/bodegas/{bodega_id}/articulos")
3107 def articulos_bodega(bodega_id: int, u: Usuario = Depends(usuario_actual)):
3108     """El extracto completo de My Inventory para esta bodega, tal cual
3109     quedo cargado del Excel: codigo, articulo, unidad y saldo del sistema."""
3110     with Sesion() as s:
3111         _requiere_acceso_bodega(s, u, bodega_id)
3112         b = s.get(Bodega, bodega_id)
3113         if b is None:
3114             raise HTTPException(404, "Bodega no encontrada.")
3115         filas = (s.query(StockSistema, Articulo)
3116                .join(Articulo, Articulo.codigo == StockSistema.articulo_codigo)
3117                .filter(StockSistema.bodega_id == bodega_id)
3118                .order_by(Articulo.nombre_oficial).all())
3119         return [{"codigo": a.codigo, "articulo": a.nombre_oficial,
3120                "unidad": a.unidad_medida, "sd": st.cantidad_sd}
3121                for st, a in filas]
3122
3123
3124 # ----- consultas y reportes -----
3125 @app.get("/api/articulos/catalogo-ligero")
3126 def catalogo_ligero(u: Usuario = Depends(usuario_actual)):
3127     """El catálogo completo pero liviano (solo codigo/nombre/unidad), para
3128     que el modo sin conexión pueda sugerir nombres oficiales guardandolo
3129     en el equipo mientras hay señal."""
3130     with Sesion() as s:
3131         return [{"codigo": a.codigo, "nombre": a.nombre_oficial, "unidad": a.unidad_medida}
3132                for a in s.query(Articulo).all()]
3133
3134
3135 @app.get("/api/articulos/consulta")
3136 def consulta_articulo(q: str, codigo: str = "", u: Usuario = Depends(usuario_actual)):
3137     from servicios.conciliacion import buscar_articulo, buscar_bodegas_candidatas
3138     # Una orden explícita ("abra kiosco taquilla ayb") se resuelve ANTES
3139     # que cualquier otra cosa: si se dejara caer primero por la búsqueda
3140     # de articulo/bodega, el propio "abra" sumaba como palabra de más en
3141     # la coincidencia por palabras contra esa misma bodega y la orden
3142     # explícita nunca llegaba a reconocerse como tal - terminaba en la
3143     # sugerencia de "vea su detalle", no en abrirla de una vez.
3144     if _VERBOS_ABRIR_BODEGA.search(q):
3145         r = _resolver_asistente("bodegas", q, u)
3146         return {
3147             "resumen": r.get("respuesta_hablada", ""),
3148             "bodegas": [], "sugerencia_bodega": None, "sugerencia_bodega_id": None,
3149             "accion": r.get("accion"), "destino": r.get("destino"),
3150             "pestanas": r.get("pestanas"), "bodega": r.get("bodega"),
3151         }
3152     cand_todos = buscar_articulo(q)
3153     # Este buscador acepta CUALQUIER frase (nombre de ingrediente, de
3154     # bodega, una pregunta, una orden) - no solo un nombre de artículo
3155     # dictado a propósito, como sí es el caso en Conteo/Pedido (donde
3156     # este mismo umbral global de 45 sigue igual, sin tocar). Con ese
3157     # umbral más flojo aquí, frases sueltas sin relación con ningún
3158     # producto ("seguir contando", "no entendí", "espera") coincidían

```

```

3159 # por casualidad con productos reales que comparten una raíz de 4
3160 # letras (SEGUIR/SEGUNDOS, ENTENDI/ENTERO...) - una coincidencia
3161 # LEGÍTIMA en el sentido estructural (la misma que hace que BLANCA
3162 # encuentre BLANCO), pero sin ninguna relación real de significado.
3163 # Probado contra el catálogo real: una búsqueda de artículo genuina,
3164 # hasta parcial o descuidada ("tomate", "vino", "tabla picar"),
3165 # siempre puntuó 84 o más; el ruido de frases sueltas se quedó entre
3166 # 45 y 80. 82 separa limpio los dos grupos.
3167 UMBRAL_CONFIANZA_BODEGAS = 82
3168 # ya eligió una alternativa específica (clic en un chip de "¿era este
3169 # otro?") - se respeta tal cual, sin el filtro extra: ahí la persona
3170 # ya confirmó cuál quería, así que la confianza original no importa.
3171 cand = cand_todos if codigo else [c for c in cand_todos if c["confianza"] >= UMBRAL_CONFIANZA_BODEGAS]
3172 if not cand:
3173     # lo dicho no es ningun articulo del catalogo - pero si SI es el
3174     # nombre de una bodega (p. ej. alguien dice "restaurante" en este
3175     # buscador de ingredientes por error, cuando quería ir a Conteo a
3176     # abrirla), vale la pena decirlo en vez de un "no encuentre" a
3177     # secas que deja a la persona sin saber por que. "restaurante" a
3178     # secas encaja en mas de una bodega real - ahí no hay una sola
3179     # que prellenar, pero igual vale la pena decir que existen y
3180     # mandar a Conteo a terminar de decir cual.
3181     with Sesion() as s:
3182         ids_permitidos = _ids_permitidos_para_buscar(s, u)
3183         candidatas = buscar_bodegas_candidatas(s, q, ids_permitidos)
3184         if ids_permitidos is not None:
3185             # buscar_bodegas_candidatas() no restringe una coincidencia
3186             # EXACTA (para que /abrir pueda decir "existe pero no es
3187             # suya" en vez de "no existe") - aqui, en cambio, esto es
3188             # solo una sugerencia informativa sin ese motivo: no hay
3189             # por qué nombrarle a un auxiliar una bodega ajena.
3190             candidatas = [c for c in candidatas if c.id in ids_permitidos]
3191         if len(candidatas) == 1:
3192             b = candidatas[0]
3193             return {
3194                 "resumen": (f"No encuentro ese artículo, pero {b.nombre_oficial.title()} "
3195                             "sí es una bodega - vea su detalle para abrirla."),
3196                 "bodegas": [], "sugerencia_bodega": b.nombre_oficial,
3197                 "sugerencia_bodega_id": b.id,
3198             }
3199         if candidatas:
3200             nombres = ", ".join(c.nombre_oficial.title() for c in candidatas[:4])
3201             return {
3202                 "resumen": (f"No encuentro ese artículo. Ese nombre coincide con varias "
3203                             f"bodegas ({nombres}) - búsquelas en Bodegas para ver cuál es."),
3204                 "bodegas": [], "sugerencia_bodega": None, "sugerencia_bodega_id": None,
3205             }
3206     # Ni articulo ni bodega: puede ser una pregunta suelta ("¿cuántas
3207     # bodegas están pendientes?") o una orden de navegar ("llévame a
3208     # reportes") - un solo buscador para todo, en vez de dos cuadros
3209     # de búsqueda separados (uno para ingredientes, otro para
3210     # cualquier otra cosa) que además terminaban dando respuestas
3211     # distintas para lo mismo.
3212     r = _resolver_asistente("bodegas", q, u)
3213     return {
3214         "resumen": r.get("respuesta_hablada", ""),
3215         "bodegas": [], "sugerencia_bodega": None, "sugerencia_bodega_id": None,
3216         "accion": r.get("accion"), "destino": r.get("destino"),
3217         "pestana": r.get("pestana"), "bodega": r.get("bodega"),
3218     }
3219 # si la persona ya eligio una de las alternativas, usa esa; si no, la mejor
3220 a = next((c for c in cand if c["codigo"] == codigo), None) or cand[0]
3221 hoy = ahora().date()
3222 with Sesion() as s:
3223     filas = s.query(StockSistema).filter_by(articulo_codigo=a["codigo"]).all()
3224     total = sum(f.cantidad_sd or 0 for f in filas)
3225     det = []
3226     for f in filas:
3227         b = s.get(Bodega, f.bodega_id)
3228         ultima = (s.query(Conteo).join(SesionConteo)
3229                 .filter(SesionConteo.bodega_id == f.bodega_id,
3230                         Conteo.articulo_codigo == a["codigo"],
3231                         Conteo.estado == "confirmado")
3232                 .order_by(Conteo.id.desc()).first())
3233         if ultima:
3234             ultima_toma = (f"hoy {ultima.creado.strftime('%H:%M')}")
3235             if ultima.creado.date() == hoy
3236             else ultima.creado.strftime("%d %b").lower())
3237         elif b and b.estado == "en conteo":
3238             ultima_toma = "en conteo"
3239         else:
3240             ultima_toma = "sin toma"
3241         det.append({"bodega": b.nombre_oficial if b else "?",
3242                   "cantidad": f.cantidad_sd, "estado": b.estado if b else "?",
3243                   "ultima_toma": ultima_toma})
3244     # consumo real del ultimo mes, sobre servicios ya legalizados

```

```

3245     hace_30 = ahora() - timedelta(days=30)
3246     lineas_mes = (s.query(LineaServicio)
3247         .filter(LineaServicio.articulo_codigo == a["codigo"],
3248             LineaServicio.estado == "legalizado",
3249             LineaServicio.creado >= hace_30).all())
3250     consumo_mes = sum(l.usado or 0 for l in lineas_mes)
3251     servicios_mes = len(lineas_mes)
3252     cobertura_dias = round(total / (consumo_mes / 30)) if consumo_mes > 0 and total > 0 else None
3253     # otros articulos parecidos y CERCANOS en puntaje: un top score alto no
3254     # basta si el segundo esta empatado o casi (ARROZ y ARROZ BASMATI
3255     # empatan al 100 cuando solo se dice "arroz"); ahi es donde se confunde
3256     # el producto sin que la persona se de cuenta.
3257     alternativas = [c for c in cand if c["codigo"] != a["codigo"]
3258         and (a["confianza"] - c["confianza"]) < 15][:2]
3259     ambiguo = bool(alternativas)
3260     resumen = f"{a['nombre']}: {total:g} {a['unidad']} en {len(filas)} bodegas."
3261     if ambiguo:
3262         resumen += (f" Encontré artículos parecidos ({', '.join(c['nombre'] for c in alternativas)); "
3263             "confirme cuál necesita si no es este.")
3264     return {"articulo": a["nombre"], "codigo": a["codigo"], "unidad": a["unidad"],
3265         "total": total, "bodegas": det, "resumen": resumen,
3266         "ambiguo": ambiguo, "alternativas": alternativas,
3267         "consumo_mes": round(consumo_mes, 2), "servicios_mes": servicios_mes,
3268         "cobertura_dias": cobertura_dias}
3269
3270
3271 @app.get("/api/articulos/{codigo}/movimientos")
3272 def movimientos_articulo(codigo: str, u: Usuario = Depends(usuario_actual)):
3273     """«Ver movimientos»: los ultimos conteos confirmados de este articulo,
3274     en cualquier bodega, con quien y cuando."""
3275     with Sesion() as s:
3276         conteos = (s.query(Conteo).filter_by(articulo_codigo=codigo, estado="confirmado")
3277             .order_by(Conteo.id.desc()).limit(20).all())
3278         salida = []
3279         for c in conteos:
3280             ses = s.get(SesionConteo, c.sesion_id)
3281             b = s.get(Bodega, ses.bodega_id) if ses else None
3282             persona = s.get(Usuario, ses.usuario_id) if ses else None
3283             salida.append({"hora": c.creado.strftime("%Y-%m-%d %H:%M"),
3284                 "bodega": b.nombre_oficial if b else "?",
3285                 "persona": persona.nombre if persona else "?",
3286                 "cantidad": c.cantidad, "unidad": c.unidad})
3287     return salida
3288
3289
3290 @app.get("/api/articulos/{codigo}/en-recetas")
3291 def articulo_en_recetas(codigo: str, u: Usuario = Depends(usuario_actual)):
3292     """«Comparar con la receta»: en que recetas aparece este articulo y
3293     cuanto pide por porcion."""
3294     with Sesion() as s:
3295         ings = s.query(RecetaIngrediente).filter_by(articulo_codigo=codigo).all()
3296         salida = []
3297         for ing in ings:
3298             r = s.get(Receta, ing.receta_id)
3299             if r:
3300                 salida.append({"receta": r.nombre, "por_porcion": ing.cantidad_por_porcion,
3301                     "rendimiento": r.rendimiento})
3302     return salida
3303
3304
3305 @app.post("/api/reportes")
3306 def reporte(formato: str = "xlsx", u: Usuario = Depends(requiere_perfil("auditor"))):
3307     ruta, filas, vista_previa = reportes.consolidado(formato)
3308     registrar(u, "REPORTE", f"Consolidado generado: {ruta} ({filas} filas)")
3309     return {"archivo": ruta, "filas": filas, "vista_previa": vista_previa}
3310
3311
3312 @app.post("/api/reportes/diferencias")
3313 def reporte_diferencias(formato: str = "xlsx", u: Usuario = Depends(requiere_perfil("auditor"))):
3314     ruta, filas, bodegas_con_descuadre = reportes.diferencias_archivo(formato)
3315     registrar(u, "REPORTE",
3316         f"Diferencias por bodega exportado: {ruta} ({filas} filas, "
3317         f"{bodegas_con_descuadre} bodegas)")
3318     return {"archivo": ruta, "filas": filas, "bodegas_con_descuadre": bodegas_con_descuadre}
3319
3320
3321 @app.post("/api/bodegas/exportar-estado")
3322 def exportar_estado_bodegas(formato: str = "xlsx", u: Usuario = Depends(requiere_perfil("auditor"))):
3323     ruta = reportes.estado_bodegas(formato)
3324     registrar(u, "REPORTE", f"Estado de bodegas exportado: {ruta}")
3325     return {"archivo": ruta}
3326
3327
3328 def _parsear_archivo_reporte(t):
3329     """«Convierte una traza cruda de tipo REPORTE en la tarjeta de archivo
3330     que muestra Reportes.jsx - sin esto, cada archivo generado se perdía

```

```

3331     en cuanto se salia de la pantalla (no habia historial real)."""
3332     d = t.detalle
3333     if ": " not in d or "reportes/" not in d:
3334         return None
3335     _, resto = d.split(": ", 1)
3336     archivo = resto.split(" ")[0].strip()
3337     filas = None
3338     if "(" in resto and "filas" in resto:
3339         try:
3340             filas = int(resto.split("(")[1].split(" filas")[0])
3341         except (ValueError, IndexError):
3342             filas = None
3343     if d.startswith("Consolidado generado"):
3344         titulo, subtitulo = "Consolidado para My Inventory", "Listo para carga"
3345     elif d.startswith("Diferencias por bodega"):
3346         titulo = "Diferencias por bodega"
3347         bodegas_txt = resto.split(", ")[-1].replace(" bodegas", "").strip() if "bodegas" in resto else None
3348         subtitulo = f"{bodegas_txt} bodegas con descuadre" if bodegas_txt else "Exportado"
3349     elif d.startswith("Estado de bodegas"):
3350         titulo, subtitulo = "Estado del tablero", "Exportado"
3351     elif d.startswith("Detalle de bodega"):
3352         # el nombre de la bodega (no solo "Detalle de bodega" a secas) va
3353         # en la clave de deduplicacion mas abajo: sin esto, exportar el
3354         # detalle de una bodega distinta hacia desaparecer de "recientes"
3355         # el de la anterior, como si nunca se hubiera generado.
3356         titulo = "Detalle de bodega"
3357         nombre_bodega = d.split("Detalle de bodega ", 1)[1].split(" exportado:")[0].strip()
3358         subtitulo = nombre_bodega.title() if nombre_bodega else "Exportado"
3359     elif d.startswith("Análisis de consumo"):
3360         titulo, subtitulo = "Análisis de consumo", "Exportado"
3361     elif d.startswith("Registro de trazabilidad"):
3362         titulo, subtitulo = "Registro de trazabilidad", "Exportado"
3363     else:
3364         titulo, subtitulo = "Reporte", "Exportado"
3365     # clave de deduplicacion: para casi todos los tipos es el titulo (solo
3366     # importa el mas reciente); "Detalle de bodega" es la excepcion, porque
3367     # el mismo titulo generico cubre un archivo distinto por cada bodega.
3368     clave = f"{titulo}:{subtitulo}" if titulo == "Detalle de bodega" else titulo
3369     return {"titulo": titulo, "subtitulo": subtitulo, "archivo": archivo, "filas": filas,
3370            "formato": archivo.rsplit(".", 1)[-1].upper() if "." in archivo else "?",
3371            "hora": t.creado.strftime("%H:%M"), "persona": (t.persona or "").title(),
3372            "clave": clave}
3373
3374
3375 @app.get("/api/reportes/recientes")
3376 def reportes_recientes(u: Usuario = Depends(requiere_perfil("auditor"))):
3377     """El historial real de archivos generados (via pantalla o por voz),
3378     leído de la trazabilidad - para que la lista sobreviva a salir de la
3379     pantalla o recargar, en vez de vivir solo en el estado del componente.
3380     Un solo archivo por tipo (el mas reciente): generar el mismo reporte
3381     varias veces en el dia no debe ir apilando copias viejas - lo que
3382     importa es el ultimo, los anteriores ya quedaron obsoletos apenas se
3383     genero uno nuevo del mismo tipo."""
3384     with Sesion() as s:
3385         trazas = (s.query(Traza).filter_by(accion="REPORTE")
3386                  .order_by(Traza.id.desc()).limit(40).all())
3387     vistos = set()
3388     salida = []
3389     for t in trazas:
3390         x = _parsear_archivo_reporte(t)
3391         if not x or x["clave"] in vistos:
3392             continue
3393         vistos.add(x["clave"])
3394         salida.append(x)
3395     return salida[:10]
3396
3397
3398 @app.post("/api/bodegas/{bodega_id}/exportar-detalle")
3399 def exportar_detalle_bodega(bodega_id: int, formato: str = "xlsx",
3400                             u: Usuario = Depends(requiere_perfil("auditor"))):
3401     ruta = reportes.detalle_bodega(bodega_id, formato)
3402     with Sesion() as s:
3403         b = s.get(Bodega, bodega_id)
3404         nombre_bodega = b.nombre_oficial if b else str(bodega_id)
3405         # el nombre (no solo el id) queda en el propio mensaje para que
3406         # _parsear_archivo_reporte pueda mostrar de cual bodega es la tarjeta,
3407         # y distinguir el archivo de esta bodega del de otra en "recientes".
3408         registrar(u, "REPORTE", f"Detalle de bodega {nombre_bodega} exportado: {ruta}")
3409     return {"archivo": ruta}
3410
3411
3412 @app.get("/api/reportes/descargar")
3413 def descargar(archivo: str, u: Usuario = Depends(requiere_perfil("auditor"))):
3414     if ".." in archivo or not archivo.startswith("reportes/"):
3415         raise HTTPException(400, "Ruta no permitida.")
3416     from servicios.archivos import leer_bytes

```

```

3417 contenido = leer_bytes(archivo)
3418 if contenido is None:
3419     raise HTTPException(404, "Archivo no encontrado.")
3420 tipo = "text/csv" if archivo.endswith(".csv") else (
3421     "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet")
3422 return Response(content=contenido, media_type=tipo, headers={
3423     "Content-Disposition": f'attachment; filename="{os.path.basename(archivo)}"'})
3424
3425
3426 @app.get("/api/reportes/vista-previa")
3427 def vista_previa_reporte(archivo: str, u: Usuario = Depends(requiere_perfil("auditor"))):
3428     """Para poder ver el contenido de cualquier archivo ya generado (no solo
3429     el que se acaba de crear) con solo dar clic en su tarjeta, sin tener
3430     que descargarlo primero. Lee el archivo tal cual quedo guardado (de la
3431     base, no de disco - ver servicios/archivos.py), así que la vista previa
3432     de un reporte viejo muestra lo que ese reporte realmente tenia, no el
3433     estado actual de la base, y sigue funcionando despues de un redeploy."""
3434     if ".." in archivo or not archivo.startswith("reportes/"):
3435         raise HTTPException(400, "Ruta no permitida.")
3436     import io
3437     import pandas as pd
3438     from servicios.archivos import leer_bytes
3439     contenido = leer_bytes(archivo)
3440     if contenido is None:
3441         raise HTTPException(404, "Archivo no encontrado.")
3442     buf = io.BytesIO(contenido)
3443     df = pd.read_csv(buf) if archivo.endswith(".csv") else pd.read_excel(buf)
3444     df = df.fillna(0)
3445     return {"filas": df.head(8).to_dict("records"), "total": len(df)}
3446
3447
3448 # ----- los tres momentos -----
3449 class PedidoIn(BaseModel):
3450     plato: str
3451     porciones: int = Field(gt=0)
3452     bodega_id: int = 1
3453
3454
3455 @app.post("/api/pedidos/calcular")
3456 def api_calcular(p: PedidoIn, u: Usuario = Depends(usuario_actual)):
3457     return calcular_pedido(p.plato, p.porciones, p.bodega_id)
3458
3459
3460 class EnviarPedidoIn(BaseModel):
3461     plato: str
3462     porciones: int = Field(gt=0)
3463     bodega_id: int
3464     servicio_id: int = 1
3465
3466
3467 # protege el bloque de verificar-duplicado + insertar: sin esto, dos
3468 # peticiones concurrentes (un reintento de red, un doble toque en la
3469 # tableta) podian pasar juntas la verificacion de "ya_existe" antes de que
3470 # cualquiera terminara de guardar, y las dos quedaban registradas como
3471 # pedidos reales en vez de que la segunda se detectara como duplicada.
3472 # Alcanza con un lock en memoria porque el servicio corre en un solo
3473 # proceso (uvicorn sin --workers, ver render.yaml); con varios procesos
3474 # haría falta una constraint a nivel de base de datos.
3475 _lock_enviar_pedido = threading.Lock()
3476
3477
3478 @app.post("/api/pedidos/enviar")
3479 def api_enviar(datos: EnviarPedidoIn, u: Usuario = Depends(usuario_actual)):
3480     # las lineas ya no se reciben del cliente: se vuelven a calcular aqui
3481     # mismo contra la receta y el stock en vivo, igual que hace "Calcular
3482     # el pedido". Confiar en las cantidades que mandara el navegador
3483     # permitia pedir cualquier cosa (cualquier codigo de articulo, en
3484     # cualquier cantidad) para cualquier plato, incluso uno sin receta.
3485     calculo = calcular_pedido(datos.plato, datos.porciones, datos.bodega_id)
3486     if not calculo["receta_encontrada"]:
3487         raise HTTPException(404, f"No encontré una receta para «{datos.plato}».")
3488     lineas_a_pedir = [l for l in calculo["lineas"] if l["falta"] > 0]
3489     if not lineas_a_pedir:
3490         raise HTTPException(400, "No hay nada que pedir: ya tiene todo en la bodega.")
3491
3492     with _lock_enviar_pedido:
3493         with Sesion() as s:
3494             # si el mismo pedido (servicio + plato + porciones) ya quedo
3495             # abierto o esperando aprobacion, no lo duplica: cubre un doble
3496             # clic, un doble disparo por voz, o un reintento tras una
3497             # desconexion que si alcanza a llegar la primera vez.
3498             ya_existe = (s.query(LineaServicio)
3499                 .filter_by(servicio_id=datos.servicio_id, plato=datos.plato,
3500                     porciones=datos.porciones)
3501                 .filter(LineaServicio.estado.in_([ "abierto", "pendiente_aprobacion" ]))
3502                 .first())

```

```

3503         if ya_existe:
3504             return {"ok": True, "duplicado": True}
3505         # un auxiliar pide, pero es el administrador quien de verdad autoriza
3506         # que salga del almacen - igual que ya pasa con productos y bodegas
3507         # creados en plena toma. El administrador autoriza su propio pedido
3508         # de una vez: no tiene sentido pedirse permiso a si mismo.
3509         estado_inicial = "abierto" if u.perfil == "auditor" else "pendiente_aprobacion"
3510         # el sufijo evita que dos pedidos distintos creados en el mismo
3511         # segundo (dos auxiliares pidiendo casi a la vez) terminen con
3512         # el mismo numero_pedido - eso los mezclaba en una sola tarjeta
3513         # de aprobacion en Auditoria, con los items de ambos revueltos.
3514         numero = f"PED-{ahora().strftime('%Y%m%d-%H%M%S')}-{secrets.token_hex(2).upper()}"
3515         n = 0
3516         items = []
3517         for l in lineas_a_pedir:
3518             s.add(LineaServicio(servicio_id=datos.servicio_id,
3519                                articulo_codigo=l["codigo"],
3520                                nombre=l["nombre"], pedido=l["falta"],
3521                                plato=datos.plato, porciones=datos.porciones,
3522                                estado=estado_inicial, bodega_id=datos.bodega_id,
3523                                creado_por_id=u.id, numero_pedido=numero))
3524             items.append({"nombre": l["nombre"], "cantidad": l["falta"],
3525                           "unidad": l.get("unidad", "")})
3526             n += 1
3527         s.commit()
3528         b = s.get(Bodega, datos.bodega_id)
3529         bodega_nombre = b.nombre_oficial if b else None
3530         if estado_inicial == "pendiente_aprobacion":
3531             registrar(u, "PEDIDO", f"Pedido {numero} enviado, pendiente de aprobacion ({n} lineas)")
3532         else:
3533             registrar(u, "PEDIDO", f"Pedido {numero} enviado y aprobado ({n} lineas)")
3534         return {"ok": True, "numero_pedido": numero, "estado": estado_inicial,
3535                "hora": ahora().strftime("%H:%M"),
3536                "bodega": bodega_nombre, "persona": u.nombre,
3537                "items": items, "total_lineas": n}
3538
3539
3540 @app.get("/api/pedidos/{numero_pedido}/estado")
3541 def estado_pedido(numero_pedido: str, u: Usuario = Depends(usuario_actual)):
3542     """Para que quien pidió pueda enterarse si un administrador ya lo
3543     aprobó o rechazó, aunque haya salido de Pedidos y vuelto - la pantalla
3544     no se refresca sola."""
3545     with Sesion() as s:
3546         fila = s.query(LineaServicio).filter_by(numero_pedido=numero_pedido).first()
3547         if fila is None:
3548             raise HTTPException(404, "Ese pedido no existe.")
3549         if fila.creado_por_id != u.id and u.perfil != "auditor":
3550             raise HTTPException(403, "Ese pedido no es suyo.")
3551         return {"numero_pedido": numero_pedido, "estado": fila.estado}
3552
3553
3554 @app.get("/api/pedidos/pendientes")
3555 def pedidos_pendientes(u: Usuario = Depends(requiere_perfil("auditor"))):
3556     """Pedidos de auxiliares esperando la firma del administrador antes de
3557     contar como enviados de verdad al almacen - el pedir no se detiene
3558     (queda registrado de una vez), pero salir del almacen si depende de
3559     esta aprobacion, igual que con productos y bodegas nuevas."""
3560     with Sesion() as s:
3561         filas = (s.query(LineaServicio)
3562                  .filter_by(estado="pendiente_aprobacion")
3563                  .order_by(LineaServicio.creado.desc()).all())
3564         grupos = {}
3565         for f in filas:
3566             g = grupos.setdefault(f.numero_pedido, {
3567                 "numero_pedido": f.numero_pedido, "plato": f.plato,
3568                 "porciones": f.porciones, "hora": f.creado.strftime("%H:%M"),
3569                 "bodega_id": f.bodega_id, "creado_por_id": f.creado_por_id,
3570                 "items": [],
3571             })
3572             g["items"].append({"nombre": f.nombre, "cantidad": f.pedido})
3573         out = []
3574         for g in grupos.values():
3575             b = s.get(Bodega, g["bodega_id"]) if g["bodega_id"] else None
3576             persona = s.get(Usuario, g["creado_por_id"]) if g["creado_por_id"] else None
3577             out.append(**g, "bodega": b.nombre_oficial if b else None,
3578                        "persona": persona.nombre if persona else "-")
3579         out.sort(key=lambda x: x["numero_pedido"], reverse=True)
3580         return out
3581
3582
3583 class ResolverPedidoIn(BaseModel):
3584     aprobar: bool
3585
3586
3587 @app.post("/api/pedidos/{numero_pedido}/resolver")
3588 def resolver_pedido(numero_pedido: str, datos: ResolverPedidoIn,

```

```

3589         u: Usuario = Depends(requiere_perfil("auditor"))):
3590     with Sesion() as s:
3591         filas = (s.query(LineaServicio)
3592             .filter_by(numero_pedido=numero_pedido, estado="pendiente_aprobacion").all())
3593         if not filas:
3594             raise HTTPException(404, "Pedido no encontrado o ya resuelto.")
3595         nuevo_estado = "abierto" if datos.aprobar else "rechazado"
3596         plato = filas[0].plato
3597         for f in filas:
3598             f.estado = nuevo_estado
3599         s.commit()
3600         accion = "aprobado" if datos.aprobar else "rechazado"
3601         registrar(u, "APROBACION", f"Pedido {numero_pedido} ({plato}) {accion}",
3602             "ok" if datos.aprobar else "alerta")
3603         return {"ok": True, "estado": nuevo_estado}
3604
3605
3606 @app.get("/api/legalizacion/{servicio_id}")
3607 def api_legalizacion(servicio_id: int, u: Usuario = Depends(usuario_actual)):
3608     return comparar_legalizacion(servicio_id)
3609
3610
3611 @app.post("/api/legalizacion/confirmar")
3612 def api_confirmar(body: dict, u: Usuario = Depends(usuario_actual)):
3613     sid = body.get("servicio_id", 1)
3614     # opcional: permite mandar lo usado junto con la confirmación en vez de
3615     # exigir un "/legalizacion/ajustar" por cada insumo antes de cerrar.
3616     usos = {item.get("codigo"): item.get("usado") for item in body.get("usos", [])}
3617     with Sesion() as s:
3618         # solo el pedido que de verdad se le mostro a la persona (el mas
3619         # reciente) - si hubiera otro pedido distinto todavia abierto para
3620         # este servicio, sigue esperando su propio turno, no se legaliza
3621         # de arrastre solo porque comparte servicio_id.
3622         for l in _filas_a_legalizar(s, sid):
3623             if l.articulo_codigo in usos and usos[l.articulo_codigo] is not None:
3624                 l.usado = usos[l.articulo_codigo]
3625             dif = (l.usado or 0) - (l.pedido or 0)
3626             if dif < 0:
3627                 # sobrante: vuelve a bodega
3628                 st = s.query(StockSistema).filter_by(
3629                     articulo_codigo=l.articulo_codigo, bodega_id=l.bodega_id).first()
3630                 if st:
3631                     st.cantidad_sd = (st.cantidad_sd or 0) + abs(dif)
3632             l.estado = "legalizado"
3633         s.commit()
3634         registrar(u, "LEGALIZACION", f"Servicio {sid} legalizado", "ok")
3635         return {"ok": True}
3636
3637
3638 @app.post("/api/legalizacion/ajustar")
3639 def api_ajustar_legalizacion(body: dict, u: Usuario = Depends(usuario_actual)):
3640     """«Ajustar por voz»: corrige lo realmente usado de un insumo antes de
3641     confirmar, dictando en vez de editar celda por celda."""
3642     from agente.cerebro import pensar
3643     sid = body.get("servicio_id", 1)
3644     texto = (body.get("texto") or "").strip()
3645     if not texto:
3646         raise HTTPException(400, "Dígame el insumo y la cantidad.")
3647     turno = pensar("Esta corrigiendo cuanto se uso de un insumo en un "
3648         "servicio de comidas ya cerrado, antes de legalizarlo.", texto)
3649     articulo_texto = (turno.get("articulo_texto") or texto).upper()
3650     cantidad = turno.get("cantidad")
3651     if cantidad is None:
3652         raise HTTPException(400, "No entendí la cantidad. ¿Me la repite?")
3653     palabras = [p for p in articulo_texto.split() if len(p) >= 3]
3654     with Sesion() as s:
3655         # mismo alcance que comparar_legalizacion/confirmar: solo el
3656         # pedido mas reciente, no cualquier otro que siga abierto para
3657         # este servicio.
3658         filas = _filas_a_legalizar(s, sid)
3659         objetivo = next((f for f in filas
3660             if any(p in f.nombre.upper() for p in palabras)), None)
3661         if objetivo is None:
3662             raise HTTPException(404, "No encontré ese insumo en este servicio.")
3663         objetivo.usado = cantidad
3664         s.commit()
3665         nombre = objetivo.nombre
3666         registrar(u, "LEGALIZACION", f"Ajuste por voz: {nombre} -> {cantidad:g}", "alerta")
3667         return comparar_legalizacion(sid)
3668
3669
3670 @app.get("/api/analisis/consumo")
3671 def api_analisis(dias: int = 30, u: Usuario = Depends(requiere_perfil("auditor"))):
3672     return analisis_consumo(dias)
3673
3674
3675 @app.post("/api/analisis/exportar")

```

```

3675 def exportar_analisis(formato: str = "xlsx", dias: int = 30,
3676                       u: Usuario = Depends(requiere_perfil("auditor"))):
3677     import pandas as pd
3678     from servicios.archivos import guardar_df
3679     datos = analisis_consumo(dias)
3680     df = pd.DataFrame(datos["subutilizados"])
3681     marca = ahora().strftime("%Y%m%d_%H%M")
3682     ruta = f"reportes/analisis_consumo_{marca}.{formato}"
3683     guardar_df(ruta, df, formato)
3684     registrar(u, "REPORTE", f"Análisis de consumo exportado: {ruta} ({len(df)} filas)")
3685     return {"archivo": ruta}
3686
3687
3688 @app.get("/api/pedidos/receta")
3689 def api_receta(plato: str, u: Usuario = Depends(usuario_actual)):
3690     return detalle_receta(plato)
3691
3692
3693 # ----- recetas: catálogo administrado -----
3694 # CuentaVoz si gestiona las recetas: crearlas, editarlas y borrarlas queda a
3695 # cargo del administrador (perfil auditor), igual que la gestión de usuarios
3696 # o de bodegas. El auxiliar solo las consulta al armar un pedido.
3697 class IngredienteIn(BaseModel):
3698     articulo_codigo: str
3699     cantidad_por_porcion: float
3700
3701
3702 class RecetaIn(BaseModel):
3703     nombre: str
3704     rendimiento: int = 1
3705     preparacion: str = ""
3706     ingredientes: list[IngredienteIn]
3707
3708
3709 @app.get("/api/recetas")
3710 def listar_recetas(u: Usuario = Depends(usuario_actual)):
3711     with Sesion() as s:
3712         recetas = s.query(Receta).order_by(Receta.nombre).all()
3713         return [{"id": r.id, "nombre": r.nombre, "rendimiento": r.rendimiento,
3714                 "ingredientes": s.query(RecetaIngrediente)
3715                     .filter_by(receta_id=r.id).count()}
3716                 for r in recetas]
3717
3718
3719 @app.get("/api/recetas/{receta_id}")
3720 def obtener_receta(receta_id: int, u: Usuario = Depends(usuario_actual)):
3721     with Sesion() as s:
3722         r = s.get(Receta, receta_id)
3723         if r is None:
3724             raise HTTPException(404, "Receta no encontrada.")
3725         ings = s.query(RecetaIngrediente).filter_by(receta_id=r.id).all()
3726         lineas = []
3727         for i in ings:
3728             art = s.get(Articulo, i.articulo_codigo)
3729             lineas.append({"articulo_codigo": i.articulo_codigo,
3730                           "nombre": art.nombre_oficial if art else i.articulo_codigo,
3731                           "unidad": art.unidad_medida if art else "",
3732                           "cantidad_por_porcion": i.cantidad_por_porcion})
3733         return {"id": r.id, "nombre": r.nombre, "rendimiento": r.rendimiento,
3734               "preparacion": r.preparacion or "", "lineas": lineas}
3735
3736
3737 def _validar_ingredientes(s, ingredientes):
3738     if not ingredientes:
3739         raise HTTPException(400, "La receta necesita al menos un ingrediente.")
3740     vistos = set()
3741     for ing in ingredientes:
3742         art = s.get(Articulo, ing.articulo_codigo)
3743         if art is None:
3744             raise HTTPException(400, f"El artículo {ing.articulo_codigo} no existe en el catálogo.")
3745         if ing.cantidad_por_porcion <= 0:
3746             raise HTTPException(400, "La cantidad por porción debe ser mayor que cero.")
3747         # el mismo artículo en dos líneas de la receta no se suma solo: el
3748         # editor no impide elegirlo dos veces por error, y calcular_pedido
3749         # terminaba mostrando dos filas separadas del mismo insumo (con el
3750         # mismo código, lo que además rompe la key de React en la tabla) en
3751         # vez de una sola cantidad combinada.
3752         if ing.articulo_codigo in vistos:
3753             raise HTTPException(400, f"«{art.nombre_oficial}» está repetido en la receta: "
3754                               "combine las cantidades en una sola línea.")
3755         vistos.add(ing.articulo_codigo)
3756
3757
3758 @app.post("/api/recetas")
3759 def crear_receta(datos: RecetaIn, u: Usuario = Depends(requiere_perfil("auditor"))):
3760     with Sesion() as s:

```

```

3761     _validar_ingredientes(s, datos.ingredientes)
3762     if s.query(Receta).filter(Receta.nombre.ilike(datos.nombre.strip())).first():
3763         raise HTTPException(400, "Ya existe una receta con ese nombre.")
3764     r = Receta(nombre=datos.nombre.strip(), rendimiento=datos.rendimiento,
3765               preparacion=datos.preparacion.strip())
3766     s.add(r)
3767     s.flush()
3768     for ing in datos.ingredientes:
3769         s.add(RecetaIngrediente(receta_id=r.id, articulo_codigo=ing.articulo_codigo,
3770                                cantidad_por_porcion=ing.cantidad_por_porcion))
3771     s.commit()
3772     rid, nombre = r.id, r.nombre
3773     registrar(u, "RECETA", f"Receta creada: {nombre} ({len(datos.ingredientes)} ingredientes)", "ok")
3774     return {"ok": True, "id": rid}
3775
3776
3777 @app.put("/api/recetas/{receta_id}")
3778 def editar_receta(receta_id: int, datos: RecetaIn,
3779                  u: Usuario = Depends(requiere_perfil("auditor"))):
3780     with Sesion() as s:
3781         r = s.get(Receta, receta_id)
3782         if r is None:
3783             raise HTTPException(404, "Receta no encontrada.")
3784         _validar_ingredientes(s, datos.ingredientes)
3785         otra = s.query(Receta).filter(Receta.nombre.ilike(datos.nombre.strip()),
3786                                       Receta.id != receta_id).first()
3787         if otra:
3788             raise HTTPException(400, "Ya existe otra receta con ese nombre.")
3789         r.nombre = datos.nombre.strip()
3790         r.rendimiento = datos.rendimiento
3791         r.preparacion = datos.preparacion.strip()
3792         s.query(RecetaIngrediente).filter_by(receta_id=receta_id).delete()
3793         for ing in datos.ingredientes:
3794             s.add(RecetaIngrediente(receta_id=receta_id, articulo_codigo=ing.articulo_codigo,
3795                                    cantidad_por_porcion=ing.cantidad_por_porcion))
3796         s.commit()
3797         nombre = r.nombre
3798         registrar(u, "RECETA", f"Receta editada: {nombre} ({len(datos.ingredientes)} ingredientes)", "ok")
3799         return {"ok": True}
3800
3801
3802 @app.delete("/api/recetas/{receta_id}")
3803 def eliminar_receta(receta_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
3804     with Sesion() as s:
3805         r = s.get(Receta, receta_id)
3806         if r is None:
3807             raise HTTPException(404, "Receta no encontrada.")
3808         nombre = r.nombre
3809         s.query(RecetaIngrediente).filter_by(receta_id=receta_id).delete()
3810         s.delete(r)
3811         s.commit()
3812         registrar(u, "RECETA", f"Receta eliminada: {nombre}", "ok")
3813         return {"ok": True}
3814
3815
3816 @app.get("/api/reportes/diferencias-por-bodega")
3817 def api_diferencias_por_bodega(u: Usuario = Depends(requiere_perfil("auditor"))):
3818     return analitica.diferencias_por_bodega(limite=50)
3819
3820
3821 @app.get("/api/panel/resumen")
3822 def api_panel_resumen(u: Usuario = Depends(requiere_perfil("auditor"))):
3823     return analitica.resumen_ejecutivo()
3824
3825
3826 @app.get("/api/panel/alertas")
3827 def api_panel_alertas(u: Usuario = Depends(requiere_perfil("auditor"))):
3828     return analitica.resumen_alertas_panel()
3829
3830
3831 # ----- alertas, usuarios y trazas -----
3832 _TIPO_ALERTA_TITULO = {
3833     "desviacion": "Desviación de cantidad", "negativo": "Cantidad negativa dictada",
3834     "unidad": "Unidad de medida", "inexistente": "Artículo fuera del catálogo",
3835 }
3836
3837
3838 @app.get("/api/alertas")
3839 def ver_alertas(resueltas: int = 0, u: Usuario = Depends(usuario_actual)):
3840     with Sesion() as s:
3841         salida = []
3842         for a in s.query(Alerta).filter_by(resuelta=resueltas).order_by(
3843             Alerta.id.desc()).all():
3844             art = bodega = persona = None
3845             if a.conteo_id:
3846                 c = s.get(Conteo, a.conteo_id)

```

```

3847         if c:
3848             ar = s.get(Articulo, c.articulo_codigo)
3849             art = ar.nombre_oficial if ar else None
3850             ses = s.get(SesionConteo, c.sesion_id)
3851             if ses:
3852                 b = s.get(Bodega, ses.bodega_id)
3853                 bodega = b.nombre_oficial if b else None
3854                 us = s.get(Usuario, ses.usuario_id)
3855                 persona = us.nombre.title() if us else None
3856             salida.append({"id": a.id, "tipo": a.tipo,
3857                           "titulo": _TIPO_ALERTA_TITULO.get(a.tipo, a.tipo),
3858                           "detalle": a.detalle, "articulo": art, "bodega": bodega,
3859                           "persona": persona, "hora": a.creado.strftime("%H:%M")})
3860     return salida
3861
3862
3863 @app.get("/api/alertas/resumen")
3864 def resumen_alertas(u: Usuario = Depends(usuario_actual)):
3865     """Estadísticas reales para la bandeja: abiertas, resueltas hoy y el
3866     tiempo medio real entre que se genera la alerta y se resuelve."""
3867     hoy = ahora().date()
3868     with Sesion() as s:
3869         abiertas = s.query(Alerta).filter_by(resuelta=0).count()
3870         resueltas_hoy = (s.query(Alerta).filter(Alerta.resuelta == 1,
3871                                                  Alerta.resuelto.isnot(None))).all()
3872         resueltas_hoy = [a for a in resueltas_hoy if a.resuelto.date() == hoy]
3873         tiempo_medio_min = None
3874         if resueltas_hoy:
3875             segundos = [(a.resuelto - a.creado).total_seconds() for a in resueltas_hoy]
3876             tiempo_medio_min = round(sum(segundos) / len(segundos) / 60, 1)
3877     return {"abiertas": abiertas, "resueltas_hoy": len(resueltas_hoy),
3878           "tiempo_medio_min": tiempo_medio_min}
3879
3880
3881 @app.post("/api/alertas/{alerta_id}/resolver")
3882 def resolver_alerta(alerta_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
3883     with Sesion() as s:
3884         a = s.get(Alerta, alerta_id)
3885         if a is None:
3886             raise HTTPException(404, "Alerta no encontrada.")
3887         a.resuelta = 1
3888         a.resuelto = ahora()
3889         s.commit()
3890         nombre = None
3891         if a.conteo_id:
3892             c = s.get(Conteo, a.conteo_id)
3893             if c:
3894                 art = s.get(Articulo, c.articulo_codigo)
3895                 nombre = art.nombre_oficial if art else None
3896         detalle = f"Alerta resuelta - {nombre}: {a.detalle}" if nombre else f"Alerta {alerta_id} resuelta"
3897         registrar(u, "ALERTA", detalle, "ok")
3898     return {"ok": True}
3899
3900
3901 @app.get("/api/usuarios")
3902 def listar_usuarios(u: Usuario = Depends(requiere_perfil("auditor"))):
3903     with Sesion() as s:
3904         salida = []
3905         for x in s.query(Usuario).all():
3906             n_bodegas = s.query(AsignacionBodega).filter_by(usuario_id=x.id).count()
3907             salida.append({"id": x.id, "nombre": x.nombre, "perfil": x.perfil,
3908                           "correo": x.correo, "activo": bool(x.activo),
3909                           "bodegas_asignadas": n_bodegas,
3910                           "ultimo_acceso": (x.ultimo_acceso.strftime("%Y-%m-%d %H:%M")
3911                                              if x.ultimo_acceso else None)})
3912     return salida
3913
3914
3915 class CrearUsuarioIn(BaseModel):
3916     nombre: str
3917     perfil: str
3918     correo: str = ""
3919     pin: str | None = None # None: se genera uno temporal, ver mas abajo
3920
3921
3922 @app.post("/api/usuarios")
3923 def crear_usuario(datos: CrearUsuarioIn, u: Usuario = Depends(requiere_perfil("auditor"))):
3924     nombre = datos.nombre.strip().lower()
3925     if datos.perfil not in ("auxiliar", "auditor"):
3926         raise HTTPException(400, "El perfil debe ser auxiliar o auditor.")
3927     # Cognito exige un correo real para crear la cuenta - antes era
3928     # opcional porque solo se guardaba en esta base; ahora sin correo no
3929     # hay como crear la cuenta de verdad, así que se exige aquí.
3930     correo = (datos.correo or "").strip()
3931     if not correo or "@" not in correo:
3932         raise HTTPException(400, "El correo es obligatorio y debe ser válido.")

```

```

3933 # sin esto, todo usuario creado desde Ajustes (la pantalla nunca pide
3934 # un PIN) quedaba con la misma clave de siempre - la misma que
3935 # aparece publicada en el README para las cuentas de demostración.
3936 # Un temporal aleatorio, distinto cada vez, obliga a que quien lo
3937 # reciba lo cambie por uno propio desde Mi perfil.
3938 pin_generado = None
3939 if datos.pin is None:
3940     pin_generado = secrets.token_urlsafe(6) + "1Aa" # cumple la política de Cognito
3941     pin_para_guardar = pin_generado
3942 else:
3943     pin_para_guardar = datos.pin
3944 if len(pin_para_guardar) < 8:
3945     raise HTTPException(400, "El PIN debe tener al menos 8 caracteres.")
3946 with Sesión() as s:
3947     if s.query(Usuario).filter_by(nombre=nombre).first():
3948         raise HTTPException(409, "Ya existe un usuario con ese nombre.")
3949     nuevo = Usuario(nombre=nombre, perfil=datos.perfil, correo=correo)
3950     s.add(nuevo)
3951     s.commit()
3952     s.refresh(nuevo)
3953     nid = nuevo.id
3954     # código de empleado: para poder ingresar con el además del nombre
3955     nuevo.codigo = f"CS-{48000 + nid}"
3956     s.commit()
3957 if not _crear_usuario_cognito(nombre, correo, pin_para_guardar):
3958     raise HTTPException(502, "La cuenta local se creó, pero no se pudo crear en Cognito. "
3959                             "Revise que el correo no esté ya registrado.")
3960 registrar(u, "USUARIO", f"Usuario {nombre} creado ({datos.perfil})", "ok")
3961 respuesta = {"ok": True, "id": nid}
3962 if pin_generado:
3963     respuesta["pin_temporal"] = pin_generado
3964 return respuesta
3965
3966 @app.put("/api/usuarios/yo")
3967 def editar_perfil(datos: dict, u: Usuario = Depends(usuario_actual)):
3968     """Cada quien edita su propio perfil - va antes de /usuarios/{usuario_id}
3969     a propósito: "yo" no es un entero, y si esta ruta queda después, esa otra
3970     la intercepta primero y revienta con un 422 (el mismo problema que ya
3971     paso con /usuarios/yo/bodegas)."""
3972     correo_nuevo = (datos.get("correo") or "").strip()
3973     # si Cognito no queda enterado del correo nuevo, el código de "olvide
3974     # mi clave" seguiría yendo al correo viejo para siempre - se sincroniza
3975     # ANTES de guardar localmente, para no dejar los dos lados desfasados.
3976     if correo_nuevo and correo_nuevo != u.correo:
3977         if not _actualizar_correo_cognito(u.nombre, correo_nuevo):
3978             raise HTTPException(502, "No se pudo actualizar el correo en este momento.")
3979     with Sesión() as s:
3980         usr = s.get(Usuario, u.id)
3981         for k in ("nombre", "correo", "telefono"):
3982             if k in datos:
3983                 setattr(usr, k, datos[k])
3984         s.commit()
3985     registrar(u, "PERFIL", "Datos personales actualizados")
3986     return {"ok": True}
3987
3988
3989
3990 class EditarUsuarioIn(BaseModel):
3991     correo: str | None = None
3992     perfil: str | None = None
3993     activo: bool | None = None
3994
3995
3996 @app.put("/api/usuarios/{usuario_id}")
3997 def editar_usuario(usuario_id: int, datos: EditarUsuarioIn,
3998                   u: Usuario = Depends(requiere_perfil("auditor"))):
3999     """Editar correo, rol o estado de OTRA persona - no es lo mismo que
4000     /usuarios/yo, que cada quien usa para su propio perfil. No se borra a
4001     nadie (rompería la trazabilidad de sus conteos y firmas pasadas):
4002     desactivar es el equivalente seguro a eliminar, y ya bloquea el
4003     ingreso (ver usuario_actual)."""
4004     if datos.perfil is not None and datos.perfil not in ("auxiliar", "auditor"):
4005         raise HTTPException(400, "El perfil debe ser auxiliar o auditor.")
4006     if usuario_id == u.id and datos.activo is False:
4007         raise HTTPException(400, "No puede desactivar su propia cuenta.")
4008     if usuario_id == u.id and datos.perfil is not None and datos.perfil != u.perfil:
4009         raise HTTPException(400, "No puede cambiar su propio rol.")
4010     with Sesión() as s:
4011         obj = s.get(Usuario, usuario_id)
4012         if obj is None:
4013             raise HTTPException(404, "Usuario no encontrado.")
4014         cambios = []
4015         if datos.correo is not None and datos.correo != obj.correo:
4016             # mismo motivo que en editar_perfil: sin esto Cognito le
4017             # seguiría mandando los códigos de recuperar clave al correo
4018             # viejo de esa persona, sin que nada lo avisara.

```

```

4019         if not _actualizar_correo_cognito(obj.nombre, datos.correo):
4020             raise HTTPException(502, "No se pudo actualizar el correo en este momento.")
4021         obj.correo = datos.correo
4022         cambios.append("correo")
4023     if datos.perfil is not None and datos.perfil != obj.perfil:
4024         obj.perfil = datos.perfil
4025         cambios.append(f"perfil -> {datos.perfil}")
4026     desactivado = False
4027     if datos.activo is not None and bool(datos.activo) != bool(obj.activo):
4028         obj.activo = int(datos.activo)
4029         desactivado = not datos.activo
4030         cambio.append("activo" if datos.activo else "inactivo")
4031     s.commit()
4032     nombre = obj.nombre
4033     if desactivado:
4034         # cierra sus sesiones vivas en Cognito - el access token ya
4035         # emitido sigue valido hasta que vence solo (ver seguridad.py)
4036         try:
4037             _cliente_cognito().admin_user_global_sign_out(
4038                 UserPoolId=COGNITO_USER_POOL_ID, Username=nombre)
4039         except Exception as e:
4040             print(f"[cognito] no se pudo cerrar la sesion de {nombre}: {e}")
4041     if cambios:
4042         registrar(u, "USUARIO", f"{nombre} editado: {'', '.join(cambios)}", "ok")
4043     return {"ok": True}
4044
4045
4046 @app.delete("/api/usuarios/{usuario_id}")
4047 def eliminar_usuario(usuario_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
4048     """Borra de verdad a alguien (no solo desactivar) - para limpiar cuentas
4049     de prueba de un ambiente antes de una demo, sin dejarlas visibles en
4050     Gestión de usuarios. A propósito NO es lo que hace editar_usuario()
4051     de arriba para el uso normal (esa sigue prefiriendo desactivar, por la
4052     misma razón documentada ahí: no romper la trazabilidad de conteos y
4053     firmas reales). Aquí sí se borra la cuenta, pero la Traza (registro
4054     inmutable) NUNCA se toca como fila - solo se le suelta la referencia al
4055     usuario (usuario_id queda en NULL; el nombre en "persona" ya es texto
4056     aparte) para que el historial real de esa persona, si lo hubo, siga
4057     intacto y legible después de borrar la cuenta."""
4058     if usuario_id == u.id:
4059         raise HTTPException(400, "No puede eliminar su propia cuenta.")
4060     with Sesión() as s:
4061         obj = s.get(Usuario, usuario_id)
4062         if obj is None:
4063             raise HTTPException(404, "Usuario no encontrado.")
4064         nombre = obj.nombre
4065         # sesiones de conteo/auditoria de esta persona: primero los Conteo
4066         # de cada sesion (y las Alertas que esos Conteo hayan generado),
4067         # despues la sesion misma - en ese orden, por las llaves foraneas.
4068         sesiones = s.query(SesionConteo).filter_by(usuario_id=usuario_id).all()
4069         for ses in sesiones:
4070             conteos = s.query(Conteo).filter_by(sesion_id=ses.id).all()
4071             for c in conteos:
4072                 s.query(Alerta).filter_by(conteo_id=c.id).delete()
4073                 s.delete(c)
4074             s.delete(ses)
4075         s.query(AsignacionBodega).filter_by(usuario_id=usuario_id).delete()
4076         # MensajeSoporte exige remitente/destinatario (no admite NULL) - sin
4077         # cuenta de uno de los dos lados, el mensaje no puede seguir existiendo.
4078         s.query(MensajeSoporte).filter(
4079             (MensajeSoporte.remitente_id == usuario_id)
4080             | (MensajeSoporte.destinatario_id == usuario_id)).delete()
4081         # estas si admiten NULL: se sueltan en vez de borrarse, para no
4082         # perder pedidos/aprobaciones reales solo porque quien los creo
4083         # ya no tiene cuenta.
4084         for linea in s.query(LineaServicio).filter_by(creado_por_id=usuario_id).all():
4085             linea.creado_por_id = None
4086         for ap in s.query(Aprobacion).filter_by(creado_por_id=usuario_id).all():
4087             ap.creado_por_id = None
4088         for ap in s.query(Aprobacion).filter_by(resuelto_por_id=usuario_id).all():
4089             ap.resuelto_por_id = None
4090         for t in s.query(Traza).filter_by(usuario_id=usuario_id).all():
4091             t.usuario_id = None
4092         s.delete(obj)
4093         s.commit()
4094     try:
4095         _cliente_cognito().admin_delete_user(UserPoolId=COGNITO_USER_POOL_ID, Username=nombre)
4096     except Exception as e:
4097         print(f"[cognito] no se pudo borrar la cuenta de {nombre}: {e}")
4098     registrar(u, "USUARIO", f"{nombre} eliminado de forma permanente", "ok")
4099     return {"ok": True}
4100
4101
4102 class ItemSemillaIn(BaseModel):
4103     articulo_codigo: str
4104     cantidad: float

```

```

4105 # solo para bodegas sin ningun stock de sistema cargado todavia (el
4106 # extracto de My Inventory nunca llego para esa bodega): crea la fila
4107 # de StockSistema con este valor ANTES de contar, para tener contra
4108 # que comparar. Si la bodega ya tiene stock de este articulo, se
4109 # ignora - nunca pisa un valor real ya cargado.
4110 cantidad_sistema_si_falta: float | None = None
4111 # una cantidad que se sale del umbral normalmente se salta (ver
4112 # docstring de sembrar_conteo) - forzar=true la guarda igual Y crea la
4113 # Alerta de "desviacion" que le corresponderia, exactamente como
4114 # cuando una persona real dice "sí, confirmo" tras la advertencia. Es
4115 # para poblar Alertas/Auditoria con casos reales en un ambiente de
4116 # demo, no para forzar cantidades negativas o de otro articulo/unidad
4117 # (esas siguen sin poder guardarse: no tendria sentido real).
4118 forzar: bool = False
4119
4120
4121 class ConteoSemillaIn(BaseModel):
4122     bodega_id: int
4123     usuario_id: int
4124     items: list[ItemSemillaIn]
4125
4126
4127 @app.post("/api/admin/sembrar-conteo")
4128 def sembrar_conteo(datos: ConteoSemillaIn, u: Usuario = Depends(requiere_perfil("auditor"))):
4129     """Crea conteos CONFIRMADOS reales (misma tabla, misma trazabilidad, se
4130     validan igual que si la persona lo hubiera dictado) para poblar un
4131     ambiente con datos de muestra antes de una demo - ej. que el panel
4132     gerencial tenga varias bodegas con diferencia real, no solo una. No
4133     pasa por reconocimiento de voz/texto: articulo y cantidad van directos,
4134     para que sembrar muchos a la vez no dependa de que el agente entienda
4135     cada frase (ni de la cuota de Gemini). Los items que dispararian una
4136     alerta (negativo, fuera de umbral...) se saltan en vez de forzarse -
4137     esto es para que la demo se vea real, no para inventar anomalias."""
4138     with Sesion() as s:
4139         persona = s.get(Usuario, datos.usuario_id)
4140         if persona is None:
4141             raise HTTPException(404, "Usuario no encontrado.")
4142         bodega = s.get(Bodega, datos.bodega_id)
4143         if bodega is None:
4144             raise HTTPException(404, "Bodega no encontrada.")
4145         ses = (s.query(SesionConteo)
4146               .filter_by(bodega_id=datos.bodega_id, usuario_id=datos.usuario_id, estado="abierta")
4147               .first())
4148         if not ses:
4149             ses = SesionConteo(bodega_id=datos.bodega_id, usuario_id=datos.usuario_id,
4150                               tipo="conteo", estado="abierta")
4151             s.add(ses)
4152             s.commit()
4153             s.refresh(ses)
4154             sesion_id = ses.id
4155             if bodega.estado == "pendiente":
4156                 bodega.estado = "en_conteo"
4157             s.commit()
4158             nombre_bodega = bodega.nombre_oficial
4159
4160     guardados = 0
4161     saltados = []
4162     for item in datos.items:
4163         with Sesion() as s:
4164             art = s.get(Articulo, item.articulo_codigo)
4165             if art is None:
4166                 saltados.append(item.articulo_codigo)
4167                 continue
4168             if item.cantidad_sistema_si_falta is not None:
4169                 existe = s.query(StockSistema).filter_by(
4170                     articulo_codigo=item.articulo_codigo, bodega_id=datos.bodega_id).first()
4171                 if not existe:
4172                     s.add(StockSistema(articulo_codigo=item.articulo_codigo,
4173                                         bodega_id=datos.bodega_id,
4174                                         cantidad_sd=item.cantidad_sistema_si_falta))
4175                     s.commit()
4176             v = validar_conteo(item.articulo_codigo, item.cantidad, art.unidad_medida, datos.bodega_id)
4177             if not v["ok"] and not (item.forzar and v["tipo"] == "desviacion"):
4178                 saltados.append(item.articulo_codigo)
4179             continue
4180         with Sesion() as s:
4181             reg = Conteo(sesion_id=sesion_id, articulo_codigo=item.articulo_codigo,
4182                          cantidad=item.cantidad, unidad=art.unidad_medida, estado="confirmado")
4183             s.add(reg)
4184             s.commit()
4185             s.refresh(reg)
4186             if not v["ok"]:
4187                 esperado = v.get("esperado") or 0
4188                 s.add(Alerta(conteo_id=reg.id, tipo="desviacion",
4189                             detalle=f"El sistema esperaba alrededor de {esperado:g}, "
4190                                     f"se contaron {item.cantidad:g}."))

```

```

4191         s.commit()
4192         guardados += 1
4193     if guardados:
4194         registrar(persona, "CONTEO",
4195             f"{guardados} referencias contadas en {nombre_bodega} (datos de muestra)")
4196     return {"ok": True, "guardados": guardados, "saltados": saltados}
4197
4198
4199 class DuracionSemillaIn(BaseModel):
4200     sesion_id: int
4201     minutos: float
4202
4203
4204 @app.post("/api/admin/sembrar-duracion")
4205 def sembrar_duracion(datos: DuracionSemillaIn, u: Usuario = Depends(requiere_perfil("auditor"))):
4206     """Ajusta cuánto "duró" una sesión de conteo (su inicio, relativo al fin
4207     que ya tiene) - para cuando una sesión de muestra se abrió hace rato
4208     (en otra prueba) y se firma/cierra recién ahora: sin esto, "tiempo
4209     promedio de conteo" del panel sale con la diferencia real entre esos
4210     dos momentos, que no fue tiempo contando de verdad."""
4211     with Sesion() as s:
4212         ses = s.get(SesionConteo, datos.sesion_id)
4213         if ses is None:
4214             raise HTTPException(404, "Sesion no encontrada.")
4215         if not ses.fin:
4216             raise HTTPException(409, "Esta sesion todavia no tiene fin (no está firmada/cerrada).")
4217         ses.inicio = ses.fin - timedelta(minutes=datos.minutos)
4218         s.commit()
4219     return {"ok": True}
4220
4221
4222 class AlertaSemillaIn(BaseModel):
4223     tipo: str # negativo | unidad | inexistente | desviacion
4224     detalle: str
4225
4226
4227 @app.post("/api/admin/sembrar-alerta")
4228 def sembrar_alerta(datos: AlertaSemillaIn, u: Usuario = Depends(requiere_perfil("auditor"))):
4229     """Crea una Alerta suelta (sin conteo asociado) para los tipos que en
4230     el flujo real NUNCA llegan a guardarse como conteo (negativo, unidad
4231     equivocada, articulo inexistente) - la persona tiene que repetir el
4232     dato, así que lo único que queda de ese intento es la alerta misma.
4233     Para poblar Auditoria/Panel con variedad real de tipos antes de una
4234     demo, no solo "desviación" (que sí se puede sembrar junto con su
4235     conteo via /admin/sembrar-conteo con forzar=true)."""
4236     if datos.tipo not in ("negativo", "unidad", "inexistente", "desviacion"):
4237         raise HTTPException(400, "Tipo de alerta no reconocido.")
4238     with Sesion() as s:
4239         s.add(Alerta(conteo_id=None, tipo=datos.tipo, detalle=datos.detalle))
4240         s.commit()
4241     return {"ok": True}
4242
4243
4244 class NegativosSemillaIn(BaseModel):
4245     inicial: int
4246
4247
4248 @app.put("/api/admin/negativos-iniciales")
4249 def ajustar_negativos_iniciales(datos: NegativosSemillaIn, u: Usuario = Depends(requiere_perfil("auditor"))):
4250     """La tarjeta "Negativos detectados en el sistema" compara el inicio de
4251     ESTE período contra el conteo actual - si nunca se fijó un punto de
4252     partida (o se fijó igual al valor de hoy, por default), la tarjeta
4253     siempre muestra "X → X" y no cuenta ninguna historia. Este valor es la
4254     foto de hace un tiempo (de antes de que My Inventory corrigiera los
4255     que ya se corrigieron), no algo que la app recalcule sola: por diseño,
4256     corregir un negativo del sistema pasa por fuera de CuentaVoz."""
4257     with Sesion() as s:
4258         existente = s.get(ConfigClave, "negativos_iniciales")
4259         valor = str(datos.inicial)
4260         if existente:
4261             existente.valor = valor
4262         else:
4263             s.add(ConfigClave(clave="negativos_iniciales", valor=valor))
4264         s.commit()
4265     return {"ok": True}
4266
4267
4268 @app.delete("/api/admin/historial-cierre/{historial_id}")
4269 def borrar_historial_cierre(historial_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
4270     """Para limpiar una foto de "Exactitud por toma de inventario" que
4271     quedó mal (ej. una bodega de muestra que se cerró antes de tener
4272     suficientes ítems contados, y después se reabrió para completarla) -
4273     a diferencia de la Traza, HistorialCierre no es un registro legal
4274     inmutable: es una serie de tiempo para la gráfica del panel, y una
4275     foto tomada por error de una prueba no debería quedar mezclada con el
4276     histórico real."""

```

```

4277     with Sesion() as s:
4278         h = s.get(HistorialCierre, historial_id)
4279         if h is None:
4280             raise HTTPException(404, "Registro no encontrado.")
4281         s.delete(h)
4282         s.commit()
4283     return {"ok": True}
4284
4285
4286 class UsoSemillaItem(BaseModel):
4287     articulo_codigo: str
4288     usado: float
4289
4290
4291 class UsoSemillaIn(BaseModel):
4292     servicio_id: int
4293     usos: list[UsoSemillaItem]
4294
4295
4296 @app.put("/api/admin/legalizacion-uso")
4297 def sembrar_uso_legalizacion(datos: UsoSemillaIn, u: Usuario = Depends(requiere_perfil("auditor"))):
4298     """Registra cuánto se usó de verdad en las líneas ya "abierto" de un
4299     servicio, SIN legalizarlo - a diferencia de /api/legalizacion/confirmar,
4300     que además cierra el servicio de una vez. Para dejar una comparación
4301     real y completa (pedido vs. usado) lista para revisar y confirmar en
4302     vivo en la demo, en vez de una pantalla vacía o con todo en cero."""
4303     with Sesion() as s:
4304         actualizadas = 0
4305         for item in datos.usos:
4306             l = (s.query(LineaServicio)
4307                 .filter_by(servicio_id=datos.servicio_id, articulo_codigo=item.articulo_codigo,
4308                           estado="abierto").first())
4309             if l:
4310                 l.usado = item.usado
4311                 actualizadas += 1
4312         s.commit()
4313     return {"ok": True, "actualizadas": actualizadas}
4314
4315
4316 @app.get("/api/usuarios/yo/bodegas")
4317 def bodegas_asignadas(u: Usuario = Depends(usuario_actual)):
4318     """Las bodegas asignadas a la persona que tiene la sesion. Va ANTES de
4319     la ruta parametrizada /usuarios/{usuario_id}/bodegas a proposito: si
4320     quedara despues, FastAPI hace match de "yo" contra {usuario_id} primero
4321     y esta ruta nunca se alcanza (por eso nunca respondia para un auxiliar)."""
4322     with Sesion() as s:
4323         asigs = s.query(AsignacionBodega).filter_by(usuario_id=u.id).all()
4324         salida = []
4325         for a in asigs:
4326             b = s.get(Bodega, a.bodega_id)
4327             if b:
4328                 refs = s.query(StockSistema).filter_by(bodega_id=b.id).count()
4329                 item = {"id": b.id, "bodega": b.nombre_oficial,
4330                        "estado": b.estado, "referencias": refs}
4331                 item.update(_contexto_bodega(s, b, refs))
4332                 salida.append(item)
4333     return salida
4334
4335
4336 @app.get("/api/usuarios/{usuario_id}/bodegas")
4337 def ver_bodegas_de_usuario(usuario_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
4338     """Para no borrar asignaciones existentes al agregar una nueva (ver
4339     «Asignar auditor» en el tablero de bodegas)."""
4340     with Sesion() as s:
4341         return [a.bodega_id for a in
4342                 s.query(AsignacionBodega).filter_by(usuario_id=usuario_id).all()]
4343
4344
4345 @app.put("/api/usuarios/{usuario_id}/bodegas")
4346 def asignar_bodegas(usuario_id: int, body: dict,
4347                     u: Usuario = Depends(requiere_perfil("auditor"))):
4348     ids = body.get("bodega_ids", [])
4349     with Sesion() as s:
4350         objetivo = s.get(Usuario, usuario_id)
4351         if objetivo is None:
4352             raise HTTPException(404, "Usuario no encontrado.")
4353         s.query(AsignacionBodega).filter_by(usuario_id=usuario_id).delete()
4354         for bid in ids:
4355             s.add(AsignacionBodega(usuario_id=usuario_id, bodega_id=bid))
4356         s.commit()
4357         nombre = objetivo.nombre
4358         registrar(u, "ASIGNACION", f"Bodegas asignadas a {nombre}: {len(ids)}", "ok")
4359     return {"ok": True}
4360
4361
4362 @app.get("/api/bodegas/asignaciones")

```

```

4363 def ver_asignaciones_completas(u: Usuario = Depends(requiere_perfil("auditor"))):
4364     """Vista de cobertura: por cada bodega, quien la tiene asignada (o
4365     nadie). Con varios auxiliares repartiendo 54 bodegas, el administrador
4366     necesita ver esto de un vistazo para no dejar bodegas sin nadie a cargo
4367     ni confundir a dos personas asignandoles las mismas por error - revisar
4368     ficha por ficha de cada persona no lo deja ver."""
4369     with Sesión() as s:
4370         por_bodega: dict[int, list] = {}
4371         for a in s.query(AsignacionBodega).all():
4372             por_bodega.setdefault(a.bodega_id, []).append(a.usuario_id)
4373         salida = []
4374         for b in s.query(Bodega).order_by(Bodega.nombre_oficial).all():
4375             personas = []
4376             for uid in por_bodega.get(b.id, []):
4377                 us = s.get(Usuario, uid)
4378                 if us:
4379                     personas.append({"id": us.id, "nombre": us.nombre, "perfil": us.perfil})
4380             salida.append({"id": b.id, "bodega": b.nombre_oficial, "asignados": personas})
4381     return salida
4382
4383
4384
4385
4386 # acciones sin valor para el resumen de "Inicio": ingresos repetidos, ajustes
4387 # de perfil y cada conteo individual (eso ya se ve en el tablero de Bodegas)
4388 _RUIDO_ACTIVIDAD = ("INGRESO", "SEGURIDAD", "PERFIL", "CONTEO")
4389
4390 def _narrar_actividad(t):
4391     p = (t.persona or "alguien").title()
4392     d = t.detalle
4393     if t.accion == "CREACION":
4394         nombre = d.split(" creado,")[0]
4395         return f"{p} creó {nombre.title()}, pendiente de aprobación"
4396     if t.accion == "APERTURA":
4397         bodega = d.split(" abierta")[0]
4398         return f"{p} abrió {bodega.title()}"
4399     if t.accion == "AUDITORIA":
4400         bodega = d.replace("Recuento ciego iniciado en ", "")
4401         return f"{p} inició el recuento ciego en {bodega.title()}"
4402     if t.accion == "FIRMA":
4403         tipo, _, bodega = d.partition(" - ")
4404         verbo = "firmó el conteo de" if tipo.lower().startswith("conteo") else "firmó la auditoría de"
4405         return f"{p} {verbo} {bodega.title()}"
4406     if t.accion == "CIERRE":
4407         bodega = d.split(" cerrada")[0]
4408         return f"{bodega.title()} quedó cerrada con doble firma"
4409     if t.accion == "REAPERTURA":
4410         motivo = d.split("motivo: ")[-1]
4411         bodega = d.split(" reabierta")[0]
4412         return f"{p} reabrió {bodega.title()} - motivo: {motivo}"
4413     if t.accion == "REPORTE":
4414         if d.startswith("Consolidado generado"):
4415             filas = f" ({d.rsplit(' ', 1)[-1]})" if "(" in d else ""
4416             return f"{p} generó el consolidado para My Inventory{filas}"
4417         if d.startswith("Estado de bodegas exportado"):
4418             return f"{p} exportó el estado del tablero"
4419         if d.startswith("Detalle de bodega"):
4420             return f"{p} exportó el detalle de una bodega"
4421         return f"{p} generó un reporte"
4422     if t.accion == "PEDIDO":
4423         return f"{p} envió un pedido al almacén"
4424     if t.accion == "LEGALIZACION":
4425         if d.startswith("Ajuste por voz"):
4426             return f"{p} ajustó un consumo por voz"
4427         return f"{p} legalizó un servicio"
4428     if t.accion == "ALERTA":
4429         if d.startswith("Alerta resuelta - "):
4430             nombre, _, msg = d[len("Alerta resuelta - "):].partition(": ")
4431             try:
4432                 esperado = msg.split("alrededor de ")[1].split(".")[0]
4433                 confirmado = msg.split("¿Confirma ")[1].rstrip("?")
4434                 return f"Alerta resuelta: {nombre} pasó de {esperado} a {confirmado}"
4435             except IndexError:
4436                 return f"{p} resolvió una alerta de {nombre}"
4437             return f"{p} resolvió una alerta"
4438     if t.accion == "CORRECCION":
4439         cambio = d.split(" (valor anterior)")[-1]
4440         return f"{p} corrigió un conteo: {cambio}"
4441     if t.accion == "USUARIO":
4442         nombre = d.split(" creado")[0].replace("Usuario ", "")
4443         return f"{p} creó el usuario {nombre.title()}"
4444     if t.accion == "ASIGNACION":
4445         return f"{p}: {d.lower()}"
4446     if t.accion == "APROBACION":
4447         if "entra al catalogo" in d:

```

```

4449         nombre = d.split(" aprobado")[0]
4450         return f"{p} aprobó el producto {nombre.title()}"
4451         nombre = d.split(" rechazado")[0]
4452         return f"{p} rechazó el producto {nombre.title()}"
4453     if t.accion == "AJUSTE":
4454         return f"{p} actualizó la configuración del sistema"
4455     if t.accion == "SOPORTE":
4456         if " le escribió a " in d:
4457             destinatario = d.split(" le escribió a ")[1].split(":")[0]
4458             return f"{p} le escribió a {destinatario.title()}"
4459         return f"{p} reportó un problema"
4460     return f"{p}: {d}"
4461
4462
4463 @app.get("/api/trazabilidad/reciente")
4464 def traza_reciente(u: Usuario = Depends(usuario_actual)):
4465     """Vista compartida para Inicio: sin datos sensibles, solo la acción y quién."""
4466     with Sesion() as s:
4467         trazas = (s.query(Traza).filter(~Traza.accion.in_(RUIDO_ACTIVIDAD))
4468                  .order_by(Traza.id.desc()).limit(8).all())
4469         return [{"hora": t.creado.strftime("%H:%M"), "persona": (t.persona or "").title(),
4470                  "accion": t.accion, "detalle": _narrar_actividad(t), "tipo": t.tipo}
4471                for t in trazas]
4472
4473
4474 # ----- aprobaciones en paralelo -----
4475 @app.get("/api/aprobaciones")
4476 def listar_aprobaciones(estado: str = "pendiente",
4477                        u: Usuario = Depends(requiere_perfil("auditor"))):
4478     with Sesion() as s:
4479         salida = []
4480         q = s.query(Aprobacion)
4481         if estado:
4482             q = q.filter_by(estado=estado)
4483         for a in q.order_by(Aprobacion.id.desc()).all():
4484             b = s.get(Bodega, a.bodega_id) if a.bodega_id else None
4485             creador = s.get(Usuario, a.creado_por_id) if a.creado_por_id else None
4486             salida.append({"id": a.id, "tipo": a.tipo, "nombre": a.nombre,
4487                           "unidad_medida": a.unidad_medida,
4488                           "cantidad_inicial": a.cantidad_inicial,
4489                           "bodega": b.nombre_oficial if b else None,
4490                           "creado_por": creador.nombre if creador else "?",
4491                           "hora": a.creado.strftime("%H:%M")})
4492     return salida
4493
4494
4495 @app.post("/api/aprobaciones/{aprobacion_id}/aprobar")
4496 def aprobar(aprobacion_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
4497     with Sesion() as s:
4498         a = s.get(Aprobacion, aprobacion_id)
4499         if a is None or a.estado != "pendiente":
4500             raise HTTPException(404, "Aprobacion no encontrada o ya resuelta.")
4501         a.estado = "aprobado"
4502         a.resuelto_por_id = u.id
4503         a.resuelto = ahora()
4504         if a.tipo == "producto" and a.articulo_codigo:
4505             existe = s.query(StockSistema).filter_by(
4506                 articulo_codigo=a.articulo_codigo, bodega_id=a.bodega_id).first()
4507             if not existe:
4508                 # el saldo del sistema arrancaba siempre en 0, sin importar
4509                 # cuanto se haya contado de verdad al crearlo (ej. "noventa
4510                 # alicornios") - el articulo quedaba marcado con una
4511                 # "diferencia" de +90 para siempre en el detalle de la
4512                 # bodega, como si nunca se hubiera revisado, aunque el
4513                 # administrador lo acababa de aprobar con ese conteo.
4514                 s.add(StockSistema(articulo_codigo=a.articulo_codigo, bodega_id=a.bodega_id,
4515                                     cantidad_sd=a.cantidad_inicial or 0))
4516             if a.conteo_id:
4517                 c = s.get(Conteo, a.conteo_id)
4518                 if c:
4519                     c.estado = "confirmado"
4520             elif a.tipo == "bodega":
4521                 s.add(Bodega(nombre_oficial=a.nombre, estado="pendiente"))
4522             s.commit()
4523             nombre = a.nombre
4524     registrar(u, "APROBACION", f"{nombre} aprobado y entra al catalogo oficial", "ok")
4525     return {"ok": True}
4526
4527
4528 @app.post("/api/aprobaciones/{aprobacion_id}/rechazar")
4529 def rechazar(aprobacion_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
4530     with Sesion() as s:
4531         a = s.get(Aprobacion, aprobacion_id)
4532         if a is None or a.estado != "pendiente":
4533             raise HTTPException(404, "Aprobacion no encontrada o ya resuelta.")
4534         a.estado = "rechazado"

```

```

4535         a.resuelto_por_id = u.id
4536         a.resuelto = ahora()
4537         if a.conteo_id:
4538             c = s.get(Conteo, a.conteo_id)
4539             if c:
4540                 c.estado = "rechazado"
4541             s.commit()
4542             nombre = a.nombre
4543             registrar(u, "APROBACION", f"{nombre} rechazado", "alerta")
4544             return {"ok": True}
4545
4546
4547 @app.post("/api/soporte/reportar")
4548 def reportar_problema(body: dict, u: Usuario = Depends(usuario_actual)):
4549     detalle = (body.get("detalle") or "").strip() or "Sin detalle."
4550     registrar(u, "SOPORTE", f"{u.nombre} reporto: {detalle}", "alerta")
4551     return {"ok": True}
4552
4553
4554 def _enviar_correo_real(destinatario: str, asunto: str, cuerpo: str) -> tuple[bool, str]:
4555     """Envía un correo de verdad por la API de Brevo (HTTPS) si hay
4556     credenciales configuradas (BREVO_API_KEY). Antes se intentó SMTP
4557     directo a Gmail (Render bloquea el puerto 587, "Network is
4558     unreachable") y luego la API de Resend (Cloudflare la rechazaba con
4559     "error code: 1010", probablemente por la reputación de las IPs
4560     compartidas de Render) - ninguna de las dos depende del código, así
4561     que se cambió de proveedor otra vez. Sin BREVO_API_KEY configurada,
4562     no intenta nada: el llamador sigue funcionando igual que antes (solo
4563     trazabilidad), el mismo patrón de "se degrada sin romperse" que ya
4564     usa el agente sin GOOGLE_API_KEY. Devuelve (enviado, motivo-si-fallo)
4565     - el motivo es temporal mientras se termina de calibrar el envío
4566     real."""
4567     api_key = os.getenv("BREVO_API_KEY", "").strip()
4568     remitente = os.getenv("BREVO_FROM", os.getenv("SMTP_CORREO", "")).strip()
4569     if not api_key or not remitente or not destinatario:
4570         return False, "sin BREVO_API_KEY/BREVO_FROM configuradas"
4571     payload = json.dumps({
4572         "sender": {"name": "CuentaVoz", "email": remitente},
4573         "to": [{"email": destinatario}],
4574         "subject": asunto, "textContent": cuerpo,
4575     }).encode("utf-8")
4576     peticion = urllib.request.Request(
4577         "https://api.brevo.com/v3/smtp/email", data=payload, method="POST",
4578         headers={"api-key": api_key, "Content-Type": "application/json",
4579                 "Accept": "application/json",
4580                 "User-Agent": "CuentaVoz/1.0 (+https://cuentavoz.onrender.com)"})
4581     # El contenedor de Render no tiene ruta de salida por IPv6, pero la
4582     # resolución de nombres a veces sí devuelve una dirección IPv6 (y
4583     # Python la intenta primero) - eso da exactamente "Network is
4584     # unreachable" aunque el IPv4 normal funcione bien. Se fuerza IPv4
4585     # solo durante esta conexión puntual.
4586     getaddrinfo_original = socket.getaddrinfo
4587
4588     def _forzar_ipv4(host, port, family=0, type=0, proto=0, flags=0):
4589         return getaddrinfo_original(host, port, socket.AF_INET, type, proto, flags)
4590
4591     socket.getaddrinfo = _forzar_ipv4
4592     try:
4593         with urllib.request.urlopen(peticion, timeout=10) as resp:
4594             return 200 <= resp.status < 300, ""
4595     except urllib.error.HTTPError as e:
4596         motivo = f"HTTP {e.code}: {e.read().decode('utf-8', 'ignore')}[:300]}"
4597         print(f"[correo] no se pudo enviar a {destinatario}: {motivo}")
4598         return False, motivo
4599     except Exception as e:
4600         motivo = f"{type(e).__name__}: {e}"
4601         print(f"[correo] no se pudo enviar a {destinatario}: {motivo}")
4602         return False, motivo
4603     finally:
4604         socket.getaddrinfo = getaddrinfo_original
4605
4606
4607 @app.post("/api/soporte/mensaje-administrador")
4608 def mensaje_administrador(body: dict, u: Usuario = Depends(usuario_actual)):
4609     """Escribirle al administrador. El mensaje siempre queda trazado -
4610     se ve en Ajustes → Registro de trazabilidad - y además se intenta un
4611     correo real (vía Brevo) si el servidor tiene BREVO_API_KEY
4612     configurada. Sin esa credencial (el caso normal por ahora: no
4613     depende de registrar una cuenta con un tercero), el frontend abre el
4614     correo ya instalado en el dispositivo de quien envía, con el mensaje
4615     listo - así igual llega un correo real, desde la cuenta de esa
4616     persona, sin que CuentaVoz tenga que gestionar ningún servicio de
4617     correo por su cuenta."""
4618     mensaje = (body.get("mensaje") or "").strip()
4619     if not mensaje:
4620         raise HTTPException(400, "Escriba el mensaje antes de enviarlo.")

```

```

4621 with Sesion() as s:
4622     admin = (s.query(Usuario)
4623             .filter(Usuario.perfil == "auditor", Usuario.activo == 1, Usuario.id != u.id)
4624             .first())
4625     if not admin:
4626         raise HTTPException(404, "No hay un administrador disponible por ahora.")
4627     nombre_admin, correo_admin, admin_id = admin.nombre, admin.correo, admin.id
4628     s.add(MensajeSoporte(remite_id=u.id, destinatario_id=admin_id, mensaje=mensaje))
4629     s.commit()
4630     registrar(u, "SOPORTE", f"{u.nombre} le escribió a {nombre_admin}: {mensaje}")
4631     cuerpo = f"Mensaje enviado desde CuentaVoz por {u.nombre}.\n\n{mensaje}"
4632     correo_enviado, _motivo = _enviar_correo_real(correo_admin, "Mensaje desde CuentaVoz", cuerpo)
4633     return {"ok": True, "administrador": nombre_admin, "correo_enviado": correo_enviado}
4634
4635
4636 @app.get("/api/soporte/mensajes")
4637 def mis_mensajes_soporte(u: Usuario = Depends(usuario_actual)):
4638     """La bandeja de "Soporte en vivo" dentro de la app: para un auxiliar,
4639     lo que ha escrito y si ya le respondieron; para el administrador, lo
4640     que le han escrito y lo que falta por responder."""
4641     with Sesion() as s:
4642         if u.perfil == "auditor":
4643             filas = (s.query(MensajeSoporte)
4644                     .filter(MensajeSoporte.destinatario_id == u.id)
4645                     .order_by(MensajeSoporte.creado.desc()).all())
4646         else:
4647             filas = (s.query(MensajeSoporte)
4648                     .filter(MensajeSoporte.remite_id == u.id)
4649                     .order_by(MensajeSoporte.creado.desc()).all())
4650         ids_personas = {m.remite_id for m in filas} | {m.destinatario_id for m in filas}
4651         personas = {p.id: p.nombre for p in s.query(Usuario).filter(Usuario.id.in_(ids_personas)).all()}
4652         return [{"id": m.id,
4653                 "de": personas.get(m.remite_id, "?"),
4654                 "para": personas.get(m.destinatario_id, "?"),
4655                 "mensaje": m.mensaje, "respuesta": m.respuesta,
4656                 "creado": m.creado.strftime("%Y-%m-%d %H:%M"),
4657                 "respondido": m.respondido.strftime("%Y-%m-%d %H:%M") if m.respondido else None}
4658                for m in filas]
4659
4660
4661 @app.post("/api/soporte/mensajes/{mensaje_id}/responder")
4662 def responder_mensaje_soporte(mensaje_id: int, body: dict, u: Usuario = Depends(usuario_actual)):
4663     """Solo quien recibió el mensaje puede responderlo - no hace falta
4664     pedir perfil "auditor" aparte: si no es el destinatario, no puede."""
4665     respuesta = (body.get("respuesta") or "").strip()
4666     if not respuesta:
4667         raise HTTPException(400, "Escriba la respuesta antes de enviarla.")
4668     with Sesion() as s:
4669         m = s.get(MensajeSoporte, mensaje_id)
4670         if not m or m.destinatario_id != u.id:
4671             raise HTTPException(404, "Ese mensaje no existe o no es suyo para responder.")
4672         if m.respuesta is not None:
4673             # sin este chequeo, dos respuestas al mismo mensaje (un doble
4674             # clic, un reintento de red, dos pestañas abiertas) pisaban la
4675             # respuesta anterior en silencio - la persona que ya la había
4676             # leído (o recibido por correo) se quedaba sin saber que
4677             # cambió, y se mandaba un segundo correo real de mas.
4678             raise HTTPException(409, "Ese mensaje ya fue respondido.")
4679         m.respuesta = respuesta
4680         m.respondido = ahora()
4681         s.commit()
4682         remitente = s.get(Usuario, m.remite_id)
4683         nombre_remitente = remitente.nombre if remitente else "?"
4684         correo_remitente = remitente.correo if remitente else None
4685         registrar(u, "SOPORTE", f"{u.nombre} le respondió a {nombre_remitente}: {respuesta}")
4686         return {"ok": True, "destinatario": nombre_remitente, "correo_destinatario": correo_remitente}
4687
4688
4689 @app.get("/api/soporte/administrador")
4690 def administrador_de_turno(u: Usuario = Depends(usuario_actual)):
4691     """Con quien puede contactarse cualquier persona (no solo el
4692     administrador) desde Ayuda - sin exponer el resto del listado de
4693     usuarios, que si es exclusivo del administrador. Si quien pregunta ya
4694     es administrador, no tiene sentido mostrarse a si mismo como contacto:
4695     se busca a OTRO administrador, y si no hay ninguno mas, no hay a quien
4696     escribirle (para eso esta la mesa de ayuda de Colsubsidio aparte)."""
4697     with Sesion() as s:
4698         admin = (s.query(Usuario)
4699                 .filter(Usuario.perfil == "auditor", Usuario.activo == 1,
4700                        Usuario.id != u.id)
4701                 .first())
4702     if not admin:
4703         return {"nombre": None, "correo": None, "es_usted": u.perfil == "auditor"}
4704     return {"nombre": admin.nombre, "correo": admin.correo, "es_usted": False}
4705
4706

```

```

4707 @app.get("/api/usuarios/yo")
4708 def ver_perfil(u: Usuario = Depends(usuario_actual)):
4709     from agente.cerebro import VOCES, VOZ_DEFEECTO
4710     dias_pin = (ahora() - (u.pin_actualizado or ahora())).days
4711     # idioma_voz guardaba un codigo de idioma de navegador (es-MX, es-CO...)
4712     # de cuando la voz salia por speechSynthesis; ahora guarda la clave de
4713     # la voz neuronal elegida (kore, puck...). Una cuenta con el valor
4714     # viejo cae a la voz por defecto en vez de romper el selector.
4715     voz = u.idioma_voz if u.idioma_voz in VOCES else VOZ_DEFEECTO
4716     return {"id": u.id, "nombre": u.nombre, "correo": u.correo, "telefono": u.telefono,
4717            "codigo": u.codigo, "perfil": u.perfil,
4718            "ultimo_acceso": u.ultimo_acceso.strftime("%Y-%m-%d %H:%M") if u.ultimo_acceso else None,
4719            "pin_vence_en_dias": max(90 - dias_pin, 0),
4720            "idioma_voz": voz, "velocidad_voz": u.velocidad_voz,
4721            "confirmacion_hablada": bool(u.confirmacion_hablada)}
4722
4723
4724 @app.get("/api/usuarios/yo/resumen")
4725 def resumen_inicio(u: Usuario = Depends(usuario_actual)):
4726     hoy = ahora().date()
4727     inicio_mes = hoy.replace(day=1)
4728     with Sesion() as s:
4729         n_bodegas = s.query(AsignacionBodega).filter_by(usuario_id=u.id).count()
4730         sesiones_usr = [x.id for x in s.query(SesionConteo).filter_by(usuario_id=u.id).all()]
4731         ref_hoy = 0
4732         if sesiones_usr:
4733             ref_hoy = sum(1 for c in (s.query(Conteo)
4734                                     .filter(Conteo.sesion_id.in_(sesiones_usr),
4735                                             Conteo.estado == "confirmado").all())
4736                           if c.creado.date() == hoy)
4737         alertas_abiertas = s.query(Alerta).filter_by(resuelta=0).count()
4738         # "Su exactitud del mes" tenia que decir de verdad SU exactitud (las
4739         # bodegas donde esta persona contó o auditó) y de verdad DEL MES
4740         # actual - antes promediaba el historial de cierres de TODAS las
4741         # bodegas de TODA la operación, sin filtrar ni por persona ni por
4742         # fecha, así que un auxiliar nuevo sin ningún cierre propio veía el
4743         # mismo numero que la administradora con mas experiencia.
4744         bodegas_suyas = {ses.bodega_id for ses in s.query(SesionConteo)
4745                         .filter(SesionConteo.usuario_id == u.id).all()}
4746         historial_mes = (s.query(HistorialCierre)
4747                         .filter(HistorialCierre.bodega_id.in_(bodegas_suyas),
4748                                 HistorialCierre.fecha >= inicio_mes).all()
4749                         if bodegas_suyas else [])
4750         exact_mes = (round(sum(h.exactitud for h in historial_mes) / len(historial_mes), 1)
4751                     if historial_mes else 100.0)
4752     return {"bodegas_asignadas": n_bodegas, "referencias_hoy": ref_hoy,
4753            "alertas_por_revisar": alertas_abiertas, "exactitud_mes": exact_mes}
4754
4755
4756 @app.put("/api/usuarios/yo/preferencias")
4757 def guardar_preferencias(body: dict, u: Usuario = Depends(usuario_actual)):
4758     with Sesion() as s:
4759         usr = s.get(Usuario, u.id)
4760         for k in ("idioma_voz", "velocidad_voz"):
4761             if k in body:
4762                 setattr(usr, k, body[k])
4763         if "confirmacion_hablada" in body:
4764             usr.confirmacion_hablada = 1 if body["confirmacion_hablada"] else 0
4765         s.commit()
4766     return {"ok": True}
4767
4768
4769 @app.post("/api/usuarios/yo/marcar-clave-cambiada")
4770 def marcar_clave_cambiada(u: Usuario = Depends(usuario_actual)):
4771     """Cognito es quien cambia la clave (Mi perfil o "olvide mi clave"),
4772     nunca este backend - pero el aviso de "su clave vence en N dias" (ver
4773     ver_perfil) se calcula desde pin_actualizado, que sin esta llamada se
4774     queda congelado en la fecha de creacion de la cuenta para siempre. El
4775     frontend llama esto justo despues de que Cognito confirma el cambio."""
4776     with Sesion() as s:
4777         usr = s.get(Usuario, u.id)
4778         usr.pin_actualizado = ahora()
4779         s.commit()
4780     return {"ok": True}
4781
4782
4783 @app.post("/api/usuarios/yo/cerrar-todas")
4784 def cerrar_todas_sesiones(u: Usuario = Depends(usuario_actual)):
4785     """Antes invalidaba con un contador propio (version_token) y devolvía
4786     un token nuevo firmado por nosotros; ahora la identidad la maneja
4787     Cognito, así que se le pide a Cognito que revoque los refresh tokens
4788     de esta persona (AdminUserGlobalSignOut: nadie con sesion abierta en
4789     otro dispositivo puede volver a pedir un access token nuevo). El
4790     access token que ya tenia emitido cada dispositivo sigue firmando
4791     valido hasta que vence solo (el User Pool los emite con vida corta,
4792     1 hora, para que esa ventana sea chica)."""

```

```

4793     cliente = _cliente_cognito()
4794     try:
4795         cliente.admin_user_global_sign_out(UserPoolId=COGNITO_USER_POOL_ID, Username=u.nombre)
4796     except Exception as e:
4797         raise HTTPException(502, f"No se pudo cerrar las sesiones en este momento: {e}")
4798     registrar(u, "SEGURIDAD", "Sesion cerrada en todos los dispositivos")
4799     return {"ok": True}
4800
4801
4802 def _tipo_imagen_real(contenido: bytes) -> str | None:
4803     """El Content-Type que manda el navegador lo elige quien sube el
4804     archivo, no el archivo: antes se guardaba y se devolvía tal cual al
4805     pedir la foto, así que subir algo con Content-Type "text/html" lo
4806     serviría después como HTML. Aquí se detecta el tipo real por los
4807     primeros bytes (firma del formato), no por lo que diga la cabecera."""
4808     if contenido.startswith(b"\xff\xd8\xff"):
4809         return "image/jpeg"
4810     if contenido.startswith(b"\x89PNG\r\n\x1a\n"):
4811         return "image/png"
4812     if contenido.startswith((b"GIF87a", b"GIF89a")):
4813         return "image/gif"
4814     if contenido[4] == b"RIFF" and contenido[8:12] == b"WEBP":
4815         return "image/webp"
4816     return None
4817
4818
4819 @app.post("/api/usuarios/yo/foto")
4820 async def subir_foto(foto: UploadFile = File(...),
4821                     u: Usuario = Depends(usuario_actual)):
4822     contenido = await foto.read()
4823     if len(contenido) > 3 * 1024 * 1024:
4824         raise HTTPException(400, "La foto no puede pesar mas de 3 MB.")
4825     tipo_real = _tipo_imagen_real(contenido)
4826     if tipo_real is None:
4827         raise HTTPException(400, "El archivo no es una imagen valida (JPEG, PNG, GIF o WebP).")
4828     with Sesion() as s:
4829         usr = s.get(Usuario, u.id)
4830         usr.foto = contenido
4831         usr.foto_tipo = tipo_real
4832         s.commit()
4833     return {"ok": True}
4834
4835
4836 @app.get("/api/usuarios/yo/foto")
4837 def ver_foto(u: Usuario = Depends(usuario_actual)):
4838     with Sesion() as s:
4839         usr = s.get(Usuario, u.id)
4840         if not usr.foto:
4841             raise HTTPException(404, "Sin foto.")
4842     return Response(content=usr.foto, media_type=usr.foto_tipo or "image/jpeg")
4843
4844
4845 @app.get("/api/voz/voces")
4846 def voces_disponibles(u: Usuario = Depends(usuario_actual)):
4847     from agente.cerebro import VOCES, VOZ_DEFECTO
4848     return {"voces": [{"clave": k, **v} for k, v in VOCES.items()],
4849           "defecto": VOZ_DEFECTO}
4850
4851
4852 class HablarIn(BaseModel):
4853     texto: str
4854     voz: str = "kore"
4855
4856
4857 @app.post("/api/voz/hablar")
4858 def api_hablar(body: HablarIn, u: Usuario = Depends(usuario_actual)):
4859     from agente.cerebro import sintetizar_voz
4860     texto = (body.texto or "").strip()
4861     if not texto:
4862         raise HTTPException(400, "Falta el texto.")
4863     wav = sintetizar_voz(texto, body.voz)
4864     if wav is None:
4865         raise HTTPException(503, "Voz neuronal no disponible en este momento.")
4866     return Response(content=wav, media_type="audio/wav")
4867
4868
4869 def _filtro_traza(s, persona: str, accion: str, rango: str):
4870     q = s.query(Traza)
4871     if persona:
4872         q = q.filter_by(persona=persona)
4873     if accion:
4874         q = q.filter_by(accion=accion)
4875     if rango == "hoy":
4876         q = q.filter(Traza.creado >= ahora().replace(hour=0, minute=0, second=0))
4877     elif rango == "semana":
4878         q = q.filter(Traza.creado >= ahora() - timedelta(days=7))

```

```

4879     elif rango == "mes":
4880         q = q.filter(Traza.creado >= ahora() - timedelta(days=30))
4881     return q
4882
4883
4884 @app.get("/api/trazabilidad")
4885 def ver_traza(persona: str = "", accion: str = "", rango: str = "",
4886             u: Usuario = Depends(requiere_perfil("auditor"))):
4887     with Sesion() as s:
4888         q = _filtro_traza(s, persona, accion, rango)
4889         return [{"id": t.id, "hora": t.creado.strftime("%H:%M:%S"),
4890                 "persona": t.persona, "accion": t.accion,
4891                 "detalle": t.detalle, "tipo": t.tipo}
4892                 for t in q.order_by(Traza.id.desc()).limit(200)]
4893
4894
4895 @app.get("/api/trazabilidad/exportar")
4896 def exportar_traza(persona: str = "", accion: str = "", rango: str = "",
4897                 formato: str = "xlsx",
4898                 u: Usuario = Depends(requiere_perfil("auditor"))):
4899     import pandas as pd
4900     from servicios.archivos import guardar_df
4901     with Sesion() as s:
4902         q = _filtro_traza(s, persona, accion, rango)
4903         filas = [{"fecha": t.creado.strftime("%Y-%m-%d %H:%M:%S"), "persona": t.persona,
4904                 "accion": t.accion, "detalle": t.detalle}
4905                 for t in q.order_by(Traza.id.desc()).all()]
4906         df = pd.DataFrame(filas)
4907         marca = ahora().strftime("%Y%m%d_%H%M")
4908         ruta = f"reportes/trazabilidad_{marca}.{formato}"
4909         guardar_df(ruta, df, formato)
4910         registrar(u, "REPORTE", f"Registro de trazabilidad exportado: {ruta} ({len(df)} filas)")
4911         return {"archivo": ruta, "filas": len(df)}
4912
4913
4914 @app.get("/api/ajustes")
4915 def ver_ajustes(u: Usuario = Depends(usuario_actual)):
4916     with Sesion() as s:
4917         offline = s.get(ConfigClave, "offline")
4918         usuarios_activos = s.query(Usuario).filter_by(activo=1).count()
4919         aprobaciones_pend = s.query(Aprobacion).filter_by(estado="pendiente").count()
4920         pedidos_pend = (s.query(LineaServicio.numero_pedido)
4921                        .filter_by(estado="pendiente_aprobacion")
4922                        .distinct().count())
4923     return {"umbral": round(umbral_actual() * 100),
4924           "bloquear_negativos": True, "confirmar_alertas": True,
4925           "offline": (offline.valor == "1") if offline else True,
4926           "version": "1.0.0", "modelo": os.getenv("MODELO", "gemini-flash-latest"),
4927           "idioma_voz": os.getenv("IDIOMA_VOZ", "es-CO"),
4928           "base_datos": "SQLite",
4929           "usuarios_activos": usuarios_activos,
4930           "aprobaciones_pendientes": aprobaciones_pend + pedidos_pend}
4931
4932
4933 @app.put("/api/ajustes")
4934 def guardar_ajustes(body: dict, u: Usuario = Depends(requiere_perfil("auditor"))):
4935     with Sesion() as s:
4936         if "umbral" in body:
4937             valor = str(max(1, min(500, float(body["umbral"]))) / 100)
4938             existente = s.get(ConfigClave, "umbral_anomalia")
4939             if existente:
4940                 existente.valor = valor
4941             else:
4942                 s.add(ConfigClave(clave="umbral_anomalia", valor=valor))
4943         if "offline" in body:
4944             valor = "1" if body["offline"] else "0"
4945             existente = s.get(ConfigClave, "offline")
4946             if existente:
4947                 existente.valor = valor
4948             else:
4949                 s.add(ConfigClave(clave="offline", valor=valor))
4950         s.commit()
4951         registrar(u, "AJUSTE", "Configuracion del sistema actualizada", "ok")
4952         return {"ok": True}
4953
4954
4955 # ----- tablero en vivo -----
4956 conexiones: list[WebSocket] = []
4957
4958
4959 def estado_bodegas():
4960     with Sesion() as s:
4961         return [{"bodega": b.nombre_oficial, "estado": b.estado, "id": b.id}
4962                 for b in s.query(Bodega).order_by(Bodega.nombre_oficial).all()]
4963
4964

```

```

4965 async def difundir_estado():
4966     for ws in list(conexiones):
4967         try:
4968             await ws.send_json(estado_bodegas())
4969         except Exception:
4970             if ws in conexiones:
4971                 conexiones.remove(ws)
4972
4973
4974 @app.websocket("/api/bodegas/estado")
4975 async def ws_estado(ws: WebSocket):
4976     from seguridad import verificar_token
4977     if not verificar_token(ws.query_params.get("token")):
4978         await ws.close(code=1008)
4979         return
4980     await ws.accept()
4981     conexiones.append(ws)
4982     await ws.send_json(estado_bodegas())
4983     try:
4984         while True:
4985             await ws.receive_text()
4986     except WebSocketDisconnect:
4987         if ws in conexiones:
4988             conexiones.remove(ws)

```

backend/modelos.py

232 LÍNEAS

Las 18 tablas en SQLAlchemy. Es el mapa del dominio.

```

1  """Las tablas del sistema: el diagrama de clases hecho código."""
2  from sqlalchemy import Column, Integer, String, Float, DateTime, ForeignKey, LargeBinary
3  from bd import Base
4  from horario import ahora
5
6
7  class Usuario(Base):
8      __tablename__ = "usuario"
9      id = Column(Integer, primary_key=True)
10     # unique=True: antes solo se evitaba un nombre duplicado revisando "ya
11     # existe" justo antes de crear (main.py:crear_usuario, orquestador.py:
12     # _crear_usuario_por_voz) - dos creaciones casi simultaneas del mismo
13     # nombre podian pasar esa revision las dos y quedar dos cuentas con el
14     # mismo nombre, rompiendo el ingreso (que busca por nombre y solo
15     # devuelve la primera que encuentra) de forma impredecible para
16     # cualquiera de las dos personas.
17     nombre = Column(String, nullable=False, unique=True)
18     perfil = Column(String, nullable=False) # auxiliar | auditor
19     # La clave la maneja Cognito, no esta base - esta columna ya no se
20     # llena (nullable=True: SQLite no permite quitarle el NOT NULL a una
21     # columna existente sin recrear la tabla; se deja aqui sin usar en
22     # vez de arriesgar esa migracion en una base ya desplegada).
23     clave_hash = Column(String, nullable=True)
24     correo = Column(String, default="")
25     telefono = Column(String, default="")
26     codigo = Column(String, default="")
27     activo = Column(Integer, default=1)
28     # NULL en cualquier cuenta creada antes de esta columna (semilla,
29     # creada a mano por un auditor, etc.) - nunca se migran, quedan fuera
30     # del filtro de aprobacion para siempre. Solo el autoregistro
31     # (main.py: registro_completado) pone 0 aqui; un auditor lo cambia a 1
32     # (aprobado) o -1 (rechazado, activo se queda en 0 para siempre).
33     aprobado = Column(Integer, nullable=True)
34     pin_actualizado = Column(DateTime, default=ahora)
35     ultimo_acceso = Column(DateTime, nullable=True)
36     idioma_voz = Column(String, default="es-MX")
37     velocidad_voz = Column(String, default="normal") # lenta | normal | rapida
38     confirmacion_hablada = Column(Integer, default=1)
39     # la foto va en la misma base (no en disco): el disco del Web Service
40     # en Render es efimero y se borra en cada deploy/reinicio, la base no.
41     foto = Column(LargeBinary, nullable=True)
42     foto_tipo = Column(String, nullable=True) # ej. "image/jpeg"
43
44
45  class AsignacionBodega(Base):
46     """Qué bodegas puede contar cada persona."""
47     __tablename__ = "asignacion_bodega"
48     id = Column(Integer, primary_key=True)
49     usuario_id = Column(Integer, ForeignKey("usuario.id"))
50     bodega_id = Column(Integer, ForeignKey("bodega.id"))
51
52
53  class Bodega(Base):
54     __tablename__ = "bodega"

```

```

55     id = Column(Integer, primary_key=True)
56     nombre_oficial = Column(String, unique=True, nullable=False)
57     estado = Column(String, default="pendiente")      # pendiente | en_conteo | en_auditoria | cerrada
58
59
60 class Artículo(Base):
61     __tablename__ = "articulo"
62     codigo = Column(String, primary_key=True)
63     nombre_oficial = Column(String, nullable=False)
64     unidad_medida = Column(String, nullable=False)
65
66
67 class StockSistema(Base):
68     """El extracto de My Inventory: lo que el sistema cree que hay."""
69     __tablename__ = "stock_sistema"
70     id = Column(Integer, primary_key=True)
71     articulo_codigo = Column(String, ForeignKey("articulo.codigo"))
72     bodega_id = Column(Integer, ForeignKey("bodega.id"))
73     cantidad_sd = Column(Float, default=0)
74
75
76 class AliasArticulo(Base):
77     """Cómo habla el equipo de verdad. El agente lo va aprendiendo."""
78     __tablename__ = "alias_articulo"
79     id = Column(Integer, primary_key=True)
80     texto_dicho = Column(String, index=True)
81     articulo_codigo = Column(String, ForeignKey("articulo.codigo"))
82     bodega_id = Column(Integer, nullable=True)
83
84
85 class SesionConteo(Base):
86     __tablename__ = "sesion_conteo"
87     id = Column(Integer, primary_key=True)
88     bodega_id = Column(Integer, ForeignKey("bodega.id"))
89     usuario_id = Column(Integer, ForeignKey("usuario.id"))
90     tipo = Column(String, default="conteo")           # conteo | auditoria
91     estado = Column(String, default="abierta")
92     firmada = Column(Integer, default=0)              # firma digital de cierre
93     inicio = Column(DateTime, default=ahora)
94     fin = Column(DateTime, nullable=True)
95
96
97 class Conteo(Base):
98     __tablename__ = "conteo"
99     id = Column(Integer, primary_key=True)
100     sesion_id = Column(Integer, ForeignKey("sesion_conteo.id"))
101     articulo_codigo = Column(String, ForeignKey("articulo.codigo"))
102     cantidad = Column(Float, nullable=False)
103     unidad = Column(String)
104     estado = Column(String, default="confirmado")
105     # confirmado | corregido | pendiente_aprobacion | borrador
106     corrige_a = Column(Integer, ForeignKey("conteo.id"), nullable=True)
107     creado = Column(DateTime, default=ahora)
108
109
110 class Alerta(Base):
111     __tablename__ = "alerta"
112     id = Column(Integer, primary_key=True)
113     conteo_id = Column(Integer, ForeignKey("conteo.id"), nullable=True)
114     tipo = Column(String)                             # unidad | negativo | desviacion | inexistente
115     detalle = Column(String)
116     resuelta = Column(Integer, default=0)
117     creado = Column(DateTime, default=ahora)
118     resuelto = Column(DateTime, nullable=True)
119
120
121 class Traza(Base):
122     """Registro inmutable: solo crece, nunca se edita ni se borra."""
123     __tablename__ = "traza"
124     id = Column(Integer, primary_key=True)
125     usuario_id = Column(Integer, ForeignKey("usuario.id"), nullable=True)
126     persona = Column(String)
127     accion = Column(String)                            # APERTURA | CIERRE | CORRECCION | ALERTA...
128     detalle = Column(String)
129     tipo = Column(String, default="info")
130     creado = Column(DateTime, default=ahora)
131
132
133 # — los tres momentos del reto: recetas y servicios —
134 class Receta(Base):
135     __tablename__ = "receta"
136     id = Column(Integer, primary_key=True)
137     nombre = Column(String, nullable=False)
138     rendimiento = Column(Integer, default=1)
139     # paso a paso de cocina, no solo la lista de ingredientes con su cantidad.
140     preparacion = Column(String, default="")

```

```

141
142
143 class RecetaIngrediente(Base):
144     __tablename__ = "receta_ingrediente"
145     id = Column(Integer, primary_key=True)
146     receta_id = Column(Integer, ForeignKey("receta.id"))
147     articulo_codigo = Column(String, ForeignKey("articulo.codigo"))
148     cantidad_por_porcion = Column(Float, nullable=False)
149
150
151 class LineaServicio(Base):
152     __tablename__ = "linea_servicio"
153     id = Column(Integer, primary_key=True)
154     servicio_id = Column(Integer, default=1)
155     articulo_codigo = Column(String, ForeignKey("articulo.codigo"))
156     nombre = Column(String)
157     pedido = Column(Float, default=0)
158     usado = Column(Float, default=0)
159     # abierto | legalizado | pendiente_aprobacion | rechazado
160     estado = Column(String, default="abierto")
161     plato = Column(String, default="")
162     porciones = Column(Integer, default=0)
163     creado = Column(DateTime, default=ahora)
164     bodega_id = Column(Integer, ForeignKey("bodega.id"), nullable=True)
165     creado_por_id = Column(Integer, ForeignKey("usuario.id"), nullable=True)
166     # agrupa las lineas de un mismo envio para poder aprobarlo/rechazarlo
167     # como una sola unidad, no linea por linea.
168     numero_pedido = Column(String, nullable=True)
169
170
171 # — aprobación en paralelo: productos y bodegas creados en plena toma —
172 class Aprobacion(Base):
173     __tablename__ = "aprobacion"
174     id = Column(Integer, primary_key=True)
175     tipo = Column(String, nullable=False) # producto | bodega
176     nombre = Column(String, nullable=False)
177     unidad_medida = Column(String, nullable=True)
178     cantidad_inicial = Column(Float, nullable=True)
179     bodega_id = Column(Integer, ForeignKey("bodega.id"), nullable=True)
180     articulo_codigo = Column(String, ForeignKey("articulo.codigo"), nullable=True)
181     conteo_id = Column(Integer, ForeignKey("conteo.id"), nullable=True)
182     creado_por_id = Column(Integer, ForeignKey("usuario.id"), nullable=True)
183     estado = Column(String, default="pendiente") # pendiente | aprobado | rechazado
184     creado = Column(DateTime, default=ahora)
185     resuelto_por_id = Column(Integer, ForeignKey("usuario.id"), nullable=True)
186     resuelto = Column(DateTime, nullable=True)
187
188
189 class HistorialCierre(Base):
190     """Una foto de la exactitud cada vez que una bodega cierra con doble firma."""
191     __tablename__ = "historial_cierre"
192     id = Column(Integer, primary_key=True)
193     bodega_id = Column(Integer, ForeignKey("bodega.id"))
194     exactitud = Column(Float, default=100)
195     referencias = Column(Integer, default=0)
196     diferencias = Column(Integer, default=0)
197     fecha = Column(DateTime, default=ahora)
198
199
200 class ConfigClave(Base):
201     """Ajustes editables desde la aplicación, con el .env como valor por defecto."""
202     __tablename__ = "config_clave"
203     clave = Column(String, primary_key=True)
204     valor = Column(String, nullable=False)
205
206
207 class ArchivoGenerado(Base):
208     """Los XLSX/CSV que se generan (consolidado, diferencias, análisis,
209     trazabilidad...) guardados en la base, no en disco - mismo motivo que
210     la foto de perfil: el disco del Web Service en Render es efímero y se
211     borra en cada deploy/reinicio, así que un archivo "generado hace una
212     hora" debe poder seguir viéndose/descargándose después de un redeploy,
213     no solo hasta el próximo `git push`. """
214     __tablename__ = "archivo_generado"
215     id = Column(Integer, primary_key=True)
216     ruta = Column(String, nullable=False, unique=True)
217     contenido = Column(LargeBinary, nullable=False)
218     creado = Column(DateTime, default=ahora)
219
220
221 class MensajeSoporte(Base):
222     """Un mensaje de "Soporte en vivo" (auxiliar -> administrador) con su
223     respuesta, si ya la dio - la bandeja dentro de la app en Ayuda,
224     aparte del correo real que tambien se manda por fuera (EmailJS). """
225     __tablename__ = "mensaje_soporte"
226     id = Column(Integer, primary_key=True)

```

```

227     remitente_id = Column(Integer, ForeignKey("usuario.id"), nullable=False)
228     destinatario_id = Column(Integer, ForeignKey("usuario.id"), nullable=False)
229     mensaje = Column(String, nullable=False)
230     respuesta = Column(String, nullable=True)
231     creado = Column(DateTime, default=ahora)
232     respondido = Column(DateTime, nullable=True)

```

backend/bd.py

94 LÍNEAS

La conexion y la sesion. Lo unico que cambia entre SQLite y PostgreSQL vive aqui.

```

1  """Conexión a la base de datos. Todo lo demás importa de aquí."""
2  import os
3  from sqlalchemy import create_engine
4  from sqlalchemy.orm import sessionmaker, DeclarativeBase
5
6
7  class Base(DeclarativeBase):
8      pass
9
10
11
12  # Ruta absoluta anclada a backend/, para que sqlite apunte siempre al mismo
13  # archivo sin importar desde que carpeta se ejecute (uvicorn corre desde
14  # backend/, pero cargar_excel.py corre desde data/).
15  _RUTA_SQLITE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "cuentavoz.db")
16  DB_URL = os.getenv("DB_URL", f"sqlite:///{_RUTA_SQLITE}")
17  # Render (como antes Heroku) entrega la cadena de Postgres con el esquema
18  # viejo "postgres://"; SQLAlchemy 2.x ya no lo traduce solo y falla al
19  # arrancar si no se corrige aquí.
20  if DB_URL.startswith("postgres://"):
21      DB_URL = DB_URL.replace("postgres://", "postgresql://", 1)
22
23  # SQLite necesita este argumento extra cuando hay varios hilos (uvicorn)
24  kwargs = {"connect_args": {"check_same_thread": False}} if DB_URL.startswith("sqlite") else {}
25  motor = create_engine(DB_URL, **kwargs)
26  Sesion = sessionmaker(bind=motor, expire_on_commit=False)
27
28
29  def iniciar_bd():
30      import modelos # noqa: F401 (registra todas las tablas)
31      Base.metadata.create_all(motor)
32      _migrar_columnas_faltantes()
33      _migrar_columnas_ya_opcionales()
34
35
36  def _migrar_columnas_faltantes():
37      """create_all() solo crea tablas nuevas, nunca agrega columnas a una
38      tabla que ya existe. Sin esto, una columna agregada al modelo (como
39      Usuario.foto) se ve localmente porque cuentavoz.db se recrea facil,
40      pero nunca llega a la base de Postgres ya desplegada en Render - hay
41      que agregarla a mano en la tabla que ya esta viva. Corre en cada
42      arranque y no hace nada si ya esta al dia."""
43      from sqlalchemy import inspect, text
44      inspector = inspect(motor)
45      with motor.begin() as conexion:
46          for tabla in Base.metadata.sorted_tables:
47              if not inspector.has_table(tabla.name):
48                  continue
49              existentes = {c["name"] for c in inspector.get_columns(tabla.name)}
50              for columna in tabla.columns:
51                  if columna.name in existentes:
52                      continue
53                  tipo = columna.type.compile(dialect=motor.dialect)
54                  conexion.execute(text(
55                      f'ALTER TABLE {tabla.name} ADD COLUMN {columna.name} {tipo}'))
56                  print(f"[bd] columna agregada: {tabla.name}.{columna.name}")
57
58
59  def _migrar_columnas_ya_opcionales():
60      """Quita el NOT NULL de las columnas que el modelo ya declara opcionales.
61
62      _migrar_columnas_faltantes solo AGREGA columnas; nunca relaja una
63      restriccion existente. Eso dejo un fallo real y silencioso en
64      produccion: al pasar la identidad a Cognito, Usuario.clave_hash se
65      volvio nullable=True en el modelo (ya nadie la llena, la clave la
66      guarda Cognito), pero la tabla de Postgres seguia con el NOT NULL de
67      cuando se creo. En local no se noto porque cuentavoz.db se recrea de
68      cero; en Render, CADA insercion de usuario fallaba - autoregistro y
69      "crear usuario" desde Ajustes - con un 500 que ademas llegaba al
70      navegador sin cabeceras CORS, o sea disfrazado de "Sin conexion con el
71      servidor. Revise el Wi-Fi".

```

```

72
73 Solo en Postgres: SQLite no sabe hacer ALTER COLUMN, y alla no hace
74 falta. Nunca toca la llave primaria, que debe seguir siendo NOT NULL.
75 """
76 if not motor.dialect.name.startswith("postgres"):
77     return
78 from sqlalchemy import inspect, text
79 inspector = inspect(motor)
80 with motor.begin() as conexion:
81     for tabla in Base.metadata.sorted_tables:
82         if not inspector.has_table(tabla.name):
83             continue
84         en_bd = {c["name"]: c for c in inspector.get_columns(tabla.name)}
85         for columna in tabla.columns:
86             actual = en_bd.get(columna.name)
87             if actual is None or columna.primary_key:
88                 continue
89             # la base exige valor pero el modelo ya dice que es opcional
90             if columna.nullable and not actual.get("nullable", True):
91                 conexion.execute(text(
92                     f'ALTER TABLE {tabla.name} '
93                     f'ALTER COLUMN {columna.name} DROP NOT NULL'))
94                 print(f"[bd] ahora opcional: {tabla.name}.{columna.name}")

```

backend/seguridad.py

168 LÍNEAS

Verificacion del access token de Cognito y el gate por perfil.

```

1  """Identidad: verificacion de tokens de AWS Cognito y permisos por perfil.
2
3  La contraseña, el registro y el login los maneja Cognito directamente (el
4  frontend habla con el SDK de Cognito, nunca manda la clave a este backend).
5  Lo unico que hace este modulo es comprobar que un access token que llega en
6  la cabecera Authorization de verdad lo firmo NUESTRO User Pool de Cognito -
7  firma (RS256, con las llaves publicas que Cognito publica), emisor,
8  audiencia (client_id) y que sea un access token, no un id token."""
9  import os
10 import jwt
11 from fastapi import Depends, HTTPException
12 from fastapi.security import OAuth2PasswordBearer
13 from bd import Sesion
14 from modelos import Usuario
15
16 # El User Pool Id y el App Client Id NO son secretos: identifican al pool,
17 # no autorizan nada, y de hecho ya viajan dentro del JavaScript que
18 # descarga cualquiera que abra la aplicacion (ver frontend/src/cognito.js,
19 # donde llevan exactamente estos mismos valores por defecto). Lo secreto
20 # son las claves de las personas (que las guarda Cognito, no este backend)
21 # y las credenciales AWS_* (que siguen siendo solo variables de entorno).
22 #
23 # Llevan valor por defecto por la misma razon que VITE_API_URL en api.js o
24 # DB_URL en bd.py: que el proyecto funcione sin depender de que alguien se
25 # acuerde de llenar una variable. Antes el backend era la unica pieza que
26 # NO lo hacia, y el resultado fue un despliegue en el que estas dos
27 # quedaron vacias y TODOS los tokens se rechazaron con "Sesion invalida o
28 # vencida." - un mensaje que ademas culpaba a la persona equivocada.
29 # Definir la variable de entorno sigue mandando: apuntar a otro User Pool
30 # (otro entorno, otra cuenta de AWS) es solo ponerla.
31 COGNITO_REGION = os.getenv("COGNITO_REGION", "").strip() or "us-east-2"
32 COGNITO_USER_POOL_ID = os.getenv("COGNITO_USER_POOL_ID", "").strip() or "us-east-2_6HbrPruvL"
33 COGNITO_APP_CLIENT_ID = os.getenv("COGNITO_APP_CLIENT_ID", "").strip() or "847jtkc5sem7mr4tb8csrrkqp"
34 _EMISOR = f"https://cognito-idp.{COGNITO_REGION}.amazonaws.com/{COGNITO_USER_POOL_ID}"
35 _JWKS_URL = f"{_EMISOR}/.well-known/jwks.json"
36
37 # PyJWKClient trae en cache las llaves publicas del User Pool (se refrescan
38 # solas si aparece un "kid" nuevo que no conoce todavia) - sin esto habria
39 # que ir a buscarlas a Cognito en cada peticion.
40 # lifespan: las llaves de firma de un User Pool son estables (no rotan
41 # cada rato). Con el valor por defecto (300 s) el backend volvía a
42 # pedir el JWKS a AWS cada 5 minutos, y CADA una de esas idas a la red
43 # era una oportunidad de que un tropiezo tumbara la sesion de todos
44 # (ver _reclamos_cognito). Una hora reduce esas idas ~12 veces.
45 # timeout: 30 s por defecto es demasiado - deja la peticion colgada. Es
46 # preferible fallar rapido y reintentar.
47 _jwks_client = (jwt.PyJWKClient(_JWKS_URL, cache_keys=True, lifespan=3600, timeout=8)
48                 if COGNITO_USER_POOL_ID else None)
49 esquema = OAuth2PasswordBearer(tokenUrl="/api/registro-completado", auto_error=False)
50
51
52 class ServicioIdentidadCaído(Exception):
53     """No se pudieron consultar las llaves publicas de Cognito.
54
55     Ojo con la distincion, que es la razon de ser de esta excepcion: esto

```

```

56 NO significa que el token sea malo, significa que este backend no pudo
57 hablar con AWS para comprobarlo. Antes ambos casos terminaban en el
58 mismo 401 "Sesion invalida o vencida.", y como el frontend cierra la
59 sesion ante cualquier 401 (ver api.js: alSesionInvalida), un tropiezo
60 de red de un segundo contra el JWKS sacaba a la persona de la
61 aplicacion y la mandaba de vuelta al login con un mensaje que ademas
62 era mentira. Se responde 503 en su lugar.""
63
64
65 def _llave_de_firma(token: str):
66     """La llave publica con la que Cognito firmo este token.
67
68     PyJWT ya reintenta solo (refrescando el JWKS) cuando el "kid" no esta
69     en la cache; lo que no reintenta es un fallo de RED al traer ese JWKS,
70     que es justo el caso que aqui interesa cubrir."""
71     from jwt.exceptions import PyJWKClientConnectionError
72     try:
73         return _jwks_client.get_signing_key_from_jwt(token).key
74     except PyJWKClientConnectionError as e:
75         print(f"[seguridad] no se pudo traer el JWKS de Cognito ({e}); reintentando")
76         try:
77             return _jwks_client.get_signing_key_from_jwt(token).key
78         except PyJWKClientConnectionError as e2:
79             raise ServicioIdentidadCaído(str(e2)) from e2
80
81
82 def _reclamos_cognito(token: str) -> dict | None:
83     """Verifica un access token de Cognito y devuelve sus datos, o None si
84     no es valido, vencio, es de otro User Pool/App Client, o es un id token
85     en vez de un access token (llevan proposito distinto: el access token
86     es para hablar con la API, el id token es para identificar a la
87     persona ante el propio frontend).
88
89     Levanta ServicioIdentidadCaído si el problema no es el token sino la
90     conexion con Cognito."""
91     if not _jwks_client:
92         return None
93     try:
94         llave = _llave_de_firma(token)
95         datos = jwt.decode(
96             token, llave, algorithms=["RS256"], issuer=_EMISOR,
97             options={"require": ["exp", "iss", "sub", "token_use", "client_id", "username"]},
98         )
99     except jwt.PyJWTError:
100         return None
101     if datos.get("token_use") != "access" or datos.get("client_id") != COGNITO_APP_CLIENT_ID:
102         return None
103     return datos
104
105
106 def usuario_actual(token: str = Depends(esquema)) -> Usuario:
107     if not token:
108         raise HTTPException(401, "Falta la sesion.")
109     try:
110         datos = _reclamos_cognito(token)
111     except ServicioIdentidadCaído as e:
112         # 503 y no 401 a proposito: el token esta bien, lo que fallo fue la
113         # consulta a Cognito. Un 401 aqui cerraria la sesion de la persona
114         # (ver api.js) por un problema que no es suyo ni de su token.
115         print(f"[seguridad] Cognito no responde: {e}")
116         raise HTTPException(
117             503, "No se pudo verificar su sesion en este momento. Intente de nuevo.")
118     if datos is None:
119         raise HTTPException(401, "Sesion invalida o vencida.")
120     nombre = (datos.get("username") or "").lower()
121     with Sesión() as s:
122         u = s.query(Usuario).filter_by(nombre=nombre).first()
123     if u is None:
124         raise HTTPException(401, "Usuario no reconocido.")
125     if not u.activo:
126         raise HTTPException(403, "Ese usuario esta inactivo.")
127     return u
128
129
130 def verificar_token(token: str):
131     """Para WebSockets: el esquema OAuth2 exige el header Authorization, que
132     un WebSocket del navegador no puede mandar, asi que aqui se valida el
133     token pasado por query string con la misma regla que usuario_actual."""
134     if not token:
135         return None
136     try:
137         datos = _reclamos_cognito(token)
138     except ServicioIdentidadCaído as e:
139         # Un WebSocket no tiene forma de decir "reintente": se rechaza la
140         # conexion y el frontend la vuelve a abrir sola mas adelante.
141         print(f"[seguridad] Cognito no responde (websocket): {e}")

```

```

142         return None
143     if datos is None:
144         return None
145     nombre = (datos.get("username") or "").lower()
146     with Sesion() as s:
147         u = s.query(Usuario).filter_by(nombre=nombre).first()
148     if u is None or not u.activo:
149         return None
150     return u
151
152
153 def requiere_perfil(perfil: str):
154     def guardia(u: Usuario = Depends(usuario_actual)) -> Usuario:
155         if u.perfil != perfil:
156             raise HTTPException(403, "Su perfil no permite esta accion.")
157         return u
158     return guardia
159
160
161 def registrar(usuario, accion: str, detalle: str, tipo: str = "info"):
162     """Deja rastro de una accion sensible. El registro solo crece."""
163     from modelos import Traza
164     with Sesion() as s:
165         s.add(Traza(usuario_id=getattr(usuario, "id", None),
166                    persona=getattr(usuario, "nombre", "sistema"),
167                    accion=accion, detalle=detalle, tipo=tipo))
168     s.commit()

```

backend/reportes.py

86 LÍNEAS

Armado de los archivos XLSX con los codigos oficiales.

```

1  """El endpoint del flujo: informacion limpia lista para el ERP."""
2  import pandas as pd
3  from sqlalchemy import text
4  from bd import motor
5  from horario import ahora
6  from servicios.archivos import guardar_df
7
8  # Los nombres de columna son a proposito los mismos que trae el extracto
9  # real de Colsubsidio (ver data/BODEGAS_Y_STOCK.xlsx: "Nr.Articulo",
10 # "Articulo", "Unidad", "SD") - Bodega/Contado/Diferencia son las unicas
11 # columnas nuevas. Asi el archivo que se descarga aqui se reconoce de una
12 # en My Inventory (Oracle) en vez de traer nombres genericos inventados.
13 CONSULTA = """
14 SELECT a.codigo AS "Nr.Artículo", a.nombre_oficial AS "Artículo",
15        a.unidad_medida AS "Unidad", b.nombre_oficial AS "Bodega",
16        c.cantidad AS "Contado", st.cantidad_sd AS "SD",
17        c.cantidad - COALESCE(st.cantidad_sd, 0) AS "Diferencia"
18 FROM conteo c
19 JOIN articulo a ON a.codigo = c.articulo_codigo
20 JOIN sesion_conteo sc ON sc.id = c.sesion_id
21 JOIN bodega b ON b.id = sc.bodega_id
22 LEFT JOIN stock_sistema st ON st.articulo_codigo = a.codigo
23     AND st.bodega_id = b.id
24 WHERE c.estado = 'confirmado'
25 """
26
27
28 def consolidado(formato: str = "xlsx") -> tuple[str, int, list[dict]]:
29     df = pd.read_sql(CONSULTA, motor)
30     # "SD" sale NULL/NaN cuando el articulo no tiene fila de stock en esa
31     # bodega (LEFT JOIN); NaN no es JSON valido para la vista previa, y
32     # aqui significa lo mismo que en el resto de la app: no hay dato, se
33     # trata como 0.
34     df["SD"] = df["SD"].fillna(0)
35     df["Diferencia"] = df["Diferencia"].round(2)
36     marca = ahora().strftime("%Y%m%d_%H%M")
37     ruta = f"reportes/consolidado_{marca}.{formato}"
38     guardar_df(ruta, df, formato)
39     vista_previa = df.head(8).to_dict("records")
40     return ruta, len(df), vista_previa
41
42
43 def diferencias_archivo(formato: str = "xlsx") -> tuple[str, int, int]:
44     """«Diferencias por bodega» descargable: solo las filas donde el
45     conteo no cuadra con el sistema, no el consolidado completo."""
46     df = pd.read_sql(CONSULTA, motor)
47     df["Diferencia"] = df["Diferencia"].round(2)
48     df = df[df["Diferencia"] != 0]
49     marca = ahora().strftime("%Y%m%d_%H%M")
50     ruta = f"reportes/diferencias_{marca}.{formato}"
51     guardar_df(ruta, df, formato)

```

```

52     return ruta, len(df), df["Bodega"].nunique() if len(df) else 0
53
54
55 def detalle_bodega(bodega_id: int, formato: str = "xlsx") -> str:
56     """«Descargar detalle»: el conteo completo de una sola bodega."""
57     # text() - no un str plano - porque un str plano hace que pandas use
58     # exec_driver_sql() (ejecuta tal cual contra el driver, sin pasar por
59     # la traducción de parametros de SQLAlchemy): en SQLite ":bodega_id"
60     # funciona porque el driver sqlite3 lo entiende nativo, pero en
61     # Postgres (psycopg2, que solo entiende %(nombre)s) ese ":bodega_id"
62     # llega como texto literal invalido y revienta con 500 - paso
63     # solo detectable en produccion, no en la base local.
64     df = pd.read_sql(text(CONSULTA + " AND b.id = :bodega_id"),
65                     motor, params={"bodega_id": bodega_id})
66     df["Diferencia"] = df["Diferencia"].round(2)
67     marca = ahora().strftime("%Y%m%d_%H%M")
68     ruta = f"reportes/bodega_{bodega_id}_{marca}.{formato}"
69     guardar_df(ruta, df, formato)
70     return ruta
71
72
73 ESTADO_BODEGAS = """
74 SELECT nombre_oficial AS bodega, estado
75 FROM bodega ORDER BY nombre_oficial
76 """
77
78
79 def estado_bodegas(formato: str = "xlsx") -> str:
80     """«Exportar estado»: la foto del tablero en vivo, para mandarla por
81     correo o adjuntarla sin tener que tomar una captura de pantalla."""
82     df = pd.read_sql(ESTADO_BODEGAS, motor)
83     marca = ahora().strftime("%Y%m%d_%H%M")
84     ruta = f"reportes/estado_bodegas_{marca}.{formato}"
85     guardar_df(ruta, df, formato)
86     return ruta

```

backend/horario.py

19 LÍNEAS

La hora local de la operacion, en un solo sitio.

```

1  """La hora de Bogotá para toda la aplicación.
2
3  El servidor (Render) corre en UTC, pero CuentaVoz es una sola operación
4  en Colombia: lo que se guarda en la base (creado, resuelto, inicio, fin...)
5  y lo que se muestra (trazabilidad, alertas, reportes) debe leerse en hora
6  de Bogotá, no en la del contenedor - si no, cada hora mostrada queda
7  adelantada 5 horas (confirmado en producción: un reporte generado a las
8  2:56pm quedó con el nombre de archivo en 19:56)."""
9  from datetime import datetime
10 from zoneinfo import ZoneInfo
11
12 _BOGOTA = ZoneInfo("America/Bogota")
13
14
15 def ahora() -> datetime:
16     """La hora de Bogotá ahora mismo, sin info de zona (naive) - para
17     seguir siendo comparable con las columnas datetime ya guardadas y con
18     el resto del código, que nunca maneja zonas horarias explícitas."""
19     return datetime.now(_BOGOTA).replace(tzinfo=None)

```

17.2 Backend · servicios

backend/servicios/interprete.py

147 LÍNEAS

El interprete local: numeros en palabras, unidades e intenciones. Es el que permite que la plataforma funcione sin Gemini.

```

1  """Interprete local: permite demostrar el flujo aun sin llave de Gemini.
2
3  No reemplaza al modelo. Reconoce numeros en palabras y las intenciones
4  mas frecuentes, para que la demo nunca se caiga por falta de internet.
5  """
6  import re
7
8  NUMEROS = {
9      "cero": 0, "un": 1, "una": 1, "uno": 1, "dos": 2, "tres": 3, "cuatro": 4,
10     "cinco": 5, "seis": 6, "siete": 7, "ocho": 8, "nueve": 9, "diez": 10,

```

```

11 "once": 11, "doce": 12, "trece": 13, "catorce": 14, "quince": 15,
12 "dieciseis": 16, "dieciséis": 16, "diecisiete": 17, "dieciocho": 18,
13 "diecinueve": 19, "veinte": 20, "veinticinco": 25, "treinta": 30,
14 "cuarenta": 40, "cincuenta": 50, "sesenta": 60, "setenta": 70,
15 "ochenta": 80, "noventa": 90, "cien": 100, "ciento": 100,
16 "doscientos": 200, "doscientas": 200, "trescientos": 300, "trescientas": 300,
17 "cuatrocientos": 400, "cuatrocientas": 400, "quinientos": 500, "quinientas": 500,
18 "seiscientos": 600, "seiscientas": 600, "setecientos": 700, "setecientas": 700,
19 "ochocientos": 800, "ochocientas": 800, "novecientos": 900, "novecientas": 900,
20 "mil": 1000,
21 }
22 UNIDADES = {
23     "kilo": "Kilogram", "kilos": "Kilogram", "kilogramo": "Kilogram",
24     "kilogramos": "Kilogram", "kg": "Kilogram",
25     "litro": "Liter", "litros": "Liter", "l": "Liter",
26     "unidad": "Unidad", "unidades": "Unidad",
27     "porcion": "Portion", "porciones": "Portion", "porción": "Portion",
28 }
29
30
31 DECENAS = (20, 30, 40, 50, 60, 70, 80, 90)
32
33
34 def _numero(texto: str):
35     """La PRIMERA cantidad de la frase. Ignora numeros que son parte
36     del nombre del producto («cazuela 16 onzas», «tabla 50x38»)."""
37     t = texto.lower()
38     m = re.search(r"-?\d+(?:[.]\d+)?", t)
39     # un dígito pegado a un guion que sigue con otro caracter (ACIDO
40     # POLIGLICOLICO "3-0", REVOLUTION "2.6-5 KG", articulos reales del
41     # catalogo) es una talla o un codigo del propio articulo, no la
42     # cantidad dictada - sin este chequeo, "hay veinte del acido
43     # poliglicolico 3-0" leia el "3" del nombre del producto en vez del
44     # "veinte" que de verdad se dicto, silenciosamente y sin ningun error.
45     if m and t[m.end():m.end() + 1] != "-":
46         val = float(m.group(0).replace(".", "."))
47         return -val if "menos" in t[:m.start()] else val
48
49     palabras = [p.strip(".", "?", "!", ",") for p in t.split()]
50     total = None
51     for i, p in enumerate(palabras):
52         if p not in NUMEROS:
53             continue
54         v = NUMEROS[p]
55         if total is None:
56             total = v
57             # «ciento ochenta»: centena seguida de decena
58             if total >= 100 and i + 1 < len(palabras) and palabras[i + 1] in NUMEROS:
59                 sig = NUMEROS[palabras[i + 1]]
60                 if sig < 100:
61                     total += sig
62             # «treinta y cinco»: decena + y + unidad
63             elif total in DECENAS and i + 2 < len(palabras) \
64                 and palabras[i + 1] == "y" and palabras[i + 2] in NUMEROS:
65                 total += NUMEROS[palabras[i + 2]]
66             break
67         # lo demas pertenece al nombre del producto
68     if total is not None and "menos" in t:
69         total = -abs(total)
70     return total
71
72 def _unidad(texto: str):
73     for p in texto.lower().replace("?", " ").split():
74         p = p.strip(".", ",")
75         if p in UNIDADES:
76             return UNIDADES[p]
77     return None
78
79
80 CANONICAS = {"Kilogram", "Liter", "Unidad", "Portion"}
81
82
83 def normalizar_unidad(dicha):
84     """Lo que sea que devuelva el agente (humano o IA) para la unidad,
85     lo lleva a uno de los 4 valores canonicos de la base de datos.
86     Gemini a veces repite la palabra tal cual la dijo la persona
87     ("kilos") en vez de traducirla ("Kilogram"); esto lo corrige aqui,
88     sin depender de que el modelo obedezca el prompt al pie de la letra."""
89     if not dicha:
90         return dicha
91     if dicha in CANONICAS:
92         return dicha
93     return UNIDADES.get(str(dicha).strip().lower(), dicha)
94
95
96 def interpretar_local(frase: str) -> dict:

```

```

97 f = frase.lower().strip()
98 cant = _numero(f)
99 uni = _unidad(f)
100
101 def sin_ruido(t):
102     t = re.sub(r"-?\d+(?:[.]\d+)?", " ", t)
103     fuera = set(NUMEROS) | set(UNIDADES) | {
104         "de", "del", "la", "el", "los", "las", "hay", "en", "y", "por",
105         "confirmo", "confirmar", "son", "hoy", "preparamos", "quiero",
106         "pedir", "insumos", "para", "cuanto", "cuánto", "cuantos",
107         "iniciar", "conteo", "abrir", "bodega", "corregir", "ultimo",
108         "último", "no", "uy", "que", "qué", "me", "genera", "generame",
109         "consolidado", "reporte", "ayuda", "menos", "onzas", "onza",
110     }
111     return " ".join(w for w in t.split() if w.strip(".", "?", "!", ",") not in fuera).strip()
112
113 # — intenciones —
114 if any(k in f for k in ("iniciar conteo", "abrir bodega", "abrir la bodega")):
115     return {"intencion": "navegar", "bodega_texto": sin_ruido(f),
116             "respuesta_hablada": "Voy a abrir esa bodega."}
117
118 if any(k in f for k in ("preparamos", "vamos a preparar", "pedir insumos")):
119     return {"intencion": "pedir", "preparacion": sin_ruido(f),
120             "porciones": int(cant) if cant else 0,
121             "respuesta_hablada": "Le calculo los insumos segun la receta."}
122
123 if f.startswith("confirmo") or f in ("confirmar", "si", "sí", "correcto", "confirmo."):
124     return {"intencion": "confirmar", "respuesta_hablada": "Listo."}
125
126 if any(k in f for k in ("corregir", "no son", "no, son", "uy no", "uy, no",
127                         "esta mal", "está mal", "me equivoque", "cambiar")):
128     return {"intencion": "corregir", "cantidad": cant, "unidad": uni,
129             "respuesta_hablada": "Corrijo ese registro."}
130
131 if any(k in f for k in ("cuanto", "cuánto", "cuantos", "donde hay", "dónde hay")):
132     return {"intencion": "consultar", "articulo_texto": sin_ruido(f),
133             "respuesta_hablada": "Reviso el inventario."}
134
135 if any(k in f for k in ("reporte", "consolidado", "archivo", "imprim")):
136     return {"intencion": "reporte", "respuesta_hablada": "Genero el archivo."}
137
138 if "ayuda" in f or "no entiendo" in f:
139     return {"intencion": "ayuda",
140             "respuesta_hablada": "Con gusto. ¿Que necesita saber?"}
141
142 if cant is not None:
143     return {"intencion": "contar", "articulo_texto": sin_ruido(f),
144             "cantidad": cant, "unidad": uni, "respuesta_hablada": ""}
145
146 return {"intencion": "ayuda",
147         "respuesta_hablada": "Perdon, no le entendi bien. ¿Me lo repite?"}

```

backend/servicios/recetas.py

182 LÍNEAS

Calculo del pedido desde la receta y comparacion de la legalizacion.

```

1  """Momentos 1 y 3 del reto: pedido por receta y legalizacion del servicio."""
2  import unicodedata
3  from datetime import timedelta
4  from bd import Sesion
5  from horario import ahora
6  from modelos import (Receta, RecetaIngrediente, Articulo,
7                       StockSistema, LineaServicio)
8
9
10 def _sin_tildes(t: str) -> str:
11     """«santafereno» dicho o escrito sin tilde («santafereno») no debe
12     fallar la busqueda - un LIKE normal en SQLite si distingue ñ de N, y
13     eso ya causo un "no encuentre la receta" con un plato que si existia."""
14     t = str(t or "").upper().strip()
15     return "".join(c for c in unicodedata.normalize("NFD", t)
16                   if unicodedata.category(c) != "Mn")
17
18
19 def _sin_plural(t: str) -> str:
20     """El chef siempre nombra el plato en plural («cincuenta ajiacos») pero
21     la receta esta guardada en singular («AJIACO SANTA FERENO»), asi que la
22     comparacion por substring nunca coincidía: la frase de ejemplo que la
23     propia pantalla de Pedidos sugiere contestaba «no encuentre una receta».
24     Se normaliza cada palabra por separado para que «ajiacos santaferenos»
25     tambien caiga en la misma forma que el nombre guardado."""
26     palabras = []
27     for p in t.split():

```

```

28         if p.endswith("ES") and len(p) > 4:
29             palabras.append(p[:-2])
30         elif p.endswith("S") and len(p) > 3:
31             palabras.append(p[:-1])
32         else:
33             palabras.append(p)
34     return " ".join(palabras)
35
36
37 def _buscar_receta(s, preparacion: str):
38     objetivo = _sin_tildes(preparacion)
39     # menos de 3 letras no alcanza a identificar un plato - una transcripcion
40     # de voz recortada o con ruido ("a", "de", "o") coincidía igual como
41     # substring de cualquier receta y la calculaba con total confianza, como
42     # si el chef de verdad hubiera dictado ese plato.
43     if len(objetivo) < 3:
44         return None
45     objetivo_sin_plural = _sin_plural(objetivo)
46     for r in s.query(Receta).all():
47         nombre = _sin_tildes(r.nombre)
48         if objetivo in nombre or objetivo_sin_plural in _sin_plural(nombre):
49             return r
50     return None
51
52
53 def calcular_pedido(preparacion: str, porciones: int, bodega_id: int) -> dict:
54     """La regla del reto: cantidad por porcion x porciones - lo que ya hay.
55
56     Si un ingrediente de la receta no esta en el catalogo, o esta pero esa
57     bodega nunca lo ha tenido registrado, no se salta en silencio: queda
58     en «faltantes_catalogo» / «sin_registro_bodega» para que la persona
59     se entere y no crea que el pedido esta completo cuando no lo esta."""
60     with Sesion() as s:
61         receta = _buscar_receta(s, preparacion)
62         if receta is None:
63             return {"lineas": [], "faltantes_catalogo": [],
64                     "sin_registro_bodega": [], "receta_encontrada": False}
65
66         ings = s.query(RecetaIngrediente).filter_by(receta_id=receta.id).all()
67         lineas, faltantes, sin_registro = [], [], []
68         for ing in ings:
69             art = s.get(Articulo, ing.articulo_codigo)
70             if art is None:
71                 faltantes.append(ing.articulo_codigo)
72                 continue
73             necesario = round(ing.cantidad_por_porcion * porciones, 3)
74             stock = s.query(StockSistema).filter_by(
75                 articulo_codigo=art.codigo, bodega_id=bodega_id).first()
76             if stock is None:
77                 sin_registro.append(art.nombre_oficial)
78             hay = stock.cantidad_sd if stock else 0
79             falta = max(0, round(necesario - hay, 3))
80             lineas.append({"codigo": art.codigo, "nombre": art.nombre_oficial,
81                           "unidad": art.unidad_medida, "necesario": necesario,
82                           "stock": hay, "falta": falta,
83                           "sin_registro_bodega": stock is None})
84     return {"lineas": lineas, "faltantes_catalogo": faltantes,
85           "sin_registro_bodega": sin_registro, "receta_encontrada": True}
86
87
88 def detalle_receta(preparacion: str) -> dict:
89     """Solo consulta: cómo está armada la receta, sin tocar stock."""
90     with Sesion() as s:
91         receta = _buscar_receta(s, preparacion)
92         if receta is None:
93             return {}
94         ings = s.query(RecetaIngrediente).filter_by(receta_id=receta.id).all()
95         lineas, faltantes = [], []
96         for ing in ings:
97             art = s.get(Articulo, ing.articulo_codigo)
98             if art is None:
99                 faltantes.append(ing.articulo_codigo)
100             continue
101             lineas.append({"codigo": art.codigo, "nombre": art.nombre_oficial,
102                           "unidad": art.unidad_medida,
103                           "por_porcion": ing.cantidad_por_porcion})
104     return {"id": receta.id, "nombre": receta.nombre, "rendimiento": receta.rendimiento,
105           "preparacion": receta.preparacion or "", "lineas": lineas,
106           "faltantes_catalogo": faltantes}
107
108
109 def _filas_a_legalizar(s, servicio_id: int):
110     """Las lineas "abierto" de un servicio pueden venir de mas de un
111     pedido (dos platos distintos pedidos el mismo dia, ninguno legalizado
112     todavia) - mezclarlas todas en una sola comparacion duplicaba
113     articulos compartidos entre platos (ej. papa criolla en ajiaco Y en

```

```

114     sancocho) bajo el nombre de uno solo, sin avisar que en realidad eran
115     dos pedidos distintos. Se legaliza un pedido a la vez, empezando por
116     el mas reciente; el resto espera su turno para la proxima vez que se
117     entre a Legalizacion.""
118     filas = s.query(LineaServicio).filter_by(
119         servicio_id=servicio_id, estado="abierto").all()
120     numeros = [f.numero_pedido for f in filas if f.numero_pedido]
121     if numeros:
122         mas_reciente = max(numeros)
123         filas = [f for f in filas if f.numero_pedido == mas_reciente]
124     return filas
125
126
127 def comparar_legalizacion(servicio_id: int) -> dict:
128     """Lo pedido contra lo usado, con la lectura en palabras. Solo lo que
129     sigue abierto: si ya se legalizo antes (el servicio de ejemplo, u otro
130     pedido con el mismo numero de servicio) no debe mezclarse aqui ni
131     volver a legalizarse por accidente."""
132     with Sesion() as s:
133         filas = _filas_a_legalizar(s, servicio_id)
134         lineas = []
135         for l in filas:
136             dif = round((l.usado or 0) - (l.pedido or 0), 3)
137             if dif < 0:
138                 lectura = "Sobran: vuelve a bodega"
139             elif dif > 0:
140                 lectura = "Merma: revisar con el chef"
141             else:
142                 lectura = "Cuadro exacto"
143             art = s.get(Articulo, l.articulo_codigo)
144             lineas.append({"codigo": l.articulo_codigo, "nombre": l.nombre,
145                 "unidad": art.unidad_medida if art else "",
146                 "pedido": l.pedido, "usado": l.usado,
147                 "diferencia": dif, "lectura": lectura})
148             plato = next((f.plato for f in filas if f.plato), "")
149             porciones = next((f.porciones for f in filas if f.porciones), 0)
150             return {"lineas": lineas, "plato": plato, "porciones": porciones}
151
152
153 def analisis_consumo(dias: int = 30) -> dict:
154     """El «suma puntos» del reto: subutilizados y sobrepedido, acotado a
155     los ultimos N dias (por defecto 30, como en el tablero de Reportes)."""
156     desde = ahora() - timedelta(days=dias)
157     with Sesion() as s:
158         filas = (s.query(LineaServicio)
159             .filter(LineaServicio.estado == "legalizado",
160                 LineaServicio.creado >= desde).all())
161         total_ped = sum(f.pedido or 0 for f in filas)
162         total_uso = sum(f.usado or 0 for f in filas)
163         servicios = len({f.servicio_id for f in filas})
164         porart = {}
165         for f in filas:
166             d = porart.setdefault(f.nombre, {"pedido": 0, "usado": 0, "veces": 0})
167             d["pedido"] += f.pedido or 0
168             d["usado"] += f.usado or 0
169             d["veces"] += 1
170         subutil = []
171         for nombre, d in porart.items():
172             sobra = round(d["pedido"] - d["usado"], 3)
173             if sobra > 0:
174                 pct = round(sobra / d["pedido"] * 100) if d["pedido"] else 0
175                 subutil.append({"nombre": nombre, "sobra": sobra,
176                     "veces": d["veces"], "sobrepedido_pct": pct})
177         subutil.sort(key=lambda x: -x["sobrepedido_pct"])
178         ahorro_potencial = round(sum(s["sobra"] for s in subutil), 2)
179         return {"pedido_total": round(total_ped, 2), "usado_total": round(total_uso, 2),
180             "aprovechamiento": round(total_uso / total_ped * 100, 1) if total_ped else 0,
181             "servicios_periodes": servicios, "ahorro_potencial": ahorro_potencial,
182             "dias": dias, "subutilizados": subutil}

```

backend/servicios/conciliacion.py

191 LÍNEAS

Emparejar lo dictado con el catalogo, y las alertas cuando algo no cuadra.

```

1  """De «tabla para picar blanca» al nombre y codigo oficiales del catalogo."""
2  import re
3  import unicodedata
4  from rapidfuzz import process, fuzz
5  from bd import Sesion
6  from modelos import Articulo, AliasArticulo, Bodega
7
8
9  # palabras que no distinguen un producto de otro

```

```

10 VACIAS = {"PARA", "DE", "DEL", "LA", "EL", "LOS", "LAS", "CON", "SIN",
11           "POR", "EN", "Y", "A", "UN", "UNA", "UNAS", "UNOS", "AL"}
12
13
14 def _tokens(t: str) -> list[str]:
15     return [p for p in t.split() if len(p) >= 3 and p not in VACIAS]
16
17
18 def _cobertura(consulta: str, candidato: str) -> float:
19     """Cuantas palabras significativas de la consulta aparecen en el nombre.
20
21     Compara por raíz de 4 letras, para que BLANCA encuentre BLANCO y
22     CAZUELAS encuentre CAZUELA - en los DOS sentidos exige que la raíz
23     tenga esas 4 letras de verdad (no baste una palabra de 3): sin ese
24     piso en el lado del candidato, una palabra corta del catalogo como
25     "RES" (de "COSTILLA DE RES") volvía "raiz" a ella misma y cualquier
26     palabra dicha que empezara igual - como "RESTAURANTE" - contaba como
27     coincidencia; sin el mismo piso del lado de lo dicho, decir "ver"
28     (de una frase suelta como "sí para ver el detalle", no una búsqueda
29     de verdad) encontraba "VERDE"/"VERDURAS" en artículos reales sin
30     relación alguna. Una palabra de 3 letras solo cuenta si coincide
31     EXACTA con alguna del candidato."""
32     tc = _tokens(consulta)
33     if not tc:
34         return 0.0
35     tn = _tokens(candidato)
36     aciertos = 0
37     for p in tc:
38         raiz = p[:4]
39         if any(p == q or (len(p) >= 4 and q.startswith(raiz))
40               or (len(q) >= 4 and p.startswith(q[:4])) for q in tn):
41             aciertos += 1
42     return aciertos / len(tc) * 100
43
44
45 def puntuar(consulta: str, candidato: str) -> float:
46     """Combina la forma general con la cobertura de palabras clave.
47
48     Sin esto, «tabla para picar blanca» devuelve la AZUL: comparte
49     tabla-picar y el color se diluye."""
50     forma = fuzz.token_set_ratio(consulta, candidato)
51     cob = _cobertura(consulta, candidato)
52     return 0.45 * forma + 0.55 * cob
53
54
55 def normalizar(t: str) -> str:
56     t = str(t).upper().strip()
57     t = "".join(c for c in unicodedata.normalize("NFD", t)
58                if unicodedata.category(c) != "Mn") # sin tildes
59     t = t.replace("P/", "PARA ")
60     # "kiosco"/"kiosko" son la misma palabra, dicha o escrita de dos
61     # formas igual de comunes en español - sin esto, decir "kiosko" no
62     # encontraba "KIOSCO TAQUILLA AYB" aunque sea exactamente la misma
63     # bodega. La K es rarísima en el catálogo salvo en estos préstamos
64     # (kilo, kiosco...), así que unificarla con la C no choca con nada.
65     t = t.replace("K", "C")
66     # el reconocimiento de voz del navegador suele cerrar la frase con un
67     # punto ("Zoológico.") aunque nadie lo haya dicho - sin quitarlo, una
68     # bodega real dejaba de encontrarse por un simple "." de más.
69     t = re.sub(r"[.,;:!?;]+", " ", t)
70     return re.sub(r"\s+", " ", t).strip()
71
72
73 def buscar_bodegas_candidatas(s, texto: str, ids_permitidos: set[int] | None = None) -> list[Bodega]:
74     """Todas las bodegas razonables para lo dicho/escrito, de más a menos
75     específica - vacía si no hay ninguna, un solo elemento si es
76     inequívoco, varios si es ambiguo ("restaurante" a secas encaja por
77     igual en "RESTAURANTE FUENTES AYB" y "RESTAURANTE FUENTES SUMIN": ahí
78     quien llama debe ofrecer las opciones, no adivinar cuál de las dos).
79
80     ids_permitidos: si se da, restringe las coincidencias PARCIALES/
81     ambiguas (por donde se filtran nombres de bodegas ajenas que un
82     auxiliar ni debería ver sugeridas). Una coincidencia EXACTA nunca se
83     restringe: si dijo el nombre completo de una bodega real que no es
84     suya, debe decir "no está asignada a usted" - no fingir que no
85     existe y ofrecerle crear una bodega duplicada."""
86     clave = normalizar(texto)
87     if not clave:
88         return []
89
90     # El catálogo de bodegas es chico (unas pocas decenas): comparar en
91     # Python, con normalizar() de los DOS lados, es lo que permite que
92     # reglas como K->C funcionen igual para lo dicho y para el nombre
93     # guardado - comparar contra la columna cruda (como antes, con SQL)
94     # solo servía porque el nombre YA estaba guardado en mayúsculas sin
95     # tildes, por pura coincidencia; una regla nueva de normalizar() no

```

```

96 # se aplicaba del lado de la base de datos.
97 todas = s.query(Bodega).all()
98
99 def _permitidas():
100     if ids_permitidos is None:
101         return todas
102     return [b for b in todas if b.id in ids_permitidos]
103
104 exacta = next((b for b in todas if normalizar(b.nombre_oficial) == clave), None)
105 if exacta:
106     return [exacta]
107 # coincidencia parcial: "kiosco" a secas es substring de "KIOSCO 2
108 # SUMINISTROS", "KIOSCO TAQUILLA AYB" y varias más - todas cuentan.
109 contienen = [b for b in _permitidas() if clave in normalizar(b.nombre_oficial)]
110 if contienen:
111     return contienen
112 # ultimo recurso: cuantas palabras dichas (de mas de 3 letras) tiene
113 # cada bodega - "autoservicios las fuentes" perdía "fuentes" antes y
114 # se quedaba con la primera bodega que solo compartiera
115 # "autoservicios"; ahora esa comparte 1 palabra y "AUTOSERVICIOS LAS
116 # FUENTES" comparte 2, así que gana sola. Un empate real (dos
117 # candidatas con el mismo máximo) se devuelve como ambigüedad.
118 palabras = [p for p in clave.split() if len(p) > 3]
119 if not palabras:
120     return []
121 candidatas = [(sum(1 for p in palabras if p in normalizar(b.nombre_oficial)), b)
122               for b in _permitidas()]
123 candidatas = [(n, b) for n, b in candidatas if n > 0]
124 if not candidatas:
125     return []
126 maximo = max(n for n, _ in candidatas)
127 return [b for n, b in candidatas if n == maximo]
128
129
130 def buscar_bodega(s, texto: str, ids_permitidos: set[int] | None = None) -> Bodega | None:
131     """La misma bodega para el nombre dicho o escrito, en un solo lugar
132     (antes /api/bodegas/abrir y el agente conversacional cada uno tenía
133     su propia version, y se desincronizaban). Nunca adivina entre varias
134     candidatas igual de buenas: mejor decir "no la encuentro" (o, para
135     quien necesite las opciones, ver buscar_bodegas_candidatas) que abrir
136     la bodega equivocada."""
137     candidatas = buscar_bodegas_candidatas(s, texto, ids_permitidos)
138     if len(candidatas) == 1:
139         return candidatas[0]
140     if ids_permitidos is not None and not candidatas:
141         # nada dentro de lo permitido - pero si el nombre dicho identifica
142         # sin ambigüedad una bodega REAL en todo el catálogo, hay que
143         # decir que existe (y dejar que quien llama la rechace por no
144         # estar asignada), no fingir que no existe y ofrecer crear un
145         # duplicado de una bodega que ya está en el sistema.
146         sin_restriccion = buscar_bodegas_candidatas(s, texto, None)
147         if len(sin_restriccion) == 1:
148             return sin_restriccion[0]
149     return None
150
151
152 def buscar_articulo(texto: str, bodega_id=None, limite: int = 3) -> list[dict]:
153     consulta = normalizar(texto)
154     if not consulta:
155         return []
156     with Sesion() as s:
157         # 1) ¿ya aprendió como lo dice este equipo?
158         alias = s.query(AliasArticulo).filter_by(texto_dicho=consulta).first()
159         if alias:
160             a = s.get(Articulo, alias.articulo_codigo)
161             if a:
162                 return [{"codigo": a.codigo, "nombre": a.nombre_oficial,
163                       "unidad": a.unidad_medida, "confianza": 100}]
164         # 2) si no, compara contra todo el catalogo
165         arts = s.query(Articulo).all()
166
167     if not arts:
168         return []
169     puntajes = []
170     for a in arts:
171         p = puntuar(consulta, normalizar(a.nombre_oficial))
172         if p >= 45:
173             puntajes.append((p, a))
174     puntajes.sort(key=lambda z: -z[0])
175     return [{"codigo": a.codigo, "nombre": a.nombre_oficial,
176           "unidad": a.unidad_medida, "confianza": round(p)}
177           for p, a in puntajes[:limite]]
178
179
180 def aprender_alias(texto: str, codigo: str, bodega_id=None):
181     """Cada eleccion confirmada hace al agente mas preciso."""

```

```

182     consulta = normalizar(texto)
183     if not consulta or not codigo:
184         return
185     with Sesion() as s:
186         ya = s.query(AliasArticulo).filter_by(texto_dicho=consulta,
187                                             articulo_codigo=codigo).first()
188         if not ya:
189             s.add(AliasArticulo(texto_dicho=consulta,
190                               articulo_codigo=codigo, bodega_id=bodega_id))
191         s.commit()

```

backend/servicios/analitica.py

166 LÍNEAS

Los indicadores del Panel y el análisis de consumo.

```

1  """Las métricas del panel gerencial y de reportes: todo calculado en vivo
2  sobre la misma base que usa el resto de la aplicación – una sola fuente."""
3  from datetime import datetime
4  from bd import Sesion
5  from modelos import (Bodega, Articulo, StockSistema, SesionConteo, Conteo,
6                      Alerta, AliasArticulo, HistorialCierre, ConfigClave, Aprobacion)
7
8
9  def _config(clave: str, defecto: str) -> str:
10     with Sesion() as s:
11         c = s.get(ConfigClave, clave)
12         return c.valor if c else defecto
13
14
15  def diferencias_por_bodega(limite: int = 6) -> list[dict]:
16     """Suma de |contado - sistema| por bodega, sobre lo confirmado."""
17     with Sesion() as s:
18         bodegas = s.query(Bodega).all()
19         salida = []
20         for b in bodegas:
21             conteos = (s.query(Conteo).join(SesionConteo)
22                       .filter(SesionConteo.bodega_id == b.id,
23                               Conteo.estado == "confirmado").all())
24             if not conteos:
25                 continue
26             total = 0.0
27             for c in conteos:
28                 st = s.query(StockSistema).filter_by(
29                     articulo_codigo=c.articulo_codigo, bodega_id=b.id).first()
30                 esperado = st.cantidad_sd if st else 0
31                 total += abs((c.cantidad or 0) - esperado)
32             if total > 0:
33                 salida.append({"bodega": b.nombre_oficial, "diferencia": round(total, 1)})
34     salida.sort(key=lambda x: -x["diferencia"])
35     return salida[:limite]
36
37
38  def stock_por_unidad() -> list[dict]:
39     with Sesion() as s:
40         arts = {a.codigo: a.unidad_medida for a in s.query(Articulo).all()}
41         acumulado = {}
42         for st in s.query(StockSistema).all():
43             u = arts.get(st.articulo_codigo, "Otra")
44             acumulado[u] = acumulado.get(u, 0) + max(st.cantidad_sd or 0, 0)
45     total = sum(acumulado.values()) or 1
46     salida = [{"unidad": u, "cantidad": round(c, 1),
47              "pct": round(c / total * 100, 1)}
48              for u, c in acumulado.items()]
49     salida.sort(key=lambda x: -x["cantidad"])
50     return salida
51
52
53  def alertas_por_tipo() -> list[dict]:
54     etiquetas = {"desviacion": "Desviación", "unidad": "Unidad",
55                 "negativo": "Saldo negativo", "inexistente": "Inexistente"}
56     with Sesion() as s:
57         conteo = {}
58         for a in s.query(Alerta).all():
59             conteo[a.tipo] = conteo.get(a.tipo, 0) + 1
60     salida = [{"tipo": etiquetas.get(t, t), "cantidad": n} for t, n in conteo.items()]
61     salida.sort(key=lambda x: -x["cantidad"])
62     return salida
63
64
65  def descuadres_recurrentes(limite: int = 5) -> list[dict]:
66     """Un artículo con diferencia en más de una toma: ahí está la causa raíz."""
67     with Sesion() as s:
68         arts = {a.codigo: a.nombre_oficial for a in s.query(Articulo).all()}

```

```

69     # un articulo "PEND-..." ya aprobado o rechazado conserva ese mismo
70     # codigo para siempre (nunca se reemplaza por uno oficial al
71     # aprobarlo) - el prefijo solo no basta para saber si de verdad
72     # sigue pendiente; sin este chequeo, uno ya aprobado hace rato
73     # seguia sugiriendo "Crear en catálogo oficial" como si nunca se
74     # hubiera resuelto.
75     pendientes_de_verdad = {a.articulo_codigo for a in
76                             s.query(Aprobacion).filter_by(
77                                 tipo="producto", estado="pendiente").all()
78                             if a.articulo_codigo}
79     por_articulo = {}
80     conteos = (s.query(Conteo).join(SesionConteo)
81                .filter(Conteo.estado == "confirmado").all())
82     for c in conteos:
83         ses = s.get(SesionConteo, c.sesion_id)
84         st = s.query(StockSistema).filter_by(
85             articulo_codigo=c.articulo_codigo, bodega_id=ses.bodega_id).first()
86         esperado = st.cantidad_sd if st else 0
87         dif = (c.cantidad or 0) - esperado
88         if abs(dif) < 0.01:
89             continue
90         d = por_articulo.setdefault(c.articulo_codigo,
91                                     {"dif": 0.0, "tomas": set(), "negativo": False})
92         d["dif"] += dif
93         d["tomas"].add(c.sesion_id)
94         if esperado < 0:
95             d["negativo"] = True
96     salida = []
97     for cod, d in por_articulo.items():
98         if len(d["tomas"]) < 2 and not d["negativo"]:
99             continue
100         salida.append({
101             "articulo": arts.get(cod, cod),
102             "tipo": "Saldo negativo" if d["negativo"] else "Desviación",
103             "diferencia": round(d["dif"], 1),
104             "tomas": len(d["tomas"]),
105             "accion": "Revisar salidas sin entrada" if d["negativo"]
106                     else "Crear en catálogo oficial" if cod in pendientes_de_verdad
107                     else "Ajustar el saldo del sistema",
108         })
109     salida.sort(key=lambda x: -abs(x["diferencia"]))
110     return salida[:limite]
111
112 def historial_exactitud(limite: int = 12) -> list[dict]:
113     with Sesion() as s:
114         filas = (s.query(HistorialCierre).order_by(HistorialCierre.fecha).all())[-limite:]
115         nombres = {b.id: b.nombre_oficial for b in s.query(Bodega).all()}
116         return [{"id": f.id, "fecha": f.fecha.strftime("%Y-%m-%d"), "exactitud": round(f.exactitud, 1),
117                 "bodega_id": f.bodega_id, "bodega": nombres.get(f.bodega_id, "")} for f in filas]
118
119 def resumen_ejecutivo() -> dict:
120     with Sesion() as s:
121         bodegas = s.query(Bodega).all()
122         cerradas = [b for b in bodegas if b.estado == "cerrada"]
123         total_conteos = s.query(Conteo).filter_by(estado="confirmado").count()
124         alertas_total = s.query(Alerta).count()
125         alertas_resueltas = s.query(Alerta).filter_by(resuelta=1).count()
126         historial = s.query(HistorialCierre).all()
127         exact = (round(sum(h.exactitud for h in historial) / len(historial), 1)
128                  if historial else 100.0)
129     return {
130         "exactitud_primera_pasada": exact,
131         "referencias_contadas": total_conteos,
132         "alertas_gestionadas": alertas_resueltas,
133         "alertas_total": alertas_total,
134         "bodegas_cerradas": len(cerradas),
135         "bodegas_total": len(bodegas),
136         "diferencia_por_bodega": diferencias_por_bodega(limite=10),
137         "stock_por_unidad": stock_por_unidad(),
138         "historial_exactitud": historial_exactitud(),
139     }
140
141 def resumen_alertas_panel() -> dict:
142     with Sesion() as s:
143         negativos_actuales = s.query(StockSistema).filter(
144             StockSistema.cantidad_sd < 0).count()
145         alias = s.query(AliasArticulo).count()
146         bodegas = s.query(Bodega).all()
147         cerradas_ses = (s.query(SesionConteo).filter_by(tipo="conteo", estado="cerrada")
148                        .filter(SesionConteo.fin.isnot(None)).all())
149         minutos = [(ses.fin - ses.inicio).total_seconds() / 60
150                    for ses in cerradas_ses if ses.fin]
151         negativos_iniciales = int(_config("negativos_iniciales", str(negativos_actuales or 0)))

```

```

155 estado_bodegas = {}
156 for b in bodegas:
157     estado_bodegas[b.estado] = estado_bodegas.get(b.estado, 0) + 1
158 return {
159     "negativos_iniciales": negativos_iniciales,
160     "negativos_actuales": negativos_actuales,
161     "tiempo_promedio_min": round(sum(minutos) / len(minutos)) if minutos else 0,
162     "alias_aprendidos": alias,
163     "estado_bodegas": estado_bodegas,
164     "alertas_por_tipo": alertas_por_tipo(),
165     "descuadres_recurrentes": descuadres_recurrentes(),
166 }

```

backend/servicios/validacion.py

66 LÍNEAS

Las reglas que se aplican antes de guardar.

```

1  """Las reglas duras.Codigo determinista: se comporta igual siempre."""
2  import os
3  from bd import Sesion
4  from modelos import Articulo, StockSistema, ConfigClave
5
6  UMBRAL_DEFECTO = float(os.getenv("UMBRAL_ANOMALIA", 0.5))
7
8
9  def umbral_actual() -> float:
10     """El umbral puede ajustarse desde Ajustes; si no, manda el .env."""
11     with Sesion() as s:
12         c = s.get(ConfigClave, "umbral_anomalia")
13         return float(c.valor) if c else UMBRAL_DEFECTO
14
15
16  def validar_conteo(codigo: str, cantidad: float, unidad: str, bodega_id) -> dict:
17     UMBRAL = umbral_actual()
18     with Sesion() as s:
19         art = s.get(Articulo, codigo)
20         if art is None:
21             return {"ok": False, "tipo": "inexistente",
22                     "mensaje": "Ese articulo no esta en el catalogo. "
23                               "¿Lo creamos? Quedara pendiente de aprobacion."}
24
25         if cantidad is None:
26             return {"ok": False, "tipo": "vacio",
27                     "mensaje": "No entendi la cantidad. ¿Me la repite?"}
28
29         # mini reto de Colsubsidio: un saldo fisico nunca baja de cero
30         if cantidad < 0:
31             return {"ok": False, "tipo": "negativo",
32                     "mensaje": "Una cantidad no puede ser negativa: un saldo "
33                               "fisico nunca baja de cero. Digame el valor correcto."}
34
35         # «cinco kilos» no se confunde con «cinco gramos»
36         if unidad and unidad.strip().lower() != art.unidad_medida.strip().lower():
37             return {"ok": False, "tipo": "unidad",
38                     "mensaje": f"Ese articulo se maneja en {art.unidad_medida}, "
39                               f"no en {unidad}. ¿Cual es la cantidad "
40                               f"en {art.unidad_medida}?"}
41
42         stock = s.query(StockSistema).filter_by(
43             articulo_codigo=codigo, bodega_id=bodega_id).first()
44         esperado = stock.cantidad_sd if stock else None
45
46         # el famoso 9 contra 90
47         if esperado is not None and esperado > 0:
48             desvio = abs(cantidad - esperado) / esperado
49             if desvio > UMBRAL:
50                 return {"ok": False, "tipo": "desviacion", "esperado": esperado,
51                         "mensaje": f"El sistema espera alrededor de {esperado:g}. "
52                                   f"¿Confirma {cantidad:g}?"}
53         # el sistema SI tiene una expectativa aqui (una fila real de
54         # StockSistema, no la ausencia de dato que ya cubre "esperado is
55         # None" arriba) y esa expectativa es CERO - "esperado > 0" arriba
56         # evita dividir por cero para calcular el porcentaje de desviacion,
57         # pero de paso dejaba sin ninguna alerta el caso de contar algo
58         # donde el sistema no esperaba nada: encontrar existencia donde
59         # deberia haber cero es tan anomalo como el 9 contra 90 de arriba,
60         # solo que no tiene un "%" que calcular.
61         elif esperado == 0 and cantidad > 0:
62             return {"ok": False, "tipo": "desviacion", "esperado": esperado,
63                     "mensaje": f"El sistema no tiene existencia registrada aqui para este articulo. "
64                               f"¿Confirma {cantidad:g}?"}
65
66     return {"ok": True, "esperado": esperado}

```

backend/servicios/archivos.py

29 LÍNEAS

Escritura y lectura de los archivos generados.

```

1  """Guardar y leer los XLSX/CSV generados desde la base, no desde disco -
2  el disco del Web Service en Render es efimero (se borra en cada deploy o
3  reinicio), asi que un reporte "generado hace una hora" quedaria
4  inalcanzable despues del siguiente `git push`. La "ruta" (ej.
5  "reportes/consolidado_20260815_1636.xlsx") se sigue usando como
6  identificador legible en toda la app - solo cambia DONDE vive el
7  contenido real."""
8  import io
9
10 from bd import Sesion
11 from modelos import ArchivoGenerado
12
13
14 def guardar_df(ruta: str, df, formato: str) -> None:
15     buf = io.BytesIO()
16     if formato == "csv":
17         df.to_csv(buf, index=False)
18     else:
19         df.to_excel(buf, index=False)
20     with Sesion() as s:
21         s.add(ArchivoGenerado(ruta=ruta, contenido=buf.getvalue()))
22         s.commit()
23
24
25 def leer_bytes(ruta: str) -> bytes | None:
26     with Sesion() as s:
27         obj = (s.query(ArchivoGenerado).filter_by(ruta=ruta)
28             .order_by(ArchivoGenerado.id.desc()).first())
29         return obj.contenido if obj else None

```

17.3 Frontend · nucleo**frontend/src/main.jsx**

18 LÍNEAS

El punto de entrada.

```

1  import React from "react";
2  import { createRoot } from "react-dom/client";
3  import * as Sentry from "@sentry/react";
4  import App from "./App";
5  import "./index.css";
6
7  // Apagado por defecto (sin VITE_SENTRY_DSN, Sentry.init() nunca llama a
8  // ningun lado) - el ErrorBoundary de abajo es inofensivo sin DSN, solo no
9  // reporta nada. Activarlo en produccion es poner la variable de entorno,
10 // sin tocar codigo.
11 const dsn = import.meta.env.VITE_SENTRY_DSN;
12 if (dsn) Sentry.init({ dsn, tracesSampleRate: 0 });
13
14 createRoot(document.getElementById("root")).render(
15     <Sentry.ErrorBoundary fallback={<p style={{ padding: 20 }}>Ocurrió un error. Recargue la página.</p>>
16         <App />
17     </Sentry.ErrorBoundary>
18 );

```

frontend/src/App.jsx

271 LÍNEAS

El menu, la vista activa, la sesion y el contexto. Agregar una pantalla empieza aqui.

```

1  import { useEffect, useRef, useState } from "react";
2  import { alSesionInvalida, borrarSesion, guardarSesion, leerSesion, pedir } from "./api";
3  import { obtenerTokenValido, cerrarSesionLocal } from "./cognito";
4  import { configurarVoz, detenerVoz } from "./voz";
5  import { leerPreferencias, aplicarPreferencias } from "./accesibilidad";
6  import VideoRecorrido from "./VideoRecorrido";
7  import { debeAbrirTutorial, EVENTO_ABRIR_VIDEO } from "./tutorial";
8  import BarraLateral from "./BarraLateral";
9  import Ingreso from "./vistas/Ingreso";
10 import Inicio from "./vistas/Inicio";
11 import Conteo from "./vistas/Conteo";

```

```

12 import Pedido from "../vistas/Pedido";
13 import Legalizacion from "../vistas/Legalizacion";
14 import Bodegas from "../vistas/Bodegas";
15 import Auditoria from "../vistas/Auditoria";
16 import Reportes from "../vistas/Reportes";
17 import Panel from "../vistas/Panel";
18 import Ajustes from "../vistas/Ajustes";
19 import Ayuda from "../vistas/Ayuda";
20 import Mensajes from "../vistas/Mensajes";
21 import MiPerfil from "../vistas/MiPerfil";
22 import CerrarSesion from "../vistas/CerrarSesion";
23
24 /* El menú lateral. Cada entrada declara las vistas que contiene:
25 este objeto es el mapa de navegación hecho código. */
26 export const MENU = [
27   { id: "inicio",      titulo: "Inicio",      vistas: ["inicio"] },
28   { id: "pedidos",     titulo: "Pedidos",     vistas: ["pedido"] },
29   { id: "conteo",      titulo: "Conteo",      vistas: ["conteo"] },
30   { id: "legalizacion", titulo: "Legalización", vistas: ["legalizacion"] },
31   { id: "bodegas",     titulo: "Bodegas",     vistas: ["bodegas"] },
32   { id: "auditoria",   titulo: "Auditoria",   vistas: ["auditoria"], soloAuditor: true },
33   { id: "reportes",    titulo: "Reportes",    vistas: ["reportes"], soloAuditor: true },
34   { id: "panel",       titulo: "Panel",       vistas: ["panel"], soloAuditor: true },
35   // El auxiliar no puede cambiar nada aquí (el umbral y el modo sin
36   // conexión ya estaban bloqueados por el backend con requiere_perfil):
37   // mostrarle la pantalla igual, solo en modo lectura, dejaba ver ajustes
38   // administrativos que no le sirven de nada y no puede tocar - mejor
39   // ocultarla del todo, igual que ya se hace con Auditoria/Reportes/Panel.
40   { id: "ajustes",     titulo: "Ajustes",     vistas: ["ajustes"], soloAuditor: true },
41   { id: "ayuda",       titulo: "Ayuda",       vistas: ["ayuda"] },
42   { id: "mensajes",    titulo: "Mensajes",    vistas: ["mensajes"] },
43   { id: "perfil",      titulo: "Mi perfil",    vistas: ["perfil"] },
44   { id: "salir",       titulo: "Cerrar sesión", vistas: ["salir"] },
45 ];
46
47 const VISTAS = {
48   inicio: Inicio, pedido: Pedido, conteo: Conteo,
49   legalizacion: Legalizacion, bodegas: Bodegas, auditoria: Auditoria,
50   reportes: Reportes, panel: Panel, ajustes: Ajustes, ayuda: Ayuda,
51   mensajes: Mensajes, perfil: MiPerfil,
52 };
53
54 /** Qué entrada del menú resaltar para una vista. Garantiza que el menú
55     señale siempre dónde vive la pantalla que se está viendo. */
56 export function menuDe(vista) {
57   const g = MENU.find((m) => m.vistas.includes(vista));
58   return g ? g.id : null;
59 }
60
61 export default function App() {
62   // arranca directo con la sesión ya guardada (si hay) en vez de forzar
63   // login otra vez: sin esto, abrir la app sin señal (o reiniciarla en
64   // medio de una caída de Wi-Fi) dejaba a cualquiera sin poder entrar,
65   // aunque ya se hubiera identificado antes en este mismo equipo.
66   const [sesion, setSesion] = useState(() => leerSesion());
67   const [vista, setVista] = useState("inicio");
68   // Antes de abrir una bodega no hay todavía un id real de sesion de
69   // conteo (ver Conteo.jsx) - un marcador de posicion fijo (antes: 1 para
70   // todo el mundo) hacia que dos personas sin ninguna bodega abierta
71   // todavía pudieran mezclar su conversacion pendiente. Uno por usuario,
72   // igual que ya hace Pedido.jsx con el suyo.
73   const [ctx, setCtx] = useState(() => ({ sessionId: -2000 - (sesion?.usuario?.id || 0) }));
74   const [salir, setSalir] = useState(false);
75   const [avisoSesion, setAvisoSesion] = useState("");
76   const [mostrarVideo, setMostrarVideo] = useState(false);
77
78   // "Ver el recorrido en video" en Ayuda.jsx reabre el recorrido bajo
79   // demanda con un evento (mismo patrón que cuentavoz:foto-actualizada) en
80   // vez de pasar la función por props a través de todas las vistas.
81   useEffect(() => {
82     function abrirVideo() { setMostrarVideo(true); }
83     window.addEventListener(EVENTO_ABRIR_VIDEO, abrirVideo);
84     return () => window.removeEventListener(EVENTO_ABRIR_VIDEO, abrirVideo);
85   }, []);
86
87   // El video del recorrido abre en CADA ingreso, no solo el primero - lo
88   // único que lo apaga es que la propia persona lo desactive desde el
89   // video (ver VideoRecorrido.jsx). "Ingreso" no es solo el instante de
90   // escribir usuario y clave: la sesión guardada (arriba, leerSesion())
91   // hace que reabrir la app o recargar la página entre DIRECTO a Inicio
92   // sin pasar por Ingreso.jsx. Enganchado solo al login manual, alguien
93   // que ya tenía sesión activa en este equipo (lo normal) nunca disparaba
94   // el video. Aquí se revisa cada vez que hay un usuario con sesión, la
95   // haya iniciado ahora o la haya retomado la app sola.
96   useEffect(() => {
97     const id = sesion?.usuario?.id;

```

```

98     if (id !== null && debeAbrirTutorial(id)) setMostrarVideo(true);
99   }, [sesion?.usuario?.id]);
100
101   // Alto contraste / tamaño de letra (Mi perfil > Accesibilidad) son
102   // preferencias de ESTE equipo (localStorage), no de la cuenta - se
103   // aplican una sola vez al montar, antes incluso de que haya sesión, para
104   // que también se vean en la pantalla de login.
105   useEffect(() => {
106     aplicarPreferencias(LeerPreferencias());
107   }, []);
108
109   // Se suscribe una sola vez: cuando pedir() encuentra un 401 (sesión
110   // cerrada desde otro dispositivo, PIN cambiado, cuenta desactivada...)
111   // vuelve aquí mismo a la pantalla de login en vez de dejar el menú y las
112   // vistas mostrando datos de una sesión que el servidor ya no reconoce.
113   useEffect(() => {
114     alSesionInvalida(() => {
115       cerrarSesionLocal();
116       setSesion(null);
117       setVista("inicio");
118       setAvisoSesion("Su sesión terminó (se cerró desde otro dispositivo o venció). Ingrese de nuevo.");
119     });
120   }, []);
121
122   // El access token de Cognito dura poco a propósito (1 hora, ver
123   // ARQUITECTURA.md), pero cada vista recibe el token como prop suelto
124   // (no llama a Cognito directo por sí misma) - sin esto, la sesión se
125   // caía cada hora en plena demo. Se revisa cada pocos minutos (getSession
126   // del SDK refresca solo con el refresh token, sin red de por medio si
127   // el access token todavía es válido) y se actualiza sesion.token para
128   // que el próximo render lo pase ya fresco a todas las pantallas.
129   useEffect(() => {
130     if (!sesion) return;
131     let vivo = true;
132     async function refrescar() {
133       const token = await obtenerTokenValido();
134       if (!vivo) return;
135       if (!token) {
136         cerrarSesionLocal();
137         setSesion(null);
138         setVista("inicio");
139         setAvisoSesion("Su sesión venció. Ingrese de nuevo.");
140         return;
141       }
142       if (token !== sesion.token) {
143         const nueva = { ...sesion, token };
144         guardarSesion(nueva);
145         setSesion(nueva);
146       }
147     }
148     refrescar();
149     const intervalo = setInterval(refrescar, 4 * 60 * 1000);
150     return () => { vivo = false; clearInterval(intervalo); };
151     // eslint-disable-next-line react-hooks/exhaustive-deps
152   }, [sesion?.usuario?.id]);
153
154   // Contador que sube en cada ir(): las pantallas con pestañas usan
155   // [tabInicial, navSeq] como dependencia de su useEffect en vez de solo
156   // [tabInicial] - sin navSeq, pedir por voz la MISMA pestaña dos veces
157   // seguidas (con un clic manual a otra pestaña en el medio) no cambiaba
158   // nada, porque tabInicial ya traía ese mismo valor de la vez anterior y
159   // React no vuelve a correr un efecto cuya dependencia no cambió de
160   // valor, aunque la persona sí haya navegado a otro lado desde entonces.
161   const navSeq = useRef(0);
162
163   /** Navega y pasa contexto: ir("bodegas", { bodegaId: 3 }) */
164   function ir(destino, contexto = {}) {
165     // sin esto, una respuesta hablada que la pantalla anterior todavía
166     // estaba esperando (la síntesis real tarda unos segundos) podía
167     // reproducirse sola varios segundos después, ya con otra pantalla en
168     // uso - sonaba como si el agente hablara de la nada, fuera de tema.
169     detenerVoz();
170     navSeq.current += 1;
171     setCtx((c) => ({ ...c, ...contexto, navSeq: navSeq.current }));
172     setVista(destino);
173   }
174
175   /** Conteo.jsx la llama apenas abre (o retoma) una bodega, con el id real
176   que le devuelve el backend - así CerrarSesion (que vive fuera de
177   Conteo, en la barra lateral) pregunta por el avance de la sesión que
178   de verdad está abierta, no por el marcador de posición de antes de
179   abrir nada. */
180   function alAbrirBodega(sesionId) {
181     setCtx((c) => ({ ...c, sesionId }));
182   }
183

```

```

184   if (!sesion)
185     return (
186       <Ingreso
187         avisoInicial={avisoSesion}
188         alEntrar={(s) => {
189           guardarSesion(s);
190           setSesion(s);
191           setAvisoSesion("");
192           setVista("inicio");
193           pedir("/api/usuarios/yo", {}, s.token).then((p) =>
194             configurarVoz({ idioma: p.idioma_voz, velocidad: p.velocidad_voz,
195               confirmacionHablada: p.confirmacion_hablada })
196           ).catch(() => {});
197         }}
198       />
199     );
200
201   const Vista = VISTAS[vista] || Inicio;
202
203   return (
204     <div className="app-root">
205       <BarraLateral
206         activo={salir ? "salir" : menuDe(vista)}
207         usuario={sesion.usuario}
208         token={sesion.token}
209         sesionId={ctx.sesionId}
210         onNavegar={(id) => {
211           detenerVoz();
212           if (id === "salir") {
213             setSalir(true);
214             return;
215           }
216           const g = MENU.find((m) => m.id === id);
217           if (g) {
218             // el menu lateral es una navegacion "limpia": sin esto, un
219             // contexto de una vista anterior (tabInicial="recetas" de
220             // Pedido, bodegaId de un detalle, etc.) se quedaba pegado para
221             // siempre - la proxima vez que se entraba a Ajustes por el
222             // menu, por ejemplo, seguia abriendo la pestana equivocada.
223             setCtx({ sesionId: ctx.sesionId });
224             setVista(g.vistas[0]);
225           }
226         }}
227       />
228
229       <div className="contenido">
230         <Vista
231           token={sesion.token}
232           usuario={sesion.usuario}
233           {...ctx}
234           ir={ir}
235           alAbrirBodega={alAbrirBodega}
236         />
237       </div>
238
239       {salir && (
240         <CerrarSesion
241           token={sesion.token}
242           sesionId={ctx.sesionId}
243           onCancel={() => setSalir(false)}
244           onSalir={() => {
245             // Revoca en Cognito (mismo mecanismo que "cerrar todas las
246             // sesiones" en Mi perfil) antes de limpiar localmente - sin
247             // esto, "Cerrar sesión" solo borraba el token de ESTE
248             // navegador; el access token seguía siendo válido en Cognito
249             // hasta vencer solo (hasta 1 hora). En una tablet compartida entre
250             // varios auxiliares, quien lo hubiera copiado antes de que la
251             // persona cerrara sesión podía seguir usándolo. No bloquea la
252             // salida si la petición falla (sin red) - la persona debe poder
253             // salir localmente de todas formas.
254             pedir("/api/usuarios/yo/cerrar-todas", { method: "POST" }, sesion.token).catch(() => {});
255             cerrarSesionLocal();
256             borrarSesion();
257             setSesion(null);
258             setSalir(false);
259             setVista("inicio");
260           }}
261         />
262       )}
263
264       {mostrarVideo && (
265         <VideoRecorrido perfil={sesion.usuario?.perfil}
266           usuarioId={sesion.usuario?.id}
267           onCerrar={() => setMostrarVideo(false)} />
268       )}
269     </div>

```

```

270 );
271 }

```

frontend/src/api.js

183 LÍNEAS

Todas las llamadas al backend, y esFalloRed, que distingue un fallo de red de un error del servidor.

```

1 // IPv4 explícito, no "localhost": en Windows el navegador suele resolver
2 // "localhost" a ::1 (IPv6), y uvicorn por defecto solo escucha en 127.0.0.1.
3 export const BASE = import.meta.env.VITE_API_URL || "http://127.0.0.1:8000";
4
5 export function guardarToken(t) {
6   localStorage.setItem("cv_token", t);
7 }
8 export function leerToken() {
9   return localStorage.getItem("cv_token");
10 }
11 export function borrarToken() {
12   localStorage.removeItem("cv_token");
13 }
14
15 // La sesión completa (token + datos del usuario), no solo el token: sin
16 // esto, reabrir la app (o perder la señal justo al recargar) obligaba a
17 // iniciar sesión otra vez contra el backend - imposible sin red, aunque
18 // la persona ya se hubiera identificado minutos antes en el mismo equipo.
19 // Con la sesión guardada, App.jsx entra directo sin depender de la red.
20 const CLAVE_SESION = "cv_sesion";
21
22 export function guardarSesion(s) {
23   localStorage.setItem(CLAVE_SESION, JSON.stringify(s));
24   if (s?.token) guardarToken(s.token); // cv_token queda siempre en sync
25 }
26 export function leerSesion() {
27   try { return JSON.parse(localStorage.getItem(CLAVE_SESION) || "null"); }
28   catch (_) { return null; }
29 }
30 export function borrarSesion() {
31   localStorage.removeItem(CLAVE_SESION);
32   borrarToken();
33 }
34
35 // App.jsx se suscribe una sola vez al montar para poder volver a la
36 // pantalla de login apenas el backend dice que el token ya no sirve
37 // (sesión cerrada desde otro dispositivo, PIN cambiado, cuenta
38 // desactivada...). Sin esto, pedir() ya borraba la sesión guardada pero la
39 // pestaña seguía mostrando el menú y las vistas como si siguiera con
40 // sesión, y cada pantalla fallaba en silencio o mostraba el error crudo
41 // del servidor como si fuera un dato más (p. ej. "0 bodegas" en vez de
42 // avisar que hay que volver a ingresar).
43 let _alSesionInvalida = null;
44 export function alSesionInvalida(fn) {
45   _alSesionInvalida = fn;
46 }
47
48 /** fetch() rechaza con un TypeError cuando la red falla (Wi-Fi caído, DNS,
49     CORS bloqueado) - la especificación lo llama "network error". Detectarlo
50     así (en vez de por texto del mensaje) es lo que ya usaba Conteo.jsx para
51     activar el modo sin conexión; se comparte aquí para que todo el frontend
52     reconozca el mismo caso una sola vez. */
53 export function esFalloRed(error) {
54   return error?.esFalloRed === true || error instanceof TypeError
55     || /failed to fetch|networkerror|load failed/i.test(error?.message || "");
56 }
57
58 /** El error técnico crudo en inglés que fetch() lanza tal cual, traducido a
59     algo que un auxiliar en una bodega con mal Wi-Fi pueda entender. Marca el
60     error con esFalloRed=true para que quien lo reciba después de pedir()
61     (que ya perdió el TypeError original) lo siga reconociendo con
62     esFalloRed(), en vez de tener que repetir la detección por texto. */
63 function errorDeRed() {
64   const error = new Error("Sin conexión con el servidor. Revise el Wi-Fi e intente de nuevo.");
65   error.esFalloRed = true;
66   return error;
67 }
68
69 /** Llama al backend adjuntando el token de la sesión. */
70 export async function pedir(ruta, opciones = {}, token) {
71   const t = token || leerToken();
72   // arma las opciones completas ANTES del try: si algo aquí lanza (un
73   // "opciones.headers" mal formado, por ejemplo), es un error de
74   // programación real y no debe disfrazarse de fallo de red.
75   const opcionesCompletas = {

```

```

76     ...opciones,
77     headers: {
78       "Content-Type": "application/json",
79       ...(t ? { Authorization: `Bearer ${t}` } : {}),
80       ...opciones.headers,
81     },
82   };
83   let res;
84   try {
85     res = await fetch(`${BASE}${ruta}`, opcionesCompletas);
86   } catch (error) {
87     if (esFalloRed(error)) throw errorDeRed();
88     throw error;
89   }
90   if (!res.ok) {
91     // 401 de verdad (no un fallo de red: el servidor sí respondió) significa
92     // que el servidor ya no reconoce este token - ni reintentar ni seguir
93     // usando la sesión cacheada sirve de nada; se limpia para que la
94     // próxima carga pida iniciar sesión de nuevo en vez de quedar
95     // atascada reusando un token que el backend rechaza siempre.
96     if (res.status === 401) { borrarSesion(); _alSesionInvalida?(); }
97     let detalle = `Error ${res.status}`;
98     try {
99       const j = await res.json();
100       detalle = j.detail || j.detalle || detalle;
101     } catch (_) {}
102     throw new Error(detalle);
103   }
104   return res.json();
105 }
106
107 // «respaldo» son las opciones que la pantalla ya tiene mostradas en
108 // pantalla ({ opciones, opcionesPara }); si el backend se reinició y
109 // perdió la memoria de la conversación, las recupera de ahí en vez de
110 // preguntar «¿cuál de los dos?» otra vez sin recordar nada.
111 export const enviarTurno = (texto, sesionId, token, respaldo = {}) =>
112   pedir("/api/agente/turno", {
113     method: "POST",
114     body: JSON.stringify({
115       texto, sesion_id: sesionId,
116       opciones_pendientes: respaldo.opciones || null,
117       opciones_para: respaldo.opcionesPara || null,
118       bodega_id_respaldo: respaldo.bodegaId || null,
119       bodega_nombre_respaldo: respaldo.bodegaNombre || null,
120       preparacion_respaldo: respaldo.preparacion || null,
121       porciones_respaldo: respaldo.porciones || null,
122     }),
123   }, token);
124
125 // Para pantallas que no cuentan ni piden (Inicio, Ajustes, Ayuda, Reportes,
126 // Panel): una pregunta suelta o una orden de navegar, sin el estado de
127 // conversación multi-turno que sí necesita enviarTurno. Ver AsistenteVoz.jsx.
128 export const preguntarAsistente = (texto, vista, token) =>
129   pedir("/api/agente/asistente", {
130     method: "POST",
131     body: JSON.stringify({ texto, vista }),
132   }, token);
133
134 export const abrirBodega = (bodega, token) =>
135   pedir("/api/bodegas/abrir", {
136     method: "POST",
137     body: JSON.stringify({ bodega }),
138   }, token);
139
140 // Resuelve un nombre dicho/escrito al nombre real del catálogo, sin abrir
141 // nada - para poder confirmar "¿abro X?" con el nombre que de verdad se
142 // va a usar, no con lo que se haya transcrito tal cual (ver Conteo.jsx).
143 export const buscarBodega = (bodega, token) =>
144   pedir("/api/bodegas/buscar", {
145     method: "POST",
146     body: JSON.stringify({ bodega }),
147   }, token);
148
149 /** Descarga un reporte generado por el backend. Un <a href> comun no sirve
150     aqui: el endpoint exige el token en la cabecera, y un enlace del
151     navegador no lo puede enviar (por eso terminaba en una pagina en blanco
152     pidiendo sesion). Se trae como blob autenticado y se dispara la
153     descarga nativa del navegador, sin salir de la aplicacion. */
154 export async function descargarReporte(archivo, token) {
155   const t = token || leerToken();
156   let res;
157   try {
158     res = await fetch(
159       `${BASE}/api/reportes/descargar?archivo=${encodeURIComponent(archivo)}`,
160       { headers: { Authorization: `Bearer ${t}` } }
161     );

```

```

162 } catch (error) {
163   if (esFalloRed(error)) throw errorDeRed();
164   throw error;
165 }
166 if (!res.ok) {
167   // mismo trato que pedir(): un 401 aqui tambien significa que la sesion
168   // ya no sirve - sin esto, descargar un reporte con la sesion vencida
169   // solo mostraba "no se pudo descargar" sin volver nunca al login, a
170   // diferencia de cualquier otra llamada de la app.
171   if (res.status === 401) { borrarSesion(); _alSesionInvalida?(); }
172   throw new Error("No se pudo descargar el archivo.");
173 }
174 const blob = await res.blob();
175 const url = URL.createObjectURL(blob);
176 const a = document.createElement("a");
177 a.href = url;
178 a.download = archivo.split("/").pop();
179 document.body.appendChild(a);
180 a.click();
181 a.remove();
182 setTimeout(() => URL.revokeObjectURL(url), 4000);
183 }

```

frontend/src/cognito.js

303 LÍNEAS

Registro, ingreso, clave y verificacion en dos pasos. La clave nunca sale de aqui hacia el backend.

```

1 // Identidad con AWS Cognito: registro, login y recuperar clave hablan
2 // DIRECTO con Cognito desde aquí (nunca por el backend - por eso el App
3 // Client no tiene secreto, está pensado para vivir en el navegador). El
4 // User Pool Id y el Client Id no son secretos (son el equivalente al
5 // "project id" de cualquier SDK de identidad que corre en el cliente);
6 // por eso llevan un valor por defecto, igual que VITE_API_URL en api.js,
7 // para que el proyecto funcione en local sin un .env aparte.
8 import {
9   CognitoUserPool,
10  CognitoUser,
11  AuthenticationDetails,
12  CognitoUserAttribute,
13 } from "amazon-cognito-identity-js";
14
15 const REGION = import.meta.env.VITE_COGNITO_REGION || "us-east-2";
16 const USER_POOL_ID = import.meta.env.VITE_COGNITO_USER_POOL_ID || "us-east-2_6HbrPrvVL";
17 const CLIENT_ID = import.meta.env.VITE_COGNITO_APP_CLIENT_ID || "847jtkc5sem7mr4tb8csrrkqp";
18
19 const poolDatos = { UserPoolId: USER_POOL_ID, ClientId: CLIENT_ID };
20 const pool = new CognitoUserPool(poolDatos);
21
22 /** Traduce los códigos de error que Cognito manda (en inglés, pensados
23     para un desarrollador) a algo que alguien en una bodega entienda. Los
24     demás mensajes de Cognito ya vienen razonablemente claros en español
25     parcial/inglés corto, así que solo se traducen los que de verdad
26     confunden. */
27 function mensajeClaro(error) {
28   const mapa = {
29     UsernameExistsException: "Ese usuario ya existe. Inicie sesión o recupere su clave.",
30     UserNotFoundException: "No existe ese usuario.",
31     NotAuthorizedException: "Usuario o clave incorrectos.",
32     CodeMismatchException: "El código no es correcto.",
33     ExpiredCodeException: "El código venció. Pida uno nuevo.",
34     InvalidPasswordException: "La clave debe tener al menos 8 caracteres, con mayúscula, "
35       + "minúscula y número.",
36     LimitExceededException: "Demasiados intentos. Espere un momento e intente de nuevo.",
37     UserNotConfirmedException: "Falta confirmar el registro con el código enviado por correo.",
38     EnableSoftwareTokenMFAException: "El código de la app autenticadora no es correcto.",
39   };
40   const msg = mapa[error?.code];
41   const final = new Error(msg || error?.message || "No se pudo completar la operación.");
42   final.code = error?.code;
43   return final;
44 }
45
46 function usuarioCognito(nombre) {
47   return new CognitoUser({ Username: nombre.trim().toLowerCase(), Pool: pool });
48 }
49
50 /** Crea la cuenta en Cognito. "name" es un atributo obligatorio del User
51     Pool (además de email) - sin él Cognito rechaza el registro. Devuelve
52     el destino enmascarado (p***@m***) que ya calcula Cognito, para
53     mostrarlo igual que su propia pantalla "Confirm your account". */
54 export function registrarse(usuario, clave, correo, nombreCompleto) {
55   return new Promise((resolve, reject) => {

```

```

56     const atributos = [
57       new CognitoUserAttribute({ Name: "email", Value: correo.trim() }),
58       new CognitoUserAttribute({ Name: "name", Value: nombreCompleto.trim() }),
59     ];
60     pool.signUp(usuario.trim().toLowerCase(), clave, atributos, null, (error, resultado) => {
61       if (error) return reject(mensajeClaro(error));
62       resolve({ destino: resultado?.codeDeliveryDetails?.Destination || correo.trim() });
63     });
64   });
65 }
66
67 /** Reenvía el código de confirmación del registro (botón "Resend code"). */
68 export function reenviarCodigoRegistro(usuario) {
69   return new Promise((resolve, reject) => {
70     usuarioCognito(usuario).resendConfirmationCode((error, resultado) => {
71       if (error) return reject(mensajeClaro(error));
72       resolve({ destino: resultado?.CodeDeliveryDetails?.Destination || "" });
73     });
74   });
75 }
76
77 /** Confirma el código de 6 dígitos que Cognito manda por correo al
78     registrarse - solo después de esto la cuenta puede iniciar sesión. */
79 export function confirmarRegistro(usuario, codigo) {
80   return new Promise((resolve, reject) => {
81     usuarioCognito(usuario).confirmRegistration(codigo, true, (error, resultado) => {
82       // Si la cuenta YA estaba confirmada, no es un fallo: el objetivo de
83       // esta función es "que quede confirmada", y ya lo está. Pasa de
84       // verdad cuando el código se confirmó bien pero el paso siguiente
85       // (crear la fila local) se cayó - al reintentar, Cognito responde
86       // UnauthorizedException, que mensajeClaro traduce a "Usuario o clave
87       // incorrectos" y deja a la persona atascada sin salida, culpándola de
88       // un dato que escribió bien. Se trata como exitoso y el flujo sigue.
89       if (error) {
90         const yaConfirmada = error.code === "NotAuthorizedException"
91           && /current status is confirmed/i.test(error.message || "");
92         if (yaConfirmada) return resolve({ yaEstaba: true });
93         return reject(mensajeClaro(error));
94       }
95       resolve(resultado);
96     });
97   });
98 }
99
100 /** Nombre completo y correo tal cual quedaron en Cognito para la cuenta
101     YA AUTENTICADA (sesión recién abierta) - se usa para completar el
102     registro local (/api/registro-completado) cuando esos datos no
103     vienen ya en memoria (p. ej. alguien que se registró, cerró la
104     pestaña antes de confirmar el código, y vuelve después: en ese
105     camino nunca se volvió a escribir el nombre/correo en el
106     formulario, así que se recuperan de Cognito en vez de mandar
107     campos vacíos). */
108 export function obtenerAtributosUsuario() {
109   return new Promise((resolve, reject) => {
110     const cu = pool.getCurrentUser();
111     if (!cu) return reject(new Error("No hay una sesión activa."));
112     cu.getSession((error) => {
113       if (error) return reject(mensajeClaro(error));
114       cu.getUserAttributes((error2, atributos) => {
115         if (error2) return reject(mensajeClaro(error2));
116         const porNombre = Object.fromEntries(
117           (atributos || []).map((a) => [a.getName(), a.getValue()]);
118         resolve({ nombreCompleto: porNombre.name || "", correo: porNombre.email || "" });
119       });
120     });
121   });
122 }
123
124 /** Inicia sesión. Si la cuenta tiene activada la verificación en dos
125     pasos (Mi perfil > Verificación en dos pasos - es opcional, no todos
126     la tienen), Cognito no da el token de una vez sino que pide el código
127     de la app autenticadora: devuelve { mfaPendiente } en vez de { token },
128     y ese CognitoUser en curso hay que pasárselo tal cual a
129     confirmarCodigoMFA() para terminar el ingreso (no se puede reconstruir
130     solo con el usuario/clave: Cognito ata el reto a esa sesión de login
131     específica). Con el token, el refresh token queda guardado por el
132     propio SDK en localStorage, con su propio ciclo de refresco automático
133     (ver obtenerTokenValido). */
134 export function iniciarSesion(usuario, clave) {
135   return new Promise((resolve, reject) => {
136     const cu = usuarioCognito(usuario);
137     const detalles = new AuthenticationDetails({
138       Username: usuario.trim().toLowerCase(), Password: clave,
139     });
140     cu.authenticateUser(detalles, {
141       onSuccess: (sesion) => resolve({ token: sesion.getAccessToken().getJwtToken() }),

```

```

142     onFailure: (error) => reject(mensajeClaro(error)),
143     totpRequired: () => resolve({ mfaPendiente: cu }),
144   });
145   });
146 }
147
148 /** Termina un ingreso que quedó pendiente de código de verificación en
149     dos pasos (ver iniciarSesion). */
150 export function confirmarCodigoMFA(mfaPendiente, codigo) {
151   return new Promise((resolve, reject) => {
152     mfaPendiente.sendMFACode(codigo, {
153       onSuccess: (sesion) => resolve(sesion.getAccessToken().getJwtToken()),
154       onFailure: (error) => reject(mensajeClaro(error)),
155     }, "SOFTWARE_TOKEN_MFA");
156   });
157 }
158
159 /** Pide el código de recuperación de clave, mandado al correo registrado.
160     Devuelve el destino enmascarado que calcula Cognito (p***@m***). */
161 export function recuperarClave(usuario) {
162   return new Promise((resolve, reject) => {
163     usuarioCognito(usuario).forgotPassword({
164       inputVerificationCode: (datos) => resolve({ destino: datos?.CodeDeliveryDetails?.Destination || "" }),
165       onSuccess: () => resolve({ destino: "" }),
166       onFailure: (error) => reject(mensajeClaro(error)),
167     });
168   });
169 }
170
171 /** Confirma el código de recuperación con la clave nueva. */
172 export function confirmarNuevaClave(usuario, codigo, claveNueva) {
173   return new Promise((resolve, reject) => {
174     usuarioCognito(usuario).confirmPassword(codigo, claveNueva, {
175       onSuccess: () => resolve(),
176       onFailure: (error) => reject(mensajeClaro(error)),
177     });
178   });
179 }
180
181 /** Cambia la clave con la sesión ya abierta (Mi perfil) - exige la clave
182     vigente, igual que el cambio de PIN de antes. */
183 export function cambiarClave(claveActual, claveNueva) {
184   return new Promise((resolve, reject) => {
185     const cu = pool.getCurrentUser();
186     if (!cu) return reject(new Error("No hay una sesión activa."));
187     cu.getSession((error) => {
188       if (error) return reject(mensajeClaro(error));
189       cu.changePassword(claveActual, claveNueva, (error2) => {
190         if (error2) return reject(mensajeClaro(error2));
191         resolve();
192       });
193     });
194   });
195 }
196
197 /** Si la persona ya autenticada tiene activa la verificación en dos pasos
198     (TOTP) - para saber qué mostrar en Mi perfil: "activar" o "desactivar". */
199 export function tieneMFAActiva() {
200   return new Promise((resolve, reject) => {
201     const cu = pool.getCurrentUser();
202     if (!cu) return reject(new Error("No hay una sesión activa."));
203     cu.getSession((error) => {
204       if (error) return reject(mensajeClaro(error));
205       cu.getUserData((error2, datos) => {
206         if (error2) return reject(mensajeClaro(error2));
207         resolve((datos?.UserMFASettingList || []).includes("SOFTWARE_TOKEN_MFA"));
208       }, { bypassCache: true });
209     });
210   });
211 }
212
213 /** Primer paso de activar la verificación en dos pasos: le pide a Cognito
214     una llave secreta nueva para esta cuenta (para el QR/la llave manual)
215     - todavía no queda activada, eso pasa en confirmarActivacionMFA(). */
216 export function iniciarActivacionMFA() {
217   return new Promise((resolve, reject) => {
218     const cu = pool.getCurrentUser();
219     if (!cu) return reject(new Error("No hay una sesión activa."));
220     cu.getSession((error) => {
221       if (error) return reject(mensajeClaro(error));
222       cu.associateSoftwareToken({
223         associateSecretCode: (llave) => resolve(llave),
224         onFailure: (error2) => reject(mensajeClaro(error2)),
225       });
226     });
227   });

```

```

228 }
229
230 /** Segundo paso: confirma con el código de 6 dígitos que generó la app
231     autenticadora a partir de la llave, y de una vez la marca como el
232     método de verificación en dos pasos de la cuenta. */
233 export function confirmarActivacionMFA(codigo) {
234     return new Promise((resolve, reject) => {
235         const cu = pool.getCurrentUser();
236         if (!cu) return reject(new Error("No hay una sesión activa."));
237         // Sin getSession() primero, cu.signInUserSession queda null (recién
238         // construido por getCurrentUser(), no hidratado desde el localStorage
239         // del SDK) - verifySoftwareToken() lo interpreta como una sesión de
240         // RETO en curso (usa this.Session, que aquí no existe) en vez de
241         // usuario-ya-autenticado (usa el AccessToken), y Cognito responde
242         // UnauthorizedException en vez de validar el código.
243         cu.getSession((error0) => {
244             if (error0) return reject(mensajeClaro(error0));
245             cu.verifySoftwareToken(codigo, "CuentaVoz", {
246                 onSuccess: () => {
247                     cu.setUserMfaPreference(null, { Enabled: true, PreferredMfa: true }, (error) => {
248                         if (error) return reject(mensajeClaro(error));
249                         resolve();
250                     });
251                 },
252                 onFailure: (error) => reject(mensajeClaro(error)),
253             });
254         });
255     });
256 }
257
258 /** Desactiva la verificación en dos pasos para esta cuenta. */
259 export function desactivarMFA() {
260     return new Promise((resolve, reject) => {
261         const cu = pool.getCurrentUser();
262         if (!cu) return reject(new Error("No hay una sesión activa."));
263         cu.getSession((error) => {
264             if (error) return reject(mensajeClaro(error));
265             cu.setUserMfaPreference(null, { Enabled: false, PreferredMfa: false }, (error2) => {
266                 if (error2) return reject(mensajeClaro(error2));
267                 resolve();
268             });
269         });
270     });
271 }
272
273 /** El URI estándar (otpauth://) que cualquier app autenticadora (Google
274     Authenticator, Authy, etc.) sabe leer desde un QR - MiPerfil.jsx lo
275     convierte en imagen con la librería "qrcode". */
276 export function uriOtpAuth(llaveSecreta, usuario) {
277     const etiqueta = encodeURIComponent(`CuentaVoz:${usuario}`);
278     return `otpauth://totp/${etiqueta}?secret=${llaveSecreta}&issuer=CuentaVoz`;
279 }
280
281 /** Access token vigente del usuario ya autenticado, refrescándolo solo
282     con el refresh token si ya venció (el SDK lo hace por su cuenta al
283     llamar getSession, sin pedirle nada a la persona) - así el access
284     token corto (1 hora, ver ARQUITECTURA.md) no obliga a reingresar cada
285     hora. Devuelve null si no hay sesión (nunca ingresó o el refresh token
286     también venció, a los 30 días). */
287 export function obtenerTokenValido() {
288     return new Promise((resolve) => {
289         const cu = pool.getCurrentUser();
290         if (!cu) return resolve(null);
291         cu.getSession((error, sesion) => {
292             if (error || !sesion?.isValid()) return resolve(null);
293             resolve(sesion.getAccessToken().getJwtToken());
294         });
295     });
296 }
297
298 /** Cierra la sesión guardada por el SDK en ESTE navegador (no revoca nada
299     en Cognito - para eso está "cerrar todas las sesiones" en Mi perfil,
300     que sí llama al backend). */
301 export function cerrarSesionLocal() {
302     pool.getCurrentUser()?.signOut();
303 }

```

frontend/src/voz.js

258 LÍNEAS

Escuchar y hablar.

```

1 /** Escuchar y hablar. Las dos piezas de la capa de voz. */
2 import { BASE, leerToken } from "../api";

```

```

3 // quitarTildes/esAfirmacion/esNegacion viven en su propio módulo sin
4 // dependencias (ni de "./api" ni del navegador) para poder probarlas con
5 // node --test igual que interpretelocal.js - se reexportan aquí para que
6 // quien ya las importaba desde "./voz" siga funcionando igual.
7 export { quitarTildes, esAfirmacion, esNegacion } from "./confirmacionVoz.js";
8
9 const IDIOMA_ESCUCHA = "es-CO"; // como reconoce lo que usted dice: fijo
10 let vozNeuronal = "kore"; // que voz neuronal usa CuentaVoz al hablar: elegible
11 const TASA = { lenta: 0.82, normal: 1.02, rapida: 1.3 };
12 let tasaActual = TASA.normal;
13 let confirmacionHablada = true;
14
15 // A que genero suena cada voz neuronal - para que, si la neuronal falla y
16 // hay que caer al respaldo del navegador, se busque una voz de navegador
17 // del mismo genero en vez de una elegida al azar. No es la MISMA voz (eso
18 // no se puede: el navegador no tiene "Kore"), pero al menos no suena como
19 // si de repente le estuviera hablando otra persona.
20 const _GENERO_VOZ = { kore: "femenina", aoede: "femenina", puck: "masculina", charon: "masculina" };
21
22 const Reconocedor =
23   typeof window !== "undefined" &&
24   (window.SpeechRecognition || window.webkitSpeechRecognition);
25
26 export const vozDisponible = () => Boolean(Reconocedor);
27
28 /** MiPerfil llama esto una vez cargadas (o cambiadas) las preferencias.
29   "idioma" quedo con ese nombre por compatibilidad con quien ya llama
30   configurarVoz(), pero ahora guarda la clave de la voz neuronal
31   (kore, puck...), no un código de idioma de navegador. */
32 export function configurarVoz({ idioma, velocidad, confirmacionHablada: ch } = {}) {
33   if (idioma && idioma !== vozNeuronal) {
34     vozNeuronal = idioma;
35     // la voz de respaldo se habia elegido (o cacheado como "no hay") con
36     // la preferencia VIEJA - sin esto, cambiar la voz en Mi perfil no
37     // cambiaba nada si en algun momento se habia caido al respaldo antes.
38     vozNavegadorBuscada = false;
39     vozNavegadorElegida = null;
40   }
41   if (velocidad && TASA[velocidad]) tasaActual = TASA[velocidad];
42   if (ch !== undefined) confirmacionHablada = ch;
43 }
44
45 /* — Respaldo: voz del navegador (speechSynthesis) —
46   Solo se usa si la voz neuronal no responde (sin internet, sin llave de
47   Gemini configurada, o la API falla) - para que el conteo por voz nunca
48   se quede completamente mudo. Suena mas robotica, pero es mejor que
49   nada mientras se recupera la conexion. */
50 let vozNavegadorElegida = null;
51 let vozNavegadorBuscada = false;
52
53 // Nombres tipicos de voces de navegador/SO en espanol, para adivinar de
54 // que genero suena cada una - el estandar Web Speech API no expone genero,
55 // asi que esto es lo mas cercano a saberlo sin reproducirla.
56 const _NOMBRES_FEM =
/sabina|helenaelvira|laura|m[ó]nica|paulina|luc[i]a|elena|isabela|marisol|conchita|pen[eé]lope|esperanza|camila|valentina|female|mujer/;
57 const _NOMBRES_MASC = /ra[uú]l|jorge|diego|pablo|carlos|alonso|enrique|miguel|male|hombre/;
58
59 function _puntuarVozNavegador(v, generoPreferido) {
60   const lang = (v.lang || "").toLowerCase();
61   const nombre = (v.name || "").toLowerCase();
62   let p = 0;
63   if (lang.startsWith("es-mx") || lang.startsWith("es-419")) p += 20;
64   else if (lang.startsWith("es-")) p += 10;
65   else if (lang.startsWith("es")) p += 5;
66   else return -1;
67   if (/natural|online|neural|wavenet|premium/.test(nombre)) p += 30;
68   if (/google/.test(nombre)) p += 10;
69   if (/desktop|compact/.test(nombre)) p -= 20;
70   // que suene del mismo genero que la voz neuronal elegida en Mi perfil
71   // pesa mas que cualquier otra cosa: es justo lo que hace notorio el
72   // cambio de voz cuando toca caer al respaldo.
73   if (generoPreferido === "femenina" && _NOMBRES_FEM.test(nombre)) p += 50;
74   else if (generoPreferido === "masculina" && _NOMBRES_MASC.test(nombre)) p += 50;
75   else if (generoPreferido === "femenina" && _NOMBRES_MASC.test(nombre)) p -= 50;
76   else if (generoPreferido === "masculina" && _NOMBRES_FEM.test(nombre)) p -= 50;
77   return p;
78 }
79
80 function _elegirVozNavegador() {
81   if (typeof window === "undefined" || !window.speechSynthesis) return null;
82   const voces = window.speechSynthesis.getVoices();
83   if (!voces.length) return null;
84   const generoPreferido = _GENERO_VOZ[vozNeuronal];
85   const candidatas = voces.map((v) => ({ v, p: _puntuarVozNavegador(v, generoPreferido) }));
86   .filter((x) => x.p >= 0).sort((a, b) => b.p - a.p);
87   return candidatas[0]?v || null;

```

```

88 }
89
90 /** Devuelve una promesa que se resuelve cuando termina de decirlo (no
91     cuando empieza) - quien vaya a escuchar() justo después de hablar()
92     necesita esperar a que la pregunta se termine de decir, o el
93     micrófono arranca mientras el navegador todavía está sonando y se
94     pierde la respuesta. */
95 function hablarConNavegador(texto) {
96     return new Promise((resolve) => {
97         if (typeof window === "undefined" || !window.speechSynthesis) { resolve(); return; }
98         if (!vozNavegadorBuscada) {
99             vozNavegadorElegida = _elegirVozNavegador();
100             if (!vozNavegadorElegida) {
101                 window.speechSynthesis.onvoiceschanged = () => { vozNavegadorElegida = _elegirVozNavegador(); };
102             } else {
103                 vozNavegadorBuscada = true;
104             }
105         }
106         window.speechSynthesis.cancel();
107         const u = new SpeechSynthesisUtterance(texto);
108         u.lang = vozNavegadorElegida?.lang || "es-MX";
109         u.rate = tasaActual;
110         u.pitch = 1.03;
111         if (vozNavegadorElegida) u.voice = vozNavegadorElegida;
112         u.onend = resolve;
113         u.onerror = resolve;
114         window.speechSynthesis.speak(u);
115     });
116 }
117
118 /* — Voz neuronal real (Gemini TTS, via el backend) — */
119 let audioActual = null;
120
121 // La síntesis real tarda unos segundos (viaja al backend y a Gemini): si
122 // mientras tanto la persona ya cambió de pantalla, esa respuesta ya no
123 // tiene contexto y no debe hablar sola de repente sobre una pantalla que
124 // ya no está en uso. Cada hablar() (y detenerVoz()) sube esta generación;
125 // una respuesta que llega tarde, de una generación vieja, se descarta en
126 // vez de reproducirse.
127 let generacion = 0;
128
129 // Si detenerVoz() corta el audio a la mitad (pause() no dispara "ended"),
130 // esto es lo que despierta a quien esté esperando hablar() - si no, un
131 // cambio de pantalla a mitad de frase dejaría esa promesa colgada para
132 // siempre.
133 let resolverAudioActual = null;
134
135 /** App.jsx la llama en cada cambio de pantalla: corta lo que se esté
136     reproduciendo y invalida cualquier hablar() todavía en camino. */
137 export function detenerVoz() {
138     generacion++;
139     if (audioActual) { audioActual.pause(); audioActual.currentTime = 0; }
140     if (typeof window !== "undefined" && window.speechSynthesis) window.speechSynthesis.cancel();
141     resolverAudioActual?.();
142     resolverAudioActual = null;
143 }
144
145 async function hablarConNeuronal(texto, miGeneracion) {
146     const res = await fetch(`${BASE}/api/voz/hablar`, {
147         method: "POST",
148         headers: {
149             "Content-Type": "application/json",
150             Authorization: `Bearer ${leerToken()}`,
151         },
152         body: JSON.stringify({ texto, voz: vozNeuronal }),
153     });
154     if (!res.ok) throw new Error("voz neuronal no disponible");
155     const blob = await res.blob();
156     if (miGeneracion !== generacion) return; // se cambió de pantalla mientras se generaba
157     const url = URL.createObjectURL(blob);
158     if (audioActual) { audioActual.pause(); audioActual.currentTime = 0; }
159     const audio = new Audio(url);
160     audio.playbackRate = tasaActual;
161     audioActual = audio;
162     // Espera a que el audio TERMINE de sonar, no solo a que arranque -
163     // audio.play() por sí solo se resuelve apenas empieza a reproducir, y
164     // quien llama a hablar() esperando poder escuchar() justo después
165     // necesita que la pregunta ya se haya terminado de decir.
166     await new Promise((resolve) => {
167         resolverAudioActual = resolve;
168         audio.addEventListener("ended", resolve, { once: true });
169         audio.addEventListener("error", resolve, { once: true });
170         audio.play().catch(resolve);
171     });
172     resolverAudioActual = null;
173     URL.revokeObjectURL(url);

```

```

174 }
175
176 // Tope de seguridad: si el navegador alguna vez no dispara "ended"/"onend"
177 // (se ha visto en algunos Chrome tras minimizar la pestaña, y en entornos
178 // sin salida de audio real), hablar() quedaría esperando para siempre y el
179 // micrófono que depende de "await hablar()" nunca llegaría a abrirse.
180 //
181 // Un numero fijo no alcanza: "Hay varias: <hasta 6 bodegas>. ¿Cuál de
182 // todas?" (Conteo.jsx, cuando el nombre dictado es ambiguo) puede pasar
183 // fácil los 200 caracteres con nombres reales del catalogo - a un ritmo
184 // de lectura normal eso son 15-20 segundos, mas que un limite fijo de 12s
185 // pensado para frases cortas. Con el limite fijo, esa frase se cortaba a
186 // medias: la promesa de hablar() se resolvía con el audio TODAVIA
187 // sonando, y el microfono que se abre justo despues terminaba
188 // "escuchando" la cola de la propia voz del agente como si fuera la
189 // respuesta de la persona. El tope ahora escala con el largo del texto
190 // (y con que tan lenta este la voz elegida en Mi perfil), con un piso
191 // para frases cortas y un techo para no esperar para siempre si el audio
192 // de verdad se atasca.
193 const TOPE_HABLAR_MS_PISO = 12000;
194 const TOPE_HABLAR_MS_TECHO = 40000;
195
196 function _topeHablar(texto) {
197   // ~12 caracteres por segundo es una lectura pausada y generosa (cubre
198   // voces mas lentas que el promedio); /tasaActual porque una voz "lenta"
199   // (0.82x) de verdad tarda mas en decir lo mismo que una "rapida" (1.3x).
200   const estimado = (texto.length / 12) * 1000 / tasaActual;
201   return Math.min(TOPE_HABLAR_MS_TECHO, Math.max(TOPE_HABLAR_MS_PISO, estimado));
202 }
203
204 /** Habla el texto y devuelve una promesa que se resuelve cuando termina
205     de decirlo (o, como mucho, al tope de seguridad calculado para ese
206     texto). Casi todo el código la llama "al aire" (sin await) y eso sigue
207     funcionando igual; quien necesite escuchar() justo después de
208     preguntar algo sí debe esperarla, o el micrófono arranca mientras
209     todavía se está hablando y se pierde parte de la respuesta. */
210 export function hablar(texto, { forzar = false } = {}) {
211   if (!texto) return Promise.resolve();
212   if (!forzar && !confirmacionHablada) return Promise.resolve();
213   generacion++;
214   const miGeneracion = generacion;
215   const intento = hablarConNeuronal(texto, miGeneracion)
216     .catch(() => { if (miGeneracion === generacion) return hablarConNavegador(texto); });
217   return Promise.race([
218     intento,
219     new Promise((resolve) => setTimeout(resolve, _topeHablar(texto))),
220   ]);
221 }
222
223 /** Escucha una frase y la devuelve como texto. */
224 export function escuchar({ alTexto, alEstado, alError }) {
225   if (!Reconocedor) {
226     alError?.("Este navegador no reconoce voz. Use Chrome o Edge.");
227     return null;
228   }
229   const rec = new Reconocedor();
230   rec.lang = IDIOMA_ESCUCHA;
231   rec.interimResults = true;
232   rec.continuous = false;
233   rec.maxAlternatives = 1;
234
235   rec.onstart = () => alEstado?.("escuchando");
236   rec.onresult = (e) => {
237     const r = e.results[e.results.length - 1];
238     const texto = r[0].transcript.trim();
239     if (r.isFinal) {
240       alEstado?.("procesando");
241       alTexto?.(texto);
242     } else {
243       alEstado?.("escuchando", texto);
244     }
245   };
246   rec.onerror = (e) => {
247     alEstado?.("listo");
248     if (e.error === "not-allowed")
249       alError?.("Debe permitir el micrófono en el navegador.");
250     else if (e.error !== "aborted" && e.error !== "no-speech")
251       alError?.("No pude escuchar (${e.error}).");
252   };
253   rec.onend = () => alEstado?.("listo");
254   try {
255     rec.start();
256   } catch (_) {}
257   return rec;
258 }

```

frontend/src/interpreteLocal.js

148 LÍNEAS

El mismo interprete del backend, pero dentro del navegador, para el modo sin conexion.

```

1  /** Puerto a JavaScript de backend/servicios/interprete.py: el mismo
2   reconocimiento de números en palabras e intenciones frecuentes, pero
3   corriendo en el navegador en vez del servidor - para que el modo sin
4   conexión entienda texto escrito de verdad (no solo un formulario
5   numérico) incluso con la red completamente apagada, cero llamadas al
6   backend. Mantener esto en sincronía con interprete.py a mano: no hay
7   una fuente única compartida entre Python y JS todavía.
8
9   No reemplaza a Gemini ni al intérprete del servidor: es el mismo
10  respaldo de última instancia, ahora también disponible sin backend. */
11
12  const NUMEROS = new Map(Object.entries({
13    cero: 0, un: 1, una: 1, uno: 1, dos: 2, tres: 3, cuatro: 4,
14    cinco: 5, seis: 6, siete: 7, ocho: 8, nueve: 9, diez: 10,
15    once: 11, doce: 12, trece: 13, catorce: 14, quince: 15,
16    dieciseis: 16, "dieciséis": 16, diecisiete: 17, dieciocho: 18,
17    diecinueve: 19, veinte: 20, veinticinco: 25, treinta: 30,
18    cuarenta: 40, cincuenta: 50, sesenta: 60, setenta: 70,
19    ochenta: 80, noventa: 90, cien: 100, ciento: 100,
20    doscientos: 200, quinientos: 500, mil: 1000,
21  }));
22
23  const UNIDADES = new Map(Object.entries({
24    kilo: "Kilogram", kilos: "Kilogram", kilogramo: "Kilogram",
25    kilogramos: "Kilogram", kg: "Kilogram",
26    litro: "Liter", litros: "Liter", l: "Liter",
27    unidad: "Unidad", unidades: "Unidad",
28    porcion: "Portion", porciones: "Portion", "porción": "Portion",
29  }));
30
31  const DECENAS = [20, 30, 40, 50, 60, 70, 80, 90];
32  const CANONICAS = new Set(["Kilogram", "Liter", "Unidad", "Portion"]);
33
34  const RE_NUMERO_DIGITOS = /-?\d+(?:[.]\d+)?/;
35  const RE_PUNTUACION_BORDE = /^[\.,\s!]+|[\.,\s!]+$/g;
36
37  function quitarPuntuacion(palabra) {
38    return palabra.replace(RE_PUNTUACION_BORDE, "");
39  }
40
41  /** La PRIMERA cantidad de la frase. Ignora números que son parte del
42   nombre del producto («cazuela 16 onzas», «tabla 50x38»). */
43  function numero(texto) {
44    const t = texto.toLowerCase();
45    const m = RE_NUMERO_DIGITOS.exec(t);
46    if (m) {
47      const val = parseFloat(m[0].replace(".", ""));
48      return t.slice(0, m.index).includes("menos") ? -val : val;
49    }
50
51    const palabras = t.trim().split(/\s+/).filter(Boolean).map(quitarPuntuacion);
52    let total = null;
53    for (let i = 0; i < palabras.length; i++) {
54      const p = palabras[i];
55      if (!NUMEROS.has(p)) continue;
56      const v = NUMEROS.get(p);
57      total = v;
58      // «ciento ochenta»: centena seguida de decena
59      if (total >= 100 && i + 1 < palabras.length && NUMEROS.has(palabras[i + 1])) {
60        const sig = NUMEROS.get(palabras[i + 1]);
61        if (sig < 100) total += sig;
62        // «treinta y cinco»: decena + y + unidad
63      } else if (DECENAS.includes(total) && i + 2 < palabras.length
64        && palabras[i + 1] === "y" && NUMEROS.has(palabras[i + 2])) {
65        total += NUMEROS.get(palabras[i + 2]);
66      }
67      break; // lo demas pertenece al nombre del producto
68    }
69    if (total !== null && t.includes("menos")) total = -Math.abs(total);
70    return total;
71  }
72
73  function unidad(texto) {
74    for (let p of texto.toLowerCase().replace(/\s+/g, " ").trim().split(/\s+/)) {
75      p = p.replace(/^[.,\s!]+|[\.,\s!]+$/g, "");
76      if (UNIDADES.has(p)) return UNIDADES.get(p);
77    }
78    return null;
79  }

```

```

80
81 /** Lo que sea que devuelva el agente (humano o el modelo) para la unidad,
82     lo lleva a uno de los 4 valores canónicos de la base de datos. */
83 export function normalizarUnidad(dicha) {
84     if (!dicha) return dicha;
85     if (CANONICAS.has(dicha)) return dicha;
86     const clave = String(dicha).trim().toLowerCase();
87     return UNIDADES.has(clave) ? UNIDADES.get(clave) : dicha;
88 }
89
90 const PALABRAS_FUERA = new Set([
91     ...NUMEROS.keys(), ...UNIDADES.keys(),
92     "de", "del", "la", "el", "los", "las", "hay", "en", "y", "por",
93     "confirmo", "confirmar", "son", "hoy", "preparamos", "quiero",
94     "pedir", "insumos", "para", "cuanto", "cuánto", "cuantos",
95     "iniciar", "conteo", "abrir", "bodega", "corregir", "ultimo",
96     "último", "no", "uy", "que", "qué", "me", "genera", "generame",
97     "consolidado", "reporte", "ayuda", "menos", "onzas", "onza",
98 ]);
99
100 function sinRuido(t) {
101     const sinDigitos = t.replace(RE_NUMERO_DIGITOS, " ");
102     return sinDigitos.trim().split(/\s+/).filter(Boolean)
103         .filter((w) => !PALABRAS_FUERA.has(quitaraPuntuacion(w)))
104         .join(" ").trim();
105 }
106
107 const contiene = (f, claves) => claves.some((k) => f.includes(k));
108
109 /** El mismo respaldo de última instancia que interprete.py, corriendo en
110     el navegador: entiende lo básico sin backend ni internet. */
111 export function interpretarLocal(frase) {
112     const f = frase.toLowerCase().trim();
113     const cant = numero(f);
114     const uni = unidad(f);
115
116     if (contiene(f, ["iniciar conteo", "abrir bodega", "abrir la bodega"])) {
117         return { intencion: "navegar", bodega_texto: sinRuido(f),
118             respuesta_hablada: "Voy a abrir esa bodega." };
119     }
120     if (contiene(f, ["preparamos", "vamos a preparar", "pedir insumos"])) {
121         return { intencion: "pedir", preparacion: sinRuido(f),
122             porciones: cant ? Math.trunc(cant) : 0,
123             respuesta_hablada: "Le calculo los insumos según la receta." };
124     }
125     if (f.startsWith("confirmo") || ["confirmar", "si", "sí", "correcto", "confirmo."].includes(f)) {
126         return { intencion: "confirmar", respuesta_hablada: "Listo." };
127     }
128     if (contiene(f, ["corregir", "no son", "no, son", "uy no", "uy, no",
129         "esta mal", "está mal", "me equivoque", "cambiar"])) {
130         return { intencion: "corregir", cantidad: cant, unidad: uni,
131             respuesta_hablada: "Corrijo ese registro." };
132     }
133     if (contiene(f, ["cuanto", "cuánto", "cuantos", "donde hay", "dónde hay"])) {
134         return { intencion: "consultar", articulo_texto: sinRuido(f),
135             respuesta_hablada: "Reviso el inventario." };
136     }
137     if (contiene(f, ["reporte", "consolidado", "archivo", "imprim"])) {
138         return { intencion: "reporte", respuesta_hablada: "Genero el archivo." };
139     }
140     if (f.includes("ayuda") || f.includes("no entiendo")) {
141         return { intencion: "ayuda", respuesta_hablada: "Con gusto. ¿Qué necesita saber?" };
142     }
143     if (cant !== null) {
144         return { intencion: "contar", articulo_texto: sinRuido(f),
145             cantidad: cant, unidad: uni, respuesta_hablada: "" };
146     }
147     return { intencion: "ayuda", respuesta_hablada: "Perdón, no le entendí bien. ¿Me lo repite?" };
148 }

```

frontend/src/colaOffline.js

17 LÍNEAS

La cola de conteos pendientes de sincronizar.

```

1 // La cola de conteos guardados en este equipo mientras no había señal
2 // (ver Conteo.jsx). Vivía solo dentro de Conteo.jsx - invisible desde
3 // cualquier otra pantalla aunque los ítems seguían pendientes de
4 // sincronizar. Se saca a un módulo aparte para que BarraLateral.jsx
5 // también pueda mostrarla, sin duplicar la lógica.
6 export const CLAVE_COLA = (sesionId) => `cv_offline_${sesionId}`;
7 export const EVENTO_COLA = "cuentavoz:cola-offline-cambiada";
8
9 export function leerCola(sesionId) {

```

```

10 try { return JSON.parse(localStorage.getItem(CLAVE_COLA(sesionId)) || "[]"); }
11 catch (_) { return []; }
12 }
13
14 export function guardarCola(sesionId, cola) {
15   localStorage.setItem(CLAVE_COLA(sesionId), JSON.stringify(cola));
16   window.dispatchEvent(new Event(EVENTO_COLA));
17 }

```

frontend/src/accesibilidad.js

20 LÍNEAS

Alto contraste y tamaño de letra.

```

1 // Preferencias de accesibilidad (alto contraste, tamaño de letra) - a
2 // propósito por dispositivo, no por servidor: son ajustes de cómo se ve la
3 // pantalla en ESTE equipo, no algo que deba viajar con la cuenta a otro
4 // dispositivo, y así siguen funcionando sin conexión.
5 const CLAVE = "cv_accesibilidad";
6
7 export function leerPreferencias() {
8   try { return JSON.parse(localStorage.getItem(CLAVE) || "{}"); }
9   catch (_) { return {}; }
10 }
11
12 export function aplicarPreferencias(p) {
13   document.documentElement.setAttribute("data-contraste", p.contraste ? "alto" : "");
14   document.documentElement.setAttribute("data-tamano", p.tamano || "");
15 }
16
17 export function guardarPreferencias(p) {
18   localStorage.setItem(CLAVE, JSON.stringify(p));
19   aplicarPreferencias(p);
20 }

```

frontend/src/confirmacionVoz.js

47 LÍNEAS

Que cuenta como un si hablado.

```

1 /** Detectar un "sí"/"no" hablado o escrito - sin depender de nada más
2   (ni del navegador, ni de la API), para poder probarlo con node --test
3   igual que interpreteLocal.js. voz.js reexporta estas mismas funciones
4   para quien ya las importaba desde ahí. */
5
6 /** Sin esto, "si" (como lo transcribe la voz, con tilde) NUNCA hacía
7   match contra /^(si|sí|...)\b/: en JavaScript \b y \w son ASCII puros,
8   así que "i" no cuenta como letra y la frontera de palabra justo
9   después de la tilde no se reconoce - "sí" (y hasta "sí ver detalle")
10  fallaban en seco, y solo "si" sin tilde funcionaba. Bug real,
11  encontrado en producción, que llevaba viva desde el principio en
12  varios confirmaciones habladas de la app. */
13 export function quitarTildes(t) {
14   return String(t).normalize("NFD").replace(/[\u0300-\u036f]/g, "");
15 }
16
17 const AFIRMACIONES_BASE = ["si", "confirmo", "confirmar", "claro", "dale", "correcto", "listo"];
18 const FRASES_AFIRMATIVAS = ["eso es", "asi es"];
19
20 /** ¿La respuesta hablada/escrita es un "sí" a lo que se le preguntó?
21   "extra" agrega palabras válidas solo en ese contexto puntual - p. ej.
22   "ver" cuando la pregunta fue "¿ve el detalle?", o el texto del botón
23   de turno ("Enviar", "Solicitar...") para que decir esa misma palabra
24   (en cualquier conjugación: "envíalo", "enviar", "envíelo") confirme
25   igual que tocar el botón. */
26 export function esAfirmacion(texto, extra = []) {
27   const t = quitarTildes(String(texto).toLowerCase()).replace(/[:,;!?:?]/g, "");
28   const palabras = t.split(/\s+/).filter(Boolean);
29   if (!palabras.length || palabras.includes("no")) return false;
30   const extrasNorm = extra.map((e) => quitarTildes(e.toLowerCase()));
31   const afirmaciones = [...AFIRMACIONES_BASE, ...extrasNorm];
32   if (palabras.some((p) => afirmaciones.includes(p))) return true;
33   if (FRASES_AFIRMATIVAS.some((f) => t.includes(f))) return true;
34   // conjugaciones del extra ("enviar" -> envía/envíalo/enviando...): se
35   // compara por raíz, quitando la terminación de infinitivo -ar/-er/-ir,
36   // en vez de enumerar cada forma verbal a mano.
37   const raices = extrasNorm.map((e) => e.replace(/(ar|er|ir)$/, "")).filter((r) => r.length >= 3);
38   return raices.length > 0 && palabras.some((p) => raices.some((r) => p.startsWith(r)));
39 }
40
41 /** ¿La respuesta es un "no" limpio? (primera palabra, no en cualquier
42   parte de la frase, para no confundir "no sé, tal vez" con un cierre). */

```

```

43 export function esNegacion(texto) {
44   const t = quitarTildes(String(texto).toLowerCase()).replace(/[,;:!\?]/g, "");
45   const palabras = t.split(/\s+/).filter(Boolean);
46   return palabras[0] === "no";
47 }

```

frontend/src/tutorial.js

27 LÍNEAS

Si el recorrido en video se abre solo.

```

1 // Lógica pura de cuándo mostrar el recorrido en video (ver
2 // VideoRecorrido.jsx), separada en un módulo .js aparte porque node:test no
3 // puede parsear JSX directamente - así esta parte sí se prueba, igual que
4 // colaOffline.js/accesibilidad.js.
5 //
6 // El video sale SIEMPRE al entrar, en cada ingreso - no solo la primera
7 // vez. Lo único que lo apaga es que la propia persona lo desactive desde
8 // un control dentro del video (VideoRecorrido.jsx), y esa bandera se
9 // guarda POR USUARIO (cv_tutorial_deshabilitado_<id>), no una sola para
10 // todo el dispositivo: en una bodega una tablet la comparten varios
11 // auxiliares, y una bandera única haría que un auxiliar apagara el video
12 // para todos los demás que usan el mismo equipo.
13 const PREFIJO_CLAVE = "cv_tutorial_deshabilitado_";
14 export const EVENTO_ABRIR_VIDEO = "cuentavoz:abrir-video";
15
16 export function debeAbrirTutorial(usuarioId) {
17   try { return !localStorage.getItem(PREFIJO_CLAVE + usuarioId); }
18   catch (_) { return true; }
19 }
20
21 export function deshabilitarTutorial(usuarioId) {
22   try { localStorage.setItem(PREFIJO_CLAVE + usuarioId, "1"); } catch (_) {}
23 }
24
25 export function habilitarTutorial(usuarioId) {
26   try { localStorage.removeItem(PREFIJO_CLAVE + usuarioId); } catch (_) {}
27 }

```

frontend/src/correoAdmin.js

52 LÍNEAS

El respaldo de envío por correo.

```

1 /** Envío real de correo para "Soporte en vivo" (mensajes y respuestas
2   entre auxiliares y administrador), compartido entre Ayuda.jsx y
3   Mensajes.jsx para no duplicar credenciales ni lógica.
4
5   La Public Key de EmailJS está hecha para ir en el código del
6   frontend (a diferencia de una clave de API común, no es secreta -
7   así lo documenta EmailJS). El correo sale desde el navegador de
8   quien escribe, usando la cuenta de Gmail conectada en el panel de
9   EmailJS - por eso no pasa por la red de Render, que es lo que
10  bloqueaba el SMTP directo a Gmail y la API de Resend. */
11 const EMAILJS_SERVICE_ID = "service_9dgu33i";
12 const EMAILJS_TEMPLATE_ID = "template_9ddkqpa";
13 const EMAILJS_PUBLIC_KEY = "raCRzjMA9m2ymYuwk";
14
15 export async function enviarPorEmailJS(destinatario, nombreRemitente, mensaje, rol, correoRemitente) {
16   try {
17     const res = await fetch("https://api.emailjs.com/api/v1.0/email/send", {
18       method: "POST",
19       headers: { "Content-Type": "application/json" },
20       body: JSON.stringify({
21         service_id: EMAILJS_SERVICE_ID,
22         template_id: EMAILJS_TEMPLATE_ID,
23         user_id: EMAILJS_PUBLIC_KEY,
24         template_params: {
25           to_email: destinatario, name: nombreRemitente, message: mensaje,
26           rol: rol || "", correo_remitente: correoRemitente || "",
27           fecha: new Date().toLocaleString("es-CO", {
28             day: "numeric", month: "long", year: "numeric", hour: "numeric", minute: "2-digit",
29           }),
30         },
31       }),
32     });
33     return res.ok;
34   } catch {
35     return false;
36   }
37 }
38

```

```

39 // admin.nombre y usuario.nombre llegan en minúscula (el resto de la app
40 // los capitaliza con CSS) - para texto plano (firma del correo, mensajes
41 // de la bandeja) hay que capitalizarlos en JS.
42 export function capitalizar(nombre) {
43   if (!nombre) return "";
44   return nombre.split(" ").map((p) => p.charAt(0).toUpperCase() + p.slice(1)).join(" ");
45 }
46
47 // Misma etiqueta que usa el resto de la app (BarraLateral, MiPerfil,
48 // Ingreso) para el rol - así la firma del correo y la bandeja dicen lo
49 // mismo que ve la persona dentro de CuentaVoz.
50 export function rolDe(usuario) {
51   return usuario?.perfil === "auditor" ? "Administrador de bodega" : "Auxiliar de inventarios";
52 }

```

17.4 Frontend · componentes compartidos

frontend/src/BarraLateral.jsx

116 LÍNEAS

El menu lateral, el avatar y el aviso de cola sin conexion.

```

1 import { useEffect, useState } from "react";
2 import { MENU } from "../App";
3 import Icono from "../Iconos";
4 import { BASE, leerToken } from "../api";
5 import { leerCola, EVENTO_COLA } from "../colaOffline";
6
7 export default function BarraLateral({ activo, usuario, token, sesionId, onNavegar }) {
8   const esAuditor = usuario?.perfil === "auditor";
9   const items = MENU.filter((m) => !m.soloAuditor || esAuditor);
10  const [fotoUrl, setFotoUrl] = useState(null);
11  const [colaLen, setColaLen] = useState(0);
12
13  // La cola de conteos guardados sin conexión (ver Conteo.jsx/colaOffline.js)
14  // antes solo se veía dentro de la pantalla de Conteo - si alguien
15  // cambiaba de pantalla con ítems todavía pendientes, no había ningún
16  // rastro de eso en el resto de la app. El evento propio cubre cambios en
17  // esta misma pestaña; "storage" cubre otra pestaña sincronizando la cola.
18  useEffect(() => {
19    function actualizar() { setColaLen(leerCola(sesionId).length); }
20    actualizar();
21    window.addEventListener(EVENTO_COLA, actualizar);
22    window.addEventListener("storage", actualizar);
23    return () => {
24      window.removeEventListener(EVENTO_COLA, actualizar);
25      window.removeEventListener("storage", actualizar);
26    };
27  }, [sesionId]);
28
29  // Igual que en Mi perfil: <img src> no puede mandar el header
30  // Authorization que /foto exige, así que se trae como blob autenticado.
31  // Se vuelve a pedir cuando Mi perfil avisa que hubo una foto nueva.
32  useEffect(() => {
33    function cargar() {
34      fetch(`${BASE}/api/usuarios/yo/foto`, {
35        headers: { Authorization: `Bearer ${token || leerToken()}` },
36      }).then((r) => (r.ok ? r.blob() : null))
37        .then((b) => setFotoUrl((prev) => {
38          if (prev) URL.revokeObjectURL(prev);
39          return b ? URL.createObjectURL(b) : null;
40        })))
41      // mismo camino que el then() de arriba (revoca la URL anterior
42      // antes de reemplazarla) - un fallo de red aquí no debe quedarse
43      // con el blob viejo vivo para siempre solo porque este intento
44      // en particular no llegó a nada.
45      .catch(() => setFotoUrl((prev) => { if (prev) URL.revokeObjectURL(prev); return null; }));
46    }
47    cargar();
48    window.addEventListener("cuentavoz:foto-actualizada", cargar);
49    return () => window.removeEventListener("cuentavoz:foto-actualizada", cargar);
50  }, [token]);
51
52  return (
53    <nav className="sidebar">
54      <div className="marca">
55        
56        <span>
57          <b>CuentaVoz</b>
58          <i>Tu voz cuenta</i>
59        </span>

```

```

60     </div>
61     <hr />
62
63     <ul>
64       {items.map((m) => (
65         <li>
66           key={m.id}
67           className={m.id === activo ? "sel" : ""}
68           onClick={() => onNavegar(m.id)}
69           role="button"
70           tabIndex={0}
71           aria-current={m.id === activo ? "page" : undefined}
72           // El menú principal solo tenía onClick - inalcanzable con
73           // teclado (Tab no lo enfocaba, nada respondía a Enter/Espacio).
74           // Para quien navega sin mouse (movilidad reducida, o un lector
75           // de pantalla), esto es el único menú de toda la app: sin
76           // esto, no había forma de cambiar de pantalla sin mouse.
77           onKeyDown={(e) => {
78             if (e.key === "Enter" || e.key === " ") {
79               e.preventDefault();
80               onNavegar(m.id);
81             }
82           }}
83         >
84           <span className="icono-menu">
85             <Icono nombre={m.id} tam={19} />
86           </span>
87           <span>{m.titulo}</span>
88         </li>
89       )})
90     </ul>
91
92     <footer>
93       {fotoUrl ? (
94         <img src={fotoUrl} alt="" className="avatar"
95           style={{ objectFit: "cover" }} />
96       ) : (
97         <span className="avatar">
98           {(usuario?.nombre || "?")[0].toUpperCase()}
99         </span>
100       )}
101       <b style={{ fontSize: ".86rem", textTransform: "capitalize" }}>
102         {usuario?.nombre}
103       </b>
104       <small>
105         {usuario?.perfil === "auditor"
106           ? "Administrador de bodega"
107           : "Auxiliar de inventarios"}
108       </small>
109       {colaLen > 0 && (
110         <span className="chip oro" style={{ marginTop: 6 }}>{colaLen} por sincronizar</span>
111       )}
112       
113     </footer>
114   </nav>
115 );
116 }

```

frontend/src/Marco.jsx

19 LÍNEAS

El encabezado comun de cada pantalla.

```

1  /** El armazón que comparten todas las vistas: barra superior con la
2   sección, la faja dorada de marca y el logo de Colsubsidio. */
3
4  import React from "react";
5  export default function Marco({ titulo, chip, children }) {
6    return (
7      <>
8        <div className="barra-sup">
9          <div className="barra-sup-inner">
10             <h1>{titulo}</h1>
11             {chip && <span className={`chip ${chip.tipo || ""}`}>{chip.texto}</span>}
12             
13           </div>
14         </div>
15         <div className="faja" />
16         <div className="area">{children}</div>
17       </>
18     );
19   }

```

frontend/src/Dialogo.jsx

221 LÍNEAS

El cuadro modal: trampa de foco, cierre con Escape, campo con voz.

```

1 import { useState, useRef, useEffect } from "react";
2 import { escuchar, hablar, esAfirmacion, esNegacion } from "../voz";
3 import { interpretarLocal } from "../interpreteLocal";
4 import Icono from "../Iconos";
5
6 /** Reemplaza a window.prompt()/window.confirm(): un modal con el estilo
7     propio de la app, en vez del cuadro genérico y feo del navegador.
8     Uso: <Dialogo titulo="..." mensaje="..." conCampo onAceptar={...} onCancel={...} />
9
10     conVoz agrega el mismo micrófono que ya existe en el resto de la app,
11     con el mismo patrón de confirmación hablada que usa el agente al abrir
12     una bodega: lo dicho llena el campo (visible, editable) Y el agente
13     pregunta en voz alta "¿confirma X?" - un "sí" acepta directo, un "no"
14     descarta y deja intentar de nuevo. El botón de Aceptar sigue ahí como
15     respaldo (para quien prefiera escribir, o si la voz no entendió el
16     sí/no). No se usa en los diálogos "Escribir en vez de hablar": esos ya
17     son el respaldo a prueba de fallos cuando la voz no funciona. */
18 export default function Dialogo({
19     titulo, mensaje, conCampo = false, conVoz = false, multilinea = false, tipo = "text",
20     valorInicial = "", placeholder = "", textoAceptar = "Aceptar",
21     textoCancelar = "Cancelar", peligro = false, confirmarAlAbrir = false, onAceptar, onCancel,
22     // Nombre accesible del campo para un lector de pantalla - por defecto se
23     // usa "mensaje" (funciona bien cuando es una pregunta corta, como "¿Cuántas
24     // porciones son en realidad?"), pero en varios diálogos "mensaje" es un
25     // párrafo explicativo largo o hasta el contenido de un mensaje ajeno
26     // (ver Mensajes.jsx "Responder mensaje": ahí incluye el mensaje ORIGINAL
27     // completo) - leerlo entero cada vez que el campo recibe foco es
28     // ruidoso e inútil como etiqueta. Estos casos pasan su propio
29     // etiquetaCampo, corto y específico.
30     etiquetaCampo,
31 }) {
32     const [valor, setValor] = useState(valorInicial);
33     const [escuchando, setEscuchando] = useState(false);
34     const [confirmando, setConfirmando] = useState(false);
35     const [errorVoz, setErrorVoz] = useState("");
36     // Igual que en el selector de bodega de conteo: cada escucha lleva su
37     // propio número, y si el micrófono anterior entrega su resultado tarde
38     // (por ejemplo, mientras todavía se está diciendo "¿confirma X?") se
39     // descarta en vez de procesarse como si fuera un nuevo valor dictado.
40     const idEscucha = useRef(0);
41     const modalRef = useRef(null);
42
43     // Accesibilidad: quien navega con teclado o con un lector de pantalla
44     // necesita que Escape cierre el modal (como cualquier diálogo nativo),
45     // y que el foco entre al modal apenas aparece - sin esto, un lector de
46     // pantalla seguía anunciando el contenido de atrás como si el modal ni
47     // existiera. Si hay un campo, ese ya se autoenfoca solo (ver más abajo);
48     // si no lo hay, el modal mismo recibe el foco.
49     useEffect(() => {
50         function alTecla(e) { if (e.key === "Escape") onCancel(); }
51         window.addEventListener("keydown", alTecla);
52         return () => window.removeEventListener("keydown", alTecla);
53     }, [onCancel]);
54
55     useEffect(() => {
56         if (!conCampo) modalRef.current?.focus();
57         // eslint-disable-next-line react-hooks/exhaustive-deps
58     }, []);
59
60     function aceptar(valorAUsar = valor) {
61         if (conCampo && !String(valorAUsar).trim()) return;
62         onAceptar(valorAUsar);
63     }
64
65     async function preguntarConfirmacion(valorDictado, miId) {
66         setConfirmando(true);
67         // Con un campo de varias líneas (un mensaje libre, no un dato corto)
68         // no se repite todo lo dictado - sería una lectura larga, y encima
69         // alarga la ventana en la que la persona ya contestó "sí"/"envíalo"
70         // pero el micrófono todavía no se reabrió (seguía sonando la
71         // pregunta). Una confirmación corta reduce esa ventana.
72         const pregunta = multilinea ? "¿Confirma el mensaje?" : `¿Confirma "${valorDictado}"`;
73         // hay que esperar a que termine de decir la pregunta antes de abrir
74         // el micrófono - si arranca apenas empieza a sonar, se pierde el
75         // principio de la respuesta.
76         await hablar(pregunta);
77         if (miId !== idEscucha.current) return;
78         let obtuvoResultado = false;
79         let yaReintentado = false;
80         escuchar({

```

```

81     alTexto: (respuesta) => {
82         if (miId !== idEscucha.current) return;
83         obtuvoResultado = true;
84         setConfirmando(false);
85         // El botón de este diálogo en particular puede decir "Enviar",
86         // "Solicitar", "Crear"... no solo "Aceptar" - decir esa misma
87         // palabra (en cualquier conjugación) también debe confirmar,
88         // igual que tocar el botón.
89         if (esAfirmacion(respuesta, [textoAceptar])) {
90             aceptar(valorDictado);
91         } else if (esNegacion(respuesta)) {
92             setValor("");
93             setErrorVoz("Cancelado. Dígalo de nuevo cuando quiera, o escríbalo.");
94         } else {
95             setErrorVoz("No le entendí un «sí» o un «no». Use el botón para confirmar.");
96         }
97     },
98     alEstado: (e) => {
99         setEscuchando(e === "escuchando");
100        // El reconocedor del navegador se cierra solo tras unos segundos
101        // de silencio, sin avisar con un error (voz.js lo ignora a
102        // propósito) - sin esto, la pregunta se quedaba "colgada" en
103        // pantalla diciendo que estaba esperando un sí/no, cuando en
104        // realidad el micrófono ya se había apagado solo. Se vuelve a
105        // preguntar y a escuchar sola, en vez de obligar a tocar el
106        // micrófono de nuevo (lo que arrancaba una dictada nueva y
107        // borraba el mensaje ya armado).
108        if (e === "listo" && obtuvoResultado && !yaReintento && miId === idEscucha.current) {
109            yaReintento = true;
110            preguntarConfirmacion(valorDictado, miId);
111        }
112    },
113    alError: (e) => { setConfirmando(false); setErrorVoz(e); },
114 });
115 }
116
117 function alTextoDictado(t, miId) {
118     if (!t) return;
119     if (tipo === "number") {
120         // el reconocimiento a veces ya entrega el número tal cual ("50"),
121         // y a veces la palabra ("cincuenta") - se intenta directo primero,
122         // y si no es un número se reusa el mismo interprete de cantidades
123         // que ya usa el conteo por voz, en vez de dejar el campo vacío.
124         const directo = Number(String(t).replace(", ", "."));
125         if (!Number.isNaN(directo)) {
126             setValor(directo);
127             preguntarConfirmacion(directo, miId);
128             return;
129         }
130         const r = interpretarLocal(t);
131         if (r.cantidad !== null) {
132             setValor(r.cantidad);
133             preguntarConfirmacion(r.cantidad, miId);
134             return;
135         }
136         setErrorVoz(`No reconocí un número en «${t}». Dígalo de nuevo o escríbalo.`);
137         return;
138     }
139     setValor(t);
140     preguntarConfirmacion(t, miId);
141 }
142
143 function porVoz() {
144     setErrorVoz("");
145     const miId = ++idEscucha.current;
146     escuchar({
147         alTexto: (t) => {
148             if (miId !== idEscucha.current) return;
149             alTextoDictado(t, miId);
150         },
151         alEstado: (e) => setEscuchando(e === "escuchando"),
152         alError: setErrorVoz,
153     });
154 }
155
156 // Cuando el valor ya llega armado por voz desde afuera (por ejemplo,
157 // "respóndele a Stephanie que ya quedó asignada la bodega" en la
158 // bandeja de Mensajes), tocar el micrófono de este diálogo para decir
159 // "envíalo" NO debe funcionar como una dictada nueva - eso pisaba el
160 // texto ya armado con la palabra "envíalo" misma. En vez de obligar a
161 // usar el botón, se pregunta la confirmación de una vez al abrir, así
162 // "envíalo"/"sí" ya confirma lo que se armó, sin tocar nada.
163 useEffect(() => {
164     if (confirmarAlAbrir && String(valorInicial || "").trim()) {
165         preguntarConfirmacion(valorInicial, ++idEscucha.current);
166     }

```

```

167 // eslint-disable-next-line react-hooks/exhaustive-deps
168 }, []);
169
170 return (
171   <div className="overlay" onClick={onCancelar}>
172     <div className="modal" onClick={(e) => e.stopPropagation()}
173       role="dialog" aria-modal="true" aria-labelledby="dialogo-titulo"
174       ref={modalRef} tabIndex={-1}>
175       <h2 id="dialogo-titulo">{titulo}</h2>
176       {mensaje && <p style={{ whiteSpace: "pre-line" }}>{mensaje}</p>}
177       {conCampo && (
178         <div style={{ display: "flex", gap: 10, marginBottom: conVoz ? 6 : 16,
179           alignItems: multilinea ? "flex-start" : "center" }}>
180           {multilinea ? (
181             <textarea autoFocus value={valor} placeholder={placeholder} rows={3}
182               onChange={(e) => setValor(e.target.value)}
183               aria-label={etiquetaCampo || mensaje || titulo}
184               style={{ flex: 1, padding: "12px 14px",
185                 border: "1px solid var(--borde)", borderRadius: 12,
186                 fontFamily: "inherit", fontSize: "1rem", resize: "vertical" }} />
187             ) : (
188               <input type={tipo} autoFocus value={valor} placeholder={placeholder}
189                 onChange={(e) => setValor(e.target.value)}
190                 onKeyDown={(e) => e.key === "Enter" && aceptar()}
191                 aria-label={etiquetaCampo || mensaje || titulo}
192                 style={{ flex: 1, padding: "12px 14px",
193                   border: "1px solid var(--borde)", borderRadius: 12,
194                   textAlign: "center", fontSize: "1.05rem" }} />
195             )}
196             {conVoz && (
197               <button type="button" className={`mic-btn ${escuchando ? "escuchando" : ""}`}
198                 style={{ width: 46, height: 46, fontSize: "1.2rem", flex: "none" }}
199                 onClick={porVoz} title="díalo en voz alta"
200                 aria-label="Díalo en voz alta"><Icono nombre="microfono" tam={22} /></button>
201             )}
202           </div>
203         )}
204         {conVoz && (
205           <p className="pista" style={{ marginTop: -2, marginBottom: 14 }}>
206             {confirmando ? "Diga «sí» para confirmar, o «no» para intentar de nuevo..."
207               : escuchando ? "Escuchando..."
208               : "Díalo y confírmelo de viva voz, o escríbalo y use el botón."}
209           </p>
210         )}
211         {errorVoz && <p className="error" style={{ marginBottom: 10 }}>{errorVoz}</p>}
212         <div className="botones">
213           <button className="btn borde" onClick={onCancelar}>{textoCancelar}</button>
214           <button className={`btn ${peligro ? "oro" : ""}`} onClick={() => aceptar()}>
215             {textoAceptar}
216           </button>
217         </div>
218       </div>
219     </div>
220   );
221 }

```

frontend/src/AsistenteVoz.jsx

159 LÍNEAS

El agente flotante de cada vista.

```

1 import { useState } from "react";
2 import { escuchar, hablar } from "../voz";
3 import { preguntarAsistente } from "../api";
4 import Icono from "../Iconos";
5
6 /** El mismo microfono que ya existe en Bodegas/Conteo/Pedido, pero para
7   pantallas sin conversacion (Inicio, Ajustes, Ayuda, Reportes, Panel):
8   una pregunta suelta sobre lo que hay en esta pantalla, o una orden de
9   ir a otra - sin el estado de "pendiente"/"opciones" que solo tiene
10  sentido en un conteo o un pedido en curso. */
11 export default function AsistenteVoz({ token, vista, ir, placeholder, alActualizar, ejemplos }) {
12   const [texto, setTexto] = useState("");
13   const [respuesta, setRespuesta] = useState("");
14   const [error, setError] = useState("");
15   const [escuchando, setEscuchando] = useState(false);
16   const [pensando, setPensando] = useState(false);
17
18   async function preguntar(dicho) {
19     if (!dicho || !dicho.trim()) return;
20     setTexto(dicho);
21     setError("");
22     // Algunas pantallas (ej. el filtro de Registro de trazabilidad)
23     // resuelven ciertas frases del todo en el frontend, sin backend ni

```

```

24 // Gemini de por medio - si un componente hijo la reconoce, llama
25 // preventDefault() y avisa así que ya quedó resuelta, para no
26 // preguntarle al agente algo que esta pantalla ya sabe contestar
27 // sola. Sin esto, decir aquí arriba exactamente uno de los
28 // ejemplos de la lista (que sí describen esas frases) no hacía
29 // nada, porque solo el micrófono chiquito junto a los filtros
30 // estaba conectado a esa lógica.
31 const evento = new CustomEvent("cuentavoz:filtro-local", {
32   detail: { texto: dicho }, cancelable: true,
33 });
34 window.dispatchEvent(evento);
35 if (evento.defaultPrevented) {
36   // El componente que lo reclamó (ej. TabTraza) es quien sabe el
37   // resultado real (cuántas filas encontró) - anuncia ESO por su
38   // cuenta de forma asíncrona en vez de un "listo, apliqué el
39   // filtro" genérico que no dice nada útil.
40   return;
41 }
42 setPensando(true);
43 try {
44   const r = await preguntarAsistente(dicho, vista, token);
45   setRespuesta(r.respuesta_hablada || "");
46   hablar(r.respuesta_hablada);
47   if (r.accion === "navegar" && r.destino && ir) {
48     // ir() combina el contexto en vez de reemplazarlo: sin fijar
49     // tabInicial explícitamente (aunque sea undefined), una pestaña
50     // pedida en una navegacion anterior por otro camino se quedaba
51     // pegada y aparecía en un destino que no la tiene. Lo mismo para
52     // bodegaSugerida: sin fijarlo a undefined cuando no aplica, un
53     // "abra kiosco" pedido antes se quedaba pegado y la próxima vez
54     // que se entraba a Conteo por otro camino la abría sola. Lo mismo
55     // para bodegaAuditar, su equivalente en Auditoría.
56     ir(r.destino, { tabInicial: r.pestana || undefined, bodegaSugerida: r.bodega || undefined,
57       bodegaAuditar: r.bodega_auditar || undefined });
58   } else if (r.archivo && r.titulo_archivo && r.pestana && ir) {
59     // Cubre dos casos: "previsualizar_reporte" (pedir VER un archivo
60     // ya generado) y "actualizar" con archivo (GENERAR uno nuevo por
61     // voz - "genera el consolidado", "exporta las diferencias") -
62     // ninguno trae "destino" (no cambian de PANTALLA: ya estamos en
63     // Reportes), pero sí pueden pedirse desde OTRA pestaña de esta
64     // misma pantalla. Sin este ir(), el evento de abajo llegaba a un
65     // TabConsolidado que ni siquiera estaba montado - solo se dibuja
66     // cuando la pestaña activa YA es "consolidado" - así que la vista
67     // previa no aparecía nunca si no se estaba, por casualidad, ya en
68     // esa pestaña (ni al pedir verla, ni al generarla). Va por props
69     // (como bodegaSugerida) en vez del evento genérico de abajo,
70     // precisamente porque necesita sobrevivir a un cambio de pestaña
71     // que todavía no ocurrió cuando se dispara el evento.
72     // r.titulo_archivo exigido a proposito: "exporta el análisis de
73     // consumo" trae archivo+pestana ("análisis") pero SIN
74     // titulo_archivo (usa su propio evento "descargar_reporte", ver
75     // TabAnálisis) - sin este chequeo, ese caso también disparaba
76     // este ir() con titulo undefined, y al volver a la pestaña
77     // "Consolidado" a mano, TabConsolidado remontaba y previsualizaba
78     // ese archivo viejo con "Vista previa · undefined" de encabezado.
79     ir(vista, { tabInicial: r.pestana,
80       archivoPrevisualizar: { archivo: r.archivo, titulo: r.titulo_archivo } });
81   }
82   // "actualizar": el agente cambió algo de verdad (ej. el modo sin
83   // conexión en Ajustes) - la pantalla que lo pidió es quien sabe
84   // cómo refrescar su propio dato, este componente no.
85   if (r.accion === "actualizar" && alActualizar) alActualizar();
86   // Cualquier otra acción (ej. "mostrar_cobertura"/"ocultar_cobertura"
87   // para abrir/cerrar un panel específico de la pantalla) se avisa
88   // como evento genérico - así un componente hijo que le interesa
89   // puede escucharlo sin que este componente necesite saber qué es
90   // cada pantalla ni acumular una prop nueva por cada acción. Ojo:
91   // "previsualizar_reporte" queda afuera a propósito (va por props,
92   // ver arriba) para no disparar el mismo aviso dos veces.
93   if (r.accion && r.accion !== "navegar" && r.accion !== "actualizar"
94     && r.accion !== "previsualizar_reporte") {
95     // detail: r - para acciones que traen datos propios, no solo la
96     // señal de que pasó algo. Las que no lo necesitan lo ignoran.
97     window.dispatchEvent(new CustomEvent(`cuentavoz:accion:${r.accion}`, { detail: r }));
98   }
99 } catch (e) {
100   // todo lo que el agente muestra en pantalla tambien lo dice en voz -
101   // este catch se quedaba solo en rojo, distinto de como ya se
102   // corrigio el mismo patron en Conteo, Pedido, Legalizacion,
103   // Auditoria, Ajustes, Ayuda y Mensajes. Como este es el componente
104   // compartido de Inicio, Mi perfil, Ajustes, Auditoria, Panel y
105   // Reportes, era el hueco mas grande: cubria seis pantallas de una
106   // sola vez.
107   setError(e.message);
108   hablar(e.message);
109 }

```

```

110     setPensando(false);
111   }
112
113   function porVoz() {
114     escuchar({
115       alTexto: preguntar,
116       alEstado: (e) => setEscuchando(e === "escuchando"),
117       alError: setError,
118     });
119   }
120
121   return (
122     <div className="card">
123       <p className="rotulo">Pregúntele al agente</p>
124       <div style={{ display: "flex", gap: 10 }}>
125         <input value={texto} onChange={(e) => setTexto(e.target.value)}
126           onKeyDown={(e) => e.key === "Enter" && preguntar(texto)}
127           placeholder={placeholder || "pregunte algo, o pida ir a otra pantalla..."}
128           aria-label="Pregúntele al agente"
129           style={{ flex: 1, minWidth: 200, padding: "13px 14px", fontSize: "1rem",
130             border: "1px solid var(--borde)", borderRadius: 12 }} />
131         <button className={`mic-btn ${escuchando ? "escuchando" : ""}`}
132           style={{ width: 46, height: 46, fontSize: "1.2rem" }}
133           onClick={porVoz} title="pregúntele por voz" aria-label="Pregúntele al agente por voz">
134           <Icono nombre="microfono" tam={22} /></button>
135       </div>
136       <p className="pista" style={{ marginTop: 8 }}>o pregunte por voz</p>
137       {ejemplos && ejemplos.length > 0 && (
138         <details style={{ marginTop: 8 }}>
139           <summary className="pista" style={{ cursor: "pointer" }}>
140             Ver frases exactas que entiende el agente aquí
141           </summary>
142           <ul style={{ margin: "8px 0 0", paddingLeft: 18 }}>
143             {ejemplos.map((ej, i) => (
144               <li key={i} className="pista" style={{ marginBottom: 4 }}><ej></li>
145             ))}
146           </ul>
147         </details>
148       )}
149       {pensando && <p className="cargando" style={{ marginTop: 10 }}>Pensando...</p>}
150       {respuesta && !pensando && (
151         <
152           <p className="rotulo" style={{ marginTop: 10 }}>CuentaVoz responde</p>
153           <p className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</p>
154         </>
155       )}
156       {error && <p className="error" style={{ marginTop: 10 }}>{error}</p>}
157     </div>
158   );
159 }

```

frontend/src/ChecklistClave.jsx

36 LÍNEAS

Los cuatro requisitos reales de la política de clave.

```

1  import Icono from "../Iconos";
2
3  // Los 4 requisitos reales de la política de clave del User Pool de
4  // Cognito (mín. 8, mayúscula, minúscula, número - SIN símbolo
5  // obligatorio, a diferencia del ejemplo de 5 requisitos de AWS: mostrar
6  // un quinto check por un requisito que Cognito en realidad no exige
7  // sería mentirle a quien está llenando el formulario).
8  const _REQUISITOS = [
9    { id: "longitud", texto: "Al menos 8 caracteres", prueba: (c) => c.length >= 8 },
10   { id: "mayuscula", texto: "Una letra mayúscula", prueba: (c) => /[A-Z]/.test(c) },
11   { id: "minusculta", texto: "Una letra minúscula", prueba: (c) => /[a-z]/.test(c) },
12   { id: "numero", texto: "Un número", prueba: (c) => /[0-9]/.test(c) },
13 ];
14
15 export function claveCumplePolitica(clave) {
16   return _REQUISITOS.every((r) => r.prueba(clave));
17 }
18
19 /** Checklist en vivo (como el de la propia pantalla de registro de
20  Cognito): cada requisito pasa a verde con un check apenas se cumple,
21  en vez de un solo mensaje de error genérico al enviar el formulario. */
22 export default function ChecklistClave({ clave }) {
23   return (
24     <ul className="checklist-clave">
25       {_REQUISITOS.map((r) => {
26         const ok = r.prueba(clave);
27         return (
28           <li key={r.id} className={ok ? "ok" : ""}>

```

```

29         {ok ? <Icono nombre="check" tam={14} /> : <span className="circulo" />}
30         {r.texto}
31       </li>
32     );
33   }}}
34 </ul>
35 );
36 }

```

frontend/src/ConfigurarMFA.jsx

86 LÍNEAS

El QR del segundo factor.

```

1 import { useEffect, useState } from "react";
2 import QRCode from "qrcode";
3 import { iniciarActivacionMFA, confirmarActivacionMFA, uriOtpAuth } from "../cognito";
4
5 /** Los 3 pasos para activar la verificación en dos pasos (instalar,
6  escanear/copiar la llave, confirmar con el código) - un solo
7  componente para no repetir esto en Mi perfil Y en el registro (ver
8  Ingreso.jsx: se ofrece justo después de confirmar el correo). Pide
9  la llave a Cognito apenas se monta - necesita una sesión ya
10 autenticada (self-service, no admin), así que solo se usa después
11 de un login o un registro ya confirmado. */
12 export default function ConfigurarMFA({ nombreUsuario, onActivado, onCancel, textoCancelar = "Cancelar" }) {
13   const [llave, setLlave] = useState("");
14   const [qr, setQr] = useState("");
15   const [mostrarLlave, setMostrarLlave] = useState(false);
16   const [codigo, setCodigo] = useState("");
17   const [err, setErr] = useState("");
18   const [cargando, setCargando] = useState(false);
19
20   useEffect(() => {
21     iniciarActivacionMFA().then(setLlave).catch((e) => setErr(e.message));
22   }, []);
23
24   // el QR se genera del lado del cliente (nadie más necesita verlo) a
25   // partir de la llave secreta que da Cognito.
26   useEffect(() => {
27     if (!llave || !nombreUsuario) { setQr(""); return; }
28     QRCode.toDataURL(uriOtpAuth(llave, nombreUsuario)).then(setQr).catch(() => setQr(""));
29   }, [llave, nombreUsuario]);
30
31   async function confirmar() {
32     setErr(""); setCargando(true);
33     try {
34       await confirmarActivacionMFA(codigo.trim());
35       onActivado();
36     } catch (e) {
37       setErr(e.message);
38     } finally {
39       setCargando(false);
40     }
41   }
42
43   return (
44     <div className="mfa-pasos">
45       <div className="mfa-paso">
46         <span className="num">1</span>
47         <div className="contenido">
48           <b>Instale una app autenticadora</b>
49           <p>Google Authenticator, Microsoft Authenticator o similar, en su celular.</p>
50         </div>
51       </div>
52       <div className="mfa-paso">
53         <span className="num">2</span>
54         <div className="contenido">
55           <b>Escanee el código QR</b>
56           <p>O ingrese la llave a mano en la app.</p>
57           {qr && <img className="mfa-qr" src={qr} alt="Código QR para configurar la app autenticadora" />}
58           <button type="button" className="enlace" style={{ marginTop: 6 }}
59             onClick={() => setMostrarLlave((v) => !v)}>
60             {mostrarLlave ? "Ocultar llave" : "Mostrar llave secreta"}
61           </button>
62           {mostrarLlave && <p className="mfa-llave-secreta">{llave}</p>}
63         </div>
64       </div>
65       <div className="mfa-paso">
66         <span className="num">3</span>
67         <div className="contenido">
68           <b>Ingresa el código</b>
69           <p>El código de 6 dígitos que muestra la app en este momento.</p>
70           <input value={codigo} inputMode="numeric" placeholder="Código"

```

```

71         onChange={(e) => setCodigo(e.target.value)}
72         aria-label="Código de la app autenticadora"
73         style={{ width: "100%", padding: "11px 13px", marginTop: 8,
74                 border: "1px solid var(--borde)", borderRadius: 12 }} />
75     </div>
76 </div>
77 {err && <p className="error" role="alert">{err}</p>}
78 <div className="grilla-botones">
79     <button className="btn" onClick={confirmar} disabled={!codigo || cargando}>
80         {cargando ? "Activando..." : "Activar"}
81     </button>
82     {onCancelar && <button className="btn gris" onClick={onCancelar}>{textoCancelar}</button>}
83 </div>
84 </div>
85 );
86 }

```

frontend/src/VideoRecorrido.jsx

102 LÍNEAS

El recorrido narrado por perfil.

```

1  import { useEffect, useRef, useState } from "react";
2  import { debeAbrirTutorial, deshabilitarTutorial, habilitarTutorial } from "../tutorial";
3
4  /* El video se sirve desde /public, no desde un servicio externo: la app se
5  usa en bodegas con señal irregular y un video en YouTube que no carga es
6  peor que no ofrecerlo. Al venir del mismo origen, el navegador lo trae
7  con el resto de la aplicación. Están fuera del precacheo del service
8  worker (ver vite.config.js) para no obligar a bajar varios megas la
9  primera vez que alguien abre el login. */
10 /* El ?v= no lo lee el servidor: es solo para el cache del navegador. Los
11 mp4 viven en una ruta fija (no llevan hash como los bundles de Vite), así
12 que sin esto quien ya vio el recorrido una vez se queda con la copia
13 vieja aunque publiquemos una corregida. Al volver a grabar un recorrido
14 hay que subirle la fecha a su version. */
15 const VIDEOS = {
16     auxiliar: {
17         src: "/recorrido-auxiliar.mp4",
18         version: "2026-08-24g",
19         titulo: "Recorrido para auxiliares de inventarios",
20     },
21     auditor: {
22         src: "/recorrido-administrador.mp4",
23         version: "2026-08-24g",
24         titulo: "Recorrido para administradores de bodega",
25     },
26 };
27
28 export default function VideoRecorrido({ perfil, usuarioId, onCerrar }) {
29     const video = VIDEOS[perfil === "auditor" ? "auditor" : "auxiliar"];
30     const fuente = `${video.src}?v=${video.version}`;
31     const modalRef = useRef(null);
32     const videoRef = useRef(null);
33     // Refleja lo que ya hay guardado para este usuario, no un estado nuevo
34     // en false: si ya lo habia desactivado antes y por lo que sea el video
35     // se abrió igual (p. ej. "Ver el recorrido en video" desde Ayuda), el
36     // interruptor no debe mostrarse como si estuviera activado.
37     const [desactivado, setDesactivado] = useState(
38         () => usuarioId !== null && !debeAbrirTutorial(usuarioId)
39     );
40
41     useEffect(() => {
42         function alTecla(e) { if (e.key === "Escape") onCerrar(); }
43         window.addEventListener("keydown", alTecla);
44         return () => window.removeEventListener("keydown", alTecla);
45     }, [onCerrar]);
46
47     useEffect(() => {
48         modalRef.current?.focus();
49         // eslint-disable-next-line react-hooks/exhaustive-deps
50     }, []);
51
52     // Se intenta reproducir solo, sin que la persona tenga que darle play:
53     // el video se abre COMO CONSECUENCIA de un clic (entrar, o "Ver el
54     // recorrido en video" en Ayuda), y ese gesto reciente es justo lo que
55     // los navegadores piden para permitir sonido automático. Si de todas
56     // formas lo bloquean (típico cuando la sesión se retomó sola, sin clic
57     // reciente), se queda pausado con los controles listos - nunca se
58     // fuerza a silenciarlo, porque un video narrado mudo no sirve de nada.
59     useEffect(() => {
60         videoRef.current?.play().catch(() => {});
61     }, []);
62

```

```

63   return (
64     <div className="overlay" onClick={onCerrar}>
65       <div className="modal" onClick={(e) => e.stopPropagation()}
66         role="dialog" aria-modal="true" aria-labelledby="video-titulo"
67         ref={modalRef} tabIndex={-1}
68         style={{ maxWidth: 780, width: "92vw" }}>
69         <h2 id="video-titulo" style={{ marginBottom: 4 }}>{video.titulo}</h2>
70         <p className="pista" style={{ marginBottom: 12 }}>
71           Narrado, con un ejemplo de cada pantalla. Puede pausarlo o cerrarlo
72           y seguir después.
73         </p>
74
75         <video ref={videoRef} src={fuente} controls preload="auto" playsInline
76           style={{ width: "100%", display: "block", borderRadius: 10,
77             background: "#000", border: "1px solid var(--borde)" }}>
78           Su navegador no puede reproducir este video.{ " " }
79           <a href={fuente} download>Descárguelo aquí.</a>
80         </video>
81
82         {usuarioId != null && (
83           <label style={{ display: "flex", alignItems: "center", gap: 8,
84             marginTop: 14, fontSize: ".9rem", color: "var(--grafito)" }}>
85             <input type="checkbox" checked={desactivado}
86               onChange={(e) => {
87                 const marcado = e.target.checked;
88                 setDesactivado(marcado);
89                 if (marcado) deshabilitarTutorial(usuarioId);
90                 else habilitarTutorial(usuarioId);
91               }} />
92             No mostrar este video automáticamente la próxima vez que ingrese
93           </label>
94         )}
95
96         <div className="botones" style={{ marginTop: 16 }}>
97           <button className="btn" onClick={onCerrar}>Cerrar</button>
98         </div>
99       </div>
100     </div>
101   );
102 }

```

frontend/src/Iconos.jsx

75 LÍNEAS

Los iconos, en SVG.

```

1  /** Iconos del menú lateral. Uno por cada entrada, en trazo simple.
2   * Heredan el color del texto (currentColor), así el CSS decide si van
3   * en dorado (activo) o en gris azulado (inactivo). Sin fondo. */
4  const TRAZO = {
5    fill: "none",
6    stroke: "currentColor",
7    strokeWidth: 1.8,
8    strokeLinecap: "round",
9    strokeLinejoin: "round",
10 };
11
12  const FORMAS = {
13    // Inicio: casa
14    inicio: <><path d="M3 10.5 12 31.9 7.5" /><path d="M5.5 9.5V20h13V9.5" /><path d="M10 20v-5h4v5" /></>,
15    // Pedidos: nota de pedido con esquina doblada
16    pedidos: <><path d="M6 3h8l4 4v14H6z" /><path d="M14 3v4h4" /><path d="M9 12h6M9 16h4" /></>,
17    // Cuento: micrófono
18    cuento: <><rect x="10" y="3" width="4" height="9" rx="2" /><path d="M6.5 11a5.5 5.5 0 0 0 11 0" /><path d="M12 16.5V20" /><path
19    d="M9 20h6" /></>,
20    // Legalización: balanza (lo pedido contra lo usado)
21    legalizacion: <><path d="M12 4v15" /><path d="M5 7h14" /><path d="M8.5 19h7" /><path d="M2.5 12 5 7l2.5 5a2.5 2.5 0 0 1-5 0z" />
22    <path d="M16.5 12 19 7l2.5 5a2.5 2.5 0 0 1-5 0z" /></>,
23    // Bodegas: bodega con techo a dos aguas
24    bodegas: <><path d="M3 9.5 12 31.9 6.5" /><path d="M5 9v11h14V9" /><path d="M5 14.5h14" /><path d="M12 9v11" /></>,
25    // Auditoría: lupa con visto bueno
26    auditoria: <><circle cx="11" cy="10.5" r="6" /><path d="M15.5 15 20 19.5" /><path d="M8.5 10.5l2 3.5-4" /></>,
27    // Reportes: documento con líneas
28    reportes: <><rect x="5" y="3" width="14" height="18" rx="2" /><path d="M8.5 8h7M8.5 12h7M8.5 16h4" /></>,
29    // Panel: barras
30    panel: <><path d="M4 20h16" /><rect x="6" y="12" width="3" height="6" /><rect x="11" y="7" width="3" height="11" /><rect x="16"
31    y="14" width="3" height="4" /></>,
32    // Ajustes: engranaje
33    ajustes: <><circle cx="12" cy="12" r="3.2" /><path d="M12 3v2.6M12 18.4V21M3 12h2.6M18.4 12h2.6M5.6 5.6l1.9 1.9M16.5 16.5l1.9
34    1.9M18.4 5.6l1.9 1.9M7.5 16.5l-1.9 1.9" /></>,
35    // Ayuda: interrogación en círculo
36    ayuda: <><circle cx="12" cy="12" r="8.5" /><path d="M9.6 9.4a2.5 2.5 0 1 1 3.4 2.3c-.63-1 .9-1 1.6v.3" /><circle cx="12" cy="17"
37    r=".9" fill="currentColor" stroke="none" /></>,
38    // Mensajes: sobre de correo

```

```

34 mensajes: <><rect x="3" y="5.5" width="18" height="13" rx="2" /><path d="M3.5 7 12 13l8.5-6" /></>,
35 // Mi perfil: persona
36 perfil: <><circle cx="12" cy="8" r="3.5" /><path d="M5 20a7 7 0 0 1 14 0" /></>,
37 // Cerrar sesión: puerta con flecha de salida
38 salir: <><path d="M14 4H6v16h8" /><path d="M11 12h9" /><path d="M17 8.5 20.5 12 17 15.5" /></>,
39 // Candado: campo de PIN en el ingreso
40 candado: <><rect x="5" y="11" width="14" height="9" rx="2" /><path d="M8 11V7.5a4 4 0 0 1 8 0V11" /></>,
41 // Descargar: flecha hacia una bandeja
42 descargar: <><path d="M12 3v11.5" /><path d="M7.5 10.5 12 15l4.5-4.5" /><path d="M4.5 17.5v2a2 2 0 0 2 2h11a2 2 0 0 2 2v-2" />
</>,
43 // Aprobar/resolver: visto bueno en círculo
44 aprobar: <><circle cx="12" cy="12" r="8.5" /><path d="M8 12.3l2.6 2.6l16 9.3" /></>,
45 // Visto bueno solo (sin círculo) - checklist de requisitos de clave
46 check: <path d="M4 12.3l5.2 5.2l20 6" />,
47 // Rechazar: equis en círculo
48 rechazar: <><circle cx="12" cy="12" r="8.5" /><path d="M9 9l6 6M15 9l-6 6" /></>,
49 // Advertencia: triángulo con exclamación (reportar un problema)
50 advertencia: <><path d="M12 3.5 21.5 20h-19z" /><path d="M12 9.5v5" /><circle cx="12" cy="17" r=".9" fill="currentColor"
stroke="none" /></>,
51 // Refrescar: dos flechas en ciclo (restaurar valores)
52 refrescar: <><path d="M4 12a8 8 0 0 1 13.66-5.66l20 8.5" /><path d="M20 4v4.5h-4.5" /><path d="M20 12a8 8 0 0 1-13.66 5.66l4 15.5"
/><path d="M4 20v-4.5h4.5" /></>,
53 // Micrófono: mismo trazo que el icono "conteo" del menú, para que
54 // TODO botón de escuchar en la app se vea igual - antes cada uno usaba
55 // el emoji 🎧, que se ve distinto (y en algunos Windows, irreconocible)
56 // según la fuente de emoji del sistema operativo.
57 microfono: <><rect x="10" y="3" width="4" height="9" rx="2" /><path d="M6.5 11a5.5 5.5 0 0 0 11 0" /><path d="M12 16.5V20" /><path
d="M9 20h6" /></>,
58 };
59
60 export default function Icono({ nombre, tam = 20 }) {
61   const forma = FORMAS[nombre];
62   if (!forma) return null;
63   return (
64     <svg
65       width={tam}
66       height={tam}
67       viewBox="0 0 24 24"
68       aria-hidden="true"
69       style={{ flex: "none" }}
70       {...TRAZO}
71     >
72       {forma}
73     </svg>
74   );
75 }

```

17.5 Frontend · las catorce vistas

frontend/src/vistas/Ingreso.jsx

657 LÍNEAS

Ingreso, registro y recuperacion.

```

1 import { useEffect, useState } from "react";
2 import { pedir, esFalloRed } from "../api";
3 import {
4   registrarse, reenviarCodigoRegistro, confirmarRegistro, iniciarSesion,
5   confirmarCodigoMFA, recuperarClave, confirmarNuevaClave, obtenerAtributosUsuario,
6 } from "../cognito";
7 import ChecklistClave, { claveCumplePolitica } from "../ChecklistClave";
8 import ConfigurarMFA from "../ConfigurarMFA";
9 import Icono from "../Iconos";
10
11 const ETIQUETAS_PERFIL = {
12   auxiliar: "Auxiliar de inventarios",
13   auditor: "Administrador de bodega",
14 };
15
16 /** Junta el token de Cognito con el perfil guardado en esta app (perfil,
17   id, nombre...) - lo que App.jsx espera recibir de alEntrar(). Cognito
18   ya confirmó a la persona, pero la fila local (creada por
19   /api/registro-completado, ver confirmarYEntrar) a veces nunca llegó a
20   crearse - por ejemplo si cerró la pestaña justo después de confirmar
21   el código, antes de esa última llamada. En vez de dejar a alguien con
22   una cuenta "a medias" varado en un 401 sin salida, se autocompleta
23   aquí mismo con los datos que Cognito sí tiene (nombre/correo) y se
24   reintenta - transparente para quien inicia sesión. */
25 // El backend responde esto (503) cuando no pudo consultarle a Cognito las
26 // llaves para verificar el token - no es culpa del token ni de la persona
27 // (ver seguridad.py: ServicioIdentidadCaído), así que vale la pena

```

```

28 // reintentar solo en vez de mandarla de vuelta al login.
29 function esCognitoNoDisponible(e) {
30   return e.message === "No se pudo verificar su sesion en este momento. Intente de nuevo.";
31 }
32
33 /** Reintenta lo que falló por algo pasajero, no por culpa de quien ingresa.
34   Dos casos, los dos reales:
35   - El backend vive en el plan gratuito de Render: si lleva 15 minutos sin
36     tráfico se duerme, y la petición que lo despierta tarda 30-50 segundos
37     o se corta en seco. Sin esto, quien se está registrando ve el botón
38     congelado en «Confirmando...», y al reintentar un «Sin conexión con el
39     servidor» que además es falso - su Wi-Fi está bien, el servidor estaba
40     dormido.
41   - Cognito no respondió al verificar el token (ver seguridad.py).
42   alEsperar avisa a la pantalla para que diga qué está pasando en vez de
43   dejar un botón mudo. */
44 // Despertar el servicio dormido de Render tarda entre 30 y 50 segundos, así
45 // que la espera total tiene que cubrir eso con margen: 2+4+6+8+10+12+15 = 57
46 // segundos repartidos en 8 intentos. Un reintentito corto (los 9 s que habia
47 // antes) se agota entero DENTRO de la ventana de arranque y falla igual.
48 const ESPERAS = [2000, 4000, 6000, 8000, 10000, 12000, 15000];
49
50 async function conPaciencia(fn, alEsperar) {
51   for (let intento = 0; ; intento++) {
52     try {
53       return await fn();
54     } catch (e) {
55       const recuperable = esFalloRed(e) || esCognitoNoDisponible(e);
56       if (!recuperable || intento >= ESPERAS.length) {
57         if (recuperable) {
58           // Ya no es "revise su Wi-Fi": la red de esta persona esta bien,
59           // el servidor no alcanza a despertar. Culpar al Wi-Fi manda a
60           // buscar el problema donde no esta.
61           const falla = new Error("El servidor está tardando en responder. "
62             + "Espere unos segundos y vuelva a intentarlo.");
63           falla.esFalloRed = true;
64           throw falla;
65         }
66         throw e;
67       }
68       alEsperar?.(intento + 1);
69       await new Promise((r) => setTimeout(r, ESPERAS[intento]));
70     }
71   }
72 }
73
74 async function sesionCompleta(token, alEsperar) {
75   const perfil = () => conPaciencia(() => pedir("/api/usuarios/yo", {}, token), alEsperar);
76   try {
77     return { token, usuario: await perfil() };
78   } catch (e) {
79     if (e.message !== "Usuario no reconocido.") throw e;
80     const atributos = await obtenerAtributosUsuario();
81     await conPaciencia(() => pedir("/api/registro-completado", {
82       method: "POST",
83       body: JSON.stringify({ nombre_completo: atributos.nombreCompleto, correo: atributos.correo }),
84     }, token), alEsperar);
85     return { token, usuario: await perfil() };
86   }
87 }
88
89 /** Aviso de código enviado por correo, con el icono de sobre y el
90   destino ya enmascarado por Cognito (p***@m***) - misma idea que la
91   pantalla "Confirm your account" de Cognito. */
92 function AvisoCodigo({ destino, onReenviar, reenviando, reenviado }) {
93   return (
94     <>
95     <div className="aviso-codigo">
96       <Icono nombre="mensajes" tam={18} />
97       <span>
98         Le enviamos un código a <b>{destino || "su correo"}</b>. Ingrésele para continuar.
99       </span>
100     </div>
101     <button type="button" className="reenviar-codigo" onClick={onReenviar} disabled={reenviando}>
102       {reenviando ? "Reenviando..." : reenviado ? "Código reenviado" : "Reenviar código"}
103     </button>
104   </>
105 );
106 }
107
108 export default function Ingreso({ alEntrar, avisoInicial }) {
109   // "login" | "login-mfa" | "registro" | "registro-codigo" | "registro-mfa" |
110   // "recuperar" | "recuperar-codigo"
111   const [modo, setModo] = useState("login");
112   // la sesión ya está lista (Cognito + fila local) apenas se confirma el
113   // registro - se guarda aquí mientras se ofrece activar la verificación

```

```

114 // en dos pasos, y solo se entrega a alEntrar() al terminar ese paso
115 // (activarla o saltarla).
116 const [sesionPendiente, setSesionPendiente] = useState(null);
117 const [usuario, setUsuario] = useState("");
118 const [clave, setClave] = useState("");
119 const [err, setErr] = useState(avisosInicial || "");
120 // errores DE UN CAMPO puntual (falta diligenciarlo, o quedó mal), para
121 // señalar justo debajo de ese campo - "err" (arriba) queda para lo que
122 // no es de un campo en particular (usuario o clave incorrectos, fallas
123 // del servidor...).
124 const [errCampo, setErrCampo] = useState({});
125 const [cargando, setCargando] = useState(false);
126 // Lo que está pasando mientras se espera - solo aparece si de verdad hay
127 // que esperar (servidor dormido, ver conPaciencia), para que el botón no
128 // se quede mudo y parezca colgado.
129 const [avisoEspera, setAvisoEspera] = useState("");
130 const avisarEspera = (intento) =>
131   setAvisoEspera("El servidor estaba en reposo y está despertando "
132     + `(intento ${intento})... puede tardar hasta un minuto.`);
133 const [perfil, setPerfil] = useState(null);
134
135 /** onChange que además borra el error de ESE campo apenas la persona
136     empieza a corregirlo, en vez de dejarlo marcado en rojo hasta el
137     próximo intento de enviar. */
138 function actualizar(setter, campo) {
139   return (e) => {
140     setter(e.target.value);
141     setErrCampo((prev) => (prev[campo] ? { ...prev, [campo]: null } : prev));
142   };
143 }
144
145 // registro
146 const [nombreCompleto, setNombreCompleto] = useState("");
147 const [codigoEmpleado, setCodigoEmpleado] = useState("");
148 const [correo, setCorreo] = useState("");
149 const [clave2, setClave2] = useState("");
150 const [codigo, setCodigo] = useState("");
151 const [destinoCodigo, setDestinoCodigo] = useState("");
152 const [reenviando, setReenviando] = useState(false);
153 const [reenviado, setReenviado] = useState(false);
154 // recuperar
155 const [claveNueva, setClaveNueva] = useState("");
156 const [claveNueva2, setClaveNueva2] = useState("");
157 // login con verificación en dos pasos
158 const [mfaPendiente, setMfaPendiente] = useState(null);
159 const [codigoMFA, setCodigoMFA] = useState("");
160
161 // Despierta el backend apenas se abre esta pantalla, sin esperar a que
162 // alguien pulse nada. En el plan gratuito de Render el servicio se duerme
163 // tras 15 minutos sin tráfico y tarda 30-50 segundos en levantarse; ese
164 // tiempo se lo comía la primera acción de la persona (confirmar el código
165 // del correo, justo el peor momento). Iniciándolo aquí, mientras se
166 // escriben los datos y llega el correo, el servidor ya está listo cuando
167 // de verdad hace falta. /api/salud no pide sesión y no cambia nada.
168 useEffect(() => {
169   pedir("/api/salud").catch(() => {});
170 }, []);
171
172 // apenas escribe el usuario se identifica el perfil solo - no hay que
173 // elegirlo a mano. Con demora corta para no disparar una llamada por
174 // cada tecla. Solo aplica en login: en registro el usuario es nuevo,
175 // no tiene perfil todavía que detectar.
176 useEffect(() => {
177   if (modo !== "login") { setPerfil(null); return; }
178   const entrada = usuario.trim();
179   if (!entrada) { setPerfil(null); return; }
180   const espera = setTimeout(() => {
181     pedir(`/api/usuarios/perfil?usuario=${encodeURIComponent(entrada)}`)
182       .then((r) => setPerfil(r.perfil))
183       .catch(() => setPerfil(null));
184   }, 350);
185   return () => clearTimeout(espera);
186 }, [usuario, modo]);
187
188 function cambiarModo(nuevo) {
189   setModo(nuevo);
190   setErr(""); setErrCampo({}); setReenviado(false); setAvisoEspera("");
191 }
192
193 /** Cierra el registro después del paso de la verificación en dos pasos,
194     se haya activado o saltado: entra directo a la aplicación. */
195 function terminarRegistro() {
196   alEntrar(sesionPendiente);
197 }
198
199 async function entrar(e) {

```

```

200     e?.preventDefault();
201     setErr("");
202     const errores = {};
203     if (!usuario.trim()) errores.usuario = "Digite su usuario.";
204     if (!clave) errores.clave = "Digite su clave.";
205     if (Object.keys(errores).length) return setErrCampo(errores);
206     setErrCampo({});
207     setCargando(true);
208     try {
209         const r = await iniciarSesion(usuario.trim(), clave);
210         if (r.mfaPendiente) {
211             setMfaPendiente(r.mfaPendiente);
212             setModo("login-mfa");
213             return;
214         }
215         alEntrar(await sesionCompleta(r.token, avisarEspera));
216     } catch (e2) {
217         if (e2.code === "UserNotConfirmedException") {
218             // se registró antes pero nunca terminó de confirmar el código
219             // (cerró la pestaña, se le olvidó...) - en vez de dejarlo
220             // varado con un error sin salida, se le reenvía un código
221             // nuevo y se le lleva directo a esa pantalla.
222             try {
223                 const { destino } = await reenviarCodigoRegistro(usuario.trim());
224                 setDestinoCodigo(destino);
225                 setModo("registro-codigo");
226             } catch (e3) {
227                 setErr(e3.message);
228             }
229         } else {
230             setErr(e2.message);
231         }
232     } finally {
233         setCargando(false);
234         setAvisoEspera("");
235     }
236 }
237
238 async function confirmarMFALogin(e) {
239     e?.preventDefault();
240     setErr("");
241     if (!codigoMFA.trim()) return setErrCampo({ codigoMFA: "Digite el código de 6 dígitos." });
242     setErrCampo({});
243     setCargando(true);
244     try {
245         const token = await confirmarCodigoMFA(mfaPendiente, codigoMFA.trim());
246         alEntrar(await sesionCompleta(token, avisarEspera));
247     } catch (e2) {
248         setErr(e2.message);
249     } finally {
250         setCargando(false);
251         setAvisoEspera("");
252     }
253 }
254
255 async function pedirRegistro(e) {
256     e?.preventDefault();
257     setErr("");
258     const errores = {};
259     if (!nombreCompleto.trim()) errores.nombreCompleto = "Digite su nombre completo.";
260     if (!correo.trim()) errores.correo = "Digite su correo.";
261     else if (!correo.includes("@") || !correo.includes(".")) errores.correo = "Ese correo no parece válido.";
262     if (!usuario.trim()) errores.usuario = "Elija un usuario.";
263     if (!clave) errores.clave = "Digite una clave.";
264     else if (!claveCumplePolitica(clave)) errores.clave = "La clave no cumple los requisitos de arriba.";
265     if (!clave2) errores.clave2 = "Confirme la clave.";
266     else if (clave !== clave2) errores.clave2 = "Las dos claves no coinciden.";
267     if (Object.keys(errores).length) return setErrCampo(errores);
268     setErrCampo({});
269     setCargando(true);
270     try {
271         const { destino } = await registrarse(usuario.trim(), clave, correo.trim(), nombreCompleto.trim());
272         setDestinoCodigo(destino);
273         setModo("registro-codigo");
274     } catch (e2) {
275         setErr(e2.message);
276     } finally {
277         setCargando(false);
278         setAvisoEspera("");
279     }
280 }
281
282 async function reenviarRegistro() {
283     setReenviando(true); setErr("");
284     try {
285         const { destino } = await reenviarCodigoRegistro(usuario.trim());

```

```

286     if (destino) setDestinoCodigo(destino);
287     setReenviado(true);
288   } catch (e2) {
289     setErr(e2.message);
290   } finally {
291     setReenviando(false);
292   }
293 }
294
295 async function confirmarYEntrar(e) {
296   e?.preventDefault();
297   setErr("");
298   if (!codigo.trim()) return setErrCampo({ codigo: "Digite el código de 6 dígitos." });
299   setErrCampo({});
300   setCargando(true);
301   try {
302     await confirmarRegistro(usuario.trim(), codigo.trim());
303     const r = await iniciarSesion(usuario.trim(), clave);
304     // (una cuenta recién registrada nunca tiene MFA activo todavía,
305     // r.mfaPendiente no aplica aquí)
306     const token = r.token;
307     // el nombre/correo se piden de nuevo a Cognito (no del formulario en
308     // memoria): si esta pantalla se llegó desde "reanudar confirmación"
309     // tras un login fallido (ver entrar()), nunca se llenó el formulario
310     // de registro en esta sesión del navegador - solo Cognito los tiene.
311     const atributos = await obtenerAtributosUsuario();
312     // crea la fila local (perfil auxiliar, siempre - ver main.py) con
313     // los datos personales que Cognito no guarda. Con conPaciencia porque
314     // esta es la PRIMERA llamada al backend de todo el registro: si el
315     // servicio estaba dormido (plan gratuito de Render), le toca a ella
316     // despertarlo, y sin reintento fallaba dejando la cuenta a medias -
317     // confirmada en Cognito pero sin fila local.
318     await conPaciencia(() => pedir("/api/registro-completado", {
319       method: "POST",
320       body: JSON.stringify({
321         nombre_completo: atributos.nombreCompleto,
322         codigo: codigoEmpleado.trim(),
323         correo: atributos.correo,
324       }),
325     }, token), avisarEspera);
326     // antes de entrar del todo, se ofrece (opcional, se puede saltar)
327     // activar la verificación en dos pasos - más fácil que alguien la
328     // active aquí, recién registrado, que que se acuerde de ir a
329     // buscarla después en Mi perfil.
330     setSesionPendiente(await sesionCompleta(token, avisarEspera));
331     setModo("registro-mfa");
332   } catch (e2) {
333     setErr(e2.message);
334   } finally {
335     setCargando(false);
336     setAvisoEspera("");
337   }
338 }
339
340 async function pedirRecuperar(e) {
341   e?.preventDefault();
342   setErr("");
343   if (!usuario.trim()) return setErrCampo({ usuario: "Digite su usuario." });
344   setErrCampo({});
345   setCargando(true);
346   try {
347     const { destino } = await recuperarClave(usuario.trim());
348     setDestinoCodigo(destino);
349     setModo("recuperar-codigo");
350   } catch (e2) {
351     setErr(e2.message);
352   } finally {
353     setCargando(false);
354     setAvisoEspera("");
355   }
356 }
357
358 async function reenviarRecuperar() {
359   setReenviando(true); setErr("");
360   try {
361     const { destino } = await recuperarClave(usuario.trim());
362     if (destino) setDestinoCodigo(destino);
363     setReenviado(true);
364   } catch (e2) {
365     setErr(e2.message);
366   } finally {
367     setReenviando(false);
368   }
369 }
370
371 async function confirmarRecuperar(e) {

```

```

372     e?.preventDefault();
373     setErr("");
374     const errores = {};
375     if (!codigo.trim()) errores.codigo = "Digite el código de 6 dígitos.";
376     if (!claveNueva) errores.claveNueva = "Digite una clave nueva.";
377     else if (!claveCumplePolitica(claveNueva)) errores.claveNueva = "La clave no cumple los requisitos de arriba.";
378     if (!claveNueva2) errores.claveNueva2 = "Confirme la clave nueva.";
379     else if (claveNueva !== claveNueva2) errores.claveNueva2 = "Las dos claves no coinciden.";
380     if (Object.keys(errores).length) return setErrCampo(errores);
381     setErrCampo({});
382     setCargando(true);
383     try {
384       await confirmarNuevaClave(usuario.trim(), codigo.trim(), claveNueva);
385       setClave(""); setCodigo(""); setClaveNueva(""); setClaveNueva2("");
386       setModo("login");
387       setErr("");
388     } catch (e2) {
389       setErr(e2.message);
390     } finally {
391       setCargando(false);
392       setAvisoEspera("");
393     }
394   }
395
396   const campo = { width: "100%", padding: "11px 13px", marginBottom: 8,
397     border: "1px solid var(--borde)", borderRadius: 12 };
398
399   return (
400     <div className="ingreso-fondo">
401       <div className="ingreso">
402         <div className="marca-cabecera">
403           
404           <h1>CuentaVoz</h1>
405           <i>Tu voz cuenta</i>
406           <p className="hackathon">Hackathon Colsubsidio x 30X</p>
407         </div>
408
409         {modo === "login" && (
410           <form className="form" onSubmit={entrar} noValidate>
411             <h2>Iniciar sesión</h2>
412             <p className="pista">Ingrese con su usuario corporativo.</p>
413
414             <label htmlFor="campo-usuario">Usuario</label>
415             <div className="campo-icone">
416               <Icono nombre="perfil" tam={18} />
417               <input
418                 id="campo-usuario"
419                 value={usuario}
420                 onChange={actualizar(setUsuario, "usuario")}
421                 autoComplete="username"
422                 aria-invalid={!errCampo.usuario}
423                 autoFocus
424               />
425             </div>
426             {errCampo.usuario && <span className="error-campo" role="alert">{errCampo.usuario}</span>}
427
428             <label htmlFor="campo-clave">Clave</label>
429             <div className="campo-icone">
430               <Icono nombre="candado" tam={18} />
431               <input
432                 id="campo-clave"
433                 type="password"
434                 value={clave}
435                 onChange={actualizar(setClave, "clave")}
436                 autoComplete="current-password"
437                 aria-invalid={!errCampo.clave}
438               />
439             </div>
440             {errCampo.clave && <span className="error-campo" role="alert">{errCampo.clave}</span>}
441
442             <div>
443               <span id="etiqueta-perfil" style={{ fontSize: ".8rem", color: "var(--grafito)" }}>
444                 Perfil
445               </span>
446               <p aria-live="polite" className="pista" style={{ minHeight: "1.1em", margin: "2px 0 4px" }}>
447                 {perfil ? `Perfil detectado: ${ETIQUETAS_PERFIL[perfil]}` : ""}
448               </p>
449               <div className="perfiles" role="group" aria-labelledby="etiqueta-perfil">
450                 <span className={perfil === "auxiliar" ? "sel" : ""}>{ETIQUETAS_PERFIL.auxiliar}</span>
451                 <span className={perfil === "auditor" ? "sel" : ""}>{ETIQUETAS_PERFIL.auditor}</span>
452               </div>
453             </div>
454
455             {err && <span className="error" role="alert">{err}</span>}
456             {avisoEspera && <span className="pista" role="status">{avisoEspera}</span>}
457

```

```

458     <button className="btn" type="submit" disabled={cargando}>
459       {cargando ? "Entrando..." : "ENTRAR"}
460     </button>
461
462     <p className="enlaces-ingreso">
463       <button type="button" className="enlace" onClick={() => cambiarModo("recuperar")}>
464         Olvidé mi clave
465       </button>
466       {" · "}
467       <button type="button" className="enlace" onClick={() => cambiarModo("registro")}>
468         Crear una cuenta
469       </button>
470     </p>
471   </form>
472 )}
473
474 {modo === "login-mfa" && (
475   <form className="form" onSubmit={confirmarMFALogin} noValidate>
476     <h2>Verificación en dos pasos</h2>
477     <p className="pista">Digite el código de 6 dígitos de su app autenticadora.</p>
478
479     <label htmlFor="mfa-codigo">Código</label>
480     <input id="mfa-codigo" style={campo} value={codigoMFA} inputMode="numeric"
481       onChange={actualizar(setCodigoMFA, "codigoMFA")}
482       aria-invalid={!errCampo.codigoMFA} autoFocus />
483     {errCampo.codigoMFA && <span className="error-campo" role="alert">{errCampo.codigoMFA}</span>}
484
485     {err && <span className="error" role="alert">{err}</span>}
486     {avisoEspera && <span className="pista" role="status">{avisoEspera}</span>}
487
488     <button className="btn" type="submit" disabled={cargando}>
489       {cargando ? "Verificando..." : "VERIFICAR Y ENTRAR"}
490     </button>
491   </form>
492 )}
493
494 {modo === "registro" && (
495   <form className="form" onSubmit={pedirRegistro} noValidate>
496     <h2>Crear una cuenta</h2>
497     <p className="pista">La cuenta queda como auxiliar de inventarios.</p>
498
499     <div className="campos-2col">
500       <div>
501         <label htmlFor="reg-nombre">Nombre completo</label>
502         <input id="reg-nombre" style={campo} value={nombreCompleto}
503           onChange={actualizar(setNombreCompleto, "nombreCompleto")}
504           aria-invalid={!errCampo.nombreCompleto} autoFocus />
505         {errCampo.nombreCompleto && <span className="error-campo" role="alert">{errCampo.nombreCompleto}</span>}
506       </div>
507       <div>
508         <label htmlFor="reg-codigo-empleado">Código de empleado (opcional)</label>
509         <input id="reg-codigo-empleado" style={campo} value={codigoEmpleado}
510           onChange={(e) => setCodigoEmpleado(e.target.value)} />
511       </div>
512     </div>
513
514     <div className="campos-2col">
515       <div>
516         <label htmlFor="reg-correo">Correo corporativo</label>
517         <input id="reg-correo" type="email" style={campo} value={correo}
518           onChange={actualizar(setCorreo, "correo")} autoComplete="email"
519           aria-invalid={!errCampo.correo} />
520         {errCampo.correo && <span className="error-campo" role="alert">{errCampo.correo}</span>}
521       </div>
522       <div>
523         <label htmlFor="reg-usuario">Usuario</label>
524         <input id="reg-usuario" style={campo} value={usuario}
525           onChange={actualizar(setUsuario, "usuario")} autoComplete="username"
526           aria-invalid={!errCampo.usuario} />
527         {errCampo.usuario && <span className="error-campo" role="alert">{errCampo.usuario}</span>}
528       </div>
529     </div>
530
531     <div className="campos-2col">
532       <div>
533         <label htmlFor="reg-clave">Clave</label>
534         <input id="reg-clave" type="password" style={campo} value={clave}
535           onChange={actualizar(setClave, "clave")} autoComplete="new-password"
536           aria-invalid={!errCampo.clave} />
537         {errCampo.clave && <span className="error-campo" role="alert">{errCampo.clave}</span>}
538       </div>
539       <div>
540         <label htmlFor="reg-clave2">Confirmar clave</label>
541         <input id="reg-clave2" type="password" style={campo} value={clave2}
542           onChange={actualizar(setClave2, "clave2")} autoComplete="new-password"
543           aria-invalid={!errCampo.clave2} />

```

```

544         {errCampo.clave2 && <span className="error-campo" role="alert">{errCampo.clave2}</span>}
545     </div>
546 </div>
547 <ChecklistClave clave={clave} />
548
549 {err && <span className="error" role="alert">{err}</span>}
550 {avisoEspera && <span className="pista" role="status">{avisoEspera}</span>}
551
552 <button className="btn" type="submit" disabled={cargando}>
553     {cargando ? "Creando cuenta..." : "CREAR CUENTA"}
554 </button>
555 <p className="enlaces-ingreso">
556     <button type="button" className="enlace" onClick={() => cambiarModo("login")}>
557         Ya tengo una cuenta
558     </button>
559 </p>
560 </form>
561 }}
562
563 {modo === "registro-codigo" && (
564     <form className="form" onSubmit={confirmarYEntrar} noValidate>
565         <h2>Confirmar correo</h2>
566         <AvisoCodigo destino={destinoCodigo} onReenviar={reenviarRegistro}
567             reenviando={reenviando} reenviado={reenviado} />
568
569         <label htmlFor="reg-codigo">Código de 6 dígitos</label>
570         <input id="reg-codigo" style={campo} value={codigo} inputMode="numeric"
571             onChange={actualizar(setCodigo, "codigo")} aria-invalid={!errCampo.codigo} autoFocus />
572         {errCampo.codigo && <span className="error-campo" role="alert">{errCampo.codigo}</span>}
573
574         {err && <span className="error" role="alert">{err}</span>}
575         {avisoEspera && <span className="pista" role="status">{avisoEspera}</span>}
576
577         <button className="btn" type="submit" disabled={cargando}>
578             {cargando ? "Confirmando..." : "CONFIRMAR Y ENTRAR"}
579         </button>
580     </form>
581 ))
582
583 {modo === "registro-mfa" && (
584     <div className="form">
585         <h2>Verificación en dos pasos</h2>
586         <p className="pista">
587             Opcional: pida un código de su celular además de la clave, al ingresar. Se puede
588             activar o desactivar después desde Mi perfil.
589         </p>
590         <ConfigurarMFA nombreUsuario={usuario.trim()}
591             onActivado={terminarRegistro}
592             onCancelar={terminarRegistro}
593             textoCancelar="Ahora no" />
594     </div>
595 ))
596
597 {modo === "recuperar" && (
598     <form className="form" onSubmit={pedirRecuperar} noValidate>
599         <h2>Olvidé mi clave</h2>
600         <p className="pista">Le enviaremos un código al correo registrado.</p>
601
602         <label htmlFor="rec-usuario">Usuario</label>
603         <input id="rec-usuario" style={campo} value={usuario}
604             onChange={actualizar(setUsuario, "usuario")} autoComplete="username"
605             aria-invalid={!errCampo.usuario} autoFocus />
606         {errCampo.usuario && <span className="error-campo" role="alert">{errCampo.usuario}</span>}
607
608         {err && <span className="error" role="alert">{err}</span>}
609         {avisoEspera && <span className="pista" role="status">{avisoEspera}</span>}
610
611         <button className="btn" type="submit" disabled={cargando}>
612             {cargando ? "Enviando..." : "ENVIAR CÓDIGO"}
613         </button>
614         <p className="enlaces-ingreso">
615             <button type="button" className="enlace" onClick={() => cambiarModo("login")}>
616                 Volver a iniciar sesión
617             </button>
618         </p>
619     </form>
620 ))
621
622 {modo === "recuperar-codigo" && (
623     <form className="form" onSubmit={confirmarRecuperar} noValidate>
624         <h2>Nueva clave</h2>
625         <AvisoCodigo destino={destinoCodigo} onReenviar={reenviarRecuperar}
626             reenviando={reenviando} reenviado={reenviado} />
627
628         <label htmlFor="rec-codigo">Código de 6 dígitos</label>
629         <input id="rec-codigo" style={campo} value={codigo} inputMode="numeric">

```

```

630         onChange={actualizar(setCodigo, "codigo")} aria-invalid={!errCampo.codigo} autoFocus />
631         {errCampo.codigo && <span className="error-campo" role="alert">{errCampo.codigo}</span>}
632
633         <label htmlFor="rec-clave">Clave nueva</label>
634         <input id="rec-clave" type="password" style={campo} value={claveNueva}
635             onChange={actualizar(setClaveNueva, "claveNueva")} autoComplete="new-password"
636             aria-invalid={!errCampo.claveNueva} />
637         {errCampo.claveNueva && <span className="error-campo" role="alert">{errCampo.claveNueva}</span>}
638         <ChecklistClave clave={claveNueva} />
639
640         <label htmlFor="rec-clave2">Confirmar clave nueva</label>
641         <input id="rec-clave2" type="password" style={campo} value={claveNueva2}
642             onChange={actualizar(setClaveNueva2, "claveNueva2")} autoComplete="new-password"
643             aria-invalid={!errCampo.claveNueva2} />
644         {errCampo.claveNueva2 && <span className="error-campo" role="alert">{errCampo.claveNueva2}</span>}
645
646         {err && <span className="error" role="alert">{err}</span>}
647         {avisoEspera && <span className="pista" role="status">{avisoEspera}</span>}
648
649         <button className="btn" type="submit" disabled={cargando}>
650             {cargando ? "Guardando..." : "GUARDAR CLAVE NUEVA"}
651         </button>
652     </form>
653 })
654 </div>
655 </div>
656 );
657 }

```

frontend/src/vistas/Inicio.jsx

171 LÍNEAS

El resumen del día.

```

1 import { useEffect, useState } from "react";
2 import { pedir } from "../api";
3 import Marco from "../Marco";
4 import AsistenteVoz from "../AsistenteVoz";
5
6 // timeZone fijo en Bogotá (no el del navegador/equipo): CuentaVoz es para
7 // la operación de Colsubsidio en Colombia, así que la fecha y el saludo
8 // deben leerse en hora de Bogotá aunque el dispositivo esté mal
9 // configurado o alguien entre desde otro huso horario.
10 const HUSO = "America/Bogota";
11 // Función, no constante: si se calculara una sola vez al cargar el módulo
12 // (como antes), un turno que deja la pestaña abierta de un día para otro
13 // -habitual en una tablet de bodega siempre encendida- seguía mostrando la
14 // fecha de ayer en el encabezado hasta recargar la página a la fuerza.
15 function fechaHoy() {
16     return new Date().toLocaleDateString("es-CO", {
17         weekday: "long", day: "numeric", month: "long", year: "numeric",
18         timeZone: HUSO,
19     });
20 }
21
22 /** "Buenos días" antes de mediodía, "Buenas tardes" hasta las 7pm, y
23     "Buenas noches" después - siempre en hora de Bogotá. Sin esto el
24     saludo decía "Buenos días" a cualquier hora, incluso de noche. */
25 function saludoSegunHora() {
26     const hora = Number(
27         new Intl.DateTimeFormat("en-US", { timeZone: HUSO, hour: "numeric", hourCycle: "h23" })
28         .format(new Date())
29     );
30     if (hora < 12) return "Buenos días";
31     if (hora < 19) return "Buenas tardes";
32     return "Buenas noches";
33 }
34
35 // Debe cubrir las mismas acciones que registra backend/seguridad.py:registrar()
36 // (la misma lista que el filtro de Ajustes › Trazabilidad) - si aquí falta
37 // alguna, esa fila cae al gris por defecto aunque tenga su propio color en
38 // todas las demás pantallas, y hasta ahora faltaba más de la mitad,
39 // incluida CONTEO: la acción más frecuente del día a día.
40 const COLOR_ACCION = {
41     FIRMA: "verde", CIERRE: "verde", APROBACION: "verde", CREACION: "verde",
42     CONTEO: "verde",
43     REPORTE: "azul", INGRESO: "azul", APERTURA: "azul", PEDIDO: "azul",
44     RECETA: "azul", LEGALIZACION: "azul", ASIGNACION: "azul", USUARIO: "azul",
45     AJUSTE: "azul", PERFIL: "azul", SOPORTE: "azul",
46     ALERTA: "oro", CORRECCION: "oro", REAPERTURA: "oro", AUDITORIA: "oro",
47     SEGURIDAD: "oro",
48 };
49 const DOT = { verde: "var(--verde)", azul: "var(--azul)",
50     oro: "var(--amarillo)", gris: "var(--grafito)" };

```

```

51
52 export default function Inicio({ token, usuario, ir }) {
53   const [resumen, setResumen] = useState(null);
54   const [actividad, setActividad] = useState([]);
55   const esAuditor = usuario?.perfil === "auditor";
56
57   // Ejemplos de lo que ya entiende el agente aquí: preguntar por los KPI de
58   // esta pantalla, o pedir directo cualquiera de los accesos rápidos de
59   // abajo - decir su nombre ya navega, sin tener que tocarlo.
60   const ejemplos = [
61     "¿cuántas bodegas tengo?", "¿cuántas alertas hay por revisar?",
62     "¿cuál es mi exactitud del mes?", "iniciar un conteo", "ver el tablero",
63     ...(esAuditor ? ["continuar auditoría", "generar reporte"]
64       : ["hacer un pedido", "legalizar servicio"]),
65   ];
66
67   useEffect(() => {
68     pedir("/api/usuarios/yo/resumen", {}, token).then(setResumen).catch(() => {});
69     pedir("/api/trazabilidad/reciente", {}, token).then(setActividad).catch(() => {});
70   }, [token]);
71
72   // "Auditoría"/"Reportes"/"Panel" ni siquiera aparecen en el menú lateral
73   // de un auxiliar (soloAuditor en App.jsx:MENU) - ofrecerlos aquí como
74   // acceso directo llevaba a una pantalla que el propio perfil no puede
75   // usar, aunque el menu de al lado dijera lo contrario.
76   const accesos = esAuditor ? [
77     { t: "Iniciar un conteo", s: "dicte y CuentaVoz registra", d: "conteo", ic: "📊" },
78     { t: "Ver el tablero", s: "estado de las bodegas", d: "bodegas", ic: "📊" },
79     { t: "Continuar auditoría", s: "recuento ciego y aprobaciones", d: "auditoria", ic: "🔍" },
80     { t: "Generar reporte", s: "consolidado del día", d: "reportes", ic: "📄" },
81   ] : [
82     { t: "Iniciar un conteo", s: "dicte y CuentaVoz registra", d: "conteo", ic: "📊" },
83     { t: "Ver el tablero", s: "estado de las bodegas", d: "bodegas", ic: "📊" },
84     { t: "Hacer un pedido", s: "receta y porciones por voz", d: "pedido", ic: "📄" },
85     { t: "Legalizar servicio", s: "sobrante y merma del turno", d: "legalizacion", ic: "👤" },
86   ];
87
88   return (
89     <Marco titulo={`Inicio · ${fechaHoy()}`}
90       chip={{ texto: "TOMA EN CURSOR", tipo: "verde" }}
91       <h2 style={{ fontSize: "1.15rem", color: "var(--azul)", marginBottom: 14 }}
92         {saludoSegunHora()}, <span style={{ textTransform: "capitalize" }}>{usuario?.nombre}</span>.
93         Esto es lo que hay para hoy.
94       </h2>
95
96       <AsistenteVoz token={token} vista="inicio" ir={ir} ejemplos={ejemplos}
97         placeholder="¿cuántas bodegas tengo?, lléveme a bodegas..." />
98
99       <div className="kpi">
100         <div className="kpi">
101           <div className="kpi-cabeza">
102             <span className="icono-kpi">📊</span>
103             <small>Bodegas asignadas a usted</small>
104           </div>
105           <b>{resumen?.bodegas_asignadas ?? "-"}</b>
106           <i>según su perfil</i>
107         </div>
108         <div className="kpi">
109           <div className="kpi-cabeza">
110             <span className="icono-kpi">📄</span>
111             <small>Referencias contadas hoy</small>
112           </div>
113           <b>{resumen?.referencias_hoy ?? "-"}</b>
114           <i>en sus sesiones de conteo</i>
115         </div>
116         <div className={`kpi ${resumen?.alertas_por_revisar ? "oro" : "verde"}`}>
117           <div className="kpi-cabeza">
118             <span className="icono-kpi">{resumen?.alertas_por_revisar ? "🚨" : "✅}</span>
119             <small>Alertas por revisar</small>
120           </div>
121           <b>{resumen?.alertas_por_revisar ?? "-"}</b>
122           <i>en toda la operación</i>
123         </div>
124         <div className="kpi verde">
125           <div className="kpi-cabeza">
126             <span className="icono-kpi">📊</span>
127             <small>Su exactitud del mes</small>
128           </div>
129           <b>{resumen ? `${resumen.exactitud_mes} %` : "-"}</b>
130           <i>promedio de bodegas cerradas</i>
131         </div>
132       </div>
133
134       <div className="card">
135         <h3>¿Qué desea hacer?</h3>
136         <div className="accesos-grid">

```

```

137     {accesos.map((a) => (
138       <button key={a.d} className="acceso-rapido"
139         onClick={() => ir(a.d)}>
140         <span className="icono-accion">{a.ic}</span>
141         <b>{a.t}</b>
142         <small>{a.s}</small>
143       </button>
144     )})
145   </div>
146 </div>
147
148 <div className="card">
149   <h3>Actividad reciente</h3>
150   {actividad.length === 0 ? (
151     <p className="vacio">Todavía no hay actividad registrada hoy.</p>
152   ) : (
153     actividad.map((a, i) => {
154       const color = COLOR_ACCION[a.accion] || "gris";
155       return (
156         <div className="registro" key={i}>
157           <span style={{ width: 9, height: 9, borderRadius: "50%", flex: "none",
158             background: DOT[color] }} />
159           <b style={{ color: "var(--grafito)", minWidth: 46 }}>{a.hora}</b>
160           <span>{a.detalle}</span>
161           <span className={`etiqueta-actividad ${color}`}>
162             {a.accion.toLowerCase()}
163           </span>
164         </div>
165       );
166     })
167   )}
168 </div>
169 </Marco>
170 );
171 }

```

frontend/src/vistas/Pedido.jsx

849 LÍNEAS

Del plato a los insumos.

```

1 import { useState, useEffect } from "react";
2 import { pedir, enviarTurno, descargarReporte } from "../api";
3 import { escuchar, hablar } from "../voz";
4 import Marco from "../Marco";
5 import Dialogo from "../Dialogo";
6 import Icono from "../Iconos";
7
8 const DIA = new Date().toLocaleDateString("es-CO", { weekday: "long" });
9 // Palabras sueltas que valen como "sí" a lo que se le esté preguntando
10 // (confirmar un guardado, o enviar el pedido calculado).
11 const AFIRMACIONES = ["confirmo", "confirmar", "sí", "si", "claro", "listo", "dale",
12   "correcto", "hazlo", "adelante", "procede"];
13 // Frases de varias palabras que también confirman, tal como las dice una
14 // persona real ("así es", "va, mándalo") y no calzarían con el chequeo por
15 // palabra exacta de arriba.
16 const FRASES_AFIRMATIVAS = ["así es", "así es", "eso es", "va pues", "de una"];
17
18 function quitarTildes(t) {
19   return t.normalize("NFD").replace(/[\u0300-\u036f]/g, "");
20 }
21
22 function esConfirmacion(texto) {
23   const t = quitarTildes(texto.toLowerCase()).replace(/[.,!¿?]/g, "");
24   const palabras = t.split(/\s+/);
25   if (palabras.includes("no")) return false; // "no lo envíes" no es un sí
26   if (palabras.some((p) => AFIRMACIONES.includes(quitarTildes(p)))) return true;
27   if (FRASES_AFIRMATIVAS.some((f) => t.includes(quitarTildes(f)))) return true;
28   // cualquier conjugación de "enviar"/"mandar" cuenta como un sí a la
29   // pregunta de mandar el pedido ("envíalo", "envíelo", "envíenmelo",
30   // "mándalo", "mándemelo...") - enumerar cada forma verbal a mano es
31   // interminable y siempre se queda corto; sin esto, decir la forma
32   // "usted" en vez de la forma "tú" hacía que la pantalla repitiera el
33   // mismo mensaje en vez de entender que la persona ya dijo que sí.
34   return /envi|mand/.test(t);
35 }
36
37 /** Un "no" limpio a lo que se le está preguntando (¿prepara otro plato?,
38   ¿lo enviamos?), sin que además mencione un plato nuevo - si mencionara
39   uno, no es un cierre sino un cambio de tema, y eso lo debe interpretar
40   Gemini como corresponde, no cerrarse en seco aquí. */
41 function esNegacionSimple(texto, platosConocidos) {
42   const t = quitarTildes(texto.toLowerCase()).replace(/[.,!¿?]/g, "");
43   const palabras = t.split(/\s+/);

```

```

44   if (!palabras.includes("no")) return false;
45   const mencionaOtroPlato = platosConocidos.some(
46     (nombre) => t.includes(quitarTildes(nombre.toLowerCase()))
47   );
48   return !mencionaOtroPlato;
49 }
50
51 // Pedidos se sale y se vuelve a entrar todo el tiempo (a mirar otra pantalla
52 // a mitad de un pedido, por ejemplo): sin guardar esto, cada regreso mostraba
53 // la pantalla en blanco como si la conversación nunca hubiera pasado, aunque
54 // el chef ya llevara todo un plato armado. Se guarda en sessionStorage (no
55 // sobrevive a cerrar la pestaña) y por persona, para que dos sesiones en el
56 // mismo navegador no se mezclen.
57 function claveEstadoPedido(sesionId) {
58   return `cv_pedido_estado_${sesionId}`;
59 }
60 function cargarEstadoPedido(sesionId) {
61   try { return JSON.parse(sessionStorage.getItem(claveEstadoPedido(sesionId)) || "{}"); }
62   catch { return {}; }
63 }
64
65 export default function Pedido({ token, usuario, ir }) {
66   const esAuditor = usuario?.perfil === "auditor";
67   // Pedidos tiene su propia conversación con el agente, aislada de la del
68   // conteo físico: usa un numero de sesion negativo (unico por persona) para
69   // que nunca choque con una sesion real de bodega (esas siempre son >= 1).
70   const sesionId = -1000 - (usuario?.id || 0);
71   const guardado = cargarEstadoPedido(sesionId);
72   // Vacío hasta que de verdad se dicte algo: mostrar «ajiacos/50» desde el
73   // arranque hacia parecer que ya se había entendido un pedido que nadie
74   // dictó todavía, y esos datos no cambiaban con nada más que se dijera.
75   const [plato, setPlato] = useState(guardado.plato || "");
76   const [porciones, setPorciones] = useState(guardado.porciones ?? null);
77   const [dicho, setDicho] = useState(guardado.dicho || "");
78   const [lineas, setLineas] = useState(guardado.lineas ?? null);
79   const [receta, setReceta] = useState(null);
80   const [enviado, setEnviado] = useState(guardado.enviado || false);
81   const [recibo, setRecibo] = useState(guardado.recibo ?? null);
82   const [respuesta, setRespuesta] = useState(
83     guardado.respuesta || "Diga el plato y las porciones: «hoy preparamos cincuenta ajiacos»."
84   );
85   const [estado, setEstado] = useState("listo");
86   const [err, setErr] = useState("");
87   const [archivo, setArchivo] = useState(guardado.archivo ?? null);
88   const [pedirPorciones, setPedirPorciones] = useState(false);
89   const [avisos, setAvisos] = useState(guardado.avisos ?? null);
90   const [opciones, setOpciones] = useState(guardado.opciones ?? null);
91   const [opcionesPara, setOpcionesPara] = useState(guardado.opcionesPara ?? null);
92   const [productoConsultado, setProductoConsultado] = useState(guardado.productoConsultado ?? null);
93   const [bodegas, setBodegas] = useState([]);
94   const [bodegaId, setBodegaId] = useState(guardado.bodegaId ?? null);
95   const [platos, setPlatos] = useState([]);
96   const [offline, setOffline] = useState(!navigator.onLine);
97
98   // a diferencia de Conteo (que solo guarda "conté X de Y" y sincroniza
99   // después), calcular un pedido necesita el stock EN VIVO de la bodega
100  // para saber cuánto falta pedir - no hay forma honesta de simularlo sin
101  // conexión sin arriesgar un cálculo con datos viejos. Por eso aquí no
102  // hay cola offline: solo se avisa con claridad que hay que esperar a
103  // tener señal, y lo que el chef ya dictó no se pierde (sigue guardado
104  // en sessionStorage, ver claveEstadoPedido arriba).
105  useEffect(() => {
106    const alVolver = () => setOffline(false);
107    const alPerder = () => setOffline(true);
108    window.addEventListener("online", alVolver);
109    window.addEventListener("offline", alPerder);
110    return () => {
111      window.removeEventListener("online", alVolver);
112      window.removeEventListener("offline", alPerder);
113    };
114  }, []);
115
116  // Guarda una foto de la conversación en cada cambio relevante, para que
117  // salir y volver a esta pantalla la deje tal cual como quedó.
118  useEffect(() => {
119    const foto = { plato, porciones, dicho, lineas, enviado, recibo, respuesta,
120      archivo, avisos, opciones, opcionesPara, productoConsultado, bodegaId };
121    sessionStorage.setItem(claveEstadoPedido(sesionId), JSON.stringify(foto));
122  }, [plato, porciones, dicho, lineas, enviado, recibo, respuesta,
123    archivo, avisos, opciones, opcionesPara, productoConsultado, bodegaId, sesionId]);
124
125  // para que el chef sepa que puede pedir sin tener que adivinar el nombre
126  // exacto de una receta que quizá no existe todavía.
127  useEffect(() => {
128    pedir("/api/recetas", {}, token).then(setPlatos).catch(() => {});
129  }, [token]);

```

```

130
131 function elegirPlato(nombre) {
132   setErr("");
133   setDicho(`«${nombre}» (elegido de la lista de recetas)`);
134   setPlato(nombre);
135   setLineas(null);
136   setReceta(null);
137   setEnviado(false);
138   setRecibo(null);
139   setProductoConsultado(null);
140   const msg = `Listo, ${nombre.toLowerCase()}. ¿Para cuántas porciones?`;
141   setRespuesta(msg);
142   hablar(msg);
143   setPedirPorciones(true);
144 }
145
146 // el agente saluda primero en vez de esperar a que el chef adivine que
147 // hay que hablar: sin esto el mensaje de bienvenida solo se veía en la
148 // pantalla, nunca se oía, y no parecía un asistente por voz de verdad.
149 // Ahora que la conversación se guarda (ver arriba), esto ya no necesita
150 // adivinar por tiempo: si ya hay un plato en curso (porque se restauró de
151 // una visita anterior), es un pedido que sigue abierto y no se interrumpe;
152 // si no hay nada, es un pedido nuevo de verdad y sí saluda.
153 useEffect(() => {
154   if (!plato) {
155     hablar("Dígame por favor qué va a preparar y para cuántas porciones.");
156   }
157   // eslint-disable-next-line react-hooks/exhaustive-deps
158 }, []);
159
160 // Con qué bodega se descuenta el pedido: por defecto el almacén de
161 // Alimentos y Bebidas (donde de verdad están los ingredientes de cocina),
162 // no una bodega al azar - antes quedaba fijo en la primera de la lista
163 // (una oficina administrativa sin ningún stock) y todo salía en cero.
164 // Si ya había una bodega elegida de una visita anterior, se respeta esa.
165 useEffect(() => {
166   pedir("/api/bodegas", {}, token).then((bs) => {
167     setBodegas(bs);
168     if (bodegaId) return;
169     const conStock = bs.find((b) => /ALMACEN\s*AYB/i.test(b.bodega))
170       || bs.find((b) => b.referencias > 0) || bs[0];
171     if (conStock) setBodegaId(conStock.id);
172   }).catch(() => {});
173   // eslint-disable-next-line react-hooks/exhaustive-deps
174 }, [token]);
175
176 // El recibo que se guarda en sessionStorage es una foto del momento en
177 // que se envió: si el administrador lo aprueba o lo rechaza después, esta
178 // pantalla no se entera sola. Sin esto, alguien podía recargar la página
179 // horas después de un rechazo real y seguir viendo "pendiente de
180 // aprobación", con CuentaVoz repitiendo que hay que esperar al
181 // administrador cuando eso ya no es cierto.
182 useEffect(() => {
183   if (!recibo || recibo.estado !== "pendiente_aprobacion") return;
184   pedir(`/api/pedidos/${recibo.numero_pedido}/estado`, {}, token)
185     .then((r) => {
186       if (r.estado === recibo.estado) return;
187       setRecibo((prev) => (prev ? { ...prev, estado: r.estado } : prev));
188       if (r.estado === "rechazado") {
189         const msg = `El administrador rechazó el pedido ${recibo.numero_pedido}. `
190           + "¿Desea armar otro pedido?";
191         setRespuesta(msg);
192         hablar(msg);
193       } else if (r.estado === "abierto") {
194         const msg = `El pedido ${recibo.numero_pedido} ya fue aprobado y enviado al almacén. `
195           + "¿Desea armar otro pedido?";
196         setRespuesta(msg);
197         hablar(msg);
198       }
199     })
200     .catch(() => {});
201   // eslint-disable-next-line react-hooks/exhaustive-deps
202 }, []);
203
204 async function verReceta(platoForzado) {
205   setErr("");
206   const pl = platoForzado ?? plato;
207   if (!pl) {
208     const msg = "Primero dígame qué va a preparar.";
209     setRespuesta(msg);
210     hablar(msg);
211     return;
212   }
213   try {
214     const r = await pedir(`/api/pedidos/receta?plato=${encodeURIComponent(pl)}`, {}, token);
215     if (!r.lineas) {

```

```

216 // todo lo que el agente muestra en pantalla también lo dice en voz -
217 // esto se quedaba solo en rojo, así que alguien sin mirar la
218 // pantalla no se enteraba de que la receta no se encontró.
219 const msg = `No encontré una receta para «${pl}». Pruebe con ajiaco o sancocho.`;
220 setErr(msg);
221 hablar(msg);
222 return;
223 }
224 setReceta(r);
225 if (r.faltantes_catalogo?.length) {
226   setAvisos({ faltantes: r.faltantes_catalogo, sinRegistro: [] });
227 }
228 } catch (e) {
229   setErr(e.message);
230   hablar(e.message);
231 }
232 }
233
234 /** La frase del chef la interpreta el agente. «mostrar» es lo que se ve
235     como «lo que dijo el chef»; «texto» es lo que de verdad se manda al
236     backend. Al tocar una tarjeta de opción se manda el código exacto,
237     nunca el nombre: si un nombre está contenido en el otro (ARROZ
238     dentro de ARROZ BASMATI), el agente no tiene con qué distinguirlos. */
239 async function dictar(texto, mostrar = texto) {
240   setErr("");
241   setDicho(`«${mostrar}»`);
242   setEstado("listo");
243
244   // «confirmo» aquí no es del conteo físico: es seguir con el pedido.
245   if (esConfirmacion(texto)) {
246     // Si lo último que dijo fue "no" a enviarlo, un "sí" suelto un rato
247     // después no debe mandarlo de golpe - puede ser un "sí" para otra
248     // cosa (confirmando que entendió, por ejemplo), no una segunda
249     // vuelta pidiendo enviarlo. Un "envíalo"/"mándalo" explícito sí
250     // pasa de una: no hay ambigüedad posible en eso.
251     const esExplicitoEnviar = /envi|mand/.test(quitarTildes(texto.toLowerCase()));
252     if (!esExplicitoEnviar && respuesta.includes("Para empezar uno nuevo")
253         && lineas && !enviado) {
254       const msg = "Para enviarlo dígalos de nuevo con «envíalo», o dígame otro plato para uno nuevo.";
255       setRespuesta(msg);
256       hablar(msg);
257       return;
258     }
259     if (lineas && !enviado && porPedir === 0) {
260       // igual que el botón, que se oculta en este caso: no hay nada que
261       // enviar, así que "envíalo" no debe crear un pedido vacío.
262       const msg = "No hay nada que enviar: ya tiene todo en la bodega. ¿Prepara otro plato?";
263       setRespuesta(msg);
264       hablar(msg);
265       return;
266     }
267     if (lineas && !enviado) return enviar();
268     if (!lineas) return calcular();
269     const msg = "Este pedido ya se envió al almacén. ¿Desea armar otro pedido?";
270     setRespuesta(msg);
271     hablar(msg);
272     return;
273   }
274
275   // un "no" limpio (sin mencionar otro plato) cierra la conversación en
276   // vez de caer en Gemini, que sin una intención propia para "declinar"
277   // terminaba repitiendo el mismo mensaje una y otra vez.
278   if (esNegacionSimple(texto, platos.map((p) => p.nombre))) {
279     // si ya le habíamos contestado con la misma frase de cierre (porque
280     // ya dijo "no" antes, a "¿lo enviamos?" o a esta misma), un segundo
281     // "no" no debe repetirla - hay que dar el cierre final de una vez,
282     // no seguir insistiendo con la misma pregunta.
283     const yaSeDespidio = respuesta.includes("Para empezar uno nuevo")
284       || respuesta.includes("estaré pendiente");
285     const msg = yaSeDespidio
286       ? "Listo, gracias. Quedo pendiente si requiere consultar otro pedido. Que tenga un buen día."
287       : (lineas && !enviado && porPedir > 0)
288         ? "A propósito no termina en «sí o no?»: un «sí» aquí se
289           // confundiría con el «sí» que envía el pedido (esConfirmacion
290           // más abajo lo manda directo con lineas && !enviado), justo lo
291           // opuesto de lo que la persona acaba de decir. En cambio se
292           // describe dónde queda el pedido y cómo empezar uno nuevo -
293           // nombrar otro plato ya lo hace, sin ambigüedad.
294           ? "Está bien, no lo envío por ahora; el pedido queda calculado aquí, "
295             + "pendiente de enviar cuando usted quiera. Para empezar uno nuevo, "
296             + "dígame otro plato."
297           : "Listo, gracias. Que tenga un buen día – aquí estaré pendiente si desea consultar otro pedido.";
298     setRespuesta(msg);
299     hablar(msg);
300     return;
301   }

```

```

302
303   try {
304     const t = await enviarTurno(texto, sesionId, token,
305       { opciones, opcionesPara, preparacion: plato, porciones });
306     // un plato nuevo es un tema distinto: la última consulta de producto
307     // ya no aplica y solo estorbaría junto al pedido que se está armando.
308     if (t.preparacion && t.porciones) setProductoConsultado(null);
309     if (t.preparacion && t.preparacion !== plato) {
310       // cambio de plato: las porciones y lo ya calculado eran del plato
311       // anterior, no de este - si no vienen porciones nuevas, se limpian
312       // en vez de dejar el numero viejo puesto como si ya se hubiera dicho.
313       setPlato(t.preparacion);
314       setPorciones(t.porciones || null);
315       setLineas(null);
316       setReceta(null);
317       setEnviado(false);
318       setRecibo(null);
319     } else if (t.porciones) {
320       setPorciones(t.porciones);
321     }
322     setRespuesta(t.respuesta_hablada || "");
323     setArchivo(t.archivo || null);
324     setOpciones(t.opciones || null);
325     setOpcionesPara(t.opciones_para || null);
326     if (t.producto_consultado) setProductoConsultado(t.producto_consultado);
327     if (t.receta) setReceta(t.receta);
328     // el chef ya dijo plato y porciones en la misma frase: el agente dice
329     // "voy a consultar la receta y los insumos" en su respuesta, así que
330     // hay que calcular de una - si no, la pantalla se queda sin nada
331     // mientras el mensaje hablado ya prometió que se iba a hacer. Ese
332     // cálculo ya trae su propio mensaje hablado (calcular() más abajo);
333     // hablar también esta respuesta intermedia disparaba dos audios casi
334     // simultáneos que competían por la misma voz neuronal, y si uno de
335     // los dos fallaba caía en la voz de respaldo del navegador, sonaba
336     // como si hablaran dos personas distintas.
337     if (t.preparacion && t.porciones) {
338       calcular(t.porciones, t.preparacion);
339     } else {
340       hablar(t.respuesta_hablada);
341     }
342   } catch (e) {
343     setErr(e.message);
344   }
345 }
346
347 function corregirCantidad() {
348   setPedirPorciones(true);
349 }
350
351 function confirmarPorciones(v) {
352   setPedirPorciones(false);
353   if (v && !isNaN(Number(v))) {
354     const p = Number(v);
355     // cero o negativo no es una cantidad real de porciones - sin este
356     // chequeo, el campo lo dejaba pasar igual (aquí "0" es un string no
357     // vacío, así que sí entra al if de arriba) y el cálculo salía con
358     // "ya tiene todo, no hace falta pedir nada" para cualquier receta.
359     if (p <= 0) {
360       const msg = "Las porciones tienen que ser un número mayor a cero. ¿Cuántas son en realidad?";
361       setRespuesta(msg);
362       hablar(msg);
363       return;
364     }
365     setPorciones(p);
366     setLineas(null);
367     // sin esto, tras elegir un plato de la lista y poner las porciones no
368     // pasaba nada visible hasta darle aparte a "Calcular el pedido" - se
369     // sentía como si el numero no hubiera servido de nada.
370     calcular(p);
371   }
372 }
373
374 /** El backend explota la receta y descuenta el stock.
375     porcionesForzadas/platoForzado: para poder calcular con el valor recién
376     confirmado sin esperar al próximo render (si no, calcular() vería el
377     plato o las porciones viejas por el cierre de la función). */
378 async function calcular(porcionesForzadas, platoForzado) {
379   setErr("");
380   const p = porcionesForzadas ?? porciones;
381   const pl = platoForzado ?? plato;
382   if (!pl || !p) {
383     const msg = "Primero dígame qué va a preparar y para cuántas porciones.";
384     setRespuesta(msg);
385     hablar(msg);
386     return;
387   }

```

```

388     setEnviado(false);
389     setAvisos(null);
390     setRecibo(null);
391     try {
392         const r = await pedir("/api/pedidos/calcular", {
393             method: "POST",
394             body: JSON.stringify({ plato: pl, porciones: Number(p), bodega_id: bodegaId }),
395         }, token);
396         if (!r.receta_encontrada) {
397             // no se queda solo con el texto rojo en silencio: cierra la
398             // conversación con una salida clara en vez de dejar la pantalla
399             // varada con un plato que no tiene receta y ningún paso siguiente.
400             const msg = `No encontré una receta para «${pl}». Elija uno de los platos de la lista o dígame otro nombre.`;
401             setErr(msg);
402             setRespuesta(msg);
403             hablar(msg);
404             setPlato("");
405             setPorciones(null);
406             return;
407         }
408         setLineas(r.lineas);
409         if (r.faltantes_catalogo?.length || r.sin_registro_bodega?.length) {
410             setAvisos({ faltantes: r.faltantes_catalogo, sinRegistro: r.sin_registro_bodega });
411         }
412         const faltan = r.lineas.filter((l) => l.falta > 0).length;
413         const msg = faltan === 0
414             ? `Ya tiene los ${r.lineas.length} insumos que necesita en la bodega; no hace falta pedir nada al almacén. ¿Prepara otro
plato?`
415             : `Necesita ${r.lineas.length} insumos. Hay que pedir ${faltan} al almacén; el resto ya está en la bodega. ¿Lo enviamos?`;
416         setRespuesta(msg);
417         hablar(msg);
418         // ya se calculó el pedido con esta receta: de una se muestra también,
419         // sin que la persona tenga que pedirla aparte con "Ver la receta".
420         verReceta(pl);
421     } catch (e) {
422         setErr(e.message);
423         hablar(e.message);
424     }
425 }
426
427 async function enviar() {
428     try {
429         const r = await pedir("/api/pedidos/enviar", {
430             method: "POST",
431             body: JSON.stringify({ servicio_id: 1, plato, porciones, bodega_id: bodegaId }),
432         }, token);
433         if (r.duplicado) {
434             const msg = "Este pedido ya se había enviado antes; no lo mandé dos veces. ¿Desea armar otro pedido?";
435             setRespuesta(msg);
436             hablar(msg);
437             setEnviado(true);
438             return;
439         }
440         const msg = r.estado === "pendiente_aprobacion"
441             ? `Pedido ${r.numero_pedido} registrado. Queda pendiente de que el administrador lo apruebe antes de salir del almacén. ¿Desea
armar otro pedido?`
442             : `Pedido ${r.numero_pedido} registrado y enviado a ${r.bodega || "el almacén"}. ¿Desea armar otro pedido?`;
443         setRespuesta(msg);
444         hablar(msg);
445         setEnviado(true);
446         setRecibo(r);
447     } catch (e) {
448         setErr(e.message);
449         hablar(e.message);
450     }
451 }
452
453 const porPedir = lineas ? lineas.filter((l) => l.falta > 0).length : 0;
454
455 return (
456     <Marco titulo="Pedidos · Cocina Piscilago"
457         chip={offline ? { texto: "SIN CONEXIÓN", tipo: "oro" }
458             : { texto: "SERVICIO ALMUERZO", tipo: "" }}>
459     {offline && (
460         <div className="banner">
461             <span className="ico">!</span>
462             <span>
463                 <b>Sin conexión</b>
464                 <span>Calcular un pedido necesita el stock en vivo de la bodega, así que
465                     no se puede hacer sin señal. Lo que ya dictó no se pierde - siga
466                     cuando vuelva la conexión y retome donde quedó.</span>
467             </span>
468         </div>
469     )}
470     <div className="chips">
471         <span className="chip">{DIA[0].toUpperCase() + DIA.slice(1)} · almuerzo</span>

```

```

472 <span className="chip">Cocina Piscilago</span>
473 <span className={`chip ${!enviado ? "oro"
474 : recibo?.estado === "rechazado" ? "rojo"
475 : recibo?.estado === "pendiente_aprobacion" ? "oro" : "verde"}}`>
476 {!enviado ? "Pedido sin enviar"
477 : recibo?.estado === "rechazado" ? "Pedido rechazado"
478 : recibo?.estado === "pendiente_aprobacion" ? "Pendiente de aprobación"
479 : "Pedido enviado"}
480 </span>
481 {bodegas.length > 0 && (
482   <select value={bodegaId} || "">
483     onChange={(e) => { setBodegaId(Number(e.target.value)); setLineas(null); }}
484     aria-label="Bodega de la que se descuenta el pedido"
485     style={{ marginLeft: "auto", padding: "6px 12px", border: "1px solid var(--borde)",
486       borderRadius: 20, fontSize: ".8rem", fontWeight: 700, color: "var(--azul)" }}>
487     {bodegas.map((b) => (
488       <option key={b.id} value={b.id}>Se descuenta de: {b.bodega}</option>
489     ))}
490   </select>
491 )}
492 </div>
493
494 <div className="conteo-cols">
495   <div className="card">
496     <h3>Lo que dijo el chef</h3>
497     <p className="cita">
498       {dicho || "Aún no ha dictado nada: pulse el micrófono y diga, por "
499       + "ejemplo, «hoy preparamos cincuenta ajiacos.»"}
500     </p>
501
502     <h3 style={{ marginTop: 18 }}>Lo que entendió CuentaVoz</h3>
503     {(productoConsultado || (plato && porciones)) ? (
504       <table>
505         <tbody>
506           {productoConsultado && (
507             <>
508               <tr><td>Último producto consultado</td><td><b style={{ color: "var(--azul)" }}>
509                 {productoConsultado.nombre}</b></td></tr>
510               <tr><td>Código · unidad</td><td><b style={{ color: "var(--azul)" }}>
511                 {productoConsultado.codigo} · {productoConsultado.unidad}</b></td></tr>
512             </>
513           )}
514           {plato && porciones && (
515             <>
516               <tr><td>Preparación</td><td><b style={{ color: "var(--azul)" }}>
517                 {plato.toUpperCase()}</b></td></tr>
518               <tr><td>Porciones</td><td><b style={{ color: "var(--azul)" }}>
519                 {porciones}</b></td></tr>
520               <tr><td>Receta aplicada</td><td><b style={{ color: "var(--azul)" }}>
521                 estándar de Colsubsidio</b></td></tr>
522               <tr><td>Momento</td><td><b style={{ color: "var(--azul)" }}>
523                 pedido al almacén</b></td></tr>
524             </>
525           )}
526         </tbody>
527       </table>
528     ) : (
529       <p className="vacio">Todavía no hay ningún plato dictado en esta conversación.</p>
530     )}
531 </div>
532
533 <div className="card mic-caja">
534   <button className={`mic-btn ${estado === "escuchando" ? "escuchando" : ""}`>
535     onClick={() => escuchar({
536       alTexto: dictar,
537       alEstado: setEstado,
538       alError: setErr,
539     })}
540     aria-label="Hable para decir el plato y las porciones">
541     <Icono nombre="microfono" tam={52} />
542   </button>
543   <b>{estado === "escuchando" ? "Escuchando..." : "Mantenga presionado y hable"}</b>
544   <small>También sirve: «pedir insumos para 30 sancochos»</small>
545   <small style={{ color: "var(--verde)", fontWeight: 700 }}>
546     Receta + stock = pedido
547   </small>
548   <details style={{ marginTop: 4, textAlign: "left", width: "100%" }}>
549     <summary className="pista" style={{ cursor: "pointer" }}>
550       Ver frases exactas que entiende el agente aquí
551     </summary>
552     <ul style={{ margin: "8px 0 0", paddingLeft: 18 }}>
553       {["hoy preparamos cincuenta ajiacos", "pedir insumos para 30 sancochos",
554        "cambia a treinta porciones", "sí", "envíalo", "no",
555        "corregir cantidad"].map((ej, i) => (
556         <li key={i} className="pista" style={{ marginBottom: 4 }}>«{ej}»</li>
557       ))}

```

```

558     </ul>
559   </details>
560 </div>
561 </div>
562
563 {platos.length > 0 && (
564   <div className="card">
565     <h3>🍲 Platos con receta registrada</h3>
566     <p className="pista" style={{ marginBottom: 10 }}>
567       Estos son los únicos platos que CuentaVoz sabe calcular por ahora. Toque uno
568       para armar el pedido sin dictarlo, o dígalos tal cual suena aquí.
569     </p>
570     <div style={{ display: "flex", flexWrap: "wrap", gap: 8 }}>
571       {platos.map((r) => (
572         <button key={r.id}
573           className={`chip ${plato.toUpperCase() === r.nombre.toUpperCase() ? "azul" : ""}`}
574           onClick={() => elegirPlato(r.nombre)}>
575         {r.nombre.toUpperCase()}
576       </button>
577     )}}
578   </div>
579 </div>
580 )}
581
582 <div className="card">
583   <p className="rotulo">CuentaVoz responde</p>
584   <div className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</div>
585   {opciones && (
586     <div className="opciones" style={{ marginTop: 14 }}>
587       {opciones.map((o, i) => (
588         <button key={o.codigo} className="opcion" onClick={() => dictar(o.codigo, o.nombre)}>
589           <span className="n">Opción {i + 1}</span>
590           <h4>{o.nombre}</h4>
591           <p>Código {o.codigo}</p>
592           <p>{o.unidad}</p>
593         </button>
594       )}}
595     </div>
596   )}
597   {archivo && (
598     <div className="grilla-botones" style={{ marginTop: 10 }}>
599       <button className="btn verde"
600         onClick={() => descargarReporte(archivo, token).catch((e) => setErr(e.message))}
601         style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
602         <Icono nombre="descargar" tam={16} />
603         Descargar archivo generado
604       </button>
605     </div>
606   )}
607   {err && <p className="error" style={{ marginTop: 10 }}>{err}</p>}
608   <div className="grilla-botones">
609     <button className="btn" onClick={() => calcular()} disabled={offline}
610       style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
611       <Icono nombre="pedidos" tam={16} />
612       Calcular el pedido
613     </button>
614     <button className="btn borde" onClick={corregirCantidad}
615       style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
616       <Icono nombre="refrescar" tam={16} />
617       Corregir cantidad
618     </button>
619     {!receta && (
620       <button className="btn oro" onClick={() => verReceta()}
621         style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
622       <Icono nombre="reportes" tam={16} />
623       Ver la receta
624     </button>
625   )}
626 </div>
627 </div>
628
629 {avisos && (avisos.faltantes.length > 0 || avisos.sinRegistro.length > 0) && (
630   <div className="banner">
631     <span className="ico">!</span>
632     <span>
633       <b>Revise antes de enviar</b>
634       {avisos.faltantes.length > 0 && (
635         <span>
636           No encontré en el catálogo: <b>{avisos.faltantes.join(", ")}</b> – esos
637           ingredientes de la receta no se pudieron calcular. Avise al administrador.
638         <br />
639       </span>
640     )}
641     {avisos.sinRegistro.length > 0 && (
642       <span>
643         Esta bodega nunca ha tenido registro de: <b>{avisos.sinRegistro.join(", ")}</b> –

```

```

644         se asumió 0 disponible; confirme con el almacén antes de descontar existencias.
645     </span>
646     })
647 </span>
648 </div>
649 })
650
651 {receta && (
652     <div className="card">
653         <div className="chips">
654             <span className="chip">{receta.nombre}</span>
655             <span className="chip">Rendimiento: {receta.rendimiento} porción</span>
656             <span className="chip">{receta.lineas.length} ingredientes</span>
657             <span className={`chip ${esAuditor ? "azul" : "gris"}`}>
658                 {esAuditor ? "ADMINISTRA ESTA RECETA" : "SOLO CONSULTA"}
659             </span>
660         </div>
661         <h3>Catálogo de Colsubsidio</h3>
662         <table>
663             <thead>
664                 <tr><th>Ingrediente</th><th>Unidad</th><th>Por porción</th>
665                 <th>Para {porciones} porciones</th></tr>
666             </thead>
667             <tbody>
668                 {receta.lineas.map((l) => (
669                     <tr key={l.codigo}>
670                         <td>{l.nombre}</td><td>{l.unidad}</td><td>{l.por_porcion}</td>
671                         <td className="dif">{Math.round(l.por_porcion * porciones * 1000) / 1000}</td>
672                     </tr>
673                 ))}
674             </tbody>
675         </table>
676         {receta.preparacion ? (
677             <>
678                 <h3 style={{ marginTop: 18 }}>Preparación paso a paso</h3>
679                 <p style={{ whiteSpace: "pre-line", fontSize: ".92rem" }}>{receta.preparacion}</p>
680             </>
681         ) : (
682             <p className="pista" style={{ marginTop: 18 }}>
683                 Esta receta todavía no tiene preparación paso a paso cargada.
684                 {esAuditor && " Agréguela desde Ajustes → Recetas."}
685             </p>
686         )}
687         <p className="pista" style={{ marginTop: 10 }}>
688             {esAuditor
689                 ? "Como administradora puede editar cantidades, agregar o quitar ingredientes, o crear un plato nuevo desde Ajustes →
690 Recetas."
691                 : "Esta receta la mantiene el administrador; si algo está desactualizado, avísele desde Reportes."}
692         </p>
693         <div className="grilla-botones">
694             <button className="btn" onClick={() => { setReceta(null); calcular(); }}
695                 style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
696                 <Icono nombre="pedidos" tam={16} />
697                 Usar para un pedido
698             </button>
699             {esAuditor && (
700                 <button className="btn borde"
701                     onClick={() => ir && ir("ajustes", { tabInicial: "recetas", recetaId: receta.id })}
702                     style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
703                     <Icono nombre="ajustes" tam={16} />
704                     Editar receta
705                 </button>
706             )}
707             <button className="btn gris" onClick={() => setReceta(null)}
708                 style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
709                 <Icono nombre="salir" tam={16} />
710                 Cerrar
711             </button>
712         </div>
713     </div>
714 })
715
716 {lineas && (
717     <>
718         <div className="kpi">
719             <div className="kpi">
720                 <div className="kpi-cabeza">
721                     <span className="icono-kpi">📋</span>
722                     <small>Insumos de la receta</small>
723                 </div>
724                 <b>{lineas.length}</b>
725                 <i>ingredientes estándar</i>
726             </div>
727             <div className="kpi verde">
728                 <div className="kpi-cabeza">
729                     <span className="icono-kpi">✅</span>

```

```

729         <small>Ya disponibles en bodega</small>
730     </div>
731     <b>{lineas.length - porPedir}</b><i>no se piden</i>
732 </div>
733 <div className="kpi oro">
734     <div className="kpi-cabeza">
735         <span className="icono-kpi">🏠</span>
736         <small>Hay que pedir al almacén</small>
737     </div>
738     <b>{porPedir}</b>
739     <i>faltante calculado</i>
740 </div>
741 </div>
742
743 <div className="card">
744     <h3>Insumos calculados para {porciones} {plato}</h3>
745     <table>
746         <thead>
747             <tr>
748                 <th>Insumo (nombre oficial)</th><th>Unidad</th>
749                 <th>Necesario</th><th>Hay en bodega</th><th>Falta: pedir</th>
750             </tr>
751         </thead>
752         <tbody>
753             {lineas.map((l) => (
754                 <tr key={l.codigo}>
755                     <td>{l.nombre}</td>
756                     <td>{l.unidad}</td>
757                     <td>{l.necesario}</td>
758                     <td>{l.stock}</td>
759                     <td className={l.falta > 0 ? "falta" : ""}>
760                         {l.falta > 0 ? l.falta : "-"}
761                     </td>
762                 </tr>
763             ))}
764         </tbody>
765     </table>
766     <p className="pista" style={{ marginTop: 10 }}>
767         La cantidad necesaria sale de la receta por porción; el faltante es
768         lo necesario menos lo que ya hay en la bodega.
769     </p>
770     {porPedir === 0 ? (
771         <p className="msg-ok" style={{ marginTop: 10 }}>
772             ✅ Ya tiene todo en la bodega: no hace falta enviar nada al almacén.
773         </p>
774     ) : !enviado && (
775         <div className="grilla-botones">
776             <button className="btn verde" onClick={enviar} disabled={offline}
777                 style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
778                 <Icono nombre="bodegas" tam={16} />
779                 Enviar pedido al almacén
780             </button>
781         </div>
782     )}
783 </div>
784 </>
785 })
786
787 {recibo && (
788     <div className="card" style={{ border: `1px solid var(--${
789         recibo.estado === "rechazado" ? "rojo"
790         : recibo.estado === "pendiente_aprobacion" ? "amarillo" : "verde"})` }}>
791         <div className="chips">
792             {recibo.estado === "rechazado" ? (
793                 <span className="chip rojo">❌ RECHAZADO POR EL ADMINISTRADOR</span>
794             ) : recibo.estado === "pendiente_aprobacion" ? (
795                 <span className="chip oro">🕒 PENDIENTE DE APROBACIÓN</span>
796             ) : (
797                 <span className="chip verde">✅ PEDIDO REGISTRADO</span>
798             )}
799             <span className="chip">{recibo.numero_pedido}</span>
800         </div>
801         <h3 style={{ marginTop: 12 }}>Recibo de pedido</h3>
802         <table>
803             <tbody>
804                 <tr><td>Número de pedido</td><td><b style={{ color: "var(--verde)" }}>
805                     {recibo.numero_pedido}</b></td></tr>
806                 <tr><td>Plato</td><td><b>{plato.toUpperCase()}</b> · {porciones} porciones</td></tr>
807                 <tr><td>Bodega de descuento</td><td><b>{recibo.bodega || "-"}</b></td></tr>
808                 <tr><td>Registrado por</td><td style={{ textTransform: "capitalize" }}>{recibo.persona}</td></tr>
809                 <tr><td>Hora</td><td>{recibo.hora}</td></tr>
810                 <tr><td>Insumos solicitados</td><td>{recibo.total_lineas}</td></tr>
811             </tbody>
812         </table>
813         {recibo.items?.length > 0 && (
814             <>

```

```

815     <h3 style={{ marginTop: 16 }}>Detalle enviado al almacén</h3>
816     <table>
817       <thead><tr><th>Insumo</th><th>Cantidad</th><th>Unidad</th></tr></thead>
818       <tbody>
819         {recibo.items.map((it, i) => (
820           <tr key={i}><td>{it.nombre}</td><td>{it.cantidad}</td><td>{it.unidad}</td></tr>
821         ))}
822       </tbody>
823     </table>
824   </>
825 ))}
826 <p className="pista" style={{ marginTop: 10 }}>
827   {recibo.estado === "rechazado"
828     ? "El administrador de bodega rechazó este pedido: no salió nada del almacén. Dígame el plato de nuevo para armar uno
nuevo."
829     : recibo.estado === "pendiente_aprobacion"
830     ? "El administrador de bodega debe aprobar este pedido en Auditoría antes de que cuente como enviado de verdad al
almacén."
831     : "Este pedido queda guardado en el sistema y aparecerá tal cual en la legalización del servicio, comparando lo pedido
contra lo consumido."}
832   </p>
833   <div className="grilla-botones">
834     <button className="btn gris" onClick={() => window.print()}>Imprimir recibo</button>
835   </div>
836 </div>
837 )}
838
839 {pedirPorciones && (
840   <Dialogo titulo={porciones ? "Corregir cantidad" : "Porciones"}
841     mensaje={porciones ? "¿Cuántas porciones son en realidad?"
842       : `¿Para cuántas porciones prepara ${plato.toLowerCase()}?`}
843     conCampo conVoz tipo="number" valorInicial={porciones ?? ""}
844     onAceptar={confirmarPorciones}
845     onCancel={(() => setPedirPorciones(false))} />
846 )}
847 </Marco>
848 );
849 }

```

frontend/src/vistas/Conteo.jsx

1174 LÍNEAS

El corazón del sistema: dictar, confirmar, corregir, y el modo sin conexión.

```

1 import { useState, useEffect, useRef } from "react";
2 import { enviarTurno, abrirBodega, buscarBodega, pedir, descargarReporte, esFalloRed } from "../api";
3 import { escuchar, hablar, vozDisponible, esAfirmacion, esNegacion } from "../voz";
4 import { interpretarLocal } from "../interpreteLocal";
5 import { leerCola, guardarCola } from "../colaOffline";
6 import Marco from "../Marco";
7 import Dialogo from "../Dialogo";
8 import Icono from "../Iconos";
9
10 const UNIDADES = ["Unidad", "Kilogram", "Liter", "Portion"];
11 const CLAVE_CATALOGO = "cv_catalogo_ligero";
12
13 function leerCatalogo() {
14   try { return JSON.parse(localStorage.getItem(CLAVE_CATALOGO) || "[]"); }
15   catch (_) { return []; }
16 }
17 function guardarCatalogo(lista) {
18   try { localStorage.setItem(CLAVE_CATALOGO, JSON.stringify(lista)); } catch (_) {}
19 }
20 function normalizarLocal(t) {
21   return (t || "").toUpperCase().normalize("NFD").replace(/[-]/g, "").trim();
22 }
23 /** Sugerencia sin internet: coincidencia de palabras contra el catálogo
24   que ya se guardó en el equipo mientras había señal. */
25 function sugerirDeCatalogo(texto, catalogo) {
26   const q = normalizarLocal(texto);
27   const palabras = q.split(/\s+/).filter((p) => p.length >= 3);
28   if (!palabras.length || !catalogo.length) return null;
29   let mejor = null, mejorPuntaje = 0;
30   for (const a of catalogo) {
31     const nombre = normalizarLocal(a.nombre);
32     const aciertos = palabras.filter((p) => nombre.includes(p)).length;
33     const puntaje = aciertos / palabras.length;
34     if (puntaje > mejorPuntaje) { mejorPuntaje = puntaje; mejor = a; }
35   }
36   return mejorPuntaje >= 0.5 ? mejor : null;
37 }
38
39 const ETIQUETA_BODEGA = {

```

```

40 pendiente: ["PENDIENTE", "gris"], en_conteo: ["EN CONTEO", "azul"],
41 en_auditoria: ["EN AUDITORÍA", "oro"], cerrada: ["CERRADA", "verde"],
42 };
43
44 export default function Conteo({ token, sessionId, ir, usuario, bodegaSugerida, alAbrirBodega }) {
45   const [bodega, setBodega] = useState(null);
46   const [asignadas, setAsignadas] = useState(null);
47   const [todas, setTodas] = useState(null);
48   // llega desde Bodegas cuando alguien buscó por voz/texto el nombre de
49   // una bodega en el buscador de INGREDIENTES (no encontró artículo, pero
50   // el nombre sí es una bodega real) - deja el nombre ya listo para
51   // confirmar con "Abrir por nombre exacto" en vez de tener que dictarlo
52   // otra vez desde cero.
53   const [textoBodega, setTextoBodega] = useState(bodegaSugerida || "");
54   const [buscarOtra, setBuscarOtra] = useState(false);
55   const [bodegaNoEncontrada, setBodegaNoEncontrada] = useState(null);
56   const [opcionesBodega, setOpcionesBodega] = useState(null);
57   // El nombre real que se le preguntó "¿confirma que abro X?" y sigue sin
58   // contestar - se guarda aparte del listener de voz porque el
59   // reconocimiento del navegador se apaga solo tras un rato de silencio: si
60   // la persona se demora en responder, ese listener ya no existe cuando por
61   // fin dice "sí", así que un segundo toque al micrófono debe seguir
62   // tratando esa respuesta como el sí/no pendiente, no como un nombre de
63   // bodega nuevo (antes terminaba ofreciendo crear una bodega llamada "sí").
64   const [bodegaParaConfirmar, setBodegaParaConfirmar] = useState(null);
65   const [msgBodega, setMsgBodega] = useState("");
66   const [dicho, setDicho] = useState("");
67   const [respuesta, setRespuesta] = useState(
68     "Abra una bodega para empezar a contar."
69   );
70   const [opciones, setOpciones] = useState(null);
71   const [opcionesPara, setOpcionesPara] = useState(null);
72   const [alerta, setAlerta] = useState(null);
73   const [alertaEsperado, setAlertaEsperado] = useState(null);
74   const [contextoAlerta, setContextoAlerta] = useState(null);
75   const [pendiente, setPendiente] = useState(null);
76   const [avance, setAvance] = useState({ hechas: 0, total: 0, alertas: 0, ultimos: [] });
77   const [estado, setEstado] = useState("listo");
78   const [err, setErr] = useState("");
79   const [crear, setCrear] = useState(null); // {nombre, unidad_medida, cantidad_inicial}
80   const [archivo, setArchivo] = useState(null);
81   const [mostrarTeclado, setMostrarTeclado] = useState(false);
82   const [offline, setOffline] = useState(!navigator.onLine);
83   const [cola, setCola] = useState(() => leerCola(sessionId));
84   const rec = useRef(null);
85   // Cada vez que se abre un micrófono para el flujo de bodega (nombre
86   // dictado, "¿cuál de todas?", "¿confirma?") sube en 1: el reconocedor
87   // anterior a veces tarda en apagarse del todo y llega a entregar un
88   // resultado extra y tardío (p. ej. recoger también el "confirмо" de la
89   // pregunta siguiente). Cada escucha guarda SU número; si para cuando
90   // responde ya no es el número vigente, se descarta sin hacer nada -
91   // así lo tardío nunca se confunde con una bodega nueva ni con una
92   // respuesta a la pregunta equivocada.
93   const idEscuchaBodega = useRef(0);
94
95   // guarda el catalogo liviano en el equipo mientras hay señal, para que
96   // el modo sin conexión pueda seguir sugiriendo nombres oficiales.
97   useEffect(() => {
98     if (navigator.onLine && token) {
99       pedir("/api/articulos/catalogo-ligero", {}, token).then(guardarCatalogo).catch(() => {});
100     }
101   }, [token]);
102
103   async function refrescar() {
104     try {
105       const bid = bodega?.bodega_id ? `?bodega_id_respaldo=${bodega.bodega_id}` : "";
106       setAvance(await pedir(`/api/sesiones/${sessionId}/avance${bid}`, {}, token));
107     } catch (_) {}
108   }
109   useEffect(() => {
110     refrescar();
111     // si quedaron registros sin sincronizar de una sesión anterior (se
112     // cerró la pestaña, o se recargó ya con señal) y ahora sí hay
113     // conexión, se reintentará al entrar en vez de esperar a que el
114     // navegador dispare otro evento "online" que quizás no vuelva a pasar.
115     if (navigator.onLine) sincronizar();
116     // eslint-disable-next-line react-hooks/exhaustive-deps
117   }, [bodega]);
118
119   useEffect(() => {
120     pedir("/api/usuarios/yo/bodegas", {}, token).then((r) => {
121       setAsignadas(r);
122       // sin bodegas propias (el caso típico del administrador, que audita
123       // en vez de contar rutinariamente) se necesita el listado completo
124       // de una vez - si no, la única opción es escribir el nombre a mano.
125       if (r.length === 0) cargarTodas();

```

```

126     }).catch(() => setAsignadas([]));
127   }, [token]);
128
129   function cargarTodas() {
130     pedir("/api/bodegas", {}, token).then(setTodas).catch(() => setTodas([]));
131   }
132
133   useEffect(() => {
134     const alVolver = () => { setOffline(false); sincronizar(); };
135     const alPerder = () => setOffline(true);
136     window.addEventListener("online", alVolver);
137     window.addEventListener("offline", alPerder);
138     return () => {
139       window.removeEventListener("online", alVolver);
140       window.removeEventListener("offline", alPerder);
141     };
142     // eslint-disable-next-line react-hooks/exhaustive-deps
143   }, [sesionId, bodega]);
144
145   async function sincronizar() {
146     const restantes = leerCola(sesionId);
147     if (!restantes.length || !bodega) return;
148     // solo se quita de la cola lo que de verdad se confirmó: si un item
149     // falla (o vuelve a caerse la red a mitad de la sincronización), el
150     // resto queda guardado para el próximo intento en vez de perderse -
151     // antes, cualquier falla borraba TODA la cola sin distinguir qué
152     // alcanzó a guardarse.
153     while (restantes.length) {
154       const item = restantes[0];
155       try {
156         await enviarTurno(`${item.articulo} ${item.cantidad} ${item.unidad}`, sesionId, token);
157         await enviarTurno("confirmando", sesionId, token);
158       } catch (_) {
159         break;
160       }
161       restantes.shift();
162       guardarCola(sesionId, restantes);
163       setCola([...restantes]);
164     }
165     refrescar();
166   }
167
168   async function abrir(nombre = textoBodega) {
169     setErr("");
170     setBodegaNoEncontrada(null);
171     setOpcionesBodega(null);
172     setBodegaParaConfirmar(null);
173     try {
174       const r = await abrirBodega(nombre, token);
175       setBodega(r);
176       // De aquí en adelante la conversación (dictar, confirmar, corregir,
177       // el avance) debe usar el id real de esta sesión de conteo, no el
178       // marcador de posición con el que se llegó a esta pantalla - sin
179       // esto, dos personas abriendo bodegas distintas casi al tiempo
180       // terminaban compartiendo la misma conversación y el conteo de una
181       // se guardaba contra la bodega de la otra.
182       alAbrirBodega?.(r.sesion_id);
183       const saludo = `Listo, ${r.bodega.toLowerCase()} abierta con ${r.referencias} referencias. Dícteme el primer producto.`;
184       setRespuesta(saludo);
185       hablar(saludo);
186       refrescar();
187     } catch (e) {
188       // todo lo que el agente muestra en pantalla también lo dice en voz -
189       // esto se quedaba solo en rojo (ej. "esta bodega ya está en conteo
190       // por otra persona"), así que alguien confirmando de viva voz sin
191       // mirar la pantalla se quedaba sin saber qué había pasado.
192       setErr(e.message);
193       hablar(e.message);
194       // igual que un artículo que no esta en el catalogo (ver
195       // FormularioCrearProducto): en vez de dejar a la persona sin salida,
196       // se ofrece pedir la bodega nueva - queda pendiente de aprobacion.
197       if (e.message === "No encuentro esa bodega." && nombre.trim()) {
198         setBodegaNoEncontrada(nombre.trim());
199       }
200     }
201   }
202
203   // Al llegar desde otra pantalla con una bodega ya resuelta (el "Ir a
204   // Conteo"/"Seguir contando" de Bodegas), se abre de una vez en lugar de
205   // solo dejarla escrita esperando un clic más: a diferencia de lo
206   // dictado por voz, este nombre ya salió de una búsqueda exacta contra
207   // el catálogo, no hay nada que confirmar.
208   useEffect(() => {
209     if (bodegaSugerida) abrir(bodegaSugerida);
210     // eslint-disable-next-line react-hooks/exhaustive-deps
211   }, []);

```

```

212
213 /** El botón "Abrir por nombre exacto" (y Enter en el campo) escribía
214     directo a abrir(), que con un nombre ambiguo ("restaurante" a
215     secas, cuando hay cinco "RESTAURANTE ...") solo sabía decir "no la
216     encuentro" y ofrecer crear una bodega duplicada - nunca mostraba
217     las que sí existían. Esto resuelve primero, igual que la voz. */
218 async function abrirPorBoton() {
219     if (!textoBodega.trim()) return;
220     setErr("");
221     setBodegaNoEncontrada(null);
222     setOpcionesBodega(null);
223     setBodegaParaConfirmar(null);
224     let resuelto;
225     try {
226         resuelto = await buscarBodega(textoBodega, token);
227     } catch (e) {
228         setErr(e.message);
229         hablar(e.message);
230         return;
231     }
232     if (resuelto.encontrada) {
233         abrir(resuelto.bodega);
234         return;
235     }
236     if (resuelto.opciones?.length) {
237         setOpcionesBodega(resuelto.opciones);
238         return;
239     }
240     setBodegaNoEncontrada(textoBodega.trim());
241 }
242
243 async function solicitarBodegaNueva() {
244     if (!bodegaNoEncontrada) return;
245     setErr("");
246     try {
247         const r = await pedir("/api/bodegas/crear-pendiente", {
248             method: "POST", body: JSON.stringify({ nombre: bodegaNoEncontrada }),
249             token;
250         });
251         setBodegaNoEncontrada(null);
252         setMsgBodega(r.respuesta_hablada || "Solicitud enviada.");
253         hablar(r.respuesta_hablada);
254     } catch (e) { setErr(e.message); hablar(e.message); }
255 }
256
257 /** Un turno de conversación: el ciclo completo del agente.
258     «mostrar» es lo que se ve como «Usted dijo»; «texto» es lo que de
259     verdad se le manda al backend. Al tocar una tarjeta de opción se
260     manda el código exacto (nunca falla), no el nombre: si un nombre
261     está contenido dentro del otro (ARROZ dentro de ARROZ BASMATI), el
262     agente no tiene con qué distinguirlos por palabra. */
263 async function procesar(texto, mostrar = texto) {
264     if (!texto) return;
265     setDicho(mostrar);
266     setErr("");
267     try {
268         const t = await enviarTurno(texto, sesionId, token, {
269             opciones, opcionesPara,
270             bodegaId: bodega?.bodega_id, bodegaNombre: bodega?.bodega,
271         });
272         setRespuesta(t.respuesta_hablada || "");
273         setOpciones(t.opciones || null);
274         setOpcionesPara(t.opciones_para || null);
275         setAlerta(t.alerta || null);
276         setAlertaEsperado(t.alerta === "desviacion" ? (t.pendiente?.cantidad ?? null) : null);
277         setContextoAlerta(t.contexto_alerta || null);
278         setPendiente(t.pendiente || null);
279         setArchivo(t.archivo || null);
280         hablar(t.respuesta_hablada);
281         if (t.guardado || t.corregido || t.bodega) refrescar();
282         if (t.bodega) {
283             setBodega({ bodega: t.bodega.nombre, referencias: t.bodega.referencias, bodega_id: t.bodega.id });
284             // igual que al abrir por el botón: de aquí en adelante hay que
285             // hablar con el id real de esta sesión de conteo, no con el
286             // marcador de posición con el que se pidió abrir por voz.
287             if (t.sesion_id) alAbrirBodega?.(t.sesion_id);
288         }
289         if (t.intencion === "crear") {
290             setCrear({ nombre: t.articulo_texto || texto, unidad_medida: "Unidad",
291                 cantidad_inicial: t.cantidad || 0 });
292         } else {
293             setCrear(null);
294         }
295     } catch (e) {
296         if (esFalloRed(e)) {
297             setOffline(true);
298             setErr("");
299         }
300     }

```

```

298     } else {
299         setErr(e.message);
300         hablar(e.message);
301     }
302 } finally {
303     setEstado("listo");
304 }
305 }
306
307 function recontar() {
308     setAlerta(null);
309     setAlertaEsperado(null);
310     setContextoAlerta(null);
311     setPendiente(null);
312     setDicho("");
313     const msg = "¿La contamos otra vez? Dígame el nuevo valor.";
314     setRespuesta(msg);
315     hablar(msg);
316 }
317
318 async function crearPendiente() {
319     setErr("");
320     try {
321         const r = await pedir("/api/conteo/crear-producto", {
322             method: "POST",
323             body: JSON.stringify({ ...crear, sesion_id: sesionId }),
324             token;
325         });
326         setRespuesta(r.respuesta_hablada);
327         hablar(r.respuesta_hablada);
328         setCrear(null);
329         setAlerta(null);
330         refrescar();
331     } catch (e) {
332         setErr(e.message);
333         hablar(e.message);
334     }
335 }
336
337 function guardarEnCola(item) {
338     const nueva = [...leerCola(sesionId), item];
339     guardarCola(sesionId, nueva);
340     setCola(nueva);
341 }
342
343 function alMicrofono() {
344     if (estado === "escuchando") {
345         rec.current?.stop();
346         return;
347     }
348     rec.current = escuchar({
349         alTexto: procesar,
350         alEstado: (e, parcial) => {
351             setEstado(e);
352             if (parcial) setDicho(parcial);
353         },
354         alError: setErr,
355     });
356 }
357
358 /** Para elegir bodega (a diferencia del conteo, que confirma hablado
359 antes de guardar): esto llena el buscador con lo que se reconoció y
360 pregunta en voz alta "¿confirma X?" - un "sí" abre directo, un "no"
361 descarta sin tocar nada. Nombres poco comunes ("Piscilago") son
362 justo donde más se equivoca el reconocimiento de voz del navegador;
363 "Abrir por nombre exacto" sigue ahí como respaldo si prefiere
364 escribir o corregir a mano en vez de confirmar hablado. */
365 function alMicrofonoBodega() {
366     if (estado === "escuchando") {
367         rec.current?.stop();
368         return;
369     }
370     setErr("");
371     setMsgBodega("");
372     // si todavía se está esperando el sí/no a "¿confirma que abro X?", este
373     // toque del micrófono es la respuesta a ESA pregunta, no un nombre de
374     // bodega nuevo - ver el comentario junto a bodegaParaConfirmar.
375     if (bodegaParaConfirmar) {
376         const miIdConfirmar = ++idEscuchaBodega.current;
377         rec.current = escuchar({
378             alTexto: (respuesta) => {
379                 if (miIdConfirmar !== idEscuchaBodega.current) return;
380                 responderConfirmacionBodega(respuesta);
381             },
382             alEstado: (e) => setEstado(e),
383             alError: setErr,
384         });
385     }
386 }

```

```

384     return;
385   }
386   setOpcionesBodega(null);
387   const miId = ++idEscuchaBodega.current;
388   rec.current = escuchar({
389     alTexto: (t) => {
390       if (miId !== idEscuchaBodega.current) return;
391       setTextoBodega(t);
392       preguntarConfirmacionBodega(t, miId);
393     },
394     alEstado: (e, parcial) => {
395       setEstado(e);
396       if (parcial) setTextoBodega(parcial);
397     },
398     alError: setErr,
399   });
400 }
401
402 /** La respuesta al "¿confirma que abro X?", venga del listener que se
403     abrió justo después de la pregunta, o de un toque posterior al
404     micrófono si la persona se demoró y ese listener ya se había
405     apagado solo (ver bodegaParaConfirmar). */
406 function responderConfirmacionBodega(respuesta) {
407   const nombreReal = bodegaParaConfirmar;
408   setBodegaParaConfirmar(null);
409   if (!nombreReal) return;
410   if (esAfirmacion(respuesta)) {
411     abrir(nombreReal);
412   } else if (esNegacion(respuesta)) {
413     setTextoBodega("");
414     const msg = "Gracias, queda pendiente. Cuando quiera abrir una bodega, dígame el nombre.";
415     setMsgBodega(msg);
416     hablar(msg);
417   } else {
418     const msg = "No le entendí un «sí» o un «no». Use «Abrir por nombre exacto», o dígame de nuevo.";
419     setErr(msg);
420     hablar(msg);
421     // sigue pendiente: no se descarta la confirmación por una respuesta
422     // que no se entendió, para no perder el hilo justo ahora.
423     setBodegaParaConfirmar(nombreReal);
424   }
425 }
426
427 /** Antes esto preguntaba "¿confirma X?" con lo que se acababa de
428     transcribir tal cual - decir "kiosco" confirmaba "kiosco" como si
429     fuera una bodega real, cuando lo que existe es "KIOSCO TAQUILLA
430     AYB". Ahora primero RESUELVE el nombre contra el catálogo
431     (/api/bodegas/buscar, sin abrir nada todavía) y confirma con el
432     nombre real que de verdad se va a abrir. Si lo dicho es ambiguo
433     ("restaurante" a secas encaja en dos bodegas reales por igual), se
434     ofrecen las opciones en vez de fallar en seco; si de verdad no
435     encuentra nada, ofrece crearla.
436
437     "miId" identifica ESTA escucha en particular (viene de
438     idEscuchaBodega, ver el comentario junto a esa ref): antes de tocar
439     cualquier estado se revisa que siga siendo la vigente, porque el
440     micrófono anterior a veces entrega su resultado tarde - sin este
441     control, ese resultado tardío se procesaba como una bodega nueva
442     dictada (por ejemplo, el "confirmo" de la pregunta de abajo llegaba
443     aquí como si alguien hubiera dicho "confirmo" como nombre de
444     bodega). */
445 async function preguntarConfirmacionBodega(nombreDictado, miId) {
446   if (!nombreDictado || !nombreDictado.trim()) return;
447   if (miId === undefined) miId = ++idEscuchaBodega.current;
448   setOpcionesBodega(null);
449   let resuelto;
450   try {
451     resuelto = await buscarBodega(nombreDictado, token);
452   } catch (e) {
453     if (miId !== idEscuchaBodega.current) return;
454     setErr(e.message);
455     hablar(e.message);
456     return;
457   }
458   if (miId !== idEscuchaBodega.current) return;
459   if (!resuelto.encontrada) {
460     if (resuelto.opciones?.length) {
461       const nombres = resuelto.opciones.map((o) => o.bodega.toLowerCase());
462       setOpcionesBodega(resuelto.opciones);
463       // reservar el id ANTES de hablar corta de una vez cualquier
464       // escucha anterior que responda tarde mientras se dice esto.
465       const miIdOpciones = ++idEscuchaBodega.current;
466       // hay que esperar a que la pregunta se termine de decir - si el
467       // micrófono arranca apenas empieza a sonar, se pierde el
468       // principio de la respuesta (o, si la voz tarda, el micrófono
469       // hasta se cansa de esperar antes de que la persona alcance a

```

```

470 // contestar).
471 await hablar(`Hay varias: ${nombres.join(", ")}. ¿Cuál de todas?`);
472 if (miIdOpciones !== idEscuchaBodega.current) return;
473 // se puede elegir tocando una tarjeta, o siguiendo la conversación
474 // y diciendo el nombre completo - que vuelve a pasar por aquí,
475 // ahora sí resolviendo a una sola.
476 rec.current = escuchar({
477   alTexto: (t) => {
478     if (miIdOpciones !== idEscuchaBodega.current) return;
479     setBodega(t);
480     preguntarConfirmacionBodega(t, miIdOpciones);
481   },
482   alEstado: (e) => setEstado(e),
483   alError: setErr,
484 });
485 return;
486 }
487 hablar(`No encuentro ${nombreDictado.toLowerCase()}. Puede solicitarla como bodega nueva.`);
488 setBodegaNoEncontrada(nombreDictado.trim());
489 return;
490 }
491 const nombreReal = resuelto.bodega;
492 setBodega(nombreReal);
493 setBodegaParaConfirmar(nombreReal);
494 const miIdConfirmar = ++idEscuchaBodega.current;
495 await hablar(`¿Confirma que abro ${nombreReal.toLowerCase()}?`);
496 if (miIdConfirmar !== idEscuchaBodega.current) return;
497 rec.current = escuchar({
498   alTexto: (respuesta) => {
499     if (miIdConfirmar !== idEscuchaBodega.current) return;
500     responderConfirmacionBodega(respuesta);
501   },
502   alEstado: (e) => setEstado(e),
503   alError: setErr,
504 });
505 }
506
507 return (
508   <Marco
509     titulo={`Conteo por voz${bodega ? " " + bodega.bodega : ""}`}
510     chip={offline ? { texto: "SIN CONEXIÓN", tipo: "oro" }
511       : bodega ? { texto: "EN CONTEO", tipo: "azul" }
512       : { texto: "SIN BODEGA", tipo: "gris" }}
513   >
514     {!bodega ? (
515       <div className="card">
516         <h3>¿Qué bodega va a contar?</h3>
517         <div style={{ display: "flex", gap: 10, alignItems: "center", marginBottom: 6 }}>
518           <button
519             className={`mic-btn ${estado === "escuchando" ? "escuchando" : ""}`}
520             style={{ width: 46, height: 46, fontSize: "1.2rem" }}
521             onClick={alMicrofonoBodega} title="diga el nombre de la bodega"
522             aria-label="Decir el nombre de la bodega por voz">
523             <Icono nombre="microfono" tam={22} />
524           </button>
525           <small style={{ color: "var(--grafito)" }}>
526             {estado === "escuchando" ? "Escuchando..."
527               : "o diga el nombre de la bodega en voz alta"}
528           </small>
529         </div>
530         <p className="pista" style={{ marginBottom: 8 }}>
531           Lo que reconozca aparece abajo, se lo pregunta de vuelta y usted
532           confirma diciendo «sí» (nombres como «Piscilago» a veces se
533           transcriben mal) — o corríjalo a mano y use «Abrir por nombre
534           exacto».
535         </p>
536         <details style={{ marginBottom: 8 }}>
537           <summary className="pista" style={{ cursor: "pointer" }}>
538             Ver frases exactas que entiende el agente aquí
539           </summary>
540           <ul style={{ margin: "8px 0 0", paddingLeft: 18 }}>
541             {["almacén general", "kiosco taquilla ayb", "sí (confirma la bodega sugerida)",
542               "no (descarta la sugerencia)"].map((ej, i) => (
543               <li key={i} className="pista" style={{ marginBottom: 4 }}>«ej»</li>
544             ))}
545           </ul>
546         </details>
547         <div style={{ display: "flex", gap: 10, flexWrap: "wrap", marginBottom: 14 }}>
548           <input
549             style={{ flex: 1, minWidth: 240, padding: "12px 14px",
550               border: "1px solid var(--borde)", borderRadius: 12 }}
551             value={textoBodega}
552             onChange={(e) => setBodega(e.target.value)}
553             onKeyDown={(e) => e.key === "Enter" && abrirPorBoton()}
554             placeholder="nombre de la bodega..."
555             aria-label="Nombre de la bodega"

```

```

556     />
557     <button className="btn" onClick={abrirPorBoton}
558         style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
559         <Icono nombre="aprobar" tam={18} />
560         Abrir por nombre exacto
561     </button>
562 </div>
563 {err && <p className="error" style={{ marginBottom: 10 }}>{err}</p>}
564 {msgBodega && <p className="msg-ok" style={{ marginBottom: 10 }}>{msgBodega}</p>}
565 {bodegaNoEncontrada && (
566     <div className="banner" style={{ marginBottom: 10 }}>
567     <span className="ico">!</span>
568     <span>
569     <b>¿Crear «{bodegaNoEncontrada}></b>
570     <span>No existe en el catálogo. Queda pendiente de aprobación del
571     administrador; el conteo no se detiene por esto.</span>
572     </span>
573     <button className="btn" style={{ flex: "none" }}
574     onClick={solicitarBodegaNueva}>
575     Solicitar bodega nueva
576     </button>
577 </div>
578 )}
579 {opcionesBodega && (
580     <div style={{ marginBottom: 14 }}>
581     <p className="pista" style={{ marginBottom: 8 }}>
582     Hay varias que encajan con lo dicho – toque una, o diga el
583     nombre completo:
584     </p>
585     <div className="opciones">
586     {opcionesBodega.map((o, i) => (
587         <button key={o.bodega_id} className="opcion"
588             onClick={() => { setOpcionesBodega(null); abrir(o.bodega); }}>
589         <span className="n">Opción {i + 1}</span>
590         <h4>{o.bodega}</h4>
591         </button>
592     ))}
593     </div>
594 </div>
595 )}
596 {asignadas === null ? (
597     <p className="cargando">Cargando sus bodegas asignadas...</p>
598 ) : asignadas.length > 0 && !buscarOtra ? (
599     <>
600     <div className="grid-bodegas">
601     {asignadas.map((b) => {
602         const [txt, cls] = ETIQUETA_BODEGA[b.estado] || [""], "gris";
603         const esOtraPersona = b.estado === "en_conteo" && b.persona
604         && b.persona.toLowerCase() !== (usuario?.nombre || "").toLowerCase();
605         let detalleLinea;
606         if (b.estado === "en_conteo" && b.persona) {
607             detalleLinea = esOtraPersona
608             ? `${b.persona} ya la está contando · ${b.avance_pct ?? 0}%`
609             : `usted la dejó a medias · ${b.avance_pct ?? 0}%`;
610         } else if (b.estado === "en_auditoria" && b.persona) {
611             detalleLinea = `${b.persona} auditando · ${b.diferencias ?? 0} dif.`;
612         } else if (b.estado === "cerrada" && b.hora_cierre) {
613             detalleLinea = `cerrada ${b.hora_cierre}`;
614         } else if (b.estado === "pendiente") {
615             detalleLinea = "sin iniciar · disponible";
616         } else {
617             detalleLinea = `${b.referencias} referencias`;
618         }
619         return (
620             <button key={b.id} className={`tarjeta-bodega ${b.estado}`}
621                 onClick={() => abrir(b.bodega)}
622                 title={esOtraPersona
623                     ? "Ya la está contando otra persona; puede intentar de todas formas."
624                     : ""}>
625             <b>{b.bodega}</b>
626             <span className={`chip ${cls} est`} >{txt}</span>
627             <small>{detalleLinea}</small>
628             </button>
629         );
630     })}
631     </div>
632     <div className="grilla-botones" style={{ marginTop: 14 }}>
633     <button className="btn borde"
634         onClick={() => { setBuscarOtra(true); if (!todas) cargarTodas(); }}>
635     Buscar otra bodega
636     </button>
637 </div>
638 </>
639 ) : (
640     <>
641     {asignadas.length > 0 && (

```

```

642     <p className="pista" style={{ marginBottom: 10 }}>
643         Buscando fuera de sus bodegas asignadas.
644     </p>
645 })
646 {todas === null ? (
647     <p className="cargando" style={{ marginTop: 10 }}>Cargando bodegas...</p>
648 ) : (
649     <div className="grid-bodegas" style={{ marginTop: 14 }}>
650         {(textoBodega.trim()
651             ? todas.filter((b) => b.bodega.toLowerCase()
652                 .includes(textoBodega.trim().toLowerCase()))
653             : todas
654         ).slice(0, 24).map((b) => {
655             const [txt, cls] = ETIQUETA_BODEGA[b.estado] || ["", "gris"];
656             return (
657                 <button key={b.id} className={`tarjeta-bodega ${b.estado}`}
658                     onClick={() => abrir(b.bodega)}>
659                     <b>{b.bodega}</b>
660                     <span className={`chip ${cls} est`} >{txt}</span>
661                     <small>{b.referencias}</small>
662                 </button>
663             );
664         })}
665     </div>
666 )}
667 {todas && !textoBodega.trim() && todas.length > 24 && (
668     <p className="pista" style={{ marginTop: 8 }}>
669         Mostrando 24 de {todas.length} – escriba para filtrar.
670     </p>
671 )}
672
673 <div className="grilla-botones" style={{ marginTop: 10 }}>
674     {asignadas.length > 0 && (
675         <button className="btn gris" onClick={() => setBuscarOtra(false)}>
676             Volver a sus bodegas asignadas
677         </button>
678     )}
679 </div>
680 </>
681 )}
682 </div>
683 ) : offline ? (
684     <FormularioOffline
685         cola={cola}
686         onGuardar={guardarEnCola}
687         onReintentar={() => { setOffline(!navigator.onLine); if (navigator.onLine) sincronizar(); }}
688     />
689 ) : (
690     <>
691         <div className="chips">
692             <span className="chip">Avance: {avance.hechas} de {avance.total}</span>
693             <span className={`chip ${avance.alertas ? "oro" : "verde"}`}>
694                 Alertas: {avance.alertas}
695             </span>
696             <span className="chip gris">Sesión bloqueada</span>
697             {cola.length > 0 && (
698                 <span className="chip oro">{cola.length} por sincronizar</span>
699             )}
700         </div>
701
702         {alerta && alerta !== "desviacion" && !crear && (
703             <div className="banner">
704                 <span className="ico">!</span>
705                 <span style={{ flex: 1 }}>
706                     <b>
707                         {alerta === "negativo" ? "Cantidad imposible"
708                         : alerta === "unidad" ? "Unidad de medida distinta"
709                         : "Artículo fuera del catálogo"}
710                     </b>
711                     <span>{respuesta}</span>
712                 </span>
713             </div>
714         )}
715
716         {alerta === "desviacion" && !crear && (
717             <AlertaDesviacion
718                 contexto={contextoAlerta} respuesta={respuesta}
719                 onRecontar={recontar}
720                 onConfirmar={() => procesar("confirmo")}
721             />
722         )}
723
724         {crear && (
725             <FormularioCrearProducto
726                 crear={crear} setCrear={setCrear}
727                 onCrear={crearPendiente} onCancelar={() => setCrear(null)}

```

```

728     bodega={bodega.bodega} err={err}
729   />
730 })
731
732 {!crear && alerta !== "desviacion" && (
733   <div className="conteo-cols">
734     <div className="card">
735       <p className="rotulo">
736         {dicho ? `Usted dijo: «${dicho}»` : "CuentaVoz responde"}
737       </p>
738       <div className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</div>
739       {archivo && (
740         <div className="grilla-botones" style={{ marginTop: 10 }}>
741           <button className="btn verde"
742             onClick={() => descargarReporte(archivo, token).catch((e) => setErr(e.message))}>
743             Descargar archivo generado
744           </button>
745         </div>
746       )}
747
748       {opciones && (
749         <div className="opciones" style={{ marginTop: 14 }}>
750           {opciones.map((o, i) => (
751             <button key={o.codigo} className="opcion"
752               onClick={() => procesar(o.codigo, o.nombre)}>
753               <span className="n">Opción {i + 1}</span>
754               <h4>{o.nombre}</h4>
755               <p>Código {o.codigo}</p>
756               <p>{o.unidad}</p>
757             </button>
758           ))}
759         </div>
760       )}
761
762       {pendiente && !opciones && (
763         <p className="pista" style={{ marginTop: 12 }}>
764           Por confirmar: <b>{pendiente.nombre}</b> – {pendiente.cantidad}{ " " }
765           {pendiente.unidad}
766         </p>
767       )}
768
769       <h3 style={{ marginTop: 18 }}>Últimos registros confirmados</h3>
770       {avance.ultimos?.length ? (
771         avance.ultimos.map((u, i) => (
772           <div className="registro" key={i}>
773             <span className="ok">✓</span>
774             <span>{u.nombre}</span>
775             <span className="cant">{u.cantidad} {u.unidad}</span>
776           </div>
777         ))
778       ) : (
779         <p className="vacio">Todavía no hay registros en esta sesión.</p>
780       )}
781     </div>
782
783     <div className="card mic-caja">
784       <button
785         className={`mic-btn ${estado === "escuchando" ? "escuchando" : ""}`}
786         onClick={alMicrofono}
787         aria-label="Toque y hable para dictar el artículo y la cantidad contada"
788       >
789         <Icono nombre="microfono" tam={52} />
790       </button>
791       <b>
792         {estado === "escuchando" ? "Escuchando..."
793           : estado === "procesando" ? "Procesando..."
794           : "Toque y hable"}
795       </b>
796       <small>
797         {vozDisponible()
798           ? "Reconocimiento en español (Colombia)"
799           : "Este navegador no reconoce voz: use el teclado"}
800       </small>
801       <details style={{ marginTop: 8, textAlign: "left" }}>
802         <summary className="pista" style={{ cursor: "pointer" }}>
803           Ver frases exactas que entiende el agente aquí
804         </summary>
805         <ul style={{ margin: "8px 0 0", paddingLeft: 18 }}>
806           [{"tres tablas para picar blancas", "hay noventa cazuelas",
807             "me faltan cinco kilos de arroz", "corregir", "son nueve",
808             "confirmo", "{cuánto arroz hay contado?}"].map((ej, i) => (
809             <li key={i} className="pista" style={{ marginBottom: 4 }}>«{ej}»</li>
810           ))}
811         </ul>
812       </details>
813       {err && <p className="error">{err}</p>}

```

```

814         </div>
815     </div>
816 })
817
818 {!crear && alerta !== "desviacion" && (
819     <>
820         <div className="grilla-botones">
821             <button className="btn verde" onClick={() => procesar("confirmo")}>
822                 Confirmar
823             </button>
824             <button className="btn borde" onClick={() => procesar("corregir")}>
825                 Corregir
826             </button>
827             <button className="btn gris" onClick={() => setMostrarTeclado(true)}>
828                 Teclado
829             </button>
830             <button className="btn oro" onClick={() => ir("ayuda")}>
831                 Pedir ayuda
832             </button>
833         </div>
834         <p className="pista" style={{ marginTop: 10 }}>
835             La voz es el camino rápido; el teclado siempre queda como respaldo.
836         </p>
837     </>
838 })
839 </>
840 })
841
842 {mostrarTeclado && (
843     <Dialogo titulo="Escribir en vez de hablar"
844         mensaje="Escriba lo que diría en voz alta:"
845         conCampo placeholder="tres tablas para picar blancas"
846         onAceptar={(t) => { setMostrarTeclado(false); if (t) procesar(t); }}
847         onCancel={() => setMostrarTeclado(false)} />
848 ))
849 </Marco>
850 );
851 }
852
853 const fmt = (n) => Number(n).toLocaleString("es-CO", { maximumFractionDigits: 2 });
854
855 function AlertaDesviacion({ contexto, respuesta, onRecontar, onConfirmar }) {
856     return (
857         <>
858             <div className="banner">
859                 <span className="ico">!</span>
860                 <span>
861                     <b>Cantidad fuera de lo esperado</b>
862                     <span>
863                         {contexto
864                             ? `El sistema espera alrededor de ${fmt(contexto.esperado)} · usted dijo `
865                             + `${fmt(contexto.dicho)} (${contexto.articulo)}`
866                             : respuesta}
867                     </span>
868                 </span>
869             </div>
870
871             <div className="card">
872                 <p className="rotulo">CuentaVoz responde</p>
873                 <div className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</div>
874             </div>
875
876             <div className="conteo-cols">
877                 {contexto ? (
878                     <div className="card">
879                         <h3>Contexto de la validación</h3>
880                         <table>
881                             <tbody>
882                                 <tr><td>Stock del sistema</td><td><b style={{ color: "var(--azul)" }}>
883                                     {fmt(contexto.esperado)} {contexto.unidad}</b></td></tr>
884                                 <tr><td>Su conteo</td><td><b style={{ color: "var(--azul)" }}>
885                                     {fmt(contexto.dicho)} {contexto.unidad}</b></td></tr>
886                                 <tr><td>Desviación</td><td className="dif">
887                                     {contexto.desviacion_pct !== null
888                                     ? `${contexto.desviacion_pct > 0 ? "+" : ""}${contexto.desviacion_pct}%`
889                                     : "-"}</td></tr>
890                             </tbody>
891                         </table>
892                     </div>
893                 ) : <div />}
894             <div style={{ display: "flex", flexDirection: "column", gap: 12 }}>
895                 <button className="btn" onClick={onRecontar}>RECONTAR</button>
896                 <button className="btn oro" onClick={onConfirmar}>
897                     CONFIRMAR {contexto ? fmt(contexto.dicho) : ""}
898                 </button>
899             </div>

```

```

900     </div>
901
902     <p className="pista">
903       Si confirma, el registro se guarda marcado para revisión del administrador de bodega.
904     </p>
905     <p className="pista" style={{ color: "var(--azul)" }}>
906       Ninguna cantidad con alerta se guarda sin una confirmación explícita de la persona.
907     </p>
908   </>
909 );
910 }
911
912 function FormularioCrearProducto({ crear, setCrear, onCreate, onCancel, bodega, err }) {
913   const [estado, setEstado] = useState("listo");
914   // El micrófono principal de Conteo se oculta mientras se decide si
915   // crear este producto (para no mezclar esa respuesta con un conteo
916   // nuevo) - sin esto, la pregunta hablada "¿lo creamos?" solo se podía
917   // contestar con clic, aunque el resto de la pantalla es por voz.
918   function alMicrofono() {
919     escuchar({
920       alTexto: (texto) => {
921         setEstado("listo");
922         if (esAfirmacion(texto, ["crea", "crear"])) { onCreate(); return; }
923         if (esNegacion(texto)) { onCancel(); return; }
924       },
925       alEstado: setEstado,
926       alError: () => {},
927     });
928   }
929   return (
930     <div className="card">
931       <div style={{ display: "flex", justifyContent: "space-between", alignItems: "center", gap: 10 }}>
932         <p className="rotulo" style={{ margin: 0 }}>CuentaVoz responde</p>
933         <button className={`mic-btn ${estado === "escuchando" ? "escuchando" : ""}`}
934           style={{ width: 46, height: 46, fontSize: "1.2rem", flex: "none" }}
935           onClick={alMicrofono} title="diga «sí» para crear, o «no» para cancelar"
936           aria-label="Responder por voz: diga sí para crear, o no para cancelar">
937           <Icono nombre="microfono" tam={20} />
938         </button>
939       </div>
940       <div className="burbuja" aria-live="polite" aria-atomic="true">
941         No encontré «{crear.nombre}» en el catálogo. Lo creamos y queda pendiente
942         del administrador.
943       </div>
944       <p className="pista" style={{ marginTop: 8 }}>o diga «sí» para crearlo, o «no» para cancelar</p>
945
946       <div className="conteo-cols" style={{ marginTop: 16 }}>
947         <div>
948           <label className="pista" htmlFor="campo-nombre-articulo">Nombre tal como lo dictó</label>
949           <input id="campo-nombre-articulo" value={crear.nombre}
950             onChange={(e) => setCrear({ ...crear, nombre: e.target.value })}
951             style={{ width: "100%", padding: "11px 13px", marginTop: 6,
952               border: "1px solid var(--borde)", borderRadius: 12,
953               textTransform: "uppercase" }} />
954         </div>
955         <div>
956           <label className="pista">Estado del registro</label>
957           <div style={{ marginTop: 6 }}>
958             <span className="chip oro">PENDIENTE DE APROBACIÓN</span>
959           </div>
960         </div>
961       </div>
962
963       <div className="dos-columnas" style={{ marginTop: 16 }}>
964         <div>
965           <label className="pista" id="etiqueta-unidad-medida">Unidad de medida</label>
966           <div className="grilla-botones" style={{ marginTop: 6 }} role="group" aria-labelledby="etiqueta-unidad-medida">
967             {UNIDADES.map((u) => (
968               <button key={u}
969                 className={`btn ${crear.unidad_medida === u ? "" : "borde"} `}
970                 onClick={() => setCrear({ ...crear, unidad_medida: u })}
971                 aria-pressed={crear.unidad_medida === u}>
972               {u}
973             </button>
974             ))}
975           </div>
976         </div>
977         <div>
978           <label className="pista" htmlFor="campo-cantidad-inicial">Cantidad inicial contada en {bodega}</label>
979           <input id="campo-cantidad-inicial" type="number" min="0" value={crear.cantidad_inicial}
980             onChange={(e) => setCrear({ ...crear, cantidad_inicial: Number(e.target.value) })}
981             style={{ width: "100%", padding: "11px 13px", marginTop: 6,
982               border: "1px solid var(--borde)", borderRadius: 12 }} />
983         </div>
984       </div>
985     </div>

```

```

986     <p className="pista" style={{ marginTop: 12 }}>
987       No entra al catálogo oficial hasta la firma del administrador de bodega.
988     </p>
989     {err && <p className="error" style={{ marginTop: 6 }}>{err}</p>}
990     <div className="grilla-botones">
991       <button className="btn" onClick={onCrear}>Crear pendiente</button>
992       <button className="btn gris" onClick={onCancelar}>Cancelar</button>
993     </div>
994     <p className="pista" style={{ marginTop: 10 }}>
995       El conteo continúa sin interrupción: la aprobación ocurre en paralelo.
996     </p>
997   </div>
998   );
999 }
1000
1001 /** Mismo ciclo de confirmación del agente real (dice el artículo y la
1002     cantidad, CuentaVoz repite y pregunta, la persona confirma) pero
1003     corriendo enteramente en el navegador con interpretelocal.js - sin
1004     tocar el backend ni el internet para nada. El reconocimiento de voz
1005     del navegador sí necesita señal (no es algo que esta app controle),
1006     así que aquí se escribe en vez de dictar; el resto de la conversación
1007     - entender la frase, sugerir el nombre oficial, pedir confirmación -
1008     sigue funcionando igual. */
1009 function FormularioOffline({ cola, onGuardar, onReintentar }) {
1010   const [catalogo] = useState(() => leerCatalogo());
1011   const [texto, setTexto] = useState("");
1012   const [dicho, setDicho] = useState("");
1013   const [respuesta, setRespuesta] = useState(
1014     "Sin conexión, pero sigo entendiendo lo que escriba. Dígame el artículo y la cantidad."
1015   );
1016   const [pendiente, setPendiente] = useState(null); // {articulo, cantidad, unidad, codigo}
1017   const [mostrarManual, setMostrarManual] = useState(false);
1018
1019   // el formulario campo por campo de siempre, como respaldo del respaldo
1020   // si una frase no se deja interpretar bien.
1021   const [articulo, setArticulo] = useState("");
1022   const [cantidad, setCantidad] = useState(1);
1023   const [unidad, setUnidad] = useState("Unidad");
1024   const sugerenciaManual = sugerirDeCatalogo(articulo, catalogo);
1025
1026   function procesarTexto(t) {
1027     if (!t.trim()) return;
1028     setDicho(t);
1029     setTexto("");
1030     const r = interpretarLocal(t);
1031     const simple = t.trim().toLowerCase();
1032     const esSi = /^(sí|sí|confirmo|correcto|dale|listo)\b/.test(simple);
1033     const esNo = /^no\b/.test(simple);
1034
1035     if (pendiente && (esSi || r.intencion === "confirmar")) {
1036       onGuardar({ articulo: pendiente.articulo, cantidad: pendiente.cantidad,
1037         unidad: pendiente.unidad });
1038       setPendiente(null);
1039       setRespuesta("Guardado en este equipo. Se sincroniza solo cuando vuelva la señal.");
1040       return;
1041     }
1042     if (pendiente && esNo) {
1043       setPendiente(null);
1044       setRespuesta("Listo, no lo guardo. Dígame el artículo y la cantidad correctos.");
1045       return;
1046     }
1047     if (pendiente && r.intencion === "corregir" && r.cantidad !== null) {
1048       setPendiente({ ...pendiente, cantidad: r.cantidad });
1049       setRespuesta(`{pendiente.articulo}: ${r.cantidad} ${pendiente.unidad}. ¿Confirma?`);
1050       return;
1051     }
1052     if (r.intencion === "contar" && r.cantidad !== null) {
1053       const sug = sugerirDeCatalogo(r.articulo_texto, catalogo);
1054       const nombreFinal = sug?.nombre || r.articulo_texto || "ese artículo";
1055       const unidadFinal = sug?.unidad || r.unidad || "Unidad";
1056       setPendiente({ articulo: nombreFinal, cantidad: r.cantidad,
1057         unidad: unidadFinal, codigo: sug?.codigo });
1058       setRespuesta(`{nombreFinal}: ${r.cantidad} ${unidadFinal}. ¿Confirma? (escriba «sí») `);
1059       return;
1060     }
1061     setRespuesta(r.respuesta_hablada
1062       || "No le entendí bien. Dígame el artículo y la cantidad, por ejemplo «hay noventa cazuelas.»");
1063   }
1064
1065   function usarSugerenciaManual() {
1066     setArticulo(sugerenciaManual.nombre);
1067     setUnidad(sugerenciaManual.unidad);
1068   }
1069   function ciclarUnidad() {
1070     setUnidad(UNIDADES[(UNIDADES.indexOf(unidad) + 1) % UNIDADES.length]);
1071   }

```

```

1072 function guardarManual() {
1073   if (!articulo.trim()) return;
1074   onGuardar({ articulo: articulo.trim(), cantidad: Number(cantidad), unidad });
1075   setArticulo("");
1076   setCantidad(1);
1077 }
1078
1079 return (
1080   <div className="conteo-cols">
1081     <div className="card">
1082       <div className="banner">
1083         <span className="ico">!/span>
1084         <span>
1085           <b>Sin conexión</b>
1086           <span>El reconocimiento de voz necesita señal, pero CuentaVoz sigue
1087             entendiendo lo que escriba. Lo que registre se guarda en este
1088             equipo y se sincroniza solo al volver la red.</span>
1089         </span>
1090       </div>
1091
1092       <p className="rotulo">{dicho ? `Usted escribió: «${dicho}»` : "CuentaVoz responde"}</p>
1093       <div className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</div>
1094
1095       <div style={{ display: "flex", gap: 10, marginTop: 12, flexWrap: "wrap" }}>
1096         <input value={texto} onChange={(e) => setTexto(e.target.value)}
1097           onKeyDown={(e) => e.key === "Enter" && procesarTexto(texto)}
1098           placeholder="hay noventa cazuelas blancas..."
1099           aria-label="Escriba el artículo y la cantidad contada"
1100           style={{ flex: 1, minWidth: 200, padding: "13px 14px", fontSize: "1rem",
1101             border: "1px solid var(--borde)", borderRadius: 12 }} />
1102         <button className="btn" onClick={() => procesarTexto(texto)}>Enviar</button>
1103       </div>
1104
1105       <div className="grilla-botones" style={{ marginTop: 10 }}>
1106         <button className="btn borde" onClick={onReintentar}>Reintentar la voz</button>
1107         <button className="btn gris" onClick={() => setMostrarManual((m) => !m)}>
1108           {mostrarManual ? "Ocultar formulario manual" : "Registrar campo por campo"}
1109         </button>
1110       </div>
1111
1112       {mostrarManual && (
1113         <div style={{ marginTop: 14, padding: 14, borderTop: "1px solid var(--borde)" }}>
1114           <label className="pista" htmlFor="campo-articulo-manual">Artículo</label>
1115           <input id="campo-articulo-manual" value={articulo} onChange={(e) => setArticulo(e.target.value)}
1116             placeholder="tabla acril..."
1117             style={{ width: "100%", padding: "11px 13px", margin: 8,
1118               border: "1px solid var(--borde)", borderRadius: 12 }} />
1119           {sugerenciaManual && (
1120             <button className="chip azul" onClick={usarSugerenciaManual}
1121               style={{ display: "block", width: "100%", textAlign: "left",
1122                 margin: 6 }}>
1123               {sugerenciaManual.nombre} {sugerenciaManual.codigo}
1124             </button>
1125           )}
1126           <label className="pista">Cantidad</label>
1127           <div style={{ display: "flex", gap: 10, margin: 6, margin: 10,
1128             alignItems: "center" }}>
1129             <button className="btn borde" style={{ minHeight: 0, padding: "8px 16px" }}
1130               onClick={() => setCantidad((c) => Math.max(0, Number(c) - 1))}
1131               aria-label="Restar uno a la cantidad"></button>
1132             <input type="number" value={cantidad}
1133               onChange={(e) => setCantidad(e.target.value)}
1134               aria-label="Cantidad contada"
1135               style={{ width: 90, padding: "11px 13px", textAlign: "center",
1136                 border: "1px solid var(--borde)", borderRadius: 12 }} />
1137             <button className="btn borde" style={{ minHeight: 0, padding: "8px 16px" }}
1138               onClick={() => setCantidad((c) => Number(c) + 1)}
1139               aria-label="Sumar uno a la cantidad"></button>
1140             <button className="chip azul" onClick={ciclarUnidad}
1141               aria-label={`Unidad: ${unidad}. Toque para cambiarla`}>{unidad}</button>
1142           </div>
1143           <button className="btn" onClick={guardarManual}>Guardar</button>
1144         </div>
1145       )}
1146
1147       <p className="pista" style={{ margin: 12 }}>
1148         El catálogo local sigue sugiriendo nombres oficiales sin internet.
1149       </p>
1150     </div>
1151
1152     <div className="card">
1153       <h3>Guardados en este equipo</h3>
1154       {cola.length === 0 ? (
1155         <p className="vacio">Nada por sincronizar todavía.</p>
1156       ) : (
1157         cola.map((c, i) => (

```

```

1158         <div className="registro" key={i}>
1159             <span>{c.articulo}</span>
1160             <span className="cant">{c.cantidad} {c.unidad}</span>
1161         </div>
1162     ))
1163 })
1164 {cola.length > 0 && (
1165     <span className="chip oro">{cola.length} registros por sincronizar</span>
1166 )}
1167 <p className="pista" style={{ marginTop: 10 }}>
1168     Al volver la señal, la cola se envía sola a la API y las validaciones
1169     se aplican igual que en línea.
1170 </p>
1171 </div>
1172 </div>
1173 );
1174 }

```

frontend/src/vistas/Legalizacion.jsx

297 LÍNEAS

Lo pedido contra lo usado.

```

1 import { useEffect, useState } from "react";
2 import { pedir } from "../api";
3 import { hablar, escuchar, esAfirmacion, esNegacion } from "../voz";
4 import Marco from "../Marco";
5 import Dialogo from "../Dialogo";
6 import Icono from "../Iconos";
7
8 const fmt = (n) => Number(n).toLocaleString("es-CO", { maximumFractionDigits: 2 });
9
10 /** Arma en palabras lo que ya se ve en la tabla, igual a como lo diría
11     CuentaVoz - sobre datos reales, no un guion fijo. */
12 function narrar(sobranante, merma) {
13     const partes = [];
14     if (sobranante.length) {
15         const detalle = sobranante
16             .map((l) => `${fmt(Math.abs(l.diferencia))} de ${l.nombre.toLowerCase()}`)
17             .join(" y ");
18         partes.push(`Sobró ${detalle}: lo devuelvo a la bodega.`);
19     }
20     if (merma.length) {
21         const detalle = merma
22             .map((l) => `${fmt(l.diferencia)} en ${l.nombre.toLowerCase()}`)
23             .join(", ");
24         partes.push(`Hubo merma de ${detalle}.`);
25     }
26     if (!partes.length) partes.push("Todo cuadró exacto entre lo pedido y lo usado.");
27     partes.push("¿Confirmo la legalización?");
28     return partes.join(" ");
29 }
30
31 export default function Legalizacion({ token, servicioId = 1, ir, usuario }) {
32     const esAuditor = usuario?.perfil === "auditor";
33     const [comp, setComp] = useState(null);
34     const [listo, setListo] = useState(false);
35     const [err, setErr] = useState("");
36     const [respuesta, setRespuesta] = useState("");
37     const [pedirAjuste, setPedirAjuste] = useState(false);
38     const [verMerma, setVerMerma] = useState(false);
39     const [estado, setEstado] = useState("listo");
40
41     function cargar() {
42         pedir(`/api/legalizacion/${servicioId}`, {}, token)
43             .then((c) => {
44                 setComp(c);
45                 const s = c.lineas.filter((l) => l.diferencia < 0);
46                 const m = c.lineas.filter((l) => l.diferencia > 0);
47                 const texto = narrar(s, m);
48                 setRespuesta(texto);
49                 // sin esto, la pregunta "¿Confirmo la legalización?" solo se veía
50                 // escrita - nunca se oía al llegar, a diferencia de cómo ya saluda
51                 // el resto de la app (Panel, Auditoría) al entrar a una pantalla.
52                 if (c.lineas.length > 0) hablar(texto);
53             })
54             .catch((e) => setErr(e.message));
55     }
56     useEffect(cargar, [servicioId, token]);
57
58     // La pregunta hablada ("¿Confirmo la legalización?") no tenía con qué
59     // contestar por voz - solo el botón. "sí/confirmo" confirma; un "no"
60     // limpio (sin nada más dicho) solo lo reconoce; cualquier otra cosa se
61     // trata como el mismo tipo de corrección que ya hace "Ajustar por voz"

```

```

62 // (texto libre: "el pollo fueron doce kilos"), así decir la corrección
63 // de una vez - sin abrir el diálogo aparte - también funciona.
64 function alMicrofono() {
65   escuchar({
66     alTexto: (texto) => {
67       setEstado("listo");
68       if (esAfirmacion(texto, ["confirma", "legaliza"])) { confirmar(); return; }
69       if (esNegacion(texto) && texto.trim().split(/\s+/).length <= 2) {
70         const msg = "Entendido, no confirmo todavía. Dígame qué insumo ajustar, "
71           + "por ejemplo «el pollo fueron doce kilos.»";
72         setRespuesta(msg);
73         hablar(msg);
74         return;
75       }
76       ajustarPorVoz(texto);
77     },
78     alEstado: setEstado,
79     alError: setErr,
80   });
81 }
82
83 async function confirmar() {
84   try {
85     await pedir("/api/legalizacion/confirmar", {
86       method: "POST",
87       body: JSON.stringify({ servicio_id: servicioId }),
88     }, token);
89     setListo(true);
90     hablar("Legalización confirmada. El sobrante volvió a la bodega y la merma quedó registrada.");
91   } catch (e) {
92     // todo lo que el agente muestra tambien lo dice en voz - aqui la
93     // persona acaba de decir "sí" o tocar el botón esperando una
94     // confirmación hablada, y un fallo que solo se ve en rojo se pierde
95     // si no está mirando la pantalla.
96     setErr(e.message);
97     hablar(e.message);
98   }
99 }
100
101 async function ajustarPorVoz(texto) {
102   setPedirAjuste(false);
103   if (!texto) return;
104   setErr("");
105   try {
106     const c = await pedir("/api/legalizacion/ajustar", {
107       method: "POST",
108       body: JSON.stringify({ servicio_id: servicioId, texto }),
109     }, token);
110     setComp(c);
111     const s = c.lineas.filter((l) => l.diferencia < 0);
112     const m = c.lineas.filter((l) => l.diferencia > 0);
113     const msg = "Listo, ajustado. " + narrar(s, m);
114     setRespuesta(msg);
115     hablar(msg);
116   } catch (e) {
117     setErr(e.message);
118     hablar(e.message);
119   }
120 }
121
122 if (err) return <Marco titulo="Legalización"><p className="error">{err}</p></Marco>;
123 if (!comp) return <Marco titulo="Legalización"><p className="cargando">Cargando...</p></Marco>;
124
125 const sobrante = comp.lineas.filter((l) => l.diferencia < 0);
126 const merma = comp.lineas.filter((l) => l.diferencia > 0);
127
128 if (listo) {
129   const fecha = new Date().toLocaleDateString("es-CO", { day: "numeric", month: "long" });
130   return (
131     <Marco titulo="Legalización · Ajuste confirmado" chip={{ texto: "LEGALIZADO", tipo: "verde" }}>
132       <div className="banner ok">
133         <span className="ico">✓</span>
134         <span>
135           <b>Legalización confirmada – servicio de almuerzo del {fecha}</b>
136           <span>El consumo real quedó registrado y el inventario de la cocina ya refleja los ajustes.</span>
137         </span>
138       </div>
139       <div className="kpis">
140         <div className="card" style={{ borderTop: "4px solid var(--verde)", marginBottom: 0 }}>
141           <h3 style={{ color: "var(--verde)" }}>Devuelto a bodega</h3>
142           {sobrante.length ? sobrante.map((l, i) => (
143             <p key={i} style={{ fontSize: ".9rem" }}>
144               {fmt(Math.abs(l.diferencia))} {l.unidad} de {l.nombre.toLowerCase()}
145             </p>
146           )) : <p className="vacio">Nada por devolver.</p>
147         </div>

```

```

148     <div className="card" style={{ borderTop: "4px solid var(--amarillo)", marginBottom: 0 }}>
149       <h3 style={{ color: "var(--amarillo-tx)" }}>Merma registrada</h3>
150       {merma.length ? merma.map((l, i) => (
151         <p key={i} style={{ fontSize: ".9rem" }}>
152           {fmt(1.diferencia)} {l.unidad} de {l.nombre.toLowerCase()} – con nota del chef
153         </p>
154       )) : <p className="vacio">Sin merma en este servicio.</p>
155     </div>
156     <div className="card" style={{ borderTop: "4px solid var(--azul)", marginBottom: 0 }}>
157       <h3 style={{ color: "var(--azul)" }}>Histórico actualizado</h3>
158       <p style={{ fontSize: ".9rem" }}>
159         Consumo real por porción, para afinar la próxima receta.
160       </p>
161     </div>
162   </div>
163   <div className="card">
164     <h3>Qué pasa ahora con esta información</h3>
165     <div className="registro"><span className="ok">✓</span>
166     <span>El archivo de ajuste queda listo para cargarse a My Inventory con los nombres y códigos oficiales.</span></div>
167     <div className="registro"><span className="ok">✓</span>
168     <span>El consumo real alimenta el histórico: si el plato gasta siempre más de un insumo que lo que dice la receta, el
169     sistema lo detecta.</span></div>
170     <div className="registro"><span className="ok">✓</span>
171     <span>La merma queda trazada con responsable y hora: la revisión ya no depende de la memoria de nadie.</span></div>
172     {esAuditor && (
173       <div className="grilla-botones">
174         <button className="btn"
175           onClick={() => ir && ir("reportes", { tabInicial: "analisis" })}>
176           Ver el reporte del servicio
177         </button>
178       </div>
179     )}
180   </div>
181 </div>
182 }
183
184 return (
185   <Marco titulo={`Legalización · ${comp.plato ? comp.plato.toLowerCase() : "servicio de almuerzo"}`}
186     chip={{ texto: "SERVICIO CERRADO", tipo: "" }}>
187     <div className="chips">
188       <span className="chip">
189         {comp.porciones ? `${comp.porciones} porciones servidas` : `${comp.lineas.length} insumos`}
190       </span>
191       <span className="chip verde">{sobrante.length} sobrantes</span>
192       <span className="chip oro">{merma.length} merma{merma.length === 1 ? "" : "s"} por revisar</span>
193     </div>
194
195     <p className="pista" style={{ marginBottom: 12 }}>
196       Al cerrar el servicio se compara lo pedido con lo realmente usado: cada
197       diferencia queda explicada como sobrante o como merma.
198     </p>
199
200     <div className="card">
201       {comp.lineas.length === 0 ? (
202         <p className="vacio">
203           No hay líneas de servicio todavía. Genere un pedido en el menú Pedidos.
204         </p>
205       ) : (
206         <table>
207           <thead>
208             <tr>
209               <th>Insumo</th><th>Pedido</th><th>Usado</th>
210               <th>Diferencia</th><th>Qué significa</th>
211             </tr>
212           </thead>
213           <tbody>
214             {comp.lineas.map((l) => (
215               <tr key={l.codigo}>
216                 <td>{l.nombre}</td>
217                 <td>{l.pedido}</td>
218                 <td>{l.usado}</td>
219                 <td className="dif">
220                   {l.diferencia > 0 ? `+${l.diferencia}` : l.diferencia}
221                 </td>
222                 <td>{l.lectura}</td>
223               </tr>
224             ))}
225           </tbody>
226         </table>
227       )}
228     </div>
229
230     {comp.lineas.length > 0 && (
231       <div className="card">
232         <div style={{ display: "flex", justifyContent: "space-between", alignItems: "center", gap: 10 }}>

```

```

233     <p className="rotulo" style={{ margin: 0 }}>CuentaVoz responde</p>
234     <button className={mic-btn ${estado === "escuchando" ? "escuchando" : ""}}
235       style={{ width: 46, height: 46, fontSize: "1.2rem", flex: "none" }}
236       onClick={alMicrofono} title="responda «sí» para confirmar, o dicte un ajuste"
237       aria-label="Responder por voz: diga sí para confirmar, o dicte un ajuste">
238       <Icono nombre="microfono" tam={20} />
239     </button>
240   </div>
241   <div className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</div>
242   <p className="pista" style={{ marginTop: 8 }}>
243     o diga «sí» para confirmar, o dicte un ajuste como «el pollo fueron doce kilos»
244   </p>
245   <details style={{ marginTop: 4 }}>
246     <summary className="pista" style={{ cursor: "pointer" }}>
247       Ver frases exactas que entiende el agente aquí
248     </summary>
249     <ul style={{ margin: "8px 0 0", paddingLeft: 18 }}>
250       {["sí", "confirmo", "legaliza",
251         "no (le pregunta qué insumo ajustar)",
252         "el pollo fueron doce kilos", "el aceite fue un litro"].map((ej, i) => (
253         <li key={i} className="pista" style={{ marginBottom: 4 }}>«{ej}»</li>
254       ))}
255     </ul>
256   </details>
257   {err && <p className="error" style={{ marginTop: 10 }}>{err}</p>}
258   <div className="grilla-botones">
259     <button className="btn verde" onClick={confirmar}
260       style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
261       <Icono nombre="aprobar" tam={16} />
262       Confirmar legalización
263     </button>
264     <button className="btn oro" onClick={() => setVerMerma(true)} disabled={!merma.length}
265       style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
266       <Icono nombre="advertencia" tam={16} />
267       Explicar la merma
268     </button>
269     <button className="btn borde" onClick={() => setPedirAjuste(true)}
270       style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
271       <Icono nombre="microfono" tam={16} />
272       Ajustar por voz
273     </button>
274   </div>
275 </div>
276 }}
277
278 {pedirAjuste && (
279   <Dialogo titulo="Ajustar por voz"
280     mensaje="Diga el insumo y cuánto se usó realmente:"
281     conCampo conVoz placeholder="el pollo fueron doce kilos"
282     onAceptar={ajustarPorVoz}
283     onCancel={() => setPedirAjuste(false)} />
284 )}
285
286 {verMerma && (
287   <Dialogo titulo="Merma de este servicio"
288     mensaje={merma.length
289       ? merma.map((l) => `${l.nombre}: +${fmt(l.diferencia)} de más que lo pedido – revisar con el chef.`).join("\n")
290       : "No hay merma en este servicio."}
291     textoAceptar="Entendido" textoCancelar="Cerrar"
292     onAceptar={() => setVerMerma(false)}
293     onCancel={() => setVerMerma(false)} />
294 )}
295 </Marco>
296 );
297 }

```

frontend/src/vistas/Bodegas.jsx

848 LÍNEAS

El parque en vivo y su detalle.

```

1 import { useEffect, useState, useRef } from "react";
2 import { pedir, BASE, leerToken, descargarReporte, preguntarAsistente } from "../api";
3 import { escuchar, hablar, esAfirmacion, esNegacion, quitarTildes } from "../voz";
4 import Marco from "../Marco";
5 import Dialogo from "../Dialogo";
6 import Icono from "../Iconos";
7
8 const ETIQUETA = {
9   pendiente: ["PENDIENTE", "gris"], en_conteo: ["EN CONTEO", "azul"],
10   en_auditoria: ["EN AUDITORÍA", "oro"], cerrada: ["CERRADA", "verde"],
11 };
12 const ICONO_ESTADO = {
13   pendiente: "🕒", en_conteo: "🕒", en_auditoria: "🔍", cerrada: "✅",

```

```

14  };
15
16  // Frases que este buscador ya entiende sin pasar por el agente general -
17  // se resuelven de una en el propio frontend (_accionLocalDeLista /
18  // _accionLocalDeResultado más abajo). Documentarlas aquí es lo mismo que
19  // el desplegable "Ver frases exactas" de las demás pantallas.
20  function _ejemplosLista(esAuditor) {
21    return [
22      "arroz, aceite, tres tablas para picar...",
23      "bodega nueva",
24      ...(esAuditor ? ["exportar estado", "asignar personas a bodegas"] : []),
25      "ver movimientos", "comparar con la receta",
26      ...(esAuditor ? ["generar reporte"] : []),
27      "volver al tablero",
28    ];
29  }
30  function _ejemplosDetalle(esAuditor, estado) {
31    return [
32      "un artículo de esta bodega",
33      ...(estado === "pendiente" || estado === "en conteo" ? ["seguir contando"] : []),
34      ...(esAuditor && estado === "cerrada" ? ["reabrir"] : []),
35      "todos los artículos",
36      ...(esAuditor ? ["descargar", "ver en el panel gerencial"] : []),
37      "cerrar detalle",
38    ];
39  }
40
41  export default function Bodegas({ token, usuario, ir }) {
42    const [lista, setLista] = useState(null);
43    const [filtro, setFiltro] = useState("todas");
44    const [detalle, setDetalle] = useState(null);
45    const [detalleId, setDetalleId] = useState(null);
46    const [cargandoDetalle, setCargandoDetalle] = useState(false);
47    const [busca, setBusca] = useState("");
48    const [consulta, setConsulta] = useState(null);
49    const [escuchando, setEscuchando] = useState(false);
50    const [movimientos, setMovimientos] = useState(null);
51    const [enRecetas, setEnRecetas] = useState(null);
52    const [articulosBodega, setArticulosBodega] = useState(null);
53    const [buscaArticuloBodega, setBuscaArticuloBodega] = useState("");
54    const [escuchandoBodega, setEscuchandoBodega] = useState(false);
55    // Cuando lo buscado en el detalle no es ningún artículo de ESTA bodega,
56    // la respuesta del agente general (pregunta suelta u orden de navegar)
57    // - mismo patrón que "consulta" en la lista principal.
58    const [respuestaAgenteDetalle, setRespuestaAgenteDetalle] = useState(null);
59    const [msg, setMsg] = useState("");
60    const [pedirMotivo, setPedirMotivo] = useState(false);
61    const [pidiendoBodegaNueva, setPidiendoBodegaNueva] = useState(false);
62    const esAuditor = usuario?.perfil === "auditor";
63    const rec = useRef(null);
64    // Igual que en el selector de bodega de Conteo: cada escucha de esta
65    // búsqueda lleva su propio número, y si el micrófono anterior entrega
66    // su resultado tarde (mientras todavía se está diciendo la sugerencia)
67    // se descarta en vez de procesarse como una búsqueda nueva.
68    const idEscuchaBusqueda = useRef(0);
69
70    useEffect(() => {
71      // si falla, resuelve a [] en vez de dejar lista en null para siempre:
72      // sin esto, un fallo de red aquí dejaba el tablero mostrando
73      // "Cargando..." sin fin, sin manera de saber que algo salió mal.
74      pedir("/api/bodegas?propias=1", {}, token).then(setLista)
75        .catch((e) => { setLista([]); setMsg(e.message); });
76      /* tablero en vivo: el WebSocket avisa a todas las tabletas */
77      const ws = new WebSocket(
78        BASE.replace("http", "ws") + "/api/bodegas/estado?token=" + (token || leerToken())
79      );
80      ws.onmessage = (e) => {
81        const estados = JSON.parse(e.data);
82        setLista((previa) => {
83          const prev = previa || [];
84          const hayNuevas = estados.some((x) => !prev.some((b) => b.id === x.id));
85          if (hayNuevas) {
86            // bodega creada despues de cargar el tablero: se necesita el
87            // listado completo (referencias, persona, avance) que el
88            // WebSocket no manda, no solo el id/estado.
89            pedir("/api/bodegas?propias=1", {}, token).then(setLista).catch(() => {});
90            return prev;
91          }
92          return prev.map((b) => {
93            const n = estados.find((x) => x.id === b.id);
94            return n ? { ...b, estado: n.estado } : b;
95          });
96        });
97      };
98      return () => ws.close();
99    }, [token]);

```

```

100
101 const vistas = (lista || []).filter((b) => filtro === "todas" || b.estado === filtro);
102 const cuenta = (e) => (lista || []).filter((b) => b.estado === e).length;
103
104 async function verDetalle(id) {
105     // Si se llega desde la sugerencia de bodega del buscador de
106     // ingredientes, "consulta" sigue puesto - sin limpiarlo, la sección
107     // de detalle no se muestra (su condición exige "!consulta").
108     setConsulta(null);
109     setDetalleId(id);
110     setArticulosBodega(null);
111     setBuscaArticuloBodega("");
112     setRespuestaAgenteDetalle(null);
113     setCargandoDetalle(true);
114     try {
115         setDetalle(await pedir(`/api/bodegas/${id}/detalle`, {}, token));
116     } finally {
117         setCargandoDetalle(false);
118     }
119 }
120
121 async function verTodosLosArticulos() {
122     if (articulosBodega) { setArticulosBodega(null); return; }
123     setArticulosBodega(await pedir(`/api/bodegas/${detalleId}/articulos`, {}, token));
124 }
125
126 async function buscarEnBodega(texto) {
127     setBuscaArticuloBodega(texto);
128     setRespuestaAgenteDetalle(null);
129     if (texto && !articulosBodega) {
130         setArticulosBodega(await pedir(`/api/bodegas/${detalleId}/articulos`, {}, token));
131     }
132 }
133
134 function _filtrarArticulosBodega(texto, lista) {
135     const palabras = texto.toLowerCase().replace(/[.,;:]/g, " ").split(/\s+/).filter(Boolean);
136     if (!palabras.length) return lista || [];
137     return (lista || []).filter((a) => {
138         const nombre = a.articulo.toLowerCase();
139         return palabras.every((p) => nombre.includes(p));
140     });
141 }
142
143 // Al confirmar (Enter, o al terminar de dictar): si lo escrito/dicho no
144 // encuentra ningún artículo DENTRO de esta bodega, cae al mismo agente
145 // general que ya usa la lista principal - un solo cuadro para todo,
146 // también aquí, en vez de un segundo "Pregúntele al agente" aparte.
147 // Igual que _accionLocalDeResultado, pero para los botones del DETALLE
148 // de una bodega (Cerrar detalle, Seguir contando...) - se revisan antes
149 // que la búsqueda de artículo/el agente general, porque esos botones
150 // no tienen equivalente en ningún otro lado: decir "cerrar detalle" sin
151 // esto caía en el agente general, que ni sabe qué botones hay en esta
152 // pantalla en particular ("Esa opción no la puedo hacer por voz").
153 function _accionLocalDeDetalle(texto) {
154     const t = quitarTildes(texto.toLowerCase().replace(/[,.;:!?]/g, "").trim());
155     if (/^b(cerrar|cierra|salir)\b/.test(t)) return "cerrar";
156     if (/^b(seguir|sigue|continua|w*|contar|cuenta)\b/.test(t)
157         && (detalle?.estado === "pendiente" || detalle?.estado === "en_conteo")) return "contar";
158     if (/^b(reabrir|reabre)\b/.test(t) && esAuditor && detalle?.estado === "cerrada") return "reabrir";
159     if (/^b(articulos?)\b/.test(t)) return "articulos";
160     if (/^b(descargar)\b/.test(t) && esAuditor) return "descargar";
161     if (/^b(panel)\b/.test(t) && esAuditor) return "panel";
162     return null;
163 }
164
165 async function confirmarBusquedaEnBodega(texto, miId) {
166     if (!texto || !texto.trim()) return;
167     if (miId === undefined) miId = ++idEscuchaBusqueda.current;
168     const accion = _accionLocalDeDetalle(texto);
169     if (accion === "cerrar") { setDetalle(null); setDetalleId(null); return; }
170     if (accion === "contar") { ir && ir("conteo", { bodegaSugerida: detalle.bodega }); return; }
171     if (accion === "reabrir") { setPedirMotivo(true); return; }
172     if (accion === "articulos") { verTodosLosArticulos(); return; }
173     if (accion === "descargar") { exportarDetalle(); return; }
174     if (accion === "panel") { ir && ir("panel"); return; }
175     setBuscaArticuloBodega(texto);
176     let lista = articulosBodega;
177     if (!lista) {
178         lista = await pedir(`/api/bodegas/${detalleId}/articulos`, {}, token);
179         if (miId !== idEscuchaBusqueda.current) return;
180         setArticulosBodega(lista);
181     }
182     if (_filtrarArticulosBodega(texto, lista).length > 0) {
183         setRespuestaAgenteDetalle(null);
184         return;
185     }
186
187     const r = await preguntarAsistente(texto, "bodegas", token);
188     if (miId !== idEscuchaBusqueda.current) return;
189     setRespuestaAgenteDetalle(r);
190     hablar(r.respuesta_hablada);
191     if (r.accion === "navegar" && r.destino && ir) {

```

```

186     ir(r.destino, { tabInicial: r.pestana || undefined, bodegaSugerida: r.bodega || undefined });
187 }
188 }
189 function buscarEnBodegaPorVoz() {
190     setMsg("");
191     const miId = ++idEscuchaBusqueda.current;
192     rec.current = escuchar({
193         alTexto: (t) => {
194             if (miId !== idEscuchaBusqueda.current) return;
195             confirmarBusquedaEnBodega(t, miId);
196         },
197         alEstado: (e) => setEscuchandoBodega(e === "escuchando"),
198         alError: setMsg,
199     });
200 }
201 // Frases que activan un botón YA VISIBLE en el resultado de una
202 // búsqueda anterior ("volver al tablero", "ver movimientos..."), en
203 // vez de tratarse como una búsqueda nueva. Van ANTES que cualquier
204 // otra cosa: "tablero" es también parte de un producto real
205 // ("BORRADOR PARA TABLERO FELPA"), así que sin este chequeo "volver al
206 // tablero" perdía contra ese artículo en la búsqueda por palabras y
207 // nunca se reconocía como la orden que era.
208 function _accionLocalDeResultado(texto) {
209     const t = quitarTildes(texto.toLowerCase()).replace(/[.,;:!?i?j]/g, "").trim();
210     if (/\\b(volver|tablero|atras|regresar)\\b/.test(t)) return "volver";
211     if (/\\bmovimientos?\\b/.test(t)) return "movimientos";
212     if (/\\brecetas?\\b/.test(t)) return "receta";
213     if (/\\breporte\\b/.test(t)) return "reporte";
214     return null;
215 }
216 // "miId" solo llega cuando la búsqueda vino de buscarPorVoz() (ver ese
217 // comentario) - una búsqueda escrita y con clic en "Buscar" no debe
218 // quedarse esperando una respuesta hablada que nadie va a dar.
219 // Órdenes sobre los botones de la lista principal ("+ Bodega nueva",
220 // "Exportar estado", "Asignar personas a bodegas") - mismo patrón que
221 // _accionLocalDeResultado/_accionLocalDeDetalle: se revisan antes que
222 // cualquier búsqueda, porque el agente general no sabe qué botones hay
223 // en esta pantalla en particular.
224 function _accionLocalDeLista(texto) {
225     const t = quitarTildes(texto.toLowerCase()).replace(/[.,;:!?i?j]/g, "").trim();
226     if (/\\bbodega\\b/.test(t) && /\\b(nueva|crear|solicitar)\\b/.test(t)) return "nueva";
227     if (/\\bexportar\\b/.test(t) && esAuditor) return "exportar";
228     if (/\\basignar\\b/.test(t) && esAuditor) return "asignar";
229     return null;
230 }
231 async function buscarArticulo(codigo = "", texto = busca, miId) {
232     if (!texto.trim()) return;
233     if (!consulta && !detalle) {
234         const accionLista = _accionLocalDeLista(texto);
235         if (accionLista === "nueva") { setPidiendoBodegaNueva(true); return; }
236         if (accionLista === "exportar") { exportarEstado(); return; }
237         if (accionLista === "asignar") { ir && ir("ajustes", { tabInicial: "usuarios" }); return; }
238     }
239     // Si queda una sugerencia de bodega pendiente en pantalla, un "sí"/
240     // "no" nuevo la responde - sin importar si llegó como continuación
241     // de la misma escucha o como una búsqueda nueva escrita/hablada
242     // aparte (tocando "Buscar" de nuevo, por ejemplo). Antes, un "sí"
243     // que no llegara EXACTAMENTE como respuesta de esa misma escucha se
244     // trataba como una búsqueda nueva de "sí" a secas, y el agente
245     // general (que no tiene ni idea de la sugerencia pendiente)
246     // respondía cualquier cosa en vez de abrir el detalle.
247     if (consulta?.sugerencia_bodega_id) {
248         if (esAfirmacion(texto, ["ver"])) { verDetalle(consulta.sugerencia_bodega_id); return; }
249         if (esNegacion(texto)) {
250             setConsulta(null);
251             setBusca("");
252             const msg = "Listo, queda ahí. Puede seguir buscando cuando quiera.";
253             setMsg(msg);
254             hablar(msg);
255             return;
256         }
257     }
258     if (consulta?.bodegas?.length > 0) {
259         const accion = _accionLocalDeResultado(texto);
260         if (accion === "volver") { volverAlTablero(); return; }
261         if (accion === "movimientos") { verMovimientos(); return; }
262         if (accion === "receta") { verEnRecetas(); return; }
263         if (accion === "reporte" && esAuditor) { generarReporteArticulo(); return; }
264     }
265     setBusca(texto);
266     setMovimientos(null);
267     setEnRecetas(null);
268     const r = await pedir(
269         `/api/articulos/consulta?q=${encodeURIComponent(texto)}&codigo=${codigo}`, {}, token);
270     if (miId !== undefined && miId !== idEscuchaBusqueda.current) return;
271     setConsulta(r);

```

```

272 // Un solo buscador para todo: si ni fue un artículo ni una bodega,
273 // el backend ya lo intentó como pregunta suelta o como orden de
274 // navegar ("llévame a reportes", "abra kiosco taquilla ayb") - si
275 // trae una acción, se seguía igual que hace AsistenteVoz.
276 if (r.accion === "navegar" && r.destino && ir) {
277   hablar(r.resumen);
278   ir(r.destino, { tabInicial: r.pestana || undefined, bodegaSugerida: r.bodega || undefined });
279   return;
280 }
281 // Si la búsqueda fue por voz y salió una bodega sugerida, no basta
282 // con decir la sugerencia y dejarla ahí esperando un clic: se
283 // pregunta y se escucha la respuesta, igual que ya hace Conteo al
284 // confirmar una bodega - "tocar el botón" no debería ser la única
285 // forma de seguir cuando se llegó hasta aquí hablando.
286 if (miId !== undefined && r.sugerencia_bodega_id) {
287   await hablar(`${r.resumen} Diga «sí» para ver el detalle.`);
288   if (miId !== idEscuchaBusqueda.current) return;
289   rec.current = escuchar({
290     alTexto: (respuesta) => {
291       if (miId !== idEscuchaBusqueda.current) return;
292       if (esAfirmacion(respuesta, ["ver"])) {
293         verDetalle(r.sugerencia_bodega_id);
294       } else if (esNegacion(respuesta)) {
295         const msg = "Listo, queda ahí. Puede seguir buscando cuando quiera.";
296         setMsg(msg);
297         hablar(msg);
298       } else {
299         setMsg("No le entendí un «sí» o un «no». Use el botón «Ver esta bodega.»);
300       }
301     },
302     alEstado: (e) => setEscuchando(e === "escuchando"),
303     alError: setMsg,
304   });
305   return;
306 }
307 hablar(r.resumen);
308 }
309 function buscarPorVoz() {
310   setMsg("");
311   const miId = ++idEscuchaBusqueda.current;
312   rec.current = escuchar({
313     alTexto: (t) => {
314       if (miId !== idEscuchaBusqueda.current) return;
315       buscarArticulo("", t, miId);
316     },
317     alEstado: (e) => setEscuchando(e === "escuchando"),
318     alError: setMsg,
319   });
320 }
321 function volverAlTablero() {
322   setConsulta(null);
323   setBusca("");
324 }
325 async function verMovimientos() {
326   setMovimientos(await pedir(`/api/articulos/${consulta.codigo}/movimientos`, {}, token));
327 }
328 async function verEnRecetas() {
329   setEnRecetas(await pedir(`/api/articulos/${consulta.codigo}/en-recetas`, {}, token));
330 }
331 async function generarReporteArticulo() {
332   setMsg("");
333   try {
334     const r = await pedir("/api/reportes?formato=xlsx", { method: "POST" }, token);
335     await descargarReporte(r.archivo, token);
336     setMsg("Reporte generado y descargado.");
337   } catch (e) { setMsg(e.message); }
338 }
339 async function reabrir(motivo) {
340   setPedirMotivo(false);
341   if (!motivo || !motivo.trim()) return;
342   try {
343     await pedir(`/api/bodegas/${detalleId}/reabrir`, {
344       method: "POST", body: JSON.stringify({ motivo }),
345     }, token);
346     setMsg("Bodega reabierto: queda registrada la justificación.");
347     verDetalle(detalleId);
348     pedir("/api/bodegas?propias=1", {}, token).then(setLista).catch(() => {});
349   } catch (e) { setMsg(e.message); }
350 }
351
352 async function solicitarBodegaNueva(nombre) {
353   setPidiendoBodegaNueva(false);
354   if (!nombre || !nombre.trim()) return;
355   try {
356     const r = await pedir("/api/bodegas/crear-pendiente", {
357       method: "POST", body: JSON.stringify({ nombre }),

```

```

358     }, token);
359     setMsg(r.respuesta_hablada || "Solicitud enviada.");
360   } catch (e) { setMsg(e.message); }
361 }
362
363 async function exportarEstado() {
364   setMsg("");
365   try {
366     const r = await pedir("/api/bodegas/exportar-estado?formato=xlsx", { method: "POST" }, token);
367     await descargarReporte(r.archivo, token);
368     setMsg("Estado del tablero exportado y descargado.");
369   } catch (e) { setMsg(e.message); }
370 }
371
372 async function exportarDetalle() {
373   setMsg("");
374   try {
375     const r = await pedir(`/api/bodegas/${detalleId}/exportar-detalle?formato=xlsx`,
376       { method: "POST" }, token);
377     await descargarReporte(r.archivo, token);
378     setMsg("Detalle de la bodega exportado y descargado.");
379   } catch (e) { setMsg(e.message); }
380 }
381
382 const tituloDetalle = detalle ? `Bodegas · ${detalle.bodega.toLowerCase().replace(/\b\w/g, (c) => c.toUpperCase())}` : "";
383 const chipDetalle = detalle ? (ETIQUETA[detalle.estado] || [detalle.estado?.toUpperCase(), ""]) : null;
384 // solo bloquea la pantalla con "Cargando..." en la primera carga de un
385 // detalle: si ya había uno (ej. reabrir() refrescando el mismo), se deja
386 // ver el dato viejo mientras llega el nuevo, en vez de taparlo con un
387 // aviso de carga que compite con el detalle todavía visible.
388 const mostrarCargandoDetalle = cargandoDetalle && !detalle;
389
390 return (
391   <Marco titulo={consulta ? "Bodegas · consulta de artículo" : detalle ? tituloDetalle : "Bodegas · estado en vivo"}
392     chip={consulta ? { texto: "BÚSQUEDA GLOBAL", tipo: "borde azul" }
393       : detalle ? { texto: chipDetalle[0], tipo: `borde ${chipDetalle[1]}` }
394       : { texto: "EN VIVO", tipo: "verde" }}>
395
396     {mostrarCargandoDetalle && <p className="cargando">Cargando...</p>}
397
398     {!detalle && !mostrarCargandoDetalle && (
399       <div className="card">
400         <div style={{ display: "flex", gap: 10, alignItems: "center", flexWrap: "wrap" }}>
401           <span style={{ color: "var(--grafito)" }}>🔍</span>
402           <input value={busca} onChange={(e) => setBusca(e.target.value)}
403             onKeyDown={(e) => e.key === "Enter" && buscarArticulo()}
404             placeholder="arroz, aceite, cazuela..."
405             aria-label="Buscar artículo, o pregúntele algo al agente"
406             style={{ flex: 1, minWidth: 200, padding: "13px 14px", fontSize: "1rem",
407               border: "1px solid var(--borde)", borderRadius: 12 }} />
408           <button className={`mic-btn ${escuchando ? "escuchando" : ""}`}
409             style={{ width: 46, height: 46, fontSize: "1.2rem" }}
410             onClick={buscarPorVoz} title="o pregunte por voz"
411             aria-label="Buscar por voz, o pregúntele al agente"><Icono nombre="microfono" tam={22} /></button>
412         </div>
413         {!consulta && (
414           <>
415             <p className="pista" style={{ marginTop: 8 }}>o pregunte por voz</p>
416             <details style={{ marginTop: 4 }}>
417               <summary className="pista" style={{ cursor: "pointer" }}>
418                 Ver frases exactas que entiende el agente aquí
419               </summary>
420               <ul style={{ margin: "8px 0 0", paddingLeft: 18 }}>
421                 {_ejemplosLista(esAuditor).map((ej, i) => (
422                   <li key={i} className="pista" style={{ marginBottom: 4 }}>«{ej}»</li>
423                 ))}
424               </ul>
425             </details>
426           </>
427         )}
428       </div>
429       {consulta && (
430         <>
431           <p className="rotulo" style={{ marginTop: 14 }}>CuentaVoz responde</p>
432           <p className="burbuja" aria-live="polite" aria-atomic="true">{consulta.resumen}</p>
433           {consulta.sugerencia_bodega && (
434             <div className="banner" style={{ marginTop: 10 }}>
435               <span className="ico">🔍</span>
436               <span>
437                 <b>{consulta.sugerencia_bodega}</b> es el nombre de una bodega, no de un artículo.
438                 <span> Este buscador es para ingredientes – vea su detalle para
439                   revisarla antes de abrirla.</span>
440               </span>
441               <button className="btn" style={{ flex: "none" }}
442                 onClick={() => verDetalle(consulta.sugerencia_bodega_id)}>
443                 Ver esta bodega

```

```

444         </button>
445     </div>
446 )}
447 {consulta.ambiguo && consulta.alternativas?.length > 0 && (
448     <div className="chips" style={{ marginTop: 10 }}>
449         <span className="pista" style={{ width: "100%" }}>¿Era este otro?</span>
450         {consulta.alternativas.map((a) => (
451             <button key={a.codigo} className="chip oro"
452                 onClick={() => buscarArticulo(a.codigo)}>
453                 {a.nombre}
454             </button>
455         ))}
456     </div>
457 )}
458
459 {consulta.bodegas?.length > 0 && (
460     <>
461         <div className="kpi" style={{ marginTop: 14 }}>
462             <div className="kpi">
463                 <div className="kpi-cabeza">
464                     <span className="icono-kpi">👤</span>
465                     <small>Total en el sistema</small>
466                 </div>
467                 <b>{consulta.total.toLocaleString("es-CO", { maximumFractionDigits: 1 })} {consulta.unidad}</b>
468                 <i>en {consulta.bodegas.length} bodegas</i>
469             </div>
470             <div className="kpi">
471                 <div className="kpi-cabeza">
472                     <span className="icono-kpi">📦</span>
473                     <small>Consumo del mes</small>
474                 </div>
475                 <b>{consulta.consumo_mes} {consulta.unidad}</b>
476                 <i>{consulta.servicios_mes} servicios</i>
477             </div>
478             <div className="kpi verde">
479                 <div className="kpi-cabeza">
480                     <span className="icono-kpi">📄</span>
481                     <small>Cobertura estimada</small>
482                 </div>
483                 <b>{consulta.cobertura_dias != null ? `${consulta.cobertura_dias} días` : "-"}</b>
484                 <i>{consulta.cobertura_dias != null ? "al ritmo actual" : "sin consumo reciente"}</i>
485             </div>
486         </div>
487
488         <table style={{ marginTop: 6 }}>
489             <thead><tr><th>Bodega</th><th>Cantidad</th><th>Última toma</th><th>Estado</th></tr></thead>
490             <tbody>
491                 {consulta.bodegas.map((b, i) => (
492                     <tr key={i}>
493                         <td>{b.bodega}</td><td>{b.cantidad} {consulta.unidad}</td>
494                         <td>{b.ultima_toma}</td><td>{ETIQUETA[b.estado]?.[0] || b.estado}</td>
495                     </tr>
496                 ))}
497             </tbody>
498         </table>
499
500         <div className="grilla-botones">
501             {esAuditor && (
502                 <button className="btn" onClick={generarReporteArticulo}>Generar reporte</button>
503             )}
504             <button className="btn borde" onClick={verMovimientos}>Ver movimientos</button>
505             <button className="btn borde" onClick={verEnRecetas}>Comparar con la receta</button>
506             <button className="btn gris" onClick={volverAlTablero}>← Volver al tablero</button>
507         </div>
508     </>
509 )}
510 </>
511 )}
512 </div>
513 )}
514
515 {msg && <p className="msg-ok">{msg}</p>}
516
517 {!consulta && detalle && (
518     <div className="card">
519         <div style={{ display: "flex", gap: 10, alignItems: "center", flexWrap: "wrap" }}>
520             <span style={{ color: "var(--grafito)" }}>🔍</span>
521             <input value={buscaArticuloBodega}
522                 onChange={(e) => buscarEnBodega(e.target.value)}
523                 onKeyDown={(e) => e.key === "Enter" && confirmarBusquedaEnBodega(buscaArticuloBodega)}
524                 placeholder="artículo en esta bodega, o pregúntele algo al agente..."
525                 aria-label="Buscar artículo en esta bodega, o pregúntele algo al agente"
526                 style={{ flex: 1, minWidth: 200, padding: "13px 14px", fontSize: "1rem",
527                     border: "1px solid var(--borde)", borderRadius: 12 }} />
528             <button className="mic-btn ${escuchandoBodega ? "escuchando" : ""}"
529                 style={{ width: 46, height: 46, fontSize: "1.2rem" }}>

```

```

530         onClick={buscarEnBodegaPorVoz} title="o pregunte por voz"
531         aria-label="Buscar por voz en esta bodega, o pregúntele al agente"><Icono nombre="microfono" tam={22} /></button>
532     </div>
533     <p className="pista" style={{ marginTop: 8, marginBottom: 4 }}>
534         o pregunte por voz - si no es un artículo de esta bodega, se lo pregunta al agente
535     </p>
536     <details style={{ marginBottom: 14 }}>
537         <summary className="pista" style={{ cursor: "pointer" }}>
538             Ver frases exactas que entiende el agente aquí
539         </summary>
540         <ul style={{ margin: "8px 0 0", paddingLeft: 18 }}>
541             {_ejemplosDetalle(esAuditor, detalle.estado).map((ej, i) => (
542                 <li key={i} className="pista" style={{ marginBottom: 4 }}><ej></li>
543             ))}
544         </ul>
545     </details>
546     {respuestaAgenteDetalle && (
547         <>
548             <p className="rotulo">CuentaVoz responde</p>
549             <p className="burbuja" style={{ marginBottom: 14 }}
550                 aria-live="polite" aria-atomic="true">{respuestaAgenteDetalle.respuesta_hablada}</p>
551         </>
552     )}
553
554     <div className="chips">
555         <span className="chip">{detalle.referencias} referencias</span>
556         {detalle.estado === "cerrada" && detalle.hora_cierre && (
557             <span className="chip verde">Cerrada {detalle.hora_cierre}</span>
558         )}
559         {detalle.personas && <span className="chip azul">{detalle.personas}</span>}
560         {detalle.alertas_resueltas > 0 && (
561             <span className="chip oro">
562                 {detalle.alertas_resueltas} alerta{detalle.alertas_resueltas === 1 ? "" : "s"} resuelta
563                 {detalle.alertas_resueltas === 1 ? "" : "s"}
564             </span>
565         )}
566     </div>
567
568     <div className="kpis">
569         <div className="kpi verde">
570             <div className="kpi-cabeza">
571                 <span className="icono-kpi">🟢</span>
572                 <small>Exactitud de esta bodega</small>
573             </div>
574             <b>{detalle.exactitud}</b>
575             <i>{detalle.diferencias.length} diferencias de {detalle.referencias}</i>
576         </div>
577         <div className="kpi">
578             <div className="kpi-cabeza">
579                 <span className="icono-kpi">🟡</span>
580                 <small>Duración del conteo</small>
581             </div>
582             <b>{detalle.duracion_min != null ? `${detalle.duracion_min} min` : "-"}</b>
583             <i>{detalle.tiempo_papel_min}
584                 ? `frente a ~${detalle.tiempo_papel_min} min en papel (estimado)`
585                 : "conteo aún en curso"</i>
586         </div>
587         <div className="kpi">
588             <div className="kpi-cabeza">
589                 <span className="icono-kpi">🟠</span>
590                 <small>Unidades en stock</small>
591             </div>
592             <b>{detalle.unidades_stock.toLocaleString("es-CO")}</b>
593             <i>sumadas todas las referencias</i>
594         </div>
595         <div className="kpi oro">
596             <div className="kpi-cabeza">
597                 <span className="icono-kpi">🟡</span>
598                 <small>Última toma anterior</small>
599             </div>
600             <b>{detalle.ultima_toma_anterior
601                 ? new Date(detalle.ultima_toma_anterior.fecha).toLocaleDateString("es-CO", { day: "numeric", month: "short" })
602                 : "-"}</b>
603             <i>{detalle.ultima_toma_anterior ? `exactitud ${detalle.ultima_toma_anterior.exactitud}%` : "sin cierres previos"}</i>
604         </div>
605     </div>
606
607     <div className="dos-columnas" style={{ gap: 16, marginTop: 4 }}>
608         <div>
609             <h3>Referencias con diferencia</h3>
610             {detalle.diferencias.length > 0 ? (
611                 <table>
612                     <thead><tr><th>Artículo</th><th>Contado</th><th>Sistema</th><th>Dif.</th></tr></thead>
613                     <tbody>
614                         {detalle.diferencias.map((d, i) => (
615                             <tr key={i}><td>{d.articulo}</td><td>{d.contado}</td>

```

```

616         <td>{d.sistema}</td><td className="dif">{d.diferencia}</td></tr>
617     }}}
618 </tbody>
619 </table>
620 ) : <p className="vacio">Todas las referencias cuadraron exactas.</p>
621 {detalle.diferencias.length > 0 && detalle.diferencias.length < detalle.referencias && (
622     <p className="pista" style={{ marginTop: 8 }}>
623         Las otras {detalle.referencias - detalle.diferencias.length} referencias cuadraron exactas.
624     </p>
625 )}
626 {detalle.diferencias.length > 0 && detalle.revisor && (
627     <p className="pista">
628         Las {detalle.diferencias.length} diferencias fueron revisadas por {detalle.revisor} al cierre.
629     </p>
630 )}
631 </div>
632
633 <div>
634     <h3>Línea de tiempo de la toma</h3>
635     {detalle.hitos.length > 0 ? detalle.hitos.map((h, i) => (
636         <div className="registro" key={i}>
637             <span style={{
638                 width: 10, height: 10, borderRadius: "50%", flex: "none",
639                 background: h.tipo === "verde" ? "var(--verde)"
640                     : h.tipo === "oro" ? "var(--amarillo)" : "var(--azul)",
641             }} />
642             <b style={{ color: "var(--grafito)" }}>{h.hora}</b>
643             <span>{h.texto}</span>
644         </div>
645     )) : <p className="vacio">Sin movimientos registrados todavía.</p>
646 </div>
647 </div>
648
649 {articulosBodega && !respuestaAgenteDetalle && (
650     <div style={{ marginTop: 16 }}>
651         <h3 style={{ margin: 0 }}>
652             {buscaArticuloBodega
653                 ? `Artículos que coinciden con "${buscaArticuloBodega}"`
654                 : `Todos los artículos (${articulosBodega.length})`}
655         </h3>
656         <div style={{ maxHeight: 420, overflowY: "auto", marginTop: 10 }}>
657             <table>
658                 <thead><tr><th>Código</th><th>Artículo</th><th>Unidad</th><th>SD</th></tr></thead>
659                 <tbody>
660                     {filtrarArticulosBodega(buscaArticuloBodega, articulosBodega).map((a) => (
661                         <tr key={a.codigo}>
662                             <td>{a.codigo}</td><td>{a.articulo}</td>
663                             <td>{a.unidad}</td><td>{a.sd.toLocaleString("es-CO")}</td>
664                         </tr>
665                     ))}
666                 </tbody>
667             </table>
668         </div>
669     </div>
670 )}
671
672 <div className="grilla-botones">
673     {(detalle.estado === "pendiente" || detalle.estado === "en_conteo") && (
674         // Este detalle es solo de lectura (estado/historial) - para
675         // de verdad contar hay que ir a Conteo. Sin este atajo,
676         // alguien que llega aquí buscando "abrir la bodega" se
677         // queda sin poder hacerlo y tiene que volver a decir/buscar
678         // el nombre desde cero en otra pantalla.
679         <button className="btn" onClick={() => ir && ir("conteo", { bodegaSugerida: detalle.bodega })}
680             style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
681             <Icono nombre="conteo" tam={18} />
682             {detalle.estado === "en_conteo" ? "Seguir contando" : "Contar esta bodega"}
683         </button>
684     )}
685     {esAuditor && detalle.estado === "cerrada" && (
686         <button className="btn borde" onClick={() => setPedirMotivo(true)}
687             style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
688             <Icono nombre="candado" tam={18} />
689             Reabrir la bodega
690         </button>
691     )}
692     <button className="btn borde" onClick={verTodosLosArticulos}
693         style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
694         <Icono nombre="reportes" tam={18} />
695         {articulosBodega ? "Ocultar artículos" : `Ver todos los artículos (${detalle.referencias})`}
696     </button>
697     {esAuditor && (
698         <button className="btn borde" onClick={exportarDetalle}
699             style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
700             <Icono nombre="descargar" tam={18} />
701             Descargar detalle

```

```

702     </button>
703   })
704   {esAuditor && (
705     <button className="btn" onClick={() => ir && ir("panel")}
706       style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
707       <Icono nombre="panel" tam={18} />
708       Ver en el panel gerencial
709     </button>
710   )}
711   <button className="btn gris" onClick={() => { setDetalle(null); setDetalleId(null); }}
712     style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
713     <Icono nombre="salir" tam={18} />
714     Cerrar detalle
715   </button>
716 </div>
717 {esAuditor && detalle.estado === "cerrada" && (
718   <p className="pista" style={{ marginTop: 10 }}>
719     Reabrir una bodega cerrada exige justificación escrita y queda en el registro de trazabilidad.
720   </p>
721 )}
722 </div>
723 )}
724
725 {!consulta && !detalle && !mostrarCargandoDetalle && lista === null && (
726   <p className="cargando">Cargando...</p>
727 )}
728
729 {!consulta && !detalle && !mostrarCargandoDetalle && lista !== null && (
730   <
731     <div className="chips">
732       {[["todas", `${lista.length} bodegas`, ""],
733         ["cerrada", `${cuenta("cerrada")} cerradas`, "verde"],
734         ["en_conteo", `${cuenta("en_conteo")} en conteo`, ""],
735         ["en_auditoria", `${cuenta("en_auditoria")} en auditoria`, "oro"],
736         ["pendiente", `${cuenta("pendiente")} pendientes`, "gris"]].map(([k, t, c]) => (
737           <button key={k} className={`chip ${filtro === k ? "azul" : c}`}
738             onClick={() => setFiltro(k)}>{t}</button>
739         ))}
740     </div>
741
742     /* Arriba, junto a los filtros, y no al final de las 54 tarjetas -
743     son acciones de la pantalla completa, no de una bodega en
744     particular, así que no tiene sentido obligar a bajar todo el
745     tablero para encontrarlas. */
746     <div className="grilla-botones" style={{ marginBottom: 16 }}>
747       {esAuditor && (
748         <button className="btn borde" onClick={exportarEstado}
749           style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
750           <Icono nombre="descargar" tam={18} />
751           Exportar estado
752         </button>
753       )}
754       {esAuditor && (
755         <button className="btn"
756           onClick={() => ir && ir("ajustes", { tabInicial: "usuarios" })}
757           style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
758           <Icono nombre="perfil" tam={18} />
759           Asignar personas a bodegas
760         </button>
761       )}
762       /* Sin esAuditor a proposito: quien suele topar con una bodega
763       que no esta en el catalogo es el auxiliar en el terreno, no
764       el administrador desde su escritorio. */
765       <button className="btn borde" onClick={() => setPidiendoBodegaNueva(true)}
766         style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
767         <Icono nombre="bodegas" tam={18} />
768         Bodega nueva
769       </button>
770     </div>
771
772     {lista.length === 0 ? (
773       <p className="vacio">No hay bodegas para mostrar.</p>
774     ) : (
775       <div className="grid-bodegas">
776         {vistas.map((b) => {
777           const [txt, cls] = ETIQUETA[b.estado] || ["?", "gris"];
778           let detalleLinea;
779           if (b.estado === "en_conteo" && b.persona) {
780             detalleLinea = `${b.persona} · ${b.avance_pct ?? 0}%`;
781           } else if (b.estado === "en_auditoria" && b.persona) {
782             detalleLinea = `${b.persona} · ${b.diferencias ?? 0} dif.`;
783           } else if (b.estado === "cerrada" && b.hora_cierre) {
784             detalleLinea = `${b.referencias} refs · ${b.hora_cierre}`;
785           } else if (b.estado === "pendiente") {
786             detalleLinea = "sin iniciar";
787           } else {

```

```

788         detalleLinea = `${b.referencias} referencias`;
789     }
790     return (
791         <button key={b.id} className={`tarjeta-bodega ${b.estado}`}
792             onClick={() => verDetalle(b.id)}>
793             <b><span className="icono-tarjeta">ICONO_ESTADO[b.estado] || "🚪"</span>{b.bodega}</b>
794             <span className={`chip ${cls} est`}>{txt}</span>
795             <small>{detalleLinea}</small>
796         </button>
797     );
798     }}}
799 </div>
800 )}
801 <p className="pista" style={{ marginTop: 12 }}>
802     Cada tarjeta se actualiza al instante por WebSocket cuando alguien abre o cierra una bodega.
803 </p>
804 </>
805 )}
806
807 {pidiendoBodegaNueva && (
808     <Dialogo titulo={esAuditor ? "Bodega nueva" : "Solicitar bodega nueva"}
809         mensaje={esAuditor
810             ? "Como administrador, la bodega entra directo al catálogo: no necesita otra aprobación."
811             : "Queda pendiente de aprobación: no entra al catálogo hasta que un administrador la confirme desde Auditoría →
Aprobaciones."}
812         conCampo conVoz placeholder="nombre de la bodega"
813         etiquetaCampo="Nombre de la bodega"
814         textoAceptar={esAuditor ? "Crear" : "Solicitar"}
815         onAceptar={solicitarBodegaNueva}
816         onCancel={(() => setPidiendoBodegaNueva(false))} />
817     )}
818
819 {pedirMotivo && (
820     <Dialogo titulo="Reabrir bodega cerrada"
821         mensaje="Esta acción exige una justificación escrita y queda registrada en el historial. Motivo:"
822         conCampo conVoz multilinea placeholder="revisión solicitada por el chef"
823         etiquetaCampo="Motivo de la reapertura"
824         textoAceptar="Reabrir" peligro
825         onAceptar={reabrir}
826         onCancel={(() => setPedirMotivo(false))} />
827     )}
828
829 {movimientos && (
830     <Dialogo titulo="Movimientos recientes"
831         mensaje={movimientos.length
832             ? movimientos.map((m) => `${m.hora} · ${m.bodega} · ${m.persona}: ${m.cantidad} ${m.unidad}`).join("\n")
833             : "Sin conteos registrados todavía para este artículo."}
834         textoAceptar="Cerrar" onAceptar={() => setMovimientos(null)}
835         onCancel={(() => setMovimientos(null))} />
836     )}
837
838 {enRecetas && (
839     <Dialogo titulo="En qué recetas aparece"
840         mensaje={enRecetas.length
841             ? enRecetas.map((r) => `${r.receta}: ${r.por_porcion} por porción (rinde ${r.rendimiento})`.join("\n")
842             : "Este artículo no aparece en ninguna receta cargada."}
843         textoAceptar="Cerrar" onAceptar={() => setEnRecetas(null)}
844         onCancel={(() => setEnRecetas(null))} />
845     )}
846 </Marco>
847 );
848 }

```

frontend/src/vistas/Auditoria.jsx

949 LÍNEAS

Las cuatro pestañas del control.

```

1 import { useEffect, useRef, useState } from "react";
2 import { pedir, enviarTurno } from "../api";
3 import { escuchar, hablar, vozDisponible } from "../voz";
4 import Marco from "../Marco";
5 import Dialogo from "../Dialogo";
6 import AsistenteVoz from "../AsistenteVoz";
7 import Icono from "../Iconos";
8
9 const TABS = [
10   { id: "recuento", t: "Recuento ciego y cierre" },
11   { id: "aprobaciones", t: "Aprobaciones" },
12   { id: "pedidos", t: "Pedidos pendientes" },
13   { id: "alertas", t: "Bandeja de alertas" },
14 ];
15
16 // Ejemplos de lo que ya entiende el agente en esta pantalla: navegar entre

```

```

17 // las cuatro pestañas, resolver aprobaciones/pedidos/alertas por nombre,
18 // seleccionar una bodega para auditar, y preguntar por los pendientes.
19 const _EJEMPLOS_AUDITORIA = [
20   "aprueba costilla de res", "rechaza el kiosco nuevo", "aprueba el pedido de Luis",
21   "rechaza el pedido de sopa de arroz", "resuelve la alerta de papa criolla",
22   "audita el almacén general", "recuento ciego y cierre", "aprobaciones",
23   "pedidos pendientes", "bandeja de alertas", "¿cuántas alertas están abiertas?",
24   "¿cuántas se resolvieron hoy?", "¿cuál es el tiempo medio?",
25   "¿cuántos pedidos están pendientes?", "¿cuántas aprobaciones hay?",
26 ];
27
28 export default function Auditoria({ token, usuario, ir, tabInicial, bodegaAuditar, navSeq }) {
29   const [tab, setTab] = useState(tabInicial || "recuento");
30   // Pedir una pestaña de esta MISMA pantalla por voz no remonta el
31   // componente - sin esto, el useState de arriba no vuelve a leer
32   // tabInicial y la pestaña se queda en la que ya estaba. navSeq (sube en
33   // cada ir()) va en la dependencia también: sin él, pedir la MISMA
34   // pestaña dos veces seguidas (con un clic manual a otra en el medio) no
35   // hacía nada, porque tabInicial no cambiaba de valor - confirmado con
36   // Playwright: "pedidos pendientes" -> clic a Aprobaciones -> "pedidos
37   // pendientes" otra vez se quedaba en Aprobaciones.
38   useEffect(() => { if (tabInicial) setTab(tabInicial); }, [tabInicial, navSeq]);
39   const [enfoco, setEnFoco] = useState(null); // { titulo, chip } para la vista activa
40   const esAuditor = usuario?.perfil === "auditor";
41
42   return (
43     <Marco titulo={enfoco ? enfoco.titulo
44       : "Auditoría · recuento ciego, aprobaciones y cierre"}
45       chip={enfoco ? enfoco.chip
46         : { texto: esAuditor ? "ADMINISTRADORA" : "SOLO LECTURA",
47           tipo: esAuditor ? "azul" : "gris" }}>
48       {!esAuditor && (
49         <div className="banner">
50           <span className="ico">!</span>
51           <span>
52             <b>Su perfil es auxiliar</b>
53             <span>Auditar, aprobar y cerrar bodegas requiere perfil de administrador.
54               Ingrese como «diana» para probarlo.</span>
55           </span>
56         </div>
57       )}
58       {esAuditor && (
59         <AsistenteVoz token={token} vista="auditoria" ir={ir} ejemplos={_EJEMPLOS_AUDITORIA}
60           placeholder="aprueba costilla de res, rechaza el kiosco..."
61           alActualizar={() => window.dispatchEvent(new Event("cuentavoz:auditoria-actualizada"))}> />
62       )}
63       <div className="chips">
64         {TABS.map((tt) => (
65           <button key={tt.id} className={`chip ${tab === tt.id ? "azul" : ""}`}
66             onClick={() => { setTab(tt.id); setEnFoco(null); }}>{tt.t}</button>
67         ))}
68       </div>
69
70       {tab === "recuento" &&
71         <TabRecuento token={token} esAuditor={esAuditor} onEnFoco={setEnFoco}
72           bodegaAuditar={bodegaAuditar} navSeq={navSeq} />
73       {tab === "aprobaciones" && <TabAprobaciones token={token} esAuditor={esAuditor} onEnFoco={setEnFoco} />
74       {tab === "pedidos" && <TabPedidosPendientes token={token} esAuditor={esAuditor} onEnFoco={setEnFoco} />
75       {tab === "alertas" && <TabAlertas token={token} esAuditor={esAuditor} onEnFoco={setEnFoco} />
76     </Marco>
77   );
78 }
79
80 /* — Recuento ciego + cierre con doble firma — */
81 function TabRecuento({ token, esAuditor, onEnFoco, bodegaAuditar, navSeq }) {
82   const [bodegas, setBodegas] = useState(null);
83   const [bodegaId, setBodegaId] = useState(null);
84   const [sesion, setSesion] = useState(null);
85   const [dicho, setDicho] = useState("");
86   const [respuesta, setRespuesta] = useState("");
87   const [comparar, setComparar] = useState(null);
88   const [firmas, setFirmas] = useState(null);
89   const [detalle, setDetalle] = useState(null);
90   const [modoCierre, setModoCierre] = useState(false);
91   const [msg, setMsg] = useState("");
92   const [estado, setEstado] = useState("listo");
93   const [mostrarTeclado, setMostrarTeclado] = useState(false);
94   const [mostrarDictado, setMostrarDictado] = useState(false);
95   const [opciones, setOpciones] = useState(null);
96   const [opcionesPara, setOpcionesPara] = useState(null);
97
98   function cargarBodegas() {
99     // ?propias=1: si este administrador tiene bodegas asignadas (supervisor
100     // de una zona), audita solo esas; sin asignaciones, ve el parque
101     // completo (administrador general) - ver /api/bodegas en el backend.
102     // si falla, resuelve a [] en vez de dejar bodegas en null para

```

```

103 // siempre: sin esto un fallo de red dejaba "Cargando..." sin fin.
104 pedir("/api/bodegas?propias=1", {}, token).then(setBodegas)
105 .catch((e) => { setBodegas([]); setMsg(e.message); });
106 }
107 useEffect(cargarBodegas, [token]);
108
109 const candidatas = (bodegas || []).filter((b) => b.estado === "en_auditoria" || b.estado === "cerrada");
110 // "cerrada" ya significa cerrada de verdad (doble firma), no "lista
111 // para cerrar" - se queda en la lista solo para poder revisarla
112 // (ver el resumen de su cierre), no como algo pendiente de hacer.
113 // Sin esta distinción, el saludo decía "lista para cerrar" o la
114 // contaba entre las "listas para auditar" aunque ya estuviera cerrada.
115 const pendientes = candidatas.filter((b) => b.estado === "en_auditoria");
116 const cerradas = candidatas.filter((b) => b.estado === "cerrada");
117 const [saludoLista, setSaludoLista] = useState("");
118 const yaSaludoLista = useRef(false);
119
120 // Al llegar a esta pestaña sin pedir una bodega puntual, el agente dice
121 // de una vez si hay algo listo para recuento o cierre y qué es. Si ya
122 // llegó con bodegaAuditar (pidió una bodega específica por voz), esto
123 // no se repite: la respuesta de esa orden ya lo dijo.
124 useEffect(() => {
125   if (bodegas === null || yaSaludoLista.current || bodegaAuditar) return;
126   yaSaludoLista.current = true;
127   let texto;
128   if (pendientes.length === 0) {
129     texto = cerradas.length > 0
130       ? `No hay ninguna bodega pendiente de recuento ciego. ${cerradas.length} `
131       + `ya ${cerradas.length === 1 ? "está cerrada" : "están cerradas"}.`
132       : "No hay ninguna bodega lista para recuento ciego o cierre en este momento.";
133   } else if (pendientes.length === 1) {
134     texto = `Tiene una bodega lista para recuento ciego: ${pendientes[0].bodega.toLowerCase()}. `
135       + "¿Quiere auditarla?";
136   } else {
137     const nombres = pendientes.slice(0, 5).map((b) => b.bodega.toLowerCase()).join(", ");
138     texto = `Tiene ${pendientes.length} bodegas listas para recuento ciego: ${nombres}. `
139       + "¿Cuál quiere auditar?";
140   }
141   setSaludoLista(texto);
142   hablar(texto);
143   // eslint-disable-next-line react-hooks/exhaustive-deps
144 }, [bodegas, bodegaAuditar]);
145
146 // "Audita <bodega>" por voz llega hasta aquí como bodegaAuditar - lo
147 // mismo que darle clic a "Abrir" en su tarjeta, sin tocar firmar/cerrar.
148 // Depende solo de bodegaAuditar (no de candidatas/bodegaId) para no
149 // reabrir la sola si la persona ya le dio "Volver a la lista" a mano.
150 useEffect(() => {
151   if (!bodegaAuditar) return;
152   const encontrada = candidatas.find((b) => b.bodega === bodegaAuditar);
153   if (encontrada) {
154     setBodegaId(encontrada.id); verFirmas(encontrada.id);
155     if (encontrada.estado === "cerrada") verDetalle(encontrada.id);
156   }
157   // eslint-disable-next-line react-hooks/exhaustive-deps
158 }, [bodegaAuditar, navSeq]);
159
160 useEffect(() => {
161   if (!bodegaId) { onEnFoco(null); return; }
162   const bActual = (bodegas || []).find((b) => b.id === bodegaId);
163   const nombre = comparar?.bodega || firmas?.bodega || detalle?.bodega || bActual?.bodega;
164   if (!nombre) return;
165   if (modoCierre) {
166     onEnFoco({ titulo: `Cierre de bodega · ${nombre}`,
167       chip: { texto: "LISTA PARA CERRAR", tipo: "borde verde" } });
168   } else if (!sesion && bActual?.estado === "cerrada") {
169     onEnFoco({ titulo: `Auditoria · ${nombre}`,
170       chip: { texto: "CERRADA", tipo: "verde" } });
171   } else {
172     onEnFoco({ titulo: `Auditoria · ${nombre}`,
173       chip: { texto: "RECONTEO CIEGO", tipo: "borde oro" } });
174   }
175 }, [bodegaId, comparar, firmas, detalle, bodegas, modoCierre, sesion]);
176
177 async function iniciar(id) {
178   setMsg(""); setComparar(null);
179   try {
180     const r = await pedir(`/api/bodegas/${id}/auditoria/iniciar`, { method: "POST", token });
181     setBodegaId(id);
182     setSesion(r);
183     setMostrarDictado(true);
184     const saludo = `Recuento ciego iniciado en ${r.bodega.toLowerCase()}. Dícteme el primer producto.`;
185     setRespuesta(saludo);
186     hablar(saludo);
187     verFirmas(id);
188   } catch (e) { setMsg(e.message); }

```

```

189   }
190
191   async function verFirmas(id) {
192     try { setFirmas(await pedir(`/api/bodegas/${id}/firmas`, {}, token)); }
193     catch (_) {}
194   }
195
196   // Solo para una bodega ya CERRADA: sin esto, abrirla aquí mostraba
197   // igual "Iniciar recuento ciego" (sesion sigue null hasta llamar
198   // iniciar()) como si le faltara auditoria, cuando ya quedó cerrada con
199   // doble firma - una pantalla casi vacía y encima engañosa.
200   async function verDetalle(id) {
201     try { setDetalle(await pedir(`/api/bodegas/${id}/detalle`, {}, token)); }
202     catch (_) {}
203   }
204
205   async function procesar(texto, mostrar = texto) {
206     if (!texto || !sesion) return;
207     setDicho(mostrar);
208     try {
209       const t = await enviarTurno(texto, sesion.sesion_id, token, {
210         opciones, opcionesPara,
211         bodegaId, bodegaNombre: sesion?.bodega,
212       });
213       setRespuesta(t.respuesta_hablada || "");
214       setOpciones(t.opciones || null);
215       setOpcionesPara(t.opciones_para || null);
216       hablar(t.respuesta_hablada);
217     } catch (e) { setMsg(e.message); hablar(e.message); }
218   }
219
220   async function verComparar() {
221     setMsg("");
222     try { setComparar(await pedir(`/api/bodegas/${bodegaId}/auditoria/comparar`, {}, token)); }
223     catch (e) { setMsg(e.message); }
224   }
225
226   async function firmarAuditoria() {
227     try {
228       await pedir(`/api/bodegas/${bodegaId}/auditoria/firmar`, { method: "POST" }, token);
229       setMsg("Auditoria firmada.");
230       verFirmas(bodegaId);
231     } catch (e) { setMsg(e.message); }
232   }
233
234   async function cerrarDefinitivo() {
235     try {
236       const r = await pedir(`/api/bodegas/${bodegaId}/cerrar`, { method: "POST" }, token);
237       setMsg(`${r.bodega} cerrada definitivamente con doble firma.`);
238       setBodegaId(null); setSesion(null); setComparar(null); setFirmas(null);
239       setDetalle(null); setModoCierre(false);
240       cargarBodegas();
241     } catch (e) { setMsg(e.message); }
242   }
243
244   function volverALaLista() {
245     setBodegaId(null); setSesion(null); setComparar(null); setFirmas(null);
246     setDetalle(null); setModoCierre(false); setMostrarDictado(false);
247   }
248
249   if (!sesAuditor) return null;
250
251   return (
252     <>
253     {msg && <p className="msg-ok">{msg}</p>}
254
255     {!bodegaId ? (
256       <div className="card">
257         <h3>Bodegas listas para recuento ciego o cierre</h3>
258         {saludoLista && (
259           <>
260             <p className="rotulo">CuentaVoz responde</p>
261             <p className="burbuja" style={{ marginBottom: 12 }}>
262               aria-live="polite" aria-atomic="true">{saludoLista}</p>
263             </>
264           </>
265         )}
266         {bodegas === null ? (
267           <p className="cargando">Cargando...</p>
268         ) : candidatas.length === 0 ? (
269           <p className="vacio">Ninguna bodega firmada por el auxiliar en este momento.</p>
270         ) : (
271           candidatas.map((b) => (
272             <div className="registro" key={b.id}>
273               <span className={`chip ${b.estado === "cerrada" ? "verde" : "oro"}`}>
274                 {b.estado === "cerrada" ? "CERRADA" : "EN AUDITORÍA"}

```

```

275     <span>{b.bodega}</span>
276     <span className="cant">{b.referencias} refs</span>
277     <button className="btn" style={{ padding: "6px 12px", minHeight: 0 }}
278       onClick={() => {
279         setBodegaId(b.id); verFirmas(b.id);
280         if (b.estado === "cerrada") verDetalle(b.id);
281       }}>
282       Abrir
283     </button>
284   </div>
285 ))
286 }}
287 <p className="pista" style={{ marginTop: 10 }}>
288   El recuento del administrador es a ciegas: no ve el conteo del auxiliar
289   hasta comparar.
290 </p>
291 </div>
292 ) : !sesion && (bodegas || []).find((b) => b.id === bodegaId)?.estado === "cerrada" ? (
293   <div className="card">
294     <div style={{ display: "flex", alignItems: "center", gap: 10, marginBottom: 4 }}>
295       <h3 style={{ margin: 0 }}>{detalle?.bodega
296         || (bodegas || []).find((b) => b.id === bodegaId)?.bodega}</h3>
297       <span className="chip verde" style={{ marginLeft: "auto" }}>CERRADA</span>
298     </div>
299     {!detalle ? (
300       <p className="cargando">Cargando...</p>
301     ) : (
302       <>
303         <div className="kpi">
304           <div className="kpi verde">
305             <div className="kpi-cabeza">
306               <span className="icono-kpi">✔</span>
307               <small>Exactitud del cierre</small>
308             </div>
309             <b>{detalle.exactitud}</b>
310             <i>{detalle.diferencias.length} diferencias de {detalle.referencias}</i>
311           </div>
312           <div className="kpi">
313             <div className="kpi-cabeza">
314               <span className="icono-kpi">🔍</span>
315               <small>Referencias</small>
316             </div>
317             <b>{detalle.referencias}</b>
318             <i>{detalle.contadas} contadas</i>
319           </div>
320           <div className="kpi">
312             <div className="kpi-cabeza">
322               <span className="icono-kpi">🕒</span>
323               <small>Hora de cierre</small>
324             </div>
325             <b>{detalle.hora_cierre || "-"}</b>
326           </div>
327           <div className="kpi">
328             <div className="kpi-cabeza">
329               <span className="icono-kpi">👂</span>
330               <small>Auditó</small>
331             </div>
332             <b>{detalle.revisor || "-"}</b>
333           </div>
334         </div>
335         <p className="pista" style={{ marginTop: 12 }}>
336           Esta bodega ya está cerrada con doble firma. Para volver a contarla hace
337           falta reabrirla desde Bodegas con una justificación escrita.
338         </p>
339       </>
340     )
341   <div className="grilla-botones" style={{ marginTop: 12 }}>
342     <button className="btn gris" onClick={volverALaLista}>Volver a la lista</button>
343   </div>
344 </div>
345 ) : !sesion ? (
346   <div className="card">
347     <h3>{(bodegas || []).find((b) => b.id === bodegaId)?.bodega}</h3>
348     <button className="btn" onClick={() => iniciar(bodegaId)}
349       style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
350       <Icono nombre="auditoria" tam={18} />
351       Iniciar recuento ciego
352     </button>
353     <div className="grilla-botones" style={{ marginTop: 12 }}>
354       <button className="btn gris" onClick={volverALaLista}>Volver a la lista</button>
355     </div>
356   </div>
357 ) : !comparar ? (
358   <div className="card">
359     <h3>{(bodegas || []).find((b) => b.id === bodegaId)?.bodega}</h3>
360     <div className="conteo-cols">

```

```

361     <div>
362     <p className="rotulo">{dicho ? `Usted dijo: «${dicho}»` : "CuentaVoz responde"}</p>
363     <div className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</div>
364     {opciones && (
365       <div className="opciones" style={{ marginTop: 14 }}>
366         {opciones.map((o, i) => (
367           <button key={o.codigo} className="opcion"
368             onClick={() => procesar(o.codigo, o.nombre)}>
369             <span className="n">Opción {i + 1}</span>
370             <h4>{o.nombre}</h4>
371             <p>Código {o.codigo}</p>
372             <p>{o.unidad}</p>
373           </button>
374         ))}
375       </div>
376     )}
377     <div className="grilla-botones" style={{ marginTop: 12 }}>
378       <button className="btn verde" onClick={() => procesar("confirmando")}>Confirmar</button>
379       <button className="btn gris" onClick={() => setMostrarTeclado(true)}>
380         Teclado
381       </button>
382     </div>
383   </div>
384   <div className="mic-caja">
385     <button className={mic-btn ${estado === "escuchando" ? "escuchando" : ""}}
386       onClick={() => escuchar({ alTexto: procesar, alEstado: setEstado })}
387       aria-label="Hable para dictar el artículo y la cantidad del recuento ciego">
388       <Icono nombre="microfono" tam={52} />
389     </button>
390     <small>{vozDisponible() ? "Reconocimiento en español" : "Use el teclado"}</small>
391   </div>
392 </div>
393 <div className="grilla-botones" style={{ marginTop: 12 }}>
394   <button className="btn borde" onClick={verComparar}>Ver comparación</button>
395   <button className="btn gris" onClick={volverALaLista}>Volver a la lista</button>
396 </div>
397 </div>
398 ) : !modoCierre ? (
399   <div className="card">
400     <div className="chips">
401       <span className="chip azul">{comparar.total} referencias</span>
402       <span className="chip oro">{comparar.filas.length} con diferencia</span>
403       <span className="chip verde">{comparar.coinciden} coinciden</span>
404     </div>
405     <p className="pista">
406       Solo se muestran las referencias con diferencia. Su recuento fue a ciegas:
407       el conteo 1 se revela al comparar.
408     </p>
409     {comparar.filas.length === 0 ? (
410       <p className="vacio">Sin diferencias entre el recuento ciego y el sistema.</p>
411     ) : (
412       <table>
413         <thead>
414           <tr><th>Artículo</th><th>Conteo 1</th><th>Su conteo</th>
415             <th>Sistema</th><th>Diferencia</th><th>Acción</th></tr>
416         </thead>
417         <tbody>
418           {comparar.filas.map((f) => (
419             <tr key={f.codigo}>
420               <td>{f.articulo}</td>
421               <td>{f.conteo1 ?? "-"}</td>
422               <td>{f.conteo2 ?? "-"}</td>
423               <td>{f.sistema}</td>
424               <td className="dif">{f.diferencia > 0 ? `+${f.diferencia}` : f.diferencia}</td>
425               <td>{f.accion}</td>
426             </tr>
427           ))}
428         </tbody>
429       </table>
430     )}
431   </div>
432   {comparar.diagnosticos?.length > 0 && (
433     <div style={{ background: "#FAFBFD", border: "1px solid var(--borde)",
434       borderRadius: "var(--radio)", padding: 14, marginTop: 14 }}>
435       <h3 style={{ marginTop: 0 }}>Diagnóstico automático</h3>
436       {comparar.diagnosticos.map((d, i) => (
437         <p key={i} style={{ fontSize: ".88rem", marginTop: i ? 8 : 0 }}>{d}</p>
438       ))}
439     </div>
440   )}
441
442   {mostrarDictado && (
443     <div className="conteo-cols" style={{ marginTop: 16 }}>
444       <div>
445         <p className="rotulo">{dicho ? `Usted dijo: «${dicho}»` : "CuentaVoz responde"}</p>
446         <div className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</div>

```

```

447         {opciones && (
448             <div className="opciones" style={{ marginTop: 14 }}>
449                 {opciones.map((o, i) => (
450                     <button key={o.codigo} className="opcion"
451                         onClick={() => procesar(o.codigo, o.nombre)}>
452                         <span className="n">Opción {i + 1}</span>
453                         <h4>{o.nombre}</h4>
454                         <p>Código {o.codigo}</p>
455                         <p>{o.unidad}</p>
456                     </button>
457                 ))}
458             </div>
459         )}
460         <div className="grilla-botones" style={{ marginTop: 12 }}>
461             <button className="btn verde" onClick={() => procesar("confirmando")}>Confirmar</button>
462             <button className="btn borde" onClick={verComparar}>Actualizar comparación</button>
463             <button className="btn gris" onClick={() => setMostrarTeclado(true)}>Teclado</button>
464         </div>
465     </div>
466     <div className="mic-caja">
467         <button className={`mic-btn ${estado === "escuchando" ? "escuchando" : ""}`}
468             onClick={() => escuchar({ alTexto: procesar, alEstado: setEstado })}
469             aria-label="Hable para dictar el artículo y la cantidad del recuento ciego">
470             <Icono nombre="microfono" tam={52} />
471         </button>
472         <small>{vozDisponible() ? "Reconocimiento en español" : "Use el teclado"}</small>
473     </div>
474 </div>
475 )}
476
477 <div className="grilla-botones" style={{ marginTop: 14 }}>
478     <button className="btn" onClick={() => setMostrarDictado((v) => !v)}>
479         {mostrarDictado ? "Ocultar dictado" : "Resolver por voz"}
480     </button>
481     <button className="btn borde" disabled={firmas?.auditoria?.firmada}
482         onClick={firmarAuditoria}>
483         Aceptar todas
484     </button>
485     <button className="btn verde" onClick={() => setModoCierre(true)}>
486         Cerrar con doble firma
487     </button>
488 </div>
489 <div className="grilla-botones" style={{ marginTop: 8 }}>
490     <button className="btn gris" onClick={volverALaLista}>Volver a la lista</button>
491 </div>
492 </div>
493 ) : (
494     <div className="card">
495         <div className="chips">
496             <span className="chip">{firmas?.referencias ?? comparar.total} referencias</span>
497             <span className="chip verde">{comparar.filas.length} diferencias resueltas</span>
498             <span className="chip verde">{firmas?.alertas_resueltas ?? 0} alertas cerradas</span>
499         </div>
500
501         <div className="conteo-cols" style={{ marginTop: 14 }}>
502             <div className="opcion" style={{
503                 borderColor: firmas?.conteo?.firmada ? "var(--verde)" : "var(--borde)",
504                 background: firmas?.conteo?.firmada ? "var(--verde-bg)" : "#fff",
505             }}>
506                 <span className="n">CONTEO 1 · AUXILIAR</span>
507                 <div style={{ display: "flex", alignItems: "center", gap: 10, marginTop: 6 }}>
508                     <span style={{ width: 34, height: 34, borderRadius: "50%", background: "var(--verde)",
509                         color: "#fff", display: "flex", alignItems: "center", justifyContent: "center",
510                         fontWeight: 700 }}>
511                         {(firmas?.conteo?.persona || "?")[0]?.toUpperCase()}
512                     </span>
513                     <h4 style={{ textTransform: "capitalize", margin: 0 }}>{firmas?.conteo?.persona || "?"}</h4>
514                 </div>
515                 {firmas?.conteo?.firmada ? (
516                     <p style={{ color: "var(--verde)", fontWeight: 700, marginTop: 10 }}>
517                         ✓ Firmado · {firmas?.conteo.hora}
518                     </p>
519                 ) : <p style={{ marginTop: 10 }}>Pendiente de su firma</p>
520                 <p style={{ fontSize: ".82rem", color: "var(--grafito)" }}>
521                     {firmas?.referencias ?? comparar.total} referencias
522                 </p>
523                 <p style={{ fontSize: ".82rem", fontStyle: "italic", color: "var(--grafito)" }}>
524                     Sin acceso al recuento de auditoría
525                 </p>
526             </div>
527             <div className="opcion" style={{
528                 borderColor: firmas?.auditoria?.firmada ? "var(--verde)" : "var(--azul)",
529             }}>
530                 <span className="n">AUDITORÍA · ADMINISTRADORA</span>
531                 <div style={{ display: "flex", alignItems: "center", gap: 10, marginTop: 6 }}>
532                     <span style={{ width: 34, height: 34, borderRadius: "50%", background: "var(--azul)",

```

```

533         color: "#fff", display: "flex", alignItems: "center", justifyContent: "center",
534         fontWeight: 700 }}>
535         {(firmas?.auditoria?.persona || "?")[0]?.toUpperCase()}
536     </span>
537     <h4 style={{ textTransform: "capitalize", margin: 0 }}>{firmas?.auditoria?.persona || "-"}</h4>
538 </div>
539 {firmas?.auditoria?.firmada ? (
540   <p style={{ color: "var(--verde)", fontWeight: 700, marginTop: 10 }}>
541     ✓ Firmado · {firmas.auditoria.hora}
542   </p>
543 ) : (
544   <div>
545     <p style={{ color: "var(--azul)", marginTop: 10 }}>Pendiente de su firma</p>
546     <button className="btn" style={{ marginTop: 8 }} onClick={firmarAuditoria}>
547       Firmar con PIN
548     </button>
549   </div>
550 )}
551 </div>
552 </div>
553
554 <div className="grilla-botones" style={{ marginTop: 16 }}>
555   <button className="btn verde" disabled={!firmas?.lista_para_cerrar}
556     onClick={cerrarDefinitivo} style={{ flex: 1, fontSize: "1.05rem" }}>
557     Cerrar bodega definitivamente
558   </button>
559 </div>
560 <p className="pista" style={{ marginTop: 10, textAlign: "center" }}>
561   Con las dos firmas la bodega pasa a CERRADA en el tablero al instante y sus datos entran al consolidado.
562 </p>
563 <p className="pista" style={{ textAlign: "center", color: "var(--azul)" }}>
564   Ninguna bodega se cierra con una sola firma: es la garantía de trazabilidad del inventario.
565 </p>
566 <div className="grilla-botones" style={{ marginTop: 8 }}>
567   <button className="btn borde" onClick={() => setModoCierre(false)}>
568     Volver a la comparación
569   </button>
570   <button className="btn gris" onClick={volverALaLista}>Volver a la lista</button>
571 </div>
572 </div>
573 )}
574
575 {mostrarTeclado && (
576   <Dialogo titulo="Escribir en vez de hablar"
577     mensaje="Escriba el producto y la cantidad:"
578     conCampo placeholder="tres tablas para picar blancas"
579     onAceptar={(t) => { setMostrarTeclado(false); if (t) procesar(t); }}
580     onCancel={() => setMostrarTeclado(false)} />
581 )}
582 </>
583 );
584 }
585
586 /* — Aprobaciones pendientes — */
587 function TabAprobaciones({ token, esAuditor, onEnFoco }) {
588   const [lista, setLista] = useState(null);
589   const [msg, setMsg] = useState("");
590   const [saludo, setSaludo] = useState("");
591   const yaSaludo = useRef(false);
592
593   function cargar() {
594     pedir("/api/aprobaciones?estado=pendiente", {}, token).then(setLista)
595       .catch((e) => { setLista([]); setMsg(e.message); });
596   }
597   useEffect(cargar, [token]);
598   // El agente ("Pregúntele al agente", arriba de las pestañas) puede
599   // aprobar/rechazar por voz - cuando lo hace, avisa con este evento
600   // para que esta lista refleje el cambio sin recargar.
601   useEffect(() => {
602     window.addEventListener("cuentavoz:auditoria-actualizada", cargar);
603     return () => window.removeEventListener("cuentavoz:auditoria-actualizada", cargar);
604   }, [token]);
605
606   useEffect(() => {
607     onEnFoco({ titulo: "Aprobaciones pendientes",
608       chip: { texto: lista === null ? "..." : `${lista.length} PENDIENTES`, tipo: "borde oro" } });
609   }, [lista]);
610
611   // Al llegar a esta pestaña (por clic o por voz), el agente dice de una
612   // vez si hay algo pendiente y qué es, en vez de esperar a que se lo
613   // pregunten - yaSaludo evita que se repita en cada refresco de la
614   // lista (ej. justo después de aprobar algo por voz).
615   useEffect(() => {
616     if (lista === null || yaSaludo.current) return;
617     yaSaludo.current = true;
618     let texto;

```

```

619   if (lista.length === 0) {
620     texto = "No hay aprobaciones pendientes.";
621   } else if (lista.length === 1) {
622     texto = `Tiene una aprobación pendiente: ${lista[0].nombre.toLowerCase()}. ¿Quiere aprobarla o rechazarla?`;
623   } else {
624     const nombres = lista.slice(0, 5).map((a) => a.nombre.toLowerCase()).join(", ");
625     texto = `Tiene ${lista.length} aprobaciones pendientes: ${nombres}. ¿Cuál quiere aprobar o rechazar?`;
626   }
627   setSaludo(texto);
628   hablar(texto);
629 }, [lista]);
630
631 async function resolver(id, accion) {
632   try {
633     await pedir(`/api/aprobaciones/${id}/${accion}`, { method: "POST" }, token);
634     setMsg(accion === "aprobar" ? "Aprobado y agregado al catálogo oficial." : "Rechazado.");
635     cargar();
636   } catch (e) { setMsg(e.message); }
637 }
638
639 if (!esAuditor) return null;
640
641 return (
642   <div className="card">
643     <p className="pista" style={{ marginTop: 0 }}>
644       Productos y bodegas creados durante la toma que esperan su firma para entrar al catálogo oficial.
645     </p>
646     {saludo && (
647       <div>
648         <p className="rotulo">CuentaVoz responde</p>
649         <p className="burbuja" style={{ marginBottom: 12 }}>
650           aria-live="polite" aria-atomic="true">{saludo}</p>
651       </div>
652     )}
653     {msg && <p className="msg-ok">{msg}</p>}
654     {lista === null ? (
655       <p className="cargando">Cargando...</p>
656     ) : lista.length === 0 ? (
657       <p className="vacio">Nada pendiente de aprobación.</p>
658     ) : (
659       lista.map((a) => (
660         <div key={a.id} style={{ display: "flex", alignItems: "flex-start", gap: 14,
661           flexWrap: "wrap",
662           border: "1px solid var(--borde)", borderRadius: 10,
663           padding: "12px 14px", marginBottom: 10 }}>
664           <span style={{ flex: 1, minWidth: 200 }}>
665             <b style={{ color: "var(--azul)" }}>{a.nombre}</b>
666             <br />
667             <span style={{ fontSize: ".85rem" }}>
668               {a.tipo === "producto"
669                 ? `Producto nuevo · ${a.unidad_medida} || ""`
670                 : "Bodega nueva"}
671             </span>
672             <br />
673             <small style={{ color: "var(--grafito)", fontStyle: "italic" }}>
674               {a.tipo === "producto"
675                 ? `${a.cantidad_inicial} ${a.unidad_medida} contados en ${a.bodega}`
676                 : "sin referencias asignadas todavía"}
677             </small>
678             <br />
679             <small style={{ color: "var(--grafito)", textTransform: "capitalize" }}>
680               {a.creado_por} · {a.hora}
681             </small>
682           </span>
683           <span style={{ display: "flex", gap: 8 }}>
684             <button className="btn gris" onClick={() => resolver(a.id, "rechazar")}
685               style={{ display: "inline-flex", alignItems: "center", gap: 6 }}>
686               <Icono nombre="rechazar" tam={16} />
687               Rechazar
688             </button>
689             <button className="btn verde" onClick={() => resolver(a.id, "aprobar")}
690               style={{ display: "inline-flex", alignItems: "center", gap: 6 }}>
691               <Icono nombre="aprobar" tam={16} />
692               Aprobar
693             </button>
694           </span>
695         </div>
696       ))
697     )}
698     <p className="pista" style={{ marginTop: 10 }}>
699       Al aprobar, el producto entra al catálogo con su código definitivo y el
700       conteo asociado deja de estar marcado. Mientras tanto el conteo nunca se
701       detiene: la aprobación ocurre en paralelo al trabajo en bodega.
702     </p>
703   </div>
704 );

```

```

705 }
706
707 /* — Pedidos de auxiliares esperando aprobación antes de salir del almacén — */
708 function TabPedidosPendientes({ token, esAuditor, onEnFoco }) {
709   const [lista, setLista] = useState(null);
710   const [msg, setMsg] = useState("");
711   const [saludo, setSaludo] = useState("");
712   const yaSaludo = useRef(false);
713
714   function cargar() {
715     pedir("/api/pedidos/pendientes", {}, token).then(setLista)
716       .catch((e) => { setLista([]); setMsg(e.message); });
717   }
718   useEffect(cargar, [token]);
719   useEffect(() => {
720     window.addEventListener("cuentavoz:auditoria-actualizada", cargar);
721     return () => window.removeEventListener("cuentavoz:auditoria-actualizada", cargar);
722   }, [token]);
723
724   useEffect(() => {
725     onEnFoco({ titulo: "Pedidos pendientes de aprobación",
726       chip: { texto: lista === null ? "..." : `${lista.length} PENDIENTES`, tipo: "borde oro" } });
727   }, [lista]);
728
729   useEffect(() => {
730     if (lista === null || yaSaludo.current) return;
731     yaSaludo.current = true;
732     let texto;
733     if (lista.length === 0) {
734       texto = "No hay pedidos pendientes.";
735     } else if (lista.length === 1) {
736       texto = `Tiene un pedido pendiente: ${lista[0].plato.toLowerCase()}, de `
737         + `${lista[0].persona}. ¿Quiere aprobarlo o rechazarlo?`;
738     } else {
739       const nombres = lista.slice(0, 5)
740         .map((p) => `${p.plato.toLowerCase()} de ${p.persona}`).join(", ");
741       texto = `Tiene ${lista.length} pedidos pendientes: ${nombres}. ¿Cuál quiere aprobar o rechazar?`;
742     }
743     setSaludo(texto);
744     hablar(texto);
745   }, [lista]);
746
747   async function resolver(numero, aprobar) {
748     try {
749       await pedir(`/api/pedidos/${numero}/resolver`, {
750         method: "POST", body: JSON.stringify({ aprobar }),
751       }, token);
752       setMsg(aprobar ? "Pedido aprobado: ya puede salir del almacén."
753         : "Pedido rechazado.");
754       cargar();
755     } catch (e) { setMsg(e.message); }
756   }
757
758   if (!esAuditor) return null;
759
760   return (
761     <div className="card">
762       <p className="pista" style={{ marginTop: 0 }}>
763         El auxiliar pide y queda registrado de una vez, pero solo sale del almacén
764         cuando usted lo aprueba aquí - igual que con productos y bodegas nuevas.
765       </p>
766       {saludo && (
767         <>
768           <p className="rotulo">CuentaVoz responde</p>
769           <p className="burbuja" style={{ marginBottom: 12 }}
770             aria-live="polite" aria-atomic="true">{saludo}</p>
771         </>
772       )}
773       {msg && <p className="msg-ok">{msg}</p>}
774       {lista === null ? (
775         <p className="cargando">Cargando...</p>
776       ) : lista.length === 0 ? (
777         <p className="vacio">Nada pendiente de aprobación.</p>
778       ) : (
779         lista.map((p) => (
780           <div key={p.numero_pedido} style={{ display: "flex", alignItems: "flex-start", gap: 14,
781             flexWrap: "wrap",
782             border: "1px solid var(--borde)", borderRadius: 10,
783             padding: "12px 14px", marginBottom: 10 }}>
784             <span style={{ flex: 1, minWidth: 200 }}>
785               <b style={{ color: "var(--azul)" }}>{p.plato}</b> · {p.porciones} porciones
786               <span className="chip" style={{ marginLeft: 8 }}>{p.numero_pedido}</span>
787             <br />
788             <small style={{ color: "var(--grafito)" }}>
789               Se descontaría de: {p.bodega || ""}
790             </small>
791           </div>
792         ))
793       )}
794     </div>
795   );

```

```

791     <br />
792     <small style={{ color: "var(--grafito)", textTransform: "capitalize" }}>
793       Pedido por {p.persona} · {p.hora}
794     </small>
795     <ul style={{ margin: "8px 0 0", paddingLeft: 18, fontSize: ".85rem" }}>
796       {p.items.map((it, i) => (
797         <li key={i}>{it.nombre}: {it.cantidad}</li>
798       ))}
799     </ul>
800   </span>
801   <span style={{ display: "flex", gap: 8, flex: "none" }}>
802     <button className="btn gris" onClick={() => resolver(p.numero_pedido, false)}
803       style={{ display: "inline-flex", alignItems: "center", gap: 6 }}>
804       <Icono nombre="rechazar" tam={16} />
805       Rechazar
806     </button>
807     <button className="btn verde" onClick={() => resolver(p.numero_pedido, true)}
808       style={{ display: "inline-flex", alignItems: "center", gap: 6 }}>
809       <Icono nombre="aprobar" tam={16} />
810       Aprobar
811     </button>
812   </span>
813 </div>
814 ))
815 })
816 <p className="pista" style={{ marginTop: 10 }}>
817   Al aprobar, el pedido queda abierto para su legalización al final del servicio.
818   Al rechazar, no cuenta como enviado y el auxiliar debe volver a intentarlo.
819 </p>
820 </div>
821 );
822 }
823
824 /* — Bandeja de alertas — */
825 const COLOR_ALERTA = { desviacion: "oro", negativo: "oro", unidad: "azul", inexistente: "azul" };
826
827 function TabAlertas({ token, esAuditor, onEnFoco }) {
828   const [alertas, setAlertas] = useState(null);
829   const [resumen, setResumen] = useState(null);
830   const [verDetalle, setVerDetalle] = useState(null);
831   const [msg, setMsg] = useState("");
832   const [saludo, setSaludo] = useState("");
833   const yaSaludo = useRef(false);
834
835   function cargar() {
836     pedir("/api/alertas?resueltas=0", {}, token).then(setAlertas)
837       .catch((e) => { setAlertas([]); setMsg(e.message); });
838     pedir("/api/alertas/resumen", {}, token).then(setResumen).catch(() => {});
839   }
840   useEffect(cargar, [token]);
841   useEffect(() => {
842     window.addEventListener("cuentavoz:auditoria-actualizada", cargar);
843     return () => window.removeEventListener("cuentavoz:auditoria-actualizada", cargar);
844   }, [token]);
845
846   useEffect(() => {
847     onEnFoco({ titulo: "Auditoría · Bandeja de alertas",
848       chip: { texto: alertas === null ? "..." : `${alertas.length} ABIERTAS`, tipo: "borde oro" } });
849   }, [alertas]);
850
851   useEffect(() => {
852     if (alertas === null || yaSaludo.current) return;
853     yaSaludo.current = true;
854     const etiqueta = (a) => a.articulo || a.bodega || a.titulo.toLowerCase();
855     let texto;
856     if (alertas.length === 0) {
857       texto = "No hay alertas abiertas.";
858     } else if (alertas.length === 1) {
859       texto = `Tiene una alerta abierta: ${etiqueta(alertas[0]).toLowerCase()}.`
860         + (esAuditor ? " ¿Quiere resolverla?" : "");
861     } else {
862       const nombres = alertas.slice(0, 5).map((a) => etiqueta(a).toLowerCase()).join(", ");
863       texto = `Tiene ${alertas.length} alertas abiertas: ${nombres}.`
864         + (esAuditor ? " ¿Cuál quiere resolver?" : "");
865     }
866     setSaludo(texto);
867     hablar(texto);
868   }, [alertas, esAuditor]);
869
870   async function resolver(id) {
871     try {
872       await pedir(`/api/alertas/${id}/resolver`, { method: "POST" }, token);
873       setMsg("Alerta resuelta.");
874       cargar();
875     } catch (e) { setMsg(e.message); }
876   }

```

```

877
878 return (
879   <div className="card">
880     <div className="chips">
881       <span className="chip oro">{resumen?.abiertas ?? (alertas || []).length} abiertas</span>
882       <span className="chip verde">{resumen?.resueltas_hoy ?? 0} resueltas hoy</span>
883       <span className="chip">
884         Tiempo medio: {resumen?.tiempo_medio_min != null ? `${resumen.tiempo_medio_min} min` : ""}
885       </span>
886     </div>
887     {saludo && (
888       <
889         <p className="rotulo">CuentaVoz responde</p>
890         <p className="burbuja" style={{ marginBottom: 12 }}
891           aria-live="polite" aria-atomic="true">{saludo}</p>
892       </>
893     )}
894     {msg && <p className="msg-ok">{msg}</p>}
895     {alertas === null ? (
896       <p className="cargando">Cargando...</p>
897     ) : alertas.length === 0 ? (
898       <p className="vacio">No hay alertas abiertas.</p>
899     ) : (
900       alertas.map((a) => (
901         <div key={a.id} style={{
902           display: "flex", alignItems: "center", gap: 14, marginTop: 10, flexWrap: "wrap",
903           border: "1px solid var(--borde)",
904           borderLeft: `4px solid ${COLOR_ALERTA[a.tipo] === "azul" ? "var(--azul)" : "var(--amarillo)"}`,
905           borderRadius: 10, padding: "12px 14px",
906         }}>
907           <span style={{ flex: 1, minWidth: 200 }}>
908             <b style={{ color: COLOR_ALERTA[a.tipo] === "azul" ? "var(--azul)" : "var(--amarillo-tx)" }}>
909               {a.titulo}
910             </b>
911             <br />
912             <span style={{ fontSize: ".88rem" }}>
913               {a.articulo || "-"}{a.bodega ? ` · ${a.bodega}` : ""}
914             </span>
915             <br />
916             <small style={{ color: "var(--grafito)", textTransform: "capitalize" }}>
917               {a.persona || "-"} · {a.hora}
918             </small>
919             <br />
920             <small style={{ color: "var(--grafito)", fontStyle: "italic" }}>{a.detalle}</small>
921           </span>
922           <span style={{ display: "flex", gap: 8, flex: "none" }}>
923             <button className="btn borde" onClick={() => setVerDetalle(a)}>Ver</button>
924             {esAuditor && (
925               <button className="btn verde" onClick={() => resolver(a.id)}
926                 style={{ display: "inline-flex", alignItems: "center", gap: 6 }}>
927                 <Icono nombre="aprobar" tam={16} />
928                 Resolver
929               </button>
930             )}
931           </span>
932         </div>
933       ))
934     )}
935     <p className="pista" style={{ marginTop: 12, fontStyle: "italic" }}>
936       Cada alerta guarda quién la generó, con qué dato y cómo se resolvió: es la trazabilidad
937       que exige una auditoría de inventario.
938     </p>
939
940     {verDetalle && (
941       <Dialogo titulo={verDetalle.titulo}
942         mensaje={` ${verDetalle.articulo || ""} ${verDetalle.bodega ? ` · ${verDetalle.bodega}` : ""}\n` +
943           ` ${verDetalle.persona || "-"} · ${verDetalle.hora}\n\n ${verDetalle.detalle}` }
944         textoAceptar="Cerrar" onAceptar={() => setVerDetalle(null)}
945         onCancel={() => setVerDetalle(null)} />
946     )}
947   </div>
948 );
949 }

```

frontend/src/vistas/Reportes.jsx

453 LÍNEAS

La salida hacia My Inventory.

```

1 import { useEffect, useState } from "react";
2 import { pedir, descargarReporte } from "../api";
3 import { hablar, esAfirmacion, esNegacion } from "../voz";
4 import Marco from "../Marco";
5 import AsistenteVoz from "../AsistenteVoz";

```

```

6 import Icono from "../Iconos";
7
8 const fmt = (n) => Number(n ?? 0).toLocaleString("es-CO", { maximumFractionDigits: 2 });
9
10 // "oro" es el modificador de clase (.chip.oro); la variable CSS real es
11 // --amarillo, no --oro - de ahí los dos mapas separados.
12 const CLASE_TITULO = {
13   "Consolidado para My Inventory": "verde",
14   "Diferencias por bodega": "oro",
15   "Detalle de bodega": "azul",
16   "Estado del tablero": "azul",
17   "Análisis de consumo": "azul",
18   "Registro de trazabilidad": "azul",
19 };
20 const VAR_TITULO = {
21   "Consolidado para My Inventory": "verde",
22   "Diferencias por bodega": "amarillo",
23   "Detalle de bodega": "azul",
24   "Estado del tablero": "azul",
25   "Análisis de consumo": "azul",
26   "Registro de trazabilidad": "azul",
27 };
28 const ICONO_TITULO = {
29   "Consolidado para My Inventory": "🏠",
30   "Diferencias por bodega": "📊",
31   "Detalle de bodega": "📋",
32   "Estado del tablero": "📅",
33   "Análisis de consumo": "📈",
34   "Registro de trazabilidad": "🕒",
35 };
36
37 // Ejemplos de lo que ya entiende el agente en esta pantalla: generar los
38 // tres tipos de archivo, ver el contenido de uno ya generado, y preguntar
39 // por lo que muestra el análisis de consumo.
40 const EJEMPLOS_REPORTES = [
41   "genera el consolidado", "exporta las diferencias", "exporta el análisis de consumo",
42   "muéstrame las diferencias", "muéstrame el estado del tablero", "muéstrame la trazabilidad",
43   "muéstrame el detalle de bodega", "muéstrame el archivo del consolidado",
44   "consolidado de la toma", "análisis de consumo",
45   "¿cuántas filas tiene el último consolidado?", "¿cuántas bodegas tienen descuadre?",
46   "¿qué archivos se han generado?", "¿cuánto se pidió en el período?",
47   "¿cuánto se usó realmente?", "¿cuál es el ahorro potencial?",
48   "¿cuántos insumos están subutilizados?", "¿cuál insumo tiene más sobrepedido?",
49 ];
50
51 export default function Reportes({ token, usuario, ir, tabInicial, navSeq, archivoPrevisualizar }) {
52   const [tab, setTab] = useState(tabInicial || "consolidado");
53   // Pedir una pestaña de esta MISMA pantalla por voz no remonta el
54   // componente - sin esto, el useState de arriba no vuelve a leer
55   // tabInicial y la pestaña se queda en la que ya estaba. navSeq (sube en
56   // cada ir()) va en la dependencia también: sin él, pedir la MISMA
57   // pestaña dos veces seguidas (con un clic manual a otra en el medio)
58   // no hacía nada, porque tabInicial no cambiaba de valor.
59   useEffect(() => { if (tabInicial) setTab(tabInicial); }, [tabInicial, navSeq]);
60
61   return (
62     <Marco titulo={tab === "consolidado" ? "Reportes · consolidado de la toma"
63       : "Reportes · análisis de consumo"}
64       chip={tab === "consolidado" ? { texto: "ARCHIVOS", tipo: "borde azul" }
65         : { texto: "ÚLTIMOS 30 DÍAS", tipo: "borde azul" }}>
66       <AsistenteVoz token={token} vista="reportes" ir={ir} ejemplos={EJEMPLOS_REPORTES}
67         placeholder="¿cuántas filas tiene el último consolidado?, lléveme al panel..."
68         alActualizar={() => window.dispatchEvent(new Event("cuentavoz:reportes-actualizados"))}> />
69       <div className="chips">
70         <button className={`chip ${tab === "consolidado" ? "azul" : ""}`}
71           onClick={() => setTab("consolidado")}>Consolidado de la toma</button>
72         <button className={`chip ${tab === "analisis" ? "azul" : ""}`}
73           onClick={() => setTab("analisis")}>Análisis de consumo</button>
74       </div>
75
76       {tab === "consolidado"
77         ? <TabConsolidado token={token} usuario={usuario}
78           navSeq={navSeq} archivoPrevisualizar={archivoPrevisualizar} />
79         : <TabAnalisis token={token} usuario={usuario} />}
80     </Marco>
81   );
82 }
83
84 /** Tabla genérica: cada tipo de reporte (consolidado, diferencias, estado,
85     detalle de bodega) trae sus propias columnas, así que en vez de mapear
86     columnas fijas se dibujan las que de verdad trae el archivo. */
87 function TablaVistaPrevia({ filas }) {
88   if (!filas || filas.length === 0) return <p className="vacio">Este archivo no tiene filas.</p>;
89   const columnas = Object.keys(filas[0]);
90   return (
91     <div className="tabla-scroll">

```

```

92     <table>
93     <thead><tr>{columnas.map((c) => <th key={c}>{c}</th>)}</tr></thead>
94     <tbody>
95     {filas.map((f, i) => (
96       <tr key={i}>
97         {columnas.map((c) => (
98           <td key={c} className={c === "Diferencia" ? "dif" : ""}>
99             {c === "Diferencia" && f[c] > 0 ? `+${f[c]}` : String(f[c])}
100           </td>
101         )})}
102       </tr>
103     )})}
104   </tbody>
105 </table>
106 </div>
107 );
108 }
109
110 /* — Consolidado de la toma — */
111 function TabConsolidado({ token, navSeq, archivoPrevisualizar }) {
112   const [recientes, setRecientes] = useState(null);
113   const [archivoActivo, setArchivoActivo] = useState(null); // {archivo, titulo}
114   const [filasPrevia, setFilasPrevia] = useState(null);
115   const [totalFilas, setTotalFilas] = useState(null);
116   const [err, setErr] = useState("");
117   const [msg, setMsg] = useState("");
118   // Se activa con cualquier vista previa (clic manual o por voz) y con
119   // cada archivo recién generado - así un "sí"/"no" hablado después
120   // sirve para descargarlo sin importar cómo se llegó hasta ahí. Ver
121   // previsualizar().
122   const [pidiendoDescarga, setPidiendoDescarga] = useState(false);
123
124   function cargar() {
125     pedir("/api/reportes/recientes", {}, token).then(setRecientes).catch(() => setRecientes([]));
126   }
127   useEffect(cargar, [token]);
128   // Generar un archivo por voz pasa por el backend directo, no por
129   // generarConsolidado/generarDiferencias de aquí abajo - sin escuchar
130   // este evento la lista de "Archivos generados" se quedaba vieja hasta
131   // que se recargaba la página a mano.
132   useEffect(() => {
133     window.addEventListener("cuentavoz:reportes-actualizados", cargar);
134     return () => window.removeEventListener("cuentavoz:reportes-actualizados", cargar);
135   }, [token]);
136
137   // anunciar: true en un clic manual a una tarjeta - sin voz de por medio,
138   // alguien con problemas de vista no tiene otra forma de saber qué
139   // encontró la vista previa (antes solo se narraba cuando el pedido
140   // llegaba por voz, porque ese camino ya trae su propia respuesta
141   // hablada del backend - un clic de mouse no). false cuando el pedido
142   // YA llegó por voz (ver el useEffect de abajo), para no narrar dos
143   // veces lo mismo que el agente acaba de decir.
144   async function previsualizar(archivo, titulo, { anunciar = false } = {}) {
145     setErr("");
146     try {
147       const d = await pedir(`/api/reportes/vista-previa?archivo=${encodeURIComponent(archivo)}`, {}, token);
148       setArchivoActivo({ archivo, titulo });
149       setFilasPrevia(d.filas);
150       setTotalFilas(d.total);
151       setPidiendoDescarga(true);
152       if (anunciar) {
153         const resumen = `Vista previa de ${titulo}, ${d.total} `
154           + `${d.total === 1 ? "fila" : "filas"}. ¿Desea descargarlo?`;
155         setMsg(resumen);
156         hablar(resumen);
157       }
158     } catch (e) { setErr(e.message); }
159   }
160   // "Muéstrame el archivo de diferencias" - lo mismo que darle clic a la
161   // tarjeta, pero pedido por voz; el backend ya resolvió CUÁL archivo es.
162   // Va por prop (como bodegaSugerida en Conteo), no por evento: si el
163   // pedido llegó desde OTRA pestaña de Reportes, este componente todavía
164   // no estaba montado cuando AsistenteVoz disparaba el evento viejo, así
165   // que la vista previa nunca aparecía. navSeq en las dependencias por la
166   // misma razón que en el resto de la app: pedir el MISMO archivo dos
167   // veces seguidas (con un clic manual a otro en el medio) no cambia el
168   // valor de archivoPrevisualizar por sí solo.
169   useEffect(() => {
170     if (archivoPrevisualizar?.archivo) {
171       previsualizar(archivoPrevisualizar.archivo, archivoPrevisualizar.titulo);
172     }
173     // eslint-disable-next-line react-hooks/exhaustive-deps
174   }, [archivoPrevisualizar, navSeq]);
175
176   // "Sí"/"no" después de que el agente preguntó "¿desea descargarlo?" -
177   // intercepta ANTES de que AsistenteVoz le pregunte al agente general

```

```

178 // (que no tiene ni idea de esa pregunta pendiente). Mismo patrón que el
179 // filtro local de Registro de trazabilidad en Ajustes.
180 useEffect(() => {
181   function interceptar(e) {
182     if (!pidiendoDescarga) return;
183     const texto = e.detail.texto;
184     if (esAfirmacion(texto)) {
185       e.preventDefault();
186       setPidiendoDescarga(false);
187       descargarReporte(archivoActivo.archivo, token).catch((err2) => setErr(err2.message));
188       const msg2 = "Descargando el archivo.";
189       setMsg(msg2);
190       hablar(msg2);
191     } else if (esNegacion(texto)) {
192       e.preventDefault();
193       setPidiendoDescarga(false);
194       const msg2 = "Listo, queda pendiente. Dígame si desea ver otro archivo.";
195       setMsg(msg2);
196       hablar(msg2);
197     }
198   }
199   window.addEventListener("cuentavoz:filtro-local", interceptar);
200   return () => window.removeEventListener("cuentavoz:filtro-local", interceptar);
201 }, [pidiendoDescarga, archivoActivo, token]);
202
203 async function generarConsolidado(formato) {
204   setErr(""); setMsg("");
205   try {
206     const d = await pedir(`/api/reportes?formato=${formato}`, { method: "POST" }, token);
207     setArchivoActivo({ archivo: d.archivo, titulo: "Consolidado para My Inventory" });
208     setFilasPrevia(d.vista_previa || null);
209     setTotalFilas(d.filas);
210     setPidiendoDescarga(true);
211     const resumen = `Consolidado generado: ${d.filas} filas. ¿Desea descargarlo?`;
212     setMsg(resumen);
213     hablar(resumen);
214     cargar();
215   } catch (e) { setErr(e.message); }
216 }
217
218 async function generarDiferencias(formato) {
219   setErr(""); setMsg("");
220   try {
221     const d = await pedir(`/api/reportes/diferencias?formato=${formato}`, { method: "POST" }, token);
222     const resumen = `Diferencias exportadas: ${d.filas} filas en ${d.bodegas_con_descuadre} `
223       + "bodegas. ¿Desea descargarlo?";
224     setMsg(resumen);
225     hablar(resumen);
226     cargar();
227     await previsualizar(d.archivo, "Diferencias por bodega");
228   } catch (e) { setErr(e.message); }
229 }
230
231 return (
232   <div className="reportes-cols">
233     <div className="card">
234       <h3>Archivos generados con los códigos oficiales de la base</h3>
235       /* Arriba, junto al título, y no al final de la lista - con muchos
236         archivos generados en el día, quedaban fuera de la vista sin
237         bajar todo el scroll, y nada indicaba que seguían ahí. */
238       <div className="grilla-botones" style={{ marginTop: 0, marginBottom: 14 }}>
239         <button className="btn" onClick={() => generarConsolidado("xlsx")}
240           style={{ display: "inline-flex", alignItems: "center", gap: 6 }}>
241           <Icono nombre="reportes" tam={16} />
242           Generar consolidado
243         </button>
244         <button className="btn" onClick={() => generarDiferencias("xlsx")}
245           style={{ display: "inline-flex", alignItems: "center", gap: 6 }}>
246           <Icono nombre="reportes" tam={16} />
247           Generar diferencias
248         </button>
249         <button className="btn gris" onClick={() => window.print()}>Imprimir</button>
250       </div>
251       {msg && <p className="msg-ok">{msg}</p>}
252       {err && <p className="error">{err}</p>}
253       {recientes === null ? (
254         <p className="cargando">Cargando...</p>
255       ) : recientes.length === 0 ? (
256         <p className="vacio">Todavía no se ha generado ningún archivo.</p>
257       ) : (
258         recientes.map((a, i) => {
259           const clase = CLASE_TITULO[a.titulo] || "azul";
260           const colorVar = clase === "oro" ? "var(--amarillo-tx)" : `var(--${VAR_TITULO[a.titulo]} || "azul")`;
261           const activo = archivoActivo?.archivo === a.archivo;
262           return (
263             <button key={i} onClick={() => previsualizar(a.archivo, a.titulo, { anunciar: true })}
264               style={{

```

```

264     display: "flex", justifyContent: "space-between", alignItems: "center",
265     width: "100%", textAlign: "left", font: "inherit",
266     borderTop: activo ? "1px solid var(--azul)" : "1px solid var(--borde)",
267     borderRight: activo ? "1px solid var(--azul)" : "1px solid var(--borde)",
268     borderBottom: activo ? "1px solid var(--azul)" : "1px solid var(--borde)",
269     borderLeft: `4px solid var(--${VAR_TITULO[a.titulo]} || "azul")`,
270     borderRadius: 10, padding: "12px 14px", marginBottom: 10,
271     cursor: "pointer", background: activo ? "var(--azul-claro)" : "transparent",
272   }}>
273   <span style={{ display: "flex", alignItems: "center", gap: 12 }}>
274     <span className="icono-kpi" style={{ fontSize: "1.1rem" }}>
275       {ICONO_TITULO[a.titulo] || "📊"}
276     </span>
277     <span>
278       <b style={{ color: colorVar }}>{a.titulo}</b>
279       <br />
280       <small style={{ color: "var(--grafito)" }}>
281         {a.formato} · hoy {a.hora}{a.filas != null ? ` · ${a.filas} filas` : ""}
282       </small>
283     </span>
284   </span>
285   <span className={`chip ${clase}`}>{a.subtitulo}</span>
286 </button>
287 );
288 }}
289 }}
290 <p className="pista" style={{ marginTop: 10 }}>
291   Dele clic a cualquier tarjeta para ver su contenido aquí al lado.
292 </p>
293 </div>
294
295 <div className="card">
296   <h3>{archivoActivo ? `Vista previa · ${archivoActivo.titulo}` : "Vista previa del consolidado"}</h3>
297   {filasPrevias === null ? (
298     <p className="vacio">Genere un archivo o dele clic a uno ya generado para ver una muestra aquí.</p>
299   ) : (
300     <>
301       <TablaVistaPrevias filas={filasPrevias} />
302       <p className="pista" style={{ marginTop: 10 }}>
303         {totalFilas != null && `Mostrando ${Math.min(8, totalFilas)} de ${totalFilas} filas.`}
304         Los códigos SIN-#### corresponden a registros del extracto que llegaron sin número de artículo.
305       </p>
306       <div className="grilla-botones">
307         <button className="btn verde"
308           onClick={() => descargarReporte(archivoActivo.archivo, token).catch((e) => setErr(e.message))}
309           style={{ display: "inline-flex", alignItems: "center", gap: 8, width: "100%",
310             justifyContent: "center" }}>
311           <Icono nombre="descargar" tam={18} />
312           Descargar este archivo
313         </button>
314       </div>
315     </>
316   )}
317 </div>
318 </div>
319 );
320 }
321
322 /* — Análisis de consumo — */
323 function TabAnálisis({ token }) {
324   const [analisis, setAnalisis] = useState(null);
325   const [err, setErr] = useState("");
326   const [msg, setMsg] = useState("");
327
328   function cargar() {
329     pedir("/api/analisis/consumo?dias=30", {}, token).then(setAnalisis).catch((e) => setErr(e.message));
330   }
331   useEffect(cargar, [token]);
332   useEffect(() => {
333     window.addEventListener("cuentavoz:reportes-actualizados", cargar);
334     return () => window.removeEventListener("cuentavoz:reportes-actualizados", cargar);
335   }, [token]);
336
337   async function exportar() {
338     setErr(""); setMsg("");
339     try {
340       const d = await pedir("/api/analisis/exportar?formato=xlsx&dias=30", { method: "POST" }, token);
341       await descargarReporte(d.archivo, token);
342       setMsg("Análisis exportado y descargado.");
343     } catch (e) { setErr(e.message); }
344   }
345   // "Exporta el análisis de consumo" por voz ya creó el archivo en el
346   // servidor (el backend respondió con accion:"descargar_reporte" y el
347   // nombre del archivo) - falta el mismo paso que hace el botón: bajarlo
348   // de verdad al dispositivo.
349   useEffect(() => {

```

```

350   async function alDescargarPorVoz(e) {
351     if (!e.detail?.archivo) return;
352     setErr(""); setMsg("");
353     try {
354       await descargarReporte(e.detail.archivo, token);
355       setMsg("Análisis exportado y descargado.");
356     } catch (err) { setErr(err.message); }
357   }
358   window.addEventListener("cuentavoz:accion:descargar_reporte", alDescargarPorVoz);
359   return () => window.removeEventListener("cuentavoz:accion:descargar_reporte", alDescargarPorVoz);
360 }, [token]);
361
362 if (err) return <p className="error">{err}</p>;
363 if (!analisis) return <p className="cargando">Cargando...</p>;
364
365 const recomendacion = analisis.subutilizados[0];
366 const maxSobrepedido = Math.max(1, ...analisis.subutilizados.map((s) => s.sobrepedido_pct));
367
368 return (
369   <>
370     {msg && <p className="msg-ok">{msg}</p>}
371     <div className="kpis">
372       <div className="kpi">
373         <div className="kpi-cabeza"><span className="icono-kpi">📦</span><small>Pedido en el período</small></div>
374         <b>{fmt(analisis.pedido_total)} kg</b>
375         <i>en {analisis.servicios_periodo} servicios</i>
376       </div>
377       <div className="kpi">
378         <div className="kpi-cabeza"><span className="icono-kpi">👤</span><small>Usado realmente</small></div>
379         <b>{fmt(analisis.usado_total)} kg</b>
380         <i>{analisis.aprovechamiento} % de lo pedido</i>
381       </div>
382       <div className="kpi oro">
383         <div className="kpi-cabeza"><span className="icono-kpi">⚠️</span><small>Insumos subutilizados</small></div>
384         <b>{analisis.subutilizados.length}</b>
385         <i>se piden y no se usan</i>
386       </div>
387       <div className="kpi verde">
388         <div className="kpi-cabeza"><span className="icono-kpi">💡</span><small>Ahorro potencial</small></div>
389         <b>{fmt(analisis.ahorro_potencial)} kg</b>
390         <i>al mes si se ajusta</i>
391       </div>
392     </div>
393
394     {analisis.subutilizados.length === 0 ? (
395       <div className="card">
396         <p className="vacio">
397           Sin servicios legalizados en los últimos {analisis.dias} días: legalice uno para ver el análisis.
398         </p>
399       </div>
400     ) : (
401       <div className="reportes-cols">
402         <div className="card">
403           <h3>📋 Sobrepedido frente a la receta</h3>
404           <div style={{ display: "flex", flexDirection: "column", gap: 10 }}>
405             {analisis.subutilizados.slice(0, 6).map((s) => (
406               <div key={s.nombre} style={{ display: "flex", alignItems: "center", gap: 10 }}>
407                 <span style={{ width: 130, fontSize: ".83rem" }}>{s.nombre}</span>
408                 <div style={{ flex: 1, background: "var(--fondo)", borderRadius: 6, height: 16 }}>
409                   <div style={{ width: `${s.sobrepedido_pct / maxSobrepedido * 100}%`, height: 16,
410                     background: s.sobrepedido_pct >= 30 ? "var(--amarillo)" : "var(--azul)",
411                     borderRadius: 6 }} />
412                 </div>
413                 <b style={{ width: 34, textAlign: "right", fontSize: ".83rem" }}>{s.sobrepedido_pct}</b>
414               </div>
415             ))}
416           </div>
417           <p className="pista" style={{ marginTop: 10 }}>% por encima de lo que pide la receta.</p>
418         </div>
419
420         <div className="card">
421           <h3>📦 Insumos subutilizados: se piden y sobran</h3>
422           {analisis.subutilizados.slice(0, 6).map((s) => (
423             <div key={s.nombre} className="registro">
424               <span style={{ textTransform: "capitalize" }}>{s.nombre.toLowerCase()}</span>
425               <span className="cant">{fmt(s.sobra)} kg sin usar</span>
426               <span className="chip oro">{s.veces} de {analisis.servicios_periodo} servicios</span>
427             </div>
428           ))}
429         </div>
430       </div>
431     )}
432
433     {recomendacion && (
434       <div className="card">
435         <h3>🤖 Recomendación automática del agente</h3>

```

```

436     <p className="burbuja">
437         {recomendacion.nombre.toLowerCase()} sobra en {recomendacion.veces} de los servicios
438         legalizados: la receta pide {recomendacion.sobrepedido_pct}% más de lo que en promedio
439         se usa. Ajustar la receta liberaría {fmt(recomendacion.sobra)} kg al mes. La decisión
440         es del chef: el agente solo muestra el patrón.
441     </p>
442     <div className="grilla-botones">
443         <button className="btn verde" onClick={exportar}
444             style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
445             <Icono nombre="descargar" tam={18} />
446             Exportar análisis
447         </button>
448     </div>
449 </div>
450 }}
451 </>
452 );
453 }

```

frontend/src/vistas/Panel.jsx

396 LÍNEAS

La vista gerencial.

```

1  import { useEffect, useRef, useState } from "react";
2  import { pedir } from "../api";
3  import { hablar } from "../voz";
4  import Marco from "../Marco";
5  import AsistenteVoz from "../AsistenteVoz";
6
7  const SERIE = ["var(--serie-1)", "var(--serie-2)", "var(--serie-3)", "var(--serie-4)"];
8  const COLOR_UNIDAD = { Unidad: "var(--azul)", Kilogram: "var(--amarillo)",
9                        Liter: "var(--verde)", Portion: "#C3CAD6" };
10 const ESTADO_COLOR = { cerrada: "var(--verde)", en_conteo: "var(--azul)",
11                       en_auditoria: "var(--amarillo)", pendiente: "#C3CAD6" };
12 const ESTADO_ETQ = { cerrada: "Cerrada", en_conteo: "Conteo",
13                    en_auditoria: "En auditoría", pendiente: "Pendiente" };
14
15 // Nombre oficial de bodega (como en el catálogo) -> version corta para
16 // que quepa en el eje del gráfico. Solo cambia como se muestra, el
17 // nombre real que se manda al backend sigue siendo el oficial.
18 const PREFIJOS_CORTOS = { RESTAURANTE: "Rest.", ALMACEN: "Almacén",
19                          ZOOLOGICO: "Zoológico", KIOSCO: "Kiosco" };
20 function nombreCorto(nombre) {
21     return (nombre || "").split(" ").map((p, i) => {
22         if (i === 0 && PREFIJOS_CORTOS[p]) return PREFIJOS_CORTOS[p];
23         if (p === "AYB") return "AyB";
24         return p.charAt(0) + p.slice(1).toLowerCase();
25     }).join(" ");
26 }
27
28 const MESES = ["ene", "feb", "mar", "abr", "may", "jun", "jul", "ago", "sep", "oct", "nov", "dic"];
29 const mesCorto = (fechaISO) => MESES[Number(fechaISO.slice(5, 7)) - 1];
30
31 // Ejemplos de lo que ya entiende el agente en esta pantalla: navegar entre
32 // las dos pestañas y preguntar por lo que muestra cada tarjeta o gráfica.
33 const _EJEMPLOS_PANEL = [
34     "resumen ejecutivo", "bodegas y alertas", "llévame a bodegas y alertas",
35     "¿cuál es la exactitud primera pasada?", "¿cuántas referencias se han contado?",
36     "¿cuántas alertas se han gestionado?", "¿cuántas bodegas están cerradas?",
37     "¿qué bodega tiene más diferencia?", "¿cómo va el stock por unidad de medida?",
38     "¿cómo va la tendencia de exactitud?", "¿cuántos negativos se han corregido?",
39     "¿cuál es el tiempo promedio de conteo?", "¿cuántos alias aprendió el agente?",
40     "¿cómo están las bodegas?", "¿cuáles son las alertas por tipo?",
41     "¿cuál es el descadre principal?",
42 ];
43
44 export default function Panel({ token, ir, tabInicial, navSeq }) {
45     const [tab, setTab] = useState(tabInicial || "resumen");
46     // Pedir una pestaña de esta MISMA pantalla por voz no remonta el
47     // componente - sin esto, el useState de arriba no vuelve a leer
48     // tabInicial y la pestaña se queda en la que ya estaba. navSeq (sube en
49     // cada ir()) tiene que ir en la dependencia también: sin él, pedir la
50     // MISMA pestaña dos veces seguidas (con un clic manual a otra en el
51     // medio) no hacía nada, porque tabInicial ya traía ese mismo valor de
52     // la vez anterior y el efecto no vuelve a correr si su dependencia no
53     // cambió de valor.
54     useEffect(() => { if (tabInicial) setTab(tabInicial); }, [tabInicial, navSeq]);
55     const [resumen, setResumen] = useState(null);
56     const [alertas, setAlertas] = useState(null);
57
58     useEffect(() => {
59         pedir("/api/panel/resumen", {}, token).then(setResumen).catch(() => {});
60         pedir("/api/panel/alertas", {}, token).then(setAlertas).catch(() => {});

```

```

61   }, [token]);
62
63   return (
64     <Marco titulo="Panel gerencial" chip={{ texto: "ACTUALIZADO AHORA", tipo: "verde" }}>
65       <AsistenteVoz token={token} vista="panel" ir={ir} ejemplos={_EJEMPLOS_PANEL}
66         placeholder="¿cuál es la exactitud primera pasada?, lléveme a inicio..." />
67       <div className="chips">
68         <button className={`chip ${tab === "resumen" ? "azul" : ""}`}
69           onClick={() => setTab("resumen")}>Resumen ejecutivo</button>
70         <button className={`chip ${tab === "alertas" ? "azul" : ""}`}
71           onClick={() => setTab("alertas")}>Bodegas y alertas</button>
72       </div>
73
74       {tab === "resumen" && <TabResumen r={resumen} />}
75       {tab === "alertas" && <TabAlertas a={alertas} />}
76     </Marco>
77   );
78 }
79
80 function TabResumen({ r }) {
81   const [saludo, setSaludo] = useState("");
82   const yaSaludo = useRef(false);
83
84   // Al llegar a esta pestaña, un titular breve en vez de esperar a que
85   // pregunten - el detalle completo de cada tarjeta/gráfica sigue
86   // disponible por voz (o en el desplegable de frases), esto es solo el
87   // resumen de resumen para no leer las cuatro tarjetas de una sentada.
88   useEffect(() => {
89     if (!r || yaSaludo.current) return;
90     yaSaludo.current = true;
91     const texto = `Resumen ejecutivo: exactitud primera pasada ${r.exactitud_primera_pasada}%, `
92       + `${r.alertas_gestionadas} de ${r.alertas_total} alertas gestionadas, `
93       + `${r.bodegas_cerradas} de ${r.bodegas_total} bodegas cerradas.`;
94     setSaludo(texto);
95     hablar(texto);
96   }, [r]);
97
98   if (!r) return <p className="cargando">Cargando...</p>;
99   const maxDif = Math.max(1, ...r.diferencia_por_bodega.map((d) => d.diferencia));
100
101   return (
102     <>
103       {saludo && (
104         <div className="card">
105           <p className="rotulo" style={{ margin: 0 }}>CuentaVoz responde</p>
106           <p className="burbuja" aria-live="polite" aria-atomic="true">{saludo}</p>
107         </div>
108       )}
109       <div className="kpi">
110         <div className="kpi verde">
111           <div className="kpi-cabeza">
112             <span className="icono-kpi">🟢</span>
113             <small>Exactitud primera pasada</small>
114           </div>
115           <b>{r.exactitud_primera_pasada}%</b><i>promedio de cierres</i></div>
116         <div className="kpi">
117           <div className="kpi-cabeza">
118             <span className="icono-kpi">🟡</span>
119             <small>Referencias contadas</small>
120           </div>
121           <b>{r.referencias_contadas}</b><i>en toda la operación</i></div>
122         <div className="kpi oro">
123           <div className="kpi-cabeza">
124             <span className="icono-kpi">🔴</span>
125             <small>Alertas gestionadas</small>
126           </div>
127           <b>{r.alertas_gestionadas} / {r.alertas_total}</b><i>resueltas del total</i></div>
128         <div className="kpi">
129           <div className="kpi-cabeza">
130             <span className="icono-kpi">🔒</span>
131             <small>Bodegas cerradas</small>
132           </div>
133           <b>{r.bodegas_cerradas} / {r.bodegas_total}</b><i>con doble firma digital</i></div>
134       </div>
135
136       <div className="conteo-cols">
137         <div className="card">
138           <h3>Diferencia absoluta por bodega</h3>
139           {r.diferencia_por_bodega.length === 0 ? (
140             <p className="vacio">Sin diferencias registradas todavía.</p>
141           ) : (
142             r.diferencia_por_bodega.map((d, i) => (
143               <div className="barra-fila" key={d.bodega}>
144                 <span className="etq">{nombreCorto(d.bodega)}</span>
145                 <div className="pista">
146                   <div className="rellena" style={{ width: `${d.diferencia / maxDif * 100}%`,

```

```

147                                     background: i === 0 ? "var(--amarillo)" : "var(--serie-1)" }} />
148                                 </div>
149                                 <span className="val">{d.diferencia}</span>
150                             </div>
151                         ))
152                     })
153                 </div>
154                 <div className="card">
155                     <h3>Stock por unidad de medida</h3>
156                     <Dona datos={r.stock_por_unidad.map((u, i) => ({
157                         etq: u.unidad, valor: u.cantidad, pct: u.pct,
158                         color: COLOR_UNIDAD[u.unidad] || SERIE[i % SERIE.length],
159                     })))} />
160                 </div>
161             </div>
162
163             <div className="card">
164                 <h3>Exactitud por toma de inventario</h3>
165                 {r.historial_exactitud.length < 2 ? (
166                     <p className="vacio">Se necesitan al menos dos cierres para ver la tendencia.</p>
167                 ) : (
168                     <Linea puntos={r.historial_exactitud} />
169                 )}
170                 <p className="pista" style={{ marginTop: 8 }}>
171                     Un punto por cada bodega cerrada con doble firma, en orden cronológico.
172                 </p>
173             </div>
174         </>
175     );
176 }
177
178 function TabAlertas({ a }) {
179     const [saludo, setSaludo] = useState("");
180     const yaSaludo = useRef(false);
181
182     useEffect(() => {
183         if (!a || yaSaludo.current) return;
184         yaSaludo.current = true;
185         const texto = a.negativos_iniciales !== a.negativos_actuales
186             ? `Bodegas y alertas: saldos negativos, ${a.negativos_iniciales} al inicio del período, `
187             + `${a.negativos_actuales} ahora. Tiempo promedio de conteo ${a.tiempo_promedio_min} minutos.`
188             : `Bodegas y alertas: ${a.negativos_actuales} saldos negativos en el sistema, se corrigen `
189             + `en My Inventory. Tiempo promedio de conteo ${a.tiempo_promedio_min} minutos.`;
190         setSaludo(texto);
191         hablar(texto);
192     }, [a]);
193
194     if (!a) return <p className="cargando">Cargando...</p>;
195     const estados = Object.entries(a.estado_bodegas);
196     const maxAlerta = Math.max(1, ...a.alertas_por_tipo.map((x) => x.cantidad));
197
198     return (
199         <>
200             {saludo && (
201                 <div className="card">
202                     <p className="rotulo" style={{ marginTop: 0 }}>CuentaVoz responde</p>
203                     <p className="burbuja" aria-live="polite" aria-atomic="true">{saludo}</p>
204                 </div>
205             )}
206             <div className="kpi">
207                 <div className="kpi oro">
208                     <div className="kpi-cabeza">
209                         <span className="icono-kpi">📉</span>
210                         <small>Saldos negativos en el sistema</small>
211                     </div>
212                     {/* La corrección de un saldo negativo pasa por fuera de
213                        CuentaVoz (My Inventory, no esta app) - dentro de UNA sola
214                        toma este número no se mueve solo, así que la flecha
215                        "antes → ahora" solo se muestra si de verdad hay una
216                        diferencia real que contar; mostrarla igualada a sí misma
217                        ("79 → 79") no cuenta nada y parece un dato inventado. */}
218                     {a.negativos_iniciales !== a.negativos_actuales ? (
219                         <>
220                             <b>{a.negativos_iniciales} → {a.negativos_actuales}</b>
221                             <i>corregidos desde el inicio de este período</i>
222                         </>
223                     ) : (
224                         <>
225                             <b>{a.negativos_actuales}</b>
226                             <i>artículos - la corrección se hace en My Inventory</i>
227                         </>
228                     )}
229                 </div>
230                 <div className="kpi">
231                     <div className="kpi-cabeza">
232                         <span className="icono-kpi">📈</span>

```

```

233     <small>Tiempo promedio de conteo por bodega</small>
234   </div>
235   <b>{a.tiempo_promedio_min ?? "-"} min</b><i>conteos ya cerrados</i></div>
236   <div className="kpi verde">
237     <div className="kpi-cabeza">
238       <span className="icono-kpi">👤</span>
239       <small>Alias aprendidos por el agente</small>
240     </div>
241     <b>{a.alias_aprendidos}</b><i>el agente habla como el equipo</i></div>
242   </div>
243
244   <div className="dos-columnas">
245     <div className="card">
246       <h3>Estado de las bodegas</h3>
247       <div className="puntos-bodegas">
248         {estados.flatMap(([estado, n]) =>
249           Array.from({ length: n }, (_, i) => (
250             <span key={` ${estado}-${i}`} style={{ background: ESTADO_COLOR[estado] }}
251               title={ESTADO_ETQ[estado]} />
252           ))
253         )}
254       </div>
255       <div className="leyenda">
256         {estados.map(([estado, n]) => (
257           <span key={estado}>
258             <span className="punto" style={{ background: ESTADO_COLOR[estado] }} />
259             {ESTADO_ETQ[estado]} · {n}
260           </span>
261         ))}
262       </div>
263     </div>
264
265     <div className="card">
266       <h3>Alertas por tipo</h3>
267       {a.alertas_por_tipo.length === 0 ? (
268         <p className="vacio">Sin alertas registradas todavía.</p>
269       ) : (
270         a.alertas_por_tipo.map((x, i) => (
271           <div className="barra-fila" key={x.tipo}>
272             <span className="etq">{x.tipo}</span>
273             <div className="pista">
274               <div className="rellena" style={{ width: `${x.cantidad / maxAlerta * 100}%`,
275                 background: i === 0 ? "var(--amarillo)" : "var(--serie-1)" }} />
276             </div>
277             <span className="val">{x.cantidad}</span>
278           </div>
279         ))
280       )}
281     </div>
282   </div>
283
284   <div className="card">
285     <h3>Descuadres recurrentes: dónde está la causa raíz</h3>
286     {a.descuadres_recurrentes.length === 0 ? (
287       <p className="vacio">Sin descuadres repetidos todavía.</p>
288     ) : (
289       <table>
290         <thead>
291           <tr><th>Artículo</th><th>Tipo de alerta</th><th>Dif. acumulada</th>
292             <th>Tomas</th><th>Acción sugerida</th></tr>
293         </thead>
294         <tbody>
295           {a.descuadres_recurrentes.map((d, i) => (
296             <tr key={i}>
297               <td>{d.articulo}</td><td>{d.tipo}</td>
298               <td className="dif">{d.diferencia}</td><td>{d.tomas}</td>
299               <td>{d.accion}</td>
300             </tr>
301           ))}
302         </tbody>
303       </table>
304     )}
305   </div>
306 </>
307 );
308 }
309
310 /** Dona hecha con conic-gradient: sin librerías, con leyenda etiquetada. */
311 function Dona({ datos }) {
312   if (!datos.length) return <p className="vacio">Sin datos de stock todavía.</p>;
313   let acumulado = 0;
314   const segmentos = datos.map((d) => {
315     const inicio = acumulado;
316     acumulado += d.pct;
317     return `${d.color} ${inicio}% ${acumulado}%`;
318   });

```

```

319   const total = datos.reduce((s, d) => s + d.valor, 0);
320   return (
321     <div style={{ display: "flex", gap: 20, alignItems: "center", flexWrap: "wrap" }}>
322       <div style={{ position: "relative", width: 140, height: 140, flex: "none" }}>
323         <div style={{
324           width: 140, height: 140, borderRadius: "50%",
325           background: `conic-gradient(${segmentos.join(",")}` ,
326         }} />
327         <div style={{
328           position: "absolute", inset: 22, borderRadius: "50%", background: "#fff",
329           display: "flex", flexDirection: "column",
330           alignItems: "center", justifyContent: "center",
331           boxShadow: "0 0 0 1px rgba(11,27,51,.04)",
332         }}>
333           <b style={{ fontSize: "1.05rem", color: "var(--azul)" }}>
334             {total.toLocaleString("es-CO")}
335           </b>
336           <span style={{ fontSize: ".68rem", color: "var(--grafito)" }}>referencias</span>
337         </div>
338       </div>
339       <div>
340         {datos.map((d) => (
341           <div key={d.etq} style={{ marginBottom: 6, fontSize: ".85rem" }}>
342             <span className="punto" style={{ background: d.color, display: "inline-block",
343               width: 10, height: 10, borderRadius: "50%", marginRight: 8 }} />
344             <b>{d.etq}</b> - {d.pct}% ({d.valor.toLocaleString("es-CO")})
345           </div>
346         ))}
347       </div>
348     </div>
349   );
350 }
351
352 /** Línea de tendencia con SVG puro: una sola serie, sin leyenda. Como
353     todos los cierres pueden caer el mismo día, el eje muestra la bodega
354     de cada punto (ya es información real y distinta punto a punto) en
355     vez de repetir el mismo mes una y otra vez. */
356 function Línea({ puntos }) {
357   const w = 460, h = 128, pad = 20, padInferior = 34;
358   const valores = puntos.map((p) => p.exactitud);
359   const min = Math.min(...valores) - 2, max = Math.max(...valores) + 2;
360   const paso = (w - pad * 2) / (puntos.length - 1);
361   const y = (v) => h - padInferior - ((v - min) / (max - min || 1)) * (h - pad - padInferior);
362   const coords = puntos.map((p, i) => [pad + i * paso, y(p.exactitud)]);
363   const linea = coords.map(([x, yy], i) => `${i === 0 ? "M" : "L"}${x},${yy}`).join(" ");
364
365   const mejora = valores[valores.length - 1] >= valores[0];
366   const color = "var(--serie-1)";
367
368   return (
369     <>
370       {puntos.length > 1 && (
371         <p style={{ textAlign: "right", fontSize: ".78rem", fontWeight: 700,
372           color: mejora ? "var(--verde)" : "var(--grafito)", marginBottom: 4 }}>
373           {mejora ? "▲ mejora sostenida" : "▼ en descenso"}
374         </p>
375       )}
376       <svg width="100%" viewBox={`0 0 ${w} ${h}`} role="img" aria-label="Tendencia de exactitud">
377         <line x1={pad} y1={h - padInferior} x2={w - pad} y2={h - padInferior} stroke="var(--borde)" strokeWidth="1" />
378         <path d={linea} fill="none" stroke={color} strokeWidth="2" />
379         {coords.map(([x, yy], i) => (
380           <circle key={i} cx={x} cy={yy} r="3.5" fill={color} />
381         ))}
382         {puntos.map((p, i) => (
383           <text key={i} x={coords[i][0]} y={h - padInferior + 11} fontSize="6.5" textAnchor="end"
384             fill="var(--grafito)" transform={`rotate(-40 ${coords[i][0]} ${h - padInferior + 11})`}>
385             <title>{nombreCorto(p.bodega)}</title>
386             {Toma ${i + 1}}
387           </text>
388         ))}
389         <text x={coords[coords.length - 1][0]} y={coords[coords.length - 1][1] - 10}
390           fontSize="11" fontWeight="700" textAnchor="middle" fill={color}>
391           {valores[valores.length - 1]}%
392         </text>
393       </svg>
394     </>
395   );
396 }

```

```

1 import { useEffect, useState } from "react";
2 import { pedir, descargarReporte } from "../api";
3 import { escuchar, hablar, quitarTildes } from "../voz";
4 import Marco from "../Marco";
5 import AsistenteVoz from "../AsistenteVoz";
6 import Icono from "../Iconos";
7
8 const _EJEMPLOS_POR_PESTANA = {
9   config: [
10     "activa el modo sin conexión",
11     "desactiva el modo sin conexión",
12   ],
13   usuarios: [
14     "crea un usuario llamado Juan perfil auxiliar",
15     "activa a Juan", "desactiva a Juan",
16     "cambia el perfil de Juan a administrador",
17     "asigne la bodega Almacen General a Juan",
18     "quítale la bodega Almacen General a Juan",
19     "¿quién tiene la bodega Almacen General?",
20     "¿cuántas bodegas sin asignar hay?",
21     "muéstrame la asignación por bodega",
22     "oculta la asignación por bodega",
23   ],
24   recetas: [
25     "crea una receta llamada Sopa, rendimiento 4 porciones, con 2 kilos de papa",
26     "agrega 300 gramos de cebolla a la receta Sopa",
27     "quita el ingrediente cebolla de la receta Sopa",
28     "cambia el rendimiento de la receta Sopa a 6 porciones",
29     "agrega la preparación a la receta Sopa: pele las papas, sofría la cebolla, y cocine todo junto veinte minutos",
30     "elimina la receta Sopa",
31   ],
32   traza: [
33     "acciones de Luis hoy",
34     "aprobaciones de esta semana",
35     "exporta el registro de trazabilidad",
36     "acciones por persona",
37     "¿cuántas acciones tiene Luis?",
38   ],
39 };
40
41 export default function Ajustes({ token, usuario, ir, tabInicial, recetaId, navSeq }) {
42   const [tab, setTab] = useState(tabInicial || "config");
43   const esAuditor = usuario?.perfil === "auditor";
44   // "llévame a gestión de usuarios" dicho desde esta MISMA pantalla no
45   // remonta el componente (sigue siendo "ajustes"), así que el useState
46   // de arriba no vuelve a leer tabInicial - sin esto, el agente decía
47   // "vamos a Gestión de usuarios" y la pestaña se quedaba en la que ya
48   // estaba. navSeq (sube en cada ir()) también va en la dependencia: sin
49   // él, pedir la MISMA pestaña dos veces seguidas (con un clic manual a
50   // otra en el medio) no hacía nada, porque tabInicial no cambiaba.
51   useEffect(() => { if (tabInicial) setTab(tabInicial); }, [tabInicial, navSeq]);
52
53   return (
54     <Marco titulo="Ajustes · configuración del sistema"
55       chip={{ texto: esAuditor ? "ADMINISTRADORA" : "SOLO LECTURA",
56         tipo: esAuditor ? "azul" : "gris" }}>
57       <AsistenteVoz token={token} vista="ajustes" ir={ir}>
58         placeholder="¿cuál es el umbral de anomalía?, lléveme a reportes..."
59         ejemplos=_EJEMPLOS_POR_PESTANA[tab]
60         alActualizar={() => window.dispatchEvent(new Event("cuentavoz:ajustes-actualizados"))} />
61       <div className="chips">
62         <button className={`chip ${tab === "config" ? "azul" : ""}`}
63           onClick={() => setTab("config")}>Configuración</button>
64         {esAuditor && (
65           <>
66             <button className={`chip ${tab === "usuarios" ? "azul" : ""}`}
67               onClick={() => setTab("usuarios")}>Gestión de usuarios</button>
68             <button className={`chip ${tab === "recetas" ? "azul" : ""}`}
69               onClick={() => setTab("recetas")}>Recetas</button>
70             <button className={`chip ${tab === "traza" ? "azul" : ""}`}
71               onClick={() => setTab("traza")}>Registro de trazabilidad</button>
72           </>
73         )}
74       </div>
75
76       {tab === "config" && <TabConfig token={token} esAuditor={esAuditor} />
77       {tab === "usuarios" && esAuditor && <TabUsuarios token={token} usuario={usuario} />
78       {tab === "recetas" && esAuditor && <TabRecetas token={token} recetaInicial={recetaId} />
79       {tab === "traza" && esAuditor && <TabTraza token={token} />
80     </Marco>
81   );
82 }
83
84 /* — Interruptor simple, sin librería — */
85 function Interruptor({ activo, onChange, disabled, etiqueta }) {
86   return (

```

```

87     <button
88       onClick={() => !disabled && onChange(!activo)}
89       disabled={disabled}
90       role="switch"
91       aria-checked={activo}
92       aria-label={etiqueta}
93       style={{
94         width: 44, height: 24, borderRadius: 14, padding: 2,
95         background: activo ? "var(--verde)" : "#C3CAD6",
96         display: "flex", justifyContent: activo ? "flex-end" : "flex-start",
97         opacity: disabled ? 0.6 : 1, cursor: disabled ? "not-allowed" : "pointer",
98       }}
99     >
100     <span style={{ width: 20, height: 20, borderRadius: "50%", background: "#fff" }} />
101   </button>
102 );
103 }
104
105 function TabConfig({ token, esAuditor }) {
106   const [cfg, setCfg] = useState(null);
107   const [umbral, setUmbral] = useState(50);
108   const [offline, setOffline] = useState(true);
109   const [msg, setMsg] = useState("");
110
111   function cargar() {
112     pedir("/api/ajustes", {}, token).then((c) => {
113       setCfg(c); setUmbral(c.umbral); setOffline(c.offline);
114     }).catch(() => {});
115   }
116   useEffect(cargar, [token]);
117   // El agente de "Pregúntele al agente" (arriba, fuera de esta pestaña)
118   // puede activar/desactivar el modo sin conexión por voz - cuando lo
119   // hace, avisa con este evento para que esta pestaña vuelva a leer el
120   // valor real en vez de quedarse mostrando el de antes.
121   useEffect(() => {
122     window.addEventListener("cuentavoz:ajustes-actualizados", cargar);
123     return () => window.removeEventListener("cuentavoz:ajustes-actualizados", cargar);
124   }, [token]);
125
126   async function guardar() {
127     try {
128       await pedir("/api/ajustes", {
129         method: "PUT", body: JSON.stringify({ umbral, offline }),
130       }, token);
131       setMsg("Configuración guardada.");
132       cargar();
133     } catch (e) { setMsg(e.message); }
134   }
135
136   if (!cfg) return <p className="cargando">Cargando...</p>;
137
138   return (
139     <>
140       <div className="kpi">
141         <div className="kpi">
142           <div className="kpi-cabeza">
143             <span className="icono-kpi">👤</span>
144             <small>Usuarios activos</small>
145           </div>
146           <b>{cfg.usuarios_activos}</b><i>con acceso al sistema</i>
147         </div>
148         <div className={cfg.aprobaciones_pendientes > 0 ? "kpi oro" : "kpi verde"}>
149           <div className="kpi-cabeza">
150             <span className="icono-kpi">✅</span>
151             <small>Aprobaciones pendientes</small>
152           </div>
153           <b>{cfg.aprobaciones_pendientes}</b>
154           <i>{cfg.aprobaciones_pendientes > 0 ? "por revisar en Auditoría" : "todo al día"}</i>
155         </div>
156         <div className={offline ? "kpi verde" : "kpi"}>
157           <div className="kpi-cabeza">
158             <span className="icono-kpi">🌐</span>
159             <small>Modo sin conexión</small>
160           </div>
161           <b>{offline ? "Activo" : "Inactivo"}</b><i>refresco del tablero en tiempo real</i>
162         </div>
163       </div>
164       {/* La versión/modelo ya se ve completa en la tarjeta "Acerca de
165        CuentaVoz" de abajo (con base de datos e idioma también) - un KPI
166        arriba con solo dos de esos cuatro datos era la misma información
167        dos veces, no una tarjeta nueva. */}
168
169       <div className="dos-columnas" style={{ gap: 16 }}>
170         <div className="card">
171           <h3><span className="icono-kpi" style={{ marginRight: 8 }}>🔒</span>Validación de datos</h3>
172           <table>

```

```

173     <tbody>
174     <tr>
175         <td>Umbral de anomalía</td>
176         <td style={{ width: 140 }}>
177             {esAuditor ? (
178                 <input type="number" min="1" max="500" value={umbral}
179                     onChange={(e) => setUmbral(Number(e.target.value))}
180                     aria-label="Umbral de anomalía, en porcentaje"
181                     style={{ width: 80, padding: "6px 10px",
182                         border: "1px solid var(--borde)", borderRadius: 8 }} />
183                 ) : <b>{cfg.umbral}</b>}
184             {esAuditor && " %"}
185         </td>
186     </tr>
187     <tr><td>Bloquear cantidades negativas</td>
188         <td><Interruptor activo disabled onChange={() => {}}
189             etiqueta="Bloquear cantidades negativas" /></td></tr>
190     <tr><td>Exigir confirmación en alertas</td>
191         <td><Interruptor activo disabled onChange={() => {}}
192             etiqueta="Exigir confirmación en alertas" /></td></tr>
193     <tr><td>Permitir crear productos pendientes</td>
194         <td><Interruptor activo disabled onChange={() => {}}
195             etiqueta="Permitir crear productos pendientes" /></td></tr>
196     </tbody>
197 </table>
198 <p className="pista" style={{ marginTop: 8 }}>
199     Las reglas fijas del reto (bloquear negativos, exigir confirmación) no se
200     pueden desactivar: son la garantía de integridad de la toma.
201 </p>
202 </div>
203
204 <div className="card">
205     <h3><span className="icono-kpi" style={{ marginRight: 8 }}><img alt="icono-kpi" data-bbox="535 390 548 400"/></span>Conexión y sincronización</h3>
206     <table>
207         <tbody>
208             <tr><td>Modo sin conexión</td>
209                 <td><Interruptor activo={offline} onChange={setOffline} disabled={!esAuditor}
210                     etiqueta="Modo sin conexión" /></td></tr>
211             <tr><td>Refresco del tablero</td><td><b>tiempo real</b></td></tr>
212         </tbody>
213     </table>
214
215     <h3 style={{ marginTop: 20 }}>
216         <span className="icono-kpi" style={{ marginRight: 8 }}><img alt="icono-kpi" data-bbox="535 505 548 515"/></span>Acerca de CuentaVoz
217     </h3>
218     <table>
219         <tbody>
220             <tr><td>Versión</td><td><b>{cfg.version}</b></td></tr>
221             <tr><td>Modelo del agente</td><td><b>{cfg.modelo}</b></td></tr>
222             <tr><td>Base de datos</td><td><b>{cfg.base_datos}</b></td></tr>
223             <tr><td>Idioma del reconocimiento</td><td><b>{cfg.idioma_voz}</b></td></tr>
224         </tbody>
225     </table>
226 </div>
227 </div>
228
229 {esAuditor && (
230     <div className="grilla-botones" style={{ marginTop: 4 }}>
231         <button className="btn" onClick={guardar}
232             style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
233             <Icono nombre="aprobar" tam={18} />
234             Guardar configuración
235         </button>
236         <button className="btn borde" onClick={cargar}
237             style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
238             <Icono nombre="refrescar" tam={18} />
239             Restaurar valores
240         </button>
241     </div>
242 )}
243 {msg && <p className="msg-ok">{msg}</p>}
244 </>
245 );
246 }
247
248 function TabUsuarios({ token, usuario }) {
249     const [usuarios, setUsuarios] = useState([]);
250     const [bodegas, setBodegas] = useState([]);
251     const [msg, setMsg] = useState("");
252     const [nuevo, setNuevo] = useState(null);
253     const [asignando, setAsignando] = useState(null); // usuario al que se le estan marcando bodegas
254     const [marcadas, setMarcadas] = useState(new Set());
255     const [cobertura, setCobertura] = useState(null);
256     const [verCobertura, setVerCobertura] = useState(false);
257     const [editando, setEditando] = useState(null); // usuario que se esta editando (correo/rol)
258     const [filtro, setFiltro] = useState("todos");

```

```

259
260 function cargar() {
261   pedir("/api/usuarios", {}, token).then(setUsuarios).catch(() => {});
262   pedir("/api/bodegas", {}, token).then(setBodegas).catch(() => {});
263   if (verCobertura) cargarCobertura();
264 }
265 useEffect(cargar, [token]);
266 // El agente ("Pregúntele al agente", arriba de las pestañas) puede
267 // activar/desactivar a alguien por voz - cuando lo hace, avisa con
268 // este evento para que esta lista refleje el cambio sin recargar.
269 useEffect(() => {
270   window.addEventListener("cuentavoz:ajustes-actualizados", cargar);
271   return () => window.removeEventListener("cuentavoz:ajustes-actualizados", cargar);
272 }, [token]);
273
274 // Estos dos modales estan armados a mano (overlay + modal), no con el
275 // componente Dialogo compartido - por eso no traian ya el mismo Escape
276 // para cerrar que ese componente sí tiene.
277 useEffect(() => {
278   function alTecla(e) {
279     if (e.key !== "Escape") return;
280     if (editando) setEditando(null);
281     else if (asignando) setAsignando(null);
282   }
283   window.addEventListener("keydown", alTecla);
284   return () => window.removeEventListener("keydown", alTecla);
285 }, [editando, asignando]);
286
287 function cargarCobertura() {
288   pedir("/api/bodegas/asignaciones", {}, token).then(setCobertura).catch(() => {});
289 }
290
291 // "muéstrame/oculta la asignación por bodega" por voz - mismo botón
292 // «Ver/Ocultar asignación por bodega», solo que disparado por el
293 // agente en vez del clic.
294 useEffect(() => {
295   const mostrar = () => { setVerCobertura(true); cargarCobertura(); };
296   const ocultar = () => setVerCobertura(false);
297   window.addEventListener("cuentavoz:accion:mostrar_cobertura", mostrar);
298   window.addEventListener("cuentavoz:accion:ocultar_cobertura", ocultar);
299   return () => {
300     window.removeEventListener("cuentavoz:accion:mostrar_cobertura", mostrar);
301     window.removeEventListener("cuentavoz:accion:ocultar_cobertura", ocultar);
302   };
303 }, [token]);
304
305 async function crear() {
306   if (!nuevo?.nombre?.trim()) return;
307   try {
308     // sin "pin" en el body: el backend genera uno temporal distinto
309     // para cada persona en vez de reusar siempre la misma clave.
310     const r = await pedir("/api/usuarios", {
311       method: "POST",
312       body: JSON.stringify({ nombre: nuevo.nombre, perfil: nuevo.perfil || "auxiliar",
313         correo: nuevo.correo || "" }),
314     }, token);
315     setMsg(`Usuario ${nuevo.nombre} creado. PIN temporal: ${r.pin_temporal}`
316       + "(cambielo al primer ingreso, en Mi perfil).");
317     setNuevo(null);
318     cargar();
319   } catch (e) { setMsg(e.message); }
320 }
321
322 function alternarBodega(id) {
323   setMarcadas((prev) => {
324     const nueva = new Set(prev);
325     nueva.has(id) ? nueva.delete(id) : nueva.add(id);
326     return nueva;
327   });
328 }
329
330 async function guardarAsignacion() {
331   const u = asignando;
332   const ids = [...marcadas];
333   setAsignando(null);
334   try {
335     await pedir(`/api/usuarios/${u.id}/bodegas`, {
336       method: "PUT", body: JSON.stringify({ bodega_ids: ids }),
337     }, token);
338     setMsg(`${ids.length} bodegas asignadas a ${u.nombre}.`);
339     cargar();
340   } catch (e) { setMsg(e.message); }
341 }
342
343 async function guardarEdicion() {
344   const u = editando;

```

```

345     setEditando(null);
346     try {
347         await pedir(`/api/usuarios/${u.id}`, {
348             method: "PUT", body: JSON.stringify({ correo: u.correo, perfil: u.perfil }),
349         }, token);
350         setMsg(`${u.nombre} actualizado.`);
351         cargar();
352     } catch (e) { setMsg(e.message); }
353 }
354
355 async function alternarActivo(persona) {
356     try {
357         await pedir(`/api/usuarios/${persona.id}`, {
358             method: "PUT", body: JSON.stringify({ activo: !persona.activo }),
359         }, token);
360         setMsg(`${persona.nombre} ${persona.activo ? "desactivado" : "activado"}`);
361         cargar();
362     } catch (e) { setMsg(e.message); }
363 }
364
365 const nAux = usuarios.filter((u) => u.perfil === "auxiliar").length;
366 const nAdmin = usuarios.filter((u) => u.perfil === "auditor").length;
367 const nInactivos = usuarios.filter((u) => !u.activo).length;
368 const usuariosFiltrados = usuarios.filter((u) => {
369     if (filtro === "auxiliar") return u.perfil === "auxiliar";
370     if (filtro === "auditor") return u.perfil === "auditor";
371     if (filtro === "inactivo") return !u.activo;
372     return true;
373 });
374
375 return (
376     <div className="card">
377         <h3><span className="icono-kpi" style={{ marginRight: 8 }}></span>
378             Gestión de usuarios {usuarios.length}</h3>
379         <div style={{ display: "flex", justifyContent: "space-between", alignItems: "center",
380             marginBottom: 12, flexWrap: "wrap", gap: 10 }}>
381             <div className="chips" style={{ marginBottom: 0 }}>
382                 <button className={`chip ${filtro === "auxiliar" ? "azul" : ""}`}
383                     onClick={() => setFiltro(filtro === "auxiliar" ? "todos" : "auxiliar")}>
384                     {nAux} auxiliares
385                 </button>
386                 <button className={`chip ${filtro === "auditor" ? "azul" : ""}`}
387                     onClick={() => setFiltro(filtro === "auditor" ? "todos" : "auditor")}>
388                     {nAdmin} administradores
389                 </button>
390                 <button className={`chip ${filtro === "inactivo" ? "azul" : ""}`}
391                     onClick={() => setFiltro(filtro === "inactivo" ? "todos" : "inactivo")}>
392                     {nInactivos} inactivos
393                 </button>
394             </div>
395             {!nuevo && (
396                 <button className="btn" onClick={() => setNuevo({})}>+ Nuevo usuario</button>
397             )}
398         </div>
399         {msg && <p className="msg-ok">{msg}</p>}
400         <table>
401             <thead><tr><th>Persona</th><th>Perfil</th><th>Bodegas asignadas</th>
402                 <th>Último acceso</th><th>Estado</th><th></th></tr></thead>
403             <tbody>
404                 {usuariosFiltrados.map((u) => (
405                     <tr key={u.id}>
406                         <td style={{ textTransform: "capitalize", display: "flex", alignItems: "center", gap: 8 }}>
407                             <span className="avatar-chico">{u.nombre[0]?.toUpperCase()}</span>
408                             {u.nombre}
409                         </td>
410                         <td>{u.perfil === "auditor" ? "Administrador" : "Auxiliar"}</td>
411                         <td>{u.bodegas_asignadas}</td>
412                         <td>{u.ultimo_acceso ? u.ultimo_acceso.slice(5).replace("-", "/") : "nunca"}</td>
413                         <td style={{ color: u.activo ? "var(--verde)" : "var(--grafito)", fontWeight: 700 }}>
414                             {u.activo ? "Activo" : "Inactivo"}
415                         </td>
416                         <td style={{ display: "flex", gap: 6, flexWrap: "wrap" }}>
417                             <button className="btn borde" style={{ padding: "4px 10px", minHeight: 0 }}
418                                 onClick={async () => {
419                                     setAsignando(u);
420                                     const propias = await pedir(`/api/usuarios/${u.id}/bodegas`, {}, token).catch(() => []);
421                                     setMarcadas(new Set(propias));
422                                 }}>
423                                 Asignar bodegas
424                             </button>
425                             <button className="btn borde" style={{ padding: "4px 10px", minHeight: 0 }}
426                                 onClick={() => setEditando({ id: u.id, nombre: u.nombre,
427                                     correo: u.correo, perfil: u.perfil })}>
428                                 Editar
429                             </button>
430                             <button className={`btn ${u.activo ? "gris" : "verde"}`>

```

```

431         style={{ padding: "4px 10px", minHeight: 0 }}
432         disabled={u.id === usuario?.id}
433         title={u.id === usuario?.id ? "No puede desactivar su propia cuenta" : ""}
434         onClick={() => alternarActivo(u)}
435         {u.activo ? "Desactivar" : "Activar"}
436       </button>
437     </td>
438   </tr>
439   )))
440 </tbody>
441 </table>
442
443 {nuevo ? (
444   <div style={{ marginTop: 16, display: "flex", gap: 10, flexWrap: "wrap" }}>
445     <input placeholder="nombre de usuario" value={nuevo.nombre || ""}
446       onChange={(e) => setNuevo({ ...nuevo, nombre: e.target.value })}
447       aria-label="Nombre de usuario"
448       style={{ padding: "10px 12px", border: "1px solid var(--borde)", borderRadius: 10 }} />
449     <input placeholder="correo (obligatorio)" value={nuevo.correo || ""}
450       onChange={(e) => setNuevo({ ...nuevo, correo: e.target.value })}
451       aria-label="Correo, obligatorio"
452       style={{ padding: "10px 12px", border: "1px solid var(--borde)", borderRadius: 10 }} />
453     <select value={nuevo.perfil || "auxiliar"}
454       onChange={(e) => setNuevo({ ...nuevo, perfil: e.target.value })}
455       aria-label="Perfil del nuevo usuario"
456       style={{ padding: "10px 12px", border: "1px solid var(--borde)", borderRadius: 10 }}>
457       <option value="auxiliar">Auxiliar</option>
458       <option value="auditor">Administrador</option>
459     </select>
460     <button className="btn" onClick={crear}>Crear (clave temporal)</button>
461     <button className="btn gris" onClick={() => setNuevo(null)}>Cancelar</button>
462   </div>
463 ) : null}
464
465 <div className="grilla-botones" style={{ marginTop: 14 }}>
466   <button className="btn borde"
467     onClick={() => {
468       const abrir = !verCobertura;
469       setVerCobertura(abrir);
470       if (abrir && !cobertura) cargarCobertura();
471     }}
472     {verCobertura ? "Ocultar asignación por bodega" : "Ver asignación por bodega"}
473   </button>
474 </div>
475
476 {verCobertura && (
477   <div style={{ marginTop: 14 }}>
478     <h3>Quién tiene cada bodega</h3>
479     <p className="pista" style={{ marginBottom: 10 }}>
480       Para repartir 54 bodegas entre varios auxiliares sin dejar ninguna sin dueño
481       ni asignarla dos veces por error.
482     </p>
483     {cobertura === null ? (
484       <p className="cargando">Cargando...</p>
485     ) : (
486       <div style={{ maxHeight: 420, overflowY: "auto" }}>
487         <table>
488           <thead><tr><th>Bodega</th><th>Asignada a</th></tr></thead>
489           <tbody>
490             {cobertura.map((b) => (
491               <tr key={b.id}>
492                 <td>{b.bodega}</td>
493                 <td>
494                   {b.asignados.length === 0 ? (
495                     <span className="chip oro">Sin asignar</span>
496                   ) : (
497                     b.asignados.map((p) => (
498                       <span key={p.id} className={`chip ${p.perfil === "auditor" ? "azul" : "gris"}`}
499                         style={{ marginRight: 6, textTransform: "capitalize" }}>
500                         {p.nombre}
501                       </span>
502                     ))
503                   )}
504                 </td>
505               </tr>
506             ))}
507           </tbody>
508         </table>
509       </div>
510     )}
511   </div>
512 )}
513
514 <div className="card" style={{ background: "var(--fondo)", marginTop: 14 }}>
515   <h3>Qué puede hacer cada perfil</h3>
516   <table>

```

```

517     <tbody>
518     <tr>
519         <td>✔ Contar por voz y corregir sus propios registros</td>
520         <td style={{ textAlign: "right", color: "var(--verde)", fontWeight: 700 }}>
521             Auxiliar y administrador
522         </td>
523     </tr>
524     <tr>
525         <td>✔ Crear productos y bodegas (quedan pendientes)</td>
526         <td style={{ textAlign: "right", color: "var(--verde)", fontWeight: 700 }}>
527             Auxiliar y administrador
528         </td>
529     </tr>
530     <tr>
531         <td>✔ Aprobar creaciones, auditar y cerrar bodegas</td>
532         <td style={{ textAlign: "right", color: "var(--azul)", fontWeight: 700 }}>
533             Solo administrador
534         </td>
535     </tr>
536     <tr>
537         <td>✔ Gestionar usuarios y cambiar los ajustes del sistema</td>
538         <td style={{ textAlign: "right", color: "var(--azul)", fontWeight: 700 }}>
539             Solo administrador
540         </td>
541     </tr>
542 </tbody>
543 </table>
544 </div>
545
546 {editando && (
547     <div className="overlay" onClick={() => setEditando(null)}>
548         <div className="modal" style={{ maxWidth: 420, textAlign: "left" }}
549             onClick={(e) => e.stopPropagation()}
550             role="dialog" aria-modal="true" aria-labelledby="titulo-editar-usuario">
551             <h2 id="titulo-editar-usuario" style={{ textTransform: "capitalize" }}>
552                 Editar a {editando.nombre}
553             </h2>
554             <label className="pista" htmlFor="campo-correo-editar">Correo</label>
555             <input id="campo-correo-editar" value={editando.correo || ""}
556                 onChange={(e) => setEditando({ ...editando, correo: e.target.value })}
557                 style={{ width: "100%", padding: "10px 12px", marginTop: 6, marginBottom: 14,
558                     border: "1px solid var(--borde)", borderRadius: 10 }} />
559             <label className="pista" htmlFor="campo-perfil-editar">Perfil</label>
560             <select id="campo-perfil-editar" value={editando.perfil}
561                 onChange={(e) => setEditando({ ...editando, perfil: e.target.value })}
562                 style={{ width: "100%", padding: "10px 12px", marginTop: 6,
563                     border: "1px solid var(--borde)", borderRadius: 10 }}
564                 disabled={editando.id === usuario?.id}>
565                 <option value="auxiliar">Auxiliar</option>
566                 <option value="auditor">Administrador</option>
567             </select>
568             {editando.id === usuario?.id && (
569                 <p className="pista" style={{ marginTop: 6 }}>
570                     No puede cambiar su propio rol desde aquí.
571                 </p>
572             )}
573             <div className="botones" style={{ marginTop: 18 }}>
574                 <button className="btn borde" onClick={() => setEditando(null)}>Cancelar</button>
575                 <button className="btn" onClick={guardarEdicion}>Guardar</button>
576             </div>
577         </div>
578     </div>
579 )}
580
581 {asignando && (
582     <div className="overlay" onClick={() => setAsignando(null)}>
583         <div className="modal" style={{ maxWidth: 520, textAlign: "left" }}
584             onClick={(e) => e.stopPropagation()}
585             role="dialog" aria-modal="true" aria-labelledby="titulo-asignar-bodegas">
586             <h2 id="titulo-asignar-bodegas" style={{ textTransform: "capitalize" }}>
587                 Bodegas para {asignando.nombre}
588             </h2>
589             <p className="pista">
590                 {asignando.perfil === "auditor"
591                     ? "Marque las bodegas de las que esta persona es responsable como administradora (recuento ciego, cierre)."
592                     : "Marque las bodegas que va a poder contar esta persona."}
593             </p>
594             <div style={{ maxHeight: 320, overflowY: "auto", margin: "12px 0",
595                 border: "1px solid var(--borde)", borderRadius: 10, padding: 8 }}>
596                 {bodegas.map((b) => (
597                     <label key={b.id} style={{ display: "flex", alignItems: "center", gap: 10,
598                         padding: "7px 6px", fontSize: ".9rem" }}>
599                         <input type="checkbox" checked={marcadas.has(b.id)}
600                             onChange={() => alternarBodega(b.id)} />
601                         {b.bodega}
602                     </label>

```

```

603         ))}
604     </div>
605     <p className="pista">{marcadas.size} bodegas marcadas</p>
606     <div className="botones">
607         <button className="btn borde" onClick={() => setAsignando(null)}>Cancelar</button>
608         <button className="btn" onClick={guardarAsignacion}>Guardar</button>
609     </div>
610 </div>
611 </div>
612     )}
613 </div>
614 );
615 }
616
617 function TabRecetas({ token, recetaInicial }) {
618     const [recetas, setRecetas] = useState([]);
619     const [catalogo, setCatalogo] = useState([]);
620     const [msg, setMsg] = useState("");
621     const [editando, setEditando] = useState(null);
622     // { id: null|number, nombre, rendimiento, lineas: [{articulo_codigo, cantidad_por_porcion}] }
623     const abrioInicial = useState(false);
624
625     function cargar() {
626         pedir("/api/recetas", {}, token).then(setRecetas).catch(() => {});
627         pedir("/api/articulos/catalogo-ligero", {}, token).then(setCatalogo).catch(() => {});
628     }
629     useEffect(cargar, [token]);
630
631     useEffect(() => {
632         window.addEventListener("cuentavoz:ajustes-actualizados", cargar);
633         return () => window.removeEventListener("cuentavoz:ajustes-actualizados", cargar);
634     }, [token]);
635
636     useEffect(() => {
637         if (recetaInicial && recetas.length && !abrioInicial[0]) {
638             abrioInicial[1](true);
639             abrir(recetaInicial);
640         }
641     }, [recetaInicial, recetas]);
642
643     // Mismo caso que en TabUsuarios: este modal esta armado a mano, no con
644     // el componente Dialogo compartido, asi que necesita su propio Escape.
645     useEffect(() => {
646         function alTecla(e) { if (e.key === "Escape" && editando) setEditando(null); }
647         window.addEventListener("keydown", alTecla);
648         return () => window.removeEventListener("keydown", alTecla);
649     }, [editando]);
650
651     async function abrir(id) {
652         try {
653             const r = await pedir(`/api/recetas/${id}`, {}, token);
654             setEditando({
655                 id: r.id, nombre: r.nombre, rendimiento: r.rendimiento,
656                 preparacion: r.preparacion || "",
657                 lineas: r.lineas.map((l) => ({ articulo_codigo: l.articulo_codigo,
658                     cantidad_por_porcion: l.cantidad_por_porcion })),
659             });
660         } catch (e) { setMsg(e.message); }
661     }
662
663     function nueva() {
664         setEditando({ id: null, nombre: "", rendimiento: 1, preparacion: "",
665             lineas: [{ articulo_codigo: "", cantidad_por_porcion: "" }] });
666     }
667
668     function actualizarLinea(i, campo, valor) {
669         setEditando(e => {
670             const lineas = e.lineas.map((l, idx) => idx === i ? { ...l, [campo]: valor } : l);
671             return { ...e, lineas };
672         });
673     }
674
675     function agregarLinea() {
676         setEditando(e => ({ ...e, lineas: [...e.lineas, { articulo_codigo: "", cantidad_por_porcion: "" }] }));
677     }
678
679     function quitarLinea(i) {
680         setEditando(e => ({ ...e, lineas: e.lineas.filter((_, idx) => idx !== i) }));
681     }
682
683     async function guardar() {
684         const cuerpo = {
685             nombre: editando.nombre.trim(),
686             rendimiento: Number(editando.rendimiento) || 1,
687             preparacion: (editando.preparacion || "").trim(),
688             ingredientes: editando.lineas

```

```

689     .filter((l) => l.articulo_codigo && l.cantidad_por_porcion)
690     .map((l) => ({ articulo_codigo: l.articulo_codigo,
691                   cantidad_por_porcion: Number(l.cantidad_por_porcion) })),
692 );
693 if (!cuerpo.nombre) { setMsg("Póngale un nombre a la receta."); return; }
694 try {
695     if (editando.id) {
696         await pedir(`/api/recetas/${editando.id}`, { method: "PUT", body: JSON.stringify(cuerpo) }, token);
697         setMsg(`Receta «${cuerpo.nombre}» actualizada.`);
698     } else {
699         await pedir("/api/recetas", { method: "POST", body: JSON.stringify(cuerpo) }, token);
700         setMsg(`Receta «${cuerpo.nombre}» creada.`);
701     }
702     setEditando(null);
703     cargar();
704 } catch (e) { setMsg(e.message); }
705 }
706
707 async function eliminar(r) {
708     if (!window.confirm(`¿Eliminar la receta «${r.nombre}»? Esta acción no se puede deshacer.`)) return;
709     try {
710         await pedir(`/api/recetas/${r.id}`, { method: "DELETE" }, token);
711         setMsg(`Receta «${r.nombre}» eliminada.`);
712         cargar();
713     } catch (e) { setMsg(e.message); }
714 }
715
716 return (
717     <div className="card">
718         <h3>
719             <span className="icono-kpi" style={{ marginRight: 8 }}>🍴</span>
720             Recetas ({recetas.length})
721         </h3>
722         <p className="pista">
723             Las porciones que dicte el chef se calculan sobre estas recetas: cantidad por
724             porción x porciones, menos lo que ya haya en la bodega. Aquí se crean, editan
725             y borran – CuentaVoz sí gestiona el catálogo de recetas y menús.
726         </p>
727         {msg && <p className="msg-ok">{msg}</p>}
728         {recetas.length === 0 ? (
729             <p className="vacio">Todavía no hay recetas creadas.</p>
730         ) : (
731             <table>
732                 <thead><tr><th>Receta</th><th>Rendimiento</th><th>Ingredientes</th><th></th></tr></thead>
733                 <tbody>
734                     {recetas.map((r) => (
735                         <tr key={r.id}>
736                             <td style={{ textTransform: "capitalize" }}>{r.nombre}</td>
737                             <td>{r.rendimiento} porción</td>
738                             <td>{r.ingredientes}</td>
739                             <td style={{ display: "flex", gap: 6 }}>
740                                 <button className="btn borde" style={{ padding: "4px 10px", minHeight: 0 }}
741                                     onClick={() => abrir(r.id)}>Editar</button>
742                                 <button
743                                     className="btn gris" style={{ padding: "4px 10px", minHeight: 0 }}
744                                     onClick={() => eliminar(r)}>Eliminar</button>
745                             </td>
746                         </tr>
747                     ))}
748                 </tbody>
749             </table>
750             <div className="grilla-botones" style={{ marginTop: 14 }}>
751                 <button className="btn" onClick={nueva}>+ Nueva receta</button>
752             </div>
753
754             <div editando && (
755                 <div className="overlay" onClick={() => setEditando(null)}>
756                     <div className="modal" style={{ maxWidth: 660, textAlign: "left", maxHeight: "88vh",
757                           overflowY: "auto" }}>
758                         onClick={() => e.stopPropagation()}
759                         role="dialog" aria-modal="true" aria-labelledby="titulo-editar-receta">
760                             <h2 id="titulo-editar-receta">
761                                 {editando.id ? `Editar ${editando.nombre}` : "Nueva receta"}
762                             </h2>
763                             <p className="pista" style={{ marginTop: -6, marginBottom: 18 }}>
764                                 Cada campo queda disponible de inmediato para calcular pedidos por voz.
765                             </p>
766
767                             <div style={{ display: "grid", gridTemplateColumns: "2fr 1fr", gap: 14, marginBottom: 20 }}>
768                                 <div>
769                                     <label className="pista" htmlFor="campo-nombre-plato" style={{ display: "block", marginBottom: 6 }}>
770                                         Nombre del plato
771                                     </label>
772                                     <input id="campo-nombre-plato" value={editando.nombre}
773                                         onChange={(e) => setEditando({ ...editando, nombre: e.target.value })}
774                                         placeholder="ej. ajiaco"

```

```

775         style={{ width: "100%", padding: "10px 12px",
776             border: "1px solid var(--borde)", borderRadius: 10 }} />
777     </div>
778     <div>
779         <label className="pista" htmlFor="campo-rendimiento" style={{ display: "block", marginBottom: 6 }}>
780             Rendimiento (porciones)
781         </label>
782         <input id="campo-rendimiento" type="number" min="1" value={editando.rendimiento}
783             onChange={(e) => setEditando({ ...editando, rendimiento: e.target.value })}
784             style={{ width: "100%", padding: "10px 12px",
785                 border: "1px solid var(--borde)", borderRadius: 10 }} />
786     </div>
787 </div>
788
789 <div style={{ background: "var(--fondo)", borderRadius: 12, padding: 16, marginBottom: 18 }}>
790     <h3 style={{ marginTop: 0, marginBottom: 4, fontSize: ".95rem", color: "var(--azul)" }}>
791         🍽 Ingredientes
792     </h3>
793     <p className="pista" style={{ marginBottom: 12 }}>Cantidad por cada porción de la receta.</p>
794     <div style={{ maxHeight: 280, overflowY: "auto" }}>
795         {editando.lineas.map((l, i) => (
796             <div key={i} style={{ display: "flex", gap: 8, alignItems: "center", marginBottom: 8 }}>
797                 <select value={l.articulo_codigo}
798                     onChange={(e) => actualizarLinea(i, "articulo_codigo", e.target.value)}
799                     aria-label={`Artículo del ingrediente ${i + 1}`}
800                     style={{ flex: 1, padding: "8px 10px", border: "1px solid var(--borde)",
801                         borderRadius: 8, background: "#fff" }}>
802                     <option value="">— elija un artículo del catálogo —</option>
803                     {catalogo.map((a) => (
804                         <option key={a.codigo} value={a.codigo}>{a.nombre} {a.unidad}</option>
805                     ))}
806                 </select>
807                 <input type="number" step="0.001" min="0" placeholder="cantidad"
808                     value={l.cantidad_por_porcion}
809                     onChange={(e) => actualizarLinea(i, "cantidad_por_porcion", e.target.value)}
810                     aria-label={`Cantidad por porción del ingrediente ${i + 1}`}
811                     style={{ width: 100, padding: "8px 10px", border: "1px solid var(--borde)",
812                         borderRadius: 8, background: "#fff" }} />
813                 <button className="btn gris" style={{ padding: "6px 10px", minHeight: 0, flex: "none" }}
814                     onClick={() => quitarLinea(i)} aria-label={`Quitar ingrediente ${i + 1}`}>X</button>
815             </div>
816         ))}
817     </div>
818     <button className="btn borde" style={{ marginTop: 4 }} onClick={agregarLinea}>
819         + Agregar ingrediente
820     </button>
821 </div>
822
823 <div style={{ background: "var(--fondo)", borderRadius: 12, padding: 16, marginBottom: 4 }}>
824     <h3 style={{ marginTop: 0, marginBottom: 4, fontSize: ".95rem", color: "var(--azul)" }}>
825         📖 Preparación paso a paso
826     </h3>
827     <p className="pista" style={{ marginBottom: 12 }}>
828         Opcional: lo que ve el auxiliar al consultar esta receta desde Pedidos.
829     </p>
830     <textarea value={editando.preparacion}
831         onChange={(e) => setEditando({ ...editando, preparacion: e.target.value })}
832         placeholder="1. Pele y pique las papas...&#10;2. Sofría la cebolla...&#10;3. ..."
833         aria-label="Preparación paso a paso, opcional"
834         rows={5}
835         style={{ width: "100%", padding: "10px 12px",
836             border: "1px solid var(--borde)", borderRadius: 10,
837             fontFamily: "inherit", resize: "vertical", background: "#fff" }} />
838 </div>
839
840 <div className="botones" style={{ marginTop: 20 }}>
841     <button className="btn borde" onClick={() => setEditando(null)}>Cancelar</button>
842     <button className="btn" onClick={guardar}>Guardar receta</button>
843 </div>
844 </div>
845 </div>
846 ))}
847 </div>
848 );
849 }
850
851 function TabTraza({ token }) {
852     const [traza, setTraza] = useState([]);
853     const [persona, setPersona] = useState("");
854     const [accion, setAccion] = useState("");
855     const [rango, setRango] = useState("hoy");
856     const [msg, setMsg] = useState("");
857     const [todasPersonas, setTodasPersonas] = useState([]);
858     const [escuchandoFiltro, setEscuchandoFiltro] = useState(false);
859
860     function cargar() {

```

```

861     const q = new URLSearchParams();
862     if (persona) q.set("persona", persona);
863     if (accion) q.set("accion", accion);
864     if (rango) q.set("rango", rango);
865     pedir(`/api/trazabilidad?${q}`, {}, token).then(setTrazas).catch(() => {});
866 }
867 useEffect(cargar, [token, persona, accion, rango]);
868 // Lista COMPLETA de personas (no solo las que aparecen en el filtro
869 // actual) para que el filtro por voz reconozca un nombre aunque el
870 // rango/persona ya aplicado lo haya dejado fuera de "traza".
871 useEffect(() => {
872     pedir("/api/usuarios", {}, token).then((us) => setTodasPersonas(us.map((x) => x.nombre)))
873     .catch(() => {});
874 }, [token]);
875
876 const acciones = ["", "INGRESO", "APERTURA", "CONTEO", "CORRECCION", "CREACION",
877     "FIRMA", "AUDITORIA", "CIERRE", "REAPERTURA", "APROBACION", "ALERTA",
878     "REPORTE", "PEDIDO", "RECETA", "LEGALIZACION", "AJUSTE", "USUARIO",
879     "ASIGNACION", "SEGURIDAD", "PERFIL", "SOPORTE"];
880 const personas = [...new Set(traza.map((t) => t.persona))].sort();
881
882 // "Muéstrame las acciones de Luis hoy" - determinístico y local, sin
883 // pasar por el agente general: solo reconoce palabras contra las
884 // listas que YA existen en esta pantalla (rango, tipos de acción,
885 // personas), así que no hace falta ninguna llamada a Gemini para
886 // filtrar. Combina lo dicho con lo que YA estaba marcado (decir solo
887 // "esta semana" no borra la persona/acción que ya tenía puesta) -
888 // por eso lee rango/persona/accion actuales como punto de partida en
889 // vez de siempre resetear a "todos".
890 function _resolverFiltroVoz(texto) {
891     const t = quitarTildes(texto.toLowerCase());
892     let coincidio = false;
893     let nuevoRango = rango, nuevaAccion = accion, nuevaPersona = persona;
894     if (/^btodo\b/.test(t)) { nuevoRango = ""; coincidio = true; }
895     else if (/^bhoy\b/.test(t)) { nuevoRango = "hoy"; coincidio = true; }
896     else if (/^semana\b/.test(t)) { nuevoRango = "semana"; coincidio = true; }
897     else if (/^bmes\b/.test(t)) { nuevoRango = "mes"; coincidio = true; }
898
899     if (/todas las acciones/.test(t)) { nuevaAccion = ""; coincidio = true; }
900     else {
901         const accionEncontrada = acciones.find((a) => a && t.includes(a.toLowerCase()));
902         if (accionEncontrada) { nuevaAccion = accionEncontrada; coincidio = true; }
903     }
904
905     if (/todas las personas/.test(t)) { nuevaPersona = ""; coincidio = true; }
906     else {
907         const personaEncontrada = todasPersonas.find(
908             (p) => p && t.includes(quitarTildes(p.toLowerCase())));
909         if (personaEncontrada) { nuevaPersona = personaEncontrada; coincidio = true; }
910     }
911     return coincidio ? { rango: nuevoRango, persona: nuevaPersona, accion: nuevaAccion } : null;
912 }
913
914 // Aplica el filtro resuelto Y dice el resultado real (cuántas filas
915 // encontró), no un "listo, apliqué el filtro" que no cuenta nada -
916 // pedido explícito del usuario. Trae los datos de una (en vez de
917 // esperar a que el useEffect de cargar() reaccione al cambio de
918 // estado) para poder anunciar la cifra exacta apenas llega.
919 async function _aplicarFiltroYAnunciar(resuelto) {
920     const { rango: r, persona: p, accion: a } = resuelto;
921     setRango(r); setPersona(p); setAccion(a);
922     setMsg("");
923     try {
924         const q = new URLSearchParams();
925         if (p) q.set("persona", p);
926         if (a) q.set("accion", a);
927         if (r) q.set("rango", r);
928         const filas = await pedir(`/api/trazabilidad?${q}`, {}, token);
929         setTrazas(filas);
930         const partes = [];
931         if (a) partes.push(`${a.toLowerCase()}`);
932         if (p) partes.push(`${p}`);
933         const etiquetaRango = r === "hoy" ? "hoy" : r === "semana" ? "en la última semana"
934             : r === "mes" ? "en el último mes" : "en total";
935         const texto = `Encontré ${filas.length} acciones${partes.length ? " " : ""} + partes.join(" ") : ""} ${etiquetaRango}`;
936         setMsg(texto);
937         hablar(texto);
938     } catch (e) { setMsg(e.message); hablar(e.message); }
939 }
940
941 // El cuadro "Pregúntele al agente" (arriba de las pestañas) es el
942 // MISMO cuadro que muestra los ejemplos «acciones de Luis hoy» - una
943 // persona que escribe/dice esa frase ahí espera que funcione, no
944 // solo desde el micrófono chiquito junto a los filtros. Como el
945 // filtro ya vive aquí (sin pasar por el backend), se intercepta la
946 // frase ANTES de que AsistenteVoz llegue a preguntarle al agente -

```

```

947 // event.preventDefault() debe llamarse de una, ANTES de cualquier
948 // await, para que AsistenteVoz vea la señal en el mismo ciclo
949 // síncrono del dispatchEvent (el fetch+anuncio siguen aparte, ya de
950 // forma asíncrona, sin que eso afecte esa señal).
951 useEffect(() => {
952   const interceptar = (e) => {
953     const resuelto = _resolverFiltroVoz(e.detail.texto);
954     if (!resuelto) return;
955     e.preventDefault();
956     _aplicarFiltroYAnunciar(resuelto);
957   };
958   window.addEventListener("cuentavoz:filtro-local", interceptar);
959   return () => window.removeEventListener("cuentavoz:filtro-local", interceptar);
960 }, [todasPersonas, acciones, token, rango, persona, accion]);
961
962 function escucharFiltro() {
963   setMsg("");
964   escuchar({
965     alTexto: (texto) => {
966       const resuelto = _resolverFiltroVoz(texto);
967       if (resuelto) _aplicarFiltroYAnunciar(resuelto);
968       else setMsg("No reconocí ningún filtro en lo que dijo.");
969     },
970     alEstado: (e) => setEscuchandoFiltro(e === "escuchando"),
971     alError: setMsg,
972   });
973 }
974
975 async function exportar() {
976   setMsg("");
977   try {
978     const q = new URLSearchParams();
979     if (persona) q.set("persona", persona);
980     if (accion) q.set("accion", accion);
981     if (rango) q.set("rango", rango);
982     const d = await pedir(`/api/trazabilidad/exportar?${q}`, { method: "GET" }, token);
983     await descargarReporte(d.archivo, token);
984     setMsg(`Exportado: ${d.filas} filas.`);
985   } catch (e) { setMsg(e.message); }
986 }
987
988 // "exporta el registro de trazabilidad" por voz - mismo botón
989 // «Exportar», con los filtros que ya estén puestos en pantalla (el
990 // agente no los conoce, viven aquí como estado de React).
991 useEffect(() => {
992   window.addEventListener("cuentavoz:accion:exportar_trazabilidad", exportar);
993   return () => window.removeEventListener("cuentavoz:accion:exportar_trazabilidad", exportar);
994 }, [token, persona, accion, rango]);
995
996 return (
997   <div className="card">
998     <h3><span className="icono-kpi" style={{ marginRight: 8 }}>📄</span>
999     Registro de trazabilidad ({traza.length} acciones)</h3>
1000     <div style={{ display: "flex", gap: 10, marginBottom: 12, flexWrap: "wrap", alignItems: "center" }}>
1001       <select value={rango} onChange={(e) => setRango(e.target.value)}
1002         aria-label="Rango de fechas del registro de trazabilidad"
1003         style={{ padding: "9px 12px", border: "1px solid var(--borde)", borderRadius: 10 }}>
1004         <option value="hoy">Hoy</option>
1005         <option value="semana">Última semana</option>
1006         <option value="mes">Último mes</option>
1007         <option value="">Todo</option>
1008       </select>
1009       <select value={persona} onChange={(e) => setPersona(e.target.value)}
1010         aria-label="Filtrar por persona"
1011         style={{ padding: "9px 12px", border: "1px solid var(--borde)", borderRadius: 10 }}>
1012         <option value="">Todas las personas</option>
1013         {personas.map((p) => <option key={p} value={p} style={{ textTransform: "capitalize" }}>{p}</option>)}
1014       </select>
1015       <select value={accion} onChange={(e) => setAccion(e.target.value)}
1016         aria-label="Filtrar por tipo de acción"
1017         style={{ padding: "9px 12px", border: "1px solid var(--borde)", borderRadius: 10 }}>
1018         {acciones.map((a) => <option key={a} value={a}>{a} || "Todas las acciones"</option>)}
1019       </select>
1020       <button className={`mic-btn ${escuchandoFiltro ? "escuchando" : ""}`}
1021         style={{ width: 46, height: 46, fontSize: "1.2rem" }}
1022         onClick={escucharFiltro} title="filtrar por voz: «acciones de Luis hoy»"
1023         aria-label="Filtrar el registro de trazabilidad por voz">
1024         <Icono nombre="microfono" tam={20} />
1025       <button className="btn verde"
1026         style={{ marginLeft: "auto", display: "inline-flex", alignItems: "center", gap: 8 }}
1027         onClick={exportar}>
1028         <Icono nombre="descargar" tam={18} />
1029         Exportar
1030       </button>
1031     </div>
1032     {msg && (

```

```

1033     <div className={`banner ${msg.startsWith("Encontré") || msg.startsWith("Exportado")} ? "ok" : ""}`}
1034         style={{ marginBottom: 12 }}>
1035         <span className="ico">
1036             {msg.startsWith("Encontré") || msg.startsWith("Exportado")} ? "✓" : "!">
1037         </span>
1038         <span>{msg}</span>
1039     </div>
1040 })
1041 {traza.length === 0 ? (
1042     <p className="vacio">Sin acciones registradas todavía.</p>
1043 ) : (
1044     <table>
1045         <thead><tr><th>Hora</th><th>Persona</th><th>Acción</th><th>Detalle</th></tr></thead>
1046         <tbody>
1047             {traza.slice(0, 25).map((t) => (
1048                 <tr key={t.id}>
1049                     <td>{t.hora}</td>
1050                     <td style={{ textTransform: "capitalize" }}>{t.persona}</td>
1051                     <td><span className={`chip ${t.tipo === "alerta" ? "oro"
1052                         : t.tipo === "ok" ? "verde" : ""}`}>{t.accion}</span></td>
1053                     <td>{t.detalle}</td>
1054                 </tr>
1055             ))}
1056         </tbody>
1057     </table>
1058 )}
1059 <p className="pista" style={{ marginTop: 10 }}>
1060     Cumple la Ley 1581 de 2012: se registra la acción, no datos personales sensibles.
1061     El registro no se puede editar ni borrar.
1062 </p>
1063 <div className="card" style={{ marginTop: 14, background: "var(--azul-claro)" }}>
1064     <h3>Por qué el registro es inmutable</h3>
1065     <p style={{ fontSize: ".87rem" }}>
1066         Una corrección no borra el valor anterior: crea un registro nuevo que apunta
1067         al original (campo <code>corrige_a</code> de la tabla Conteo). Así, si alguien
1068         pregunta «¿quién cambió este dato y por qué?», la respuesta está en el
1069         sistema y no en la memoria de nadie.
1070     </p>
1071 </div>
1072 </div>
1073 );
1074 }

```

frontend/src/vistas/Ayuda.jsx

364 LÍNEAS

Preguntas, agente y soporte.

```

1  import { useEffect, useState, useRef } from "react";
2  import { pedir, preguntarAsistente } from "../api";
3  import { EVENTO_ABRIR_VIDEO } from "../tutorial";
4  import { escuchar, hablar, quitarTildes } from "../voz";
5  import { enviarPorEmailJS, capitalizar, rolDe } from "../correoAdmin";
6  import Marco from "../Marco";
7  import Dialogo from "../Dialogo";
8  import Icono from "../Iconos";
9
10 const FAQ = [
11     ["¿Cómo corrijo un conteo ya confirmado?",
12         "Diga «corregir» y luego el valor correcto. El valor anterior se conserva."],
13     ["El agente no me entiende un producto",
14         "Dígallo como aparece en la etiqueta. Si no existe, se puede crear: queda pendiente."],
15     ["¿Qué hago si sale una alerta?",
16         "Recuente. Si el número es correcto, confirme: queda marcado para el administrador."],
17     ["Se cayó el internet en plena bodega",
18         "Siga contando: el intérprete local mantiene el flujo y sincroniza al volver la señal."],
19     ["¿Puedo contar dos bodegas a la vez?",
20         "No en el mismo dispositivo. El candado de sesión evita conteos duplicados."],
21 ];
22
23 const COMANDOS = [
24     ["«Iniciar conteo en almacén de suministros», "abrir una bodega"],
25     ["«Tres tablas para picar blancas», "registrar un conteo"],
26     ["«Corregir» · «Son nueve», "corregir sin borrar el original"],
27     ["«¿Cuánto arroz hay y en qué bodegas?», "consultar el inventario"],
28     ["«Hoy preparamos cincuenta ajíacos», "pedir insumos por receta"],
29 ];
30
31 export default function Ayuda({ token, usuario, ir }) {
32     const [salud, setSalud] = useState(null);
33     const [busca, setBusca] = useState("");
34     const [reportando, setReportando] = useState(false);
35     const [msg, setMsg] = useState("");
36     const [pedirDetalle, setPedirDetalle] = useState(false);

```

```

37 const [escribiendoAdmin, setEscribiendoAdmin] = useState(false);
38 const [enviandoAdmin, setEnviandoAdmin] = useState(false);
39 const [admin, setAdmin] = useState(null);
40 const [miPerfil, setMiPerfil] = useState(null);
41 // Si lo escrito/dicho no encuentra nada en las preguntas frecuentes ni
42 // en la guía de comandos, la respuesta del agente general - mismo
43 // patrón que el buscador de Bodegas: un solo cuadro para todo, en vez
44 // de dos cuadros separados (uno para el agente, otro para filtrar)
45 // que además se veían mal juntos, uno encima del otro.
46 const [respuestaAgente, setRespuestaAgente] = useState(null);
47 const [escuchando, setEscuchando] = useState(false);
48 const [errorBusca, setErrorBusca] = useState("");
49 const rec = useRef(null);
50 const idEscucha = useRef(0);
51
52 useEffect(() => {
53   pedir("/api/salud", {}, token).then(setSalud).catch(() => setSalud({ api: "caido" }));
54   pedir("/api/soporte/administrador", {}, token).then(setAdmin).catch(() => {});
55   // Para la firma del correo a el administrador (nombre, rol y correo
56   // de quien escribe) - "usuario" (la sesión) solo trae id/nombre/perfil.
57   pedir("/api/usuarios/yo", {}, token).then(setMiPerfil).catch(() => {});
58 }, [token]);
59
60 async function reportarProblema(detalle) {
61   setPedirDetalle(false);
62   if (!detalle || !detalle.trim()) return;
63   setReportando(true);
64   try {
65     await pedir("/api/soporte/reportar", {
66       method: "POST", body: JSON.stringify({ detalle }),
67     }, token);
68     const listo = "Reportado a la mesa de ayuda. Quedó en el registro de trazabilidad.";
69     setMsg(listo);
70     hablar(listo);
71   } catch (e) { setMsg(e.message); hablar(e.message); }
72   setReportando(false);
73 }
74
75 // Reemplaza el enlace mailto: de antes - dependía de que el equipo
76 // tuviera un cliente de correo instalado, y sin uno mostraba el cuadro
77 // genérico del sistema operativo ("seleccionar una aplicación para
78 // abrir este vínculo") en vez de algo propio de CuentaVoz. Este envío
79 // queda en el registro de trazabilidad, con el mismo aviso honesto que
80 // ya usa "Reportar un problema": no promete un correo real que la app
81 // no puede garantizar.
82 async function enviarMensajeAdministrador(mensaje) {
83   setEscribiendoAdmin(false);
84   if (!mensaje || !mensaje.trim()) return;
85   setEnviandoAdmin(true);
86   try {
87     const r = await pedir("/api/soporte/mensaje-administrador", {
88       method: "POST", body: JSON.stringify({ mensaje }),
89     }, token);
90     const nombre = capitalizar(admin?.nombre) || "el administrador";
91     const rol = rolDe(usuario);
92     let listo;
93     if (r?.correo_enviado) {
94       listo = `Correo enviado a ${nombre}.`;
95     } else if (admin?.correo) {
96       await enviarPorEmailJS(admin.correo, capitalizar(usuario?.nombre) || "Usuario",
97         mensaje, rol, miPerfil?.correo) {
98         listo = `Correo enviado a ${nombre}.`;
99       } else if (admin?.correo) {
100         // Último respaldo si EmailJS tampoco responde: abre el correo ya
101         // instalado en el dispositivo con todo listo.
102         const asunto = encodeURIComponent("Mensaje desde CuentaVoz");
103         const cuerpo = encodeURIComponent(`De: ${usuario?.nombre} || "usted"\n\n${mensaje}`);
104         window.location.href = `mailto:${admin.correo}?subject=${asunto}&body=${cuerpo}`;
105         listo = `Mensaje registrado. Se abrió su correo para enviarlo a ${nombre}.`;
106       } else {
107         listo = "Mensaje registrado.";
108       }
109       setMsg(listo);
110       hablar(listo);
111     } catch (e) { setMsg(e.message); hablar(e.message); }
112     setEnviandoAdmin(false);
113   }
114
115 // Devuelve lo que hay que DECIR cuando lo preguntado ya está resuelto
116 // localmente (sin pasar por el agente general), o null si no coincide
117 // con nada. Antes solo se sabía SI coincidía (booleano) y la pantalla
118 // se limitaba a filtrar la lista visualmente, sin decir nada - quien
119 // pregunta por voz sin mirar la pantalla se quedaba sin ninguna
120 // respuesta hablada, justo lo contrario de lo que promete "pregúntele
121 // al agente" en esta misma pantalla.
122 function _respuestaLocal(texto) {

```

```

123     const t = texto.trim().toLowerCase();
124     const faq = FAQ.find(([p, r]) => p.toLowerCase().includes(t) || r.toLowerCase().includes(t));
125     if (faq) return faq[1];
126     const comando = COMANDOS.find(([c, d]) => c.toLowerCase().includes(t) || d.toLowerCase().includes(t));
127     if (comando) return `${comando[0]} sirve para ${comando[1]}.`;
128     return null;
129 }
130
131 function alEscribir(texto) {
132     setBusca(texto);
133     setRespuestaAgente(null);
134 }
135
136 // Los botones de "Soporte en vivo" ("Escribirle al administrador",
137 // "Reportar un problema") también son órdenes reconocibles - se
138 // revisan antes que cualquier otra cosa, porque el agente general no
139 // sabe qué botones hay en esta pantalla en particular (por eso decía
140 // "no puedo enviarle mensajes... esa función no está disponible", aun
141 // cuando el botón para hacerlo está ahí mismo, a la vista).
142 function _accionLocalDeAyuda(texto) {
143     const t = quitarTildes(texto.toLowerCase()).replace(/[.,;:!?;]/g, "").trim();
144     // "\bdescrib" (sin \b de cierre) para que agarre CUALQUIER conjugación
145     // - "escribir", "escribale", y sobre todo "escribirle" (como dice el
146     // botón mismo: "Escribirle al administrador") - un \b\escribir\b no
147     // hacía match dentro de "escribirle" porque ahí "escribir" no es una
148     // palabra completa, es solo el principio de una más larga.
149     if (/\/badministrador\b/.test(t) && /\b(escrib|w*|contactar|mensaje)\b/.test(t)) return "escribir";
150     if (/\/bproblema\b|berror\b/.test(t) && /\breportw*\.test(t)) return "reportar";
151     return null;
152 }
153
154 // Al confirmar (Enter, o al terminar de dictar): primero las órdenes
155 // sobre los botones de esta pantalla; si no aplica ninguna y lo dicho
156 // no encuentra nada en las preguntas frecuentes ni en la guía de
157 // comandos, cae al agente general - un solo cuadro para todo.
158 async function confirmarBusqueda(texto, miId) {
159     if (!texto || !texto.trim()) return;
160     if (miId === undefined) miId = ++idEscucha.current;
161     setBusca(texto);
162     const accion = _accionLocalDeAyuda(texto);
163     if (accion === "escribir" && admin) { setEscribiendoAdmin(true); return; }
164     if (accion === "reportar") { setPedirDetalle(true); return; }
165     const respuestaLocal = _respuestaLocal(texto);
166     if (respuestaLocal) {
167         setRespuestaAgente(null);
168         hablar(respuestaLocal);
169         return;
170     }
171     setErrorBusca("");
172     const r = await preguntarAsistente(texto, "ayuda", token);
173     if (miId !== idEscucha.current) return;
174     setRespuestaAgente(r);
175     hablar(r.respuesta_hablada);
176     if (r.accion === "navegar" && r.destino && ir) {
177         ir(r.destino, { tabInicial: r.pestana || undefined });
178     }
179 }
180
181 function buscarPorVoz() {
182     setErrorBusca("");
183     const miId = ++idEscucha.current;
184     rec.current = escuchar({
185         alTexto: (t) => {
186             if (miId !== idEscucha.current) return;
187             confirmarBusqueda(t, miId);
188         },
189         alEstado: (e) => setEscuchando(e === "escuchando"),
190         alError: setErrorBusca,
191     });
192 }
193
194 const q = busca.trim().toLowerCase();
195 const faqFiltrado = q ? FAQ.filter(([p, r]) =>
196     p.toLowerCase().includes(q) || r.toLowerCase().includes(q)) : FAQ;
197 const comandosFiltrados = q ? COMANDOS.filter(([c, d]) =>
198     c.toLowerCase().includes(q) || d.toLowerCase().includes(q)) : COMANDOS;
199
200 const servicios = [
201     ["API y base de datos", salud?.api === "ok"],
202     ["Datos del inventario cargados", (salud?.stock || 0) > 0],
203     ["Agente Gemini (AI Studio)", Boolean(salud?.gemini)],
204 ];
205
206 return (
207     <Marco titulo="Ayuda · cómo usar CuentaVoz" chip={{ texto: "SOPORTE", tipo: "azul" }}>
208     <div className="card">

```

```

209 <div className="grilla-botones" style={{ justifyContent: "flex-end", marginBottom: 10 }}>
210   <button className="btn borde"
211     onClick={() => window.dispatchEvent(new Event(EVENTO_ABRIR_VIDEO))}>
212     Ver el recorrido en video
213   </button>
214 </div>
215 <p className="rotulo">Pregúntele al agente</p>
216 <div style={{ display: "flex", gap: 10, alignItems: "center" }}>
217   <input value={busca} onChange={(e) => alEscribir(e.target.value)}
218     onKeyDown={(e) => e.key === "Enter" && confirmarBusqueda(busca)}
219     placeholder="pregúntele al agente, o escriba para filtrar las preguntas frecuentes..."
220     aria-label="Pregúntele al agente, o escriba para filtrar las preguntas frecuentes"
221     style={{ flex: 1, padding: "12px 14px",
222       border: "1px solid var(--borde)", borderRadius: 12 }} />
223   <button className={`mic-btn ${escuchando ? "escuchando" : ""}`}
224     style={{ width: 46, height: 46, fontSize: "1.2rem" }}
225     onClick={buscarPorVoz} title="o pregunte por voz"
226     aria-label="Pregúntele al agente por voz"><Icono nombre="microfono" tam={22} /></button>
227 </div>
228 <p className="pista" style={{ marginTop: 8 }}>o pregunte por voz</p>
229 {respuestaAgente && (
230   <
231     <p className="rotulo" style={{ marginTop: 10 }}>CuentaVoz responde</p>
232     <p className="burbuja" aria-live="polite" aria-atomic="true">{respuestaAgente.respuesta_hablada}</p>
233   </>
234 )}
235 {errorBusca && <p className="error" style={{ marginTop: 8 }}>{errorBusca}</p>}
236 </div>
237 <div className="conteo-cols">
238   <div className="card">
239     <h3><span className="icono-kpi" style={{ marginRight: 8 }}>?</span> Preguntas frecuentes</h3>
240     {faqFiltrado.length === 0 && <p className="vacio">Sin resultados para «{busca}».</p>}
241     {faqFiltrado.map([(p, r), i] => (
242       <div key={i} style={{ marginBottom: 12, paddingBottom: 12,
243         borderBottom: i < faqFiltrado.length - 1 ? "1px solid var(--borde)" : "none" }}>
244         <b style={{ color: "var(--azul)", fontSize: ".92rem" }}>{p}</b>
245         <p style={{ fontSize: ".86rem", color: "var(--grafito)", marginTop: 3 }}>{r}</p>
246       </div>
247     ))}
248
249     <h3 style={{ marginTop: 18 }}>
250       <span className="icono-kpi" style={{ marginRight: 8 }}>🗣️</span>
251       Guía rápida de comandos de voz
252     </h3>
253     {comandosFiltrados.length === 0 && <p className="vacio">Sin resultados para «{busca}».</p>}
254     {comandosFiltrados.map([(c, d), i] => (
255       <div className="registro" key={i}>
256         <span style={{ color: "var(--azul)", fontWeight: 700 }}>{c}</span>
257         <span className="cant">{d}</span>
258       </div>
259     ))}
260   </div>
261
262   <div>
263     <div className="card">
264       <h3><span className="icono-kpi" style={{ marginRight: 8 }}>👤</span> Soporte en vivo</h3>
265       {admin?.nombre ? (
266         <
267           <p style={{ fontSize: ".88rem" }}>
268             Administrador de bodega · <span style={{ textTransform: "capitalize" }}>{admin.nombre}</span>
269             <span style={{ color: "var(--verde)", fontWeight: 700 }}> · disponible</span>
270           </p>
271           <div className="grilla-botones">
272             <button className="btn" disabled={enviandoAdmin}
273               onClick={() => setEscribiendoAdmin(true)}
274               style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
275               <Icono nombre="mensajes" tam={18} />
276               {enviandoAdmin ? "Enviando..." : "Escribirle al administrador"}
277             </button>
278           </div>
279         </>
280       ) : admin?.es_usted ? (
281         <p style={{ fontSize: ".88rem" }}>
282           Usted es la administradora de turno: los auxiliares le escriben a
283           usted. Para algo que se salga de su alcance, use la mesa de ayuda
284           de Colsubsidio.
285         </p>
286       ) : (
287         <p style={{ fontSize: ".88rem" }}>
288           No hay otro administrador registrado por ahora. Use la mesa de
289           ayuda de Colsubsidio.
290         </p>
291       )}
292       {msg && <p className="msg-ok">{msg}</p>}
293     <p className="pista" style={{ margin: "10px 0" }}>
294       Mesa de ayuda Colsubsidio · ext. 4040 · 7 por 24

```

```

295     </p>
296     <div className="grilla-botones">
297       {(admin?.nombre || admin?.es_usted) && (
298         <button className="btn borde" onClick={() => ir("mensajes")}
299           style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
300           <Icono nombre="mensajes" tam={18} />
301           Ver mensajes
302         </button>
303       )}
304       <button className="btn oro" disabled={reportando} onClick={() => setPedirDetalle(true)}
305         style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
306         <Icono nombre="advertencia" tam={18} />
307         {reportando ? "Enviando..." : "Reportar un problema"}
308       </button>
309     </div>
310   </div>
311
312   <div className="card" style={{ marginTop: 16 }}>
313     <h3><span className="icono-kpi" style={{ marginRight: 8 }}></span>Estado del sistema</h3>
314     <div className="kpis" style={{ gridTemplateColumns: "1fr" }}>
315       {servicios.map([t, ok], i) => (
316         <div className={ok ? "kpi verde" : "kpi"} key={i}
317           style={{ display: "flex", alignItems: "center", justifyContent: "space-between",
318             padding: "10px 16px" }}>
319           <div className="kpi-cabeza" style={{ marginBottom: 0 }}>
320             <span className="icono-kpi" style={{
321               background: ok ? "var(--verde-bg)" : "#FBE8E6",
322               color: ok ? "var(--verde)" : "#B3261E" }}>
323               {ok ? "✓" : "!"}
324             </span>
325             <small>{t}</small>
326           </div>
327           <b style={{ fontSize: ".92rem", color: ok ? "var(--verde)" : "#B3261E" }}>
328             {ok ? "operativo" : "revisar"}
329           </b>
330         </div>
331       )}
332     </div>
333     {!salud?.gemini && (
334       <p className="pista" style={{ marginTop: 10 }}>
335         Sin la llave de Gemini el agente usa el intérprete local: el flujo
336         funciona igual, pero entiende menos variantes de frase. Ponga
337         GOOGLE_API_KEY en el archivo .env para activarlo.
338       </p>
339     )}
340   </div>
341 </div>
342 </div>
343
344 {pedirDetalle && (
345   <Dialogo titulo="Reportar un problema"
346     mensaje="Describe el problema que encontró:"
347     conCampo conVoz multilinea placeholder="Se cayó el micrófono al confirmar un conteo..."
348     textoAceptar="Enviar"
349     onAceptar={reportarProblema}
350     onCancel={() => setPedirDetalle(false)} />
351 )}
352
353 {escribiendoAdmin && (
354   <Dialogo titulo="Escribirle al administrador"
355     mensaje={ `Para: ${capitalizar(admin?.nombre) || "Administrador"}\nDe: ${capitalizar(usuario?.nombre) ||
"Usted"}\nAsunto: Mensaje desde CuentaVoz\n\nEscriba su mensaje:` }
356     conCampo conVoz multilinea placeholder="Necesito que me asigne la bodega de Kiosco Taquilla..."
357     etiquetaCampo={ `Su mensaje para ${capitalizar(admin?.nombre) || "el administrador"}` }
358     textoAceptar="Enviar"
359     onAceptar={enviarMensajeAdministrador}
360     onCancel={() => setEscribiendoAdmin(false)} />
361 )}
362 </Marco>
363 );
364 }

```

frontend/src/vistas/Mensajes.jsx

248 LÍNEAS

La bandeja del equipo.

```

1 import { useEffect, useRef, useState } from "react";
2 import { pedir } from "../api";
3 import { escuchar, hablar, quitarTildes } from "../voz";
4 import { enviarPorEmailJS, capitalizar, rolDe } from "../correoAdmin";
5 import Marco from "../Marco";
6 import Dialogo from "../Dialogo";
7 import Icono from "../Iconos";

```

```

8
9  /** La bandeja completa de "Soporte en vivo": para un auxiliar, lo que ha
10 escrito y si ya le respondieron; para el administrador, lo que le
11 han escrito y lo que falta por responder. Antes esta lista vivía
12 dentro de la tarjeta de Ayuda, pero con varios auxiliares escribiendo
13 (Luis, Valentina...) esa tarjeta se llenaba y quedaba desordenada -
14 aquí tiene su propio espacio, como Auditoría o Reportes. */
15 export default function Mensajes({ token, usuario }) {
16   const [mensajes, setMensajes] = useState(null);
17   const [miPerfil, setMiPerfil] = useState(null);
18   const [respondiendoId, setRespondiendoId] = useState(null);
19   const [respuestaPrellenada, setRespuestaPrellenada] = useState("");
20   const [enviandoRespuesta, setEnviandoRespuesta] = useState(false);
21   const [msg, setMsg] = useState("");
22   const [escuchandoOrden, setEscuchandoOrden] = useState(false);
23   const [ordenVoz, setOrdenVoz] = useState("");
24   const idEscuchaOrden = useRef(0);
25   const esAdmin = usuario?.perfil === "auditor";
26
27   useEffect(() => {
28     cargar();
29     pedir("/api/usuarios/yo", {}, token).then(setMiPerfil).catch(() => {});
30   }, [token]);
31
32   function cargar() {
33     pedir("/api/soporte/mensajes", {}, token).then(setMensajes).catch(() => setMensajes([]));
34   }
35
36   async function responderMensaje(respuesta) {
37     const id = respondiendoId;
38     setRespondiendoId(null);
39     setRespuestaPrellenada("");
40     if (!respuesta || !respuesta.trim() || !id) return;
41     setEnviandoRespuesta(true);
42     try {
43       const r = await pedir(`/api/soporte/mensajes/${id}/responder`, {
44         method: "POST", body: JSON.stringify({ respuesta }),
45       }, token);
46       if (r?.correo_destinatario) {
47         await enviarPorEmailJS(r.correo_destinatario, capitalizar(usuario?.nombre) || "Usuario",
48           respuesta, rolDe(usuario), miPerfil?.correo);
49       }
50       const listo = `Respuesta enviada a ${capitalizar(r?.destinatario) || "la persona"}`;
51       setMsg(listo);
52       hablar(listo);
53       cargar();
54     } catch (e) { setMsg(e.message); hablar(e.message); }
55     setEnviandoRespuesta(false);
56   }
57
58   // "Respóndele a Stephanie que ya quedó asignada la bodega" - con un
59   // solo mensaje pendiente no hace falta nombrar a nadie, lo dicho es
60   // directamente la respuesta. Con varios, busca el nombre de quien
61   // escribió dentro de la frase (mismo patrón de emparejar por nombre
62   // que ya usa el resto de la app) y separa el resto como el texto de
63   // la respuesta.
64   function _extraerRespuestaPorVoz(texto, listaPendientes) {
65     if (listaPendientes.length === 1) {
66       return { mensaje: listaPendientes[0], texto: texto.trim() };
67     }
68     const t = quitarTildes(texto.toLowerCase());
69     const encontrado = listaPendientes.find(
70       (m) => m.de && t.includes(quitarTildes(m.de.toLowerCase()));
71     if (!encontrado) return { mensaje: null, texto: null };
72     // el nombre de quien escribió no tiene restricción de caracteres al
73     // crear el usuario en Ajustes - sin escapar los que son especiales en
74     // una expresión regular (paréntesis, corchetes...), un nombre con uno
75     // sin su pareja ("Juan (temporal)" tumbaba esto con una excepción sin
76     // capturar y la respuesta por voz se quedaba sin funcionar, sin
77     // ningún aviso de qué pasó.
78     const nombreEscapado = encontrado.de.replace(/[\.\*\?\^\$\{\}\|\[\]\\\]/g, "\\$&");
79     const patron = new RegExp(`^.*?\b${nombreEscapado}\\b\\s*(que|:)?\\s*`, "i");
80     const textoFinal = texto.replace(patron, "").trim() || texto.trim();
81     return { mensaje: encontrado, texto: textoFinal };
82   }
83
84   function alTextoOrdenVoz(texto, miId) {
85     if (miId !== idEscuchaOrden.current || !texto) return;
86     setOrdenVoz(texto);
87     const pendientesLista = (mensajes || []).filter((m) => !m.respuesta);
88     if (pendientesLista.length === 0) return;
89     const { mensaje, texto: respuestaTexto } = _extraerRespuestaPorVoz(texto, pendientesLista);
90     if (!mensaje) {
91       const nombres = pendientesLista.map((m) => capitalizar(m.de)).join(", ");
92       const aviso = `Hay varios mensajes pendientes: ${nombres}. Diga el nombre de quién le escribió.`;
93       setMsg(aviso);

```

```

94     hablar(aviso);
95     return;
96 }
97 setRespuestaPrellenada(respuestaTexto);
98 setRespondiendoId(mensaje.id);
99 setOrdenVoz("");
100 }
101
102 function escucharOrdenVoz() {
103     setMsg("");
104     const miId = ++idEscuchaOrden.current;
105     escuchar({
106         alTexto: (t) => alTextoOrdenVoz(t, miId),
107         alEstado: (e) => setEscuchandoOrden(e === "escuchando"),
108         alError: setMsg,
109     });
110 }
111
112 const original = mensajes?.find((m) => m.id === respondiendoId);
113 const pendientes = mensajes?.filter((m) => !m.respuesta).length ?? 0;
114 const respondidos = mensajes?.filter((m) => m.respuesta).length ?? 0;
115
116 return (
117     <Marco titulo="Mensajes · soporte en vivo"
118     chip={{ texto: esAdmin ? "BANDEJA DEL ADMINISTRADOR" : "MIS MENSAJES", tipo: "azul" }}>
119     {mensajes !== null && mensajes.length > 0 && (
120         <div className="kpi" style={{ marginBottom: 16 }}>
121             <div className="kpi">
122                 <div className="kpi-cabeza">
123                     <span className="icono-kpi">  </span>
124                     <small>{esAdmin ? "Mensajes recibidos" : "Mensajes enviados"}</small>
125                 </div>
126                 <b>{mensajes.length}</b>
127                 <i>en total</i>
128             </div>
129             <div className={`kpi ${pendientes ? "oro" : "verde"} `}>
130                 <div className="kpi-cabeza">
131                     <span className="icono-kpi">{pendientes ? "🕒" : "✅"}</span>
132                     <small>Pendientes de respuesta</small>
133                 </div>
134                 <b>{pendientes}</b>
135                 <i>{esAdmin ? "por responder" : "esperando al administrador"}</i>
136             </div>
137             <div className="kpi verde">
138                 <div className="kpi-cabeza">
139                     <span className="icono-kpi">  </span>
140                     <small>Respondidos</small>
141                 </div>
142                 <b>{respondidos}</b>
143                 <i>ya resueltos</i>
144             </div>
145         </div>
146     )}
147
148     {esAdmin && pendientes > 0 && (
149         <div className="card" style={{ marginBottom: 16 }}>
150             <h3>Responder por voz</h3>
151             <div style={{ display: "flex", gap: 10, alignItems: "center" }}>
152                 <input
153                     value={ordenVoz}
154                     onChange={(e) => setOrdenVoz(e.target.value)}
155                     onKeyDown={(e) => e.key === "Enter" && alTextoOrdenVoz(ordenVoz, idEscuchaOrden.current)}
156                     placeholder={pendientes === 1
157                         ? "Diga o escriba la respuesta..."
158                         : "«Respóndele a Stephanie que ya quedó asignada la bodega...»"}
159                     aria-label="Responder por voz o escribiendo"
160                     style={{ flex: 1, padding: "12px 14px",
161                         border: "1px solid var(--borde)", borderRadius: 12 }} />
162                 <button
163                     className={mic-btn ${escuchandoOrden ? "escuchando" : ""}}
164                     style={{ width: 46, height: 46, fontSize: "1.2rem" }}
165                     onClick={escucharOrdenVoz}
166                     title="responder por voz"
167                     aria-label="Responder por voz"><Icono nombre="microfono" tam={22} /></button>
168             </div>
169             <p className="pista" style={{ marginTop: 8 }}>
170                 {pendientes === 1
171                     ? "Un solo mensaje pendiente: lo que diga se usa directo como respuesta."
172                     : "Varios mensajes pendientes: diga primero el nombre de quién le escribió."}
173             </p>
174         </div>
175     )}
176
177     {msg && <p className="msg-ok" style={{ marginBottom: 10 }}>{msg}</p>}
178
179     {mensajes === null ? (
180         <div className="card"><p className="pista">Cargando...</p></div>
181     ) : mensajes.length === 0 ? (
182         <div className="card">
183             <p className="vacio">

```

```

180         {esAdmin ? "Nadie le ha escrito todavía." : "No ha escrito ningún mensaje todavía."}
181     </p>
182 </div>
183 ) : (
184     mensajes.map((m) => (
185         <div className="card" key={m.id} style={{ marginBottom: 12 }}>
186             <div style={{ display: "flex", alignItems: "flex-start", gap: 12 }}>
187                 <span className="avatar-chico" style={{ marginTop: 2 }}>
188                     {(esAdmin ? m.de : m.para)?.[0]?.toUpperCase() || "?"}
189                 </span>
190                 <div style={{ flex: 1, minWidth: 0 }}>
191                     <div style={{ display: "flex", justify-content: "space-between",
192                         alignItems: "center", gap: 10, flex-wrap: "wrap" }}>
193                         <strong style={{ textTransform: "capitalize", color: "var(--azul-prof)" }}>
194                             {esAdmin ? capitalizar(m.de) : `Para ${capitalizar(m.para)}`}
195                         </strong>
196                         <span style={{ display: "flex", alignItems: "center", gap: 10 }}>
197                             <span className={`etiqueta-actividad ${m.respuesta ? "verde" : "oro"}`}
198                                 style={{ marginLeft: 0 }}>
199                                 {m.respuesta ? "respondido" : "pendiente"}
200                             </span>
201                             <span className="pista">{m.creado}</span>
202                         </span>
203                     </div>
204                     <p style={{ fontSize: ".92rem", marginTop: 6, lineHeight: 1.5 }}>{m.mensaje}</p>
205
206                     {m.respuesta ? (
207                         <div style={{ display: "flex", alignItems: "flex-start", gap: 10,
208                             marginTop: 12, padding: 12, borderTop: "1px solid var(--borde)" }}>
209                             <span className="avatar-chico" style={{ background: "var(--verde-bg)", color: "var(--verde)" }}>
210                                 {(esAdmin ? usuario?.nombre : m.para)?.[0]?.toUpperCase() || "?"}
211                             </span>
212                             <div style={{ flex: 1 }}>
213                                 <div style={{ display: "flex", justify-content: "space-between", alignItems: "baseline" }}>
214                                     <strong style={{ color: "var(--verde)", fontSize: ".85rem" }}>
215                                         {esAdmin ? "Su respuesta" : `Respuesta de ${capitalizar(m.para)}`}
216                                     </strong>
217                                     <span className="pista">{m.respondido}</span>
218                                 </div>
219                                 <p style={{ fontSize: ".9rem", marginTop: 3, lineHeight: 1.5 }}>{m.respuesta}</p>
220                             </div>
221                         </div>
222                     ) : esAdmin ? (
223                         <button className="btn borde" disabled={enviandoRespuesta}
224                             style={{ marginTop: 12, fontSize: ".85rem", padding: "6px 16px" }}
225                             onClick={() => { setRespuestaPrellenada(""); setRespondiendoId(m.id); }}>
226                             Responder
227                         </button>
228                     ) : null}
229                 </div>
230             </div>
231         </div>
232     ))
233 )}
234
235 {respondiendoId && (
236     <Dialogo titulo="Responder mensaje"
237         mensaje={ `Para: ${capitalizar(original?.de) || "la persona"}\nDe: ${capitalizar(usuario?.nombre) ||
"Usted"}\n\nMensaje original:\n${original?.mensaje || ""}\n\nEscriba su respuesta:` }
238         conCampo conVoz multilinea placeholder="Ya quedó asignada esa bodega, revise en unos minutos..."
239         etiquetaCampo=`Su respuesta a ${capitalizar(original?.de) || "la persona"}`
240         valorInicial={respuestaPrellenada}
241         confirmarAlAbrir={Boolean(respuestaPrellenada)}
242         textoAceptar="Enviar"
243         onAceptar={responderMensaje}
244         onCancel={() => { setRespondiendoId(null); setRespuestaPrellenada(""); }} />
245     )}
246 </Marco>
247 );
248 }

```

frontend/src/vistas/MiPerfil.jsx

429 LÍNEAS

La cuenta propia.

```

1 import { useEffect, useState } from "react";
2 import { pedir, BASE, leerToken, borrarSesion, esFalloRed } from "../api";
3 import { cambiarClave, cerrarSesionLocal, tieneMFAActiva, desactivarMFA } from "../cognito";
4 import { configurarVoz, hablar } from "../voz";
5 import { leerPreferencias, guardarPreferencias } from "../accesibilidad";
6 import ChecklistClave, { claveCumplePolitica } from "../ChecklistClave";
7 import ConfigurarMFA from "../ConfigurarMFA";
8 import Marco from "../Marco";

```

```

9 import Dialogo from "../Dialogo";
10 import AsistenteVoz from "../AsistenteVoz";
11
12 function formatoAcceso(fechaStr) {
13   if (!fechaStr) return "-";
14   const [fecha, hora] = fechaStr.split(" ");
15   const hoy = new Date().toISOString().slice(0, 10);
16   return fecha === hoy ? `hoy ${hora}` : `${fecha.slice(5).replace("-", "/")} ${hora}`;
17 }
18
19 /* — Interruptor simple, sin librería (mismo patrón que Ajustes.jsx) — */
20 function Interruptor({ activo, onChange, etiqueta }) {
21   return (
22     <button
23       onClick={() => onChange(!activo)}
24       role="switch"
25       aria-checked={activo}
26       aria-label={etiqueta}
27       style={{
28         width: 44, height: 24, borderRadius: 14, padding: 2,
29         background: activo ? "var(--verde)" : "#C3CAD6",
30         display: "flex", justifyContent: activo ? "flex-end" : "flex-start",
31       }}
32     >
33     <span style={{ width: 20, height: 20, borderRadius: "50%", background: "#fff" }} />
34     </button>
35   );
36 }
37
38 // Lo que YA se puede cambiar por voz aquí (la voz, la velocidad, la
39 // confirmación hablada, y preguntar por los datos de la cuenta). La clave
40 // y la foto son a propósito solo manuales, por seguridad - el agente lo
41 // explica si se le pide, en vez de fingir que puede hacerlo.
42 const _EJEMPLOS_PERFIL = [
43   "cambia mi voz a puck", "usa la voz aoede", "habla más lento", "velocidad normal",
44   "activa la confirmación hablada", "desactiva la confirmación hablada",
45   "¿cuándo fue mi último acceso?", "¿cuántos días le quedan a mi PIN?",
46   "¿qué bodegas tengo asignadas?",
47 ];
48
49 export default function MiPerfil({ token, ir }) {
50   const [datos, setDatos] = useState(null);
51   const [bodegas, setBodegas] = useState([]);
52   const [msg, setMsg] = useState("");
53   const [claveActual, setClaveActual] = useState("");
54   const [claveNueva, setClaveNueva] = useState("");
55   const [claveNueva2, setClaveNueva2] = useState("");
56   const [err, setErr] = useState("");
57   const [fotoUrl, setFotoUrl] = useState(null);
58   const [confirmarCierre, setConfirmarCierre] = useState(false);
59   const [voces, setVoces] = useState([]);
60   // verificación en dos pasos (MFA con app autenticadora)
61   const [mfaActiva, setMfaActiva] = useState(null); // null = todavía no se sabe
62   const [configurandoMFA, setConfigurandoMFA] = useState(false);
63   const [confirmarQuitarMFA, setConfirmarQuitarMFA] = useState(false);
64   // Alto contraste / tamaño de letra - preferencia de este equipo
65   // (localStorage), no de la cuenta, así que se aplica al toque, sin
66   // pasar por "Guardar cambios" del resto de la pantalla.
67   const [accesibilidad, setAccesibilidad] = useState(() => leerPreferencias());
68   function cambiarAccesibilidad(cambios) {
69     const nuevas = { ...accesibilidad, ...cambios };
70     setAccesibilidad(nuevas);
71     guardarPreferencias(nuevas);
72   }
73
74   // <img src> no puede mandar el header Authorization, y /foto exige
75   // sesion - por eso se trae como blob autenticado (mismo patron que
76   // descargarReporte) en vez de apuntar la imagen directo al endpoint.
77   function cargarFoto() {
78     fetch(`${BASE}/api/usuarios/yo/foto`, {
79       headers: { Authorization: `Bearer ${token} || leerToken()` },
80     }).then((r) => (r.ok ? r.blob() : null))
81     .then((b) => setFotoUrl((prev) => {
82       if (prev) URL.revokeObjectURL(prev);
83       return b ? URL.createObjectURL(b) : null;
84     })))
85     .catch(() => setFotoUrl(null));
86   }
87
88   useEffect(() => {
89     pedir("/api/usuarios/yo", {}, token).then(setDatos).catch(() => {});
90     pedir("/api/usuarios/yo/bodegas", {}, token).then(setBodegas).catch(() => {});
91     pedir("/api/voz/voces", {}, token).then((r) => setVoces(r.voces)).catch(() => {});
92     tieneMFAActiva().then(setMfaActiva).catch(() => setMfaActiva(false));
93     cargarFoto();
94     // eslint-disable-next-line react-hooks/exhaustive-deps

```

```

95   }, [token]);
96
97   async function guardar() {
98     setErr(""); setMsg("");
99     try {
100       await pedir("/api/usuarios/yo", { method: "PUT", body: JSON.stringify(datos) }, token);
101       await pedir("/api/usuarios/yo/preferencias", {
102         method: "PUT", body: JSON.stringify({
103           idioma_voz: datos.idioma_voz, velocidad_voz: datos.velocidad_voz,
104           confirmacion_hablada: datos.confirmacion_hablada,
105         }),
106       }, token);
107       setMsg("Cambios guardados correctamente.");
108     } catch (e) { setErr(e.message); }
109   }
110   async function subirFoto(e) {
111     const f = e.target.files?.[0];
112     if (!f) return;
113     setErr(""); setMsg("");
114     const form = new FormData();
115     form.append("foto", f);
116     try {
117       const r = await fetch(`${BASE}/api/usuarios/yo/foto`, {
118         method: "POST",
119         headers: { Authorization: `Bearer ${token || leerToken()}` },
120         body: form,
121       });
122       if (!r.ok) throw new Error("No se pudo subir la foto.");
123       cargarFoto();
124       window.dispatchEvent(new Event("cuentavoz:foto-actualizada"));
125       setMsg("Foto actualizada.");
126     } catch (e2) {
127       setErr(esFalloRed(e2)
128         ? "Sin conexión con el servidor. Revise el Wi-Fi e intente de nuevo."
129         : e2.message);
130     }
131   }
132   async function cambiarClavePropia() {
133     setErr(""); setMsg("");
134     if (!claveActual) return setErr("Digite su clave actual.");
135     if (!claveCumplePolitica(claveNueva)) return setErr("La clave nueva no cumple todos los requisitos.");
136     if (claveNueva !== claveNueva2) return setErr("Las dos claves no coinciden.");
137     try {
138       // habla directo con Cognito (no con este backend, que nunca ve la
139       // clave) - a diferencia del PIN de antes, esto NO invalida la sesión
140       // actual, así que no hace falta recargar la página.
141       await cambiarClave(claveActual, claveNueva);
142       setClaveActual(""); setClaveNueva(""); setClaveNueva2("");
143       setMsg("Clave actualizada.");
144       setDatos((d) => ({ ...d, pin_vence_en_dias: 90 }));
145       pedir("/api/usuarios/yo/marcar-clave-cambiada", { method: "POST" }, token).catch(() => {});
146     } catch (e) { setErr(e.message); }
147   }
148   async function cerrarTodas() {
149     try {
150       await pedir("/api/usuarios/yo/cerrar-todas", { method: "POST" }, token);
151       // revoca las sesiones en TODOS los dispositivos, incluido este -
152       // por consistencia se cierra tambien aqui en vez de seguir en la
153       // pantalla con un token que ya no sirve para pedir uno nuevo.
154       cerrarSesionLocal();
155       borrarSesion();
156       window.location.reload();
157     } catch (e) { setErr(e.message); }
158   }
159   function alActivarMFA() {
160     setConfigurandoMFA(false);
161     setMfaActiva(true);
162     setMsg("Verificación en dos pasos activada.");
163   }
164   async function quitarMFA() {
165     setErr(""); setMsg("");
166     try {
167       await desactivarMFA();
168       setMfaActiva(false);
169       setMsg("Verificación en dos pasos desactivada.");
170     } catch (e) { setErr(e.message); }
171   }
172   function probarVoz() {
173     hablar("Hola, soy CuentaVoz. Así voy a sonar de ahora en adelante.", { forzar: true });
174   }
175
176   // Solo actualiza en memoria (para que "Probar voz" suene distinto de una
177   // vez): el guardado real de estas preferencias queda para "Guardar
178   // cambios", junto con los datos personales - un solo botón, un solo
179   // guardado, como el resto de la pantalla.
180   function elegirPreferencia(cambios) {

```

```

181     const nuevos = { ...datos, ...cambios };
182     setDatos(nuevos);
183     configurarVoz({ idioma: nuevos.idioma_voz, velocidad: nuevos.velocidad_voz,
184                   confirmacionHablada: nuevos.confirmacion_hablada });
185 }
186
187 // El agente ("Pregúntele al agente", arriba) Sí puede cambiar la voz, la
188 // velocidad y la confirmación hablada directo en la base de datos - a
189 // diferencia de elegirPreferencia() (que solo actualiza en memoria hasta
190 // "Guardar cambios"), aquí ya quedó guardado en el servidor, así que se
191 // trae de nuevo (en vez de adivinar qué cambió) y se aplica de una con
192 // configurarVoz(), para que la SIGUIENTE frase que diga el agente ya
193 // suene con la preferencia nueva sin esperar a recargar la página.
194 function recargarPreferencias() {
195     pedir("/api/usuarios/yo", {}, token).then((d) => {
196         setDatos(d);
197         configurarVoz({ idioma: d.idioma_voz, velocidad: d.velocidad_voz,
198                       confirmacionHablada: d.confirmacion_hablada });
199     }).catch(() => {});
200 }
201
202 if (!datos) return <Marco titulo="Mi perfil"><p className="cargando">Cargando...</p></Marco>;
203
204 return (
205     <Marco titulo="Mi perfil · datos personales"
206       chip={{ texto: "SESIÓN ACTIVA", tipo: "verde" }}>
207       <AsistenteVoz token={token} vista="perfil" ir={ir} ejemplos={EJEMPLOS_PERFIL}
208         placeholder="cambia mi voz a puck, ¿cuándo fue mi último acceso?..."
209         alActualizar={recargarPreferencias} />
210       <div className="perfil-cols">
211         <div className="card mic-caja">
212           <fotoUrl ? (
213             <img src={fotoUrl} alt=""
214               style={{ width: 110, height: 110, borderRadius: "50%",
215                 objectFit: "cover", border: "3px solid var(--azul)" }} />
216           ) : (
217             <span className="avatar-grande">{{datos.nombre || "?"}[0].toUpperCase()}</span>
218           )
219           <b style={{ textTransform: "capitalize" }}>{{datos.nombre}}</b>
220           <small>{{datos.perfil === "auditor" ? "Administrador de bodega"
221             : "Auxiliar de inventarios"}}</small>
222           <label className="btn borde" style={{ textAlign: "center" }}>
223             Cambiar foto
224             <input type="file" accept="image/*" hidden onChange={subirFoto} />
225           </label>
226         </div>
227
228         <div className="card">
229           <h3>Datos personales</h3>
230           <div className="campos-2col">
231             {[["Nombre completo", "nombre"], ["Correo corporativo", "correo"],
232               ["Teléfono de contacto", "telefono"], ["Código de empleado", "codigo"]].map(([l, k]) => (
233               <div key={k} style={{ marginBottom: 10 }}>
234                 <label className="pista" htmlFor={`campo-perfil-${k}`}>{{l}}</label>
235                 <input id={`campo-perfil-${k}`} value={{datos[k] || ""}}
236                   onChange={(e) => setDatos({ ...datos, [k]: e.target.value })}}
237                 style={{ width: "100%", padding: "11px 13px",
238                   border: "1px solid var(--borde)", borderRadius: 12 }} />
239               </div>
240             ))}
241           </div>
242           <label className="pista" id="etiqueta-bodegas-asignadas">Bodegas asignadas</label>
243           <div className="chips" style={{ marginTop: 6 }} aria-labelledby="etiqueta-bodegas-asignadas">
244             {bodegas.length === 0 ? (
245               <span className="pista">Sin bodegas asignadas todavía.</span>
246             ) : bodegas.map((b) => <span key={b.id} className="chip">{{b.bodega}}</span>)}
247           </div>
248         </div>
249       </div>
250
251       <div className="dos-columnas">
252         <div className="card">
253           <h3>Seguridad de la cuenta</h3>
254           <p style={{ fontSize: ".87rem", marginBottom: 4 }}>
255             Último acceso: <b>{{formatoAcceso(datos.ultimo_acceso)}}</b>
256           </p>
257           <p style={{ fontSize: ".87rem", marginBottom: 14 }}>
258             Su PIN vence en <b>{{datos.pin_vence_en_dias}} días</b>
259           </p>
260
261           {{datos.pin_vence_en_dias <= 14 && (
262             <div className="banner" style={{ marginBottom: 14 }}>
263               <span className="ico">!</span>
264               <span>
265                 <b>Su clave vence pronto</b>
266                 <span>Le quedan {{datos.pin_vence_en_dias}} días. Cámbiela abajo cuando quiera.</span>

```

```

267         </span>
268     </div>
269 })
270
271 <input type="password" placeholder="Clave actual" value={claveActual}
272     autoComplete="current-password"
273     onChange={(e) => setClaveActual(e.target.value)}
274     aria-label="Clave actual"
275     style={{ width: "100%", padding: "11px 13px", marginBottom: 8,
276         border: "1px solid var(--borde)", borderRadius: 12 }} />
277 <input type="password" placeholder="Clave nueva"
278     value={claveNueva}
279     autoComplete="new-password"
280     onChange={(e) => setClaveNueva(e.target.value)}
281     aria-label="Clave nueva"
282     style={{ width: "100%", padding: "11px 13px", marginBottom: 4,
283         border: "1px solid var(--borde)", borderRadius: 12 }} />
284 <ChecklistClave clave={claveNueva} />
285 <input type="password" placeholder="Confirmar clave nueva" value={claveNueva2}
286     autoComplete="new-password"
287     onChange={(e) => setClaveNueva2(e.target.value)}
288     aria-label="Confirmar clave nueva"
289     style={{ width: "100%", padding: "11px 13px",
290         border: "1px solid var(--borde)", borderRadius: 12 }} />
291 <div className="grilla-botones">
292     <button className="btn borde" onClick={cambiarClavePropia}>Cambiar clave</button>
293 </div>
294
295 <div className="grilla-botones" style={{ marginTop: 10 }}>
296     <button className="btn gris" onClick={() => setConfirmarCierre(true)}>
297         Cerrar sesión en todos los dispositivos
298     </button>
299 </div>
300 <p className="pista" style={{ marginTop: 8 }}>
301     La identidad y la clave las administra AWS Cognito, no CuentaVoz -
302     este backend nunca ve ni guarda su clave.
303 </p>
304
305 <hr style={{ border: "none", borderTop: "1px solid var(--borde)", margin: "18px 0" }} />
306 <h3 style={{ fontSize: "1rem" }}>Verificación en dos pasos</h3>
307 {mfaActiva === null ? (
308     <p className="pista">Comprobando...</p>
309 ) : configurandoMFA ? (
310     <ConfigurarMFA nombreUsuario={datos.nombre} onActivado={alActivarMFA}
311         onCancel={() => setConfigurandoMFA(false)} />
312 ) : mfaActiva ? (
313     <>
314         <p className="pista" style={{ color: "var(--verde)", fontStyle: "normal" }}>
315             ✓ Activada con una app autenticadora.
316         </p>
317         <div className="grilla-botones">
318             <button className="btn gris" onClick={() => setConfirmarQuitarMFA(true)}>Desactivar</button>
319         </div>
320     </>
321 ) : (
322     <>
323         <p className="pista">
324             Pida un código de 6 dígitos generado en su celular además de la clave, al ingresar. Opcional.
325         </p>
326         <div className="grilla-botones">
327             <button className="btn borde" onClick={() => setConfigurandoMFA(true)}>Activar</button>
328         </div>
329     </>
330 )}
331 </div>
332
333 <div className="card">
334     <h3>Preferencias de voz</h3>
335     <label className="pista" htmlFor="campo-voz">Voz de CuentaVoz</label>
336     <select id="campo-voz" value={datos.idioma_voz}
337         onChange={(e) => elegirPreferencia({ idioma_voz: e.target.value })}
338         style={{ width: "100%", marginTop: 6, marginBottom: 8, padding: "10px 12px",
339             border: "1px solid var(--borde)", borderRadius: 10 }}>
340         {voces.map((v) => (
341             <option key={v.clave} value={v.clave}>{v.nombre} – {v.etiqueta}</option>
342         ))}
343     </select>
344     <div className="grilla-botones" style={{ marginBottom: 6 }}>
345         <button className="btn borde" onClick={probarVoz}>🔊 Probar voz</button>
346     </div>
347     <p className="pista" style={{ marginBottom: 12 }}>
348         Voz neuronal real (no la del navegador): suena humana y fluida.
349         El reconocimiento de lo que usted dice sigue siempre en español
350         de Colombia; esto solo cambia cómo suena CuentaVoz al responder.
351     </p>
352

```

```

353     <label className="pista" id="etiqueta-velocidad">Velocidad de la respuesta</label>
354     <div className="grilla-botones" style={{ marginTop: 6, marginBottom: 12 }}
355       role="group" aria-labelledby="etiqueta-velocidad">
356       {[ "lenta", "normal", "rapida" ].map((v2) => (
357         <button key={v2}
358           className={`btn ${datos.velocidad_voz === v2 ? "" : "borde"}`}
359           onClick={() => elegirPreferencia({ velocidad_voz: v2 })}
360           aria-pressed={datos.velocidad_voz === v2}>
361         {v2[0].toUpperCase() + v2.slice(1)}
362       </button>
363     )]}
364     </div>
365
366     <label className="pista" id="etiqueta-confirmacion">Confirmación hablada</label>
367     <div className="grilla-botones" style={{ marginTop: 6, marginBottom: 18 }}
368       role="group" aria-labelledby="etiqueta-confirmacion">
369       <button className={`btn ${datos.confirmacion_hablada ? "" : "borde"}`}
370         onClick={() => elegirPreferencia({ confirmacion_hablada: true })}
371         aria-pressed={!datos.confirmacion_hablada}>
372         Activada
373       </button>
374       <button className={`btn ${!datos.confirmacion_hablada ? "" : "borde"}`}
375         onClick={() => elegirPreferencia({ confirmacion_hablada: false })}
376         aria-pressed={!datos.confirmacion_hablada}>
377         Desactivada
378       </button>
379     </div>
380
381     {err && <p className="error" role="alert">{err}</p>}
382     {msg && <p className="msg-ok" aria-live="polite">{msg}</p>}
383     <button className="btn" style={{ width: "100%" }} onClick={guardar}>
384       Guardar cambios
385     </button>
386   </div>
387 </div>
388
389 <div className="card">
390   <h3>Accesibilidad</h3>
391   <div className="grilla-botones" style={{ alignItems: "center", justifyContent: "flex-start", gap: 10 }}>
392     <Interruptor activo={!accesibilidad.contraste} etiqueta="Alto contraste"
393       onChange={(v) => cambiarAccesibilidad({ contraste: v })} />
394     <span className="pista">Alto contraste</span>
395   </div>
396   <label className="pista" htmlFor="campo-tamano-letra" style={{ display: "block", marginTop: 12 }}>
397     Tamaño de letra
398   </label>
399   <select id="campo-tamano-letra" value={accesibilidad.tamano || "normal"}
400     onChange={(e) => cambiarAccesibilidad({ tamano: e.target.value === "normal" ? "" : e.target.value })}
401     style={{ width: "100%", maxWidth: 260, marginTop: 6, padding: "10px 12px",
402       border: "1px solid var(--borde)", borderRadius: 10 }}>
403     <option value="normal">Normal</option>
404     <option value="grande">Grande</option>
405     <option value="mas-grande">Más grande</option>
406   </select>
407   <p className="pista" style={{ marginTop: 10 }}>
408     Se aplica de una vez en este equipo, sin necesidad de guardar.
409   </p>
410 </div>
411
412 {confirmarCierre && (
413   <Dialogo titulo="Cerrar sesión en todos los dispositivos"
414     mensaje="Esto cierra la sesión en todas las tabletas donde haya entrado, incluida esta. ¿Continuar?"
415     textoAceptar="Cerrar todas" peligro
416     onAceptar={() => { setConfirmarCierre(false); cerrarTodas(); }}
417     onCancel={() => setConfirmarCierre(false)} />
418 )}
419
420 {confirmarQuitarMFA && (
421   <Dialogo titulo="Desactivar verificación en dos pasos"
422     mensaje="La próxima vez que entre, bastará con la clave - ya no se pedirá el código de la app autenticadora.
¿Continuar?"
423     textoAceptar="Desactivar" peligro
424     onAceptar={() => { setConfirmarQuitarMFA(false); quitarMFA(); }}
425     onCancel={() => setConfirmarQuitarMFA(false)} />
426 )}
427 </Marco>
428 );
429 }

```

```

1 import { useEffect, useRef, useState } from "react";
2 import { pedir } from "../api";
3
4 export default function CerrarSesion({ token, sessionId, onCancel, onSalir }) {
5   const [pend, setPend] = useState(null);
6   const [noSePudoVerificar, setNoSePudoVerificar] = useState(false);
7   const modalRef = useRef(null);
8
9   // Igual que Dialogo.jsx: Escape cierra (aquí, "seguir trabajando" - la
10  // opción que no pierde nada), y el modal recibe el foco al aparecer. Sin
11  // esto, quien navega con teclado o con un lector de pantalla no tenía
12  // forma de descartar este diálogo en particular, aunque todos los demás
13  // de la app sí la ofrecen.
14  useEffect(() => {
15    function alTecla(e) { if (e.key === "Escape") onCancel(); }
16    window.addEventListener("keydown", alTecla);
17    return () => window.removeEventListener("keydown", alTecla);
18  }, [onCancel]);
19
20  useEffect(() => {
21    // el modal no existe en el DOM hasta que "pend" resuelve (ver el
22    // "if (!pend) return null" más abajo) - el foco solo puede entrar una
23    // vez que de verdad aparece, no en el montaje del componente.
24    if (pend) modalRef.current?.focus();
25  }, [pend]);
26
27  useEffect(() => {
28    setNoSePudoVerificar(false);
29    pedir(`/api/sesiones/${sessionId}/avance`, {}, token)
30      .then((a) => setPend({ bodega: a.bodega, hechas: a.hechas }))
31      // si la comprobación falla (sin conexión, sesión vencida...) no hay
32      // que asumir que no hay trabajo pendiente: eso es la respuesta más
33      // tranquilizadora posible justo cuando menos se sabe. Se avisa que
34      // no se pudo comprobar en vez de decir "puede salir sin problema"
35      // sin tener manera de saberlo.
36      .catch(() => { setNoSePudoVerificar(true); setPend({ bodega: null, hechas: 0 }); });
37  }, [sessionId, token]);
38
39  if (!pend) return null;
40  const hayTrabajo = Boolean(pend.bodega);
41
42  return (
43    <div className="overlay" onClick={onCancel}>
44      <div className="modal" onClick={(e) => e.stopPropagation()}
45        role="dialog" aria-modal="true" aria-labelledby="cerrar-sesion-titulo"
46        ref={modalRef} tabIndex=-1>
47        <h2 id="cerrar-sesion-titulo">¿Cerrar la sesión?</h2>
48        {noSePudoVerificar ? (
49          <p>
50            No se pudo comprobar si tiene trabajo sin guardar. Si tiene una bodega abierta,
51            lo contado hasta ahora ya quedó guardado y podrá continuar al volver.
52          </p>
53        ) : hayTrabajo ? (
54          <p>
55            Tiene <b>{pend.bodega}</b> abierta con {pend.hechas} referencias contadas.
56          </p>
57          <div className="aviso">
58            Lo registrado queda guardado y podrá continuar al volver.
59          </div>
60        ) : (
61          <p>No tiene trabajo pendiente: puede salir sin problema.</p>
62        )}
63        <div className="botones">
64          <button className="btn borde" onClick={onCancel}>Seguir trabajando</button>
65          <button className="btn" onClick={onSalir}>Cerrar sesión</button>
66        </div>
67      </div>
68    </div>
69  );
70 }
71
72 }

```

17.6 Pruebas

backend/tests/conftest.py

135 LÍNEAS

La base en memoria que usan todas.

```

1  """Infraestructura aislada para las pruebas de regresión de la API."""
2  from __future__ import annotations
3
4  import importlib
5  import sys
6  from pathlib import Path
7  from typing import Iterator
8
9  import pytest
10 from fastapi.testclient import TestClient
11
12
13 BACKEND_DIR = Path(__file__).resolve().parents[1]
14 if str(BACKEND_DIR) not in sys.path:
15     sys.path.insert(0, str(BACKEND_DIR))
16
17
18 def _descargar_modulos_backend() -> None:
19     """Fuerza que cada prueba lea DB_URL antes de crear el motor SQLAlchemy."""
20     directos = {"bd", "modelos", "seguridad", "main", "reportes"}
21     prefijos = ("agente", "servicios")
22     for nombre in list(sys.modules):
23         if nombre in directos or nombre.startswith(prefijos):
24             sys.modules.pop(nombre, None)
25
26
27 def _reclamos_prueba(token: str) -> dict[str, object] | None:
28     """Reemplaza la verificación real de un access token de Cognito (que
29     exigiría red y un User Pool de verdad) por una regla trivial: el
30     "token" que emite /api/ingresar de prueba (ver mas abajo) ES el
31     nombre de usuario, y aquí simplemente se acepta tal cual - las
32     pruebas de este archivo verifican la lógica de negocio (permisos,
33     asignaciones, agente...), no la integración real con Cognito, que
34     ya se probó a mano contra el User Pool real."""
35     if not token:
36         return None
37     return {"username": token, "token_use": "access", "client_id": "prueba"}
38
39
40 @pytest.fixture
41 def app_modulos(tmp_path: Path, monkeypatch: pytest.MonkeyPatch) -> Iterator[object]:
42     """Importa una aplicación nueva conectada a un SQLite temporal por prueba."""
43     ruta_db = tmp_path / "cuentavoz-pruebas.db"
44     monkeypatch.setenv("DB_URL", f"sqlite:///({ruta_db.as_posix()})")
45     monkeypatch.setenv("COGNITO_REGION", "us-east-2")
46     monkeypatch.setenv("COGNITO_USER_POOL_ID", "us-east-2-pruebas")
47     monkeypatch.setenv("COGNITO_APP_CLIENT_ID", "prueba")
48     # datos_regresion (mas abajo) espera encontrar a "luis" ya creado por
49     # arranque() - explicito aqui, no implicito via un .env de desarrollo
50     # que quien corra esto en otra maquina o en CI no tiene por que tener.
51     monkeypatch.setenv("SEMBRAR_DEMO", "1")
52     _descargar_modulos_backend()
53
54     modulo = importlib.import_module("main")
55     # main.py hace "from seguridad import usuario_actual, ..." (no "import
56     # seguridad"), así que hay que importar el modulo aparte para parchar
57     # _reclamos_cognito - sys.modules ya tiene la MISMA instancia que main
58     # uso, importlib no la vuelve a crear.
59     modulo_seguridad = importlib.import_module("seguridad")
60     monkeypatch.setattr(modulo_seguridad, "_reclamos_cognito", _reclamos_prueba)
61
62     from fastapi import Form
63
64     @modulo.app.post("/api/ingresar")
65     def _ingresar_prueba(username: str = Form(...), password: str = Form(...)):
66         """Solo para pruebas: en la app real el login lo hace Cognito
67         directo desde el frontend (ver cognito.js), este backend nunca
68         recibe una clave. Emite un "token" de prueba que _reclamos_prueba
69         acepta, para no reescribir cada prueba que necesita una sesión."""
70         from fastapi import HTTPException
71         with modulo.Sesion() as s:
72             u = s.query(modulo.Usuario).filter_by(nombre=username.lower()).first()
73             if u is None or not u.activo:
74                 raise HTTPException(401, "Usuario o clave incorrectos.")
75             return {"token": u.nombre, "perfil": u.perfil}
76
77     try:
78         yield modulo
79     finally:
80         # Libera el archivo SQLite antes de que pytest elimine tmp_path en Windows.
81         modulo.Sesion.kw["bind"].dispose() if hasattr(modulo.Sesion, "kw") else None
82         _descargar_modulos_backend()
83
84
85 @pytest.fixture
86 def client(app_modulos: object) -> Iterator[TestClient]:

```

```

87     """Cliente con el ciclo de vida de FastAPI activo (incluye iniciar_bd)."""
88     with TestClient(app_modulos.app) as cliente:
89         yield cliente
90
91
92 @pytest.fixture
93 def datos_regresion(client: TestClient) -> dict[str, object]:
94     """Crea un auxiliar, dos bodegas y el mismo articulo en ambas bodegas."""
95     from bd import Sesion
96     from modelos import ( # Se importan después de fijar DB_URL en app_modulos.
97         Articulo,
98         AsignacionBodega,
99         Bodega,
100         StockSistema,
101         Usuario,
102     )
103
104     with Sesion() as sesion:
105         auxiliar = sesion.query(Usuario).filter_by(nombre="luis").one()
106         asignada = Bodega(nombre_oficial="BODEGA ASIGNADA")
107         no_asignada = Bodega(nombre_oficial="BODEGA RESTRINGIDA")
108         articulo = Articulo(codigo="ART-REG-001", nombre_oficial="ARTICULO REGRESION",
109                             unidad_medida="unidad")
110         sesion.add_all([asignada, no_asignada, articulo])
111         sesion.flush()
112         sesion.add(AsignacionBodega(usuario_id=auxiliar.id, bodega_id=asignada.id))
113         sesion.add_all([
114             StockSistema(articulo_codigo=articulo.codigo, bodega_id=asignada.id,
115                           cantidad_sd=100),
116             StockSistema(articulo_codigo=articulo.codigo, bodega_id=no_asignada.id,
117                           cantidad_sd=200),
118         ])
119         sesion.commit()
120         datos = {
121             "auxiliar_id": auxiliar.id,
122             "bodega_asignada_id": asignada.id,
123             "bodega_no_asignada_id": no_asignada.id,
124             "bodega_asignada": asignada.nombre_oficial,
125             "bodega_no_asignada": no_asignada.nombre_oficial,
126             "articulo_codigo": articulo.codigo,
127         }
128
129     respuesta = client.post(
130         "/api/ingresar",
131         data={"username": "luis", "password": "StockXperts1"},
132     )
133     assert respuesta.status_code == 200, respuesta.text
134     datos["headers"] = {"Authorization": f"Bearer {respuesta.json()['token']}"}
135     return datos

```

backend/tests/test_cerebro.py

85 LÍNEAS

El orquestador del agente.

```

1  """Regresiones del «pensar» del agente: la limpieza de negaciones (para que
2  "no quiero preparar arroz" nunca se lea como un pedido de arroz) y la
3  degradación a respaldo local cuando no hay llave de Gemini - la garantía
4  de que la demo no se cae por falta de internet."""
5  from __future__ import annotations
6
7  import pytest
8
9  from agente.cerebro import _limpiar_si_niega, pensar
10
11
12 # ----- _limpiar_si_niega -----
13
14 def test_niega_pedido_borra_preparacion_y_baja_intencion():
15     datos = {"intencion": "pedir", "preparacion": "ARROZ", "porciones": 4,
16             "articulo_texto": None, "cantidad": None}
17     _limpiar_si_niega("no quiero preparar arroz", datos)
18     assert datos["intencion"] == "explicar"
19     assert datos["preparacion"] is None
20     assert datos["porciones"] is None
21
22
23 def test_niega_conteo_borra_articulo_y_cantidad():
24     datos = {"intencion": "contar", "articulo_texto": "ARROZ", "cantidad": 5,
25             "preparacion": None, "porciones": None}
26     _limpiar_si_niega("ya no necesito eso", datos)
27     assert datos["intencion"] == "explicar"
28     assert datos["articulo_texto"] is None
29     assert datos["cantidad"] is None

```

```

30
31
32 @pytest.mark.parametrize("frase", [
33     "cancela el ajiao", "cancelar el pedido", "quitale de la lista",
34     "no me sirve ese", "olvida eso", "no es arroz",
35 ])
36 def test_frases_de_negacion_reconocidas(frase):
37     datos = {"intencion": "pedir", "preparacion": "ALGO", "porciones": 1,
38             "articulo_texto": None, "cantidad": None}
39     _limpiar_si_niega(frase, datos)
40     assert datos["preparacion"] is None
41
42
43 def test_frase_sin_negacion_no_se_toca():
44     datos = {"intencion": "pedir", "preparacion": "ARROZ", "porciones": 4,
45             "articulo_texto": None, "cantidad": None}
46     _limpiar_si_niega("hoy preparamos arroz para cuatro", datos)
47     assert datos["intencion"] == "pedir"
48     assert datos["preparacion"] == "ARROZ"
49     assert datos["porciones"] == 4
50
51
52 def test_intencion_distinta_de_pedir_o_contar_no_se_degrada():
53     # una negación limpia los campos igual, pero no toca intenciones que
54     # ya no son "pedir"/"contar" (ej. "consultar" no debe volverse "explicar").
55     datos = {"intencion": "consultar", "preparacion": None, "porciones": None,
56             "articulo_texto": "ARROZ", "cantidad": None}
57     _limpiar_si_niega("no quiero ese, mejor cancelalo", datos)
58     assert datos["intencion"] == "consultar"
59     assert datos["articulo_texto"] is None
60
61
62 # ----- pensar(): respaldo sin Gemini -----
63
64 def test_pensar_sin_llave_cae_al_interprete_local(monkeypatch):
65     """Sin GOOGLE_API_KEY, pensar() no debe lanzar ni bloquear la
66     conversación: debe devolver exactamente lo que interpretaría el
67     intérprete local, con su propio mensaje hablado de aviso."""
68     monkeypatch.delenv("GOOGLE_API_KEY", raising=False)
69     import agente.cerebro as cerebro
70     monkeypatch.setattr(cerebro, "_cliente", None)
71
72     resultado = pensar("Bodega activa: ninguna.", "hay noventa cazuelas")
73
74     assert resultado["intencion"] == "contar"
75     assert resultado["cantidad"] == 90
76     assert "llave" in resultado["respuesta_hablada"].lower()
77
78
79 def test_pensar_sin_llave_reconoce_confirmacion(monkeypatch):
80     monkeypatch.delenv("GOOGLE_API_KEY", raising=False)
81     import agente.cerebro as cerebro
82     monkeypatch.setattr(cerebro, "_cliente", None)
83
84     resultado = pensar("Pendiente de confirmar: arroz 90.", "confirmo")
85     assert resultado["intencion"] == "confirmar"

```

backend/tests/test_interprete.py

161 LÍNEAS

El interprete local.

```

1  """Regresiones del intérprete local: el respaldo que sostiene la demo si
2  falla el Wi-Fi o no hay llave de Gemini. Es reconocimiento de palabras
3  clave, no NLU real - por eso necesita cobertura explícita de cada intención
4  y de las formas de decir números que ya se sabe que aparecen en la demo."""
5  from __future__ import annotations
6
7  from servicios.interprete import interpretar_local, normalizar_unidad
8
9
10 # ----- números -----
11
12 def test_numero_digito_simple():
13     assert interpretar_local("hay 12 arroces")["cantidad"] == 12
14
15
16 def test_numero_decimal_con_punto():
17     assert interpretar_local("hay 2.5 kilos de arroz")["cantidad"] == 2.5
18
19
20 def test_numero_decimal_con_coma():
21     assert interpretar_local("hay 2,5 kilos de arroz")["cantidad"] == 2.5
22

```

```

23
24 def test_numero_negativo_con_menos():
25     assert interpretar_local("hay menos 5 cazuelas")["cantidad"] == -5
26
27
28 def test_numero_en_palabras_simple():
29     assert interpretar_local("hay cinco cazuelas")["cantidad"] == 5
30
31
32 def test_numero_compuesto_decena_y_unidad():
33     # "treinta y cinco" -> 35
34     assert interpretar_local("hay treinta y cinco tablas")["cantidad"] == 35
35
36
37 def test_numero_compuesto_centena_y_decena():
38     # "ciento ochenta" -> 180
39     assert interpretar_local("hay ciento ochenta unidades")["cantidad"] == 180
40
41
42 def test_numero_en_palabras_mil():
43     assert interpretar_local("hay mil servilletas")["cantidad"] == 1000
44
45
46 def test_numero_en_palabras_con_menos():
47     assert interpretar_local("hay menos cinco cazuelas")["cantidad"] == -5
48
49
50 def test_sin_numero_ni_palabra_clavecae_en_ayuda():
51     # sin número reconocible ni ninguna palabra clave, cae en el
52     # "no le entendí" por defecto en vez de fingir una cantidad.
53     r = interpretar_local("no logro ver nada aqui")
54     assert r["intencion"] == "ayuda"
55     assert r.get("cantidad") is None
56
57
58 # ----- unidades -----
59
60 def test_unidad_kilo_variantes():
61     for palabra in ("kilo", "kilos", "kilogramo", "kilogramos", "kg"):
62         assert interpretar_local(f"hay 3 {palabra} de arroz")["unidad"] == "Kilogram"
63
64
65 def test_unidad_litro_variantes():
66     for palabra in ("litro", "litros", "l"):
67         assert interpretar_local(f"hay 3 {palabra} de aceite")["unidad"] == "Liter"
68
69
70 def test_unidad_no_reconocida_es_none():
71     assert interpretar_local("hay 3 cazuelas blancas")["unidad"] is None
72
73
74 def test_normalizar_unidad_pasa_canonicas_intactas():
75     assert normalizar_unidad("Kilogram") == "Kilogram"
76
77
78 def test_normalizar_unidad_traduce_lo_dicho():
79     assert normalizar_unidad("kilos") == "Kilogram"
80     assert normalizar_unidad("litros") == "Liter"
81     assert normalizar_unidad("unidades") == "Unidad"
82     assert normalizar_unidad("porciones") == "Portion"
83
84
85 def test_normalizar_unidad_desconocida_se_devuelve_igual():
86     assert normalizar_unidad("bultos") == "bultos"
87
88
89 def test_normalizar_unidad_vacia():
90     assert normalizar_unidad(None) is None
91     assert normalizar_unidad("") == ""
92
93
94 # ----- intenciones -----
95
96 def test_intencion_navegar_iniciar_conteo():
97     r = interpretar_local("iniciar conteo en almacen ayb")
98     assert r["intencion"] == "navegar"
99     assert "almacen ayb" in r["bodega_texto"] or "ayb" in r["bodega_texto"]
100
101
102 def test_intencion_navegar_abrir_bodega():
103     r = interpretar_local("abrir bodega restaurante fuentes")
104     assert r["intencion"] == "navegar"
105
106
107 def test_intencion_pedir_con_porciones():
108     r = interpretar_local("hoy preparamos cincuenta ajiacos")

```

```

109     assert r["intencion"] == "pedir"
110     assert r["porciones"] == 50
111
112
113 def test_intencion_pedir_sin_porciones_queda_en_cero():
114     r = interpretar_local("vamos a preparar sancocho")
115     assert r["intencion"] == "pedir"
116     assert r["porciones"] == 0
117
118
119 def test_intencion_confirmar_variantes():
120     for frase in ("confirmo", "si", "sí", "correcto"):
121         assert interpretar_local(frase)["intencion"] == "confirmar"
122
123
124 def test_intencion_corregir_variantes():
125     for frase in ("no son esos, corregir", "uy no, esta mal", "cambiar la cantidad"):
126         assert interpretar_local(frase)["intencion"] == "corregir"
127
128
129 def test_intencion_consultar():
130     r = interpretar_local("cuanto arroz hay")
131     assert r["intencion"] == "consultar"
132     assert "arroz" in r["articulo_texto"]
133
134
135 def test_intencion_reporte():
136     for frase in ("generame el reporte", "el consolidado por favor", "imprimeme el archivo"):
137         assert interpretar_local(frase)["intencion"] == "reporte"
138
139
140 def test_intencion_ayuda_explicita():
141     for frase in ("ayuda", "no entiendo"):
142         assert interpretar_local(frase)["intencion"] == "ayuda"
143
144
145 def test_intencion_contar_con_cantidad_y_articulo():
146     r = interpretar_local("hay noventa cazuelas blancas")
147     assert r["intencion"] == "contar"
148     assert r["cantidad"] == 90
149     assert "cazuelas" in r["articulo_texto"] or "blancas" in r["articulo_texto"]
150
151
152 def test_intencion_desconocida_pide_repetir():
153     r = interpretar_local("xyzyz blorp qwerty")
154     assert r["intencion"] == "ayuda"
155     assert "repite" in r["respuesta_hablada"].lower()
156
157
158 def test_articulo_texto_sin_ruido_quita_palabras_vacias():
159     r = interpretar_local("hay noventa cazuelas blancas")
160     for palabra_vacia in ("hay", "noventa"):
161         assert palabra_vacia not in r["articulo_texto"].split()

```

backend/tests/test_agente_conversacion.py

317 LÍNEAS

Conversaciones completas.

```

1  """Integración de extremo a extremo del agente conversacional: guía
2  conversaciones reales por /api/agente/turno. Corre sin GOOGLE_API_KEY a
3  propósito (usa el intérprete local, determinista) para no depender de una
4  red externa ni de la variabilidad de un modelo de lenguaje - exactamente
5  el camino que sostiene la demo si falla el Wi-Fi."""
6  from __future__ import annotations
7
8  import pytest
9  from fastapi.testclient import TestClient
10
11
12 @pytest.fixture
13 def datos_agente(client: TestClient, monkeypatch: pytest.MonkeyPatch) -> dict:
14     """Una bodega asignada a luis con dos artículos deliberadamente
15     ambiguos (ARROZ / ARROZ BASMATI, el ejemplo documentado en
16     orquestador.py) y uno sin ambigüedad (ACEITE), para ejercitar tanto
17     el camino directo como la desambiguación."""
18     monkeypatch.delenv("GOOGLE_API_KEY", raising=False)
19     import agente.cerebro as cerebro
20     monkeypatch.setattr(cerebro, "_cliente", None)
21
22     from bd import Sesion
23     from modelos import Articulo, AsignacionBodega, Bodega, StockSistema, Usuario
24
25     with Sesion() as sesion:

```

```

26     auxiliar = sesion.query(Usuario).filter_by(nombre="luis").one()
27     bodega = Bodega(nombre_oficial="BODEGA AGENTE")
28     arroz = Articulo(codigo="ARR-001", nombre_oficial="ARROZ", unidad_medida="Kilogram")
29     basmati = Articulo(codigo="ARR-002", nombre_oficial="ARROZ BASMATI",
30                        unidad_medida="Kilogram")
31     aceite = Articulo(codigo="ACE-001", nombre_oficial="ACEITE", unidad_medida="Liter")
32     sesion.add_all([bodega, arroz, basmati, aceite])
33     sesion.flush()
34     sesion.add(AsignacionBodega(usuario_id=auxiliar.id, bodega_id=bodega.id))
35     sesion.add_all([
36         StockSistema(articulo_codigo="ARR-001", bodega_id=bodega.id, cantidad_sd=100),
37         StockSistema(articulo_codigo="ARR-002", bodega_id=bodega.id, cantidad_sd=50),
38         StockSistema(articulo_codigo="ACE-001", bodega_id=bodega.id, cantidad_sd=100),
39     ])
40     sesion.commit()
41     bodega_id = bodega.id
42
43     respuesta = client.post("/api/ingresar",
44                             data={"username": "luis", "password": "StockXperts1"})
45     assert respuesta.status_code == 200, respuesta.text
46     headers = {"Authorization": f"Bearer {respuesta.json()['token']}"}
47     return {"headers": headers, "bodega_id": bodega_id}
48
49
50 def _turno(client: TestClient, headers: dict, texto: str, sesion_id: int = 777) -> dict:
51     r = client.post("/api/agente/turno", headers=headers,
52                    json={"texto": texto, "sesion_id": sesion_id})
53     assert r.status_code == 200, r.text
54     return r.json()
55
56
57 def _ultimo_conteo(sesion_id: int):
58     """_guardar() en orquestador.py escribe Conteo.sesion_id con el mismo
59     entero de la conversación que llega a /api/agente/turno (no el id de
60     SesiónConteo) - se consulta igual aquí."""
61     from bd import Sesión
62     from modelos import Conteo
63     with Sesión() as sesion:
64         return (sesion.query(Conteo).filter_by(sesion_id=sesion_id, estado="confirmado")
65                 .order_by(Conteo.id.desc()).first())
66
67
68 def test_abrir_bodega_pide_confirmar_antes_de_abrir_y_luego_deja_contar(
69     client: TestClient, datos_agente: dict,
70 ) -> None:
71     headers = datos_agente["headers"]
72
73     ofrece = _turno(client, headers, "iniciar conteo en bodega agente")
74     assert ofrece.get("bodega") is None # todavía no abrió nada
75     assert "confirma" in ofrece["respuesta_hablada"].lower()
76
77     abrir = _turno(client, headers, "confirmo")
78     assert abrir["bodega"]["id"] == datos_agente["bodega_id"]
79
80     contar = _turno(client, headers, "hay cien aceites")
81     assert contar.get("pendiente") is not None
82     assert contar.get("alerta") is None # coincide exacto con el sistema: sin alerta
83
84     confirmar = _turno(client, headers, "confirmo")
85     assert confirmar.get("guardado") is True
86
87     guardado = _ultimo_conteo(777)
88     assert guardado is not None
89     assert guardado.articulo_codigo == "ACE-001"
90     assert guardado.cantidad == 100
91
92
93 def test_abrir_bodega_por_voz_encuentra_el_nombre_aunque diga_tilde(
94     client: TestClient, monkeypatch: pytest.MonkeyPatch,
95 ) -> None:
96     """Bug real reportado desde producción: el catálogo de bodegas viene
97     sin tildes del Excel ("ALMACEN SUMINISTROS"), pero el reconocimiento
98     de voz transcribe con tilde ("almacén suministros") - antes eso caía
99     en "no la encuentro, ¿la creamos?" para una bodega que sí existía."""
100     monkeypatch.delenv("GOOGLE_API_KEY", raising=False)
101     import agente.cerebro as cerebro
102     monkeypatch.setattr(cerebro, "_cliente", None)
103
104     from bd import Sesión
105     from modelos import Bodega, Usuario, AsignacionBodega
106
107     with Sesión() as sesion:
108         auxiliar = sesion.query(Usuario).filter_by(nombre="luis").one()
109         bodega = Bodega(nombre_oficial="ALMACEN CENTRAL")
110         sesion.add(bodega)
111         sesion.flush()

```

```

112     sesion.add(AsignacionBodega(usuario_id=auxiliar.id, bodega_id=bodega.id))
113     sesion.commit()
114     bodega_id = bodega.id
115
116     respuesta = client.post("/api/ingresar",
117                             data={"username": "luis", "password": "StockXperts1"})
118     headers = {"Authorization": f"Bearer {respuesta.json()['token']}"}
119
120     ofrece = _turno(client, headers, "iniciar conteo en almacén central", sesion_id=779)
121     assert ofrece.get("intencion") != "crear_bodega", ofrece
122
123     r = _turno(client, headers, "confirmo", sesion_id=779)
124     assert r.get("bodega", {}).get("id") == bodega_id, r
125
126
127 def test_abrir_bodega_por_voz_no_confunde_nombres_parecidos(
128     client: TestClient, monkeypatch: pytest.MonkeyPatch,
129 ) -> None:
130     """Bug real reportado desde producción: decir "autoservicios las
131     fuentes" abría "AUTOSERVICIOS CASCADA" - el respaldo por palabras se
132     quedaba con la PRIMERA bodega que contuviera "autoservicios", sin
133     llegar a comparar "fuentes" para desempatar."""
134     monkeypatch.delenv("GOOGLE_API_KEY", raising=False)
135     import agente.cerebro as cerebro
136     monkeypatch.setattr(cerebro, "_cliente", None)
137
138     from bd import Sesión
139     from modelos import Bodega, Usuario, AsignacionBodega
140
141     with Sesión() as sesion:
142         auxiliar = sesion.query(Usuario).filter_by(nombre="luis").one()
143         cascada = Bodega(nombre_oficial="AUTOSERVICIOS CASCADA")
144         fuentes = Bodega(nombre_oficial="AUTOSERVICIOS LAS FUENTES")
145         sesion.add_all([cascada, fuentes])
146         sesion.flush()
147         sesion.add_all([
148             AsignacionBodega(usuario_id=auxiliar.id, bodega_id=cascada.id),
149             AsignacionBodega(usuario_id=auxiliar.id, bodega_id=fuentes.id),
150         ])
151         sesion.commit()
152         fuentes_id = fuentes.id
153
154     respuesta = client.post("/api/ingresar",
155                             data={"username": "luis", "password": "StockXperts1"})
156     headers = {"Authorization": f"Bearer {respuesta.json()['token']}"}
157
158     _turno(client, headers, "iniciar conteo en autoservicios las fuentes", sesion_id=780)
159     r = _turno(client, headers, "confirmo", sesion_id=780)
160     assert r.get("bodega", {}).get("id") == fuentes_id, r
161
162
163 def test_decir_no_a_la_confirmacion_de_bodega_no_abre_nada(
164     client: TestClient, datos_agente: dict,
165 ) -> None:
166     """Si el reconocimiento de voz entendió mal el nombre, decir "no" a
167     la confirmación debe dejar todo intacto - nada de sesión creada, nada
168     de bodega marcada en conteo."""
169     headers = datos_agente["headers"]
170     sid = 783
171     ofrece = _turno(client, headers, "iniciar conteo en bodega agente", sesion_id=sid)
172     assert ofrece.get("bodega") is None
173
174     r = _turno(client, headers, "no", sesion_id=sid)
175     assert r.get("bodega") is None
176     assert "no la abro" in r["respuesta_hablada"].lower()
177
178     from bd import Sesión
179     from modelos import Bodega, SesiónConteo
180     with Sesión() as sesion:
181         b = sesion.get(Bodega, datos_agente["bodega_id"])
182         assert b.estado != "en_conteo"
183         assert sesion.query(SesiónConteo).filter_by(bodega_id=b.id).count() == 0
184
185
186 def test_bodega_ya_abierta_por_otro_dice_quien_la_tiene(
187     client: TestClient, monkeypatch: pytest.MonkeyPatch,
188 ) -> None:
189     """Antes decía "ya está en conteo por otra persona" a secas - sin
190     saber si era normal o un error, ni con quién hablar. Ahora dice el
191     nombre real de quién la tiene."""
192     monkeypatch.delenv("GOOGLE_API_KEY", raising=False)
193     import agente.cerebro as cerebro
194     monkeypatch.setattr(cerebro, "_cliente", None)
195
196     from bd import Sesión
197     from modelos import Bodega, Usuario, AsignacionBodega, SesiónConteo

```

```

198
199     with Sesion() as sesion:
200         luis = sesion.query(Usuario).filter_by(nombre="luis").one()
201         diana = sesion.query(Usuario).filter_by(nombre="diana").one()
202         bodega = Bodega(nombre_oficial="BODEGA COMPARTIDA", estado="en_conteo")
203         sesion.add(bodega)
204         sesion.flush()
205         sesion.add(AsignacionBodega(usuario_id=luis.id, bodega_id=bodega.id))
206         sesion.add(SesionConteo(bodega_id=bodega.id, usuario_id=diana.id,
207                                tipo="conteo", estado="abierta"))
208         sesion.commit()
209
210     respuesta = client.post("/api/ingresar",
211                             data={"username": "luis", "password": "StockXperts1"})
212     headers = {"Authorization": f"Bearer {respuesta.json()['token']}"}
213
214     r = _turno(client, headers, "iniciar conteo en bodega compartida", sesion_id=784)
215     assert r.get("bodega") is None
216     assert "diana" in r["respuesta_hablada"].lower()
217
218
219 def test_sin_bodega_abierta_no_deja_contar(client: TestClient, datos_agente: dict) -> None:
220     headers = datos_agente["headers"]
221     r = _turno(client, headers, "hay diez aceites", sesion_id=778)
222     assert "abra una bodega" in r["respuesta_hablada"].lower()
223     assert r.get("pendiente") is None
224
225
226 def test_arroz_ambiguo_se_desambigua_y_dispara_desviacion(
227     client: TestClient, datos_agente: dict,
228 ) -> None:
229     headers = datos_agente["headers"]
230     sid = 779
231     _turno(client, headers, "iniciar conteo en bodega agente", sesion_id=sid)
232     _turno(client, headers, "confirmo", sesion_id=sid)
233
234     ambiguo = _turno(client, headers, "hay noventa arroces", sesion_id=sid)
235     assert ambiguo.get("opciones") is not None
236     nombres = {o["nombre"] for o in ambiguo["opciones"]}
237     assert nombres == {"ARROZ", "ARROZ BASMATI"}
238
239     # el sistema espera 50 de la basmati; decir noventa es una desviación
240     # del 80%, muy por encima del umbral - debe alertar y NO auto-confirmar.
241     elegido = _turno(client, headers, "la basmati", sesion_id=sid)
242     assert elegido.get("alerta") == "desviacion"
243     assert elegido["contexto_alerta"]["articulo"] == "ARROZ BASMATI"
244     assert elegido.get("guardado") is not True
245
246     confirmar = _turno(client, headers, "confirmo", sesion_id=sid)
247     assert confirmar.get("guardado") is True
248
249     guardado = _ultimo_conteo(sid)
250     assert guardado.articulo_codigo == "ARR-002"
251     assert guardado.cantidad == 90
252
253
254 def test_declina_opciones_con_no_cancela_sin_romper(
255     client: TestClient, datos_agente: dict,
256 ) -> None:
257     headers = datos_agente["headers"]
258     sid = 780
259     _turno(client, headers, "iniciar conteo en bodega agente", sesion_id=sid)
260     _turno(client, headers, "confirmo", sesion_id=sid)
261     ambiguo = _turno(client, headers, "hay noventa arroces", sesion_id=sid)
262     assert ambiguo.get("opciones") is not None
263
264     cancelado = _turno(client, headers, "no", sesion_id=sid)
265     assert "cancelado" in cancelado["respuesta_hablada"].lower()
266
267     # sin opciones pendientes, un tercer turno normal ya no debe seguir
268     # atascado en la desambiguación anterior.
269     siguiente = _turno(client, headers, "hay cien aceites", sesion_id=sid)
270     assert siguiente.get("pendiente") is not None
271
272
273 def test_corregir_antes_de_confirmar_cambia_la_cantidad_pendiente(
274     client: TestClient, datos_agente: dict,
275 ) -> None:
276     headers = datos_agente["headers"]
277     sid = 781
278     _turno(client, headers, "iniciar conteo en bodega agente", sesion_id=sid)
279     _turno(client, headers, "confirmo", sesion_id=sid)
280     _turno(client, headers, "hay cien aceites", sesion_id=sid)
281
282     corregido = _turno(client, headers, "no, son noventa", sesion_id=sid)
283     assert corregido.get("pendiente") is not None

```

```

284
285     _turno(client, headers, "confirmando", sesion_id=sid)
286
287     guardado = _ultimo_conteo(sid)
288     assert guardado.cantidad == 90          # el valor corregido, no los cien originales
289
290
291 def test_cantidad_negativa_no_se_guarda_y_pide_repetir(
292     client: TestClient, datos_agente: dict,
293 ) -> None:
294     headers = datos_agente["headers"]
295     sid = 782
296     _turno(client, headers, "iniciar conteo en bodega agente", sesion_id=sid)
297     _turno(client, headers, "confirmando", sesion_id=sid)
298
299     negativo = _turno(client, headers, "hay menos cinco aceites", sesion_id=sid)
300     assert negativo.get("alerta") == "negativo"
301     assert negativo.get("pendiente") is None
302
303     # nada quedo pendiente: un "confirmando" no debe inventar un guardado.
304     confirmar = _turno(client, headers, "confirmando", sesion_id=sid)
305     assert confirmar.get("guardado") is not True
306
307     assert _ultimo_conteo(sid) is None
308
309
310 def test_modos_pedido_extrae_preparacion_y_porciones_sin_bodega(
311     client: TestClient, datos_agente: dict,
312 ) -> None:
313     headers = datos_agente["headers"]
314     r = _turno(client, headers, "hoy preparamos cincuenta ajiacos", sesion_id=-777)
315     assert r["intencion"] == "pedir"
316     assert r["porciones"] == 50
317     assert "ajiaco" in (r.get("preparacion") or "").lower()

```

backend/tests/test_conciliacion.py

68 LÍNEAS

Emparejamiento y alertas.

```

1  """Coincidencias de artículo por texto dicho/escrito (servicios/conciliacion.py)."""
2  from servicios.conciliacion import _cobertura, normalizar, puntuar
3
4  UMBRAL = 45 # el mismo que usa buscar_articulo() para aceptar un candidato
5
6
7  def test_restaurante_no_confunde_con_costilla_de_res():
8      """Decir "restaurante" (buscando una bodega, en la pantalla equivocada)
9      no debe encontrar "COSTILLA DE RES" como si fuera un artículo parecido.
10
11      Antes, la raíz de comparación de 4 letras se quedaba corta cuando la
12      palabra del catálogo tenía menos de 4 letras ("RES"): "RESTAURANTE"
13      empieza igual que "RES" y eso contaba como coincidencia completa, sin
14      que las palabras tengan relación alguna."""
15      consulta = normalizar("restaurante")
16      candidato = normalizar("COSTILLA DE RES")
17      assert _cobertura(consulta, candidato) == 0.0
18      assert puntuar(consulta, candidato) < UMBRAL
19
20
21  def test_una_palabra_corta_del_catologo_no_hace_de_raiz_para_cualquier_cosa():
22      """Lo mismo, en general: ninguna palabra de 3 letras del catálogo
23      (RES, PAN...) debe servir de "raíz" para aceptar una palabra dicha
24      mucho más larga solo porque empieza igual."""
25      assert _cobertura(normalizar("restaurante"), normalizar("ARROZ RES")) == 0.0
26      assert _cobertura(normalizar("panculado"), normalizar("QUESO PAN")) == 0.0
27
28
29  def test_blanca_sigue_encontrando_blanco():
30      """La raíz de 4 letras entre dos palabras largas (>=4) debe seguir
31      funcionando igual que antes - lo que se corrigió es solo el caso de
32      una palabra del catálogo más corta que la raíz."""
33      consulta = normalizar("tabla para picar blanca")
34      candidato = normalizar("TABLA PARA PICAR BLANCO")
35      assert _cobertura(consulta, candidato) == 100.0
36
37
38  def test_cazuelas_sigue_encontrando_cazuela():
39      assert _cobertura(normalizar("cazuelas"), normalizar("CAZUELA DE BARRO")) == 100.0
40
41
42  def test_palabra_dicha_de_3_letras_ya_no_encuentra_candidato_largo_por_prefijo():
43      """El caso contrario al de "restaurante": antes, una palabra DICHA de
44      3 letras ("ver", de una frase como "sí para ver el detalle" - no una

```

```

45     búsqueda real) encontraba cualquier artículo cuyo nombre EMPEZARA
46     igual ("VERDE", "VERDURAS"), sin relación alguna. Ahora una palabra
47     de 3 letras solo cuenta si coincide EXACTA - "sal" ya no encuentra
48     "SALSA" solo por el prefijo (haría falta decir "sal" tal cual, no
49     "salsa", para que cuente)."""
50     assert _cobertura(normalizar("ver"), normalizar("VERDE")) == 0.0
51     assert _cobertura(normalizar("sal"), normalizar("SALSA DE TOMATE")) == 0.0
52
53
54 def test_bug_real_si_para_ver_el_detalle_no_encuentra_un_articulo_al_azar():
55     """Bug real reportado: decir "sí para ver el detalle" (el eco de la
56     pregunta hablada del selector de bodega, no una búsqueda de
57     artículo) encontraba "PASTILLA LIMPIADOR HORNO VERDE BALDEX150"
58     porque "ver" es prefijo de "VERDE"."""
59     consulta = normalizar("Sí para ver el detalle.")
60     candidato = normalizar("PASTILLA LIMPIADOR HORNO VERDE BALDEX150")
61     assert _cobertura(consulta, candidato) == 0.0
62     assert puntuar(consulta, candidato) < UMBRAL
63
64
65 def test_palabra_dicha_de_4_letras_sigue_encontrando_candidato_largo_por_prefijo():
66     """A partir de 4 letras, el prefijo real sigue contando en los dos
67     sentidos - eso es lo que hace que BLANCA encuentre BLANCO."""
68     assert _cobertura(normalizar("cazu"), normalizar("CAZUELA DE BARRO")) == 100.0

```

backend/tests/test_regresiones_seguridad.py

125 LÍNEAS

Fallos de seguridad que ya ocurrieron una vez.

```

1  """Regresiones de autorización, sesión y legalización de inventario."""
2  from __future__ import annotations
3
4  import pytest
5  from fastapi.testclient import TestClient
6  from starlette.websockets import WebSocketDisconnect
7
8
9  def test_auxiliar_no_puede_abrir_bodega_no_asignada(
10     client: TestClient, datos_regresion: dict[str, object]
11 ) -> None:
12     """Una asignación limita tanto la lista visible como la apertura de bodega."""
13     headers = datos_regresion["headers"]
14
15     listado = client.get("/api/bodegas", headers=headers)
16     assert listado.status_code == 200, listado.text
17     ids_visibles = {fila["id"] for fila in listado.json()}
18     assert datos_regresion["bodega_asignada_id"] in ids_visibles
19     assert datos_regresion["bodega_no_asignada_id"] not in ids_visibles
20
21     respuesta = client.post(
22         "/api/bodegas/abrir",
23         headers=headers,
24         json={"bodega": datos_regresion["bodega_no_asignada"]},
25     )
26     assert respuesta.status_code == 403, respuesta.text
27
28
29 def test_cerrar_todas_las_sesiones_pide_a_cognito_revocar_las_del_usuario_correcto(
30     client: TestClient, datos_regresion: dict[str, object],
31     monkeypatch: pytest.MonkeyPatch,
32 ) -> None:
33     """La clave y el login los maneja Cognito directamente, así que esta
34     ruta ya no puede invalidar un token propio (no hay "token propio") -
35     lo que sigue siendo responsabilidad de este backend, y lo que prueba
36     esto, es pedirle a Cognito (AdminUserGlobalSignOut) que revoque las
37     sesiones exactamente de quien hizo la llamada, no de otra persona."""
38     import main as modulo
39
40     llamadas = []
41
42     class _ClienteCognitoFalso:
43         def admin_user_global_sign_out(self, UserPoolId, Username):
44             llamadas.append((UserPoolId, Username))
45
46     monkeypatch.setattr(modulo, "_cliente_cognito", lambda: _ClienteCognitoFalso())
47
48     headers = datos_regresion["headers"]
49     r = client.post("/api/usuarios/yo/cerrar-todas", headers=headers)
50     assert r.status_code == 200, r.text
51     assert r.json() == {"ok": True}
52     assert llamadas == [(modulo.COGNITO_USER_POOL_ID, "luis")]
53
54

```

```

55 def test_cerrar_todas_las_sesiones_devuelve_502_si_cognito_falla(
56     client: TestClient, datos_regresion: dict[str, object],
57     monkeypatch: pytest.MonkeyPatch,
58 ) -> None:
59     """Si Cognito no responde, se avisa con un error - no se calla el
60     fallo como si las sesiones sí se hubieran cerrado."""
61     import main as modulo
62
63     class _ClienteCognitoFalso:
64         def admin_user_global_sign_out(self, UserPoolId, Username):
65             raise RuntimeError("Cognito no disponible")
66
67     monkeypatch.setattr(modulo, "_cliente_cognito", lambda: _ClienteCognitoFalso())
68
69     headers = datos_regresion["headers"]
70     r = client.post("/api/usuarios/yo/cerrar-todas", headers=headers)
71     assert r.status_code == 502, r.text
72
73
74 def test_auxiliar_no_puede_ver_detalle_de_bodega_ajena(
75     client: TestClient, datos_regresion: dict[str, object]
76 ) -> None:
77     """La asignación también protege el detalle/firmas/inventario, no solo
78     el listado: antes se podía pedir estos datos directo por bodega_id sin
79     que la asignación se revisara para nada."""
80     headers = datos_regresion["headers"]
81     ajena = datos_regresion["bodega_no_asignada_id"]
82
83     for ruta in (f"/api/bodegas/{ajena}/detalle", f"/api/bodegas/{ajena}/firmas",
84                 f"/api/bodegas/{ajena}/articulos"):
85         r = client.get(ruta, headers=headers)
86         assert r.status_code == 403, f"{ruta}: {r.text}"
87
88     propia = datos_regresion["bodega_asignada_id"]
89     r = client.get(f"/api/bodegas/{propia}/detalle", headers=headers)
90     assert r.status_code == 200, r.text
91
92
93 def test_auxiliar_no_puede_abrir_bodega_ajena_por_voz(
94     client: TestClient, datos_regresion: dict[str, object],
95     monkeypatch: pytest.MonkeyPatch,
96 ) -> None:
97     """El mismo límite de asignación que /api/bodegas/abrir aplica al pedir
98     lo mismo por el agente conversacional (/api/agente/turno) - antes
99     bastaba con dictar el nombre de la bodega para saltárselo. Se fuerza
100     el intérprete local (sin Gemini) para que el resultado no dependa de
101     cómo un modelo externo decida frasear "bodega_texto" ese día - lo que
102     se prueba aquí es la autorización, no la interpretación del lenguaje."""
103     monkeypatch.delenv("GOOGLE_API_KEY", raising=False)
104     import agente.cerebro as cerebro
105     monkeypatch.setattr(cerebro, "_cliente", None)
106
107     headers = datos_regresion["headers"]
108     nombre_ajena = datos_regresion["bodega_no_asignada"]
109
110     r = client.post("/api/agente/turno", headers=headers,
111                    json={"texto": f"iniciar conteo en {nombre_ajena}", "sesion_id": 999})
112     assert r.status_code == 200, r.text
113     datos = r.json()
114     assert datos.get("bodega") is None
115     assert "no esta asignada" in datos["respuesta_hablada"].lower() \
116         or "no está asignada" in datos["respuesta_hablada"].lower()
117
118
119 def test_websocket_rechaza_conexion_sin_token(client: TestClient) -> None:
120     """El tablero en vivo no puede exponer estado de bodegas sin identidad."""
121     with pytest.raises(WebsocketDisconnect) as error:
122         with client.websocket_connect("/api/bodegas/estado") as websocket:
123             websocket.receive_json()
124
125     assert error.value.code == 1008

```

backend/tests/test_regresiones_legalizacion.py

56 LÍNEAS

Fallos de legalizacion que ya ocurrieron una vez.

```

1 """Regresiones para evitar alteraciones repetidas o cruzadas de inventario."""
2 from __future__ import annotations
3
4 from fastapi.testclient import TestClient
5
6
7 def _stock(codigo: str, bodega_id: int) -> float:

```

```

8     from bd import Sesion
9     from modelos import StockSistema
10
11     with Sesion() as sesion:
12         fila = sesion.query(StockSistema).filter_by(
13             articulo_codigo=codigo, bodega_id=bodega_id
14         ).one()
15         return float(fila.cantidad_sd)
16
17
18 def test_legalizacion_es_idempotente_y_afecta_solo_su_bodega(
19     client: TestClient, datos_regresion: dict[str, object]
20 ) -> None:
21     """Reintentar una legalización no duplica el sobrante en ninguna bodega."""
22     from bd import Sesion
23     from modelos import LineaServicio
24
25     servicio_id = 777
26     codigo = str(datos_regresion["articulo_codigo"])
27     bodega_origen = int(datos_regresion["bodega_asignada_id"])
28     otra_bodega = int(datos_regresion["bodega_no_asignada_id"])
29     headers = datos_regresion["headers"]
30
31     with Sesion() as sesion:
32         sesion.add(LineaServicio(
33             servicio_id=servicio_id,
34             articulo_codigo=codigo,
35             nombre="ARTICULO REGRESION",
36             pedido=10,
37             usado=0,
38             bodega_id=bodega_origen,
39         ))
40         sesion.commit()
41
42     payload = {
43         "servicio_id": servicio_id,
44         "usos": [{"codigo": codigo, "usado": 7}],
45     }
46     primera = client.post("/api/legalizacion/confirmar", headers=headers, json=payload)
47     assert primera.status_code == 200, primera.text
48     assert _stock(codigo, bodega_origen) == 103
49     assert _stock(codigo, otra_bodega) == 200
50
51     segunda = client.post("/api/legalizacion/confirmar", headers=headers, json=payload)
52     # Un reintento puede responder éxito ya aplicado o conflicto; en ambos casos
53     # la condición de idempotencia es que el inventario permanezca intacto.
54     assert segunda.status_code in (200, 409), segunda.text
55     assert _stock(codigo, bodega_origen) == 103
56     assert _stock(codigo, otra_bodega) == 200

```

frontend/src/interpreteLocal.test.js

139 LÍNEAS

El interprete del navegador.

```

1 // Regresiones del intérprete local en JS: los mismos casos que
2 // backend/tests/test_interprete.py, para confirmar que el puerto a
3 // JavaScript (interpreteLocal.js) se comporta igual que el original en
4 // Python que corre en el servidor. Corre con: node --test src/*.test.js
5 import { test } from "node:test";
6 import assert from "node:assert/strict";
7 import { interpretarLocal, normalizarUnidad } from "../interpreteLocal.js";
8
9 // — números —
10 test("numero digito simple", () => {
11     assert.equal(interpretarLocal("hay 12 arroces").cantidad, 12);
12 });
13 test("numero decimal con punto", () => {
14     assert.equal(interpretarLocal("hay 2.5 kilos de arroz").cantidad, 2.5);
15 });
16 test("numero decimal con coma", () => {
17     assert.equal(interpretarLocal("hay 2,5 kilos de arroz").cantidad, 2.5);
18 });
19 test("numero negativo con menos", () => {
20     assert.equal(interpretarLocal("hay menos 5 cazuelas").cantidad, -5);
21 });
22 test("numero en palabras simple", () => {
23     assert.equal(interpretarLocal("hay cinco cazuelas").cantidad, 5);
24 });
25 test("numero compuesto decena y unidad", () => {
26     assert.equal(interpretarLocal("hay treinta y cinco tablas").cantidad, 35);
27 });
28 test("numero compuesto centena y decena", () => {
29     assert.equal(interpretarLocal("hay ciento ochenta unidades").cantidad, 180);

```

```

30 });
31 test("numero en palabras mil", () => {
32   assert.equal(interpretarLocal("hay mil servilletas").cantidad, 1000);
33 });
34 test("numero en palabras con menos", () => {
35   assert.equal(interpretarLocal("hay menos cinco cazuelas").cantidad, -5);
36 });
37 test("sin numero ni palabra clave cae en ayuda", () => {
38   const r = interpretarLocal("no logro ver nada aqui");
39   assert.equal(r.intencion, "ayuda");
40   assert.equal(r.cantidad ?? null, null);
41 });
42
43 // — unidades —
44 test("unidad kilo variantes", () => {
45   for (const palabra of ["kilo", "kilos", "kilogramo", "kilogramos", "kg"]) {
46     assert.equal(interpretarLocal(`hay 3 ${palabra} de arroz`).unidad, "Kilogram");
47   }
48 });
49 test("unidad litro variantes", () => {
50   for (const palabra of ["litro", "litros", "l"]) {
51     assert.equal(interpretarLocal(`hay 3 ${palabra} de aceite`).unidad, "Liter");
52   }
53 });
54 test("unidad no reconocida es null", () => {
55   assert.equal(interpretarLocal("hay 3 cazuelas blancas").unidad, null);
56 });
57 test("normalizarUnidad pasa canonicas intactas", () => {
58   assert.equal(normalizarUnidad("Kilogram"), "Kilogram");
59 });
60 test("normalizarUnidad traduce lo dicho", () => {
61   assert.equal(normalizarUnidad("kilos"), "Kilogram");
62   assert.equal(normalizarUnidad("litros"), "Liter");
63   assert.equal(normalizarUnidad("unidades"), "Unidad");
64   assert.equal(normalizarUnidad("porciones"), "Portion");
65 });
66 test("normalizarUnidad desconocida se devuelve igual", () => {
67   assert.equal(normalizarUnidad("bultos"), "bultos");
68 });
69 test("normalizarUnidad vacia", () => {
70   assert.equal(normalizarUnidad(null), null);
71   assert.equal(normalizarUnidad(""), "");
72 });
73
74 // — intenciones —
75 test("intencion navegar iniciar conteo", () => {
76   const r = interpretarLocal("iniciar conteo en almacen ayb");
77   assert.equal(r.intencion, "navegar");
78 });
79 test("intencion navegar abrir bodega", () => {
80   assert.equal(interpretarLocal("abrir bodega restaurante fuentes").intencion, "navegar");
81 });
82 test("intencion pedir con porciones", () => {
83   const r = interpretarLocal("hoy preparamos cincuenta ajiacos");
84   assert.equal(r.intencion, "pedir");
85   assert.equal(r.porciones, 50);
86 });
87 test("intencion pedir sin porciones queda en cero", () => {
88   const r = interpretarLocal("vamos a preparar sancocho");
89   assert.equal(r.intencion, "pedir");
90   assert.equal(r.porciones, 0);
91 });
92 test("intencion confirmar variantes", () => {
93   for (const frase of ["confirmo", "si", "sí", "correcto"]) {
94     assert.equal(interpretarLocal(frase).intencion, "confirmar");
95   }
96 });
97 test("intencion corregir variantes", () => {
98   for (const frase of ["no son esos, corregir", "uy no, esta mal", "cambiar la cantidad"]) {
99     assert.equal(interpretarLocal(frase).intencion, "corregir");
100   }
101 });
102 test("intencion consultar", () => {
103   const r = interpretarLocal("cuanto arroz hay");
104   assert.equal(r.intencion, "consultar");
105   assert.ok(r.articulo_texto.includes("arroz"));
106 });
107 test("intencion reporte", () => {
108   for (const frase of ["generame el reporte", "el consolidado por favor", "imprimeme el archivo"]) {
109     assert.equal(interpretarLocal(frase).intencion, "reporte");
110   }
111 });
112 test("intencion ayuda explicita", () => {
113   for (const frase of ["ayuda", "no entiendo"]) {
114     assert.equal(interpretarLocal(frase).intencion, "ayuda");
115   }

```

```

116 });
117 test("intencion contar con cantidad y articulo", () => {
118   const r = interpretarLocal("hay noventa cazuelas blancas");
119   assert.equal(r.intencion, "contar");
120   assert.equal(r.cantidad, 90);
121 });
122 test("intencion desconocida pide repetir", () => {
123   const r = interpretarLocal("xyzyz blorp qwerty");
124   assert.equal(r.intencion, "ayuda");
125   assert.ok(r.respuesta_hablada.toLowerCase().includes("repite"));
126 });
127 test("articulo_texto sin ruido quita palabras vacias", () => {
128   const r = interpretarLocal("hay noventa cazuelas blancas");
129   const palabras = r.articulo_texto.split(" ");
130   assert.ok(!palabras.includes("hay"));
131   assert.ok(!palabras.includes("noventa"));
132 });
133
134 // — casos propios del puerto a JS (ambigüedad centena/decena real) —
135 test("arroz vs arroz basmati: frase completa se limpia igual que en Python", () => {
136   const r = interpretarLocal("hay noventa arroces");
137   assert.equal(r.cantidad, 90);
138   assert.ok(r.articulo_texto.includes("arroces"));
139 });

```

frontend/src/colaOffline.test.js

47 LÍNEAS

La cola sin conexion.

```

1 // Corre con: node --test src/*.test.js
2 // node:test no trae localStorage/window (a diferencia del navegador) -
3 // se simulan aquí mismo, con lo mínimo que colaOffline.js necesita, en vez
4 // de meter jsdom solo para esto.
5 import { test } from "node:test";
6 import assert from "node:assert/strict";
7
8 const almacen = new Map();
9 globalThis.localStorage = {
10   getItem: (k) => (almacen.has(k) ? almacen.get(k) : null),
11   setItem: (k, v) => almacen.set(k, v),
12 };
13 let eventosDisparados = [];
14 globalThis.window = {
15   dispatchEvent: (e) => eventosDisparados.push(e.type),
16 };
17 globalThis.Event = class { constructor(tipo) { this.type = tipo; } };
18
19 const { leerCola, guardarCola, CLAVE_COLA, EVENTO_COLA } = await import("./colaOffline.js");
20
21 test("leerCola devuelve [] cuando no hay nada guardado", () => {
22   assert.deepEqual(leerCola("nueva-sesion"), []);
23 });
24
25 test("leerCola devuelve [] si lo guardado no es JSON válido (corrupción de localStorage)", () => {
26   almacen.set(CLAVE_COLA("rota"), "{no es json}");
27   assert.deepEqual(leerCola("rota"), []);
28 });
29
30 test("guardarCola y leerCola hacen ida y vuelta con los mismos datos", () => {
31   const items = [{ articulo: "ARROZ", cantidad: 5, unidad: "Kilogram" }];
32   guardarCola("s1", items);
33   assert.deepEqual(leerCola("s1"), items);
34 });
35
36 test("guardarCola dispara el evento para que otras pantallas se enteren", () => {
37   eventosDisparados = [];
38   guardarCola("s2", []);
39   assert.ok(eventosDisparados.includes(EVENTO_COLA));
40 });
41
42 test("la cola es independiente por sesión", () => {
43   guardarCola("sesion-a", [{ articulo: "A" }]);
44   guardarCola("sesion-b", [{ articulo: "B" }]);
45   assert.deepEqual(leerCola("sesion-a"), [{ articulo: "A" }]);
46   assert.deepEqual(leerCola("sesion-b"), [{ articulo: "B" }]);
47 });

```

frontend/src/accesibilidad.test.js

44 LÍNEAS

Contraste y tamaño de letra.

```

1 // Corre con: node --test src/*.test.js
2 // node:test no trae localStorage/document (a diferencia del navegador) -
3 // se simulan aquí mismo con lo mínimo que accesibilidad.js necesita.
4 import { test } from "node:test";
5 import assert from "node:assert/strict";
6
7 const almacen = new Map();
8 globalThis.localStorage = {
9   getItem: (k) => (almacen.has(k) ? almacen.get(k) : null),
10  setItem: (k, v) => almacen.set(k, v),
11 };
12 const atributos = {};
13 globalThis.document = {
14   documentElement: {
15     setAttribute: (k, v) => { atributos[k] = v; },
16   },
17 };
18
19 const { leerPreferencias, guardarPreferencias, aplicarPreferencias } =
20   await import("../accesibilidad.js");
21
22 test("leerPreferencias devuelve {} cuando no hay nada guardado", () => {
23   assert.deepEqual(leerPreferencias(), {});
24 });
25
26 test("leerPreferencias devuelve {} si lo guardado no es JSON válido", () => {
27   almacen.set("cv_accesibilidad", "no-es-json");
28   assert.deepEqual(leerPreferencias(), {});
29   almacen.delete("cv_accesibilidad");
30 });
31
32 test("guardarPreferencias y leerPreferencias hacen ida y vuelta", () => {
33   guardarPreferencias({ contraste: true, tamano: "grande" });
34   assert.deepEqual(leerPreferencias(), { contraste: true, tamano: "grande" });
35 });
36
37 test("aplicarPreferencias pone data-contraste=alto solo si contraste es true", () => {
38   aplicarPreferencias({ contraste: true, tamano: "grande" });
39   assert.equal(atributos["data-contraste"], "alto");
40   assert.equal(atributos["data-tamano"], "grande");
41   aplicarPreferencias({ contraste: false, tamano: "" });
42   assert.equal(atributos["data-contraste"], "");
43   assert.equal(atributos["data-tamano"], "");
44 });

```

frontend/src/confirmacionVoz.test.js

71 LÍNEAS

Que cuenta como un si.

```

1 // Bug real, encontrado en producción: "sí" (como lo transcribe la voz,
2 // CON tilde) nunca hacía match contra /^(si|sí|...)\b/ porque en
3 // JavaScript \b y \w son ASCII puros - "i" no cuenta como letra, así que
4 // la frontera de palabra justo después de la tilde no se reconocía.
5 // Corre con: node --test src/*.test.js
6 import { test } from "node:test";
7 import assert from "node:assert/strict";
8 import { quitarTildes, esAfirmacion, esNegacion } from "../confirmacionVoz.js";
9
10 test("quitarTildes deja sí como si", () => {
11   assert.equal(quitarTildes("sí"), "si");
12 });
13 test("quitarTildes no toca palabras sin tilde", () => {
14   assert.equal(quitarTildes("confirmo"), "confirmo");
15 });
16
17 test("si con tilde SI confirma (el bug real reportado)", () => {
18   assert.equal(esAfirmacion("sí"), true);
19 });
20 test("Si con tilde y mayuscula y punto, como lo entrega la voz", () => {
21   assert.equal(esAfirmacion("Sí."), true);
22 });
23 test("si sin tilde tambien confirma", () => {
24   assert.equal(esAfirmacion("si"), true);
25 });
26 test("confirmo confirma", () => {
27   assert.equal(esAfirmacion("confirmo"), true);
28 });
29 test("asi es (con y sin tilde) confirma", () => {
30   assert.equal(esAfirmacion("así es"), true);
31   assert.equal(esAfirmacion("asi es"), true);
32 });
33 test("una palabra extra del contexto (ver) confirma solo si se pasa", () => {
34   assert.equal(esAfirmacion("ver"), false);
35 });

```

```

35  assert.equal(esAfirmacion("ver", ["ver"]), true);
36  });
37  test("una conjugacion del verbo extra (el texto del boton) tambien confirma", () => {
38    // Bug real reportado: el boton de un diálogo decía "Enviar", pero solo
39    // la palabra EXACTA "enviar" contaba - decir "envíalo" (como diría una
40    // persona real) no calzaba con nada y quedaba en "no le entendí".
41    assert.equal(esAfirmacion("envíalo", ["Enviar"]), true);
42    assert.equal(esAfirmacion("Enviar.", ["Enviar"]), true);
43    assert.equal(esAfirmacion("envíelo por favor", ["Enviar"]), true);
44    assert.equal(esAfirmacion("mándalo", ["Enviar"]), false); // no es conjugación de "enviar"
45  });
46  test("una palabra sin relacion con el extra no confirma por casualidad", () => {
47    assert.equal(esAfirmacion("entonces qué", ["Enviar"]), false);
48    assert.equal(esAfirmacion("no envíes nada", ["Enviar"]), false); // "no" niega primero
49  });
50  test("no NO confirma aunque diga sí en otra parte de la frase", () => {
51    assert.equal(esAfirmacion("no, mejor sí despues"), false);
52  });
53  test("una frase sin nada reconocible no confirma", () => {
54    assert.equal(esAfirmacion("no le entendí"), false);
55    assert.equal(esAfirmacion("kiosco taquilla ayb"), false);
56  });
57  test("vacío no confirma", () => {
58    assert.equal(esAfirmacion(""), false);
59  });
60
61  test("no limpio niega", () => {
62    assert.equal(esNegacion("no"), true);
63    assert.equal(esNegacion("No."), true);
64  });
65  test("no en medio de la frase no cuenta como negacion limpia", () => {
66    assert.equal(esNegacion("no se, tal vez"), true); // "no" es la primera palabra
67    assert.equal(esNegacion("mejor no"), false);      // "no" no es la primera palabra
68  });
69  test("si no niega", () => {
70    assert.equal(esNegacion("sí"), false);
71  });

```

frontend/src/tutorial.test.js

32 LÍNEAS

Cuando se abre el recorrido.

```

1  // Corre con: node --test src/*.test.js
2  import { test } from "node:test";
3  import assert from "node:assert/strict";
4
5  const almacen = new Map();
6  globalThis.localStorage = {
7    getItem: (k) => (almacen.has(k) ? almacen.get(k) : null),
8    setItem: (k, v) => almacen.set(k, v),
9    removeItem: (k) => almacen.delete(k),
10 };
11
12 const { debeAbrirTutorial, deshabilitarTutorial, habilitarTutorial } =
13   await import("../tutorial.js");
14
15 test("debeAbrirTutorial es true por defecto (nadie lo ha desactivado)", () => {
16   assert.equal(debeAbrirTutorial(11), true);
17 });
18
19 test("deshabilitarTutorial hace que debeAbrirTutorial pase a false, solo para ESE usuario", () => {
20   deshabilitarTutorial(11);
21   assert.equal(debeAbrirTutorial(11), false);
22   // otro usuario en el mismo dispositivo (tablet compartida en bodega)
23   // debe seguir viendo el video: la desactivacion es por persona.
24   assert.equal(debeAbrirTutorial(22), true);
25 });
26
27 test("habilitarTutorial revierte la desactivacion", () => {
28   deshabilitarTutorial(33);
29   assert.equal(debeAbrirTutorial(33), false);
30   habilitarTutorial(33);
31   assert.equal(debeAbrirTutorial(33), true);
32 });

```